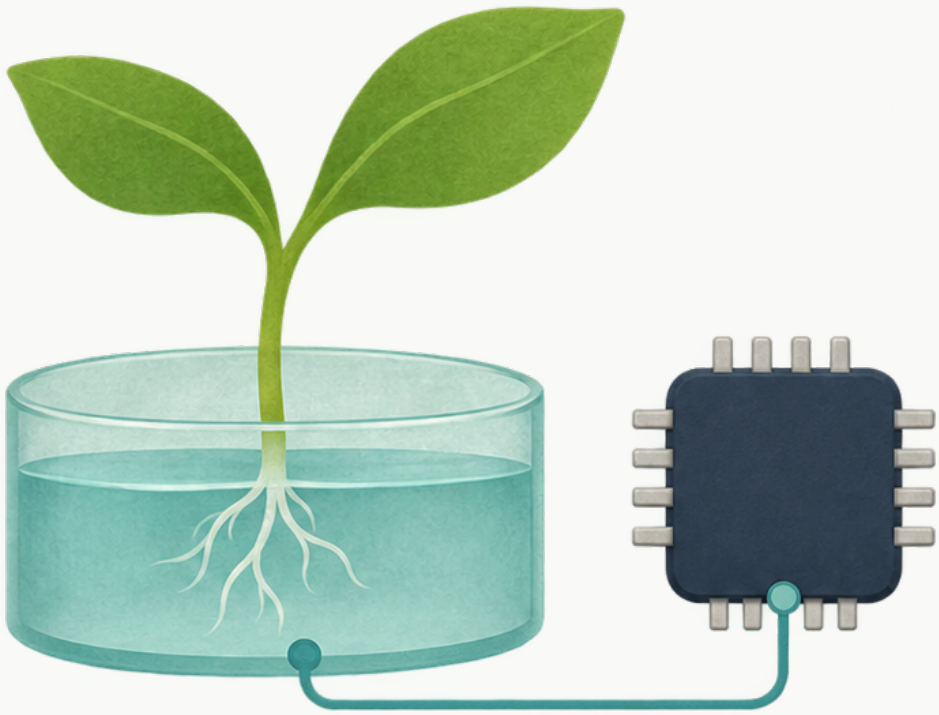
THE GUIDED HYDROPONICS TEXTBOOK

Writing
ESP32 Firmware

Learn C and FreeRTOS through
a complete hydroponics controller.



ESP32-S3 / C / FreeRTOS

BEGINNER GUIDED EDITION

You are going to read a complete controller.

A line of firmware should tell you what changes, when it changes and which earlier fact makes that change valid. When a line is difficult, we will slow down and supply the missing C or RTOS concept before asking you to follow it.

The opening lessons build small examples you can calculate yourself. Then each chapter turns to the production files that use those ideas. We will read their complete source in manageable sections. You will see declarations, normal paths, error paths and cleanup, rather than only the attractive part of a function.

The program is a hydroponics controller for an ESP32-S3. It uses C for most application code, FreeRTOS for task execution, and two C++ adapters for BLE and Wi-Fi libraries. Seven physical sensors produce ten metrics. Acquisition obtains observations; Safety owns the relay decision; other services coordinate power, settings and communication.

You need basic familiarity with variables, an if statement and a loop. Pointers, storage duration, queues, notifications, driver tables, callbacks and state machines are developed as the controller needs them. You can pause and use the linked earlier lesson when a detail has faded.

The source here is the reference implementation carried by the supplied Hydroponics firmware book. File paths and line numbers name that fixed source. We preserve its code while examining its assumptions. A comment about a design budget or a past measurement is not a new measurement made by reading the program.

Keep a notebook or editor beside the book. At each guard, predict which branch a concrete input takes. At each state update, name the object that changes. At each return, name the next caller that can make progress. Those small predictions connect the language to the system.

Read with two questions: why, then how?

Lessons before the production files

Begin each chapter with the worked concept lessons. The following source readings use that vocabulary immediately. The source contents pages let you reopen a file directly when reviewing the controller.

Every source range has a location

The code boxes show the path and original line numbers. Every line in the supplied application, adapters, bring-up programs and native tests is present. A grey > marks a visual continuation. It is not a character to type into the source.

Use the links as memory aids

Blue lesson references include printed page numbers. Function calls inside production listings also link to their implementation when the target is unambiguous. Your viewer's Back command returns from a jump. Margin notes appear only for a useful tip, pitfall or reminder; other pages use the full reading width.

Keep teaching code and production code distinct

A teaching example simplifies one question. Production listings carry their real file paths. The small learning program is a separate target; its fabricated input helps you observe task communication before connecting a probe.

Keep the original files nearby

Reference_Firmware_Source.zip and Learning_Program.zip are embedded in this PDF. Extract them from a desktop PDF viewer's Attachments panel when you want to compile or compare the files. The printed ranges are for reading; the archive supplies exact bytes.

Ask what the next owner is allowed to assume

A copied value may still be stale. A READY driver may still report a failed observation. A queued command may still be waiting to apply. Each chapter follows those distinctions into the next relevant caller without requiring you to understand an unseen implementation first.

Contents

01 Make One Decision 13

Begin with a value and a decision that you can trace yourself.

02 Read C As Instructions 17

Read declarations, branches and arithmetic by predicting their results.

03 Use Addresses And Records 22

Give a value an address, then put related values into one record.

04 Make State And Files Predictable 28

Understand what survives a call and how a program is assembled.

05 Let Two Jobs Share A CPU 39

Learn what the scheduler saves, selects and waits for.

06 Move Values Through Queues 52

Make a sender and receiver exchange an actual copy.

07 Wake For A Reason Or A Deadline 60

Use notifications for prompts, clocks for due times, and retained state for facts.

Contents / continued

08 Protect Shared State 70

Choose protection from the operation that must stay indivisible.

09 Write A Driver In Steps 78

Replace a physical wait with remembered state and a due time.

10 Decide Whether A Reading Is Usable 90

A number needs identity, timing and a validity verdict.

11 Build The Light Measurement 114

Use one analog path to connect voltage, time, filtering and a Reading.

12 Read The BME280 Over I2C 149

Separate bus transactions from conversion and compensation.

13 Wait For Water Temperature 182

Use a long conversion without freezing the acquisition service.

14 Turn A pH Voltage Into A Claim 213

Recover the module voltage, then justify its calibration.

Contents / continued

15 Compensate The TDS Estimate 234

Make a measurement dependency explicit instead of assuming 25 degrees.

16 Keep The SGP30 On Its Cadence 269

Understand a driver whose past affects its present result.

17 Measure An Ultrasonic Pulse 309

Separate microsecond edge timing from task scheduling.

18 Read The Acquisition Engine 338

Bring the individual drivers together without giving up their timing rules.

19 Give Safety The Final Decision 393

Combine a lamp request, alarm state and input validity in one output owner.

20 Survive A Power Transition 463

Move ownership only after the old owner stops using shared hardware.

21 Save And Apply Configuration 519

Separate accepting a request, storing it and using it.

Contents / continued

22 Encode And Decode Messages 567

Move between bounded bytes and explicit application requests.

23 Build The BLE Device Link 628

Keep callback work short and deliver commands through their owners.

24 Manage A Wi-Fi Station 675

Reconnect in bounded steps without borrowing the sensor path.

25 Bring The Controller To Life 697

Start dependencies in order and observe meaningful progress.

26 Test The Pure Behaviour 744

Supply controlled inputs and inspect the exact returned state.

27 Follow Complete Controller Events 817

Connect code you have read into full execution paths.

Production source contents

src/domain/types.h 36

include/app_config.h 47

src/drivers/sensor_driver.h 84

src/drivers/sensor_driver.c 89

src/domain/reading.h 94

src/domain/reading.c 99

src/services/snapshot_store.h 105

src/services/snapshot_store.c 106

src/domain/sensor_set.h 108

src/domain/sensor_set.c 110

src/hal/adc_channel.h 118

src/hal/adc_channel.c 120

src/domain/calibration.h 122

src/domain/calibration.c 125

src/drivers/light.h 130

src/drivers/light.c 133

test/bringup/bringup_harness.h 142

test/bringup/light_main.c 147

src/hal/i2c_bus.h 155

src/hal/i2c_bus.c 157

src/drivers/bme280.h 162

src/drivers/bme280.c 164

test/bringup/bme280_main.c 179

src/hal/onewire_bus.h 187

src/hal/onewire_bus.c 189

Production source contents

src/domain/ds18b20_codec.h 196

src/domain/ds18b20_codec.c 198

src/drivers/ds18b20.h 200

src/drivers/ds18b20.c 202

test/bringup/ds18b20_main.c 210

src/drivers/ph.h 218

src/drivers/ph.c 222

test/bringup/ph_main.c 229

src/domain/tds_codec.h 238

src/domain/tds_codec.c 239

src/drivers/tds.h 240

src/drivers/tds.c 244

test/bringup/tds_main.c 251

test/bringup/water_main.c 255

src/domain/sgp30_codec.h 272

src/domain/sgp30_codec.c 273

src/drivers/sgp30.h 275

src/drivers/sgp30.c 280

test/bringup/sgp30_main.c 296

test/bringup/climate_main.c 299

src/domain/ultrasonic.h 314

src/domain/ultrasonic.c 315

src/hal/pulse_capture.h 316

src/hal/pulse_capture.c 319

src/drivers/hcsr04.h 325

Production source contents

src/drivers/hcsr04.c 327

test/bringup/hcsr04_main.c 335

src/hal/clock.h 345

src/hal/clock.c 346

src/domain/schedule.h 347

src/domain/schedule.c 349

src/services/acquisition.h 352

src/services/acquisition.c 357

src/domain/alarm.h 398

src/domain/alarm.c 401

include/policy_config.h 403

src/domain/alarm_sticky.h 407

src/domain/alarm_sticky.c 409

src/domain/event_journal.h 411

src/domain/event_journal.c 413

src/domain/lamp_control.h 415

src/domain/lamp_control.c 417

src/domain/safety_core.h 420

src/domain/safety_core.c 426

src/hal/gpio_safe.h 435

src/hal/gpio_safe.c 436

src/hal/relay.h 437

src/hal/relay.c 438

src/services/safety.h 439

src/services/safety.c 442

Production source contents

test/bringup/lamp_main.c 458

src/services/rail_state.h 467

src/services/rail_state.c 468

src/domain/power_mode.h 470

src/domain/power_mode.c 475

src/services/power_coordinator.h 483

src/services/power_coordinator.c 485

test/bringup/power_sense_main.c 493

src/domain/cal_capture.h 522

src/domain/cal_capture.c 524

src/domain/config_blob.h 526

src/domain/config_blob.c 531

src/services/flash_gate.h 539

src/services/flash_gate.c 540

src/services/configuration.h 542

src/services/configuration.c 546

src/domain/frame_codec.h 570

src/domain/frame_codec.c 572

src/domain/protocol_codec.h 576

src/domain/protocol_codec.c 583

src/transports/protocol.h 619

src/transports/protocol.c 620

src/transports/frames.h 621

src/transports/frames.c 623

src/transports/ble_gatt.h 633

Production source contents

src/transports/ble_gatt.cpp 637

src/transports/ble_transport.h 649

src/transports/ble_transport.c 651

src/domain/backoff.h 677

src/domain/backoff.c 678

src/transports/wifi_lan.h 680

src/transports/wifi_lan.cpp 681

src/transports/wifi_mqtt.h 683

src/transports/wifi_mqtt.c 685

include/board_profile.h 701

platformio.ini 706

src/hal/arduino_entry.h 713

src/main.c 714

src/diagnostics/logger.h 718

src/diagnostics/logger.c 719

src/diagnostics/progress.h 723

src/diagnostics/progress.c 724

src/services/supervisor.h 725

src/services/supervisor.c 727

tools/serial_log.py 740

.gitattributes 742

.gitignore 743

test/native/test_domain/test_main.c 746

test/native/test_power/test_main.c 776

test/native/test_protocol/test_main.c 787

What must your program actually do?

Suppose the water temperature is 24.5 degrees Celsius. Your first job is to obtain that number. Your second job is to decide whether it is suitable for a control decision. Those are different jobs: a disconnected probe can leave an old 24.5 in memory even while the water is heating.

Teaching example - C

```
float waterC = 24.5f;
bool tooWarm = waterC >= 30.0f;
```

Read the first line as: reserve an object named waterC that holds an approximate real number, and initialise it to 24.5. The f suffix asks for a float constant. Read the second line as: compare waterC with 30.0, then store the comparison result in tooWarm. For this input, tooWarm is false. If waterC becomes 31.0 before the comparison, the result is true.

Nothing in those two lines reads a sensor. You supplied the number yourself. This is useful: you can first check your decision with known inputs. Later, the sensor driver will supply the number. Keeping that substitution possible is the first reason to separate a decision from hardware access.

Why the real firmware carries more fields

To act on a value, you will eventually need its unit, its age and an indication of whether the measurement succeeded. We will build that record after learning structures. For now, keep one question in mind: **what evidence would make this number safe to use?**

In the project, DS18B20 is the water-temperature sensor. The driver produces observations; a service decides how to use them. You do not need either file yet. We will open the driver in the water-temperature chapter (p. 182), when a long conversion gives us a reason to organise time.

TIP

The 30.0 threshold is a teaching value. It is not a commissioned limit for your plants.

COMMON PITFALL

A plausible number is not proof of a connected or working sensor.

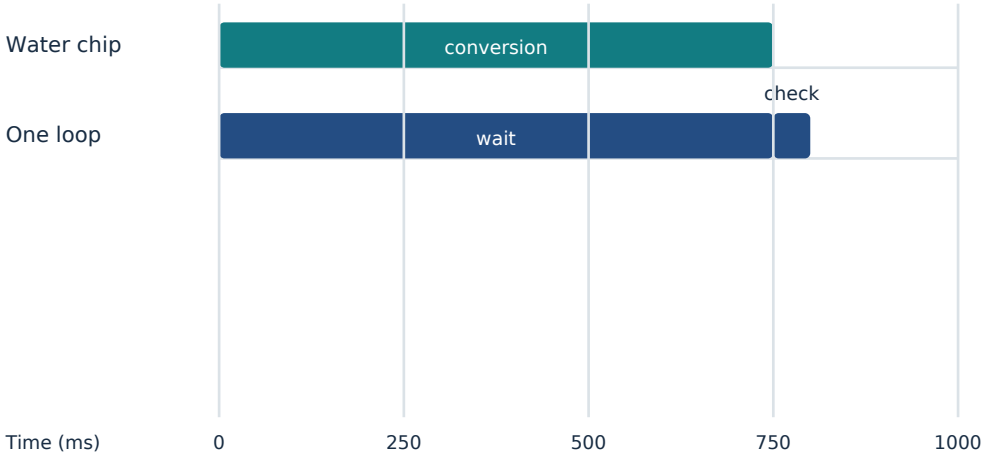
Why one long wait causes trouble

Start with one loop. It asks for a water-temperature conversion, waits 750ms, reads the result, and checks the lamp. Assume the instruction that waits really prevents this loop from advancing.

Teaching pseudocode - helper bodies deliberately omitted

```
for (;;) {
    beginWaterConversion();
    waitUntilWaterIsReady();  // up to 750ms
    readWaterResult();
    updateLampDecision();
}
```

If you press a lamp-off button 1ms after the wait begins, this loop cannot reach the lamp decision for roughly another 749ms. Making the CPU clock twice as fast does not halve a sensor's physical conversion time. The delay is in the order of operations.



Worked example: the sensor is busy while the loop postpones the lamp check.

You have two possible repairs. First, turn the sensor operation into steps: start it, remember when it should finish, and return to other work. Second, put independent work into separate RTOS tasks so one task can wait while another runs. Our firmware uses both repairs. A task organises a service; a stepped driver avoids making that service wait for every physical conversion.

Do not choose a task merely because you have a sensor. Seven sensors do not automatically require seven tasks. They may share buses and benefit from one owner. We will earn that design later, after a two-task example makes waiting concrete.

REMEMBER

A loop only advances when the current call returns. Other RTOS tasks may run if that call blocks through the RTOS.

Separate the question from the pin

Teaching example - C

```
bool waterIsTooWarm(float waterC) {
    return waterC >= 30.0f;
}
```

Call waterIsTooWarm(24.5f). The argument 24.5 is copied into the parameter waterC. The comparison produces false, and return hands that result to the caller. Call it with 31.0f and it returns true. The function neither knows a GPIO number nor reads a bus.

Now imagine the physical probe moves to a different pin. You should change the hardware setup, not the comparison. Conversely, a new agreed limit should not force you to rewrite the sensor protocol. That is a concrete reason for the project's folders.

Folder	Question it answers
hal/	How do I operate this ESP32 peripheral?
drivers/	How do I ask this sensor for a measurement?
domain/	What does this value or event mean?
services/	When do operations run, and who owns their state?
transports/	How do external messages enter and leave?

HAL means hardware abstraction layer: a small set of functions that hide the peripheral-specific calls. For example, adcChannelReadMilliVolts() obtains a voltage. The light driver uses that voltage to produce a Reading. Safety later decides whether the lamp should be on. Each step has a different reason to change.

For the next few chapters, we will write tiny functions like the one above. We will not start by memorising the folder tree. When we return to the tree in startup (p. 697), you will already understand the operations it connects.

COMMON PITFALL

Putting every function in a separate file does not create a good architecture. Separate responsibilities that need different information or change for different reasons.

Trace the value before adding machinery

Here is a complete small C program for your computer. It uses invented input, so you can check the decision without a sensor or an ESP32.

Teaching example - C

```c
#include <stdbool.h>
#include <stdio.h>

static bool waterIsTooWarm(float waterC) {
    return waterC >= 30.0f;
}

int main(void) {
    float waterC = 31.0f;
    bool tooWarm = waterIsTooWarm(waterC);
    printf("warm=%d\n", (int)tooWarm);
    return 0;
}
```

Execution enters main(), creates waterC, and calls the function with its value. The called function creates its parameter, evaluates 31.0 >= 30.0, and returns true. printf prints warm=1. The cast (int) makes the intended integer argument explicit. main returns zero to report normal termination to your computer.

On a PC with GCC, save this as first.c and run gcc -std=c11 -Wall -Wextra first.c -o first, then run ./first. Change only 31.0f to 24.5f; the printed result becomes warm=0. If you remove the function call, no decision is computed automatically.

The ESP32 project starts differently: Arduino calls setup(), and setup creates long-lived tasks. We will reach that entry point in the boot chapter (p. 697). The function call and value-copy rules you just traced stay the same.

You can now explain a tiny path from input to output. The next chapter adds enough C to express that path reliably. We will introduce each operator where it changes the program's behaviour.

TIP

main() here is a PC entry point. Do not paste it into the production firmware as another entry point.

A declaration gives a value a place

Teaching example - C

```
#include <stdint.h>
#include <stdbool.h>

uint32_t elapsedMs = 750;
float waterC = 24.5f;
bool valid = true;
```

uint32_t is an unsigned integer type with exactly 32 bits. It can represent 0 through 4294967295. It has no negative values. float can represent fractions, approximately. bool holds false or true. Choosing a type states which values you expect and which operations make sense.

The text after = is an initializer. It supplies an object's first value. An ordinary local declaration such as uint32_t elapsedMs; does not give you a meaningful starting number. Reading that local before assigning a value is an error. Set a starting value when you create it.

COMMON PITFALL

uint32_t is a type, not a unit. A variable containing 750 could mean milliseconds, microseconds or ADC counts; your naming and interface must say which.

Names and types do different jobs

Teaching example - C

```
typedef uint64_t MonoMs;
MonoMs now = 1000;
MonoMs deadline = now + 750;
```

typedef gives an existing type another name. MonoMs helps you remember that the number is monotonic milliseconds: elapsed time within this run, unaffected by a correction to the calendar clock. It does not create a clock and does not prevent you from accidentally assigning microseconds. In this example, deadline becomes 1750. Our project defines this alias in src/domain/types.h.

A semicolon ends a declaration or expression statement. Braces group a function body or control-flow body. Indentation helps you read the grouping; C uses the braces, not the indentation, to decide it. We will use braces even around short if bodies in teaching examples.

The exact-width types come from stdint.h; bool in this C11-style source comes from stdbool.h. An include makes those declarations available to the compiler. It does not read a sensor or run a function.

Read a function call in both directions

Teaching example - C

```
static float echoToMm(uint32_t widthUs) {
    return (float)widthUs * 343.0f / 2000.0f;
}

float distanceMm = echoToMm(1000);
```

At the call, evaluate the argument 1000. Copy it into widthUs. Enter the body. Convert the integer to float, multiply by 343.0, divide by 2000.0, then return 171.5. The caller stores that result in distanceMm. A return type tells you what comes back; the parameter list tells you what must go in.

void as a return type means there is no returned value. void in an otherwise empty parameter list means the function takes no parameters. A function can still change objects through pointer parameters; we will introduce that mechanism in output parameters (p. 23).

A local name does not escape with the result

widthUs is a parameter local to this call. Returning its calculated value does not give the caller access to that parameter object. If another call uses 2000, it gets its own parameter value and returns 343.0. Keep this copy behaviour in mind: a normal argument does not let the function rewrite the caller's variable.

A grey > at the start of a displayed code line marks a wrapped continuation. It is a page-layout mark, not a character to type into your C source. The attached source retains its original lines.

The initial static makes this function name private to this C translation unit. For now, read it as: this helper is for this file. We will separate that meaning from persistent storage in the static lesson (p. 28).

This calculation will be useful in the ultrasonic chapter (p. 309). There, an interrupt will provide widthUs. The arithmetic will remain simple because we will have separated obtaining the pulse width from converting it.

The speed 343m/s is the reference firmware's nominal value. It is not temperature compensated.

Reject a bad input before using it

Teaching example - C

```
bool accepted = false;
if (valid && waterC >= 30.0f) {
    accepted = true;
}
```

&& means logical AND. C evaluates valid first. If it is false, the second comparison is not evaluated. If it is true, C evaluates waterC >= 30.0f. Only two true operands enter the body. This evaluation order is called short-circuit evaluation.

Read valid && waterC >= 30.0f in order: first decide whether the observation is eligible; then compare its number. A guard is an early check that rejects an input before an operation that requires it. The next chapter will apply the same order to a pointer: check that an object exists before reading through its address.

Expression	Meaning
a = b	Assign b to a.
a == b	Compare for equality.
a != b	Compare for inequality.
!ready	True when ready is false.
a || b	True when either operand is true.

A comparison does not remember a previous decision. Each evaluation uses the values available then. To keep the lamp's previous command inside a threshold band, we will need a stored state, not just a longer condition. You will see that in lamp hysteresis (p. 393).

COMMON PITFALL

if (ready = true) assigns true and then tests it. It usually enters the branch. Use == for comparison, or simply if (ready).

REMEMBER

&& and || can skip their right operand. Put a required safety check before the operation it protects.

Units decide the arithmetic

Teaching example - C

```
uint32_t widthUs = 1000;
float wrong = widthUs / 2000;
float right = (float)widthUs / 2000.0f;
```

Both operands of the first division are integers. The quotient is the integer 0; assigning it to float later gives 0.0. The second expression performs floating-point division and gives 0.5. The destination type does not change an operation that has already happened.

For distance, multiply by speed as well: 1000 microseconds times 343 metres per second equals 343000 micrometres. Divide by two for the outgoing and returning path, then by 1000 to convert micrometres to millimetres. That gives 171.5mm. The code combines those last factors as /2000.0f.

Bits are useful when several yes/no facts travel together

Teaching example - C

```
uint16_t flags = 0;
flags |= (1u << 3);
bool late = (flags & (1u << 3)) != 0;
```

1u << 3 moves the one bit three places left: binary 00001000, decimal 8. |= keeps existing set bits and adds this one. & keeps only the bit positions present in both operands. Here late becomes true. The firmware uses this pattern for qualityFlags; a measurement may be both uncompensated and late.

Integer overflow is another reason to inspect operands. If a multiplication needs 64 bits, cast an operand before multiplying. Writing uint64_t result = (uint64_t)a * b makes that multiplication wide; casting the already-overflowed result would be too late.

COMMON PITFALL

0.1 is not stored exactly in ordinary binary float. Compare measured values with meaningful tolerances, not equality tests against every decimal you expect.

REMEMBER

& has two meanings: bitwise AND between values, and address-of before one object. The position tells you which meaning applies.

Read a byte before interpreting it

Teaching example - hexadecimal notation

```
uint16_t raw = 0x0190;  // the same number as 400
uint8_t high = 0x01;    // 1
uint8_t low  = 0x90;    // 144
// 1 * 256 + 144 = 400
```

The prefix 0x writes an integer in base 16. Its digits are 0 through 9, then A through F for ten through fifteen. Moving one digit left multiplies its place value by 16. Thus 0x90 means 9 times 16, or 144. Hexadecimal does not change the number stored; it gives you a convenient way to see groups of four bits.

On this target, a byte is eight bits. Two hexadecimal digits describe one byte, and a sixteen-bit quantity occupies two bytes. high and low above name the numerical parts of 400. A device protocol still decides which byte travels first. DS18B20 sends low first; the gas protocol sends its word's high byte first. We will decode each from its own documented order.

The suffix u, as in 1u, requests an unsigned integer constant. The f in 24.5f requests float. These suffixes affect C arithmetic types; neither suffix adds a physical unit. A field called raw could contain ADC counts, a temperature code or microseconds, so its interface must state the interpretation.

REMEMBER

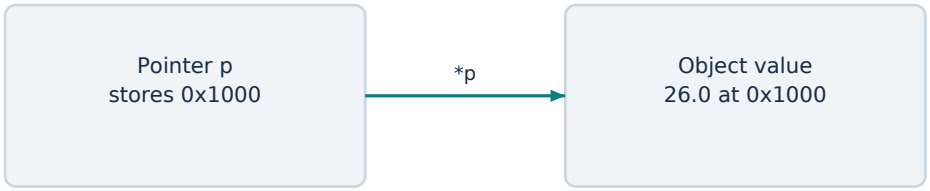
Representation comes first: bytes, order and scale. The physical unit comes from the driver contract.

A pointer stores an address

Teaching example - C

```
float waterC = 24.5f;
float *p = &waterC;
*p = 26.0f;
```

waterC is the object holding a float. &waterC asks for its address. p is a second object, a pointer, which stores that address. *p means the float object at the address stored in p. The final assignment therefore changes waterC to 26.0. It does not replace p's address.



&value gives the address; *p uses the object there.

Illustrative addresses: the actual numeric addresses depend on the build.

In a declaration, float *p says that p has pointer-to-float type. In an expression, *p accesses the pointed-to object. The same symbol appears in two different grammatical roles. Read the full statement before deciding what the star does.

Copying p copies an address. It does not create another waterC. If q = p, then *q and *p refer to the same float. Changing that object through either pointer is visible through the other. This will matter when two tasks receive addresses into shared memory.

A pointer is useful only while the target object exists and the address is appropriate for that type. NULL is a deliberate missing pointer. An uninitialised pointer is not a useful substitute for NULL. Do not dereference either one.

We will use & when asking a queue to copy an object, and * when a helper fills caller-owned storage. Neither operation means that a task automatically gets private or protected memory.

COMMON PITFALL

float *p creates a pointer object. It does not reserve another float for p to point to. Supply the address of a real object before using *p.

Let a function fill your object

Teaching example - C

```c
static bool readFake(float *out) {
    if (out == NULL) {
        return false;
    }
    *out = 24.5f;
    return true;
}

float waterC = 0.0f;
bool ok = readFake(&waterC);
```

At the call, &waterC is copied into out. The pointer itself is passed by value, just like the earlier integer parameter. Inside the function, *out = 24.5f follows that copied address and changes the caller's waterC. The function returns true separately to report success.

You now have two outputs with different purposes: waterC contains the measurement, and ok tells you whether the function supplied one. If readFake returns false, the caller must not pretend a fresh result exists. This is the pattern behind many HAL functions.

This will be useful when the ultrasonic HAL writes the captured width into a variable supplied by its caller. We will already understand the output-parameter syntax when we reach pulseCapturePoll().

COMMON PITFALL

readFake(waterC) passes the number. readFake(&waterC) passes its address. The function declaration tells you which one it needs.

REMEMBER

Return values often report success or status; pointer parameters can carry the actual data.

Keep facts about one result together

Teaching example - C

```
typedef struct {
    float valueC;
    uint64_t measuredMs;
    bool valid;
} Sample;

Sample a = { .valueC = 24.5f,
             .measuredMs = 1000,
             .valid = true };
```

A structure groups several fields into one object. a.valueC selects the float field inside a. a.measuredMs selects its timestamp. The dot in the initializer names the field being initialised; the dot in a.valueC selects the field later. Initializers with field names make a long record easier to check.

Teaching example - C

```
Sample b = a;
a.valueC = 99.0f;
```

The assignment copies the record's field values. b.valueC remains 24.5. b does not become another name for a. This is the reason a queue can pass a small measurement by copy: the receiver can own its own record after the sender continues.

Be careful when a structure contains pointers. Copying such a structure copies its pointer fields, not the objects those pointers identify. Two copied pointer fields can still reach the same buffer. Our simple Sample has only values, so it avoids that extra ownership problem.

The project's Reading is a larger record with the same purpose. It keeps a value, identity, unit, timing and validity information together. We will open its complete declaration in the Reading lesson (p. 90), after we know why tasks need a message.

COMMON PITFALL

Do not send a native C structure directly as a network format. Padding, byte order and type representation need an explicit encoding.

REMEMBER

The compiler may insert padding between fields to meet alignment rules. sizeof(Sample) can exceed the sum of field sizes.

Read the arrow and const precisely

Teaching example - C

```
Sample a = { .valueC = 24.5f, .valid = true };
Sample *p = &a;
p->valueC = 26.0f;
```

p->valueC means (*p).valueC: follow the pointer, then select a field. The arrow combines those two operations. a.valueC uses a structure object directly. p->valueC uses a pointer to one. The final line changes a.valueC to 26.0.

Teaching example - C

```
const Sample *view = &a;
Sample *const fixed = &a;
```

Through view, you may read the Sample but may not change its fields. You may assign view another address. fixed is a pointer whose stored address cannot change after initialisation, but its target remains writable. Thus fixed->valueC = 27.0f is allowed; fixed = anotherPointer is not.

Declaration	What is restricted?
const Sample *p	Changes through p to the Sample.
Sample *const p	Changes to p's address.
const Sample *const p	Both forms of change through this declaration.

const describes access through a declaration. It is not a lock. If another permitted pointer changes a, view can observe the new value. It also does not extend a's lifetime. We will use const Reading * when a rule should inspect a measurement without editing the supplied record.

When you see const SensorDriverOps *ops later, read it as a pointer to an operations table that this access path cannot modify. We will build that table only after a stepped driver gives us a reason to call different implementations.

COMMON PITFALL

const does not make a shared record immune to concurrent writes. Synchronisation is a separate question.

Read a real output-parameter call

You can now read both the output pointers and the arrow in a real HAL helper. AdcChannel is a record describing one ADC channel. Its pin_ field holds the assigned GPIO number. We will examine the ADC setup and clipping threshold in the analog chapter; here, follow where the returned information is written.

Original firmware - src/hal/adc_channel.c, lines 23-32

```
bool adcChannelReadMilliVolts(AdcChannel* ch, uint32_t* mv, bool* saturated) {
  if (mv) *mv = 0;
  if (saturated) *saturated = false;
  if (ch->pin_ < 0) return false;
  // eFuse-calibrated conversion (esp_adc_cal). One conversion, a few tens of us.
  const uint32_t value = analogReadMilliVolts((uint8_t)ch->pin_);
  if (mv) *mv = value;
  if (saturated) *saturated = value >= kSaturationMv;
  return true;
}
```

Here mv and saturated are output pointers. The HAL can return a voltage through one and a clipping indication through the other. if (mv) tests whether the pointer is non-null before writing through it. value is an ordinary local number returned by the Arduino ADC function.

The source allows omitted outputs by accepting NULL, but a teaching helper does not have to. Decide whether an output is required and enforce the contract. A pointer parameter alone does not tell you whether NULL is permitted.

Follow a successful call with local variables voltageMv and clipped. The caller supplies &voltageMv and &clipped. Inside the function, mv and saturated hold those addresses. *mv writes the voltage into the first caller object; *saturated writes a true/false clipping result into the second. The returned bool is a third result: whether the call could perform this operation.

The earlier readFake example (p. 23) required its output pointer. This HAL permits either output to be omitted. Read the null checks in each interface rather than assuming every pointer parameter follows the same rule.

Arrays provide storage; pointers do not count it

Teaching example - C

```
uint32_t samples[4] = {100, 101, 99, 102};
uint32_t first = samples[0];
size_t count = sizeof(samples) / sizeof(samples[0]);
```

This array reserves four consecutive uint32_t objects. The valid indices are 0, 1, 2 and 3. samples[4] is outside the array. first becomes 100, and count becomes 4 because the expression still sees the actual array object.

Teaching example - C

```
static float mean(const uint32_t *values, size_t n) {
    uint64_t sum = 0;
    for (size_t i = 0; i < n; ++i) {
        sum += values[i];
    }
    return n ? (float)sum / (float)n : 0.0f;
}
```

At a call such as mean(samples, 4), the array expression supplies a pointer to its first element. The function does not receive the array's size automatically. n is the length the caller promises. The loop visits exactly n elements. A false promise can make it read outside valid storage.

++i adds one to i. The condition i < n is checked before each visit. The ?: operator chooses its middle expression when n is nonzero and its last expression when n is zero. Here it prevents division by zero, although a production interface might report an explicit error instead.

size_t is an unsigned integer type used for object sizes and array lengths; sizeof produces a size_t result. Include stddef.h when a standalone C file needs that type. A size_t can hold a size supported by the target, but it does not make an incorrect buffer length safe.

Strings are byte arrays with a zero byte marking the end. A char buffer[8] can hold at most seven ordinary single-byte characters plus that terminator. A received radio payload is not automatically a terminated string; later we will pass an explicit length into its decoder.

COMMON PITFALL

A pointer and a length must agree. Checking n alone does not prove that values points to n real elements.

REMEMBER

sizeof(pointer) is the size of an address object, not the length of the buffer behind it.

What static actually keeps

Teaching example - C

```
static uint32_t countVisits(void) {
    static uint32_t count = 0;
    ++count;
    return count;
}
```

Call this function three times during one run. It returns 1, then 2, then 3. The local static count exists for the program's lifetime, even while no call is executing. Its name is visible only inside this function. A normal local initialised to zero would start over on each call.

The first static, before the function's return type, has a different role. It gives the function name internal linkage: another separately compiled C file cannot call it by that name. It does not make a private task stack or prevent two tasks from calling the function together.

Teaching example - C

```
static Sample latest;  // file scope
```

At file scope, static keeps the name private to this translation unit. The object also has static storage duration. A file-scope object without static would still live for the program's lifetime; the visibility of its name would differ. Static-storage objects without an explicit initializer are zero-initialised before application execution.

Persistent in this context means until reset or program termination. It does not mean saved in flash. To retain calibration across an ESP32 reboot, the Configuration service writes NVS, which we will study in saved configuration (p. 520).

If two tasks call countVisits(), they share the same count. ++count is a read-modify-write sequence. Without protection, concurrent calls can lose an increment. A private C name is not a synchronisation mechanism. This distinction will matter when we inspect the acquisition module's static driver objects.

COMMON PITFALL

static is not one copy per task. A static local belongs to the program, even when its declaration sits inside a task function.

Keep an address valid for long enough

Teaching counterexample - deliberately incorrect

```
static Sample *badAddress(void) {
    Sample temporary = { .valueC = 24.5f };
    return &temporary;  // invalid after return
}
```

temporary has automatic storage duration. Once the function returns, that object's lifetime ends. Its old address does not become safe because the bytes happen to remain unchanged for a while. A later call may reuse the storage. Do not use the returned pointer.

Storage	When does the object stop existing?
Automatic local	At the end of its enclosing block; each call has its own instance.
Static object	At program termination or reset in this firmware.
Allocated heap block	When released with the matching allocator's free operation.

A task keeps its current automatic locals while it is blocked. Blocking is not returning from the function. The RTOS preserves the task's execution state so it can resume. This is why a local variable inside a task loop can still be valid after xQueueReceive waits.

Heap allocation asks for storage at runtime. You must check failure, decide who owns the block, and free it exactly once when appropriate. The project often supplies static queue storage and task stacks instead. That makes much of its memory reservation visible before tasks begin.

Static allocation does not prove that the reservation is large enough. Stack overflow is still possible, and a buffer index can still be wrong. We will inspect actual stack usage and checks in debugging memory (p. 699).

REMEMBER

A queue that copies a record can safely copy a local before it goes out of scope. A queue that copies its pointer does not extend that local's life.

A header lets the compiler check a call

Teaching example - two files shown together

```
// water.h
#pragma once
#include <stdbool.h>
bool waterIsTooWarm(float waterC);

// water.c
#include "water.h"
bool waterIsTooWarm(float waterC) {
    return waterC >= 30.0f;
}
```

The header declares the interface: function name, return type and parameter types. The .c file defines the body. A caller includes water.h so the compiler can check its arguments. The compiler builds each source file into an object file; the linker then resolves calls between those object files.

#include is a preprocessing operation. It makes header text available while compiling. It does not call the functions declared there. #pragma once prevents a header from being processed repeatedly in one translation unit on this toolchain. A translation unit is one source file after its includes and preprocessing have been applied.

Teaching example - C

```
// in a header
extern uint32_t sampleCount;
// in exactly one .c file
uint32_t sampleCount = 0;
```

COMMON PITFALL

Do not put ordinary writable global definitions in a header included by several .c files. You may create multiple definitions rather than one shared object.

A header lets the compiler check a call / continued

extern declares an externally defined object without making another ordinary definition here. Prefer functions that return copies of shared state when you can; a globally writable object makes ownership harder to see. The firmware's snapshotPeek() interface is an example we will read in the snapshot store (p. 57).

You will also see small function bodies written as static inline in a header. static gives each translation unit its own internally linked helper. inline allows the compiler to consider expanding that helper at a call site; it does not guarantee that choice or create a task. The shared types header uses this form for small timestamp comparisons.

If compilation fails, ask whether a declaration or type is visible. If linking fails with an undefined reference, ask whether the defining source was built and whether its symbol has the expected linkage. These are different stages, so they need different repairs.

Name outcomes instead of guessing numbers

Teaching example - C

```
typedef uint8_t Status;
enum { STATUS_OK = 0, STATUS_PENDING, STATUS_FAULT };

Status st = STATUS_PENDING;
if (st == STATUS_PENDING) {
    // The operation has not finished yet.
}
```

The anonymous enum supplies named integer constants. With the explicit zero first, STATUS_PENDING is 1 and STATUS_FAULT is 2. The typedef makes Status an alias for uint8_t. This project uses that pattern to keep record fields small and their intent readable.

The name is a reading aid, not a distinct type barrier: Status and another uint8_t alias can still be assigned to one another in C. Use clear parameter names and checks where an accidental interchange would matter.

Completion and validity are different questions

A driver can finish an operation and report that the measurement failed. In this project, DRIVER_READY means the driver has an outcome available. The returned Reading may carry HEALTH_TIMEOUT or HEALTH_FAULT. READY does not guarantee a usable physical value.

Why keep both? The acquisition engine needs to know when it can collect an outcome and start scheduling the next job. A consumer needs to know whether the stored number may be used. Combining those questions into one boolean would hide useful information.

The production driver interface will use a PollResult record containing a status and the earliest sensible time to poll again. Before opening that interface, read two more C operations used throughout these files: selecting a named status and copying a byte range. Then we can ask what an RTOS task does while it waits.

COMMON PITFALL

Do not write if (status) unless the interface defines truthiness. STATUS_OK is zero here, so a successful outcome would test false.

Choose one branch with switch

Teaching example - uses Status from the previous lesson

```
static const char *statusName(Status st) {
    switch (st) {
        case STATUS_OK:      return "OK";
        case STATUS_PENDING: return "PENDING";
        default:             return "FAULT";
    }
}
```

switch evaluates st, then begins at the case whose integer constant matches. With STATUS_PENDING, execution starts at the second case and returns the string "PENDING". default handles a value that matches no listed case. Here the return type is a pointer to characters that this function does not permit you to change.

A case label does not automatically end its branch. In this example, return ends the whole function. In a branch that should continue after the switch, break exits just the switch. Without either, execution continues into the following case; this is called fall-through.

Teaching fragment - accepted is a previously declared bool

```
switch (st) {
    case STATUS_OK:
    case STATUS_PENDING:
        accepted = true;
        break;
    default:
        accepted = false;
        break;
}
```

The two adjacent labels deliberately share one body. Either OK or PENDING assigns true, then break skips the default branch. Later, reading.c groups several metric labels this way because one BME280 produces temperature, humidity and pressure. You now know why several cases can lead to one return.

COMMON PITFALL

A missing break can execute the next case body. Check where every branch actually ends.

Copy the bytes of a complete object

Teaching fragment - Sample was defined in the records lesson

```
#include <string.h>

Sample a = { .valueC = 24.5f,
             .measuredMs = 1000, .valid = true };
Sample copied;
memset(&copied, 0, sizeof(copied));
memcpy(&copied, &a, sizeof(copied));
```

memset receives a destination address, a byte value and a byte count. This call fills every byte of copied with zero. It does not fetch a measurement. memcpy then receives destination, source and count; it replaces those bytes with the complete representation of a. string.h declares both functions.

After the copy, copied.valueC is 24.5, copied.measuredMs is 1000 and copied.valid is true. Changing a.valueC afterward does not change copied.valueC. For this record, copied = a would be the clearer ordinary assignment. The byte functions matter because you will also see fixed buffers, queue storage and checksum-covered records.

The count is sizeof(copied), the size of the whole destination object, including any padding. sizeof(&copied) would instead give the size of a pointer and copy the wrong amount. Both addresses must name enough live storage; memcpy does not check your promise. Its ranges must not overlap. Use memmove when the ranges can overlap, as when shifting an existing byte history.

Zero-filled bytes are not a universal constructor for every C type. Use the type's initializer or reset helper when you need a meaningful initial state. In this example the complete copy follows immediately, so we do not interpret the intermediate zero-filled record. Later, ConfigBlob deliberately initialises its bytes before calculating a checksum that also covers padding.

REMEMBER

A byte count describes storage. It does not prove that the address points to that much storage.

Choose code before the program runs

Teaching fragment - preprocessing choices

```
#define SAMPLE_PERIOD_MS 1000u

#ifdef FW_ENABLE_BLE
void startRadio(void);
#endif
```

A macro is a name the preprocessor replaces while preparing a source file for compilation. SAMPLE_PERIOD_MS becomes the token 1000u wherever this macro is used. The define does not create a writable variable, reserve a timer or make a task periodic. A later function call must use that value to do something.

#ifdef asks whether a macro name is defined. Its enclosed text reaches the compiler only in that case. #endif closes the choice; #else can supply another branch. This differs from a C if statement: an ordinary if chooses while the program runs, whereas preprocessing chooses which source is compiled into the image.

A build option such as -D FW_ENABLE_BLE=1 defines the name without adding a define line to the source. With #ifdef, even defining it as zero still counts as defined. The firmware uses these choices to omit radio implementations from isolated bring-up images. You will recognise those blocks before needing to understand their contents.

The shared types header follows the enum lesson because it uses enum, typedef and static inline together. Later headers reuse those names through #include. The chain is now concrete: preprocessing supplies declarations, compilation checks each file, and linking connects the built functions.

Reference: WG14: public C11 committee draft N1570

COMMON PITFALL

Changing a macro changes the next build. It cannot change the program already flashed onto the board.

Give time an explicit representation

You can now read typedef, enum, static inline and a header declaration. This production header combines those ideas into the vocabulary every other file uses: timestamps, sensor identity, metric identity and units.

A Reading uses these names later; for now recognise which category each field belongs to. Review the enum lesson (p. 32) for the numbered names before following their production lists.

src/domain/types.h / lines 1-14

```
// Basic shared types. Domain layer: pure C, no Arduino/FreeRTOS includes.
//
// Enumerations are stored as uint8_t (the typedefs below) with the values in
// an anonymous enum. This keeps every record exactly the size it had with the
// C++ `enum class X : uint8_t` types (queue item sizes, RAM).
#pragma once
#include <stdbool.h>
#include <stdint.h>

// Monotonic milliseconds since boot. Used for every age and deadline.
// Never wall-clock/NTP time (§8). 64-bit so it does not wrap in practice.
typedef uint64_t MonoMs;

static inline bool monoReached(MonoMs now, MonoMs deadline) { return now >= deadline; }
```

MonoMs names a 64-bit unsigned integer whose unit is milliseconds. The alias makes intent visible while leaving ordinary integer arithmetic available. monoReached compares two timestamps; it does not wait or schedule anything. If now is 1749 and deadline is 1750, the return is false. At 1750 it is true.

The inline helper lives in the header so each caller sees its definition. static keeps a separate internal name in each translation unit. Neither keyword creates an RTOS task. We will supply now from the monotonic clock rather than a calendar clock.

COMMON PITFALL

A type alias cannot detect a microsecond value accidentally assigned to a millisecond variable.

Name sensors separately from measurements

src/domain/types.h / lines 15-41

```
15
16 typedef uint8_t SensorId;
17 enum {
18   SENSOR_BME280 = 0,
19   SENSOR_DS18B20,
20   SENSOR_LIGHT,   // DFR0026
21   SENSOR_HCSR04,
22   SENSOR_PH,      // pH-4502C
23   SENSOR_TDS,     // SEN0244
24   SENSOR_SGP30,   // TVOC / CO2eq (replaced the MQ-7, D18)
25   SENSOR_COUNT
26 };
27
28 // One entry per published metric. A sensor may produce
 >   several metrics.
29 typedef uint8_t MetricId;
30 enum {
31   METRIC_AIR_TEMPERATURE = 0,  // BME280
32   METRIC_AIR_HUMIDITY,         // BME280
33   METRIC_AIR_PRESSURE,         // BME280
34   METRIC_WATER_TEMPERATURE,    // DS18B20
35   METRIC_LIGHT_LEVEL,          // DFR0026 (mV until
 >   calibrated, then lux)
36   METRIC_WATER_DISTANCE,       // HC-SR04
37   METRIC_PH,                   // pH-4502C
38   METRIC_TDS,                  // SEN0244
39   METRIC_TVOC,                 // SGP30, ppb
40   METRIC_CO2EQ,                // SGP30, ppm (estimate, not
 >   measured CO2)
41   METRIC_COUNT
```

SensorId identifies a physical device. MetricId identifies one reported quantity. Count the BME280 entries: one sensor produces temperature, humidity and pressure. SGP30 contributes TVOC and CO2eq. This is why the counts differ.

The enumeration starts at an explicit zero and increments each following value. SENSOR_COUNT and METRIC_COUNT are sentinels used for array bounds; they are not real devices or observations. A loop with index < METRIC_COUNT visits every valid metric index without visiting the sentinel.

COMMON PITFALL

SensorId and MetricId are both uint8_t aliases. Their different names explain intent but do not make accidental interchange a C type error.

REMEMBER

Review named status values (p. 32) for enum numbering and number notation (p. 21) for the byte representation.

Keep the unit beside the number

src/domain/types.h / lines 42-57

```
42 };
43
44 typedef uint8_t Unit;
45 enum {
46   UNIT_NONE = 0,
47   UNIT_DEG_C,
48   UNIT_PERCENT_RH,
49   UNIT_PASCAL,
50   UNIT_MILLIVOLT,
51   UNIT_MILLIMETRE,
52   UNIT_PH,
53   UNIT_PPM,
54   UNIT_ADC_RAW,
55   UNIT_PPB,
56   UNIT_LUX   // light after an on-device fit (WP8 calibration). Appended: values above
 >   unchanged
57 };
```

The Unit constants label the interpretation of Reading.value. The same float storage can hold 24.5 degrees Celsius or 24.5 millivolts, but consumers must not compare those as though they meant the same physical quantity.

Notice that UNIT_LUX is appended. Stable numeric identifiers matter when records or protocols use existing values. In a new design you could use a different representation, but while reviewing this one, do not casually reorder an enum whose values cross a module or communication boundary.

Name the work the kernel will organise

Firmware is the program that makes this controller operate its hardware after reset. In this project, the Arduino framework already starts FreeRTOS. Your setup function creates application tasks within that running system. You do not need to start a second scheduler.

RTOS means real-time operating system. Its kernel keeps task execution state and implements scheduling and communication operations. The scheduler selects which eligible task runs on each core. Application code still decides which reading is usable, which deadline matters and what a failed operation should do.

Real-time asks whether a result arrives in time to serve its purpose. A water conversion can legitimately take 750ms. An ultrasonic edge needs a much finer timestamp. Giving both the same task priority or wait duration would ignore the physical jobs they perform. You will learn to separate elapsed conversion time, occupied CPU time and allowed response delay.

Responsibility	Owner in this project
Keep task state and select execution	FreeRTOS kernel and scheduler
Start and collect sensor operations	The active acquisition worker
Choose the final relay command	Safety
Save accepted settings	Configuration

Read the next task function as ordinary C executed in a kernel-managed context. We will first watch it wait and resume. Then we will create a second task and follow the messages that connect them.

Reference: Espressif: ESP-IDF FreeRTOS SMP, v4.4.2 (S3)

REMEMBER

A task has execution state. A sensor is a physical device. The architecture decides how many devices one task manages.

A task is a resumable function

Teaching fragment - checkLatestValue is a stand-in

```
static void readerTask(void *arg) {
    (void)arg;
    for (;;) {
        checkLatestValue();
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

This function repeatedly checks a value and asks the RTOS to wait. A task is more than the function body: the kernel also keeps its stack, current execution position, saved CPU registers and scheduling information. That saved context lets it pause here and resume after the wait.

The stack is the task's working storage for nested calls and automatic local variables. For example, a local Sample inside readerTask must survive while that task is blocked in a wait. The kernel preserves the task's saved execution context so the same call can continue with its existing locals. A later function return, unlike a wait, can end a local object's lifetime.

At the delay, readerTask becomes Blocked. The kernel can run a different Ready task on that core. When the delay expires, readerTask becomes Ready. It resumes only when the scheduler selects it; expiry does not promise immediate execution.

(void)arg deliberately ignores an unused parameter. The void * parameter is a generic pointer accepted by the task API. We will pass a real address through such a parameter when needed. for (;;) is an infinite loop: none of its three optional control expressions is supplied.

A normal helper returns to its caller. A long-lived task usually loops instead. If a finite task finishes its work, it should use the port's task-deletion API rather than return from its task entry function. In our first controller both tasks live until reset.

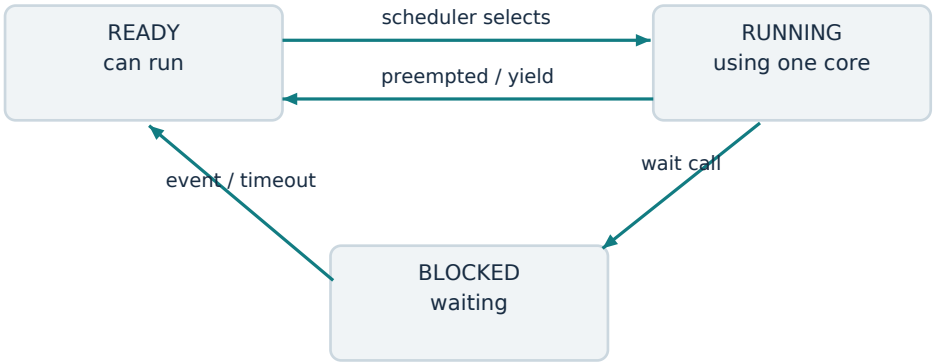
The RTOS does not rewrite your code into parallel instructions. Each core still executes one task at a time, interrupted when necessary. On one core, two tasks make progress by taking turns. On two cores, two eligible tasks may actually execute at the same time.

Reference: Espressif: ESP-IDF FreeRTOS SMP, v4.4.2 (S3)

REMEMBER

Task context is the execution state needed to resume a task. A normal C function call does not create a task.

Ready is permission, not execution



The three states needed for our examples. Suspended is a separate explicit state.

COMMON PITFALL

A task becoming Ready does not mean the instruction after its wait has executed. Keep that delay in your response-time reasoning.

Running means the task is currently executing on a core. Ready means it could execute if selected. Blocked means it is waiting for an event or a time limit and is not a scheduling candidate. Suspended means an explicit suspend operation has removed it from normal scheduling until resumed.

Suppose Reader waits on an empty queue for 100ms. At 40ms, Producer inserts a value. Reader becomes Ready because the event arrived. If a higher-priority eligible task still needs the core, Reader remains Ready until that task blocks or is preempted by another event.

If no value arrives, the timeout instead makes Reader Ready at the relevant tick. When Reader eventually runs, the queue call can return failure. The return value tells you whether it actually obtained an item; simply resuming is not proof of success.

Blocked time consumes elapsed time, not CPU execution

A blocked task still has its allocated stack and kernel record. It simply does not use instructions to repeatedly test its condition. If all application tasks block, the idle task gets the core; the system may then use its configured power-saving behaviour.

This will be useful for the ultrasonic driver: the acquisition task can wait until the next useful poll time while an interrupt records the echo edges. We will separate the event measurement from the task's later processing of it.

Reference: Espressif: ESP-IDF FreeRTOS SMP, v4.4.2 (S3)

Priority chooses among eligible tasks

In the application table, Safety has priority 6, acquisition has priority 4, and SerialLogger has priority 1. A larger number means a higher application task priority. When all three are Ready and pinned to core1, Safety runs first. Once Safety blocks, acquisition can run. Once acquisition blocks, the logger can run.

Core	Ready application tasks	Highest eligible choice
0	PowerCoord 5; BLE 2; Supervisor 1	PowerCoord
1	Safety 6; AcqNormal 4; Logger 1	Safety

This is a worked scheduling snapshot, not a complete system trace. ESP32 framework tasks and interrupts also exist. Core affinity restricts where an application task may run: a task pinned to core1 cannot fill unused time on core0. The two largest priority numbers across the whole application are therefore not always the two running tasks.

Preemption means a higher-priority eligible task can replace the current task without waiting for it to finish its function. The kernel preserves the interrupted context. Time slicing gives eligible equal-priority tasks opportunities to run, but the ESP-IDF dual-core implementation has affinity-related details; do not depend on an exact alternating order.

Priority is not CPU speed. A priority-6 task executes the same addition at the same clock rate as a priority-1 task. Priority changes when it gets CPU time. If the high-priority task remains Ready forever, lower-priority work can starve.

The source table documents a reason for pinning Safety to core1: the project had observed Wi-Fi startup interference on core0. Treat that comment as project history, not a fresh latency measurement from this edition. Pinning still does not remove every possible delay.

Reference: Espressif: ESP-IDF FreeRTOS SMP, v4.4.2 (S3)

REMEMBER

Eligible means Ready and allowed on this core. Interrupts are not ordinary Ready tasks.

Read the task-creation call

Teaching example - C

```
TaskHandle_t handle = NULL;
BaseType_t ok = xTaskCreatePinnedToCore(
    readerTask, "Reader", 4096, NULL,
    2, &handle, 1);
if (ok != pdPASS) {
    // Do not continue as if Reader exists.
}
```

The first argument is the entry function, not a call to it. readerTask names the function; readerTask(...) would call it now. The second is a diagnostic name. On this ESP-IDF port, 4096 specifies stack bytes. NULL is the argument delivered to readerTask. 2 is the task priority. &handle lets the kernel write back a task handle. 1 pins it to core1.

BaseType_t is a FreeRTOS integer typedef used for many API outcomes and flags. Here ok records the task-creation outcome; pdPASS is the success value this API documents. Later queue calls compare their result with pdTRUE. Read each API's promised result instead of treating these names as a measurement-valid flag. StaticTask_t, used by the static creation API, holds the kernel's task bookkeeping; the separate stack array holds the task's working storage.

A handle identifies a kernel object. It is not the measurement that the task produces. You use a task handle with APIs such as xTaskNotifyGive(). You use a queue handle with queue APIs. Keep those roles distinct even if both are represented by pointer-like typedefs.

COMMON PITFALL

Creating a task twice makes two execution contexts. An initialisation function should guard against accidental repeated creation.

Read the task-creation call / continued

This dynamic creation API allocates the task's memory. The original firmware usually uses xTaskCreateStaticPinnedToCore(), supplying persistent stack storage and a StaticTask_t control block itself. That gives the application control over the storage; the resulting task still uses the same scheduling states.

A created task can run before the creation function's caller has finished its next line. Initialise objects it depends on first. For our learning program, create the queue before creating either task. Later, production setup() also creates and publishes configuration before starting its consumers.

Stack depth units differ between vanilla FreeRTOS and ESP-IDF. Read the API for the pinned version you actually compile. Do not multiply a byte count by four because another tutorial used words.

Reference: Espressif: ESP-IDF FreeRTOS SMP, v4.4.2 (S3)

A wait is not a spin loop

Teaching example - C

```c
while (!ready) {
    // Rechecks continuously: remains Ready/Running.
}

vTaskDelay(pdMS_TO_TICKS(100));
// Blocks this task for a tick-based interval.
```

REMEMBER

Yield means give the scheduler another choice. Block means stop being eligible until something changes.

The while loop is a scheduling sketch: assume ready is obtained through a valid sharing mechanism. An ordinary bool shared between tasks is not, by itself, that mechanism. We will examine shared-state protection in the volatile lesson (p. 71). Here, compare repeatedly checking a condition with asking the kernel to block.

The empty while loop spends CPU time loading ready and testing it. A blocking RTOS wait removes the task from the Ready set until its condition or timeout changes. That gives other tasks a useful opportunity to run.

The RTOS measures many waits in ticks. If the configured tick rate were 100Hz, one tick would be 10ms; at 1000Hz it would be 1ms. pdMS_TO_TICKS converts a duration to the tick count used by the build. It does not create a microsecond timer. Read the configured rate rather than assuming one.

Conversion can round small values down to zero. A zero-time queue receive checks immediately. vTaskDelay(0) does not request a positive delay; it can yield to an eligible equal-priority task, but it does not make lower-priority work win over a still-Ready higher-priority task. taskYIELD() has the same basic limitation.

Even a positive one-tick delay is not a guaranteed minimum physical delay of one full tick period from the instant you call it; the call may occur just before a tick. For minimum sensor conversion waits, remember the absolute due time and check the monotonic clock again before reading.

A measured interval and a wake request have different precision. We will use microsecond timestamps to measure an ultrasonic pulse and a coarse wait to decide when a task should inspect the finished result. That separation avoids trying to time a 1000us pulse with a scheduler tick.

Reference: Espressif: ESP-IDF FreeRTOS SMP, v4.4.2 (S3), FreeRTOS: delay-until API

Select the running task on each core

Use a simplified scheduling snapshot with only five application tasks. PowerCoord has priority 5 on core0; BLE has priority 2 on core0. Safety has priority 6 on core1; acquisition has priority 4 on core1; logger has priority 1 on core1. Ignore interrupts, framework tasks and equal-priority ties for this calculation.

Change	Core0 runs	Core1 runs
Initially: PowerCoord, BLE, acquisition and logger Ready; Safety Blocked	PowerCoord	Acquisition
Safety becomes Ready; other states unchanged	PowerCoord	Safety
PowerCoord blocks; Safety stays Ready	BLE	Safety
Safety blocks; acquisition stays Ready	BLE	Acquisition

In the first row, Safety's larger priority does not help because it is Blocked. In the second, it preempts acquisition on core1. PowerCoord cannot move to core1, and acquisition cannot occupy core0 when PowerCoord blocks. Affinity and readiness are checked before comparing priorities for that core.

These rows are predictions under stated conditions, not a measured controller trace. The table gives you a method to use when reading TaskSpec: list each core's eligible candidates, select its highest priority, and update the candidates after every block or wake. An unpinned task introduces an additional rule: one task cannot execute on both cores at once.

Reference: Espressif: ESP-IDF FreeRTOS SMP, v4.4.2 (S3)

REMEMBER

A high-priority Blocked task uses no core. A pinned Ready task must wait for its assigned core.

Separate release periods from call budgets

You have learned what a task and a timeout are. This header now shows the controller's actual scheduling and memory choices. Read the constants as the application's intended budgets; the later driver and service chapters show how they are used.

Configuration values do not enforce themselves. Find the call that consumes each value before assuming it changes runtime behaviour.

include/app_config.h / lines 1-25

```
1  // Application timing, queue and task budgets.
2  // Source: Firmware Architecture §6 (tasks), §7 (schedule),
   >   §8 (staleness),
3  // §9 (queues), §11 (power). These are DESIGN BUDGETS, not
   >   measurements:
4  // replace with measured values as each stage is validated.
5  //
6  // Pure C (no Arduino headers).
7  #pragma once
8  #include <stdint.h>
9
10 // ---- Schedule (§7)
   >   ------------------------------------------------------------
   >   -
11 #define CFG_FRAME_PERIOD_MS ((uint32_t)1000)         //
   >   ordinary sensors
12 #define CFG_BME280_PERIOD_MS ((uint32_t)250)         // fast
   >    climate check
13 #define CFG_BME280_BUDGET_MS ((uint32_t)12)          //
   >   conversion + read
14 #define CFG_DS18B20_CONVERSION_MAX_MS ((uint32_t)750)  //
   >   12-bit
15 #define CFG_DS18B20_BUDGET_MS ((uint32_t)20)
16 #define CFG_HCSR04_TIMEOUT_MS ((uint32_t)30)
17 #define CFG_ANALOG_GROUP_BUDGET_MS ((uint32_t)5)
18 #define CFG_SNAPSHOT_BUDGET_MS ((uint32_t)10)
19 #define CFG_FRAME_RESERVATION_MS ((uint32_t)150)
20 #define CFG_SERVICE_STEP_TARGET_MS ((uint32_t)5)
21
22 // ---- Staleness (§8)
   >   ------------------------------------------------------------
   >   --
23 #define CFG_STALE_BME280_MS ((uint32_t)1000)
24 #define CFG_STALE_ORDINARY_MS ((uint32_t)3000)
25
```

COMMON PITFALL

Do not add all elapsed conversion times as if the worker spends CPU time on every one of them.

Separate release periods from call budgets / continued

The ordinary frame repeats every 1000ms; climate slots repeat every 250ms. These say how often work is requested. A 750ms DS18B20 conversion allowance says how long a physical conversion can take. A 20ms transaction budget describes a shorter piece of CPU or bus work. Those are different dimensions of the schedule.

include/app_config.h / lines 26-34

```
26 // ---- Queues (§9) ------------------------------------------------------
27 #define CFG_ALARM_DETAIL_QUEUE_LEN ((uint32_t)16)
28 #define CFG_ALARM_JOURNAL_LEN ((uint32_t)64)
29 #define CFG_COMMAND_QUEUE_LEN ((uint32_t)8)
30 #define CFG_TELEMETRY_BACKLOG_LEN ((uint32_t)60)
31 #define CFG_LOG_QUEUE_LEN ((uint32_t)32)
32
33 // ---- Power transitions (§11) ------------------------------------------
34 #define CFG_POWER_LOSS_DEBOUNCE_MS ((uint32_t)50)
```

A 5ms service-step target is a design goal, not proof that every call finishes in 5ms. When the source later admits a longer blocking operation, acquisition uses overlap guards rather than pretending that a macro made it short.

Here an **ordinary frame** means a recurring release of the ordinary sensor jobs. A **climate slot** is one opportunity to release the faster BME280 job. Starting a frame does not mean every sensor finishes together. The communication chapter will also use frame for a length-prefixed message; that is a different use of the word. We will trace the actual sensor releases in acquisition scheduling (p. 340).

Choose how long information may wait

include/app_config.h / lines 35-58

```
35 #define CFG_POWER_RETURN_STABLE_MS ((uint32_t)500)
36 #define CFG_TRANSITION_ACK_MS ((uint32_t)200)
37 #define CFG_NETWORK_SHUTDOWN_ACK_MS ((uint32_t)2000)
38
39 // ---- Network (§15) ------------------------------------------------------
40 #define CFG_MQTT_BACKOFF_INITIAL_MS ((uint32_t)1000)
41 #define CFG_MQTT_BACKOFF_MAX_MS ((uint32_t)60000)
42 #define CFG_BLE_STATE_PERIOD_MS ((uint32_t)2000)
43 #define CFG_BLE_TELEMETRY_PERIOD_MS ((uint32_t)3000)
44
45 // ---- Safety / supervision (§10, §16) ------------------------------------
46 #define CFG_SAFETY_HEALTH_CHECK_MS ((uint32_t)100)
47 #define CFG_SUPERVISOR_PERIOD_MS ((uint32_t)1000)
48 #define CFG_TASK_WATCHDOG_MS ((uint32_t)5000)
49
50 // ---- Tasks (§6). Higher number = higher priority. Stack in bytes. -------
51 typedef uint8_t TaskId;
52 enum {
53   TASK_ID_SAFETY = 0,
54   TASK_ID_POWER_COORDINATOR,
55   TASK_ID_ACQUISITION_NORMAL,
56   TASK_ID_ACQUISITION_LOW_POWER,
57   TASK_ID_WIFI_MQTT,
58   TASK_ID_BLE_TRANSPORT,
```

The stale limits describe when a retained observation stops being eligible. The queue lengths describe storage capacity. Debounce and stable-return values describe power qualification. A task can wake for one purpose while checking another limit against the monotonic clock.

The detail queue has 16 slots, while the journal retains 64 events. The two buffers serve different readers and failure contracts. A full queue does not imply the journal is full, and increasing the journal does not prevent an immediate detail-queue overflow.

Assign names to the task table

include/app_config.h / lines 59-85

```
  TASK_ID_CONFIGURATION,
  TASK_ID_SUPERVISOR,
  TASK_ID_SERIAL_LOGGER,
  TASK_ID_COUNT
};

typedef struct TaskSpec {
  const char* name;
  uint8_t core;
  uint8_t priority;
  uint32_t stackBytes;
} TaskSpec;

static inline const TaskSpec* cfgTaskSpec(TaskId id) {
  // Indexed by TaskId. Keep order in sync with the enum
  above.
  static const TaskSpec kTaskSpecs[TASK_ID_COUNT] = {
      // Core 1 since 28 Oct 2026 (D14 fallback): the Wi-Fi
  driver's start-up
      // starved Safety on core 0 for 40-110 ms; on core 1
  the worst case is
      // 145 us. Frozen-header change made on the project
  owner's decision.
      {"Safety", 1, 6, 3 * 1024},
      {"PowerCoord", 0, 5, 4 * 1024},
      {"AcqNormal", 1, 4, 4 * 1024},
      {"AcqLowPower", 0, 4, 4 * 1024},
      {"WifiMqtt", 0, 2, 6 * 1024},
      {"BleTransport", 0, 2, 6 * 1024},
      {"Configuration", 0, 2, 4 * 1024},
      {"Supervisor", 0, 1, 3 * 1024},
```

TaskId supplies array indices for a fixed table of TaskSpec records. Each record groups a diagnostic name, core, priority and stack-byte budget. A caller passes its TaskId to cfgTaskSpec and obtains a pointer to the matching constant record.

The function returns an address, not a new task. A later service uses that address to obtain creation parameters. This is a useful place to pause: declaration, configuration lookup and task creation are three separate operations.

COMMON PITFALL

The helper assumes id is within its enum range. It is an internal lookup, not a validator for arbitrary external input.

Read the actual priority and affinity choices

include/app_config.h / lines 86-92

```
86        {"SerialLogger", 1, 1, 2 * 1024},
87    };
88    return &kTaskSpecs[id];
89 }
90
91 // ---- CPU frequency (§13) - 80 MHz low power is only a
 >    candidate --------------------
92 #define CFG_CPU_MHZ_NORMAL ((uint32_t)160)
```

Safety is assigned core 1 and priority 6; normal acquisition uses core 1 and priority 4. Power coordination uses core 0 and priority 5. The active worker, framework tasks and interrupts still determine the real execution trace.

The source comment records a past observation about Wi-Fi startup interference. It explains why this project chose an affinity. It is not a guarantee for another board, another SDK or another execution path. Use the table to predict eligibility, then use measurements to assess delay.

COMMON PITFALL

The declared stackBytes values use this port's byte convention. Keep that convention consistent with the allocation and creation call.

Reserve space for messages

Teaching example - C

```
QueueHandle_t samples = xQueueCreate(
    4, sizeof(Sample));
if (samples == NULL) {
    // Allocation failed: no queue is available.
}
```

A queue is a kernel-managed buffer with a maximum number of items and a fixed item size. This call reserves four slots, each large enough for a Sample. The queue also has internal bookkeeping. It is not just a plain array that both tasks update without coordination.

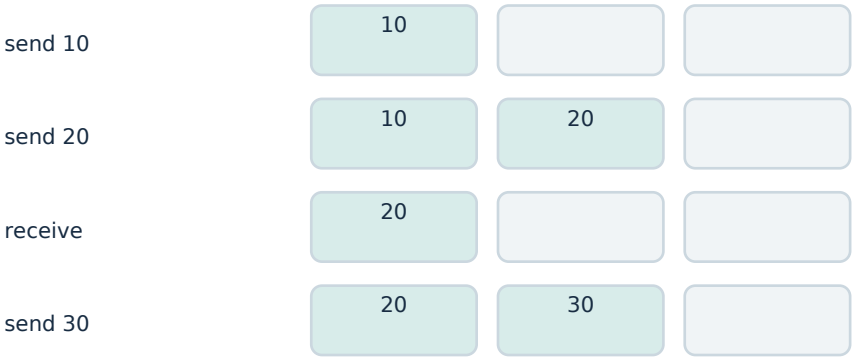
The first parameter is the number of messages, not bytes. The second is bytes per message, not a pointer to the type. sizeof(Sample) asks the compiler for the actual record size, including alignment padding. Every send and receive must use storage compatible with that size.

For a FIFO queue, a successful send to the back adds an item after existing items; a receive removes the oldest one. FIFO means first in, first out. A queue can preserve order among accepted messages, but it cannot accept an unlimited stream if the consumer is slower than the producer.

COMMON PITFALL

A successful queue creation does not mean it contains an item. A new queue starts empty.

Reserve space for messages / continued



Each row shows the queue after the named operation. The left slot is oldest.

The production snapshot store instead needs one latest value. It creates a one-item queue and overwrites that item. We will compare that contract with a FIFO after first following an ordinary send and receive.

The teaching program uses dynamic creation for a short listing. The production snapshot queue has caller-supplied static storage.

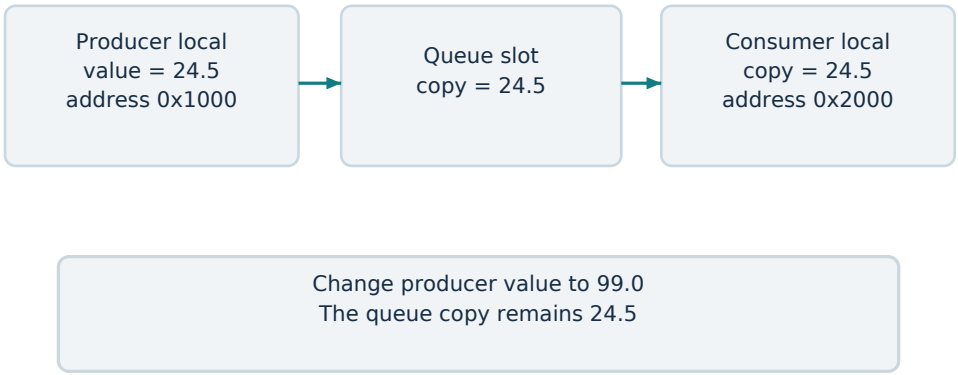
Reference: FreeRTOS: queue concepts

Sending copies bytes from your object

Teaching example - C

```
Sample sent = { .valueC = 24.5f,
                .measuredMs = 1000, .valid = true };
BaseType_t ok = xQueueSend(samples, &sent, 0);
sent.valueC = 99.0f;
```

The queue was created with sizeof(Sample). If send succeeds, it copies that many bytes from the address &sent into a queue slot before returning. Changing sent.valueC afterward does not change the stored record. The final argument 0 says: do not wait for space if the queue is full.



The addresses are illustrative. The three records occupy different storage.

The API accepts a pointer because it needs to know where the source bytes are. That does not mean the queue stores that pointer as the message. The queue's configured item size decides what it copies.

COMMON PITFALL

&sent is the source address for a record copy. &p is the source address for a pointer copy. Match both to the queue's item size.

Sending copies bytes from your object / continued

Teaching contrast - a queue of pointers

```
QueueHandle_t addresses = xQueueCreate(4, sizeof(Sample *));
Sample *p = &sent;
xQueueSend(addresses, &p, 0);
```

This second queue copies only the pointer object p. Its receiver can still reach the same sent object, whose value may now be 99.0. You must define how long sent lives and who may modify it. For small sensor records, copying the value avoids that extra agreement.

This is why the Hydroponics Reading contains plain values rather than a pointer to a temporary text buffer. A queued copy can outlive the driver function that produced it.

Reference: FreeRTOS: queue concepts

Check what the wait actually returned

Teaching fragment - useSample/reportNoNewSample are stand-ins

```
Sample received = {0};
BaseType_t got = xQueueReceive(
    samples, &received, pdMS_TO_TICKS(100));
if (got == pdTRUE) {
    useSample(&received);
} else {
    reportNoNewSample();
}
```

If a message is already queued, receive copies it into received and removes it immediately. If the queue is empty, the task can block for up to the requested tick interval. An arriving item can wake it early. The task then resumes according to priority and core eligibility.

If the wait ends without an item, the call returns failure and does not supply a fresh Sample. The previous contents of received are not evidence of a new result. The else branch matters even if the last successful value still looks plausible.

Trace a successful wake

At 0ms Reader calls receive on an empty queue. At 40ms Producer sends 24.5. Reader becomes Ready. When it runs, receive returns pdTRUE, and received.valueC is 24.5. The queue slot is now free. The 100ms timeout was an upper waiting budget, not an instruction to delay for the full 100ms.

Trace a timeout

No producer sends anything. The timeout expires; Reader becomes Ready. When it runs, receive returns failure. Your code must choose what to do: retry later, retain the previous display with an age, or declare missing data. The kernel does not choose the application's fault policy.

useSample() is a placeholder for the action being taught. It is not a FreeRTOS API or an original project function.

Reference: FreeRTOS: queue concepts

REMEMBER

An output parameter only contains a new result when the function's success contract says it does.

Keep the latest state for many readers

Original firmware - src/services/snapshot_store.c, lines 10-24

```
bool snapshotStoreInit(void) {
  if (s_queue == NULL) {
    s_queue = xQueueCreateStatic(1, sizeof(Snapshot),
>   s_queueStorage, &s_queueControl);
  }
  return s_queue != NULL;
}

void snapshotPublish(const Snapshot* snapshot) {
  if (s_queue != NULL) xQueueOverwrite(s_queue, snapshot);
}

bool snapshotPeek(Snapshot* out) {
  if (s_queue == NULL || out == NULL) return false;
  return xQueuePeek(s_queue, out, 0) == pdTRUE;
}
```

This is the complete behaviour that matters in snapshot_store.c. The static storage declarations above it reserve one Snapshot item and its queue control block. snapshotStoreInit creates the queue once. snapshotPublish replaces the current item. snapshotPeek copies the current item into caller-owned storage and leaves the queue full.

Use xQueueReceive when a message should be consumed. Use xQueuePeek when the item should remain available. Safety and BLE can each peek the latest Snapshot without stealing it from one another. Their copies may come from different publications if acquisition updates the queue between the two peeks.

COMMON PITFALL

A latest-value mailbox cannot preserve every alarm transition. A later healthy state could overwrite a brief fault before a reader sees it.

Keep the latest state for many readers / continued

xQueueOverwrite is intended for a queue of length one. It is a good match here because the question is: what is the newest retained system state? If acquisition publishes A, B and C before BLE reads, BLE gets C. Losing A and B is allowed by this interface.

A Snapshot is one coherent memory copy, not simultaneous sensing. Its ten readings may have different acquisition times. A water-temperature value can be older than the air-temperature value beside it. Consumers must check each reading's timing instead of assuming the frame timestamp makes every field fresh.

We will return to this store in the result path (p. 343). For now you can read its purpose without knowing every field of Snapshot: one writer publishes a retained copy; several readers copy it without consuming it.

Reference: FreeRTOS: queue concepts

Choose what happens when the queue is full

Suppose Producer sends 10 samples per second and Consumer handles 6 per second. The backlog grows by four samples per second. A queue with 16 initially empty slots fills after roughly 4s of this steady mismatch. A larger queue postpones the failure; it does not fix the rate mismatch.

Need	Useful behaviour
Newest state only	One slot; overwrite the previous state.
Every accepted command in order	FIFO; reject or back-pressure when full.
A wake-up to inspect retained state	Notification; store the state separately.
A critical trip despite detail loss	Retained trip state plus a bounded detail path.

A blocking send can slow the producer while space becomes available. That is back-pressure: the consumer's capacity limits the sender. It is useful when waiting is acceptable. It is a poor choice inside a measurement path that must promptly return to other sensor deadlines.

A zero-wait send returns failure if no slot is free. The caller must then do something defined. Telemetry can count dropped records; a command interface can return busy. A safety trip needs a stronger fallback than silently losing the event. The original Safety service uses a retained sticky path, which we will explain only after alarms themselves make sense.

Queue length is therefore an architectural choice. Ask what information may be lost and what delay the sender can tolerate before choosing the API. The RTOS protects the queue mechanism; you still choose its meaning.

TIP

Rate arithmetic here assumes steady rates and no initial backlog. Bursts need a separate worst-case backlog calculation.

COMMON PITFALL

Ignoring xQueueSend's result turns a documented capacity limit into silent data loss.

A notification tells one task to look

Teaching example - C

```c
// Sender, after storing the new state:
xTaskNotifyGive(readerHandle);

// Reader:
uint32_t count = ulTaskNotifyTake(
    pdTRUE, pdMS_TO_TICKS(100));
```

Each task has notification state managed by the kernel. In this usage, give increments a notification value. take waits for that value to become nonzero or for its timeout. It returns the value observed before consuming it. The notification is directed to one task identified by readerHandle.

If three gives arrive before take runs, the value can be 3. With pdTRUE, take returns 3 and clears the value to zero. With pdFALSE, it returns 3 and decrements the value to 2; the next take can return immediately. These are different consumption rules.

For a wake-up that means inspect the latest retained state, clearing all accumulated gives is often appropriate. The current state can already include all changes. For counted independent work, you may need one decrement per unit or a queue containing actual work records. A notification count does not hold three different temperature values.

If a give happens before the task starts waiting, the pending notification remains available. That helps avoid a simple lost-wakeup race. It does not fix a design that sends the wake-up before making the referenced state visible.

In the firmware, Safety wakes on a submitted event or after its 100ms health-check interval. The event detail is stored separately. When Safety wakes, it drains the available work and checks the latest state. The wake-up is a prompt, not the event record.

Reference: FreeRTOS: ulTaskNotifyTake

COMMON PITFALL

ulTaskNotifyTake returns zero on timeout. A nonzero count means notifications were pending, not that a particular queue receive must succeed.

Store the fact before waking its reader

Teaching producer fragment

```
xQueueOverwrite(latestQueue, &sample);
xTaskNotifyGive(readerHandle);
```

The ordering is deliberate. First place the sample in the queue; then send the prompt. A higher-priority Reader may run as soon as the give unblocks it. It should find the state that caused the wake already available.

Teaching consumer fragment

```
ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(100));
Sample local;
if (xQueuePeek(latestQueue, &local, 0) == pdTRUE) {
    inspectSample(&local);
}
```

The consumer still checks peek. At boot, no sample may have been published. A timeout may cause the pass. Another allowed path may have sent a notification for a different reason. Checking the data source keeps those cases explicit.

Why not simply receive from a one-item FIFO? That works well for one consumer that should consume each accepted message. The retained queue plus notification is useful when several services should inspect the newest state, or when the same task must wake for commands and periodic health checks as well as new data.

If two publications happen before Reader runs, it can inspect only the newest retained sample and consume both prompts at once. That is correct only when intermediate states may be skipped. We will keep alarm trip details on their separate path.

This is the reason for a later line such as safetySubmitEvent(&event): it first retains the safety information, then attempts the detail queue, then wakes Safety. We will read that actual ordering in alarm delivery (p. 395).

COMMON PITFALL

Reversing these two producer lines can wake a reader that finds only the previous state.

REMEMBER

A wake-up answers when to look. The queue or protected object answers what happened.

An early wake does not move a deadline

Suppose a sensor starts at 1000ms and may be read after 1750ms. You want to sleep until that due time, but a configuration change may wake you early. If you wake at 1300ms, only 300ms elapsed. The chip still needs another 450ms.

Teaching fragment - small, bounded interval assumed

```
MonoMs due = 1750;
MonoMs now = monoNowMs();
if (now < due) {
    uint32_t remaining = (uint32_t)(due - now);
    ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(remaining));
}
now = monoNowMs();
if (now >= due) {
    // It is now time to inspect completion.
}
```

The notification can end the wait early, so the code reads the clock again. It does not subtract a nominal wait duration and assume that time passed. An absolute deadline is a timestamp; a timeout argument is the maximum time this particular wait should block.

In a real worker loop, handle the event that woke you, calculate the next useful deadline, and wait again. If the interval can exceed uint32_t, bound it before casting rather than truncating a 64-bit difference. The firmware's ordinary schedule intervals are small, but that fact should be part of your reasoning.

Tick rounding can produce a zero wait for a positive remaining interval. Then a loop that repeatedly recalculates the same near deadline can stay Ready until the clock advances. For a minimum physical wait, check absolute time and choose a deliberate rounding or peripheral-timer strategy. Do not interpret the RTOS conversion macro as a deadline guarantee.

COMMON PITFALL

A notification is not a deadline-expired event unless your design explicitly makes it one.

REMEMBER

monoNowMs() is the project clock wrapper. Its definition is linked in the clock lesson (p. 93).

Keep periodic work on a time grid

Imagine a task whose work takes 20ms. If it then delays 1000ms, its starts occur near 0, 1020, 2040 and 3060ms. The work duration accumulates into the period. Variable work produces variable drift.

Teaching fragment - periodic RTOS task

```
TickType_t previous = xTaskGetTickCount();
for (;;) {
    doOneShortPass();
    vTaskDelayUntil(&previous, pdMS_TO_TICKS(1000));
}
```

Delay-until uses a reference release time rather than adding the period after the work finishes. If the work stays within budget and scheduling allows, its intended releases remain on a 1000ms grid. It still cannot force a task to execute at an exact instant when a higher-priority task occupies the core.

The Hydroponics scheduler uses its own monotonic jobs and skips expired releases. Suppose the next BME280 slot was 250ms, the period is 250ms, and the worker gets to it at 820ms. Lateness is 570ms. Two further grid points, 500 and 750, have passed. It runs one current job and advances the next due time to 1000ms.

It does not issue three old conversions in a burst to pretend it measured the past. A skipped observation cannot be recreated by reading the sensor three times now. The schedule records the skipped count so you can see that the timing budget was missed.

We will read scheduleJobMarkRun() in the acquisition chapter (p. 340). Here the important choice is already visible: periodic intent, late execution and missed observations are separate facts.

Reference: FreeRTOS: delay-until API

COMMON PITFALL

An absolute schedule prevents one kind of drift. It does not prove deadlines are met. Record lateness and measure execution time.

Run the two-task learning program

Here is a complete small program you can run after the notification lessons. It uses synthetic water values so you can observe a retained record, a wake-up and a stale-data check before introducing sensor faults. The production source readings begin in the same chapters as their corresponding devices.

REMEMBER

You can observe an RTOS handoff without first debugging a sensor.

Complete learning platformio.ini - reference S3 N16R8 board

```
[env:learning]
platform = espressif32@6.12.0
framework = arduino
board = esp32-s3-devkitc-1
board_build.arduino.memory_type = qio_opi
board_build.flash_size = 16MB
board_upload.flash_size = 16MB
board_build.partitions = default_16MB.csv
monitor_speed = 115200
build_flags = -D BOARD_HAS_PSRAM
```

From the directory extracted from Learning_Program.zip, run pio run -e learning. Then use pio run -e learning -t upload -t monitor with the board's configured console connection. This version prints through the framework's standard UART0 console at 115200baud. Check the correct port if you use a board connection configured differently.

Extract **Learning_Program.zip**, the separate attachment named on the navigation page. Run the commands from its directory containing platformio.ini and src/main.c. Its learning environment is not defined in Reference_Firmware_Source.zip. Read the next three source sections as that one main.c.

The synthetic input cycles through 24.0, 25.0, 26.0 and 27.0. warm is a teaching decision, not a relay command.

Learning program / objects and producer

Learning main.c - lines 1-25

```
#include <stdbool.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <esp_timer.h>
#include <freertos/FreeRTOS.h>
#include <freertos/queue.h>
#include <freertos/task.h>

void setup(void) __asm__("_Z5setupv");
void loop(void) __asm__("_Z4loopv");

typedef struct {
    float valueC;
    uint64_t measuredMs;
    uint32_t sequence;
    bool valid;
} Sample;

static QueueHandle_t latest;
static TaskHandle_t readerHandle;

static uint64_t nowMs(void) {
    return (uint64_t)(esp_timer_get_time() / 1000);
}
```

Learning main.c - lines 27-39

```
static void producer(void *arg) {
    (void)arg;
    uint32_t sequence = 0;
    for (;;) {
        Sample s = { .valueC = 24.0f + (float)(sequence % 4),
                     .measuredMs = nowMs(),
                     .sequence = sequence, .valid = true };
        xQueueOverwrite(latest, &s);
        xTaskNotifyGive(readerHandle);
        ++sequence;
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

COMMON PITFALL

The ordinary 1s delay in this small lab includes work-time drift. The production engine uses its release grid because its cadence requirements are stricter.

Learning program / objects and producer / continued

The includes supply types and API declarations. The two assembler-labelled declarations match the symbols Arduino expects from its C++ entry code. Sample is the message type. latest is a queue handle, while readerHandle identifies the task to notify.

Each producer iteration creates a local Sample, copies it into the retained queue, then wakes Reader. sequence%4 is the remainder after division by four, producing 0, 1, 2, 3 repeatedly. Adding 24.0 gives the synthetic temperatures. The task's local s can be reused on the next iteration because overwrite already copied its bytes.

Read the two __asm__ labels as a connection to the Arduino framework. They tell this GNU compiler which linker names to give setup and loop so the framework can call them. They do not execute assembly instructions here. Keep them unchanged for the lab; the production entry header (p. 713) explains why this particular build needs them.

Learning program / reader and freshness

Learning main.c - lines 41-59

```
static void reader(void *arg) {
    (void)arg;
    uint32_t seen = UINT32_MAX;
    bool wasStale = false;
    for (;;) {
        ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(100));
        Sample s;
        if (xQueuePeek(latest, &s, 0) != pdTRUE) continue;
        bool stale = nowMs() - s.measuredMs > 1500;
        bool warm = s.valid && !stale && s.valueC >= 26.0f;
        if (s.sequence != seen || stale != wasStale) {
            printf("seq=%lu water=%.1f state=%s warm=%d\n",
                   (unsigned long)s.sequence, (double)s.valueC,
                   stale ? "STALE" : "FRESH", (int)warm);
            seen = s.sequence;
            wasStale = stale;
        }
    }
}
```

REMEMBER

This loop still checks freshness when no new sample arrives. That is the purpose of the timeout, not a requirement to process empty data.

Reader wakes either because Producer gave a notification or because its 100ms health-check timeout expired. It peeks rather than receives, so the latest record remains available during a later health check. If no sample exists yet, continue goes directly to the next loop iteration and its wait.

The age calculation uses the current monotonic time minus the sample timestamp. A record becomes stale only after its age is greater than 1500ms. warm requires three facts together: valid input, acceptable age and temperature at least 26.0. The short-circuit AND checks them in that order.

The print condition suppresses identical repeated output. It prints on a new sequence or a freshness-state change. UINT32_MAX is an initial sentinel so the first ordinary sequence, zero, is recognised as new. The printf casts match the chosen format specifiers; floating arguments are promoted to double.

Notice what is private: Reader's s is a copy on Reader's own stack. Producer cannot alter that record by writing its own local. The queue handle is shared, but the RTOS controls access to its stored item. No shared live pointer to a producer local is needed.

Learning program / startup and first trace

Learning main.c - lines 61-74

```
void setup(void) {
    latest = xQueueCreate(1, sizeof(Sample));
    if (latest == NULL) abort();
    BaseType_t ok = xTaskCreatePinnedToCore(
        reader, "Reader", 4096, NULL, 3, &readerHandle, 1);
    if (ok != pdPASS) abort();
    ok = xTaskCreatePinnedToCore(
        producer, "Producer", 4096, NULL, 2, NULL, 1);
    if (ok != pdPASS) abort();
}

void loop(void) {
    vTaskDelay(portMAX_DELAY);
}
```

Create the one-slot queue first. Then create Reader, which may run immediately and block waiting for data. Its handle is established before Producer can notify it. Create Producer last. Failure calls abort rather than pretending the partially started demonstration is healthy.

COMMON PITFALL

Suspending or blocking Producer does not stop Reader. Accidentally blocking Reader would test a different failure.

Learning program / startup and first trace / continued

Publication	Synthetic value	Expected fresh decision
sequence 0	24.0	warm=0
sequence 1	25.0	warm=0
sequence 2	26.0	warm=1
sequence 3	27.0	warm=1
sequence 4	24.0	warm=0

Those are logical expected values, not a measured serial trace. Their observation times vary with tick phase, logging and scheduling. A latest-value queue allows intermediate publications to be skipped if Reader is delayed for long enough.

To expose the stale path, add if (sequence == 6) vTaskDelay(portMAX_DELAY); immediately after ++sequence in Producer. Reader keeps its own 100ms health check. Once the retained result is older than 1500ms and Reader gets CPU time, it prints STALE with warm=0. The numeric temperature can remain 25.0; its age changed, not its stored value.

Remove that deliberate stall for normal runs. Reset restarts the sequence. This one experiment connects elapsed time, a retained queue, notification coalescing and the meaning of a health check without depending on any physical sensor.

Two correct lines can lose an update

Teaching example - C

```c
// Shared object, initially zero:
static uint32_t count = 0;
// Task A and Task B each execute:
++count;
```

Think of ++count as read, add one, write. A reads zero. B reads zero. A computes one. B computes one. Both write one. Two increments occurred, but the stored result is one. This is a race: the result depends on an interleaving that the code did not control.

On two cores the reads can overlap in real time. On one core, preemption between read and write can create the same problem. Pinning tasks to one core does not make a multi-instruction update indivisible.

Step	Task A	Task B
1	Read 0	
2		Read 0
3	Write 1	
4		Write 1

Even if a single aligned word load is indivisible on your target, a compound operation on it may not be. A multi-field record has another problem: a reader can see a new value with an old timestamp if the writer updates the fields separately.

You can avoid shared mutation by sending a record copy to one owner. When shared access is necessary, use a primitive whose contract covers the operation: an atomic operation for suitable small state, a brief critical section for a tiny shared update, or a mutex for task-level exclusive access.

The project uses queue copies for snapshots and short portMUX-protected regions for several cross-core records. We will inspect that distinction before applying it to a driver handover.

COMMON PITFALL

Testing the race once and seeing count=2 proves only that one interleaving happened to work.

volatile does not make a conversation safe

Teaching counterexample - incomplete synchronisation

```
static volatile bool ready;
static Sample shared;

// Writer:
shared.valueC = 24.5f;
ready = true;
```

volatile tells the compiler that accesses to that volatile object have observable significance. It is used for things such as hardware registers and some low-level interrupt-visible state. It does not automatically establish a correct shared-memory protocol for the non-volatile Sample beside it.

If another task waits for ready and then reads shared, you must also reason about ordering, visibility, concurrent writers and the lifetime of every field. Marking ready volatile does not turn shared into one coherent published record. It does not make ++count atomic either.

For application code, a queue copy is often easier to reason about. The RTOS provides the communication operation; the sender finishes writing its local record before sending it. The receiver gets its own copy when receive or peek succeeds.

The original firmware contains volatile fields in specialised ISR and diagnostic paths. Do not copy that keyword into every shared variable and assume the problem is solved. Inspect each path's writer, reader, affinity, width and publication rules. A source comment is a claim to review, not a substitute for that reasoning.

const and volatile answer different questions. const restricts modification through a declaration. volatile affects how accesses are treated by the compiler. Neither one is the same as a mutex, a queue or a complete atomic publication protocol.

COMMON PITFALL

A busy loop over a volatile flag can still starve the task expected to set it.

REMEMBER

Before choosing a keyword, name both actors and the exact data they exchange.

A mutex gives one task temporary ownership

Teaching fragment - mutex previously created

```
if (xSemaphoreTake(busMutex, pdMS_TO_TICKS(20)) == pdTRUE) {
    performBoundedTransaction();
    xSemaphoreGive(busMutex);
} else {
    // No ownership was obtained. Do not use the bus.
}
```

A mutex is an exclusive-access object for tasks. A successful take gives this task permission to enter the protected operation. Another task attempting a blocking take can sleep until the mutex becomes available or its timeout expires. The holder must give it back on every completion and error path.

Why not protect only individual I2C bytes? Suppose the first task writes a register address, a second task changes the device's pointer, then the first reads. The bytes may each be electrically valid while the logical transaction is wrong. Protect the whole operation that must remain together.

FreeRTOS mutexes provide priority inheritance: a low-priority holder can temporarily inherit a waiting higher priority under the mutex rules. That can reduce priority inversion, where urgent work waits behind less urgent work holding a resource. It does not make long lock holds harmless or remove all possible deadlocks.

The Hydroponics acquisition engine normally avoids bus competition by giving one worker all sensor drivers. Its flash gate is a different mutex: a ping holds it across the echo window so Configuration cannot start a flash write at the same time. The gate protects a timing condition, not just a data structure.

Mutexes are task-level objects. Do not take a blocking mutex in an interrupt. Interrupt code must finish promptly and use primitives permitted in that context.

Reference: Espressif: ESP-IDF FreeRTOS SMP, v4.4.2 (S3)

COMMON PITFALL

If Task A holds lock X and waits for Y while Task B holds Y and waits for X, neither can progress. Define a consistent lock order.

See why the lock holder may need a higher priority

Take three tasks on one core: Low at priority 1, Medium at 4 and High at 6. Low already holds a mutex for a resource. High becomes Ready, preempts Low, tries to take that same mutex with a blocking wait and becomes Blocked. High cannot finish until Low releases it.

Without priority inheritance, a Ready Medium could keep winning over Low even though High is waiting for Low's resource. High's urgency would not help Low reach its release line. This dependency is priority inversion.

Step with a FreeRTOS mutex	Effect in this one-mutex example
High blocks waiting for Low's mutex	Low temporarily inherits High's priority.
The scheduler selects the boosted Low	Low can finish its protected operation ahead of Medium.
Low gives its only held mutex	High can become Ready and acquire it; Low returns to its base priority.

The mechanism shortens this particular delay. It cannot make a blocked peripheral finish or repair a circular lock dependency. FreeRTOS uses simplified inheritance; multiple held mutexes have additional rules. Keep the lock order and completion paths understandable instead of relying on inheritance to repair an arbitrary design.

In the controller, the flash gate coordinates an echo interval with Configuration's flash write. The declaration of the lock does not tell you its whole cost. Follow the owner from successful take through normal completion, timeout and cancellation to the matching give.

Reference: FreeRTOS: mutexes and priority inheritance

COMMON PITFALL

A binary semaphore does not provide a mutex's ownership and priority-inheritance contract.

Keep a critical section tiny

Teaching example - C

```
static portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;
static uint32_t completed;

portENTER_CRITICAL(&mux);
++completed;
portEXIT_CRITICAL(&mux);
```

REMEMBER

Critical sections on a dual-core chip need cross-core coordination. Disabling interrupts on just one core is insufficient by itself.

In this ESP-IDF port, portMUX-based critical sections combine interrupt control on the current core with a spinlock that coordinates the other core. A competing core can spin while the holder finishes. This is not the sleeping wait of a blocking mutex take.

Use this for a few simple loads or stores, such as copying a small status record. Keep the protected region short, with a clear entry and exit. Both readers and writers that need the invariant must use the same protection. Locking only the writer does not stop an unprotected reader from observing a partial update.

Do not log, wait on a queue, write NVS, or run a sensor transaction while holding a portMUX. Any of those can take too long or call code that is not valid inside the critical section. Copy the required facts under the lock; perform slow work afterward.

For a large Snapshot, the project uses the queue mechanism instead of hand-writing a huge lock-protected copy protocol. For acquisition statistics, it uses a short critical section. Those choices follow the operation being protected, not a rule that one primitive is always better.

A driver will keep persistent state that one owner advances. Before reading that interface, compare the remaining signalling tools in this chapter. They organise events and conditions; they do not supply another owner for the same driver.

Reference: Espressif: ESP-IDF FreeRTOS SMP, v4.4.2 (S3)

Choose a token or an owner

A binary semaphore holds either zero or one available token. An event producer can give it; a task can take it and optionally block while no token exists. Unlike a mutex, it is not an ownership lock with priority inheritance.

Teaching fragment - include freertos/semphr.h

```
SemaphoreHandle_t done = xSemaphoreCreateBinary();
// New binary semaphore starts empty. Check done != NULL.

// Event producer in task context:
xSemaphoreGive(done);

// Waiting task:
if (xSemaphoreTake(done, pdMS_TO_TICKS(100)) == pdTRUE) {
    // At least one completion token was obtained.
}
```

If the producer gives twice before a take, a binary semaphore cannot retain two tokens. The second give can fail because the one-token capacity is full. This is suitable for a coalesced event, but not for preserving every independent completion.

A counting semaphore can hold up to a configured maximum count. If you have three interchangeable free buffers, start with three tokens. Taking one reserves a buffer opportunity; giving one returns it. The semaphore tracks quantity, so you still need a separate way to identify which specific buffer was reserved.

A mutex instead expresses: this task owns this protected resource until it releases it. Its priority-inheritance behaviour is useful for task-level exclusion. Do not replace a bus mutex with a binary semaphore merely because both APIs have Take and Give in their names.

For waking one known worker, a direct task notification is often simpler and avoids another kernel object. Choose by the required semantics: ownership, event count, actual message contents, or a retained state prompt.

Reference: FreeRTOS: binary semaphores

COMMON PITFALL

A semaphore token does not contain the sensor result. You still need defined storage and publication for the data.

Wait for named conditions together

An event group stores a set of bits. Each bit can represent a condition, such as network ready or configuration loaded. A task can wait for any selected bit or all selected bits. The bits are conditions; repeated sets of the same bit do not count repeated events.

Teaching fragment - group created; include freertos/event_groups.h

```
const EventBits_t NET = 1u << 0;
const EventBits_t CFG = 1u << 1;
EventBits_t wanted = NET | CFG;
EventBits_t bits = xEventGroupWaitBits(
    group, wanted, pdFALSE, pdTRUE, pdMS_TO_TICKS(500));
bool both = (bits & wanted) == wanted;
```

pdFALSE asks not to clear the requested bits on a successful exit. pdTRUE asks to wait for all requested bits. At 100ms, NET may be set while CFG is still clear. The task remains blocked. If CFG is set at 200ms, the all-bits condition is met and the task becomes Ready.

If 500ms expires first, the returned bit value may contain NET without CFG. Check the mask; do not treat every return as success. If you choose clear-on-exit, understand which bits are cleared when the condition is met. Clearing changes what later waiters observe.

This is useful for coarse readiness conditions. It is unsuitable for ten distinct sensor results, command payloads, or a pulse-width history. Those need records or counters with the correct capacity. A group bit also does not prove that a condition remains true forever after a task wakes.

The reference acquisition design mainly uses notifications and copied records. Event groups are shown here so you can recognise an alternative RTOS tool without forcing it into a path that already has a clear contract.

Reference: FreeRTOS: xEventGroupWaitBits

COMMON PITFALL

Event bits coalesce. Setting READY five times leaves READY set once, not a count of five.

Let a timer prompt work, then return

Teaching fragment - include freertos/timers.h

```
static void onTimer(TimerHandle_t timer) {
    (void)timer;
    xTaskNotifyGive(workerHandle);
}

TimerHandle_t timer = xTimerCreate(
    "Refresh", pdMS_TO_TICKS(1000), pdTRUE,
    NULL, onTimer);
// Check timer != NULL, then check xTimerStart(timer, 0).
```

pdTRUE selects an auto-reload timer: after expiry it keeps running periodically until stopped or changed. A software-timer callback runs in the timer service task. A GPIO interrupt handler runs directly in interrupt context; this callback runs as task code in the shared timer service. The interrupt chapter will develop that distinction. Creating a timer does not create a private task for its callback.

That shared callback context must stay responsive. A blocking delay in one callback holds up other timer callbacks and command processing. Use a short action such as a notification, then let the owning worker perform longer work.

xTimerStart submits a command through the timer mechanism. A successful submission is not evidence that the callback has already run. Tick resolution and service-task scheduling affect when it executes. A 1s refresh prompt is an appropriate use; precise timing of ultrasonic edges is not.

The project instead stores many driver poll times in its acquisition scheduler. That keeps sensor scheduling decisions in one owner and makes overlap guards visible. You could use software timers to prompt that owner, but callbacks should not independently operate the same driver objects.

You now have several ways to arrange a future action. Use a task delay for the task's own waiting, a deadline scheduler for several stepped jobs, a software timer for a short deferred prompt, and a suitable peripheral or timestamped ISR for fine event timing. They solve different time scales.

Reference: FreeRTOS: timer API header

COMMON PITFALL

Do not use a nonzero queue or semaphore wait from a software-timer callback. It blocks the shared timer service.

Remember enough to return later

Teaching example - C

```
typedef struct {
    bool busy;
    uint64_t dueMs;
    float resultC;
} FakeDriver;

static void begin(FakeDriver *d, uint64_t nowMs) {
    d->busy = true;
    d->dueMs = nowMs + 750;
}
```

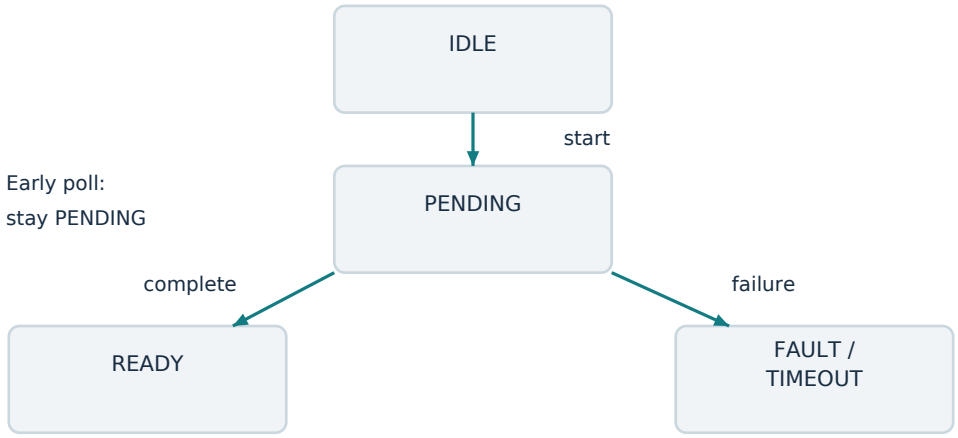
The sensor takes time, so begin stores the information a future call needs. If nowMs is 1000, the object remembers busy=true and dueMs=1750. The function then returns. The caller can service other sensors or block the task instead of sitting in a wait loop.

At 1300ms, the driver is still busy and the due time has not arrived. At 1750ms or later, a poll function can collect the outcome. The driver object must live across those calls. That is why the acquisition module keeps its driver objects in static storage rather than as locals that disappear after construction.

REMEMBER

Long elapsed conversion time does not require long CPU execution time.

Remember enough to return later / continued



One operation moves from idle through pending to a completed or failed outcome.

A state machine is simply stored state plus rules for changing it when an event or time condition occurs. busy is enough for this toy. A more complex driver may store an enum for idle, measuring and reading a baseline. It still changes state one short step at a time.

begin does not create a task. The caller chooses when to call it and when to poll again. That distinction lets one acquisition task advance several independent physical operations.

Return both status and the next useful time

Teaching simplification - caller promises begin happened first

```
typedef struct {
    bool ready;
    uint64_t nextMs;
} FakePoll;

static FakePoll poll(FakeDriver *d, uint64_t nowMs) {
    if (d->busy && nowMs < d->dueMs) {
        return (FakePoll){ false, d->dueMs };
    }
    d->busy = false;
    d->resultC = 24.5f;
    return (FakePoll){ true, nowMs };
}
```

The return expressions are compound literals: (FakePoll){…} creates a record value of the named type. The first return says: no result yet; do not bother again before dueMs. The second stores the invented result and says it is available.

Trace begin at 1000, poll at 1300, and poll at 1750. The first poll returns false with 1750. The second returns true with 1750 and stores 24.5. The caller did not need to poll every microsecond between those times.

The toy omits several checks to keep this trace small. A real driver must reject poll before start, handle a failed bus read, bound every call, and support cancellation. The original PollResult uses DRIVER_PENDING, DRIVER_READY and failure statuses rather than a boolean.

Cancellation matters when the rail disappears or a frame deadline passes. It must stop treating a later hardware event as part of the abandoned operation. In the ultrasonic chapter, disarming capture before the next ping prevents an old falling edge being credited to a new measurement.

COMMON PITFALL

Returning PENDING without a sensible future poll time can make the acquisition worker spin. Returning a far-future time can delay a result unnecessarily.

Let one caller use several drivers

Teaching fragment - driver is a persistent FakeDriver object

```
typedef struct {
    void (*start)(FakeDriver *d, uint64_t nowMs);
    FakePoll (*poll)(FakeDriver *d, uint64_t nowMs);
} FakeOps;

static const FakeOps ops = {
    .start = begin,
    .poll = poll
};

ops.start(&driver, 1000);
```

void (*start)(...) declares a field named start that stores a pointer to a function. The pointed-to function returns void and accepts the listed parameters. The parentheses around *start are necessary: without them, you would describe a different declaration.

The initializer stores begin's address in the start field and poll's address in the poll field. It does not run either function. ops.start(...) later selects the stored function and calls it. This is indirect dispatch: the same calling code can choose a different implementation by using a different table.

Why do this? Acquisition needs to start, poll and collect results from seven sensors. If every sensor exposes the same shape of interface, the acquisition loop can work through an array of driver pointers. It does not need a separate giant switch for every action on every sensor.

There is a cost: the table must point to functions with compatible types, and the correct state object must accompany each call. A bad cast does not make an incompatible function safe. The project uses a shared SensorDriver base and a table with a single agreed set of signatures.

COMMON PITFALL

A function pointer is not a task handle. Calling through it runs a normal function in the calling task.

REMEMBER

Function name without call parentheses selects the function. Parentheses with arguments request execution.

Read the original operations table

Original firmware - src/drivers/sensor_driver.h, lines 51-62

```
  DriverStatus (*init)(SensorDriver* self);

  // Begin one measurement (e.g. trigger a conversion). Must
>   not block on it.
  DriverStatus (*start)(SensorDriver* self, MonoMs now);

  // Advance the measurement. Never blocks beyond the
>   documented budget.
  PollResult (*poll)(SensorDriver* self, MonoMs now);

  // Copy up to maxOut readings of the last completed
>   measurement.
  // Returns the number written. Health/quality are filled by
>   the driver;
  // powerEpoch/configRevision are stamped by the acquisition
>   service.
  uint8_t (*readResult)(SensorDriver* self, Reading* out,
>   uint8_t maxOut);
```

Original firmware - src/drivers/sensor_driver.h, lines 72-87

```
struct SensorDriver {
  const SensorDriverOps* ops;
};

static inline SensorId sensorDriverId(const SensorDriver* d) {
>   return d->ops->id(d); }
static inline const char* sensorDriverName(const SensorDriver*
>   d) { return d->ops->name(d); }
static inline uint8_t sensorDriverMetricCount(const
>   SensorDriver* d) {
  return d->ops->metricCount(d);
}
static inline DriverStatus sensorDriverInit(SensorDriver* d) {
>   return d->ops->init(d); }
static inline DriverStatus sensorDriverStart(SensorDriver* d,
>   MonoMs now) {
  return d->ops->start(d, now);
}
static inline PollResult sensorDriverPoll(SensorDriver* d,
>   MonoMs now) {
  return d->ops->poll(d, now);
}
```

COMMON PITFALL

A constructor fills the object and its table pointer. It does not necessarily configure the physical sensor; init does that.

Read the original operations table / continued

SensorDriver contains an ops pointer. That pointer selects a constant SensorDriverOps table. In sensorDriverStart(), d->ops selects the table, ->start selects the function pointer in it, and (d, now) calls that function. now is the timestamp supplied by the acquisition owner.

Each concrete driver puts SensorDriver base as its first structure member. For example, LightDriver begins with SensorDriver base, then its configuration and private state. Its constructor sets base.ops to the light table. Passing &light.base gives acquisition a generic view; the light implementation converts that initial-member pointer back to LightDriver * to reach its own fields.

This conversion relies on the actual object layout and the correct table/object pairing. It is not permission to cast any arbitrary pointer to any driver type. Keep base first and construct the object before calling through its table.

The interface asks init to configure the device, start to begin an operation, poll to advance it, readResult to copy completed records, cancel to abandon work, and preparePowerOff to make interface pins suitable for a lost rail. These are ordinary calls made by the owning task.

We can now read sensor implementations without learning a new calling convention each time. Before opening the first one, we will define the result record they all return.

The source table also contains identity and metric-count functions. They let acquisition label a driver and allocate enough result space.

Distinguish progress from a valid measurement

Every concrete driver must answer the same lifecycle questions. This is the shared interface that lets acquisition manage different devices through an array of SensorDriver pointers.

Read one operation at a time. The sensor chapters fill in the concrete bodies behind this interface.

COMMON PITFALL

Do not use if(status) as a universal success check; DRIVER_OK is zero.

src/drivers/sensor_driver.h / lines 1-23

```
// Common sensor driver contract (Firmware Architecture §5, §14).
// FROZEN CONTRACT - all seven drivers implement exactly this interface.
//
// Rules for every driver:
//  - Know hardware, report status. No MQTT/BLE/WiFi includes, no task
//    creation, no power-mode decisions, no relay changes.
//  - No hidden long delays. start() begins an operation; poll() returns
//    Pending + the next time it is worth polling again. The single
//    acquisition worker interleaves sensors using those deadlines.
//  - Worst-case blocking of every call must be documented and bounded.
//  - Configuration (bus, pins, address) is passed to the constructor
//    (<name>DriverConstruct()).
//
// C object model: a driver is a struct whose FIRST member is a SensorDriver.
// SensorDriver holds a pointer to the driver's const operations table (the
// "vtable"). Callers use the sensorDriver*() functions below on a
// SensorDriver*, which dispatch through that table; the driver casts the
// SensorDriver* back to its own type.
#pragma once
#include <stdbool.h>
#include <stdint.h>
#include "domain/reading.h"

```

Distinguish progress from a valid measurement / continued

DriverStatus tells the caller what happened to the operation. OK means an operation such as start was accepted. PENDING means there is more work later. READY means an outcome can be collected. The outcome itself still has a Health value, so READY can accompany a failed physical measurement.

src/drivers/sensor_driver.h / lines 24-37

```
typedef uint8_t DriverStatus;
enum {
  DRIVER_OK = 0,          // call accepted (init/start)
  DRIVER_PENDING,         // operation in progress, poll again at nextPollMs
  DRIVER_READY,           // result available via readResult()
  DRIVER_NOT_PRESENT,     // device missing / wrong identity
  DRIVER_FAULT,           // bus, CRC or range error
  DRIVER_TIMEOUT,         // exceeded its budget
  DRIVER_UNPOWERED,       // supply rail not available
  DRIVER_NOT_IMPLEMENTED  // stub (Stage 1 skeleton)
};

typedef struct PollResult {
  DriverStatus status;
```

PollResult pairs that status with nextPollMs. If the driver is pending until 1750ms, returning that timestamp lets the caller do useful work or block until it becomes worth checking again.

Read a function-pointer member from its name

src/drivers/sensor_driver.h / lines 38-63

```
38   MonoMs nextPollMs;  // meaningful when status ==
 >   DRIVER_PENDING
39 } PollResult;
40
41 typedef struct SensorDriver SensorDriver;
42
43 typedef struct SensorDriverOps {
44   SensorId (*id)(const SensorDriver* self);
45   const char* (*name)(const SensorDriver* self);
46
47   // Number of Reading records this driver produces per
 >   result.
48   uint8_t (*metricCount)(const SensorDriver* self);
49
50   // Probe and configure the device. May be called again
 >   after a fault.
51   DriverStatus (*init)(SensorDriver* self);
52
53   // Begin one measurement (e.g. trigger a conversion). Must
 >    not block on it.
54   DriverStatus (*start)(SensorDriver* self, MonoMs now);
55
56   // Advance the measurement. Never blocks beyond the
 >   documented budget.
57   PollResult (*poll)(SensorDriver* self, MonoMs now);
58
59   // Copy up to maxOut readings of the last completed
 >   measurement.
60   // Returns the number written. Health/quality are filled
 >   by the driver;
61   // powerEpoch/configRevision are stamped by the
 >   acquisition service.
62   uint8_t (*readResult)(SensorDriver* self, Reading* out,
 >   uint8_t maxOut);
63
```

In uint8_t (*readResult)(SensorDriver *self, Reading *out, uint8_t maxOut), readResult stores a function address. Calling it passes a generic driver pointer, destination records and a capacity, then receives the number of records written.

COMMON PITFALL

A function table does not enforce bounded execution. Each concrete implementation must keep its promise.

Read a function-pointer member from its name / continued

src/drivers/sensor_driver.h / lines 64-70

```
64   // Abandon any pending operation (deadline miss, quiesce request).
65   void (*cancel)(SensorDriver* self);
66
67   // Put the interface pins into a safe state before the rail is removed
68   // (§12: no back-feeding an unpowered module).
69   void (*preparePowerOff)(SensorDriver* self);
70 } SensorDriverOps;
```

init configures the hardware; start initiates one operation; poll advances it; readResult copies outcomes. cancel abandons a pending operation. preparePowerOff also makes interface pins suitable for the lost rail. The driver owns protocol knowledge, while the acquisition owner chooses when to call these operations.

Follow the indirection without inventing a task

src/drivers/sensor_driver.h / lines 71-94

```
71
72 struct SensorDriver {
73   const SensorDriverOps* ops;
74 };
75
76 static inline SensorId sensorDriverId(const SensorDriver* d)
 >    { return d->ops->id(d); }
77 static inline const char* sensorDriverName(const
 >   SensorDriver* d) { return d->ops->name(d); }
78 static inline uint8_t sensorDriverMetricCount(const
 >   SensorDriver* d) {
79   return d->ops->metricCount(d);
80 }
81 static inline DriverStatus sensorDriverInit(SensorDriver* d)
 >    { return d->ops->init(d); }
82 static inline DriverStatus sensorDriverStart(SensorDriver*
 >   d, MonoMs now) {
83   return d->ops->start(d, now);
84 }
85 static inline PollResult sensorDriverPoll(SensorDriver* d,
 >   MonoMs now) {
86   return d->ops->poll(d, now);
87 }
88 static inline uint8_t sensorDriverReadResult(SensorDriver*
 >   d, Reading* out, uint8_t maxOut) {
89   return d->ops->readResult(d, out, maxOut);
90 }
91 static inline void sensorDriverCancel(SensorDriver* d) {
 >   d->ops->cancel(d); }
92 static inline void sensorDriverPreparePowerOff(SensorDriver*
 >    d) { d->ops->preparePowerOff(d); }
93
94 const char* driverStatusName(DriverStatus s);
```

SensorDriver contains one pointer to a constant operations table. sensorDriverStart reads that pointer, selects the start function and calls it with the original object address and now. The called body executes in the current acquisition task.

The thin inline wrappers make calling code readable without duplicating dispatch syntax everywhere. They assume a constructed driver with a valid table; they do not check every pointer at runtime. A concrete constructor must establish base.ops before any wrapper is used.

REMEMBER

The first-member base conversion is explained in the lesson immediately before this source reading.

Convert a status into a diagnostic string

This implementation supplies diagnostic names for the operation statuses. It is intentionally small because the lifecycle behaviour belongs to the concrete driver.

A readable log string is an interpretation of a stored enum; it is not a new status transition.

src/drivers/sensor_driver.c / lines 1-15

```
#include "drivers/sensor_driver.h"

const char* driverStatusName(DriverStatus s) {
  switch (s) {
    case DRIVER_OK: return "OK";
    case DRIVER_PENDING: return "PENDING";
    case DRIVER_READY: return "READY";
    case DRIVER_NOT_PRESENT: return "NOT_PRESENT";
    case DRIVER_FAULT: return "FAULT";
    case DRIVER_TIMEOUT: return "TIMEOUT";
    case DRIVER_UNPOWERED: return "UNPOWERED";
    case DRIVER_NOT_IMPLEMENTED: return "NOT_IMPLEMENTED";
  }
  return "?";
}
```

Each case returns a pointer to a string literal. Those literals have static storage duration, so the pointer remains valid after the function returns. The caller must treat the text as read-only.

The fallback handles an unexpected numeric value with a visible unknown marker. That makes diagnostics more robust, but it does not repair the invalid state that produced the value. Look at the status producer if this fallback appears in a log.

Read the record in useful groups

Original firmware - src/domain/reading.h, lines 32-48

```
typedef struct Reading {
  SensorId sensor;
  MetricId metric;
  Unit unit;
  Health health;
  uint16_t qualityFlags;      // QualityFlag bits
  bool hasRaw;
  float value;                // in `unit`; meaningless unless
>   health == Ok/Uncalibrated
  int32_t raw;                // ADC counts / mV / raw register
>    value if hasRaw
  MonoMs intervalStartMs;     // acquisition interval
  MonoMs intervalEndMs;
  MonoMs lastGoodMs;          // last usable result (OK or
>   UNCALIBRATED); 0 = never
  uint32_t sequence;          // per-metric sample counter
  uint32_t powerEpoch;        // rail epoch when measured (§12)
  uint16_t configRevision;
  uint16_t calibrationRevision;
} Reading;
```

Start with sensor, metric and unit. A BME280 is one sensor, but it publishes air temperature, humidity and pressure as three metrics. The unit tells you whether value is degrees Celsius, pascals, millivolts or something else. The same float type cannot tell those apart.

REMEMBER

A record copy preserves related fields together. It does not make the underlying physical measurements simultaneous.

Read the record in useful groups / continued

Next inspect health and qualityFlags. Health is the main verdict about this observation. Quality flags add qualifications such as uncompensated or saturated. hasRaw tells you whether raw is meaningful; raw's interpretation depends on the metric. For light it holds filtered pin millivolts, while for distance it holds pulse microseconds.

The interval fields bracket the operation. sequence distinguishes a newly published observation from a repeated peek at the same one. Acquisition increments the published sequence for failed observations as well, so a new fault does not disappear as an apparent resend of the previous good sample.

The revision fields identify the configuration and calibration used. powerEpoch identifies the rail history associated with the result. You do not need their full mechanisms yet; each is a tag that will let a consumer reject a result created under outdated conditions. We will give epoch its worked example in the power chapter (p. 463).

Reading is 56 bytes in this source layout. reading.c has a compile-time assertion for that size.

Stored OK can become too old

Original firmware - src/domain/reading.c, lines 21-33

```
bool readingIsActionable(const Reading* r) { return r->health
>   == HEALTH_OK; }

Health readingHealthAt(const Reading* r, MonoMs now, uint32_t
>   staleLimitMs) {
  if (r->health != HEALTH_OK) return r->health;
  if (now > r->intervalEndMs && (now - r->intervalEndMs) >
>   staleLimitMs) {
    return HEALTH_STALE;
  }
  return HEALTH_OK;
}

bool readingIsActionableAt(const Reading* r, MonoMs now,
>   uint32_t staleLimitMs) {
  return readingHealthAt(r, now, staleLimitMs) == HEALTH_OK;
}
```

COMMON PITFALL

lastGoodMs is historical information. A current FAULT does not become actionable because a good sample existed earlier.

readingIsActionable() only checks stored health. readingHealthAt() also compares the observation's end time with the caller's current time. readingIsActionableAt() combines that age check with the OK requirement. Use the At form for decisions whose input can age while no new sample arrives.

Take an OK Reading whose intervalEndMs is 1000 and whose stale limit is 3000ms. At now=4000, its age is exactly 3000 and the source still returns OK. At 4001, the age is 3001 and it returns STALE. The comparison is greater-than, not greater-than-or-equal.

The function calculates an effective health; it does not write STALE into the original record. A different caller can evaluate the same retained record at another time. If the stored health is already FAULT, the function returns FAULT before doing the age calculation.

UNCALIBRATED means acquisition produced a useful raw or provisional value, but the numeric physical conversion is not accepted for ordinary rules. DEPENDENCY_INVALID means another needed input, such as water temperature for TDS, was unsuitable. Neither should silently be relabelled OK because the float looks reasonable.

There is a specific exception for lamp control: its commissioned control quantity is pin millivolts, so it can use an uncalibrated light reading in those raw units. That is an explicit policy, not a rule that every uncalibrated sensor may control an output.

Measure ages with the monotonic clock

Original firmware - src/hal/clock.c, lines 1-5

```
#include "hal/clock.h"

#include <esp_timer.h>

MonoMs monoNowMs(void) { return (MonoMs)(esp_timer_get_time() /
>    1000); }
```

esp_timer_get_time() provides microseconds since timer startup. Dividing by 1000 gives whole milliseconds. monoNowMs() names that conversion in one place so application code has a common time source. MonoMs is the 64-bit unsigned alias introduced earlier.

Use this clock for deadlines, age and elapsed intervals. Wall-clock time, such as a calendar time received from a network, can be corrected forward or backward. If a clock correction moves 20s backward, a freshness calculation based on that calendar time could mistakenly keep an old reading fresh.

A monotonic timestamp still needs context. It starts again after reset; do not subtract a pre-reboot timestamp from a post-reboot timestamp as if they belonged to one continuous run. Logs and external consumers need a boot or session identity if they compare records across restarts.

Why use two resolutions?

Sensor schedules and stale limits operate in milliseconds. An echo pulse is measured in microseconds. Converting the edge timestamps to milliseconds before subtracting would discard useful information: a 1000us pulse could collapse into only one tick or less of apparent width. Keep the fine resolution until the width is calculated.

The firmware's pulse HAL deliberately stores 32-bit microsecond timestamps and subtracts them unsigned. That shorter counter wraps; the interval operation can still work across one wrap for a suitably bounded pulse. We will calculate an example in the timeout lesson (p. 313).

Reference: Espressif: high-resolution timer, v4.4.7 (S3)

REMEMBER

The wrapper's resolution is 1ms. That does not guarantee a task will wake or finish within 1ms.

Name the observation verdict and its qualifications

You have seen why a consumer needs more than a float. This header gives the production record its complete shape. Group the fields by the question each one answers.

All drivers, acquisition, Safety and encoders share this contract.

COMMON PITFALL

A quality bit does not cancel a health fault. Consumers need a declared acceptance policy for both fields.

Name the observation verdict and its qualifications / continued

src/domain/reading.h / lines 1-28

```
1  // Reading contract and latest-snapshot record (Firmware Architecture §8).
2  // FROZEN CONTRACT - every driver, service and transport depends on it.
3  // Domain layer: pure C, no Arduino/FreeRTOS includes.
4  #pragma once
5  #include <stdbool.h>
6  #include <stdint.h>
7  #include "domain/types.h"
8
9  typedef uint8_t Health;
10 enum {
11   HEALTH_STARTING = 0,        // no valid result yet
12   HEALTH_WARMING,             // heater/filter/first conversion settling
13   HEALTH_OK,                  // fresh, valid, declared units/calibration
14   HEALTH_STALE,               // last good value exceeded its age limit
15   HEALTH_FAULT,               // transport, CRC, range or conversion error
16   HEALTH_TIMEOUT,             // operation did not complete in its budget
17   HEALTH_UNPOWERED,           // rail off or power epoch changed during measurement
18   HEALTH_UNCALIBRATED,        // acquisition works, physical units not trustworthy
19   HEALTH_DEPENDENCY_INVALID   // e.g. TDS without a valid water temperature
20 };
21
22 // Additional quality bits (combine with Health).
23 enum QualityFlag {
24   kQualityNone = 0,
25   kQualitySaturated = 1u << 0,     // ADC at/near rail
26   kQualityUncompensated = 1u << 1, // compensation input missing
27   kQualityFilterPartial = 1u << 2, // filter window not full
28   kQualityLate = 1u << 3,          // completed after its deadline
```

Health is one main verdict: starting, warming, OK, stale, fault, timeout, unpowered, uncalibrated or dependency-invalid. QualityFlag adds independent details. Several quality bits can be set simultaneously because each occupies a different bit position.

src/domain/reading.h / lines 29-29

```
29 };
```

The distinction prevents one enum from needing a separate value for every combination. A distance can be otherwise valid and still use nominal sound speed, so its health and uncompensated flag answer different questions.

Keep interpretation and history with the value

src/domain/reading.h / lines 30-48

```
30
31 // Fixed-size record, one per metric. Plain data: safe to
 >   copy through queues.
32 typedef struct Reading {
33   SensorId sensor;
34   MetricId metric;
35   Unit unit;
36   Health health;
37   uint16_t qualityFlags;      // QualityFlag bits
38   bool hasRaw;
39   float value;                // in `unit`; meaningless
 >   unless health == Ok/Uncalibrated
40   int32_t raw;                // ADC counts / mV / raw
 >   register value if hasRaw
41   MonoMs intervalStartMs;     // acquisition interval
42   MonoMs intervalEndMs;
43   MonoMs lastGoodMs;          // last usable result (OK or
 >   UNCALIBRATED); 0 = never
44   uint32_t sequence;          // per-metric sample counter
45   uint32_t powerEpoch;        // rail epoch when measured
 >   (§12)
46   uint16_t configRevision;
47   uint16_t calibrationRevision;
48 } Reading;
```

Reading stores identity, unit and health before the numeric fields. hasRaw says whether raw may be interpreted. intervalStartMs and intervalEndMs bracket the observation; lastGoodMs records the last usable outcome; sequence identifies publication changes.

powerEpoch, configRevision and calibrationRevision tag the conditions used for the result. They are not alternate clocks. A changed tag can make two equal numeric values represent different histories. We will use those tags when a configuration or rail changes.

COMMON PITFALL

The old float can remain in memory after a new fault. Health determines whether that field currently represents a usable observation.

A Snapshot is a retained collection

src/domain/reading.h / lines 49-68

```
49
50 // Latest-snapshot record published by the acquisition lease
 >    holder (§8, §9).
51 // A coherent copy, NOT a simultaneous measurement of all
 >   sensors.
52 typedef struct Snapshot {
53   uint32_t frameSequence;
54   MonoMs publishedMs;
55   uint32_t powerEpoch;
56   Reading readings[METRIC_COUNT];  // indexed by MetricId
57 } Snapshot;
58
59 // Initialise a record to STARTING with identity fields set.
60 void readingReset(Reading* r, SensorId sensor, MetricId
 >   metric, Unit unit);
61
62 // True if the stored health is OK. Does NOT check age: a
 >   consumer that acts on
63 // the value (Safety, control rules) must use
 >   readingIsActionableAt() instead.
64 bool readingIsActionable(const Reading* r);
65
66 // True only for fresh, valid, calibrated data (eligible for
 >    numeric rules):
67 // stored health OK and not older than staleLimitMs at `now`
 >    (§10).
68 bool readingIsActionableAt(const Reading* r, MonoMs now,
 >   uint32_t staleLimitMs);
```

COMMON PITFALL

Do not refresh every metric merely by updating Snapshot.publishedMs.

Snapshot groups the latest record for each metric in a fixed indexed array. publishedMs describes publication of that copy, not simultaneous conversion of all ten metrics. An air sample may have completed much later than the retained water sample beside it.

The helper declarations expose reset and age-aware interpretation. readingIsActionable checks stored health only; readingIsActionableAt also considers the caller's now. A consumer that can act after an arbitrary delay needs the latter question.

Expose names and reset operations

src/domain/reading.h / lines 69-78

```
69
70 // Returns Stale if an Ok reading is older than staleLimitMs at `now`,
71 // otherwise the reading's current health.
72 Health readingHealthAt(const Reading* r, MonoMs now, uint32_t staleLimitMs);
73
74 const char* healthName(Health h);
75 const char* metricName(MetricId m);
76 const char* unitName(Unit u);
77
78 void snapshotReset(Snapshot* s);
```

The name helpers turn health, metric and unit identifiers into display strings. snapshotReset establishes the full collection's initial identity and starting states. These functions keep repeated initialisation and naming logic in one domain implementation.

The header declares what callers may request. It does not create a global Snapshot instance or start acquisition. We will now read the implementation to see exactly what each reset and age check changes.

Start from a known record

A helper this small is worth tracing completely. It establishes the boundary between a stored record and the consumer's current interpretation.

Keep a sample ending at 1000ms in mind while reading the age checks.

COMMON PITFALL

A compile-time size assertion checks layout, not physical correctness or sufficient task stack space.

src/domain/reading.c / lines 1-19

```
#include "domain/reading.h"

#include <string.h>

// The records must keep the layout they had in C++ (enum
  class : uint8_t):
// they are copied through queues and sized into RAM
  budgets.
_Static_assert(sizeof(Reading) == 56, "Reading layout
  changed");
// 528 -> 584 bytes on 22 Oct 2026: MQ-7 CO signal replaced
  by SGP30 TVOC + CO2eq.
_Static_assert(sizeof(Snapshot) == 584, "Snapshot layout
  changed");

void readingReset(Reading* r, SensorId sensor, MetricId
  metric, Unit unit) {
  memset(r, 0, sizeof(*r));
  r->sensor = sensor;
  r->metric = metric;
  r->unit = unit;
  r->health = HEALTH_STARTING;
  r->qualityFlags = kQualityNone;
  r->hasRaw = false;
}
```

The compile-time size checks fail the build if Reading or Snapshot no longer has the expected layout. memset zeroes the entire record, including padding, then explicit assignments establish identity and HEALTH_STARTING.

Zeroing alone would not communicate why a record is invalid. The explicit starting health says no accepted observation exists yet. The record can be copied safely, but it is not yet a measurement.

Calculate effective health at the time of use

src/domain/reading.c / lines 20-33

```
20
21 bool readingIsActionable(const Reading* r) { return r->health == HEALTH_OK; }
22
23 Health readingHealthAt(const Reading* r, MonoMs now, uint32_t staleLimitMs) {
24   if (r->health != HEALTH_OK) return r->health;
25   if (now > r->intervalEndMs && (now - r->intervalEndMs) > staleLimitMs) {
26     return HEALTH_STALE;
27   }
28   return HEALTH_OK;
29 }
30
31 bool readingIsActionableAt(const Reading* r, MonoMs now, uint32_t staleLimitMs) {
32   return readingHealthAt(r, now, staleLimitMs) == HEALTH_OK;
33 }
```

The stored-health helper returns true only for OK. readingHealthAt returns existing non-OK health immediately, then checks age only for OK. At exactly the stale limit the code retains OK; one millisecond beyond it becomes STALE.

The function does not edit the record through its const pointer. Different consumers can ask the same retained record about different now values. This is why a timed health check can discover staleness without waiting for another publication.

Give diagnostics a stable vocabulary

src/domain/reading.c / lines 34-61

```
34
35 const char* healthName(Health h) {
36   switch (h) {
37     case HEALTH_STARTING: return "STARTING";
38     case HEALTH_WARMING: return "WARMING";
39     case HEALTH_OK: return "OK";
40     case HEALTH_STALE: return "STALE";
41     case HEALTH_FAULT: return "FAULT";
42     case HEALTH_TIMEOUT: return "TIMEOUT";
43     case HEALTH_UNPOWERED: return "UNPOWERED";
44     case HEALTH_UNCALIBRATED: return "UNCALIBRATED";
45     case HEALTH_DEPENDENCY_INVALID: return "DEPENDENCY_INVALID";
46   }
47   return "?";
48 }
49
50 const char* metricName(MetricId m) {
51   switch (m) {
52     case METRIC_AIR_TEMPERATURE: return "air_temperature";
53     case METRIC_AIR_HUMIDITY: return "air_humidity";
54     case METRIC_AIR_PRESSURE: return "air_pressure";
55     case METRIC_WATER_TEMPERATURE: return "water_temperature";
56     case METRIC_LIGHT_LEVEL: return "light_level";
57     case METRIC_WATER_DISTANCE: return "water_distance";
58     case METRIC_PH: return "ph";
59     case METRIC_TDS: return "tds";
60     case METRIC_TVOC: return "tvoc";
61     case METRIC_CO2EQ: return "co2eq";
```

healthName and metricName use switches to return meaningful string literals. Read their mappings alongside types.h: the names are not inferred from the numeric values. An enum constant and a protocol spelling are separate definitions that must stay consistent.

src/domain/reading.c / lines 62-65

```
62     case METRIC_COUNT: break;
63   }
64   return "?";
65 }
```

The default path exposes an unexpected identifier as unknown. For a normal metric, the returned literal can be used immediately in a bounded log or encoder. It should not be freed or modified.

Label the numerical scale

src/domain/reading.c / lines 66-82

```
66
67 const char* unitName(Unit u) {
68   switch (u) {
69     case UNIT_NONE: return "";
70     case UNIT_DEG_C: return "degC";
71     case UNIT_PERCENT_RH: return "%RH";
72     case UNIT_PASCAL: return "Pa";
73     case UNIT_MILLIVOLT: return "mV";
74     case UNIT_MILLIMETRE: return "mm";
75     case UNIT_PH: return "pH";
76     case UNIT_PPM: return "ppm";
77     case UNIT_ADC_RAW: return "raw";
78     case UNIT_PPB: return "ppb";
79     case UNIT_LUX: return "lux";
80   }
81   return "?";
82 }
```

unitName maps UNIT_PASCAL to Pa and UNIT_MILLIVOLT to mV, for example. It does not convert a number. A value of 101325 remains unchanged when you choose its label; any conversion to another scale must be performed elsewhere.

Keeping naming separate from conversion avoids a display helper silently changing the control quantity. When reviewing telemetry, check the stored unit field as well as the name returned here.

Recover the source device from a metric

src/domain/reading.c / lines 83-99

```
83
84 static SensorId metricSensor(MetricId m) {
85   switch (m) {
86     case METRIC_AIR_TEMPERATURE:
87     case METRIC_AIR_HUMIDITY:
88     case METRIC_AIR_PRESSURE: return SENSOR_BME280;
89     case METRIC_WATER_TEMPERATURE: return SENSOR_DS18B20;
90     case METRIC_LIGHT_LEVEL: return SENSOR_LIGHT;
91     case METRIC_WATER_DISTANCE: return SENSOR_HCSR04;
92     case METRIC_PH: return SENSOR_PH;
93     case METRIC_TDS: return SENSOR_TDS;
94     case METRIC_TVOC:
95     case METRIC_CO2EQ: return SENSOR_SGP30;
96     case METRIC_COUNT: break;
97   }
98   return SENSOR_COUNT;
99 }
```

Several case labels intentionally share one return. Temperature, humidity and pressure map to BME280; TVOC and CO2eq map to SGP30. This is a many-to-one relationship written directly as a switch.

The sentinel return SENSOR_COUNT denotes no valid mapping. Callers must not use that sentinel as an array index without a range check. These mappings become useful when configuration disables a whole sensor and every metric from it must change state.

Reset all metric slots with their identities

src/domain/reading.c / lines 100-125

```
100
101 // The unit each driver publishes when calibrated. pH and light may publish
102 // mV while uncalibrated; the Reading's own unit field says which.
103 static Unit metricUnit(MetricId m) {
104   switch (m) {
105     case METRIC_AIR_TEMPERATURE: return UNIT_DEG_C;
106     case METRIC_AIR_HUMIDITY: return UNIT_PERCENT_RH;
107     case METRIC_AIR_PRESSURE: return UNIT_PASCAL;
108     case METRIC_WATER_TEMPERATURE: return UNIT_DEG_C;
109     case METRIC_LIGHT_LEVEL: return UNIT_MILLIVOLT;
110     case METRIC_WATER_DISTANCE: return UNIT_MILLIMETRE;
111     case METRIC_PH: return UNIT_PH;
112     case METRIC_TDS: return UNIT_PPM;
113     case METRIC_TVOC: return UNIT_PPB;
114     case METRIC_CO2EQ: return UNIT_PPM;
115     case METRIC_COUNT: break;
116   }
117   return UNIT_NONE;
118 }
119
120 void snapshotReset(Snapshot* s) {
121   memset(s, 0, sizeof(*s));
122   for (uint8_t i = 0; i < METRIC_COUNT; ++i) {
123     readingReset(&s->readings[i], metricSensor((MetricId)i), (MetricId)i,
  >   metricUnit((MetricId)i));
124   }
125 }
```

metricUnit supplies the initial unit expected for each metric. snapshotReset clears publication metadata, then walks through every valid metric index and calls readingReset with its sensor, metric and unit.

The light slot starts in millivolts because lux requires a fitted mapping. Later drivers may publish units and health that make their current calibration status explicit. The loop creates no sensor traffic; it only prepares RAM records.

Publish and peek instead of exporting a writable global

This service gives other tasks a narrow way to obtain a complete retained copy. Its interface hides how the copy is stored.

Safety and the transports call this interface; they do not borrow the acquisition working object.

src/services/snapshot_store.h / lines 1-10

```
1  // Latest-snapshot mailbox (§8, §9). Length-one queue.
2  // Writer: the acquisition lease holder only (xQueueOverwrite).
3  // Readers: Safety, BLE, MQTT peek with zero wait - never consume.
4  #pragma once
5  #include <stdbool.h>
6  #include "domain/reading.h"
7
8  bool snapshotStoreInit(void);
9  void snapshotPublish(const Snapshot* snapshot);
10 bool snapshotPeek(Snapshot* out);  // false if nothing published yet
```

snapshotPublish accepts a const source because the store copies from it. snapshotPeek accepts a writable destination because the service fills caller-owned storage. A false result means no valid queue item was obtained; it does not supply a new Snapshot.

This ownership direction is visible in the parameter types. The interface never returns a pointer into its internal queue storage. The consumer receives a copy it can age-check without blocking the producer's future updates.

Reserve the actual retained item

The complete storage mechanism is one statically allocated queue item. You already know why overwrite and peek fit a latest-state contract.

Read the implementation with sizeof(Snapshot) as the item size, not a pointer size.

src/services/snapshot_store.c / lines 1-15

```
#include "services/snapshot_store.h"

#include <freertos/FreeRTOS.h>
#include <freertos/queue.h>

static StaticQueue_t s_queueControl;
static uint8_t s_queueStorage[sizeof(Snapshot)];
static QueueHandle_t s_queue = NULL;

bool snapshotStoreInit(void) {
  if (s_queue == NULL) {
    s_queue = xQueueCreateStatic(1, sizeof(Snapshot),
  s_queueStorage, &s_queueControl);
  }
  return s_queue != NULL;
}
```

s_queueStorage has enough bytes for one Snapshot, and s_queueControl supplies the kernel bookkeeping. The queue handle begins null. The null guard makes repeated initialisation return the existing queue rather than allocating another one.

xQueueCreateStatic receives capacity one, item size, storage and control addresses. The item is initially absent: creating its storage is not the same as publishing an initial Snapshot.

COMMON PITFALL

Static queue storage must outlive the queue. A local array inside this init function would not be suitable.

Copy into and out of the mailbox

src/services/snapshot_store.c / lines 16-24

```
16
17 void snapshotPublish(const Snapshot* snapshot) {
18   if (s_queue != NULL) xQueueOverwrite(s_queue, snapshot);
19 }
20
21 bool snapshotPeek(Snapshot* out) {
22   if (s_queue == NULL || out == NULL) return false;
23   return xQueuePeek(s_queue, out, 0) == pdTRUE;
24 }
```

Publish ignores a request if initialisation failed; otherwise overwrite installs the latest full copy. Peek rejects a missing queue or destination and asks for an immediate copy without removing the retained item.

Multiple consumers can inspect the latest state independently. They may see different publications if acquisition runs between their peeks, but each successful peek is one internally coordinated queue copy. A reader must still inspect individual Reading ages.

Read a sensor set as several yes/no bits

Configuration and power policy often select whole groups of sensors. This header represents such a group as a bit mask.

A sensor bit answers membership, while SensorId still identifies one device.

src/domain/sensor_set.h / lines 1-27

```
// Fixed sensor set as the app sees it (FirmwareContract.md v2, "Sensors").
// Pure C. Work package WP8 (Stage 5).
//
// Sensors are fixed by the board (pins in board_profile.h). The app can only
// enable or disable them; the enabled set is a SensorMask persisted by
// Configuration. Keys are the protocol names, stable across firmware versions.
#pragma once
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

#include "domain/reading.h"
#include "domain/types.h"

typedef uint8_t SensorMask;  // bit per SensorId
#define SENSOR_MASK_ALL ((SensorMask)((1u << SENSOR_COUNT) - 1u))
// Sensors on the external, switched 5 V rail (§14), gated on the rail epoch.
#define SENSOR_MASK_RAIL_5V ((SensorMask)((1u << SENSOR_HCSR04) | (1u << SENSOR_PH) | (1u << SENSOR_TDS)))

static inline bool sensorMaskHas(SensorMask mask, SensorId s) {
  return s < SENSOR_COUNT && (mask & (1u << s)) != 0;
}

// Protocol key ("bme280", "ds18b20", "light", "hcsr04", "ph", "tds", "sgp30").
const char* sensorKey(SensorId s);
const char* sensorLabel(SensorId s);

```

The mask constants combine 1u << SENSOR_... positions. OR builds a set; AND tests membership; complement can remove permitted bits when intersected with a known all-sensors mask. sensorMaskHas protects against shifting by an invalid sensor index.

COMMON PITFALL

Always constrain a complemented mask to supported bits. ~mask alone also sets unrelated high bits.

Read a sensor set as several yes/no bits / continued

Worked mask - pH is SensorId 4 in types.h

```
uint8_t all = 0x7Fu;              // bits 0 through 6
uint8_t phBit = (uint8_t)(1u << 4); // 0x10
uint8_t withoutPh = (uint8_t)(all & ~phBit);
// withoutPh = 0x6F = 111
```

Read ~ as bitwise complement: it flips each bit of the promoted integer. It differs from logical !, which gives a true/false answer. AND with all keeps only the seven supported positions and clears bit 4. The result includes every supported sensor except pH. This same operation will produce Safety's disabled-sensor mask from the enabled set.

src/domain/sensor_set.h / lines 28-42

```
28 // What disabling this sensor does to the others (app warning), "" if nothing.
29 const char* sensorDependsNote(SensorId s);
30
31 // SENSOR_COUNT if the key (len bytes, not NUL-terminated) is unknown.
32 SensorId sensorFromKey(const char* key, size_t len);
33
34 // Sensor that produces a metric; SENSOR_COUNT for an invalid metric.
35 SensorId sensorOfMetric(MetricId m);
36
37 // Consumer-side rail check (§12), on a snapshot copy: acquisition checks the
38 // epoch when it accepts a result, but the rail can fail after that. Every 5 V
39 // reading measured in another epoch than railEpoch, or any 5 V reading while
40 // the rail is not valid now, becomes UNPOWERED (never shown, captured or
41 // acted on). STARTING readings are left as they are.
42 void snapshotGateRail(Snapshot* s, bool railValid, uint32_t railEpoch);
```

The header also declares names, dependency descriptions and rail-gating helpers. These are mappings over existing records, not driver calls. Later the low-power policy can select retained-rail sensors without teaching the protocol encoder about individual GPIOs.

Map external names to checked identities

This file translates between human-readable sensor keys, metric ownership and rail membership. It lets services apply a whole-sensor decision consistently.

The mappings are shared policy facts. A transport should not duplicate a private, slightly different list.

src/domain/sensor_set.c / lines 1-26

```
#include "domain/sensor_set.h"

#include <string.h>

typedef struct SensorInfo {
  const char* key;
  const char* label;
  const char* dependsNote;
} SensorInfo;

// Indexed by SensorId.
static const SensorInfo kSensors[SENSOR_COUNT] = {
    {"bme280", "BME280 air temperature, humidity, pressure",
     "SGP30 loses humidity compensation"},
    {"ds18b20", "DS18B20 water temperature", "TDS loses temperature compensation"},
    {"light", "DFR0026 light level", "Automatic lamp stays OFF"},
    {"hcsr04", "HC-SR04 water level", ""},
    {"ph", "pH-4502C solution pH", ""},
    {"tds", "SEN0244 TDS", ""},
    {"sgp30", "SGP30 TVOC / CO2eq", ""},
};

const char* sensorKey(SensorId s) { return s < SENSOR_COUNT ? kSensors[s].key : "?"; }

const char* sensorLabel(SensorId s) { return s < SENSOR_COUNT ? kSensors[s].label : "?";
   }

```

The tables hold a machine key, a display label and a dependency note for each sensor. The lookup helpers guard the index before selecting an entry. sensorFromKey accepts a pointer and an explicit byte count: the input need not end with a NUL byte. It requires both matching length and matching bytes.

Map external names to checked identities / continued

src/domain/sensor_set.c / lines 27-32

```
27 const char* sensorDependsNote(SensorId s) {
28   return s < SENSOR_COUNT ? kSensors[s].dependsNote : "";
29 }
30
31 SensorId sensorFromKey(const char* key, size_t len) {
32   for (uint8_t i = 0; i < SENSOR_COUNT; ++i) {
```

src/domain/sensor_set.c / lines 33-37

```
33     const char* k = kSensors[i].key;
34     if (strlen(k) == len && memcmp(k, key, len) == 0) return (SensorId)i;
35   }
36   return SENSOR_COUNT;
37 }
```

The return value is a SensorId, not a boolean. A match returns its array index; an unknown key returns SENSOR_COUNT. The protocol parser will check result < SENSOR_COUNT before accepting that identifier. An unknown key must never become an array index.

Make old rail-dependent values visibly unusable

src/domain/sensor_set.c / lines 38-46

```
38
39 void snapshotGateRail(Snapshot* s, bool railValid, uint32_t railEpoch) {
40   for (uint8_t m = 0; m < METRIC_COUNT; ++m) {
41     Reading* r = &s->readings[m];
42     if (!sensorMaskHas(SENSOR_MASK_RAIL_5V, sensorOfMetric((MetricId)m))) continue;
43     if (r->health == HEALTH_STARTING || r->health == HEALTH_UNPOWERED) continue;
44     if (!railValid || r->powerEpoch != railEpoch) r->health = HEALTH_UNPOWERED;
45   }
46 }
```

snapshotGateRail edits the caller's Snapshot copy. For a 5V sensor, a down rail or mismatched epoch makes the record UNPOWERED. The function does not write the original publication, toggle a supply or read an ADC.

Safety uses this as a second acceptance boundary: even a queued Snapshot that was valid when published can become invalid before the current decision. The caller's fresh rail state is therefore part of the interpretation.

Keep the metric-to-sensor relationship explicit

src/domain/sensor_set.c / lines 47-63

```
47
48 SensorId sensorOfMetric(MetricId m) {
49   switch (m) {
50     case METRIC_AIR_TEMPERATURE:
51     case METRIC_AIR_HUMIDITY:
52     case METRIC_AIR_PRESSURE: return SENSOR_BME280;
53     case METRIC_WATER_TEMPERATURE: return SENSOR_DS18B20;
54     case METRIC_LIGHT_LEVEL: return SENSOR_LIGHT;
55     case METRIC_WATER_DISTANCE: return SENSOR_HCSR04;
56     case METRIC_PH: return SENSOR_PH;
57     case METRIC_TDS: return SENSOR_TDS;
58     case METRIC_TVOC:
59     case METRIC_CO2EQ: return SENSOR_SGP30;
60     case METRIC_COUNT: break;
61   }
62   return SENSOR_COUNT;
63 }
```

sensorOfMetric groups all BME280 metrics under BME280 and both gas metrics under SGP30, then handles the single-metric sensors. An invalid metric returns SENSOR_COUNT.

When you add a metric, review this switch together with types.h, snapshotReset and protocol output. Otherwise the value may be measured correctly but fail to follow its sensor's enable, rail or dependency policy.

COMMON PITFALL

There are mapping helpers in reading.c as well. Duplicated mappings need coordinated updates when the metric set changes.

First obtain millivolts at the pin

The light module changes its output voltage with illumination. The ESP32 ADC converts an input voltage into a digital result. The HAL hides the ADC-specific setup and returns calibrated millivolts. The driver then decides how to label and filter them.

Original firmware - src/hal/adc_channel.c, lines 14-21

```
bool adcChannelBegin(AdcChannel* ch, int8_t pin) {
  ch->pin_ = ADC_CHANNEL_NO_PIN;
  if (pin < kAdc1FirstPin || pin > kAdc1LastPin) return false;
  analogReadResolution(12);
  analogSetPinAttenuation((uint8_t)pin, ADC_11db);
  ch->pin_ = pin;
  return true;
}
```

adcChannelBegin first marks the channel unassigned. It rejects pins outside this project's ADC1 range, sets 12-bit resolution and configures attenuation, then stores the valid pin number. Returning true means setup was accepted; it does not prove the sensor is connected or that its voltage is physically accurate.

12 bits give 4096 possible raw codes, 0 through 4095. An ideal 3.3V full-scale converter would have a step near 0.806mV, but this ADC's transfer curve and attenuation require calibration. Do not convert every raw code with 3300/4095 and assume that is the board's measured behaviour.

The HAL calls analogReadMilliVolts(). ADC calibration maps conversion results to voltage; light calibration later maps voltage to lux. Those are two different calibrations. A good millivolt reading does not by itself supply a lux model.

For this board, the light signal goes through 10kOhm and has 1uF to ground at the input. The nominal RC time constant is 10ms. That circuit smooths the signal, but the driver still collects a time-spread window because lamp flicker can remain.

Reference: Espressif: ADC, v4.4.7 (S3)

TIP

Board pin assignments belong to this ESP32-S3 snapshot. Use board_profile.h rather than copying pins to another ESP32 model.

COMMON PITFALL

Attenuation changes the measurable input range. It does not authorise exceeding a pin's electrical ratings.

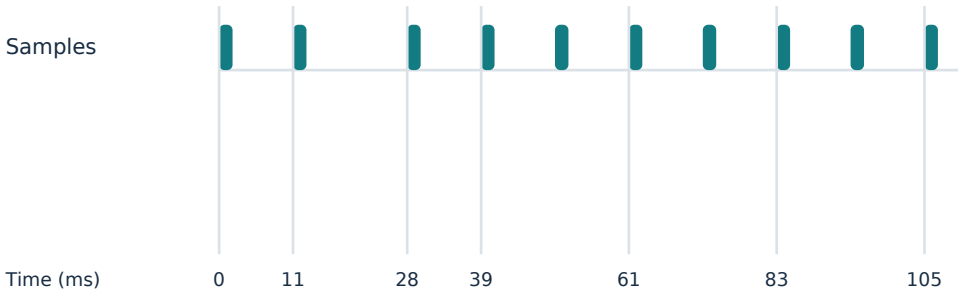
Spread samples across time

The light driver takes ten samples, with each new sample due 11ms after the previous actual one. The first sample happens in start(). The remaining nine happen in later poll() calls. With no delay, the window spans at least 9 x 11 = 99ms.

Original firmware - src/drivers/light.c, lines 59-66

```
  // Spacing is measured from the previous actual sample, never
>    from the window
  // start: after a late poll, catching up would compress the
>   filter taps.
  const MonoMs due = self->lastSampleMs_ +
>   LIGHT_SAMPLE_SPACING_MS;
  if (now < due) {
    r.status = DRIVER_PENDING;
    r.nextPollMs = due;
    return r;
  }
```

due is calculated from lastSampleMs_, not from the window start plus a count. Suppose a sample was intended near 22ms but the task actually gets it at 28ms. The next sample is due at 39ms. Taking two immediate samples at 28ms to catch up would compress the observation window.



Worked light window: one late sample moves the next useful tap, not the past.

Why 11ms? With 100Hz flicker, the light waveform repeats every 10ms. Sampling exactly every 10ms could repeatedly hit the same phase. An 11ms interval walks through that phase instead. It does not guarantee perfect rejection of every lighting waveform, but it gives the chosen filter a reason to use separated observations.

When poll returns PENDING, the acquisition task can work on another sensor or wait. The window occupies elapsed time without making ten back-to-back ADC calls. You can now connect the driver's nextPollMs to the deadline wait you already learned.

COMMON PITFALL

Ten rapid samples are not equivalent to ten samples spread across a flicker cycle. The number of samples alone does not define a filter.

Calculate a filtered value by hand

The driver sorts ten millivolt samples and removes the two lowest and two highest. It averages the remaining six. This is a trimmed mean: isolated extreme values have less influence than they would in an ordinary mean.

Worked data - sorted values, not executable C

```
10, 98, 99, 100, 100, 101, 101, 102, 103, 900
// Remove: 10, 98, 103, 900
// Keep:   99, 100, 100, 101, 101, 102
// Sum = 603; mean = 603 / 6 = 100.5mV
```

An ordinary mean of all ten would be 171.4mV because of the 900mV spike. The trimmed mean is 100.5mV. That shows precisely which sort of disturbance this filter helps with. If all ten values are biased upward, trimming does not recover the true light level.

Original firmware - src/domain/calibration.c, lines 34-48

```
float trimmedMeanU32(uint32_t* samples, uint8_t n, uint8_t
>   trim) {
  if (n <= 2u * trim) return 0.0f;
  for (uint8_t i = 1; i < n; ++i) {  // insertion sort: n is ~10
    const uint32_t v = samples[i];
    uint8_t j = i;
    while (j > 0 && samples[j - 1] > v) {
      samples[j] = samples[j - 1];
      --j;
    }
    samples[j] = v;
  }
  uint64_t sum = 0;
  for (uint8_t i = trim; i < n - trim; ++i) sum += samples[i];
  return (float)sum / (float)(n - 2u * trim);
}
```

The function uses insertion sort because the array is small. v holds the next value to insert. j moves backward while earlier values are larger, shifting them one slot right. The final assignment places v in its sorted position. The function sorts the supplied array in place; it does not preserve the original order.

The accumulator is uint64_t so adding several uint32_t values does not immediately overflow 32 bits. The denominator n - 2*trim is the number of retained values. The initial guard rejects a window with no retained values. For our ten-sample, two-trim case, that denominator is six.

REMEMBER

A filter reduces selected disturbances. It does not prove connection, calibration or freshness.

Publish the unit you can justify

Original firmware - src/drivers/light.c, lines 87-94

```
  Reading* r = &self->result_;
  r->intervalStartMs = self->startMs_;
  r->intervalEndMs = now;
  r->hasRaw = true;
  r->raw = (int32_t)(pinMv + 0.5f);
  // Clipped = brighter than the ADC can show: a lower bound,
>   not a fault.
  r->qualityFlags = self->saturated_ ? kQualitySaturated :
>   kQualityNone;
  r->calibrationRevision = self->config_.calibration.revision;
```

pinMv is the filtered result. raw stores rounded pin millivolts so the lamp policy can use the same control quantity even after a lux calibration. calibrationRevision records which fit produced the physical value.

Without a valid light calibration, the driver publishes value=pinMv, unit=MILLIVOLT and health=UNCALIBRATED. It has measured a voltage, but has not established a trustworthy lux conversion. With a valid gain, it can publish lux and OK, provided the computed value is finite.

For a worked calibration, suppose a reference meter says 500lux while the filtered pin voltage is 100mV. A gain-through-zero model uses 5lux/mV. At 140mV it predicts 700lux. This is a chosen model; a single point does not establish linearity over the whole range or every light spectrum.

Saturation has a metric-specific interpretation. The ADC may reach its ceiling in bright light. The light driver keeps a lower-bound indication and sets a quality flag rather than pretending it knows the full illumination. The lamp can interpret saturation as bright. That same approach would be wrong for a pH voltage clipped at the ADC ceiling.

A disconnected light signal is harder: the capacitor may retain a plausible voltage, and near-zero voltage can be real darkness. The driver cannot reliably identify every unplugged condition from these samples. Later policy cannot manufacture information the electrical interface never supplied.

COMMON PITFALL

Do not label a voltage lux merely because a library or variable name sounds like a physical unit.

Keep the ADC state smaller than the sensor state

Begin the production light path at the electrical boundary. This interface promises a pin-voltage result; it deliberately says nothing about lux, pH or ppm.

The three analog drivers reuse this adapter with different pins and conversion policies.

COMMON PITFALL

The approximate range in the source comment is a board-characterisation assumption, not a universal guarantee for every attenuation and chip.

Keep the ADC state smaller than the sensor state / continued

src/hal/adc_channel.h / lines 1-24

```h
// ADC1 channel adapter (HAL). Work package WP3 (shared by light, pH and TDS).
// ADC1 only: ADC2 conflicts with Wi-Fi (§3). Returns calibrated millivolts at
// the ESP32 pin; divider recovery and units belong to domain/calibration.h.
//
// 12-bit, 11 dB attenuation (S3 usable range about 0-3100 mV), eFuse calibration.
// Worst-case blocking: one conversion per read, well under 1 ms.
#pragma once
#include <stdbool.h>
#include <stdint.h>

#define ADC_CHANNEL_NO_PIN ((int8_t)-1)

// Treat the members as private to adc_channel.c. Initialise with ADC_CHANNEL_INIT.
typedef struct AdcChannel {
  int8_t pin_;
} AdcChannel;

#define ADC_CHANNEL_INIT {ADC_CHANNEL_NO_PIN}

// false if the pin is not an ADC1 pin (GPIO1-10 on the S3); the channel then
// stays unusable and every read fails.
bool adcChannelBegin(AdcChannel* ch, int8_t pin);
// One conversion. false on error. Reports saturation near full scale.
bool adcChannelReadMilliVolts(AdcChannel* ch, uint32_t* mv, bool* saturated);
```

AdcChannel stores only an assigned pin. ADC_CHANNEL_INIT gives it the unassigned sentinel before setup. A negative sentinel fits int8_t; it would not mean the same thing if stored in an unsigned pin field.

The read function can fill a millivolt output and a saturation flag. Those are optional output addresses in this implementation. The boolean return concerns the adapter's accepted operation; a plausible voltage alone cannot prove a physically connected probe.

Make a channel unusable before validating it

Now follow the adapter from validation to one conversion. Notice which checks it can perform and which it cannot.

The light driver will turn this result into a filtered window rather than changing this shared HAL.

src/hal/adc_channel.c / lines 1-21

```
#include "hal/adc_channel.h"

#include <Arduino.h>
#include <stddef.h>

// ESP32-S3: ADC1 is GPIO1-10 (ADC2 conflicts with Wi-Fi,
  §3).
static const int8_t kAdc1FirstPin = 1;
static const int8_t kAdc1LastPin = 10;

// At 11 dB (ADC_ATTEN_DB_12 in IDF 5) the S3 measures about
   0-3100 mV. Treat a
// calibrated result at or above this ceiling as clipped.
  Characterise per board.
static const uint32_t kSaturationMv = 3050;

bool adcChannelBegin(AdcChannel* ch, int8_t pin) {
  ch->pin_ = ADC_CHANNEL_NO_PIN;
  if (pin < kAdc1FirstPin || pin > kAdc1LastPin) return
  false;
  analogReadResolution(12);
  analogSetPinAttenuation((uint8_t)pin, ADC_11db);
  ch->pin_ = pin;
  return true;
}
```

The function first writes the unassigned sentinel, then validates the requested pin against this S3 ADC1 range. A rejected call therefore does not leave a previously assigned pin accidentally usable.

After setting resolution and attenuation, it records the valid pin. The function does not allocate a task, reserve a sample window or perform light calibration. Those operations belong to later layers.

COMMON PITFALL

A hardware configuration call affects a peripheral. Review its scope before assuming every setting is private to one AdcChannel object.

Return voltage and clipping as separate outputs

src/hal/adc_channel.c / lines 22-32

```
22
23 bool adcChannelReadMilliVolts(AdcChannel* ch, uint32_t* mv, bool* saturated) {
24   if (mv) *mv = 0;
25   if (saturated) *saturated = false;
26   if (ch->pin_ < 0) return false;
27   // eFuse-calibrated conversion (esp_adc_cal). One conversion, a few tens of us.
28   const uint32_t value = analogReadMilliVolts((uint8_t)ch->pin_);
29   if (mv) *mv = value;
30   if (saturated) *saturated = value >= kSaturationMv;
31   return true;
32 }
```

The function clears requested outputs before checking the channel. It rejects an unassigned pin, then calls analogReadMilliVolts and compares the result with the chosen saturation threshold. A caller can distinguish a completed voltage conversion from the flag saying it is near the useful ceiling.

Notice a limitation of this API path: the wrapper has no separate hardware error result returned by analogReadMilliVolts. Its true return mainly means the channel was assigned and the call path completed. Do not describe it as independent proof that the ADC and probe both passed diagnostics.

Describe a fitted line and its validity

This header holds mathematical helpers shared by light, pH and TDS. Learn the data shape now; the pH and TDS chapters will show why the same fields mean different physical mappings.

A calibration record is copied into a driver configuration. It is not a pointer to a live editing screen.

src/domain/calibration.h / lines 1-25

```
1  // Unit conversion and calibration stages (§3, §14). Pure C.
2  // Work packages: WP5 (pH/TDS), WP3 (light).
3  #pragma once
4  #include <stdbool.h>
5  #include <stdint.h>
6
7  // Recover the module output voltage from the ADC pin voltage through a
8  // resistor divider: Vmodule = Vadc * (Rtop + Rbottom) / Rbottom.
9  // Use MEASURED resistor values once available (§3).
10 float dividerRecoverMv(float adcMv, uint32_t rTopOhm, uint32_t rBottomOhm);
11
12 // Linear fit y = slope * x + offset, with a revision for traceability.
13 typedef struct LinearCalibration {
14   float slope;
15   float offset;
16   uint16_t revision;
17   bool valid;  // false => publish raw with HEALTH_UNCALIBRATED
18 } LinearCalibration;
19
20 float linearApply(const LinearCalibration* c, float x);
21
22 // True if slope and offset are finite numbers. Every fit maker below refuses
23 // non-finite input or output; config_blob refuses a stored non-finite fit.
24 bool linearCalibrationFinite(const LinearCalibration* c);
25
```

LinearCalibration stores slope, offset, revision and valid. linearApply evaluates slope*x+offset. valid is an acceptance tag supplied by fit creation and configuration validation; the arithmetic helper does not infer it from how reasonable a result looks.

Describe a fitted line and its validity / continued

src/domain/calibration.h / lines 26-34

```
26 // Two-point fit through (x1, y1) and (x2, y2), e.g. module mV read in two pH
27 // buffers. false (and *out left invalid) if the x values are too close.
28 bool linearFromTwoPoints(float x1, float y1, float x2, float y2, uint16_t revision,
29                          LinearCalibration* out);
30
31 // Sort `samples` in place, drop `trim` values at each end and return the mean
32 // of the rest (spike-rejecting filter, pH-4502C tutorial: 10 samples, mean of
33 // the middle 6). Returns 0 if n <= 2 * trim.
34 float trimmedMeanU32(uint32_t* samples, uint8_t n, uint8_t trim);
```

The divider helper recovers the upstream module voltage. Its coefficients are resistor values, not calibration points. Keep electrical scaling and physical calibration as separate transformations so you can test each with known numbers.

Distinguish a reference fit from a compiled default

src/domain/calibration.h / lines 35-53

```
35
36 // ---- On-device calibration (WP8, Calibration page) ----------------------------
37 // Which stored fits the user made on the device (ConfigBlob.calOverridden).
38 // A fit without its bit is the compiled default and is refreshed at boot.
39 #define CAL_BIT_PH 0x01u
40 #define CAL_BIT_TDS 0x02u
41 #define CAL_BIT_LIGHT 0x04u
42 #define CAL_BITS_ALL 0x07u
43
44 // One-point gain through the origin, y = (y1 / x1) * x (TDS K-factor, light
45 // lux per mV). false if x1 or y1 is not positive or the gain is outside
46 // [minGain, maxGain].
47 bool linearGainFromPoint(float x1, float y1, float minGain, float maxGain, uint16_t
 >   revision,
48                          LinearCalibration* out);
49
50 // Two-point pH buffer fit, rejected unless |slope| (pH per module mV) is in
51 // [minAbsSlope, maxAbsSlope] and the buffers are at least 1 pH apart.
52 bool phFitFromBuffers(float mv1, float ph1, float mv2, float ph2, float minAbsSlope,
 >   float maxAbsSlope,
53                       uint16_t revision, LinearCalibration* out);
```

The CAL_BIT constants record which sensor fits came from a user calibration. They let configuration refresh compiled defaults while preserving a deliberately fitted replacement. The bit mask is history about the fit's origin, not its mathematical formula.

The one-point helper constrains a gain through zero. The pH helper fits two points and checks both buffer separation and slope magnitude. Those restrictions embody chosen models; do not replace a two-point pH fit with a gain-only light model because both use LinearCalibration storage.

Recover a voltage, then evaluate a line

We will read every helper with numerical inputs. None of these functions touches a peripheral, which makes the same bodies testable on a PC.

The light driver calls the filter and line evaluation; later calibration commands call the fit makers.

src/domain/calibration.c / lines 1-14

```
#include "domain/calibration.h"

#include <math.h>

float dividerRecoverMv(float adcMv, uint32_t rTopOhm,
  uint32_t rBottomOhm) {
  if (rBottomOhm == 0) return 0.0f;
  return adcMv * (float)(rTopOhm + rBottomOhm) /
  (float)rBottomOhm;
}

float linearApply(const LinearCalibration* c, float x) {
  return c->slope * x + c->offset;
}

bool linearCalibrationFinite(const LinearCalibration* c) {
  return isfinite(c->slope) && isfinite(c->offset); }
```

dividerRecoverMv rejects a zero lower resistor before division. With 10kOhm above and 15kOhm below, an ADC input of 1800mV corresponds to 3000mV at the module. linearApply then uses the supplied line without changing its record.

linearCalibrationFinite checks slope and offset with isfinite. NaN and infinity are floating-point representations, not usable calibration coefficients. A nonzero or valid-looking flag alone does not make them acceptable.

COMMON PITFALL

The helper returns zero for an invalid divider denominator; callers still need to establish whether zero is a meaningful result or a rejected configuration.

Create a line only when two points determine one

src/domain/calibration.c / lines 15-32

```
15
16 bool linearFromTwoPoints(float x1, float y1, float x2, float y2, uint16_t revision,
17                          LinearCalibration* out) {
18   out->slope = 1.0f;
19   out->offset = 0.0f;
20   out->revision = revision;
21   out->valid = false;
22   if (!isfinite(x1) || !isfinite(y1) || !isfinite(x2) || !isfinite(y2)) return false;
23   const float dx = x2 - x1;
24   if (!(dx <= -1e-3f || dx >= 1e-3f)) return false;
25   const float slope = (y2 - y1) / dx;
26   const float offset = y1 - slope * x1;
27   if (!isfinite(slope) || !isfinite(offset)) return false;
28   out->slope = slope;
29   out->offset = offset;
30   out->valid = true;
31   return true;
32 }
```

The output begins with an invalid, neutral-looking line. Each early return leaves valid false. The finite checks reject malformed numeric input, and the dx test rejects coincident or nearly coincident voltages before dividing.

Once slope and offset are finite, the function commits all fields and sets valid true. For (3000,7) and (3500,4), it produces -0.006 and 25. Calling code must check the boolean; merely seeing slope=1 after failure is not a successful identity calibration.

Watch insertion sort change the array

src/domain/calibration.c / lines 33-48

```
33
34 float trimmedMeanU32(uint32_t* samples, uint8_t n, uint8_t
 >   trim) {
35   if (n <= 2u * trim) return 0.0f;
36   for (uint8_t i = 1; i < n; ++i) {  // insertion sort: n is
 >    ~10
37     const uint32_t v = samples[i];
38     uint8_t j = i;
39     while (j > 0 && samples[j - 1] > v) {
40       samples[j] = samples[j - 1];
41       --j;
42     }
43     samples[j] = v;
44   }
45   uint64_t sum = 0;
46   for (uint8_t i = trim; i < n - trim; ++i) sum +=
 >   samples[i];
47   return (float)sum / (float)(n - 2u * trim);
48 }
```

For each new element, v remembers the value while larger earlier entries move right. j stops at the beginning or at a value that should remain before v. The assignment after the while loop completes that insertion.

The sum excludes trim items from each end and uses uint64_t for accumulation. For ten samples and trim two, six items remain. Sorting edits the input buffer; copy the samples first if another consumer needs their original temporal order.

COMMON PITFALL

This filter rejects isolated extremes. It cannot correct a persistent offset shared by every sample.

Build a bounded gain through zero

src/domain/calibration.c / lines 49-66

```
49
50 static void invalidFit(uint16_t revision, LinearCalibration* out) {
51   out->slope = 1.0f;
52   out->offset = 0.0f;
53   out->revision = revision;
54   out->valid = false;
55 }
56
57 bool linearGainFromPoint(float x1, float y1, float minGain, float maxGain, uint16_t
 >   revision,
58                          LinearCalibration* out) {
59   invalidFit(revision, out);
60   if (!(x1 > 0.0f) || !(y1 > 0.0f) || !isfinite(x1) || !isfinite(y1)) return false;
61   const float gain = y1 / x1;
62   if (!isfinite(gain) || !(gain >= minGain && gain <= maxGain)) return false;
63   out->slope = gain;
64   out->valid = true;
65   return true;
66 }
```

invalidFit establishes the same failure state for all later paths. The helper requires positive finite x and y, computes y/x, and checks the agreed gain range before marking the result valid.

A 500ppm reference over a nominal 400ppm prediction gives 1.25. The offset stays zero because this helper's model is a gain, not an arbitrary line. A single reference does not tell you an independent offset.

Add pH-specific plausibility to a general line

src/domain/calibration.c / lines 67-79

```
67
68 bool phFitFromBuffers(float mv1, float ph1, float mv2, float ph2, float minAbsSlope,
 >   float maxAbsSlope,
69                       uint16_t revision, LinearCalibration* out) {
70   invalidFit(revision, out);
71   const float dph = ph2 - ph1;
72   if (!(dph <= -1.0f || dph >= 1.0f)) return false;  // also NaN
73   LinearCalibration fit;
74   if (!linearFromTwoPoints(mv1, ph1, mv2, ph2, revision, &fit)) return false;
75   const float mag = fit.slope < 0.0f ? -fit.slope : fit.slope;
76   if (!(mag >= minAbsSlope && mag <= maxAbsSlope)) return false;
77   *out = fit;
78   return true;
79 }
```

phFitFromBuffers first requires the reference pH values to differ by at least one unit. It delegates the algebra to linearFromTwoPoints, then checks the absolute slope against policy bounds. Negative slopes are allowed because the electrical response can run opposite to pH.

Only the final assignment copies the accepted fit into out. The helper does not collect a new sensor sample or store anything in flash. Configuration later supplies captured points and decides whether to persist the result.

Read the light-specific physical assumptions

This is the first full concrete driver object. Match its members to the actions you have already traced in the lesson.

The configuration points to an already configured ADC channel, while the driver owns its window and result.

src/drivers/light.h / lines 1-28

```
// dfr0026_light driver. Work package WP3 (Stage 2). Architecture §3, §14.
// Source: DFR0026.PDF (DFRobot Analog Ambient Light Sensor V2, pages 1-4).
//
// Retained 3.3 V rail. S -> 10 kOhm series -> ADC1 pin (GPIO1), 1 uF to GND
// (tau ~10 ms). No divider: pin mV = module output mV. More light, higher
// voltage. Datasheet range 1-6000 lux, but the output has no lux unit: the FAQ
// requires a reference-meter mapping (D13). Without one every result is mV.
// With an on-device fit (Calibration page: one reference-meter reading, gain
// through zero) the value is lux. raw stays pin mV either way, and the lamp
// thresholds use it (lamp_control), so calibrating never moves the lamp.
//
// Filter: LIGHT_SAMPLES conversions spaced LIGHT_SAMPLE_SPACING_MS apart,
// sorted, mean of the middle LIGHT_SAMPLES - 2*LIGHT_TRIM. Mains lamps flicker
// at 100 Hz (120 Hz on 60 Hz grids) and the RC only cuts 100 Hz to ~16 %, so
// the spacing is deliberately NOT a multiple of 10 ms: 11 ms steps the flicker
// phase by 1 ms per tap and the window covers the whole period. The window is
// spread over successive poll() calls, never taken as a burst (§9): each tap is
// due LIGHT_SAMPLE_SPACING_MS after the previous actual sample.
//
// Results (poll() returns READY for all; raw = filtered pin mV):
//   calibration.valid -> value = lux, UNIT_LUX, HEALTH_OK
//   otherwise         -> value = module mV, UNIT_MILLIVOLT, HEALTH_UNCALIBRATED
//   any sample clipped -> kQualitySaturated, value kept as a lower bound
//                         (bright light on a 3.3 V supply can exceed the S3's
//                         ~3.1 V range; this is not a fault)
//   ADC error          -> HEALTH_FAULT
// No no-signal threshold: ~0 mV is real darkness. With 1 uF on the pin and no
// pull-down, an unplugged S line holds its last level while the capacitor leaks;
```

The introductory contract explains the pin RC network, the absence of a justified lux mapping, and the decision to preserve raw millivolts for control. These comments describe why the same numeric field may use MILLIVOLT before calibration and LUX afterward.

src/drivers/light.h / lines 29-30

```
// the driver cannot tell that apart. Characterise it at bring-up.
//
```

The output cannot always distinguish unplugged signal from real darkness. That is a property of the electrical observations available to the code. Treat it as a diagnostic boundary when reviewing the later health branches, rather than adding a zero-voltage rule without evidence.

Connect elapsed filtering time to short calls

src/drivers/light.h / lines 31-55

```
31 // Worst-case blocking (one ADC conversion C, well under 1 ms):
32 //   init()    1 C   (throwaway conversion, checks the channel)
33 //   start()   1 C   (first sample of the window)
34 //   poll()    1 C   Pending until the window is full (>= 99 ms)
35 //   others    none
36 //
37 // Implements the SensorDriver contract in drivers/sensor_driver.h.
38 #pragma once
39 #include "drivers/sensor_driver.h"
40 #include "hal/adc_channel.h"
41 #include "domain/calibration.h"
42
43 #define LIGHT_SAMPLES 10
44 #define LIGHT_TRIM 2
45 #define LIGHT_SAMPLE_SPACING_MS 11
46
47 typedef struct LightConfig {
48   AdcChannel* adc;                // already begun on BOARD_PIN_LIGHT_ADC
49   LinearCalibration calibration;  // x = pin mV, y = lux; invalid = publish mV
50 } LightConfig;
51
52 // Members after `base` are private to light.c.
53 typedef struct LightDriver {
54   SensorDriver base;  // must stay first
55   LightConfig config_;
```

The filter uses ten taps, trims two from each end and spaces actual taps by 11ms. start takes one conversion; each eligible poll takes at most one more. A minimum 99ms window can therefore coexist with a much shorter per-call execution budget.

The header promises that partial work can be cancelled. A schedule deadline may end a window before all samples arrive; the service must not pretend that a full-window result was obtained.

Identify which fields survive the next return

src/drivers/light.h / lines 56-71

```
56   bool initialised_;
57   bool busy_;
58   bool saturated_;
59   uint8_t count_;
60   uint32_t samples_[LIGHT_SAMPLES];  // pin mV
61   MonoMs startMs_;
62   MonoMs lastSampleMs_;  // next tap is due LIGHT_SAMPLE_SPACING_MS after this
63   Reading result_;
64 } LightDriver;
65
66 // Constructor. Use &driver.base wherever a SensorDriver* is expected.
67 void lightDriverConstruct(LightDriver* self, const LightConfig* config);
68
69 // New lux fit from the applied configuration (on-device calibration, WP8).
70 // Called by the acquisition owner between windows; applies from the next one.
71 void lightDriverSetCalibration(LightDriver* self, const LinearCalibration* calibration);
```

base must remain first for generic dispatch. config_ holds an ADC pointer and a calibration copy. count_, samples_, startMs_ and lastSampleMs_ retain the in-progress window. result_ retains the latest completed outcome.

You can now predict construction: install the operations table, copy configuration, clear state, and reset result identity. Reinitialising the hardware should not erase the historical meaning of every earlier result unless the contract explicitly says so.

Confirm the channel before accepting work

Follow one complete window through the real implementation. Keep the ten sample times and one retained Reading separate in your mind.

Acquisition will call these bodies through SensorDriverOps; the code here does not know a queue or relay.

COMMON PITFALL

A successful init does not establish a lux calibration or prove that the signal cable is connected.

Confirm the channel before accepting work / continued

src/drivers/light.c / lines 1-27

```
#include "drivers/light.h"

#include <math.h>

#include "domain/calibration.h"

static inline LightDriver* asLight(SensorDriver* d) { return (LightDriver*)d; }

static PollResult finish(LightDriver* self, MonoMs now);

static SensorId lightId(const SensorDriver* d) { (void)d; return SENSOR_LIGHT; }
static const char* lightName(const SensorDriver* d) { (void)d; return "dfr0026_light"; }
static uint8_t lightMetricCount(const SensorDriver* d) { (void)d; return 1; }

static DriverStatus lightInit(SensorDriver* d) {
  LightDriver* self = asLight(d);
  self->initialised_ = false;
  self->busy_ = false;
  if (self->config_.adc == 0) return DRIVER_FAULT;
  // Throwaway conversion: fails if the channel was never begun on an ADC1 pin,
  // and keeps any one-time ADC characterisation cost out of poll().
  uint32_t mv = 0;
  bool saturated = false;
  if (!adcChannelReadMilliVolts(self->config_.adc, &mv, &saturated)) return
  DRIVER_FAULT;
  self->initialised_ = true;
  return DRIVER_OK;
}
```

The cast helper recovers LightDriver from its first-member base pointer. Identity helpers supply the physical sensor, diagnostic name and one-metric count. They return constants and do not inspect hardware.

lightInit clears active state, rejects a missing ADC pointer and performs one throwaway voltage conversion. That moves any one-time setup cost out of normal poll steps. Only the successful path sets initialised_ true.

Take exactly one tap

src/drivers/light.c / lines 28-38

```
28
29 // Take one sample of the window at `now`. false on an ADC error.
30 static bool sample(LightDriver* self, MonoMs now) {
31   uint32_t mv = 0;
32   bool saturated = false;
33   if (!adcChannelReadMilliVolts(self->config_.adc, &mv, &saturated)) return false;
34   self->samples_[self->count_++] = mv;
35   self->lastSampleMs_ = now;
36   self->saturated_ = self->saturated_ || saturated;
37   return true;
38 }
```

sample obtains one voltage, appends it at count_, records the actual sample time and accumulates saturation with logical OR. Once any tap saturates, saturated_ remains true for the rest of that window.

The post-increment uses the old count as the index, then advances it. The caller's state machine must keep count below the array capacity. This helper does not check the index independently; its safe use depends on the start/poll logic.

Start a new window with a real first sample

src/drivers/light.c / lines 39-52

```
39
40 static DriverStatus lightStart(SensorDriver* d, MonoMs now) {
41   LightDriver* self = asLight(d);
42   if (!self->initialised_) return DRIVER_FAULT;
43   self->count_ = 0;
44   self->saturated_ = false;
45   self->startMs_ = now;
46   if (!sample(self, now)) {
47     self->initialised_ = false;
48     return DRIVER_FAULT;
49   }
50   self->busy_ = true;
51   return DRIVER_OK;
52 }
```

The function rejects uninitialised state, resets the count and saturation tag, remembers now, and immediately obtains the first tap. Failure clears initialisation so the caller cannot keep starting as if the channel were healthy.

busy_ is set only after that first tap succeeds. start therefore returns OK with a real partial window, not merely an intention to sample later. The next poll time can be derived from the recorded actual tap time.

Let timing decide whether another tap is useful

src/drivers/light.c / lines 53-79

```
53
54 static PollResult lightPoll(SensorDriver* d, MonoMs now) {
55   LightDriver* self = asLight(d);
56   PollResult r = {DRIVER_FAULT, now};
57   if (!self->busy_) return r;
58
59   // Spacing is measured from the previous actual sample, never from the window
60   // start: after a late poll, catching up would compress the filter taps.
61   const MonoMs due = self->lastSampleMs_ + LIGHT_SAMPLE_SPACING_MS;
62   if (now < due) {
63     r.status = DRIVER_PENDING;
64     r.nextPollMs = due;
65     return r;
66   }
67   if (!sample(self, now)) {
68     self->busy_ = false;
69     self->initialised_ = false;
70     self->result_.health = HEALTH_FAULT;
71     self->result_.intervalStartMs = self->startMs_;
72     self->result_.intervalEndMs = now;
73     return r;
74   }
75   if (self->count_ < LIGHT_SAMPLES) {
76     r.status = DRIVER_PENDING;
77     r.nextPollMs = self->lastSampleMs_ + LIGHT_SAMPLE_SPACING_MS;
78     return r;
79   }
```

A poll without active work returns the initial FAULT outcome. Otherwise due equals the last actual sample time plus the spacing. Before due, the function returns PENDING without taking another conversion.

src/drivers/light.c / lines 80-81

```
80   return finish(self, now);
81 }
```

When a tap succeeds, count decides whether the window is complete. If not, another PENDING return supplies the next useful time. If the ADC helper fails, the function ends the window and records a new fault interval. This branch is different from simply receiving a plausible low light voltage.

Filter the window and retain its electrical scale

src/drivers/light.c / lines 82-105

```
82
83 static PollResult finish(LightDriver* self, MonoMs now) {
84   self->busy_ = false;
85   const float pinMv = trimmedMeanU32(self->samples_, LIGHT_SAMPLES, LIGHT_TRIM);
86
87   Reading* r = &self->result_;
88   r->intervalStartMs = self->startMs_;
89   r->intervalEndMs = now;
90   r->hasRaw = true;
91   r->raw = (int32_t)(pinMv + 0.5f);
92   // Clipped = brighter than the ADC can show: a lower bound, not a fault.
93   r->qualityFlags = self->saturated_ ? kQualitySaturated : kQualityNone;
94   r->calibrationRevision = self->config_.calibration.revision;
95   if (self->config_.calibration.valid) {
96     r->unit = UNIT_LUX;
97     r->value = linearApply(&self->config_.calibration, pinMv);
98     if (r->value < 0.0f) r->value = 0.0f;
99     r->health = HEALTH_OK;
100    if (!isfinite(r->value)) {  // a broken fit is a fault, never a value
101      r->health = HEALTH_FAULT;
102      PollResult bad = {DRIVER_READY, now};
103      return bad;
104    }
105  } else {
```

finish clears busy_ and computes the trimmed pin voltage. It stamps the interval, retains rounded raw millivolts and records the saturation and calibration revision. These are the facts shared by both calibrated and uncalibrated outcomes.

With a valid fit, value changes to lux and negative predicted lux is clamped to zero. The finite check still rejects a broken coefficient path. HEALTH_OK is meaningful only after that check; assignment order inside this local result does not mean a consumer saw the earlier intermediate state.

Keep an uncalibrated value honest

src/drivers/light.c / lines 106-114

```
106      r->unit = UNIT_MILLIVOLT;
107      r->value = pinMv;  // no divider: pin mV = module mV
108      r->health = HEALTH_UNCALIBRATED;  // no lux mapping until a reference meter (D13)
109    }
110    r->lastGoodMs = now;  // last usable result (OK or UNCALIBRATED)
111    ++r->sequence;
112    PollResult pr = {DRIVER_READY, now};
113    return pr;
114  }
```

Without a valid mapping, the result stays in millivolts and is labelled UNCALIBRATED. The driver nevertheless updates lastGoodMs and its usable-sample sequence because the raw observation can be useful to the lamp's specific raw-mV policy.

READY reports that the outcome is available. It does not erase the distinction between OK lux and UNCALIBRATED mV. Ordinary numeric alarm rules use their own acceptance policy and do not inherit the lamp exception automatically.

Copy results and abandon partial windows

src/drivers/light.c / lines 115-130

```
115
116 static uint8_t lightReadResult(SensorDriver* d, Reading* out, uint8_t maxOut) {
117   if (maxOut < 1) return 0;
118   out[0] = asLight(d)->result_;
119   return 1;
120 }
121
122 static void lightCancel(SensorDriver* d) {
123   LightDriver* self = asLight(d);
124   self->busy_ = false;
125   self->count_ = 0;
126 }
127
128 // Retained rail, and the ADC pin is a high-impedance input behind 10 kOhm:
129 // nothing to drive. Drop any partial window.
130 static void lightPreparePowerOff(SensorDriver* d) { lightCancel(d); }
```

readResult checks capacity and copies one retained record. cancel drops busy_ and count_ so a partial window cannot continue. preparePowerOff delegates to that cancellation because this retained-rail analog input has no output line that needs to be driven low.

These functions do not publish a Snapshot. The owner first collects the copy and later adds service metadata. Keeping those responsibilities separate lets this same driver run in the small bring-up task.

Install the table once, then update fits between windows

src/drivers/light.c / lines 131-156

```
131
132 static const SensorDriverOps kLightOps = {
133     .id = lightId,
134     .name = lightName,
135     .metricCount = lightMetricCount,
136     .init = lightInit,
137     .start = lightStart,
138     .poll = lightPoll,
139     .readResult = lightReadResult,
140     .cancel = lightCancel,
141     .preparePowerOff = lightPreparePowerOff,
142 };
143
144 void lightDriverConstruct(LightDriver* self, const LightConfig* config) {
145   self->base.ops = &kLightOps;
146   self->config_ = *config;
147   self->initialised_ = false;
148   self->busy_ = false;
149   self->saturated_ = false;
150   self->count_ = 0;
151   self->startMs_ = 0;
152   self->lastSampleMs_ = 0;
153   readingReset(&self->result_, SENSOR_LIGHT, METRIC_LIGHT_LEVEL, UNIT_MILLIVOLT);
154   self->result_.calibrationRevision = config->calibration.revision;
155 }
156
```

kLightOps names each concrete function for the generic lifecycle. The constructor points base.ops to that constant table, copies configuration and establishes predictable initial state and Reading identity.

src/drivers/light.c / lines 157-159

```
157 void lightDriverSetCalibration(LightDriver* self, const LinearCalibration* calibration)
  >   {
158   self->config_.calibration = *calibration;
159 }
```

SetCalibration copies a complete fit. It does not take its own lock or cancel a current window; the acquisition owner is responsible for calling it at an appropriate boundary. This is an ownership precondition, not a hidden property of the struct assignment.

Carry a driver and a period into one task

Before looking at the light test entry point, read the reusable task that runs one production driver. It is a small bridge from lessons to a working board experiment.

The harness deliberately omits multi-sensor ownership handover and output policy so one device can be observed in isolation.

test/bringup/bringup_harness.h / lines 1-27

```
1  // Minimal RTOS bring-up harness shared by every test/bringup/<sensor>_main.c.
2  // Architecture §17 stage 1-3, requirement R9.
3  //
4  // Runs ONE production driver from a task pinned like AcquisitionNormal
5  // (core 1, priority 4), on an absolute period (vTaskDelayUntil). Each period:
6  // init (retried until OK) -> start -> poll until READY/error -> readResult.
7  // Prints every reading with its health and the start->ready duration so the
8  // conversion budget in app_config.h can be checked against reality.
9  #pragma once
10 #include <Arduino.h>
11 #include <stdio.h>
12
13 #include "app_config.h"
14 #include "domain/reading.h"
15 #include "drivers/sensor_driver.h"
16 #include "hal/arduino_entry.h"
17 #include "hal/clock.h"
18
19 typedef struct BringupContext {
20   SensorDriver* driver;
21   uint32_t periodMs;
22   // Optional, called from the harness task after every READY result (so it
23   // may use driver getters without breaking single bus ownership). NULL = none.
24   void (*afterResult)(SensorDriver* driver);
25 } BringupContext;
26
27 // One compact line per measurement, e.g.
```

BringupContext contains the generic driver pointer, requested period and an optional function pointer called after a READY result. The callback executes in the same harness task, so it can inspect driver-owned state without introducing another concurrent bus user.

A null callback means no extra action. Do not interpret it as a missing driver function: the normal lifecycle still runs through ops.

Print a value only under a declared health policy

test/bringup/bringup_harness.h / lines 28-47

```
28 //   [bme280] 27.98 degC | 54.20 %RH | 101602.04 Pa   (11 ms)
29 // An UNCALIBRATED reading shows its raw-unit value with "(uncal)", e.g.
30 //   [ph4502c] 2498.31 mV (uncal)   (271 ms)
31 // Any other non-OK reading shows its metric and health instead of a value.
32 static inline void bringupPrintResult(const char* name, const Reading* r, uint8_t n,
33                                        uint32_t elapsedMs) {
34   printf("[%s] ", name);
35   for (uint8_t i = 0; i < n; ++i) {
36     if (i > 0) printf(" | ");
37     if (r[i].health == HEALTH_OK) {
38       printf("%.2f %s", (double)r[i].value, unitName(r[i].unit));
39     } else if (r[i].health == HEALTH_UNCALIBRATED) {
40       printf("%.2f %s (uncal)", (double)r[i].value, unitName(r[i].unit));
41     } else {
42       printf("%s=%s", metricName(r[i].metric), healthName(r[i].health));
43       if (r[i].hasRaw) printf(" (raw %ld)", (long)r[i].raw);
44     }
45   }
46   printf("   (%lu ms)\n", (unsigned long)elapsedMs);
47 }
```

The formatter walks the returned records. OK values print their units; UNCALIBRATED values print with an explicit tag; other health states print the metric and verdict, optionally raw data.

The casts match variadic printf expectations. Floating arguments are passed as double, and the explicit long casts match integer formats. This is a diagnostic interface, not a new calibration step. A formatted number retains the meaning of the record supplied to it.

Release one observation on an absolute period

test/bringup/bringup_harness.h / lines 48-72

```
48
49 static inline void bringupTask(void* arg) {
50   BringupContext* ctx = (BringupContext*)arg;
51   SensorDriver* drv = ctx->driver;
52   bool initialised = false;
53   TickType_t lastWake = xTaskGetTickCount();
54
55   for (;;) {
56     vTaskDelayUntil(&lastWake, pdMS_TO_TICKS(ctx->periodMs));
57
58     if (!initialised) {
59       DriverStatus st = sensorDriverInit(drv);
60       printf("[%s] init -> %s\n", sensorDriverName(drv), driverStatusName(st));
61       if (st != DRIVER_OK) continue;
62       initialised = true;
63     }
64
65     const MonoMs t0 = monoNowMs();
66     DriverStatus st = sensorDriverStart(drv, t0);
67     if (st != DRIVER_OK) {
68       printf("[%s] start -> %s\n", sensorDriverName(drv), driverStatusName(st));
69       if (st != DRIVER_PENDING) initialised = false;
70       continue;
71     }
72
```

The task recovers its context pointer, remembers the tick grid and waits until the next release. It retries init on later periods until accepted. It does not spin in an unlimited inner init loop.

test/bringup/bringup_harness.h / lines 73-76

```
73     PollResult pr = sensorDriverPoll(drv, monoNowMs());
74     while (pr.status == DRIVER_PENDING) {
75       const MonoMs now = monoNowMs();
76       if (now - t0 >= ctx->periodMs) {  // harness-level guard, not the driver budget
```

After init, t0 captures the start time. A rejected start prints its status and usually requires reinitialisation. The special PENDING handling preserves the lifecycle convention that some already-busy conditions should not cause immediate destructive reconfiguration.

Poll with a task-level guard

test/bringup/bringup_harness.h / lines 77-99

```
77          sensorDriverCancel(drv);
78          pr.status = DRIVER_TIMEOUT;
79          break;
80        }
81        uint32_t waitMs = 1;
82        if (pr.nextPollMs > now) waitMs = (uint32_t)(pr.nextPollMs - now);
83        TickType_t ticks = pdMS_TO_TICKS(waitMs);
84        vTaskDelay(ticks > 0 ? ticks : 1);
85        pr = sensorDriverPoll(drv, monoNowMs());
86      }
87
88      const uint32_t elapsedMs = (uint32_t)(monoNowMs() - t0);
89      if (pr.status != DRIVER_READY) {
90        printf("[%s] poll -> %s after %lu ms\n", sensorDriverName(drv),
91               driverStatusName(pr.status), (unsigned long)elapsedMs);
92        if (pr.status == DRIVER_NOT_IMPLEMENTED || pr.status == DRIVER_NOT_PRESENT) {
93          initialised = false;
94        }
95        continue;
96      }
97
98      Reading out[4];
99      const uint8_t n = sensorDriverReadResult(drv, out, 4);
```

The loop checks the driver's next useful time, converts the interval to ticks and forces at least one tick when conversion would yield zero. That avoids a high-priority busy loop over a near deadline.

The harness cancels a pending operation after one configured period. This is its own outer guard, not a replacement for the driver's shorter protocol timeout. Error status is printed and the next period becomes another bounded opportunity to make progress.

Collect a copy before calling the optional observer

test/bringup/bringup_harness.h / lines 100-110

```
100     bringupPrintResult(sensorDriverName(drv), out, n,
  >   elapsedMs);
101     if (ctx->afterResult != NULL) ctx->afterResult(drv);
102   }
103 }
104
105 // Start the harness task with the AcquisitionNormal
  >   placement from §6.
106 static inline void bringupStart(BringupContext* ctx) {
107   const TaskSpec* spec =
  >   cfgTaskSpec(TASK_ID_ACQUISITION_NORMAL);
108   xTaskCreatePinnedToCore(bringupTask, "bringup",
  >   spec->stackBytes, ctx,
109                           spec->priority, NULL, spec->core);
110 }
```

A READY result is copied into a four-record local array, then printed. The callback runs afterward, when the driver has a completed outcome rather than a half-written one. The array is safe here because printing finishes before its stack lifetime ends.

bringupStart uses acquisition's placement parameters to create the test task. Notice that this test helper does not check the creation return value. If no test output appears after setup, creation failure is one of the paths to investigate; the helper does not turn failure into a usable task.

COMMON PITFALL

A bring-up helper is executable code to review, not a guarantee that every failure has already been reported well.

Define one complete test object graph

The light experiment now becomes a small setup file using the production driver and harness. Read it with a real observation plan in mind.

The test replaces production main for its build environment; it does not run alongside it.

test/bringup/light_main.c / lines 1-26

```
// Bring-up entry point for light (WP3).  pio run -e test_light -t upload -t monitor
// Retained 3.3 V rail. S -> 10 kOhm -> GPIO1 (ADC1_CH0), 1 uF to GND.
// Replaces src/main.c in this env only; links the production driver.
//
// Every line prints module mV with "(uncal)": there is no lux mapping (D13).
// Procedure (README "DFR0026 light bring-up"), note the mV for each step:
//  1. Dark: cover the sensor completely.
//  2. Room light, sensor uncovered.
//  3. Phone torch close to the lens: find where it clips (~3050 mV at the pin;
//     the reading carries kQualitySaturated).
//  4. Under LED or fluorescent light: note the spread between successive lines
//     (flicker rejection of the 11 ms tap spacing).
//  5. Unplug S: watch how the reading decays (the 1 uF holds the last level).
#include <Arduino.h>
#include <stdio.h>

#include "app_config.h"
#include "board_profile.h"
#include "bringup_harness.h"
#include "hal/adc_channel.h"
#include "drivers/light.h"

static AdcChannel s_adc = ADC_CHANNEL_INIT;
static LightConfig s_config = {&s_adc};
static LightDriver s_driver;
static BringupContext s_ctx = {&s_driver.base, CFG_FRAME_PERIOD_MS, NULL};
```

The source procedure asks you to compare darkness, room light, bright illumination and a disconnected signal. Those cases test range, clipping, flicker and diagnostic limits.

s_config points to the static ADC object. s_driver owns its own state. s_ctx points to the driver's base and supplies the ordinary frame period. The objects persist after setup returns, so the created task does not inherit dangling addresses.

Construct first, configure the channel, then create work

test/bringup/light_main.c / lines 27-40

```
27
28 void setup(void) {
29   lightDriverConstruct(&s_driver, &s_config);
30   delay(200);
31   printf("\n=== bring-up: %s ===\n", sensorDriverName(&s_driver.base));
32   if (!adcChannelBegin(&s_adc, BOARD_PIN_LIGHT_ADC)) {
33     printf("GPIO%d is not an ADC1 pin\n", (int)BOARD_PIN_LIGHT_ADC);
34   }
35   printf("%d taps, %d ms apart, trimmed mean; printing module mV (uncal)\n",
36          LIGHT_SAMPLES, LIGHT_SAMPLE_SPACING_MS);
37   bringupStart(&s_ctx);
38 }
39
40 void loop(void) { vTaskDelay(portMAX_DELAY); }
```

setup constructs the driver, waits for console visibility, configures the ADC channel and prints the chosen filter settings before starting the harness. Constructor and ADC setup solve different prerequisites.

loop blocks because the task owns repeated work. During this test, use the reported raw millivolts to establish electrical behaviour; do not expect lux until a reference mapping exists. The uncalibrated label is the intended result of this setup, not evidence that the task failed.

Follow one addressed read on the bus

I2C uses SDA for data and SCL for the clock. The ESP32 is the controller here; the sensor is the addressed target. They share the lines with other I2C devices. On this board, external pull-ups return released lines high, while a participant can pull them low.

A transfer sends eight data bits and then an acknowledgement clock. The receiver pulling SDA low on that ninth clock is ACK; leaving it high is NACK. ACK means the addressed participant acknowledged those bytes. It does not mean that a physical conversion is finished or the requested sensor model was verified.

Part of a BME280 identity read	Meaning
START, device 0x76, write direction	Select the chip to receive a register number.
Register number 0xD0	Select its identity register.
Repeated START, device 0x76, read direction	Change to reading without ending the combined transaction first.
Read one byte, then end the read	Obtain the identity value for the driver to check.

A repeated START begins another addressed phase without an intervening STOP. The HAL library generates these wire actions. The driver supplies address 0x76, register 0xD0, destination and length. In this source, the accepted BME280 identity byte is 0x60. Choosing the right address and reading the right identity are separate checks.

Now read i2cBusReadRegs in the source section. Match each parameter to the table before following the timeout and got==len checks. The bus helper obtains bytes; the BME280 driver decides whether their identity and meaning are acceptable.

Reference: NXP: I2C-bus specification UM10204, Bosch Sensortec: BME280 datasheet

REMEMBER

A device address selects a participant. A register number selects a location inside that participant.

Ask for bytes as one transaction

I2C is a shared bus addressed by device. On this board the BME280 normally uses address 0x76 and the SGP30 uses 0x58. A device address selects the chip; a register address selects a location inside that chip. Those addresses are not interchangeable.

Original firmware - src/hal/i2c_bus.c, lines 34-40

```
bool i2cBusReadRegs(I2cBus* bus, uint8_t address, uint8_t reg,
>   uint8_t* buf, size_t len) {
  if (!bus->started_ || buf == NULL || len == 0) return false;
  // Register write, repeated start, read: one bounded
>   transaction.
  size_t got = 0;
  i2cWriteReadNonStop(kI2cPort, address, &reg, 1, buf, len,
>   I2C_BUS_TIMEOUT_MS, &got);
  return got == len;  // a partial or failed read is dropped
}
```

To read registers, the HAL sends the register number and performs a repeated-start read into buf. &reg supplies the address of the register-number byte. buf already points to the caller's destination array. len is the requested byte count, and &got lets the library report how many bytes arrived.

The success check requires got==len. If you wanted eight bytes and got five, you cannot decode the missing three as if they were fresh data. A partial transaction is rejected rather than padded with whatever previously occupied the buffer.

A transaction needs a bounded timeout. Without one, an absent or stuck device could hold the acquisition worker inside a call forever. A PENDING driver does not rescue you from an unbounded HAL call hidden inside poll().

The single acquisition owner serialises the bus use. It can start a conversion on one chip, use the bus for another chip while the first converts internally, then return for the first result. Sharing elapsed conversion time is safe only when the device protocols permit it.

COMMON PITFALL

An ACK only shows that something acknowledged that address. Read the device identity before assuming the expected chip is present.

REMEMBER

The bus moves bytes. The driver knows what those bytes mean.

Trigger once, then return at the right time

Original firmware - src/drivers/bme280.c, lines 128-139

```
static DriverStatus bme280Start(SensorDriver* d, MonoMs now) {
  Bme280Driver* self = asBme280(d);
  if (!self->initialised_) return DRIVER_FAULT;
  if (!i2cBusWriteReg(self->config_.bus, self->config_.address, kRegCtrlMeas,
                      kCtrlMeasForcedX1)) {
    self->initialised_ = false;
    return DRIVER_FAULT;
  }
  self->startMs_ = now;
  self->busy_ = true;
  return DRIVER_OK;
}
```

bme280Start writes the forced-mode control register. If the write fails, it clears initialised_ and returns a fault. Otherwise it stores startMs_, marks busy_ and returns OK. There is no conversion-length sleep in this start path.

Original firmware - src/drivers/bme280.c, lines 148-154

```
  // The measuring bit may not be set yet right after the trigger, so the
  // first status check waits until the maximum conversion time has passed.
  if (now < startMs + kFirstPollMs) {
    PollResult r = {DRIVER_PENDING, startMs + kFirstPollMs};
    return r;
  }
```

The first poll is deferred until startMs + 10ms in this implementation. The source explains why: immediately after the trigger, the measuring status may not yet show the conversion in progress. Checking too early could misread not-started as already-finished.

Trigger once, then return at the right time / continued

After that delay, poll reads configuration and status. If the chip is still measuring and the 12ms conversion budget has expired, it reports TIMEOUT. Otherwise it returns PENDING with another near-future poll time. If the device has lost its expected humidity setting, it reports failure so the caller can reinitialise it.

When ready, one burst obtains all eight data bytes. The driver uses the device's coherent burst behaviour instead of making unrelated reads that could cross an update boundary. init had already read factory coefficients needed to turn raw values into physical units.

The 250ms application release period is much longer than this roughly 10ms conversion. Do not confuse sample period with conversion duration. The first controls how often you ask; the second controls how long a particular request takes.

These 10ms and 12ms choices describe the archived driver and its selected settings, not every possible BME280 configuration.

Reference: Bosch Sensortec: BME280 datasheet

Turn an integer scale into a physical value

A device can report fractions without returning a C float. It can store an integer that counts small fixed steps. If a temperature routine reports 2450 in hundredths of a degree, its value is 2450/100 = 24.50 degrees Celsius. You must know that scale before interpreting the integer.

Worked fixed-point decoding - no peripheral access

```
uint32_t pressureQ = 25939200u;
float pressurePa = (float)pressureQ / 256.0f;
// pressurePa = 101325.0
```

In Q24.8 notation, the eight says that eight stored bits represent the fractional part. One whole unit is therefore 2 to the power 8, or 256 stored counts. The 24 names the integer-bit part of this representation. Dividing by 256 recovers the physical value; shifting by eight is an integer operation that would discard the fraction.

Returned representation	Stored example	Decoded quantity
Hundredths of a degree	2450	24.50 degrees Celsius
Pressure with eight fractional bits	25939200	101325Pa
Humidity with ten fractional bits	51200	50.0 percent RH

The compensation routines also use intermediate scales to keep arithmetic in integers. You do not need to reconstruct Bosch's full transfer model to follow the call chain. You do need to preserve coefficient signedness, intermediate width, call order and each routine's returned scale. Those are the code-review questions explained beside the actual functions.

Reference: Bosch Sensortec: BME280 datasheet

COMMON PITFALL

The C type uint32_t supplies a range of integers. It does not encode the scale or the physical unit.

Unpack bits, then apply the correct scale

Worked example - BME280 20-bit register packing

```
uint8_t bytes[3] = {0x65, 0x5A, 0xC0};
int32_t raw = ((int32_t)bytes[0] << 12)
            | (bytes[1] << 4)
            | (bytes[2] >> 4);
// raw = 0x655AC = 415148
```

The first byte supplies the top eight bits. Shifting it left 12 puts them at their final positions. The next byte contributes eight more bits after a left shift of four. Only the top nibble of the third byte is used, so shifting right four discards its unused low bits. Bitwise OR joins the non-overlapping fields.

This raw integer is not degrees Celsius. The chip's factory coefficients correct its transfer characteristics. compensateT() runs first because it also writes tFine_, an intermediate value needed by pressure and humidity compensation. Reordering those calls could use an old intermediate even if every function individually compiles.

The returned integer scales also differ. A temperature result of 2450 means 24.50 degrees Celsius because the driver divides by 100. A Q24.8 pressure value has eight fractional bits, so divide by 256. A humidity result with ten fractional bits is divided by 1024. These scale factors follow the selected compensation formulas, not a general I2C rule.

One completed conversion can therefore produce three Reading records with different metrics and units. Acquisition stores them at their metric indices in Snapshot. The SGP30 later uses the temperature and humidity as dependency inputs; we will explain that connection in its own chapter.

Reference: Bosch Sensortec: BME280 datasheet

COMMON PITFALL

Do not insert /1000 because a raw number looks large. Follow the documented scale from the conversion routine to the published unit.

Keep a bus configuration distinct from a device address

This object records how the shared I2C controller was configured. It exposes bounded transactions rather than the entire Arduino Wire object.

BME280 and SGP30 use the same I2cBus instance under the acquisition owner.

src/hal/i2c_bus.h / lines 1-24

```
// I2C bus adapter (HAL). Work package WP1 (with BME280).
// Single owner: only the acquisition lease holder calls it (§9).
// Every transaction must have a bounded timeout; i2cBusRecover() performs a
// limited stuck-bus recovery (§16), never a blocking retry loop.
//
// Worst-case blocking: I2C_BUS_TIMEOUT_MS per transaction (probe/writeReg/
// readRegs/write/read); recover() ~0.2 ms of bit-banging plus one begin().
#pragma once
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

#define I2C_BUS_TIMEOUT_MS ((uint16_t)5)

// Treat the members as private to i2c_bus.c. Initialise with I2C_BUS_INIT.
typedef struct I2cBus {
  bool started_;
  int8_t sda_;
  int8_t scl_;
  uint32_t clockHz_;
} I2cBus;

#define I2C_BUS_INIT {false, -1, -1, 0}

```

I2cBus retains pins, clock and started state. Each operation also receives a device address because a shared bus can serve several chips. Register reads and arbitrary command reads are different helpers on that same bus.

COMMON PITFALL

Do not use an address-only ACK as proof of device identity. The driver must check the expected chip or feature identifier.

Keep a bus configuration distinct from a device address / continued

src/hal/i2c_bus.h / lines 25-37

```
25 bool i2cBusBegin(I2cBus* bus, int8_t sda, int8_t scl, uint32_t clockHz);
26 bool i2cBusProbe(I2cBus* bus, uint8_t address);
27 bool i2cBusWriteReg(I2cBus* bus, uint8_t address, uint8_t reg, uint8_t value);
28 bool i2cBusReadRegs(I2cBus* bus, uint8_t address, uint8_t reg, uint8_t* buf, size_t
 >   len);
29 // Raw transfers for command-based devices (SGP30): each is one transaction
30 // ending in STOP, so a device that NACKs while busy is never held by a
31 // repeated start. A read returns true only if all `len` bytes arrived.
32 bool i2cBusWrite(I2cBus* bus, uint8_t address, const uint8_t* tx, size_t len);
33 bool i2cBusRead(I2cBus* bus, uint8_t address, uint8_t* buf, size_t len);
34 // Call after a failed transaction. Does nothing (returns true) when SDA is
35 // high, i.e. the failure was only a NACK; recovers only a bus held low.
36 bool i2cBusRecover(I2cBus* bus);
37 void i2cBusEnd(I2cBus* bus);
```

The timeout belongs to a transaction. The driver's conversion deadline is a different limit above this adapter. An I2C read can fail quickly while a correctly requested chip conversion legitimately remains pending.

Start or reuse the configured controller

Read the exact peripheral calls hidden by the bus interface. This is where a logical request becomes bytes on the configured controller.

One acquisition owner prevents two protocol sequences from interleaving on these helpers.

src/hal/i2c_bus.c / lines 1-20

```
#include "hal/i2c_bus.h"

#include <Arduino.h>
#include <driver/gpio.h>
#include <esp32-hal-i2c.h>

// Controller 0 (what the C++ version reached through the Arduino `Wire`
// object). Calls go straight to the core's C I2C layer, with the same
// per-transaction timeout Wire.setTimeOut() used to apply.
static const uint8_t kI2cPort = 0;

bool i2cBusBegin(I2cBus* bus, int8_t sda, int8_t scl, uint32_t clockHz) {
  if (bus->started_) i2cBusEnd(bus);
  bus->sda_ = sda;
  bus->scl_ = scl;
  bus->clockHz_ = clockHz;
  // As Wire.begin(): a controller that is already running counts as started.
  bus->started_ = i2cIsInit(kI2cPort) || i2cInit(kI2cPort, sda, scl, clockHz) == ESP_OK;
  return bus->started_;
}
```

The adapter selects controller 0. If its own object is marked started, it ends that session before changing pins or speed. It records the requested setup and accepts a controller already initialised by the core or a successful new i2cInit call.

The stored bus state is a software description; it is not a hardware mutex. A second independently configured object targeting controller 0 could still disturb the same peripheral. The application's one-owner rule is part of the safe-use contract.

Keep the register pointer and read together

src/hal/i2c_bus.c / lines 21-40

```
21
22 bool i2cBusProbe(I2cBus* bus, uint8_t address) {
23   if (!bus->started_) return false;
24   // Address-only write: ACK => present.
25   return i2cWrite(kI2cPort, address, NULL, 0,
 >   I2C_BUS_TIMEOUT_MS) == ESP_OK;
26 }
27
28 bool i2cBusWriteReg(I2cBus* bus, uint8_t address, uint8_t
 >   reg, uint8_t value) {
29   if (!bus->started_) return false;
30   const uint8_t tx[2] = {reg, value};
31   return i2cWrite(kI2cPort, address, tx, sizeof(tx),
 >   I2C_BUS_TIMEOUT_MS) == ESP_OK;
32 }
33
34 bool i2cBusReadRegs(I2cBus* bus, uint8_t address, uint8_t
 >   reg, uint8_t* buf, size_t len) {
35   if (!bus->started_ || buf == NULL || len == 0) return
 >   false;
36   // Register write, repeated start, read: one bounded
 >   transaction.
37   size_t got = 0;
38   i2cWriteReadNonStop(kI2cPort, address, &reg, 1, buf, len,
 >   I2C_BUS_TIMEOUT_MS, &got);
39   return got == len;  // a partial or failed read is dropped
40 }
```

Probe performs an address-only write. WriteReg sends two bytes: register address and value. ReadRegs uses the repeated-start helper to set the register pointer and obtain the requested bytes as one bounded transaction.

ReadRegs checks the number actually received. It does not accept a partially filled buffer. A caller should therefore branch on the returned boolean before unpacking any register fields.

COMMON PITFALL

The array size passed to sizeof must describe the actual source or destination object, not the size of a pointer parameter.

Support command-oriented devices without inventing registers

src/hal/i2c_bus.c / lines 41-52

```
41
42 bool i2cBusWrite(I2cBus* bus, uint8_t address, const uint8_t* tx, size_t len) {
43   if (!bus->started_ || tx == NULL || len == 0) return false;
44   return i2cWrite(kI2cPort, address, tx, len, I2C_BUS_TIMEOUT_MS) == ESP_OK;
45 }
46
47 bool i2cBusRead(I2cBus* bus, uint8_t address, uint8_t* buf, size_t len) {
48   if (!bus->started_ || buf == NULL || len == 0) return false;
49   size_t got = 0;
50   i2cRead(kI2cPort, address, buf, len, I2C_BUS_TIMEOUT_MS, &got);
51   return got == len;  // a partial or failed read is dropped
52 }
```

The arbitrary write and read helpers pass a byte buffer and length directly to the core. SGP30 uses such command sequences rather than BME280-style register addresses. Both reject an unstarted bus or an unusable buffer/length combination.

The destination may still contain old bytes after a failed call. Success is the permission to decode, not the presence of nonzero memory. This is the same output-contract discipline used for queues and analog reads.

Recover only when the physical line needs it

src/hal/i2c_bus.c / lines 53-74

```
53
54 // Release a slave holding SDA low: up to 9 SCL pulses, then a STOP.
55 // Bounded (~0.2 ms); no retry loop.
56 bool i2cBusRecover(I2cBus* bus) {
57   const int8_t sda = bus->sda_;
58   const int8_t scl = bus->scl_;
59   if (sda < 0 || scl < 0) return false;
60   // An idle bus has SDA high: the failure was a NACK (e.g. a device that is
61   // absent or busy), not a stuck bus. Leave the controller running so the
62   // other device on the bus is not disturbed (audit L5).
63   if (bus->started_ && gpio_get_level((gpio_num_t)sda) != 0) return true;
64   if (bus->started_) i2cBusEnd(bus);
65
66   pinMode(sda, INPUT_PULLUP);
67   pinMode(scl, OUTPUT_OPEN_DRAIN);
68   digitalWrite(scl, HIGH);
69   delayMicroseconds(5);
70   for (uint8_t i = 0; i < 9 && digitalRead(sda) == LOW; ++i) {
71     digitalWrite(scl, LOW);
72     delayMicroseconds(5);
73     digitalWrite(scl, HIGH);
74     delayMicroseconds(5);
```

Recovery first reads SDA. If the controller is running and SDA is already high, the source treats the failure as something such as a NACK and leaves the controller running. It avoids resetting the whole shared bus for every absent device.

If SDA is stuck, the adapter ends the controller and manually generates up to nine clock pulses, checking whether the device releases the line. The loop is bounded. Another device on the same bus can be affected, so this belongs to the shared owner rather than a random radio callback.

Generate a stop and rebuild the peripheral state

src/hal/i2c_bus.c / lines 75-95

```
  }

  // STOP: SDA low -> SCL high -> SDA high.
  pinMode(sda, OUTPUT_OPEN_DRAIN);
  digitalWrite(scl, LOW);
  digitalWrite(sda, LOW);
  delayMicroseconds(5);
  digitalWrite(scl, HIGH);
  delayMicroseconds(5);
  digitalWrite(sda, HIGH);
  delayMicroseconds(5);

  pinMode(sda, INPUT_PULLUP);
  const bool sdaReleased = digitalRead(sda) == HIGH;
  return i2cBusBegin(bus, sda, scl, bus->clockHz_) && sdaReleased;
}

void i2cBusEnd(I2cBus* bus) {
  if (bus->started_ && i2cIsInit(kI2cPort)) i2cDeinit(kI2cPort);
  bus->started_ = false;
}
```

The manual sequence brings SDA from low to high while SCL is high, then restores SDA as an input and calls Begin again. The return combines successful restart with the observed line release.

End shuts down the selected controller when the object says it was started, then clears the software flag. Recovery cannot guarantee that an electrically damaged or miswired bus becomes usable; the boolean lets the driver report that the attempt did not restore its required condition.

Read the selected conversion policy

The BME280 needs factory coefficients as well as conversion state. This header gives those persistent values named fields instead of leaving them in unexplained byte arrays.

One driver publishes three Reading records from one conversion.

src/drivers/bme280.h / lines 1-28

```
// bme280 driver. Work package WP1 (Stage 2). Architecture §4, §7, §14.
// Datasheet: Bosch BST-BME280-DS001 rev 1.24.
//
// Verify chip ID 0x60 (rejects BMP280); read compensation coefficients once
// in init(). Forced mode, x1 oversampling T/P/H, IIR off; Bosch integer
// compensation (§4.2.3). Produces AirTemperature, AirHumidity, AirPressure.
//
// Worst-case blocking (I2C_BUS_TIMEOUT_MS = T per transaction):
//   init()    ~13 ms + 16 T  (soft reset + bounded NVM-copy wait)
//   start()   1 T            (one register write)
//   poll()    2 T            (ctrl_hum/status + one 8-byte burst); Pending
//                            until start+10 ms (t_max 9.3 ms), Timeout
//                            after CFG_BME280_BUDGET_MS
//   others    no bus access
// Any bus fault, timeout or invalid sample clears the initialised state, so
// the next start() returns Fault and the caller re-runs init(). poll() also
// detects a module power-cycle (ctrl_hum back to reset 0): it fails the
// sample and clears the initialised state; the caller then re-runs init().
//
// Implements the SensorDriver contract in drivers/sensor_driver.h.
#pragma once
#include "drivers/sensor_driver.h"
#include "hal/i2c_bus.h"

typedef struct Bme280Config {
  I2cBus* bus;
  uint8_t address;  // BOARD_BME280_I2C_ADDRESS
} Bme280Config;
```

The contract describes forced conversions, three result metrics and the intended blocking limits. A forced request lets acquisition choose release times rather than consuming an independently cycling stream whose phase is unknown.

The I2C configuration contains a bus pointer and device address. Moving the address does not require changing the compensation mathematics. This is the separation between locating a device and interpreting its result.

Keep coefficients, intermediate state and results separate

src/drivers/bme280.h / lines 29-55

```
29
30 #define BME280_METRICS 3  // results_ index: 0 T, 1 H, 2 P
31
32 typedef struct Bme280Calib {
33   uint16_t t1;
34   int16_t t2, t3;
35   uint16_t p1;
36   int16_t p2, p3, p4, p5, p6, p7, p8, p9;
37   uint8_t h1, h3;
38   int16_t h2, h4, h5;
39   int8_t h6;
40 } Bme280Calib;
41
42 // Members after `base` are private to bme280.c.
43 typedef struct Bme280Driver {
44   SensorDriver base;  // must stay first
45   Bme280Config config_;
46   Bme280Calib calib_;
47   int32_t tFine_;
48   MonoMs startMs_;
49   bool initialised_;
50   bool busy_;
51   Reading results_[BME280_METRICS];
52 } Bme280Driver;
53
54 // Constructor. Use &driver.base wherever a SensorDriver* is expected.
55 void bme280DriverConstruct(Bme280Driver* self, const Bme280Config* config);
```

Bme280Calib distinguishes signed and unsigned coefficient fields. Bme280Driver stores that calibration, tFine_, a start timestamp, lifecycle flags and three retained results. The fine-temperature intermediate is shared by the pressure and humidity formulas.

The constructor receives configuration and installs a generic base. It does not discover a new bus on every sample. Read the implementation next with these storage groups in mind: device setup, one operation and completed observations.

Name each register before using it

We will follow identification, reset, coefficient loading, one forced measurement and every compensation helper. You do not need to memorise the coefficient formulas; you do need to understand their inputs, order, scale and failure checks.

Keep the BME280 datasheet open for register definitions when checking a packed field.

src/drivers/bme280.c / lines 1-27

```
#include "drivers/bme280.h"

#include <Arduino.h>
#include <string.h>

#include "app_config.h"

// Registers (datasheet §5.3).
static const uint8_t kRegCalib00 = 0x88;  // ..0xA1, 26 bytes
static const uint8_t kRegChipId = 0xD0;
static const uint8_t kRegReset = 0xE0;
static const uint8_t kRegCalib26 = 0xE1;  // ..0xE7, 7 bytes
static const uint8_t kRegCtrlHum = 0xF2;
static const uint8_t kRegStatus = 0xF3;
static const uint8_t kRegCtrlMeas = 0xF4;
static const uint8_t kRegConfig = 0xF5;
static const uint8_t kRegData = 0xF7;     // ..0xFE, 8 bytes (P, T, H)

static const uint8_t kChipId = 0x60;
static const uint8_t kResetCmd = 0xB6;
static const uint8_t kStatusMeasuring = 1u << 3;
static const uint8_t kStatusImUpdate = 1u << 0;

static const uint8_t kCtrlHumX1 = 0x01;     // osrs_h = x1
static const uint8_t kConfigNoFilter = 0x00;  // IIR off, t_sb unused in forced mode
static const uint8_t kCtrlMeasForcedX1 = (1u << 5) | (1u << 2) | 0x01;  // 0x25

```

The constants name register locations, expected identity, control bits and ranges. kCtrlMeasForcedX1 combines three bit fields with OR; it is not three sequential commands. u16le and s16le join little-endian calibration bytes with different signed interpretations.

src/drivers/bme280.c / lines 28-34

```
static const uint32_t kFirstPollMs = 10;     // > t_max 9.3 ms at x1/x1/x1 (§9.1)
static const uint8_t kResetWaitMaxMs = 10;   // bounded NVM-copy wait in init()

// Operating range (§1): -40..85 degC, 0..100 %RH, 300..1100 hPa.
static const int32_t kTminCenti = -4000, kTmaxCenti = 8500;
static const float kPminPa = 30000.0f, kPmaxPa = 110000.0f;

```

Name each register before using it / continued

src/drivers/bme280.c / lines 35-48

```
35 static inline uint16_t u16le(const uint8_t* b) { return (uint16_t)(b[0] | (b[1] << 8));
 >   }
36 static inline int16_t s16le(const uint8_t* b) { return (int16_t)u16le(b); }
37
38 static inline Bme280Driver* asBme280(SensorDriver* d) { return (Bme280Driver*)d; }
39
40 static bool readCalibration(Bme280Driver* self);
41 static PollResult fail(Bme280Driver* self, DriverStatus status, Health health, MonoMs
 >   now);
42 static int32_t compensateT(Bme280Driver* self, int32_t adcT);   // 0.01 degC, sets
 >   tFine_
43 static uint32_t compensateP(Bme280Driver* self, int32_t adcP);  // Q24.8 Pa, 0 = invalid
44 static uint32_t compensateH(Bme280Driver* self, int32_t adcH);  // Q22.10 %RH
45
46 static SensorId bme280Id(const SensorDriver* d) { (void)d; return SENSOR_BME280; }
47 static const char* bme280Name(const SensorDriver* d) { (void)d; return "bme280"; }
48 static uint8_t bme280MetricCount(const SensorDriver* d) { (void)d; return
 >   BME280_METRICS; }
```

The forward declarations let later helper bodies be called before their definitions appear. They do not execute those bodies. The identity helpers then supply the fixed generic-driver metadata.

Identify the chip before accepting its state

src/drivers/bme280.c / lines 49-74

```
49
50 static DriverStatus bme280Init(SensorDriver* d) {
51   Bme280Driver* self = asBme280(d);
52   self->initialised_ = false;
53   self->busy_ = false;
54   I2cBus* bus = self->config_.bus;
55   const uint8_t address = self->config_.address;
56
57   uint8_t id = 0;
58   if (!i2cBusReadRegs(bus, address, kRegChipId, &id, 1)) {
59     i2cBusRecover(bus);
60     return DRIVER_FAULT;
61   }
62   if (id != kChipId) return DRIVER_NOT_PRESENT;
63
64   if (!i2cBusWriteReg(bus, address, kRegReset, kResetCmd)) return DRIVER_FAULT;
65   // t_startup <= 2 ms (Table 1). delay(n) may return up to 1 tick early, so
66   // a failed read is "not ready yet" until the bounded wait runs out.
67   delay(3);
68   bool ready = false;
69   for (uint8_t i = 0; i < kResetWaitMaxMs && !ready; ++i) {
70     uint8_t status = kStatusImUpdate;
71     ready = i2cBusReadRegs(bus, address, kRegStatus, &status, 1) &&
72             (status & kStatusImUpdate) == 0;
73     if (!ready) delay(1);
74   }
```

Init clears lifecycle flags and reads the identity register. A transport failure and the wrong chip identity return different statuses. A correct address ACK without the expected identity is insufficient.

The reset write is followed by a bounded wait for the NVM-copy status to clear. The chip may still be starting even if an early read fails. The loop gives it limited opportunities to become ready; it does not retry forever inside acquisition.

Verify configuration before setting initialised

src/drivers/bme280.c / lines 75-94

```
  if (!ready) return DRIVER_TIMEOUT;

  if (!readCalibration(self)) return DRIVER_FAULT;

  // Sleep mode after reset, so config writes are accepted (§5.4.6).
  // ctrl_hum takes effect on the ctrl_meas write in start() (§5.4.3).
  uint8_t check[2] = {0, 0};
  if (!i2cBusWriteReg(bus, address, kRegCtrlHum, kCtrlHumX1) ||
      !i2cBusWriteReg(bus, address, kRegConfig, kConfigNoFilter) ||
      !i2cBusReadRegs(bus, address, kRegCtrlHum, &check[0], 1) ||
      !i2cBusReadRegs(bus, address, kRegConfig, &check[1], 1)) {
    return DRIVER_FAULT;
  }
  if ((check[0] & 0x07) != kCtrlHumX1 || (check[1] & 0xFC) != kConfigNoFilter) {
    return DRIVER_FAULT;
  }

  self->initialised_ = true;
  return DRIVER_OK;
}
```

After coefficient loading, init writes humidity oversampling and the selected configuration, then reads them back. Masking compares only the relevant bits. A failed write, read or mismatch returns before initialised_ becomes true.

That final true assignment is the commit point for device setup. The next start depends on it. Do not move it ahead of the readback merely because the earlier writes returned success.

Decode factory coefficients with the right signedness

src/drivers/bme280.c / lines 95-122

```
95
96  static bool readCalibration(Bme280Driver* self) {
97    uint8_t a[26];
98    uint8_t b[7];
99    if (!i2cBusReadRegs(self->config_.bus, self->config_.address, kRegCalib00, a,
 >    sizeof(a)) ||
100       !i2cBusReadRegs(self->config_.bus, self->config_.address, kRegCalib26, b,
 >    sizeof(b))) {
101     return false;
102   }
103   Bme280Calib* c = &self->calib_;
104   // Table 16. a[0] = 0x88, b[0] = 0xE1.
105   c->t1 = u16le(&a[0]);
106   c->t2 = s16le(&a[2]);
107   c->t3 = s16le(&a[4]);
108   c->p1 = u16le(&a[6]);
109   c->p2 = s16le(&a[8]);
110   c->p3 = s16le(&a[10]);
111   c->p4 = s16le(&a[12]);
112   c->p5 = s16le(&a[14]);
113   c->p6 = s16le(&a[16]);
114   c->p7 = s16le(&a[18]);
115   c->p8 = s16le(&a[20]);
116   c->p9 = s16le(&a[22]);
117   c->h1 = a[25];                                  // 0xA1
118   c->h2 = s16le(&b[0]);                           // 0xE1/0xE2
119   c->h3 = b[2];                                   // 0xE3
120   c->h4 = (int16_t)(((int8_t)b[3] * 16) | (b[4] & 0x0F));  // E4[11:4] E5[3:0]
121   c->h5 = (int16_t)(((int8_t)b[5] * 16) | (b[4] >> 4));    // E6[11:4] E5[7:4]
122   c->h6 = (int8_t)b[6];                           // 0xE7
```

Two burst reads cover the coefficient regions. The code then maps byte offsets to named coefficient fields. Most use 16-bit little-endian helpers, but h4 and h5 span nibbles across adjacent bytes and need explicit signed handling.

src/drivers/bme280.c / lines 123-126

```
123
124   // Blank NVM or a bus returning 0x00/0xFF would make these 0/0xFFFF.
125   return c->t1 != 0 && c->p1 != 0 && c->t1 != 0xFFFF && c->p1 != 0xFFFF;
126 }
```

For h4, the signed high fragment is multiplied by 16 before the low nibble is ORed in. This avoids shifting a negative signed value left. The final nonzero/non-all-ones checks reject obvious blank or disconnected data, although they do not prove every possible corrupt coefficient set is detected.

Request one conversion without occupying its duration

src/drivers/bme280.c / lines 127-139

```
127
128 static DriverStatus bme280Start(SensorDriver* d, MonoMs now) {
129   Bme280Driver* self = asBme280(d);
130   if (!self->initialised_) return DRIVER_FAULT;
131   if (!i2cBusWriteReg(self->config_.bus, self->config_.address, kRegCtrlMeas,
132                       kCtrlMeasForcedX1)) {
133     self->initialised_ = false;
134     return DRIVER_FAULT;
135   }
136   self->startMs_ = now;
137   self->busy_ = true;
138   return DRIVER_OK;
139 }
```

Start rejects a driver that has not completed init. It writes the control value, clears initialisation on a write failure, then records startMs_ and busy_ only after the request succeeded.

The return does not contain temperature. Its job is to establish an in-progress operation whose next poll can compare now with the stored start. The bus is then free for another permitted transaction.

Check conversion progress after the initial guard

src/drivers/bme280.c / lines 140-162

```
140
141 static PollResult bme280Poll(SensorDriver* d, MonoMs now) {
142   Bme280Driver* self = asBme280(d);
143   I2cBus* bus = self->config_.bus;
144   const uint8_t address = self->config_.address;
145   const MonoMs startMs = self->startMs_;
146
147   if (!self->busy_) return fail(self, DRIVER_FAULT, HEALTH_FAULT, now);
148
149   // The measuring bit may not be set yet right after the trigger, so the
150   // first status check waits until the maximum conversion time has passed.
151   if (now < startMs + kFirstPollMs) {
152     PollResult r = {DRIVER_PENDING, startMs + kFirstPollMs};
153     return r;
154   }
155
156   // ctrl_hum + status in one burst. ctrl_hum back at its reset value means the
157   // module was power-cycled and lost its settings: fail, so the caller re-inits.
158   uint8_t cs[2] = {0, 0};
159   if (!i2cBusReadRegs(bus, address, kRegCtrlHum, cs, sizeof(cs))) {
160     i2cBusRecover(bus);
161     return fail(self, DRIVER_FAULT, HEALTH_FAULT, now);
162   }
```

Poll recovers the concrete object and rejects a call without busy_. Before the first useful time it returns PENDING without reading status. After that guard, it reads expected humidity setup and measuring state together.

Check conversion progress after the initial guard / continued

src/drivers/bme280.c / lines 163-176

```
163   if ((cs[0] & 0x07) != kCtrlHumX1) return fail(self, DRIVER_FAULT, HEALTH_FAULT, now);
164   const uint8_t status = cs[1];
165   if (status & kStatusMeasuring) {
166     if (now - startMs >= CFG_BME280_BUDGET_MS) {
167       return fail(self, DRIVER_TIMEOUT, HEALTH_TIMEOUT, now);
168     }
169     PollResult r = {DRIVER_PENDING, now + 1};
170     return r;
171   }
172
173   // One burst read: shadowing keeps P/T/H consistent only within it (§4.1).
174   uint8_t dat[8];
175   if (!i2cBusReadRegs(bus, address, kRegData, dat, sizeof(dat))) {
176     i2cBusRecover(bus);
```

A configuration reset is treated as failure, while a still-busy chip gets another poll until its conversion budget is exhausted. This distinguishes lost setup, failed transport and an operation that simply has not completed yet.

Read one coherent raw block and preserve bit positions

src/drivers/bme280.c / lines 177-203

```
    return fail(self, DRIVER_FAULT, HEALTH_FAULT, now);
  }
  self->busy_ = false;

  const int32_t adcP = ((int32_t)dat[0] << 12) | (dat[1] << 4) | (dat[2] >> 4);
  const int32_t adcT = ((int32_t)dat[3] << 12) | (dat[4] << 4) | (dat[5] >> 4);
  const int32_t adcH = ((int32_t)dat[6] << 8) | dat[7];

  // All-ones means a floating bus (impossible as real T/P data: at x1 the low
  // 4 bits are 0). 0x80000/0x8000 are NOT checked: at x1 they are valid
  // mid-scale readings (lost settings are detected via ctrl_hum above).
  if (adcT == 0xFFFFF || adcP == 0xFFFFF || adcH == 0xFFFF) {
    return fail(self, DRIVER_FAULT, HEALTH_FAULT, now);
  }

  // Temperature first: it provides tFine_ for P and H.
  const int32_t tCenti = compensateT(self, adcT);
  if (tCenti < kTminCenti || tCenti > kTmaxCenti) {
    return fail(self, DRIVER_FAULT, HEALTH_FAULT, now);
  }
  const uint32_t pQ8 = compensateP(self, adcP);
  const uint32_t hQ10 = compensateH(self, adcH);

  const float values[BME280_METRICS] = {
      (float)tCenti / 100.0f,
      (float)hQ10 / 1024.0f,
      (float)pQ8 / 256.0f,
```

The burst fills eight bytes. The shifts reconstruct 20-bit pressure and temperature and 16-bit humidity. The all-ones guard rejects an obvious invalid response. Temperature compensation runs first because it updates tFine_.

The pressure and humidity routines then use that same intermediate. Call order is part of the algorithm. A compiler accepts reversed calls, but pressure and humidity could use stale state if you reorder them.

Turn scaled integers into three independent verdicts

src/drivers/bme280.c / lines 204-231

```
  };
  const int32_t raws[BME280_METRICS] = {adcT, adcH, adcP};
  const bool valid[BME280_METRICS] = {
      true,
      true,  // compensateH clamps to 0..100 %RH
      pQ8 != 0 && values[2] >= kPminPa && values[2] <= kPmaxPa,
  };

  for (uint8_t i = 0; i < BME280_METRICS; ++i) {
    Reading* r = &self->results_[i];
    r->intervalStartMs = startMs;
    r->intervalEndMs = now;
    r->hasRaw = true;
    r->raw = raws[i];
    r->qualityFlags = kQualityNone;
    if (valid[i]) {
      r->health = HEALTH_OK;
      r->value = values[i];
      r->lastGoodMs = now;
      ++r->sequence;
    } else {
      r->health = HEALTH_FAULT;
    }
  }

  PollResult r = {DRIVER_READY, now};
  return r;
}
```

Temperature uses hundredths of a degree, pressure uses eight fractional bits and humidity uses ten. The float conversions divide by 100, 256 and 1024 respectively. The values array and raw array are ordered to match the three result identities established by construction.

Each result gets interval and raw data. Accepted metrics update value and lastGoodMs; a rejected pressure can carry FAULT even when the same conversion's temperature is acceptable. READY says the set of outcomes is available, not that every member passed.

Keep failure and temperature arithmetic local

src/drivers/bme280.c / lines 232-256

```
232
233 // Abort the measurement, mark every metric with `health` and force a re-init
234 // on the next start(). The last value is kept but never reported as Ok.
235 static PollResult fail(Bme280Driver* self, DriverStatus status, Health health, MonoMs
  >   now) {
236   self->busy_ = false;
237   self->initialised_ = false;
238   for (uint8_t i = 0; i < BME280_METRICS; ++i) {
239     self->results_[i].health = health;
240     self->results_[i].intervalStartMs = self->startMs_;
241     self->results_[i].intervalEndMs = now;
242   }
243   PollResult r = {status, now};
244   return r;
245 }
246
247 // Datasheet §4.2.3 (rev 1.1 reference code), fixed-point.
248 static int32_t compensateT(Bme280Driver* self, int32_t adcT) {
249   const Bme280Calib* c = &self->calib_;
250   const int32_t t1 = c->t1;
251   int32_t var1 = ((((adcT >> 3) - (t1 << 1))) * (int32_t)c->t2) >> 11;
252   int32_t var2 = (((((adcT >> 4) - t1) * ((adcT >> 4) - t1)) >> 12) *
253                   (int32_t)c->t3) >> 14;
254   self->tFine_ = var1 + var2;
255   return (self->tFine_ * 5 + 128) >> 8;
256 }
```

fail ends the operation, requires reinitialisation and marks all three result intervals with the requested health. It returns a PollResult consistent with that operation failure.

compensateT uses factory coefficients to obtain tFine_ and a centi-degree result. The casts and shifts preserve the fixed-point scale expected by the reference formula. The shift by 8 near the return divides by 256 at that stage; it does not mean the final value is already degrees Celsius.

Use wide intermediates before multiplication

src/drivers/bme280.c / lines 257-273

```
257
258 static uint32_t compensateP(Bme280Driver* self, int32_t adcP) {
259   const Bme280Calib* c = &self->calib_;
260   int64_t var1 = (int64_t)self->tFine_ - 128000;
261   int64_t var2 = var1 * var1 * c->p6;
262   var2 = var2 + ((var1 * c->p5) * 131072);              // << 17
263   var2 = var2 + ((int64_t)c->p4 * 34359738368LL);       // << 35
264   var1 = ((var1 * var1 * c->p3) >> 8) + ((var1 * c->p2) * 4096);  // << 12
265   var1 = (((int64_t)1 << 47) + var1) * c->p1 >> 33;
266   if (var1 == 0) return 0;  // avoid division by zero
267   int64_t p = 1048576 - adcP;
268   p = (((p << 31) - var2) * 3125) / var1;
269   var1 = ((int64_t)c->p9 * (p >> 13) * (p >> 13)) >> 25;
270   var2 = ((int64_t)c->p8 * p) >> 19;
271   p = ((p + var1 + var2) >> 8) + ((int64_t)c->p7 * 16);  // << 4; p7 is signed
272   return (uint32_t)p;
273 }
```

Pressure compensation uses int64_t intermediates because coefficient products exceed a comfortable 32-bit range. The conversion to a wider type occurs before the large arithmetic. A cast after overflow would not repair the lost value.

var1 is checked before division. Returning zero marks the pressure result invalid in the caller. The return scale is Q24.8 pascals, so the caller divides by 256. Follow that agreement rather than guessing a decimal scale from the size of the integer.

Clamp the humidity representation deliberately

src/drivers/bme280.c / lines 274-288

```
274
275 static uint32_t compensateH(Bme280Driver* self, int32_t adcH) {
276   const Bme280Calib* c = &self->calib_;
277   int32_t v = self->tFine_ - 76800;
278   // h4 is signed: multiply, not << 20 (a negative left shift is undefined in C).
279   v = (((((adcH << 14) - ((int32_t)c->h4 * 1048576) -
280           ((int32_t)c->h5 * v)) + 16384) >> 15) *
281        (((((((v * (int32_t)c->h6) >> 10) *
282             (((v * (int32_t)c->h3) >> 11) + 32768)) >> 10) + 2097152) *
283              (int32_t)c->h2 + 8192) >> 14));
284   v = v - (((((v >> 15) * (v >> 15)) >> 7) * (int32_t)c->h1) >> 4);
285   v = v < 0 ? 0 : v;
286   v = v > 419430400 ? 419430400 : v;
287   return (uint32_t)(v >> 12);
288 }
```

The humidity expression also depends on tFine_. Signed coefficient h4 is multiplied rather than shifted left. The lower and upper clamps constrain the fixed-point humidity representation before its final shift.

This dense arithmetic is a good place to preserve a known reference implementation and test known vectors. Simplifying parentheses without following every intermediate scale can silently change the function even when it still compiles. The driver later divides the returned Q-format integer by 1024.

Copy only the capacity the caller supplied

src/drivers/bme280.c / lines 289-306

```
289
290 static uint8_t bme280ReadResult(SensorDriver* d, Reading* out, uint8_t maxOut) {
291   Bme280Driver* self = asBme280(d);
292   if (out == NULL) return 0;
293   const uint8_t n = maxOut < BME280_METRICS ? maxOut : BME280_METRICS;
294   for (uint8_t i = 0; i < n; ++i) out[i] = self->results_[i];
295   return n;
296 }
297
298 // A forced conversion finishes by itself and the chip returns to sleep.
299 static void bme280Cancel(SensorDriver* d) { asBme280(d)->busy_ = false; }
300
301 // The 3.3 V rail stays up in low power (§3); just require a fresh init().
302 static void bme280PreparePowerOff(SensorDriver* d) {
303   Bme280Driver* self = asBme280(d);
304   self->busy_ = false;
305   self->initialised_ = false;
306 }
```

ReadResult chooses the smaller of maxOut and the three metrics, copies that many records and returns the count. A null destination returns zero. The caller must iterate only over the returned count.

Cancel clears busy_. PreparePowerOff also requires a later init. The physical forced conversion may finish internally; cancellation means the software no longer treats its eventual bytes as this accepted pending operation.

Construct identity once and preserve result history

src/drivers/bme280.c / lines 307-332

```
307
308 static const SensorDriverOps kBme280Ops = {
309     .id = bme280Id,
310     .name = bme280Name,
311     .metricCount = bme280MetricCount,
312     .init = bme280Init,
313     .start = bme280Start,
314     .poll = bme280Poll,
315     .readResult = bme280ReadResult,
316     .cancel = bme280Cancel,
317     .preparePowerOff = bme280PreparePowerOff,
318 };
319
320 // Readings are reset once here so sequence/lastGoodMs survive re-inits.
321 void bme280DriverConstruct(Bme280Driver* self, const Bme280Config* config) {
322   self->base.ops = &kBme280Ops;
323   self->config_ = *config;
324   memset(&self->calib_, 0, sizeof(self->calib_));
325   self->tFine_ = 0;
326   self->startMs_ = 0;
327   self->initialised_ = false;
328   self->busy_ = false;
329   readingReset(&self->results_[0], SENSOR_BME280, METRIC_AIR_TEMPERATURE, UNIT_DEG_C);
330   readingReset(&self->results_[1], SENSOR_BME280, METRIC_AIR_HUMIDITY, UNIT_PERCENT_RH);
331   readingReset(&self->results_[2], SENSOR_BME280, METRIC_AIR_PRESSURE, UNIT_PASCAL);
332 }
```

The operations table links each lifecycle function. Construction installs it, copies the bus configuration, clears coefficient and execution state, and gives the three results their distinct identities and units.

Results are reset at construction rather than every init so their retained history can survive recovery attempts. Acquisition later supplies its own published observation sequence, including failures, without asking this driver to know about queues.

Prepare persistent objects and identify likely variants

Use this entry point to distinguish bus addressing, device identity and a complete compensated reading. Each is observable separately.

The generic harness supplies the repeated start/poll/read loop you already read.

test/bringup/bme280_main.c / lines 1-22

```
// Bring-up entry point for bme280 (WP1).  pio run -e test_bme280 -t upload -t monitor
// 3.3 V rail. Expect 3 readings every 250 ms, took <= 12 ms.
// Replaces src/main.c in this env only; links the production driver.
#include <Arduino.h>
#include <stdio.h>

#include "app_config.h"
#include "board_profile.h"
#include "bringup_harness.h"
#include "hal/i2c_bus.h"
#include "drivers/bme280.h"

static I2cBus s_i2c = I2C_BUS_INIT;
static Bme280Config s_config;  // address chosen at runtime in setup()
static Bme280Driver s_driver;
static BringupContext s_ctx = {&s_driver.base, CFG_BME280_PERIOD_MS, NULL};

static const char* chipName(uint8_t id) {
  if (id == 0x60) return "BME280";
  if (id == 0x56 || id == 0x57 || id == 0x58) return "BMP280 (no humidity!)";
  return "unknown";
}
```

The bus, driver, configuration and context live for the program's lifetime. The context uses the faster climate period. chipName distinguishes the expected BME280 identity from BMP280 variants that lack humidity.

This helper only prints a diagnostic interpretation. The driver still enforces its own expected identity. A convenient label from this setup code does not bypass that check.

Probe addresses before constructing the chosen device

test/bringup/bme280_main.c / lines 23-45

```
23
24 void setup(void) {
25   delay(200);
26   printf("\n=== bring-up: bme280 (SDA %d, SCL %d, %lu Hz) ===\n", BOARD_PIN_I2C_SDA,
27          BOARD_PIN_I2C_SCL, (unsigned long)BOARD_I2C_CLOCK_HZ);
28
29   const bool busOk = i2cBusBegin(&s_i2c, BOARD_PIN_I2C_SDA, BOARD_PIN_I2C_SCL,
30                                  BOARD_I2C_CLOCK_HZ);
31   printf("i2c begin -> %s\n", busOk ? "OK" : "FAILED");
32
33   // Pick whichever address ACKs (SDO strap); default if neither does.
34   uint8_t address = BOARD_BME280_I2C_ADDRESS;
35   const bool ack76 = i2cBusProbe(&s_i2c, BOARD_BME280_I2C_ADDRESS);
36   const bool ack77 = i2cBusProbe(&s_i2c, BOARD_BME280_I2C_ADDRESS_ALT);
37   printf("probe 0x%02X -> %s, 0x%02X -> %s\n", BOARD_BME280_I2C_ADDRESS,
38          ack76 ? "ACK" : "-", BOARD_BME280_I2C_ADDRESS_ALT, ack77 ? "ACK" : "-");
39   if (!ack76 && ack77) address = BOARD_BME280_I2C_ADDRESS_ALT;
40
41   uint8_t id = 0;
42   if (i2cBusReadRegs(&s_i2c, address, 0xD0, &id, 1)) {
43     printf("chip id @0x%02X = 0x%02X (%s)\n", address, id, chipName(id));
44   } else {
45     printf("chip id @0x%02X -> read failed (check wiring/pull-ups)\n", address);
```

Setup begins the bus and probes both supported address straps. If only the alternate responds, it selects that one. It then reads the identity register and prints what arrived.

Probe success, register-read success and accepted chip identity are separate checkpoints. If neither address responds, retaining the default address lets the harness report init failure rather than making up a successful device. Read the output in that order when diagnosing wiring.

Pass the selected address into the production driver

test/bringup/bme280_main.c / lines 46-54

```
  }

  s_config.bus = &s_i2c;
  s_config.address = address;
  bme280DriverConstruct(&s_driver, &s_config);
  bringupStart(&s_ctx);
}

void loop(void) { vTaskDelay(portMAX_DELAY); }
```

The chosen bus and address are stored in s_config before construction. The harness then runs the unmodified production driver. Setup does not perform compensation itself.

loop blocks, so a failure to produce repeated records belongs to the task, lifecycle or bus path. The expectation of three metrics every 250ms is a target to check against elapsed readings, not a consequence of printing that expectation in a comment.

Start the DS18B20 and leave it working

The DS18B20 uses 1-Wire: a bus where the controller and sensor exchange timed bits on one data line. This project supplies the sensor externally from the retained 3.3V rail. The bus transaction needs CPU attention; the internal temperature conversion does not need the bus held busy for its whole duration.

Original firmware - src/drivers/ds18b20.c, lines 79-91

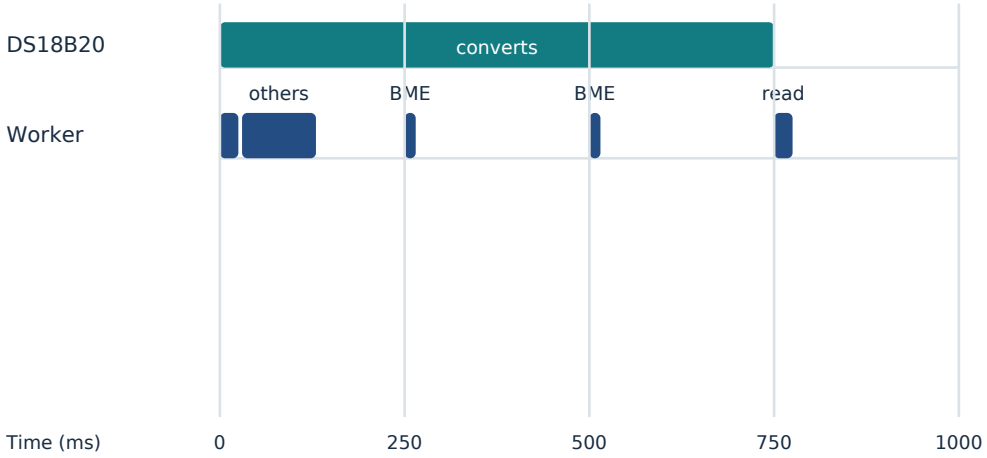
```
static DriverStatus ds18b20Start(SensorDriver* d, MonoMs now) {
  Ds18b20Driver* self = asDs18b20(d);
  if (!self->initialised_) return DRIVER_FAULT;
  if (!selectDevice(self)) {
    self->initialised_ = false;
    markNotPresent(self, now, now);
    return DRIVER_NOT_PRESENT;
  }
  oneWireBusWriteByte(self->config_.bus, kCmdConvertT);
  self->startMs_ = now;
  self->busy_ = true;
  return DRIVER_OK;
}
```

The driver checks initialisation, selects the discovered sensor, writes Convert T, stores the start time and returns. If selection fails, it marks the sensor missing and does not claim a conversion began. startMs_ becomes the reference for the later conversion wait.

REMEMBER

Waiting for physical conversion and transmitting a 1-Wire command are different kinds of time.

Start the DS18B20 and leave it working / continued



Illustrative 1s frame: the water chip converts while the worker handles other jobs.

At the chosen 12-bit resolution, the project allows 750ms before reading. A frame releases ordinary jobs every 1000ms, so the water conversion can span most of a frame. That is elapsed time inside the chip, not a 750ms busy loop in the worker.

The drawing shows conceptual ordering, not a measured trace or a hard timing guarantee. In real code, individual 1-Wire calls can block briefly and may mask interrupts. Acquisition must place those calls carefully relative to the ultrasonic echo window. We will return to that conflict after learning echo capture.

Reference: Analog Devices: DS18B20 datasheet

Validate the bytes before dividing them

Original firmware - src/drivers/ds18b20.c, lines 98-116

```
  // Fixed worst-case conversion time: no bus traffic while
>   converting.
  const MonoMs due = self->startMs_ +
>   CFG_DS18B20_CONVERSION_MAX_MS;
  if (now < due) {
    r.status = DRIVER_PENDING;
    r.nextPollMs = due;
    return r;
  }
  self->busy_ = false;
  uint8_t sp[DS18B20_SCRATCHPAD_LEN];
  if (!readScratchpad(self, sp)) {
    self->initialised_ = false;
    markNotPresent(self, self->startMs_, now);
    r.status = DRIVER_NOT_PRESENT;
    return r;
  }
  finish(self, sp, now);
  r.status = DRIVER_READY;
  return r;
}
```

Before due, poll returns PENDING with that same absolute timestamp. At or after due, it clears busy_, reads the scratchpad and calls finish(). The scratchpad is the sensor's small result-and-configuration memory area.

A successful transaction is not enough. The codec checks a CRC, a check byte calculated from the message bytes. A mismatch means the received bytes do not match the expected codeword, so the driver rejects the observation before interpreting temperature. CRC detects many transmission errors; it does not prove the probe's physical accuracy.

COMMON PITFALL

A familiar temperature value is not a replacement for CRC and lifecycle checks. Reset defaults can look like real measurements.

Validate the bytes before dividing them / continued

Worked decode - temperature bytes after a valid CRC

```
uint8_t low = 0x90;
uint8_t high = 0x01;
int16_t raw = (int16_t)((uint16_t)high << 8 | low);
float waterC = (float)raw / 16.0f;
// 0x0190 = 400; 400 / 16 = 25.0 degrees Celsius
```

This device places the low byte first, unlike the big-endian words we will meet in the SGP30. The signed 16-bit result supports negative temperatures. For example, raw=-160 gives -10.0 degrees Celsius after division by 16.

The driver also checks the configured resolution and application range. It treats the +85-degree power-on register value as a fault in this hydroponics application because that value can indicate a reset without a completed conversion. That is a project-specific diagnostic choice; +85 degrees Celsius is not universally an impossible DS18B20 measurement.

Reference: Analog Devices: DS18B20 datasheet

Give the next sensor a copy, not a live pointer

The TDS conversion needs water temperature. How should it get it? The acquisition owner already holds the latest water-temperature Reading in its working Snapshot. Before starting TDS, it calls tdsDriverSetWaterTemperature() with that record.

Original firmware - src/drivers/tds.c, lines 179-185

```
void tdsDriverSetWaterTemperature(TdsDriver* self, const
>   Reading* waterTemperature) {
  self->waterTemperature_ = *waterTemperature;
}

void tdsDriverSetCalibration(TdsDriver* self, const
>   LinearCalibration* calibration) {
  self->config_.calibration = *calibration;
}
```

The structure assignment makes a copy. TDS keeps that observation and its timestamp; it does not follow a changing pointer back into the Snapshot. That makes the dependency used for this measurement stable while the next water conversion progresses.

Suppose the previous water Reading ended at 900ms. TDS starts at 1100ms, before the new 750ms water conversion completes. At finish, it can use the previous 25.0-degree record if it remains fresh and OK. A dependency does not always have to come from the same frame; it has to satisfy the declared validity and age limits.

If the water sensor stops responding, the retained record eventually becomes stale. TDS then marks its own dependency invalid instead of silently using the last known temperature forever. This is the practical connection between timestamps, structure copies and compensation.

The source permits sharing an older-but-fresh dependency, so Snapshot cannot mean all sensors measured at one instant. When you review any calculated metric, identify both its direct input and the retained inputs used to compensate it.

The code path that supplies this copy lives in acquisition.c: prepareInputs(). You will read that caller in the acquisition pass (p. 341).

REMEMBER

A copied dependency is stable, but it still gets older. Copying does not refresh its timestamp.

Separate line timing from temperature interpretation

The long water conversion is implemented above a very short-bit protocol. This header makes the physical bus operations explicit.

The driver selects one discovered DS18B20; this adapter does not create a task or own the conversion schedule.

src/hal/onewire_bus.h / lines 1-24

```
// 1-Wire bus adapter (HAL). Work package WP2 (with DS18B20).
// Externally powered DS18B20 on BOARD_PIN_ONE_WIRE (4.7 kOhm pull-up).
//
// Bit-banged, open-drain GPIO (the pin only pulls low; the pull-up drives
// high). Standard-speed timing from the DS18B20 datasheet (slot 70 us).
// Each time slot runs with interrupts off on this core for at most ~70 us;
// the reset's 480 us low phase runs with interrupts on.
// Worst-case blocking: reset ~1 ms, one byte ~0.6 ms, search ~15 ms.
// No parasite power: there is no strong pull-up.
// WP7: do not run a 1-Wire transaction on the core that services the HC-SR04
// echo ISR while an echo is in flight - a masked edge is timestamped late
// (up to ~70 us, ~1 cm).
#pragma once
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

// Treat the members as private to onewire_bus.c. Initialise with ONE_WIRE_BUS_INIT.
typedef struct OneWireBus {
  int8_t pin_;
} OneWireBus;

#define ONE_WIRE_BUS_INIT {-1}

```

OneWireBus stores the assigned pin. Begin configures the electrical mode; Reset detects presence; byte functions encode timed bits; SearchFirst obtains one ROM identifier; Select addresses that device.

Separate line timing from temperature interpretation / continued

src/hal/onewire_bus.h / lines 25-34

```
25 // false if the pin is not a usable GPIO; the bus then stays unusable.
26 bool oneWireBusBegin(OneWireBus* bus, int8_t pin);
27 // true = presence pulse detected. false also if the bus does not idle high
28 // (shorted to GND or missing pull-up).
29 bool oneWireBusReset(OneWireBus* bus);
30 void oneWireBusWriteByte(OneWireBus* bus, uint8_t value);
31 uint8_t oneWireBusReadByte(OneWireBus* bus);
32 // Search ROM [F0h] for the first device (lowest branch at each discrepancy).
33 // Does its own reset. true if a code with a valid CRC was read into rom.
34 bool oneWireBusSearchFirst(OneWireBus* bus, uint8_t rom[8]);
```

src/hal/onewire_bus.h / lines 35-37

```
35 // Match ROM [55h] + code. Call right after a successful reset.
36 void oneWireBusSelect(OneWireBus* bus, const uint8_t rom[8]);
37 uint8_t oneWireBusCrc8(const uint8_t* data, size_t len);
```

CRC is exposed here for callers but implemented in the pure codec. A successful bus reset means a presence response was observed, not that a new temperature conversion has finished. Those milestones remain separate throughout the driver.

Name the bit-slot timing before producing pulses

Read these microsecond slots carefully. They explain why the acquisition scheduler later keeps 1-Wire work away from an ultrasonic echo window.

The critical sections protect slot timing; the rest intervals are kept outside when possible.

src/hal/onewire_bus.c / lines 1-28

```
#include "hal/onewire_bus.h"

#include <Arduino.h>
#include <driver/gpio.h>
#include <string.h>

#include "domain/ds18b20_codec.h"

// ESP32-S3 GPIO numbers run 0-48.
static const int8_t kMaxGpio = 48;

static const uint8_t kCmdSearchRom = 0xF0;
static const uint8_t kCmdMatchRom = 0x55;

// Slot timing (us), DS18B20 datasheet AC characteristics: tSLOT 60-120,
// tLOW1 1-15, tLOW0 60-120, tRDV 15, tREC >= 1, tRSTL/tRSTH >= 480,
// presence low starts 15-60 us after release and lasts 60-240 us.
enum {
  kResetLowUs = 480,
  kPresenceSampleUs = 70,
  kResetHighRestUs = 410,  // 70 + 410 = tRSTH
  kWrite1LowUs = 6,
  kWrite1RestUs = 64,
  kWrite0LowUs = 60,
  kWrite0RestUs = 10,
  kReadLowUs = 3,
  kReadSampleUs = 10,  // sampled 13 us into the slot, inside tRDV
  kReadRestUs = 53
```

The constants express reset low time, presence sampling, write-one and write-zero low durations, and read sampling. A logical one and zero differ in pulse timing rather than in a stored byte sent to a hardware UART.

Name the bit-slot timing before producing pulses / continued

src/hal/onewire_bus.c / lines 29-34

```
29 };
30
31 // Interrupts off for one slot only (bit timing must not be stretched).
32 static portMUX_TYPE s_mux = portMUX_INITIALIZER_UNLOCKED;
33
34 static inline void busLow(const OneWireBus* bus) { gpio_set_level((gpio_num_t)bus->pin_,
 >    0); }
```

src/hal/onewire_bus.c / lines 35-38

```
35 static inline void busRelease(const OneWireBus* bus) {
 >   gpio_set_level((gpio_num_t)bus->pin_, 1); }
36 static inline bool busLevel(const OneWireBus* bus) {
37   return gpio_get_level((gpio_num_t)bus->pin_) != 0;
38 }
```

busLow actively pulls the open-drain line low. busRelease writes the released state, allowing the external pull-up to make it high. Reading the level therefore observes the shared line, not merely the controller's last requested output.

Configure open-drain operation and detect presence

src/hal/onewire_bus.c / lines 39-66

```
39
40 bool oneWireBusBegin(OneWireBus* bus, int8_t pin) {
41   bus->pin_ = -1;
42   if (pin < 0 || pin > kMaxGpio) return false;
43   gpio_config_t io;
44   memset(&io, 0, sizeof(io));
45   io.pin_bit_mask = 1ULL << (uint8_t)pin;
46   io.mode = GPIO_MODE_INPUT_OUTPUT_OD;  // drive low or
 >   release; read back the line
47   io.pull_up_en = GPIO_PULLUP_DISABLE;  // external 4.7 kOhm
 >    to 3.3 V
48   io.pull_down_en = GPIO_PULLDOWN_DISABLE;
49   io.intr_type = GPIO_INTR_DISABLE;
50   if (gpio_config(&io) != ESP_OK) return false;
51   bus->pin_ = pin;
52   busRelease(bus);
53   return true;
54 }
55
56 bool oneWireBusReset(OneWireBus* bus) {
57   if (bus->pin_ < 0) return false;
58   busRelease(bus);
59   if (!busLevel(bus)) return false;  // stuck low: short or
 >   no pull-up
60   busLow(bus);
61   delayMicroseconds(kResetLowUs);
62   portENTER_CRITICAL(&s_mux);
63   busRelease(bus);
64   delayMicroseconds(kPresenceSampleUs);
65   const bool presence = !busLevel(bus);
66   portEXIT_CRITICAL(&s_mux);
```

Begin rejects an invalid pin before configuring input/output open-drain mode. It disables internal pulls because the board uses an external pull-up, then releases the line.

src/hal/onewire_bus.c / lines 67-69

```
67   delayMicroseconds(kResetHighRestUs);
68   return presence;
69 }
```

Reset first rejects a stuck-low idle line. It generates a reset pulse, releases the line and samples for a low presence response within a protected interval. The function returns that observed presence, not a decoded ROM or temperature.

COMMON PITFALL

Timing protection on this core can delay other edge-sensitive work. A short protocol slot is still a real interruption to other handlers.

Transmit and sample one bit in bounded steps

src/hal/onewire_bus.c / lines 70-90

```
70
71 static void writeBit(OneWireBus* bus, bool bit) {
72   portENTER_CRITICAL(&s_mux);
73   busLow(bus);
74   delayMicroseconds(bit ? kWrite1LowUs : kWrite0LowUs);
75   busRelease(bus);
76   portEXIT_CRITICAL(&s_mux);
77   delayMicroseconds(bit ? kWrite1RestUs : kWrite0RestUs);
78 }
79
80 static bool readBit(OneWireBus* bus) {
81   portENTER_CRITICAL(&s_mux);
82   busLow(bus);
83   delayMicroseconds(kReadLowUs);
84   busRelease(bus);
85   delayMicroseconds(kReadSampleUs);
86   const bool bit = busLevel(bus);
87   portEXIT_CRITICAL(&s_mux);
88   delayMicroseconds(kReadRestUs);
89   return bit;
90 }
```

writeBit chooses the low duration from the intended bit, releases the line, exits the critical section and waits the remaining slot time. readBit pulls low briefly, releases, waits to its sample point and records the shared level.

The protected portion covers the edge-to-sample relationship. The trailing rest is outside the critical section. This reduces how long interrupts are masked, but the caller still spends elapsed microseconds completing each slot.

Build bytes in least-significant-bit order

src/hal/onewire_bus.c / lines 91-104

```
91
92 void oneWireBusWriteByte(OneWireBus* bus, uint8_t value) {
93   if (bus->pin_ < 0) return;
94   for (uint8_t i = 0; i < 8; ++i) writeBit(bus, (value >> i) & 1u);  // LSB first
95 }
96
97 uint8_t oneWireBusReadByte(OneWireBus* bus) {
98   if (bus->pin_ < 0) return 0xFF;
99   uint8_t value = 0;
100  for (uint8_t i = 0; i < 8; ++i) {
101    if (readBit(bus)) value |= (uint8_t)(1u << i);
102  }
103  return value;
104 }
```

WriteByte shifts the source byte right by the bit index and sends its lowest bit. ReadByte tests each sampled bit and ORs 1u << i into the destination. This device protocol sends the least-significant bit first.

An unassigned read returns 0xFF in this implementation. That sentinel can also be ordinary data, so the higher-level presence and CRC checks are necessary. Do not interpret one byte alone as a complete health verdict.

Resolve one branch of the ROM search

src/hal/onewire_bus.c / lines 105-123

```
105
106 bool oneWireBusSearchFirst(OneWireBus* bus, uint8_t rom[8]) {
107   if (!oneWireBusReset(bus)) return false;
108   oneWireBusWriteByte(bus, kCmdSearchRom);
109   uint8_t id[8] = {0};
110   for (uint8_t i = 0; i < 64; ++i) {
111     const bool bit = readBit(bus);
112     const bool complement = readBit(bus);
113     if (bit && complement) return false;  // no device took part
114     // Different: every device has `bit` here. Equal (both 0): devices differ,
115     // take the 0 branch, which is the first device in search order.
116     const bool direction = bit != complement ? bit : false;
117     if (direction) id[i / 8] |= (uint8_t)(1u << (i % 8));
118     writeBit(bus, direction);
119   }
120   if (oneWireCrc8(id, 7) != id[7]) return false;
121   memcpy(rom, id, 8);
122   return true;
123 }
```

For each ROM position, the adapter reads a bit and its complement. Two ones mean no participating device; differing values select the available bit; two zeros mean competing devices, so this implementation deliberately follows the zero branch.

It writes that choice back to keep only matching devices in the search. After 64 positions it validates the ROM CRC before copying the temporary identifier to the caller. SearchFirst finds one device in this policy, not every device on a multi-drop bus.

Address the chosen device explicitly

src/hal/onewire_bus.c / lines 124-130

```
124
125 void oneWireBusSelect(OneWireBus* bus, const uint8_t rom[8]) {
126   oneWireBusWriteByte(bus, kCmdMatchRom);
127   for (uint8_t i = 0; i < 8; ++i) oneWireBusWriteByte(bus, rom[i]);
128 }
129
130 uint8_t oneWireBusCrc8(const uint8_t* data, size_t len) { return oneWireCrc8(data, len);
  >     }
```

Select sends Match ROM and all eight identifier bytes so later commands address the selected sensor. The CRC wrapper reuses the domain implementation rather than keeping a second private algorithm.

The adapter knows bytes and bus timing. The DS18B20 driver must still check family identity, configuration resolution, conversion timing and the scratchpad result. This boundary keeps physical timing separate from temperature acceptance.

Define what a valid DS18B20 record means

The codec names the scratchpad layout and numerical interpretation without depending on ESP32 hardware.

These functions are suitable for known-byte tests on a PC.

src/domain/ds18b20_codec.h / lines 1-24

```
1  // DS18B20 scratchpad decoding and 1-Wire CRC (WP2). Pure C, host-tested.
2  // Datasheet: Maxim DS18B20 (local copy in Electrical System Study/Datasheets/
3  // DS18B20.pdf): Figure 9 (memory map), Figure 10 (configuration register),
4  // Table 1 (temperature format), "CRC Generation".
5  #pragma once
6  #include <stdbool.h>
7  #include <stddef.h>
8  #include <stdint.h>
9
10 #define DS18B20_FAMILY_CODE 0x28
11 #define DS18B20_SCRATCHPAD_LEN 9
12 #define DS18B20_POWER_ON_RAW ((int16_t)0x0550)  // +85 degC, reset value of the register
13 #define DS18B20_CONFIG_12BIT 0x7F               // R1 = R0 = 1, reserved bits as read
14 #define DS18B20_MIN_C (-55.0f)
15 #define DS18B20_MAX_C 125.0f
16
17 // Dallas/Maxim CRC-8 (x^8 + x^5 + x^4 + 1), LSB first, init 0. Running it over
18 // data plus its CRC byte gives 0.
19 uint8_t oneWireCrc8(const uint8_t* data, size_t len);
20
21 // ROM code: CRC over the first 7 bytes matches byte 7 and the family is 28h.
22 // Rejects the all-zero code a bus stuck low produces.
23 bool ds18b20RomValid(const uint8_t rom[8]);
24
```

The constants name the family code, scratchpad size, chosen resolution, power-on raw value and application limits. The validation declarations distinguish ROM validity from scratchpad validity.

Define what a valid DS18B20 record means / continued

src/domain/ds18b20_codec.h / lines 25-35

```
25 // Scratchpad CRC matches AND the configuration register has its fixed bits
26 // (bit 7 = 0, bits 0-4 = 1). The second check rejects an all-zero read (bus
27 // stuck low), whose CRC is also 0.
28 bool ds18b20ScratchpadValid(const uint8_t scratchpad[DS18B20_SCRATCHPAD_LEN]);
29
30 // True if the configuration register selects 12-bit resolution.
31 bool ds18b20Is12Bit(uint8_t config);
32
33 // Signed temperature register (bytes 0-1), 1/16 degC per LSB at 12 bits.
34 int16_t ds18b20RawTemperature(const uint8_t scratchpad[DS18B20_SCRATCHPAD_LEN]);
35 float ds18b20RawToDegC(int16_t raw);
```

RawTemperature returns a signed integer; RawToDegC supplies its 1/16-degree scale. Keeping these steps separate makes endianness, signedness and scale individually inspectable. The driver decides when those bytes may be read and whether a reset-like value is acceptable here.

Advance the CRC one incoming bit at a time

Follow the bytes through CRC, configuration validation and a signed temperature. No RTOS state is involved in this file.

Use the worked 0x0190 value from the lesson to check the final two functions.

src/domain/ds18b20_codec.c / lines 1-15

```
#include "domain/ds18b20_codec.h"

uint8_t oneWireCrc8(const uint8_t* data, size_t len) {
  uint8_t crc = 0;
  for (size_t i = 0; i < len; ++i) {
    uint8_t in = data[i];
    for (uint8_t b = 0; b < 8; ++b) {
      const uint8_t mix = (crc ^ in) & 0x01;
      crc >>= 1;
      if (mix) crc ^= 0x8C;  // reflected 0x31
      in >>= 1;
    }
  }
  return crc;
}
```

The accumulator begins at zero. Each iteration combines the next incoming bit with the CRC's low bit, shifts the CRC and conditionally XORs the reflected polynomial. The input byte shifts right so the next bit becomes available.

This is deterministic processing of supplied bytes. It does not read the 1-Wire line. If a received scratchpad changes one protected byte, recalculating this code and comparing its check byte can expose the corruption.

Check structure before interpreting signed temperature

src/domain/ds18b20_codec.c / lines 16-32

```
16
17 bool ds18b20RomValid(const uint8_t rom[8]) {
18   return rom[0] == DS18B20_FAMILY_CODE && oneWireCrc8(rom, 7) == rom[7];
19 }
20
21 bool ds18b20ScratchpadValid(const uint8_t scratchpad[DS18B20_SCRATCHPAD_LEN]) {
22   if (oneWireCrc8(scratchpad, 8) != scratchpad[8]) return false;
23   return (scratchpad[4] & 0x9F) == 0x1F;
24 }
25
26 bool ds18b20Is12Bit(uint8_t config) { return (config & 0x60) == 0x60; }
27
28 int16_t ds18b20RawTemperature(const uint8_t scratchpad[DS18B20_SCRATCHPAD_LEN]) {
29   return (int16_t)(uint16_t)((uint16_t)scratchpad[1] << 8 | scratchpad[0]);
30 }
31
32 float ds18b20RawToDegC(int16_t raw) { return (float)raw / 16.0f; }
```

ROM validation checks family and CRC. Scratchpad validation checks its CRC and the expected configuration-bit pattern. Resolution checking then determines whether the selected twelve-bit mode is active.

RawTemperature combines high and low bytes into uint16_t before interpreting the signed pattern as int16_t on this target. RawToDegC divides by 16. For raw = 400, the result is 25.0; for raw = -160, it is -10.0. The reset-value policy remains in the driver, not hidden in the arithmetic helper.

Read the conversion and reset assumptions

The driver holds a selected ROM, pending-conversion time and one retained temperature result. The header also explains the intended short blocking cost of each bus action.

The retained supply keeps this sensor available in the project's low-power sensor set.

src/drivers/ds18b20.h / lines 1-28

```
// ds18b20 driver. Work package WP2 (Stage 2). Architecture §7, §14.
// Datasheet: Maxim DS18B20 (local copy in Electrical System Study/Datasheets/
// DS18B20.pdf). Domain math (CRC, decoding) in domain/ds18b20_codec.h.
//
// Retained 3.3 V supply (external power, no parasite mode), 1-Wire on GPIO14
// with a 4.7 kOhm pull-up. Produces WaterTemperature in degC.
//
// init() finds the first device with Search ROM, checks the ROM CRC and the
// family code (28h) and keeps that ROM: every later transaction uses Match
// ROM, so a second device on the bus is never read by mistake. It then reads
// the scratchpad and, if the EEPROM selected a lower resolution, writes 12-bit
// to the scratchpad only (no Copy Scratchpad: no EEPROM wear, re-applied on
// every init).
// start() sends Convert T and returns; poll() is Pending until
// CFG_DS18B20_CONVERSION_MAX_MS (750 ms, 12-bit tCONV max) has passed, then
// reads the 9-byte scratchpad once.
//
// Results, highest priority first (poll() returns READY for all of them):
//   scratchpad CRC or fixed config bits wrong -> HEALTH_FAULT, no raw
//   temperature register = 0550h (+85 degC power-on value) -> HEALTH_FAULT;
//                     the device reset without converting, so the next start()
//                     fails and init() re-applies the resolution
//   outside -55..+125 degC -> HEALTH_FAULT
//   otherwise         -> value in degC, HEALTH_OK
// raw = temperature register (1/16 degC) whenever the CRC is good.
// No presence pulse in start()/poll() -> DRIVER_NOT_PRESENT (re-init needed).
// Before the first result readResult() gives HEALTH_STARTING, never 0 degC.
//
```

The contract states the externally supplied operating mode and conversion allowance. It also describes why the power-on temperature register value is rejected for this water application.

src/drivers/ds18b20.h / lines 29-30

```
// Worst-case blocking (1-Wire HAL; slots run with interrupts off <= 70 us each):
//   init()    ~50 ms  (search ~15 ms, scratchpad read ~12 ms, and when the
```

A hardware datasheet can permit a value that the application treats as a diagnostic sign under its expected operating range. Keep that distinction explicit: the driver is enforcing this controller's policy, not redefining the device's universal capability.

Keep address selection and result history across calls

src/drivers/ds18b20.h / lines 31-57

```
//                       resolution must be set: write ~9 ms + re-read ~12 ms)
//   start()   ~7 ms   (reset, Match ROM, Convert T)
//   poll()    ~12 ms  once the conversion time is over (reset, Match ROM,
//                       Read Scratchpad, 9 bytes); Pending polls: none
//   others    none
//
// Implements the SensorDriver contract in drivers/sensor_driver.h.
#pragma once
#include "drivers/sensor_driver.h"
#include "hal/onewire_bus.h"

typedef struct Ds18b20Config {
  OneWireBus* bus;  // already begun on BOARD_PIN_ONE_WIRE
} Ds18b20Config;

// Members after `base` are private to ds18b20.c.
typedef struct Ds18b20Driver {
  SensorDriver base;  // must stay first
  Ds18b20Config config_;
  bool initialised_;
  bool busy_;
  bool hasRom_;
  uint8_t rom_[8];
  MonoMs startMs_;
  Reading result_;
} Ds18b20Driver;

```

The configuration points to an existing OneWireBus. The object retains ROM identity, whether discovery succeeded, lifecycle flags, start time and a result. GetRom copies the selected identifier for diagnostics rather than returning writable internal storage.

src/drivers/ds18b20.h / lines 58-62

```
// Constructor. Use &driver.base wherever a SensorDriver* is expected.
void ds18b20DriverConstruct(Ds18b20Driver* self, const Ds18b20Config* config);

// ROM code selected by the last successful init(). false before that.
bool ds18b20DriverGetRom(const Ds18b20Driver* self, uint8_t rom[8]);
```

As with light, the base member is first. Construction supplies generic dispatch, and init performs actual discovery. A later reinit can select a device again without reconstructing the whole task.

Choose one device before every addressed operation

Now join the timed bus primitives with the conversion lifecycle. The important order is select, convert, leave, return and validate.

The first successful init establishes the addressed sensor and its resolution.

src/drivers/ds18b20.c / lines 1-26

```
#include "drivers/ds18b20.h"

#include <string.h>

#include "app_config.h"
#include "domain/ds18b20_codec.h"

static const uint8_t kCmdConvertT = 0x44;
static const uint8_t kCmdReadScratchpad = 0xBE;
static const uint8_t kCmdWriteScratchpad = 0x4E;

static inline Ds18b20Driver* asDs18b20(SensorDriver* d) { return (Ds18b20Driver*)d; }

static void finish(Ds18b20Driver* self, const uint8_t sp[DS18B20_SCRATCHPAD_LEN], MonoMs
    now);

// No presence pulse: publish a FAULT verdict, as the other drivers do, so the
// last OK reading is not served as current. lastGoodMs keeps its value.
static void markNotPresent(Ds18b20Driver* self, MonoMs start, MonoMs now) {
  Reading* r = &self->result_;
  r->intervalStartMs = start;
  r->intervalEndMs = now;
  r->qualityFlags = kQualityNone;
  r->hasRaw = false;
  r->health = HEALTH_FAULT;
}

```

markNotPresent records a failed observation interval without pretending zero degrees was measured. The identity helpers provide generic metadata. selectDevice requires a discovered ROM, resets the bus and sends Match ROM before the next command.

src/drivers/ds18b20.c / lines 27-35

```
static SensorId ds18b20Id(const SensorDriver* d) { (void)d; return SENSOR_DS18B20; }
static const char* ds18b20Name(const SensorDriver* d) { (void)d; return "ds18b20"; }
static uint8_t ds18b20MetricCount(const SensorDriver* d) { (void)d; return 1; }

// Reset + Match ROM. false if no presence pulse.
static bool selectDevice(Ds18b20Driver* self) {
  OneWireBus* bus = self->config_.bus;
  if (!oneWireBusReset(bus)) return false;
  oneWireBusSelect(bus, self->rom_);
```

Choose one device before every addressed operation / continued

src/drivers/ds18b20.c / lines 36-48

```
  return true;
}

// Read all 9 scratchpad bytes. false if no presence pulse; the content is
// checked by the caller.
static bool readScratchpad(Ds18b20Driver* self, uint8_t sp[DS18B20_SCRATCHPAD_LEN]) {
  if (!selectDevice(self)) return false;
  oneWireBusWriteByte(self->config_.bus, kCmdReadScratchpad);
  for (uint8_t i = 0; i < DS18B20_SCRATCHPAD_LEN; ++i) {
    sp[i] = oneWireBusReadByte(self->config_.bus);
  }
  return true;
}
```

readScratchpad then requests the scratchpad and obtains all bytes. It reports transport-level success here; the codec validation performed by its callers decides whether those bytes can be trusted.

Discover, validate and establish resolution

src/drivers/ds18b20.c / lines 49-74

```
49
50 static DriverStatus ds18b20Init(SensorDriver* d) {
51   Ds18b20Driver* self = asDs18b20(d);
52   self->initialised_ = false;
53   self->busy_ = false;
54   if (self->config_.bus == 0) return DRIVER_FAULT;
55
56   uint8_t rom[8];
57   if (!oneWireBusSearchFirst(self->config_.bus, rom)) return DRIVER_NOT_PRESENT;
58   if (!ds18b20RomValid(rom)) return DRIVER_NOT_PRESENT;  // not a DS18B20
59   memcpy(self->rom_, rom, sizeof(rom));
60   self->hasRom_ = true;
61
62   uint8_t sp[DS18B20_SCRATCHPAD_LEN];
63   if (!readScratchpad(self, sp)) return DRIVER_NOT_PRESENT;
64   if (!ds18b20ScratchpadValid(sp)) return DRIVER_FAULT;
65   if (!ds18b20Is12Bit(sp[4])) {
66     // TH and TL are written back unchanged; the scratchpad only, not EEPROM.
67     if (!selectDevice(self)) return DRIVER_NOT_PRESENT;
68     oneWireBusWriteByte(self->config_.bus, kCmdWriteScratchpad);
69     oneWireBusWriteByte(self->config_.bus, sp[2]);
70     oneWireBusWriteByte(self->config_.bus, sp[3]);
71     oneWireBusWriteByte(self->config_.bus, DS18B20_CONFIG_12BIT);
72     if (!readScratchpad(self, sp)) return DRIVER_NOT_PRESENT;
73     if (!ds18b20ScratchpadValid(sp) || !ds18b20Is12Bit(sp[4])) return DRIVER_FAULT;
74   }
```

Init clears active state, searches one ROM and rejects the wrong family or bad identifier CRC. It copies the selected address only after that check. It then reads and validates the scratchpad.

src/drivers/ds18b20.c / lines 75-77

```
75   self->initialised_ = true;
76   return DRIVER_OK;
77 }
```

If the resolution differs, it writes the desired configuration while preserving the existing alarm bytes, rereads and verifies the result. It does not burn EEPROM merely to establish this runtime configuration. The final true assignment means these setup conditions were satisfied.

Begin physical work after selection succeeds

src/drivers/ds18b20.c / lines 78-91

```
78
79 static DriverStatus ds18b20Start(SensorDriver* d, MonoMs now) {
80   Ds18b20Driver* self = asDs18b20(d);
81   if (!self->initialised_) return DRIVER_FAULT;
82   if (!selectDevice(self)) {
83     self->initialised_ = false;
84     markNotPresent(self, now, now);
85     return DRIVER_NOT_PRESENT;
86   }
87   oneWireBusWriteByte(self->config_.bus, kCmdConvertT);
88   self->startMs_ = now;
89   self->busy_ = true;
90   return DRIVER_OK;
91 }
```

Start rejects uninitialised state, selects the device, writes Convert T and stores the start time. Selection failure marks a failed observation and requires recovery.

Returning OK does not claim a temperature is ready. It says the caller can now manage a pending conversion using startMs_. No traffic is needed on this bus during the conversion allowance in this implementation.

Use the deadline to decide when to read

src/drivers/ds18b20.c / lines 92-116

```
92
93 static PollResult ds18b20Poll(SensorDriver* d, MonoMs now) {
94   Ds18b20Driver* self = asDs18b20(d);
95   PollResult r = {DRIVER_FAULT, now};
96   if (!self->busy_) return r;
97
98   // Fixed worst-case conversion time: no bus traffic while converting.
99   const MonoMs due = self->startMs_ + CFG_DS18B20_CONVERSION_MAX_MS;
100  if (now < due) {
101    r.status = DRIVER_PENDING;
102    r.nextPollMs = due;
103    return r;
104  }
105  self->busy_ = false;
106  uint8_t sp[DS18B20_SCRATCHPAD_LEN];
107  if (!readScratchpad(self, sp)) {
108    self->initialised_ = false;
109    markNotPresent(self, self->startMs_, now);
110    r.status = DRIVER_NOT_PRESENT;
111    return r;
112  }
113  finish(self, sp, now);
114  r.status = DRIVER_READY;
115  return r;
116 }
```

Before start+750ms, the function returns PENDING with that exact due time. It does not repeatedly probe completion bits. At or after due, it ends busy state and requests a scratchpad.

A failed read reports not-present and forces later init. A completed read passes through finish, which may produce a health fault even though the operation returns READY. This is the concrete driver-status versus observation-health distinction.

Reject reset-like and corrupt results before acceptance

src/drivers/ds18b20.c / lines 117-144

```
117
118 static void finish(Ds18b20Driver* self, const uint8_t sp[DS18B20_SCRATCHPAD_LEN], MonoMs
  >     now) {
119   Reading* r = &self->result_;
120   r->intervalStartMs = self->startMs_;
121   r->intervalEndMs = now;
122   r->qualityFlags = kQualityNone;
123   r->hasRaw = false;
124
125   // READY in every case below: health carries the verdict.
126   if (!ds18b20ScratchpadValid(sp)) {
127     r->health = HEALTH_FAULT;  // corrupted read; the content is not trusted
128     return;
129   }
130   const int16_t raw = ds18b20RawTemperature(sp);
131   r->hasRaw = true;
132   r->raw = raw;
133   if (raw == DS18B20_POWER_ON_RAW) {
134     // The register still holds its reset value: the device restarted and did
135     // not convert. Water never reads exactly +85 degC here.
136     r->health = HEALTH_FAULT;
137     self->initialised_ = false;  // re-apply the resolution
138     return;
139   }
140   const float degC = ds18b20RawToDegC(raw);
141   if (degC < DS18B20_MIN_C || degC > DS18B20_MAX_C) {
142     r->health = HEALTH_FAULT;
143     return;
144   }
```

finish stamps interval fields and initially clears raw availability. CRC/config validation must pass before the signed raw value is stored. The power-on raw value then triggers a fault and reinitialisation in this application.

src/drivers/ds18b20.c / lines 145-149

```
145   r->value = degC;
146   r->health = HEALTH_OK;
147   r->lastGoodMs = now;
148   ++r->sequence;
149 }
```

Only an in-range conversion reaches value, OK, lastGoodMs and the driver's usable sequence increment. A failure can preserve an old numeric value in the record while changing health. A consumer must not ignore that new verdict.

Copy, cancel and require setup again

src/drivers/ds18b20.c / lines 150-162

```
150
151 static uint8_t ds18b20ReadResult(SensorDriver* d, Reading* out, uint8_t maxOut) {
152   if (maxOut < 1) return 0;
153   out[0] = asDs18b20(d)->result_;
154   return 1;
155 }
156
157 // A conversion in progress finishes on its own; its result is never read.
158 static void ds18b20Cancel(SensorDriver* d) { asDs18b20(d)->busy_ = false; }
159
160 // Retained rail, and the open-drain pin only ever releases the line between
161 // slots: nothing to drive. Drop any pending conversion.
162 static void ds18b20PreparePowerOff(SensorDriver* d) { ds18b20Cancel(d); }
```

readResult respects the caller's one-record capacity. Cancel drops the pending flag. PreparePowerOff also clears initialisation, so a later permitted use must rediscover or verify the sensor.

The functions contain no queue calls. The same completed Reading can be collected by the isolated harness or production acquisition. That reuse is possible because the driver reports outcomes rather than owning system policy.

Construct the instance and expose a ROM copy

src/drivers/ds18b20.c / lines 163-186

```
163
164 static const SensorDriverOps kDs18b20Ops = {
165     .id = ds18b20Id,
166     .name = ds18b20Name,
167     .metricCount = ds18b20MetricCount,
168     .init = ds18b20Init,
169     .start = ds18b20Start,
170     .poll = ds18b20Poll,
171     .readResult = ds18b20ReadResult,
172     .cancel = ds18b20Cancel,
173     .preparePowerOff = ds18b20PreparePowerOff,
174 };
175
176 void ds18b20DriverConstruct(Ds18b20Driver* self, const Ds18b20Config* config) {
177   self->base.ops = &kDs18b20Ops;
178   self->config_ = *config;
179   self->initialised_ = false;
180   self->busy_ = false;
181   self->hasRom_ = false;
182   memset(self->rom_, 0, sizeof(self->rom_));
183   self->startMs_ = 0;
184   readingReset(&self->result_, SENSOR_DS18B20, METRIC_WATER_TEMPERATURE, UNIT_DEG_C);
185 }
186
```

The operations table ties the lifecycle together. Construction clears ROM and state and resets one water-temperature Reading. GetRom returns false until an address exists, otherwise copies eight bytes into caller storage.

src/drivers/ds18b20.c / lines 187-191

```
187 bool ds18b20DriverGetRom(const Ds18b20Driver* self, uint8_t rom[8]) {
188   if (!self->hasRom_) return false;
189   memcpy(rom, self->rom_, sizeof(self->rom_));
190   return true;
191 }
```

The returned boolean is the condition under which that diagnostic buffer is new. The caller should compare or print it only after success. The production path does not need to expose the driver object itself to other tasks.

Remember which identity was already printed

This test adds a ROM observer to the shared harness so discovery and temperature are visible without a full system boot.

The observer runs in the driver-owning harness task after results complete.

test/bringup/ds18b20_main.c / lines 1-28

```
// Bring-up entry point for ds18b20 (WP2).  pio run -e test_ds18b20 -t upload -t monitor
// Retained 3.3 V supply, DQ on GPIO14 with a 4.7 kOhm pull-up to 3.3 V.
// 12-bit conversion (750 ms max) runs inside the 1 s period.
// Replaces src/main.c in this env only; links the production driver.
//
// Prints the selected ROM code once, then one temperature per second.
// "init -> NOT_PRESENT": no presence pulse (DQ open, no supply, missing
// pull-up) or the first device is not a DS18B20 (family 28h).
#include <Arduino.h>
#include <stdio.h>
#include <string.h>

#include "app_config.h"
#include "board_profile.h"
#include "bringup_harness.h"
#include "hal/onewire_bus.h"
#include "drivers/ds18b20.h"

static OneWireBus s_bus = ONE_WIRE_BUS_INIT;
static Ds18b20Config s_config = {&s_bus};
static Ds18b20Driver s_driver;

// Print the ROM code whenever init() selected a different one.
static void printRomIfNew(SensorDriver* driver) {
  (void)driver;
  static uint8_t last[8];
  static bool printed = false;
  uint8_t rom[8];
```

The callback gets a fresh ROM copy, compares it with a persistent last array and prints only on the first identity or a changed identity. memcmp compares bytes; memcpy updates the retained copy.

Remember which identity was already printed / continued

test/bringup/ds18b20_main.c / lines 29-35

```
29   if (!ds18b20DriverGetRom(&s_driver, rom)) return;
30   if (printed && memcmp(rom, last, sizeof(rom)) == 0) return;
31   memcpy(last, rom, sizeof(rom));
32   printed = true;
33   printf("# ds18b20 ROM %02X-%02X%02X%02X%02X%02X%02X crc %02X\n", rom[0], rom[6],
 >   rom[5],
34           rom[4], rom[3], rom[2], rom[1], rom[7]);
35 }
```

The local rom array is safe because the callback finishes using it before returning. The persistent last array survives later calls. This is a useful example of two different storage durations in one short function.

Supply the callback as data, then start the harness

test/bringup/ds18b20_main.c / lines 36-49

```
36
37 static BringupContext s_ctx = {&s_driver.base, CFG_FRAME_PERIOD_MS, printRomIfNew};
38
39 void setup(void) {
40   ds18b20DriverConstruct(&s_driver, &s_config);
41   delay(200);
42   printf("\n=== bring-up: %s ===\n", sensorDriverName(&s_driver.base));
43   if (!oneWireBusBegin(&s_bus, BOARD_PIN_ONE_WIRE)) {
44     printf("GPIO%d cannot be used for 1-Wire\n", (int)BOARD_PIN_ONE_WIRE);
45   }
46   bringupStart(&s_ctx);
47 }
48
49 void loop(void) { vTaskDelay(portMAX_DELAY); }
```

The context stores printRomIfNew without calling it. Setup constructs the driver, configures the pin and starts the common task. The task later decides when a READY result permits the callback.

The diagnostic procedure can distinguish no discovery from CRC rejection or delayed conversion. If the device reappears with a different ROM, the next successful result can make that change visible without introducing another bus-reading task.

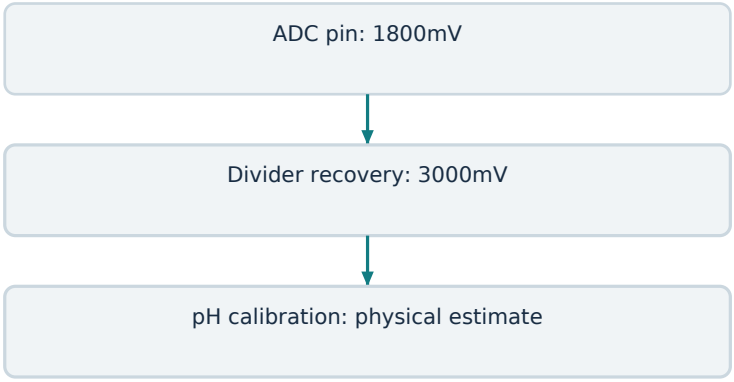
Undo the divider before applying the fit

The pH module's output is divided before reaching GPIO7. In this source board profile, the top resistor is 10kOhm and the lower resistor is 15kOhm. The ADC pin therefore sees 15/(10+15)=0.6 of the module voltage.

Original firmware - src/domain/calibration.c, lines 5-8

```
float dividerRecoverMv(float adcMv, uint32_t rTopOhm, uint32_t
>   rBottomOhm) {
  if (rBottomOhm == 0) return 0.0f;
  return adcMv * (float)(rTopOhm + rBottomOhm) /
>   (float)rBottomOhm;
}
```

If the filtered ADC voltage is 1800mV, recover 1800 x 25/15 = 3000mV at the module. A fit calibrated against module millivolts must use 3000, not 1800. Applying it to the ADC voltage would shift every pH result even if the line-fitting code were correct.



The numerical example follows one observation. Calibration is deliberately separate from ADC setup.

The helper guards against a zero bottom resistance before dividing. Its parameters are resistor values in ohms, not measured node voltages. The casts make the arithmetic floating-point. The sum and ratio are then applied to the input voltage.

The driver takes a spread sample window and filters it before recovering the module voltage. Filtering reduces selected noise. Divider recovery changes the electrical scale. Calibration changes voltage into the reported physical quantity. Keep those operations separate in your explanation and in your checks.

COMMON PITFALL

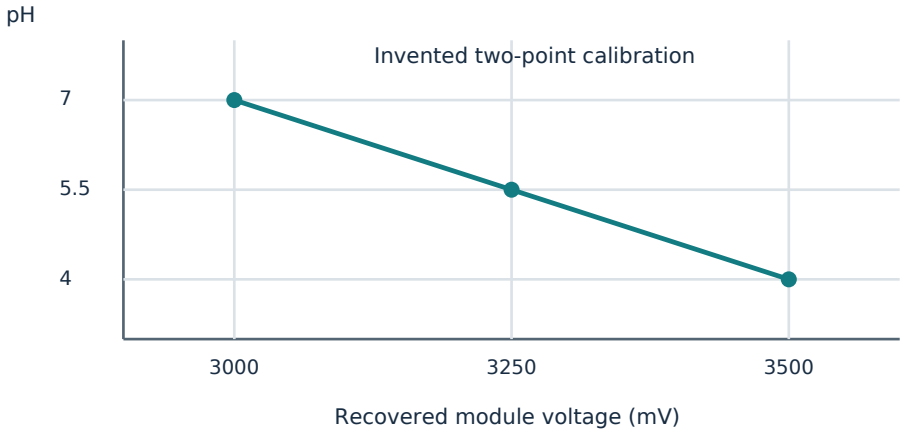
A resistor tolerance changes the real divider ratio. Nominal recovery is a model; precision work needs measured ratios or calibration that absorbs their effect.

Fit two known solutions, then test the fit

Use a worked, invented calibration: the module gives 3000mV in pH7 buffer and 3500mV in pH4 buffer. For a line y=m*x+b, the slope is (4-7)/(3500-3000)=-0.006 pH/mV. The offset is 7-(-0.006 x 3000)=25.

Worked calibration model - invented points

```
float ph = -0.006f * moduleMv + 25.0f;
// moduleMv = 3250 -> pH = 5.5
```



Worked calibration points and their midpoint. These are invented data, not probe measurements.

The negative slope is not a coding mistake in this example: higher module voltage corresponds to lower pH. The sign comes from the measured pair order. Always calculate it rather than choosing a positive sign because it looks simpler.

COMMON PITFALL

Do not use two readings from the same solution as two independent reference points. A tiny voltage difference can create an enormous slope.

Fit two known solutions, then test the fit / continued

linearFromTwoPoints() rejects non-finite inputs and two x coordinates that are too close. phFitFromBuffers() also requires the buffer pH values to be sufficiently separated and the slope magnitude to lie within the project's allowed range. These guards prevent division by an almost-zero voltage difference and implausible fits.

Two points determine a line; they do not validate it away from those points. A third known reference can test whether the chosen model remains reasonable. The book's numbers are a calculation lesson, not measured calibration values for your probe.

The archived firmware has a compiled nominal pH fit. It deliberately labels results UNCALIBRATED until a buffer-based fit overrides it. That label prevents a tutorial coefficient from silently becoming evidence that this particular probe was calibrated.

The driver applies the fit after filtering a complete raw window. Configuration ownership (p. 519) will explain why a replacement fit is copied at the acquisition boundary rather than edited from a radio callback.

Read the order of the result checks

Original firmware - src/drivers/ph.c, lines 97-115

```
  // READY in every case below: health carries the verdict, raw
>    the level.
  PollResult pr = {DRIVER_READY, now};
  if (pinMv < (float)PH_NO_SIGNAL_PIN_MV) {
    r->health = HEALTH_UNPOWERED;
    return pr;
  }
  if (self->saturated_) {
    r->health = HEALTH_FAULT;
    return pr;
  }

  if (!cfg->calibration.valid) {
    r->unit = UNIT_MILLIVOLT;
    r->value = moduleMv;
    r->health = HEALTH_UNCALIBRATED;
    r->lastGoodMs = now;  // last usable result (OK or
>   UNCALIBRATED), as the SGP30
    ++r->sequence;  // a fresh sample, in raw units
    return pr;
  }
```

The result is READY in every branch here: the observation ended. Health explains the outcome. A pin level below the configured no-signal threshold is labelled UNPOWERED by this driver. A saturated ADC level is a fault because a clipped pH voltage cannot support the intended conversion.

REMEMBER

DRIVER_READY means collect the outcome. HEALTH_OK means this observation passed the driver's acceptance checks.

Read the order of the result checks / continued

If no calibration is valid, the driver publishes recovered module millivolts and UNCALIBRATED. If a calibration exists, it applies the line, checks for a finite result, checks the range for a non-provisional fit, and then publishes pH. A provisional nominal fit can remain UNCALIBRATED even though its unit is pH.

The checks must precede acceptance. Once the ADC clips, averaging more clipped values will not reconstruct the lost voltage. Once a computed value is NaN or infinity, comparing it with a normal plant threshold is not a sound acceptance policy.

The driver-level no-signal test is a heuristic specific to this module path. Acquisition also checks the external 5V rail history before publishing. Those checks answer different questions: the analog level may suggest a dead output, while the rail epoch can prove that a power transition happened during the operation.

When reviewing this code, follow value, unit and health together. The presence of a float in value does not promise that consumers are permitted to use it. Existing memory may retain an earlier number after a new failure.

Keep nominal conversion separate from acceptance

The pH driver reuses the analog window pattern, but its conversion and fault meanings differ from light. Read the contract before assuming the same voltage has the same significance.

The input calibration uses recovered module millivolts, while raw retains pin millivolts.

src/drivers/ph.h / lines 1-23

```
// ph4502c driver. Work package WP5 (Stage 3). Architecture §3, §7, §14.
// Source: PH-Sensor-4502C.pdf (a tutorial, not a manufacturer specification).
//
// Switched 5 V rail. PO -> 10k/15k divider -> ADC1 pin (GPIO7). DO/TO unused.
// Pin mV -> divider recovery -> module mV -> two-point buffer fit -> pH.
// Module output is 0-5 V, so at most 3 V at the pin (inside the S3's ~3.1 V
// range at 11 dB). With the offset pot set so a shorted BNC reads 2.5 V at PO
// (pH 7), the tutorial's nominal slope is about -5.7 pH/V.
//
// Filter (tutorial sketch): PH_SAMPLES conversions spaced PH_SAMPLE_SPACING_MS
// apart, sorted, mean of the middle PH_SAMPLES - 2*PH_TRIM. The window is spread
// over successive poll() calls, never taken as a burst (§9): each tap is due
// PH_SAMPLE_SPACING_MS after the previous actual sample, so a late poll delays
// the rest of the window instead of compressing it.
// No temperature compensation: the tutorial reports < 0.01 pH below 30 degC.
//
// Results:
//   calibration.valid  -> value in pH, HEALTH_OK (FAULT outside 0-14)
//     + calibrationProvisional -> value in pH, HEALTH_UNCALIBRATED (a nominal
//                         fit, e.g. shorted-BNC offset + tutorial slope)
//                         (also outside 0-14: the nominal fit may not match the
//                         probe yet; capturable for calibration, never actionable)
//   otherwise          -> value = module mV, UNIT_MILLIVOLT, HEALTH_UNCALIBRATED
```

COMMON PITFALL

A source comment claiming a temperature error does not implement temperature compensation. This driver has no water-temperature input.

Keep nominal conversion separate from acceptance / continued

src/drivers/ph.h / lines 24-28

```
24 //   window mean < PH_NO_SIGNAL_PIN_MV -> HEALTH_UNPOWERED (a 0 V output is a
25 //                         dead rail or open lead, never "very acidic", §10).
26 //                         Also seen before the offset pot is set: factory
27 //                         offset puts pH 7 near 0 V.
28 //   any sample clipped -> kQualitySaturated + HEALTH_FAULT
```

The contract lists three possible output interpretations: calibrated pH, provisional pH and uncalibrated module voltage. It also identifies the no-signal and clipping checks. READY means any of those outcomes can be collected.

src/drivers/ph.h / lines 29-31

```
29 // poll() returns READY for all of these; the health carries the verdict.
30 // raw = filtered pin mV in every case.
31 //
```

The cited module reference in this source is a tutorial, not a uniquely authoritative specification for every board sold under the same module name. The implementation's offset and no-signal assumptions should be checked on the actual module.

Store the divider and fit with the window state

src/drivers/ph.h / lines 32-58

```
32 // Worst-case blocking (one ADC conversion C, well under 1 ms):
33 //   init()    1 C   (throwaway conversion, checks the channel)
34 //   start()   1 C   (first sample of the window)
35 //   poll()    1 C   Pending until the window is full (>= 270 ms)
36 //   others    none
37 //
38 // Implements the SensorDriver contract in drivers/sensor_driver.h.
39 #pragma once
40 #include "drivers/sensor_driver.h"
41 #include "hal/adc_channel.h"
42 #include "domain/calibration.h"
43
44 #define PH_SAMPLES 10
45 #define PH_TRIM 2
46 #define PH_SAMPLE_SPACING_MS 30
47 // Provisional: below this at the pin the module is treated as unpowered.
48 // pH 14 is ~1.3 V at PO (~0.8 V at the pin), so 100 mV is far from real data.
49 #define PH_NO_SIGNAL_PIN_MV 100
50
51 typedef struct PhConfig {
52   AdcChannel* adc;             // already begun on BOARD_PIN_PH_ADC
53   uint32_t dividerTopOhm;      // BOARD_PH_DIVIDER_TOP_OHM
54   uint32_t dividerBottomOhm;   // BOARD_PH_DIVIDER_BOTTOM_OHM
55   LinearCalibration calibration;  // x = module mV, y = pH
56   bool calibrationProvisional;    // true: fit not checked against buffers
57 } PhConfig;
58
```

PhConfig adds divider values and calibrationProvisional to the ADC pointer and line record. The object then keeps the familiar count, samples, timing and retained result fields.

Store the divider and fit with the window state / continued

src/drivers/ph.h / lines 59-73

```
// Members after `base` are private to ph.c.
typedef struct PhDriver {
  SensorDriver base;  // must stay first
  PhConfig config_;
  bool initialised_;
  bool busy_;
  bool saturated_;
  uint8_t count_;
  uint32_t samples_[PH_SAMPLES];  // pin mV
  MonoMs startMs_;
  MonoMs lastSampleMs_;  // next tap is due PH_SAMPLE_SPACING_MS after this
  Reading result_;
} PhDriver;

// Constructor. Use &driver.base wherever a SensorDriver* is expected.
```

src/drivers/ph.h / lines 74-78

```
void phDriverConstruct(PhDriver* self, const PhConfig* config);

// New fit from the applied configuration (on-device calibration, WP8).
// Called by the acquisition owner between windows; applies from the next one.
void phDriverSetCalibration(PhDriver* self, const LinearCalibration* calibration, bool
  provisional);
```

The spacing is 30ms: ten taps require at least 270ms from the first to the last. The provisional flag allows a numerical pH estimate to remain explicitly uncalibrated until buffer-based verification replaces it. Changing the flag changes eligibility, not the arithmetic storage type.

Prepare the channel without certifying the probe

Follow the complete pH path from one tap to its verdict. Compare the shared window pattern with light, then pay attention to the different finish policy.

The owner applies calibration changes between windows through the public setter.

src/drivers/ph.c / lines 1-27

```
#include "drivers/ph.h"

#include <math.h>

static const float kPhMin = 0.0f, kPhMax = 14.0f;

static inline PhDriver* asPh(SensorDriver* d) { return (PhDriver*)d; }

static PollResult finish(PhDriver* self, MonoMs now);

static SensorId phId(const SensorDriver* d) { (void)d; return SENSOR_PH; }
static const char* phName(const SensorDriver* d) { (void)d; return "ph4502c"; }
static uint8_t phMetricCount(const SensorDriver* d) { (void)d; return 1; }

static DriverStatus phInit(SensorDriver* d) {
  PhDriver* self = asPh(d);
  self->initialised_ = false;
  self->busy_ = false;
  if (self->config_.adc == 0 || self->config_.dividerBottomOhm == 0) return
  DRIVER_FAULT;
  // Throwaway conversion: fails if the channel was never begun on an ADC1 pin,
  // and keeps any one-time ADC characterisation cost out of poll().
  uint32_t mv = 0;
  bool saturated = false;
  if (!adcChannelReadMilliVolts(self->config_.adc, &mv, &saturated)) return
  DRIVER_FAULT;
  self->initialised_ = true;
  return DRIVER_OK;
}
```

The cast and identity helpers establish the concrete instance and generic metadata. Init clears current execution flags, rejects a missing ADC pointer and performs a test conversion.

This confirms the adapter can be used under its contract. It does not establish buffer calibration, temperature compensation or probe immersion. Those are different claims from initialised_ being true.

Accumulate evidence across one new window

src/drivers/ph.c / lines 28-52

```
28
29 // Take one sample of the window at `now`. false on an ADC error.
30 static bool sample(PhDriver* self, MonoMs now) {
31   uint32_t mv = 0;
32   bool saturated = false;
33   if (!adcChannelReadMilliVolts(self->config_.adc, &mv, &saturated)) return false;
34   self->samples_[self->count_++] = mv;
35   self->lastSampleMs_ = now;
36   self->saturated_ = self->saturated_ || saturated;
37   return true;
38 }
39
40 static DriverStatus phStart(SensorDriver* d, MonoMs now) {
41   PhDriver* self = asPh(d);
42   if (!self->initialised_) return DRIVER_FAULT;
43   self->count_ = 0;
44   self->saturated_ = false;
45   self->startMs_ = now;
46   if (!sample(self, now)) {
47     self->initialised_ = false;
48     return DRIVER_FAULT;
49   }
50   self->busy_ = true;
51   return DRIVER_OK;
52 }
```

sample stores a pin-voltage tap, advances count, records actual time and retains any clipping flag. Start resets the window and takes its first tap before accepting the operation.

A failed first conversion ends that attempt and clears initialisation. A successful start leaves persistent state for later polls; its local pointer is only an access path to that state, not a new driver copy.

Delay future taps after a late one

src/drivers/ph.c / lines 53-79

```
53
54 static PollResult phPoll(SensorDriver* d, MonoMs now) {
55   PhDriver* self = asPh(d);
56   PollResult r = {DRIVER_FAULT, now};
57   if (!self->busy_) return r;
58
59   // Spacing is measured from the previous actual sample, never from the window
60   // start: after a late poll, catching up would compress the filter taps.
61   const MonoMs due = self->lastSampleMs_ + PH_SAMPLE_SPACING_MS;
62   if (now < due) {
63     r.status = DRIVER_PENDING;
64     r.nextPollMs = due;
65     return r;
66   }
67   if (!sample(self, now)) {
68     self->busy_ = false;
69     self->initialised_ = false;
70     self->result_.health = HEALTH_FAULT;
71     self->result_.intervalStartMs = self->startMs_;
72     self->result_.intervalEndMs = now;
73     return r;
74   }
75   if (self->count_ < PH_SAMPLES) {
76     r.status = DRIVER_PENDING;
77     r.nextPollMs = self->lastSampleMs_ + PH_SAMPLE_SPACING_MS;
78     return r;
79   }
```

Poll compares now with the last actual sample time plus 30ms. It does not try to recreate missed taps by taking an immediate burst. A failed tap records a fault interval; an incomplete window returns the next useful timestamp.

src/drivers/ph.c / lines 80-81

```
80   return finish(self, now);
81 }
```

After the tenth sample, finish becomes responsible for filter, divider recovery and physical interpretation. The polling function itself does not compare a pH limit or write the relay.

Recover voltage before classifying an unusable output

src/drivers/ph.c / lines 82-106

```
82
83 static PollResult finish(PhDriver* self, MonoMs now) {
84   self->busy_ = false;
85   const PhConfig* cfg = &self->config_;
86   const float pinMv = trimmedMeanU32(self->samples_, PH_SAMPLES, PH_TRIM);
87   const float moduleMv = dividerRecoverMv(pinMv, cfg->dividerTopOhm,
 >   cfg->dividerBottomOhm);
88
89   Reading* r = &self->result_;
90   r->intervalStartMs = self->startMs_;
91   r->intervalEndMs = now;
92   r->hasRaw = true;
93   r->raw = (int32_t)(pinMv + 0.5f);
94   r->qualityFlags = self->saturated_ ? kQualitySaturated : kQualityNone;
95   r->calibrationRevision = cfg->calibration.revision;
96
97   // READY in every case below: health carries the verdict, raw the level.
98   PollResult pr = {DRIVER_READY, now};
99   if (pinMv < (float)PH_NO_SIGNAL_PIN_MV) {
100    r->health = HEALTH_UNPOWERED;
101    return pr;
102  }
103  if (self->saturated_) {
104    r->health = HEALTH_FAULT;
105    return pr;
106  }
```

Finish computes a trimmed pin voltage and recovers module voltage using the configured divider. It retains raw pin millivolts and the calibration revision before choosing a health branch.

The no-signal threshold and saturation are checked before numerical pH acceptance. A clipped waveform cannot be repaired by a line fit. The no-signal classification is the driver's heuristic for this configured analog path; acquisition separately checks the known rail history.

Preserve the distinction between fit availability and trust

src/drivers/ph.c / lines 107-134

```
107
108   if (!cfg->calibration.valid) {
109     r->unit = UNIT_MILLIVOLT;
110     r->value = moduleMv;
111     r->health = HEALTH_UNCALIBRATED;
112     r->lastGoodMs = now;  // last usable result (OK or UNCALIBRATED), as the SGP30
113     ++r->sequence;  // a fresh sample, in raw units
114     return pr;
115   }
116   const float ph = linearApply(&cfg->calibration, moduleMv);
117   r->unit = UNIT_PH;
118   if (!isfinite(ph)) {  // a broken fit is a fault, never a value
119     r->health = HEALTH_FAULT;
120     return pr;
121   }
122   if ((ph < kPhMin || ph > kPhMax) && !cfg->calibrationProvisional) {
123     r->health = HEALTH_FAULT;
124     return pr;
125   }
126   // Out of range on the provisional (nominal) fit: the fit may simply not
127   // match this probe yet. Published UNCALIBRATED (never actionable), so the
128   // Calibration page can still capture it and fix the fit.
129   r->value = ph;
130   ++r->sequence;
131   r->lastGoodMs = now;
132   if (cfg->calibrationProvisional) {  // pH units, but not buffer-verified (D13)
133     r->health = HEALTH_UNCALIBRATED;
134     return pr;
```

With no valid fit, the driver publishes module millivolts and UNCALIBRATED. Otherwise it evaluates pH and rejects non-finite arithmetic. A non-provisional fit must also stay inside the allowed pH range.

src/drivers/ph.c / lines 135-138

```
135   }
136   r->health = HEALTH_OK;
137   return pr;
138 }
```

A provisional nominal fit can publish even an out-of-range numerical estimate as UNCALIBRATED so the calibration workflow can inspect its raw input and replace the fit. The point is visibility for calibration, not permission for a numeric control rule to act on it.

Copy only a completed outcome and discard partial work

src/drivers/ph.c / lines 139-154

```
139
140 static uint8_t phReadResult(SensorDriver* d, Reading* out, uint8_t maxOut) {
141   if (maxOut < 1) return 0;
142   out[0] = asPh(d)->result_;
143   return 1;
144 }
145
146 static void phCancel(SensorDriver* d) {
147   PhDriver* self = asPh(d);
148   self->busy_ = false;
149   self->count_ = 0;
150 }
151
152 // The ADC pin is a high-impedance input behind the divider: nothing to drive
153 // before the 5 V rail goes down. Drop any partial window.
154 static void phPreparePowerOff(SensorDriver* d) { phCancel(d); }
```

ReadResult copies the retained observation when capacity permits. Cancel clears busy and sample count. PreparePowerOff uses that cancellation because the ADC signal is an input, not a line the driver should drive into an unpowered module.

These paths preserve the lifecycle boundary: no later poll should finish the abandoned partial window. The service will decide whether the published record should become UNPOWERED or TIMEOUT for the wider system event.

Construct the instance and copy a coherent replacement fit

src/drivers/ph.c / lines 155-180

```
155
156 static const SensorDriverOps kPhOps = {
157     .id = phId,
158     .name = phName,
159     .metricCount = phMetricCount,
160     .init = phInit,
161     .start = phStart,
162     .poll = phPoll,
163     .readResult = phReadResult,
164     .cancel = phCancel,
165     .preparePowerOff = phPreparePowerOff,
166 };
167
168 void phDriverConstruct(PhDriver* self, const PhConfig* config) {
169   self->base.ops = &kPhOps;
170   self->config_ = *config;
171   self->initialised_ = false;
172   self->busy_ = false;
173   self->saturated_ = false;
174   self->count_ = 0;
175   self->startMs_ = 0;
176   self->lastSampleMs_ = 0;
177   readingReset(&self->result_, SENSOR_PH, METRIC_PH, UNIT_PH);
178   self->result_.calibrationRevision = config->calibration.revision;
179 }
180
```

The table supplies the generic callbacks. Construction resets state and one pH result, while retaining the selected calibration revision. SetCalibration copies both the line and its provisional status.

src/drivers/ph.c / lines 181-184

```
181 void phDriverSetCalibration(PhDriver* self, const LinearCalibration* calibration, bool
  >   provisional) {
182   self->config_.calibration = *calibration;
183   self->config_.calibrationProvisional = provisional;
184 }
```

Those two changes belong together: the same coefficients may have different trust after actual reference work. The setter does not serialise competing tasks; the acquisition owner must apply the pair at its declared boundary.

Separate placeholders from measured calibration points

This experiment makes the module-voltage input visible beside the pH estimate. You can therefore inspect a fit without losing sight of what the ADC actually observed.

The procedure in the source is a particular commissioning workflow, not values already measured for a new probe.

test/bringup/ph_main.c / lines 1-25

```
// Bring-up entry point for ph (WP5).  pio run -e test_ph -t upload -t monitor
// Switched 5 V rail must be ON. PO -> 10k/15k -> GPIO7 (ADC1_CH6).
// Replaces src/main.c in this env only; links the production driver.
//
// Procedure (README "pH-4502C calibration"). Every reading is followed by a
// "module NNNN mV" line: the module PO voltage the buffer fit is made from,
// whatever the pH line shows (nominal fit, buffer fit or none).
//  1. Offset: probe off, BNC shorted. Turn the pot nearest the BNC until the
//     module line reads ~2500 mV - that is the module PO voltage (pH 7).
//     A factory-set module starts near 0 V and prints "ph=UNPOWERED": check
//     the 5 V rail, then keep turning the pot until mV values appear.
//  2. Probe in, rinse, pH 6.86 buffer, wait >= 2 min, note the module mV.
//     Rinse, pH 4.01 buffer, wait >= 2 min, note the module mV.
//  3. Put both mV values below and re-flash: readings switch to pH / OK.
// Without buffers: pH comes from the nominal fit below and prints "(uncal)".
#include <Arduino.h>
#include <stdio.h>

#include "app_config.h"
#include "board_profile.h"
#include "bringup_harness.h"
#include "hal/adc_channel.h"
#include "domain/calibration.h"
#include "drivers/ph.h"

```

The buffer millivolt macros are initially zero, meaning not yet measured. The alternative nominal fit uses a recorded offset and tutorial slope. Neither placeholder becomes a new buffer measurement because the program is compiled.

test/bringup/ph_main.c / lines 26-34

```
// Module mV measured in each buffer (step 2). 0 = not measured yet: uncalibrated.
#define PH_BUFFER_A_PH 6.86f
#define PH_BUFFER_A_MV 0.0f
#define PH_BUFFER_B_PH 4.01f
#define PH_BUFFER_B_MV 0.0f
#define PH_CALIBRATION_REVISION 1

// Nominal fit, used until both buffer values are set. Offset: module mV with the
// BNC shorted (pH 7), measured 17 Oct 2026. Re-measure if the offset pot moves.
```

Separate placeholders from measured calibration points / continued

test/bringup/ph_main.c / lines 35-40

```
35 // Slope: tutorial sketch, pH = -5.70 * V. Accuracy roughly +/-0.5-1 pH.
36 #define PH_NOMINAL_MV_AT_PH7 2758.0f
37 #define PH_NOMINAL_PH_PER_MV (-0.0057f)
38
39 static AdcChannel s_adc = ADC_CHANNEL_INIT;
40 static PhConfig s_config = {&s_adc, BOARD_PH_DIVIDER_TOP_OHM,
 >   BOARD_PH_DIVIDER_BOTTOM_OHM,
```

s_config stores the divider and an initially invalid fit; setup chooses the actual policy before construction. If the analog offset is changed, any calibration tied to the previous voltage relationship must be revisited.

Inspect raw voltage through the same driver owner

test/bringup/ph_main.c / lines 41-49

```
                              {1.0f, 0.0f, 0, false}, false};  // set in setup()
static PhDriver s_driver;
// The module mV for the procedure, from the pin mV the driver publishes as raw.
static void printModuleMv(SensorDriver* driver) {
  Reading r;
  if (sensorDriverReadResult(driver, &r, 1) != 1 || !r.hasRaw) return;
  printf("    module %.0f mV\n",
         (double)dividerRecoverMv((float)r.raw, BOARD_PH_DIVIDER_TOP_OHM,
  BOARD_PH_DIVIDER_BOTTOM_OHM));
}
```

The callback requests one copied result, checks raw availability and recovers module millivolts for printing. It does not call the ADC independently.

That keeps one observation path: the pH line and the module-voltage line come from the same filtered result. A parallel ADC read could observe a different instant and confuse calibration comparisons.

Choose a measured two-point fit or a provisional fallback

test/bringup/ph_main.c / lines 50-69

```
50
51 static BringupContext s_ctx = {&s_driver.base, CFG_FRAME_PERIOD_MS, printModuleMv};
52
53 void setup(void) {
54   if (PH_BUFFER_A_MV > 0.0f && PH_BUFFER_B_MV > 0.0f) {
55     linearFromTwoPoints(PH_BUFFER_A_MV, PH_BUFFER_A_PH, PH_BUFFER_B_MV, PH_BUFFER_B_PH,
56                         PH_CALIBRATION_REVISION, &s_config.calibration);
57   } else {
58     s_config.calibration.slope = PH_NOMINAL_PH_PER_MV;
59     s_config.calibration.offset = 7.0f - PH_NOMINAL_PH_PER_MV * PH_NOMINAL_MV_AT_PH7;
60     s_config.calibration.revision = 0;
61     s_config.calibration.valid = true;
62     s_config.calibrationProvisional = true;
63   }
64   phDriverConstruct(&s_driver, &s_config);
65   delay(200);
66   printf("\n=== bring-up: %s ===\n", sensorDriverName(&s_driver.base));
67   if (!adcChannelBegin(&s_adc, BOARD_PIN_PH_ADC)) {
68     printf("GPIO%d is not an ADC1 pin\n", (int)BOARD_PIN_PH_ADC);
69   }
```

When both buffer voltages are positive, setup calls the general two-point fit maker. Otherwise it builds the nominal line and sets provisional true. Construction then copies the selected configuration.

The test does not check the fit-maker boolean directly, but a failed maker leaves its calibration invalid. The later print path exposes that state. In a user-facing command path, explicit error reporting is stronger than relying only on a later diagnostic line.

Report which interpretation the experiment is using

test/bringup/ph_main.c / lines 70-84

```
  if (s_config.calibrationProvisional) {
    printf("nominal pH (not buffer-calibrated): pH 7 at %.0f mV, %.4f pH/mV\n",
           (double)PH_NOMINAL_MV_AT_PH7, (double)PH_NOMINAL_PH_PER_MV);
  } else if (s_config.calibration.valid) {
    // Tutorial nominal is about -0.0057 pH/mV; far from it means a bad buffer reading.
    printf("calibration rev %u: pH = %.6f * mV + %.3f\n",
           (unsigned)s_config.calibration.revision, (double)s_config.calibration.slope,
           (double)s_config.calibration.offset);
  } else {
    printf("uncalibrated: printing module PO voltage in mV\n");
  }
  bringupStart(&s_ctx);
}

void loop(void) { vTaskDelay(portMAX_DELAY); }
```

After ADC setup, the source prints whether it is using nominal pH, a valid fitted revision or raw module voltage. These labels explain the unit and health you should expect from the harness.

The shared task then performs repeated windows. Observe settling and record reference voltage under the procedure's conditions; do not use an instant after probe movement as though the physical signal had already stabilised.

Follow the polynomial with units intact

Original firmware - src/domain/tds_codec.c, lines 3-13

```
float tdsCompensationCoefficient(float waterTempC) {
  return 1.0f + 0.02f * (waterTempC - 25.0f);
}

float tdsPpmFromModuleMv(float moduleMv, float coefficient) {
  if (moduleMv <= 0.0f || coefficient <= 0.0f) return 0.0f;
  const float v = moduleMv / 1000.0f;
  const float ppm = (133.42f * v * v * v - 255.86f * v * v +
>   857.39f * v) * 0.5f;
  return ppm / coefficient;  // compensate the conductivity,
>   not the voltage
}
```

The input is module millivolts. Dividing by 1000.0 changes it to volts before the polynomial. For moduleMv=1000 and coefficient=1, v=1. The polynomial becomes (133.42-255.86+857.39) x 0.5 = 367.475ppm.

For a water temperature of 30 degrees Celsius, the project coefficient is 1 + 0.02 x (30-25) = 1.10. The reported nominal estimate is then 367.475/1.10 = 334.07ppm. The source divides the polynomial result by the temperature coefficient.

The placement matters. A polynomial is nonlinear: applying it to v/1.10 generally does not equal applying it to v and dividing the result by 1.10. Do not move compensation to another point in the expression without checking the intended model.

The source uses a nominal conversion plus an optional fitted gain. TDS here is an estimate based on the electrical response and the selected model. It does not identify individual dissolved ions, and its calibration is not automatically transferable to every solution.

This is a useful file to test on your PC: it has no ADC access and no RTOS calls. Known numeric inputs should produce known outputs. That is the benefit of keeping the arithmetic in domain/ while the driver handles timing and validity.

Reference: DFRobot: SEN0244 reference

COMMON PITFALL

Feeding 1000 as volts rather than millivolts makes the cubic term enormous. Parameter units are part of the function's contract.

Reject a calculation with a bad dependency

Original firmware - src/drivers/tds.c, lines 85-99

```
static bool temperatureUsable(const TdsDriver* self, MonoMs
>   now) {
  const Reading* t = &self->waterTemperature_;
  return readingIsActionableAt(t, now, CFG_STALE_ORDINARY_MS) &&
         t->value >= TDS_MIN_WATER_TEMP_C && t->value <=
>   TDS_MAX_WATER_TEMP_C;
}

static PollResult finish(TdsDriver* self, MonoMs now) {
  self->busy_ = false;
  const TdsConfig* cfg = &self->config_;
  const float pinMv = trimmedMeanU32(self->samples_,
>   TDS_SAMPLES, TDS_TRIM);  // = module mV

  const bool compensated = temperatureUsable(self, now);
  const float coefficient =
      compensated ?
>   tdsCompensationCoefficient(self->waterTemperature_.value) :
>   1.0f;
  float ppm = tdsPpmFromModuleMv(pinMv, coefficient);
```

temperatureUsable checks both freshness and the allowed temperature range. It does not merely ask whether a float exists. The driver has a copied water-temperature record from prepareInputs(), so its own finish time decides whether that dependency is still acceptable.

If the dependency is not usable, the implementation can still calculate a nominal number with coefficient 1.0, but it labels the outcome DEPENDENCY_INVALID. That number may be useful for diagnostics. It must not quietly become an ordinary calibrated control input.

REMEMBER

Compensation is a dependency on another observation, not just an extra multiplication.

Reject a calculation with a bad dependency / continued

After the calculation, the driver rejects non-finite results, saturation and estimates beyond its declared full-scale range. With a usable dependency but no valid reference calibration, it publishes UNCALIBRATED. Only the accepted, calibrated path reaches OK.

Take two cases with the same 1000mV input. Case A has an OK 25-degree Reading from 200ms ago. Case B has a FAULT temperature record whose old value field still says 25.0. Case A can use coefficient 1.0 as a valid dependency. Case B cannot: the number alone is the same, but the evidence is different.

This is the architectural reason for carrying the complete Reading between modules. A pointer to a float would discard the age and health needed to distinguish those cases.

The ordinary stale limit in this snapshot is 3000ms. The driver checks it at the time of use.

Know which faults the signal cannot reveal

At 0mV, the polynomial returns zero. A low-conductivity solution can produce a low output, but a disconnected or unpowered analog path may also produce one. A voltage-only interface may not contain enough information to distinguish them.

Do not invent a universal rule that every zero voltage is a broken probe. Conversely, do not claim that every in-range voltage proves a healthy probe. The driver can check clipping, finite arithmetic, rated range, temperature dependency and known rail validity. It cannot directly prove that the electrodes are immersed or connected correctly in every failure condition.

Use a fault experiment to establish observability

During isolated bring-up, record the output in a known solution, with the probe removed, with the signal unplugged, and with module power removed. Observe whether the conditions produce distinct levels or time patterns. A firmware diagnostic needs a distinguishable signal to classify those cases reliably.

If two conditions overlap, document the limitation or improve the electrical interface. A more complicated if statement cannot recover missing evidence. This is one reason the project also records rail status and retained health rather than reducing everything to a single ppm field.

For reference calibration, a gain-only model compares a known reference ppm with the nominal prediction. If the reference is 500ppm and the nominal prediction is 400ppm, gain=1.25. The policy constrains the allowed gain. After applying it, test a different reference point instead of treating a perfect fit at the first point as full validation.

COMMON PITFALL

A flat, stable trace can mean a stable solution, a clipped input, a disconnected input or a frozen acquisition path. Use sequence, timing and fault tests together.

Name the input scale before using the formula

This interface isolates the nominal TDS model and its accepted temperature and full-scale limits.

The driver supplies filtered millivolts and a validated temperature coefficient.

src/domain/tds_codec.h / lines 1-27

```
// SEN0244 TDS conversion (WP5). Pure C, host-tested.
// Source: DFRobot Gravity Analog TDS Sensor SEN0244 (local copy in
// Electrical System Study/Datasheets/SEN0244.PDF) for the curve, and DFRobot's
// GravityTDS library for where the temperature compensation is applied:
//   ppm(T)      = (133.42 V^3 - 255.86 V^2 + 857.39 V) * 0.5
//   coefficient = 1 + 0.02 * (T - 25)
//   ppm25       = ppm(T) / coefficient
// The datasheet's sample sketch divides the VOLTAGE by the coefficient
// instead. Because the curve is not linear, that over-corrects: on 24 Oct 2026
// the same tap water at 11.4 / 33.6 / 11.6 degC gave 683 / 584 / 713 ppm with
// the sketch's method and 631 / 620 / 653 ppm compensating the ppm (EC), as
// the library does. Conductivity is what changes by ~2 %/degC.
// The sample's VREF/1024 scaling is not ported: the ADC HAL already returns
// calibrated millivolts, and the module output reaches the pin undivided.
#pragma once

#define TDS_FULL_SCALE_PPM 1000.0f   // rated range 0-1000 ppm, +/-10 % F.S.
#define TDS_MIN_WATER_TEMP_C 0.0f    // below: frozen water, not a usable input
#define TDS_MAX_WATER_TEMP_C 55.0f   // datasheet probe limit

// Temperature compensation coefficient, 1.0 at 25 degC.
float tdsCompensationCoefficient(float waterTempC);

// Module output (mV) to TDS (ppm) referred to 25 degC: the curve's ppm divided
// by `coefficient` (1.0 = no compensation). 0 for a non-positive voltage or
// coefficient.
float tdsPpmFromModuleMv(float moduleMv, float coefficient);
```

The declarations distinguish the water-temperature coefficient from the millivolt-to-ppm model. The limits belong to the selected sensor/model assumptions. A float parameter does not carry its unit mechanically, so the names and caller must keep millivolts and degrees straight.

The codec can calculate a value for supplied numbers without knowing whether they came from fresh sensors. The driver makes that acceptance decision around the arithmetic.

Compensate the model output at the intended stage

Read the entire nominal conversion with the lesson's 1000mV example.

The optional fitted gain is applied by the driver after this helper returns.

src/domain/tds_codec.c / lines 1-12

```c
#include "domain/tds_codec.h"

float tdsCompensationCoefficient(float waterTempC) {
  return 1.0f + 0.02f * (waterTempC - 25.0f);
}

float tdsPpmFromModuleMv(float moduleMv, float coefficient) {
  if (moduleMv <= 0.0f || coefficient <= 0.0f) return 0.0f;
  const float v = moduleMv / 1000.0f;
  const float ppm = (133.42f * v * v * v - 255.86f * v * v + 857.39f * v) * 0.5f;
  return ppm / coefficient;  // compensate the conductivity, not the voltage
}
```

At 25 degrees the coefficient is 1.0; at 30 it is 1.10. The polynomial first converts module millivolts to volts, produces the nominal ppm estimate, then divides that estimate by the coefficient.

A cubic is nonlinear, so changing the input voltage before the polynomial is not generally equivalent. The guards return zero for nonpositive voltage or coefficient; they do not by themselves provide an observation-health verdict. The caller must still reject invalid configuration and dependencies.

Read the order of failure meanings

The TDS object adds a retained water-temperature record to the analog window you already understand. That extra record is the architectural difference to watch.

The temperature setter copies data supplied by acquisition and never touches the DS18B20 bus.

src/drivers/tds.h / lines 1-28

```
// sen0244_tds driver. Work package WP5 (Stage 3). Architecture §8, §14.
// Source: DFRobot Gravity Analog TDS Sensor SEN0244 datasheet (wiki + sample).
//
// Switched 5 V rail. A -> GPIO6 (ADC1_CH5), 100 nF to GND, no divider: pin mV
// = module mV. Output 0-2.3 V nominal, inside the S3's ~3.1 V range at 11 dB.
// Pin mV -> datasheet cubic -> temperature compensation of the ppm (EC, as
// DFRobot GravityTDS; domain/tds_codec.h)
// -> optional K-factor (calibration: x = nominal ppm, y = ppm) -> ppm.
//
// Filter: TDS_SAMPLES conversions spaced TDS_SAMPLE_SPACING_MS apart, sorted,
// mean of the middle TDS_SAMPLES - 2*TDS_TRIM (stands in for the sample's
// median). The sample takes 30 taps 40 ms apart (~1.2 s), which overruns the
// 1 s frame, so the window is shortened to ~560 ms. Spread over successive
// poll() calls, never a burst (§9), spacing measured from the previous tap.
//
// Temperature compensation needs a fresh DS18B20 reading via
// tdsDriverSetWaterTemperature(): OK and not older than CFG_STALE_ORDINARY_MS
// at the end of the window, within 0-55 degC (probe limit). Otherwise the
// value is computed uncompensated - never silently assumed to be 25 degC.
//
// Results (unit ppm, raw = filtered pin mV in every case), highest first:
//   any tap clipped or ppm > 1000 -> kQualitySaturated + HEALTH_FAULT (value
//                         kept; above the rated range, never actionable)
//   no valid temperature  -> kQualityUncompensated + HEALTH_DEPENDENCY_INVALID
//   !calibration.valid    -> HEALTH_UNCALIBRATED (nominal datasheet curve, D13)
//   otherwise             -> HEALTH_OK
// poll() returns READY for all of these; the health carries the verdict.
//
```

Read the order of failure meanings / continued

The contract gives clipping or excessive ppm priority over missing temperature, then distinguishes a nominal uncalibrated model from a reference-calibrated result. The raw voltage stays available across those verdicts for diagnosis.

src/drivers/tds.h / lines 29-32

```
// Not detectable here: 0 V is distilled water, a probe in air, an open lead or
// a dead rail alike, so there is no UNPOWERED threshold. Rail state is left to
// acquisition (rail epoch, WP9).
//
```

It also says why there is no zero-voltage UNPOWERED threshold: several physical conditions can produce the same low signal. Known rail state is therefore handled above the driver rather than inferred from an indistinguishable analog level.

Keep a stable dependency for this calculation

src/drivers/tds.h / lines 33-60

```
33 // Worst-case blocking (one ADC conversion C, well under 1
 >   ms):
34 //   init()    1 C   (throwaway conversion, checks the
 >   channel)
35 //   start()   1 C   (first sample of the window)
36 //   poll()    1 C   Pending until the window is full (>=
 >   560 ms)
37 //   others    none
38 //
39 // Implements the SensorDriver contract in
 >   drivers/sensor_driver.h.
40 #pragma once
41 #include "drivers/sensor_driver.h"
42 #include "hal/adc_channel.h"
43 #include "domain/calibration.h"
44
45 #define TDS_SAMPLES 15
46 #define TDS_TRIM 5
47 #define TDS_SAMPLE_SPACING_MS 40
48
49 typedef struct TdsConfig {
50   AdcChannel* adc;                // already begun on
 >   BOARD_PIN_TDS_ADC
51   LinearCalibration calibration;  // K-factor: x = nominal
 >   ppm, y = ppm
52 } TdsConfig;
53
54 // Members after `base` are private to tds.c.
55 typedef struct TdsDriver {
56   SensorDriver base;  // must stay first
57   TdsConfig config_;
58   bool initialised_;
59   bool busy_;
60   bool saturated_;
```

Fifteen taps 40ms apart span at least 560ms; trimming five from each end leaves five values for the mean. The state object retains the sample window and a complete Reading for water temperature.

src/drivers/tds.h / lines 61-66

```
61   uint8_t count_;
62   uint32_t samples_[TDS_SAMPLES];  // pin mV
63   MonoMs startMs_;
64   MonoMs lastSampleMs_;  // next tap is due
 >   TDS_SAMPLE_SPACING_MS after this
65   Reading waterTemperature_;
66   Reading result_;
```

COMMON PITFALL

A copied dependency becomes older while your own operation runs. Validate it at use, not only at assignment.

Keep a stable dependency for this calculation / continued

src/drivers/tds.h / lines 67-78

```
67 } TdsDriver;
68
69 // Constructor. Use &driver.base wherever a SensorDriver* is expected.
70 void tdsDriverConstruct(TdsDriver* self, const TdsConfig* config);
71
72 // Compensation input from the latest DS18B20 reading (copied, not shared).
73 // Checked for health and age when the window finishes; no ADC access here.
74 void tdsDriverSetWaterTemperature(TdsDriver* self, const Reading* waterTemperature);
75
76 // New K-factor from the applied configuration (on-device calibration, WP8).
77 // Called by the acquisition owner between windows; applies from the next one.
78 void tdsDriverSetCalibration(TdsDriver* self, const LinearCalibration* calibration);
```

That dependency includes health and time, not merely valueC. The setter can copy a previously completed result before the new TDS window starts, and finish can decide whether that same record is still fresh when it is used.

Establish a usable ADC channel

The early lifecycle resembles the other analog drivers. We will read it completely, then concentrate on the point where another sensor's evidence changes the outcome.

All dependency and calibration setters still belong to the single driver owner.

src/drivers/tds.c / lines 1-28

```
#include "drivers/tds.h"

#include <math.h>

#include "app_config.h"
#include "domain/tds_codec.h"

static inline TdsDriver* asTds(SensorDriver* d) { return (TdsDriver*)d; }

static PollResult finish(TdsDriver* self, MonoMs now);

static SensorId tdsId(const SensorDriver* d) { (void)d; return SENSOR_TDS; }
static const char* tdsName(const SensorDriver* d) { (void)d; return "sen0244_tds"; }
static uint8_t tdsMetricCount(const SensorDriver* d) { (void)d; return 1; }

static DriverStatus tdsInit(SensorDriver* d) {
  TdsDriver* self = asTds(d);
  self->initialised_ = false;
  self->busy_ = false;
  if (self->config_.adc == 0) return DRIVER_FAULT;
  // Throwaway conversion: fails if the channel was never begun on an ADC1 pin,
  // and keeps any one-time ADC characterisation cost out of poll().
  uint32_t mv = 0;
  bool saturated = false;
  if (!adcChannelReadMilliVolts(self->config_.adc, &mv, &saturated)) return
  DRIVER_FAULT;
  self->initialised_ = true;
  return DRIVER_OK;
}
```

The generic cast and metadata helpers identify this one-metric driver. Init clears active state, rejects a missing ADC pointer and performs a throwaway conversion before accepting later windows.

The optional calibration starts as a record in config_. It is not established by ADC initialisation, and a valid ADC result is not a reference solution measurement.

Retain taps and clipping across returns

src/drivers/tds.c / lines 29-53

```
29
30 // Take one sample of the window at `now`. false on an ADC error.
31 static bool sample(TdsDriver* self, MonoMs now) {
32   uint32_t mv = 0;
33   bool saturated = false;
34   if (!adcChannelReadMilliVolts(self->config_.adc, &mv, &saturated)) return false;
35   self->samples_[self->count_++] = mv;
36   self->lastSampleMs_ = now;
37   self->saturated_ = self->saturated_ || saturated;
38   return true;
39 }
40
41 static DriverStatus tdsStart(SensorDriver* d, MonoMs now) {
42   TdsDriver* self = asTds(d);
43   if (!self->initialised_) return DRIVER_FAULT;
44   self->count_ = 0;
45   self->saturated_ = false;
46   self->startMs_ = now;
47   if (!sample(self, now)) {
48     self->initialised_ = false;
49     return DRIVER_FAULT;
50   }
51   self->busy_ = true;
52   return DRIVER_OK;
53 }
```

sample appends one millivolt result and accumulates saturation. Start resets count, clipping and timing, then takes the first tap. The object persists while the task returns to other work.

The state machine ensures a later poll, not a second concurrent caller, supplies each next sample. Separate stacks would not protect samples_ if two owners advanced this same object.

Respect the next actual sample time

src/drivers/tds.c / lines 54-80

```
54
55 static PollResult tdsPoll(SensorDriver* d, MonoMs now) {
56   TdsDriver* self = asTds(d);
57   PollResult r = {DRIVER_FAULT, now};
58   if (!self->busy_) return r;
59
60   // Spacing is measured from the previous actual sample, never from the window
61   // start: after a late poll, catching up would compress the filter taps.
62   const MonoMs due = self->lastSampleMs_ + TDS_SAMPLE_SPACING_MS;
63   if (now < due) {
64     r.status = DRIVER_PENDING;
65     r.nextPollMs = due;
66     return r;
67   }
68   if (!sample(self, now)) {
69     self->busy_ = false;
70     self->initialised_ = false;
71     self->result_.health = HEALTH_FAULT;
72     self->result_.intervalStartMs = self->startMs_;
73     self->result_.intervalEndMs = now;
74     return r;
75   }
76   if (self->count_ < TDS_SAMPLES) {
77     r.status = DRIVER_PENDING;
78     r.nextPollMs = self->lastSampleMs_ + TDS_SAMPLE_SPACING_MS;
79     return r;
80   }
```

Poll returns early when no operation exists or the next 40ms spacing has not elapsed. ADC-helper failure ends the window and stamps a fault interval. A successful but partial window returns PENDING; a complete one calls finish.

src/drivers/tds.c / lines 81-82

```
81   return finish(self, now);
82 }
```

There is no delay loop inside the driver. The returned timestamp is the mechanism that lets an acquisition task wait efficiently while still servicing other drivers.

Choose compensation only from acceptable evidence

src/drivers/tds.c / lines 83-110

```
83
84 // True if the stored water temperature can drive compensation at `now`.
85 static bool temperatureUsable(const TdsDriver* self, MonoMs now) {
86   const Reading* t = &self->waterTemperature_;
87   return readingIsActionableAt(t, now, CFG_STALE_ORDINARY_MS) &&
88          t->value >= TDS_MIN_WATER_TEMP_C && t->value <= TDS_MAX_WATER_TEMP_C;
89 }
90
91 static PollResult finish(TdsDriver* self, MonoMs now) {
92   self->busy_ = false;
93   const TdsConfig* cfg = &self->config_;
94   const float pinMv = trimmedMeanU32(self->samples_, TDS_SAMPLES, TDS_TRIM);  // =
 >   module mV
95
96   const bool compensated = temperatureUsable(self, now);
97   const float coefficient =
98       compensated ? tdsCompensationCoefficient(self->waterTemperature_.value) : 1.0f;
99   float ppm = tdsPpmFromModuleMv(pinMv, coefficient);
100  if (cfg->calibration.valid) ppm = linearApply(&cfg->calibration, ppm);
101
102  Reading* r = &self->result_;
103  r->intervalStartMs = self->startMs_;
104  r->intervalEndMs = now;
105  r->hasRaw = true;
106  r->raw = (int32_t)(pinMv + 0.5f);
107  r->unit = UNIT_PPM;
108  r->value = ppm;
109  r->qualityFlags = compensated ? kQualityNone : kQualityUncompensated;
110  r->calibrationRevision = cfg->calibration.revision;
```

temperatureUsable checks the copied temperature's health, age and allowed physical range. Finish computes the filtered pin voltage, uses a coefficient only when that dependency passes, applies the nominal polynomial and then an optional calibration line.

The result retains raw voltage and tags uncompensated output explicitly. A number calculated with coefficient 1.0 because temperature was missing is not the same evidence as an accepted 25-degree dependency that legitimately gives coefficient 1.0.

Let the strongest rejection win before setting OK

src/drivers/tds.c / lines 111-135

```
111
112   // READY in every case below: health carries the verdict, raw the level.
113   PollResult pr = {DRIVER_READY, now};
114   if (!isfinite(ppm)) {  // a broken fit is a fault, never a value
115     r->health = HEALTH_FAULT;
116     return pr;
117   }
118   if (self->saturated_ || ppm > TDS_FULL_SCALE_PPM) {
119     r->qualityFlags |= kQualitySaturated;
120     r->health = HEALTH_FAULT;  // above the rated 0-1000 ppm: never actionable
121     return pr;
122   }
123   if (!compensated) {
124     r->health = HEALTH_DEPENDENCY_INVALID;
125     return pr;
126   }
127   ++r->sequence;
128   r->lastGoodMs = now;  // last usable result (OK or UNCALIBRATED)
129   if (!cfg->calibration.valid) {  // nominal datasheet curve, no reference check (D13)
130     r->health = HEALTH_UNCALIBRATED;
131     return pr;
132   }
133   r->health = HEALTH_OK;
134   return pr;
135 }
```

Non-finite arithmetic returns FAULT. Clipping or a value beyond full scale sets the saturation qualification and FAULT. Missing temperature returns DEPENDENCY_INVALID before a nominal or calibrated value can be accepted.

Only a usable dependency advances the usable-result history. Without a valid reference fit the result remains UNCALIBRATED; otherwise it becomes OK. Those ordered returns explain why simply looking at the ppm float cannot reconstruct the measurement's eligibility.

End a window without driving its analog input

src/drivers/tds.c / lines 136-151

```
136
137 static uint8_t tdsReadResult(SensorDriver* d, Reading* out, uint8_t maxOut) {
138   if (maxOut < 1) return 0;
139   out[0] = asTds(d)->result_;
140   return 1;
141 }
142
143 static void tdsCancel(SensorDriver* d) {
144   TdsDriver* self = asTds(d);
145   self->busy_ = false;
146   self->count_ = 0;
147 }
148
149 // The ADC pin is a high-impedance input (100 nF to GND): nothing to drive
150 // before the 5 V rail goes down. Drop any partial window.
151 static void tdsPreparePowerOff(SensorDriver* d) { tdsCancel(d); }
```

ReadResult copies one retained Reading when capacity permits. Cancel drops busy and count. PreparePowerOff delegates to cancel because the input has no output control line to make safe.

The service can independently label this observation UNPOWERED when a rail epoch changed. This division prevents the ADC driver from guessing a known power event from voltage alone.

Construct both result identities and update dependencies by copy

src/drivers/tds.c / lines 152-178

```
152
153 static const SensorDriverOps kTdsOps = {
154     .id = tdsId,
155     .name = tdsName,
156     .metricCount = tdsMetricCount,
157     .init = tdsInit,
158     .start = tdsStart,
159     .poll = tdsPoll,
160     .readResult = tdsReadResult,
161     .cancel = tdsCancel,
162     .preparePowerOff = tdsPreparePowerOff,
163 };
164
165 void tdsDriverConstruct(TdsDriver* self, const TdsConfig* config) {
166   self->base.ops = &kTdsOps;
167   self->config_ = *config;
168   self->initialised_ = false;
169   self->busy_ = false;
170   self->saturated_ = false;
171   self->count_ = 0;
172   self->startMs_ = 0;
173   self->lastSampleMs_ = 0;
174   readingReset(&self->waterTemperature_, SENSOR_DS18B20, METRIC_WATER_TEMPERATURE,
  >   UNIT_DEG_C);
175   readingReset(&self->result_, SENSOR_TDS, METRIC_TDS, UNIT_PPM);
176   self->result_.calibrationRevision = config->calibration.revision;
177 }
178
```

Construction installs the operations table, initialises the window and resets both waterTemperature_ and the TDS result. The dependency begins STARTING rather than silently assuming 25 degrees.

src/drivers/tds.c / lines 179-185

```
179 void tdsDriverSetWaterTemperature(TdsDriver* self, const Reading* waterTemperature) {
180   self->waterTemperature_ = *waterTemperature;
181 }
182
183 void tdsDriverSetCalibration(TdsDriver* self, const LinearCalibration* calibration) {
184   self->config_.calibration = *calibration;
185 }
```

The two setters copy complete records: one for a measured dependency and one for a calibration fit. Neither fetches the data itself or acquires a bus. Ownership and timing of these calls remain visible in acquisition.

Create a nominal driver with no invented temperature

The isolated TDS experiment deliberately begins without a water sensor. That makes the dependency-invalid path observable instead of hiding it with a default temperature.

A manual thermometer input in this experiment is a test substitution, not a production DS18B20 reading.

test/bringup/tds_main.c / lines 1-28

```
// Bring-up entry point for tds (WP5).  pio run -e test_tds -t upload -t monitor
// Switched 5 V rail must be ON. A -> GPIO6 (ADC1_CH5), 100 nF to GND.
// Replaces src/main.c in this env only; links the production driver.
//
// Procedure (README "SEN0244 TDS bring-up"):
//  1. Probe in the liquid, away from the container wall (datasheet), stir
//     gently and wait for the reading to settle. Water below 55 degC only.
//  2. Without a water temperature every line is "tds=DEPENDENCY_INVALID
//     (raw <pin mV>)": this env runs the TDS driver alone, without the DS18B20.
//  3. Measure the water with a thermometer, put it in TDS_MANUAL_WATER_TEMP_C
//     and re-flash: lines switch to "<ppm> ppm (uncal)" (nominal curve).
// "tds=FAULT (raw ...)" means above 1000 ppm (raw >~ 2160 mV at 25 degC), not
// a wiring fault: an ADC error shows as "poll -> FAULT" instead.
#include <Arduino.h>
#include <stdio.h>

#include "app_config.h"
#include "board_profile.h"
#include "bringup_harness.h"
#include "hal/adc_channel.h"
#include "hal/clock.h"
#include "drivers/tds.h"

// Water temperature from a thermometer, degC. 0 = off (no compensation).
// BRING-UP ONLY: stands in for the DS18B20 and is re-stamped as fresh every
// frame. Production feeds the real DS18B20 reading.
#define TDS_MANUAL_WATER_TEMP_C 0.0f

```

Create a nominal driver with no invented temperature / continued

The manual temperature macro is zero to disable substitution. The configuration has no valid K-factor, so even an accepted compensated model would remain UNCALIBRATED.

test/bringup/tds_main.c / lines 29-33

```
29 static AdcChannel s_adc = ADC_CHANNEL_INIT;
30 static TdsConfig s_config = {&s_adc, {1.0f, 0.0f, 0, false}};  // nominal curve, no
 >   K-factor
31 static TdsDriver s_driver;
32
33 // Feed the manual temperature as a fresh OK DS18B20 reading.
```

The source procedure explains expected NO TEMP and over-range outcomes. A fault label can mean a measured model limit was exceeded, not necessarily that an electrical wire broke. Use raw voltage and the returned operation status to distinguish those paths.

Make the test substitution explicit in the record

test/bringup/tds_main.c / lines 34-45

```
34 static void feedManualTemperature(SensorDriver* driver) {
35   (void)driver;
36   Reading t;
37   readingReset(&t, SENSOR_DS18B20, METRIC_WATER_TEMPERATURE, UNIT_DEG_C);
38   const MonoMs now = monoNowMs();
39   t.health = HEALTH_OK;
40   t.value = TDS_MANUAL_WATER_TEMP_C;
41   t.intervalStartMs = now;
42   t.intervalEndMs = now;
43   t.lastGoodMs = now;
44   tdsDriverSetWaterTemperature(&s_driver, &t);
45 }
```

The helper resets a water-temperature Reading with DS18B20 identity, stamps the current time and marks the manually supplied value OK before copying it into TDS. It is a deliberate fake dependency for isolated bring-up.

Refreshing its timestamp every result keeps that test constant fresh. Doing this in production would conceal a stopped real sensor. The implementation is acceptable as an experiment only because its origin and scope are explicit.

Enable the optional substitution before repeated work

test/bringup/tds_main.c / lines 46-67

```
46
47 static BringupContext s_ctx = {&s_driver.base, CFG_FRAME_PERIOD_MS, NULL};
48
49 void setup(void) {
50   tdsDriverConstruct(&s_driver, &s_config);
51   delay(200);
52   printf("\n=== bring-up: %s ===\n", sensorDriverName(&s_driver.base));
53   if (!adcChannelBegin(&s_adc, BOARD_PIN_TDS_ADC)) {
54     printf("GPIO%d is not an ADC1 pin\n", (int)BOARD_PIN_TDS_ADC);
55   }
56   if (TDS_MANUAL_WATER_TEMP_C > 0.0f) {
57     printf("manual water temperature %.1f degC (bring-up only, not a DS18B20)\n",
58            (double)TDS_MANUAL_WATER_TEMP_C);
59     feedManualTemperature(&s_driver.base);
60     s_ctx.afterResult = feedManualTemperature;
61   } else {
62     printf("no water temperature: readings are DEPENDENCY_INVALID, raw = pin mV\n");
63   }
64   bringupStart(&s_ctx);
65 }
66
67 void loop(void) { vTaskDelay(portMAX_DELAY); }
```

Setup constructs the driver and ADC channel. If the manual value is enabled, it supplies an initial dependency and installs the refresh callback. Otherwise the driver continues reporting the missing dependency with raw data.

The harness then tests the same production window and conversion logic. A useful experiment compares the two modes at a known temperature, rather than judging an uncalibrated nominal ppm estimate against an unspecified solution.

Read the table as several kinds of evidence

This combined experiment is the first complete demonstration of two drivers advancing within one task. You have now read both drivers and their dependency contract, so the interleaving should be visible.

It is a bring-up scheduler for water probes, not the production acquisition engine.

test/bringup/water_main.c / lines 1-28

```
// Combined bring-up for the water probes: DS18B20 (WP2) + SEN0244 TDS (WP5).
//   pio run -e test_water -t upload -t monitor
// DS18B20 on GPIO14 (1-Wire, retained 3.3 V); TDS A on GPIO6 (ADC1_CH5,
// switched 5 V rail must be ON). Not the production scheduler (WP7): one task
// owns both buses and runs a 1 s frame. Each frame it passes the latest
// DS18B20 reading to the TDS driver (tdsDriverSetWaterTemperature), then runs
// both measurements interleaved: the DS18B20 converts for 750 ms while the
// TDS window (~560 ms) is sampled, so both fit one frame. TDS is therefore
// compensated with the previous frame's water temperature (~1 s old, inside
// the 3 s staleness limit); the first frame has none (NO TEMP).
//
// One table row per frame (header repeated every WATER_HEADER_EVERY rows):
//         | DS18B20 |  SEN0244 TDS                                  |
//    time | water T |    raw | uncomp |   @25C | state   | trend    |
//      12 |   24.81 |   1245 |    470 |    466 | UNCAL   | stable   |
//   water T  DS18B20 degC, or why not: FAULT, NOPRES (no presence pulse),
//            TIMEOUT (frame guard), "-" (no result yet)
//   raw      TDS pin mV (filtered window)
//   uncomp   ppm from raw with NO temperature compensation
//   @25C     ppm compensated to 25 degC (what production uses); "-" without
//            a fresh water temperature
//   state    OK, UNCAL (nominal curve, no K-factor, D13), NO TEMP (no fresh
//            water temperature), OVER (above 1000 ppm or ADC clipped), FAULT
//   trend    stable once the ppm shown (@25C, or uncomp when there is none)
//            has stopped drifting: over the last WATER_TREND_ROWS rows the mean
//            of the newer half is within 0.4 % (min 2 ppm) of the older half,
//            and no row is more than 2 % off. Otherwise settling. Judged on
//            @25C, comparing half-means, because the DS18B20 creeps towards
```

The source introduces raw voltage, uncompensated model output, temperature-compensated output, health and a settling indicator. Each column answers a different question. The table does not turn a nominal curve into a reference-calibrated measurement.

test/bringup/water_main.c / lines 29-34

```
//            the water temperature for a minute or more (~0.15 %/s in ppm),
//            slower than the row-to-row noise.
// Events (init, faults, ROM code, compensation on/off, first timings) print
// as their own "# ..." lines between rows.
// Replaces src/main.c in this env only; links the production drivers.
#include <Arduino.h>
```

Read the table as several kinds of evidence / continued

test/bringup/water_main.c / lines 35-46

```
35 #include <stdio.h>
36 #include <string.h>
37
38 #include "app_config.h"
39 #include "board_profile.h"
40 #include "bringup_harness.h"
41 #include "domain/tds_codec.h"
42 #include "drivers/ds18b20.h"
43 #include "drivers/tds.h"
44 #include "hal/adc_channel.h"
45 #include "hal/clock.h"
46 #include "hal/onewire_bus.h"
```

The first frame lacks a previous temperature. Later frames can use the preceding water observation while the new 750ms conversion proceeds. This is why the printed compensation can be valid without using a simultaneous same-frame temperature.

Represent per-driver progress and a trend window

test/bringup/water_main.c / lines 47-71

```
47
48 #define WATER_HEADER_EVERY 20     // rows between repeated table headers
49 #define WATER_TREND_ROWS 10           // rows in the window (even: two halves)
50 #define WATER_TREND_DRIFT_PCT 0.4f    // newer-half vs older-half mean, % ...
51 #define WATER_TREND_MIN_PPM 2.0f      // ... but never tighter than this
52 #define WATER_TREND_SPREAD_PCT 2.0f   // any single row vs the mean
53
54 static OneWireBus s_oneWire = ONE_WIRE_BUS_INIT;
55 static Ds18b20Config s_dsConfig = {&s_oneWire};
56 static Ds18b20Driver s_ds;
57 static AdcChannel s_adc = ADC_CHANNEL_INIT;
58 static TdsConfig s_tdsConfig = {&s_adc, {1.0f, 0.0f, 0, false}};  // nominal curve, no
 >   K-factor
59 static TdsDriver s_tds;
60
61 typedef struct DriverRun {
62   SensorDriver* driver;
63   bool initialised;
64   bool pending;    // started, not yet READY or failed
65   bool ready;      // this frame produced a result
66   DriverStatus failure;  // why not ready (DRIVER_OK = not started this frame)
67   MonoMs t0;
68   MonoMs nextPollMs;
69   uint32_t elapsedMs;
70 } DriverRun;
71
```

DriverRun stores whether each lifecycle stage is pending, ready or failed, plus its timing. Trend stores a circular sequence of recent ppm values and the compensation mode that gave them meaning.

test/bringup/water_main.c / lines 72-79

```
72 // Last rows, for the trend column.
73 typedef struct Trend {
74   uint8_t count;  // valid rows, up to WATER_TREND_ROWS
75   uint8_t next;
76   bool compensated;  // which column the window holds (@25C or uncomp)
77   float ppm[WATER_TREND_ROWS];
78 } Trend;
79
```

The two driver objects still own their private protocol state. These records belong to the test scheduler and presentation logic. Do not merge the two layers merely because both contain a pending flag.

Start one run without blocking the other conversion

test/bringup/water_main.c / lines 80-105

```
// init (if needed) -> start. Same logic as bringupTask(), without waiting.
static void beginRun(DriverRun* run, MonoMs now) {
  SensorDriver* drv = run->driver;
  run->pending = false;
  run->ready = false;
  run->failure = DRIVER_OK;
  if (!run->initialised) {
    const DriverStatus st = sensorDriverInit(drv);
    printf("# %s init -> %s\n", sensorDriverName(drv), driverStatusName(st));
    if (st != DRIVER_OK) {
      run->failure = st;
      return;
    }
    run->initialised = true;
  }
  run->t0 = now;
  const DriverStatus st = sensorDriverStart(drv, now);
  if (st != DRIVER_OK) {
    printf("# %s start -> %s\n", sensorDriverName(drv), driverStatusName(st));
    if (st != DRIVER_PENDING) run->initialised = false;
    run->failure = st;
    return;
  }
  run->pending = true;
  run->nextPollMs = now;
}
```

The helper resets this frame's scheduler flags, performs init only when required and records a failed init or start without making pending true. A successful start stores the first due-poll opportunity.

The caller can now invoke beginRun for the second driver. A long physical conversion in the first does not prevent the second from starting, because the first call already returned.

Advance due work and enforce an outer frame guard

test/bringup/water_main.c / lines 106-133

```
106
107 // One poll if it is due. Clears `pending` once READY or failed.
108 static void stepRun(DriverRun* run, MonoMs now) {
109   if (!run->pending || now < run->nextPollMs) return;
110   SensorDriver* drv = run->driver;
111   const PollResult pr = sensorDriverPoll(drv, now);
112   if (pr.status == DRIVER_PENDING) {
113     run->nextPollMs = pr.nextPollMs > now ? pr.nextPollMs : now + 1;
114     return;
115   }
116   run->pending = false;
117   run->elapsedMs = (uint32_t)(monoNowMs() - run->t0);
118   if (pr.status != DRIVER_READY) {
119     printf("# %s poll -> %s after %lu ms\n", sensorDriverName(drv),
120            driverStatusName(pr.status), (unsigned long)run->elapsedMs);
121     if (pr.status == DRIVER_NOT_PRESENT) run->initialised = false;
122     run->failure = pr.status;
123     return;
124   }
125   run->ready = true;
126 }
127
128 // Harness-level guard, not a driver budget: abandon what is left of the frame.
129 static void cancelRun(DriverRun* run) {
130   if (!run->pending) return;
131   sensorDriverCancel(run->driver);
132   run->pending = false;
133   run->failure = DRIVER_TIMEOUT;
```

stepRun calls poll only when pending and due. PENDING updates nextPollMs, ensuring it advances beyond now. Other outcomes end pending state and record elapsed time and failure or readiness.

Advance due work and enforce an outer frame guard / continued

test/bringup/water_main.c / lines 134-141

```
134   printf("# %s TIMEOUT (frame guard)\n", sensorDriverName(run->driver));
135 }
136
137 static void printTimingOnce(const DriverRun* run, bool* reported) {
138   if (*reported || !run->ready) return;
139   printf("# %s took %lu ms\n", sensorDriverName(run->driver), (unsigned
  >   long)run->elapsedMs);
140   *reported = true;
141 }
```

cancelRun abandons unfinished work at the test frame boundary. printTimingOnce reports the first completed timing rather than printing a repeated target from configuration. That measurement helps compare the real driver with its intended budget.

Format identity, absence and health separately

test/bringup/water_main.c / lines 142-169

```
142
143 static void printRomIfNew(void) {
144   static uint8_t last[8];
145   static bool printed = false;
146   uint8_t rom[8];
147   if (!ds18b20DriverGetRom(&s_ds, rom)) return;
148   if (printed && memcmp(rom, last, sizeof(rom)) == 0) return;
149   memcpy(last, rom, sizeof(rom));
150   printed = true;
151   printf("# ds18b20 ROM %02X-%02X%02X%02X%02X%02X%02X crc %02X\n", rom[0], rom[6],
  >   rom[5],
152          rom[4], rom[3], rom[2], rom[1], rom[7]);
153 }
154
155 static void printHeader(void) {
156   printf("\n");
157   printf("        | DS18B20 |  SEN0244 TDS                                |\n");
158   printf("   time | water T |    raw | uncomp |   @25C | state   | trend    |\n");
159   printf("      s |    degC |     mV |    ppm |    ppm |         |          |\n");
160   printf("--------+---------+--------+--------+--------+---------+----------+\n");
161 }
162
163 // Water temperature cell: the value, or why there is none this frame.
164 static void waterCell(const DriverRun* run, const Reading* t, char* cell, size_t len) {
165   if (!run->ready) {
166     const char* why = run->failure == DRIVER_NOT_PRESENT ? "NOPRES"
167                     : run->failure == DRIVER_TIMEOUT   ? "TIMEOUT"
168                     : run->failure == DRIVER_OK        ? "-"
169                                                        : "FAULT";
```

The ROM observer notices a changed selected sensor. The header labels each table column. waterCell distinguishes not-yet-ready, missing presence, timeout and a completed temperature observation. tdsStateName shortens health names for the table while preserving the key distinctions.

test/bringup/water_main.c / lines 170-178

```
170     snprintf(cell, len, "%s", why);
171   } else if (t->health == HEALTH_OK) {
172     snprintf(cell, len, "%.2f", (double)t->value);
173   } else {
174     snprintf(cell, len, "%s", t->health == HEALTH_STARTING ? "-" :
  >   healthName(t->health));
175   }
176 }
177
178 static const char* tdsStateName(const Reading* r) {
```

Format identity, absence and health separately / continued

test/bringup/water_main.c / lines 179-187

```
  switch (r->health) {
    case HEALTH_OK: return "OK";
    case HEALTH_UNCALIBRATED: return "UNCAL";
    case HEALTH_DEPENDENCY_INVALID: return "NO TEMP";
    case HEALTH_STARTING: return "-";
    case HEALTH_FAULT: return (r->qualityFlags & kQualitySaturated) ? "OVER" : "FAULT";
    default: return healthName(r->health);
  }
}
```

These are formatting decisions over retained facts. None should perform another sensor transaction. Reading a second value while formatting would make the row internally confusing.

Compare older and newer halves of one consistent window

test/bringup/water_main.c / lines 188-211

```
188
189 // Add this row and return "stable" / "settling" / "-". `ppm` is the value
190 // shown: @25C when compensated, else uncomp. A gap or a switch between the
191 // two restarts the window.
192 static const char* trendUpdate(Trend* tr, bool tdsOk, bool compensated, float ppm) {
193   if (!tdsOk) {
194     tr->count = 0;
195     return "-";
196   }
197   if (tr->count > 0 && compensated != tr->compensated) tr->count = 0;
198   if (tr->count == 0) tr->next = 0;
199   tr->compensated = compensated;
200   tr->ppm[tr->next] = ppm;
201   tr->next = (uint8_t)((tr->next + 1) % WATER_TREND_ROWS);
202   if (tr->count < WATER_TREND_ROWS) ++tr->count;
203   if (tr->count < WATER_TREND_ROWS) return "settling";
204
205   // Oldest row is at `next` (the window is full).
206   const uint8_t half = WATER_TREND_ROWS / 2;
207   float older = 0.0f, newer = 0.0f;
208   for (uint8_t i = 0; i < WATER_TREND_ROWS; ++i) {
209     const float v = tr->ppm[(tr->next + i) % WATER_TREND_ROWS];
210     if (i < half) older += v; else newer += v;
211   }
```

A gap or a change between compensated and uncompensated mode resets the trend window. Once full, next identifies the oldest element in the ring. The loop then computes the older-half and newer-half means in chronological order.

Compare older and newer halves of one consistent window / continued

test/bringup/water_main.c / lines 212-225

```
  older /= half;
  newer /= half;
  const float mean = (older + newer) / 2.0f;
  float drift = mean * WATER_TREND_DRIFT_PCT / 100.0f;
  if (drift < WATER_TREND_MIN_PPM) drift = WATER_TREND_MIN_PPM;
  float spread = mean * WATER_TREND_SPREAD_PCT / 100.0f;
  if (spread < WATER_TREND_MIN_PPM) spread = WATER_TREND_MIN_PPM;
  if (newer - older > drift || older - newer > drift) return "settling";
  for (uint8_t i = 0; i < WATER_TREND_ROWS; ++i) {
    const float d = tr->ppm[i] - mean;
    if (d > spread || -d > spread) return "settling";
  }
  return "stable";
}
```

Drift checks compare those half means; spread checks reject an isolated row too far from the overall mean. Minimum ppm tolerances keep the relative thresholds from becoming unrealistically tight near zero. Stable means this numerical window satisfies the chosen test; it does not prove probe calibration.

Display the same observation through several models

test/bringup/water_main.c / lines 226-253

```
226
227 static void printRow(MonoMs now, const DriverRun* ds, const Reading* t, const DriverRun*
  >    tds,
228                       const Reading* r, Trend* tr) {
229   char water[12], raw[12], uncomp[12], comp[12];
230   waterCell(ds, t, water, sizeof(water));
231
232   const bool tdsOk = tds->ready && r->hasRaw;
233   const char* state = tds->ready ? tdsStateName(r) : "-";
234   if (tdsOk) {
235     snprintf(raw, sizeof(raw), "%ld", (long)r->raw);
236     snprintf(uncomp, sizeof(uncomp), "%.0f", (double)tdsPpmFromModuleMv((float)r->raw,
  >   1.0f));
237   } else {
238     snprintf(raw, sizeof(raw), "-");
239     snprintf(uncomp, sizeof(uncomp), "-");
240   }
241   // value is compensated unless the driver flagged it uncompensated.
242   const bool compensated = tdsOk && !(r->qualityFlags & kQualityUncompensated);
243   if (compensated) {
244     snprintf(comp, sizeof(comp), "%.0f", (double)r->value);
245   } else {
246     snprintf(comp, sizeof(comp), "-");
247   }
248   const float shown = compensated ? r->value : tdsPpmFromModuleMv((float)r->raw, 1.0f);
249   const char* trend = trendUpdate(tr, tdsOk, compensated, shown);
250
251   printf("%7lu | %7s | %6s | %6s | %6s | %-7s | %-8s |\n", (unsigned long)(now / 1000),
  >   water,
252          raw, uncomp, comp, state, trend);
253 }
```

The function fills bounded text cells, obtains uncompensated ppm from raw millivolts and uses the retained value for the compensated column only when the flag permits it. It passes the displayed quantity and its compensation mode into the trend helper.

The raw, nominal and compensated columns intentionally share one observed voltage. Their differences expose the model's effect rather than differences between several ADC reads. Inspect health as well as the nicely aligned table.

Pass the previous temperature before starting both jobs

test/bringup/water_main.c / lines 254-281

```
254
255 static void waterTask(void* arg) {
256   (void)arg;
257   DriverRun ds = {&s_ds.base, false, false, false, DRIVER_OK, 0, 0, 0};
258   DriverRun tds = {&s_tds.base, false, false, false, DRIVER_OK, 0, 0, 0};
259   Trend trend;
260   memset(&trend, 0, sizeof(trend));
261   bool dsTimed = false;
262   bool tdsTimed = false;
263   int lastCompensated = -1;  // unknown
264   uint32_t rows = 0;
265   TickType_t lastWake = xTaskGetTickCount();
266
267   for (;;) {
268     vTaskDelayUntil(&lastWake, pdMS_TO_TICKS(CFG_FRAME_PERIOD_MS));
269     const MonoMs frameStart = monoNowMs();
270
271     // Latest DS18B20 result (previous frame) -> TDS compensation input. A
272     // failed or missing reading is passed as is: the TDS driver rejects it.
273     Reading water;
274     sensorDriverReadResult(&s_ds.base, &water, 1);
275     tdsDriverSetWaterTemperature(&s_tds, &water);
276     const bool fresh = readingIsActionableAt(&water, frameStart, CFG_STALE_ORDINARY_MS);
277     if ((int)fresh != lastCompensated) {
278       printf(fresh ? "# tds compensation ON (from DS18B20)\n"
279                    : "# DS18B20 not fresh -> tds uncompensated (NO TEMP)\n");
280       lastCompensated = fresh;
281     }
```

waterTask creates scheduler records and a trend, then releases frames on an absolute grid. At each frame start it copies the latest completed water result into TDS. A change in freshness is printed as a compensation-state event.

test/bringup/water_main.c / lines 282-284

```
282
283     beginRun(&ds, monoNowMs());
284     beginRun(&tds, monoNowMs());
```

Both runs then start. This is the concrete reason a previous-but-fresh dependency is useful: TDS can collect its 560ms window during the new water conversion instead of waiting until 750ms before beginning.

Sleep until the earliest useful poll and retain one owner

test/bringup/water_main.c / lines 285-312

```
285     while (ds.pending || tds.pending) {
286       MonoMs now = monoNowMs();
287       if (now - frameStart >= CFG_FRAME_PERIOD_MS) {
288         cancelRun(&ds);
289         cancelRun(&tds);
290         break;
291       }
292       stepRun(&ds, now);
293       stepRun(&tds, monoNowMs());
294
295       // Sleep until the earlier of the two next polls.
296       now = monoNowMs();
297       MonoMs wake = now + 1;
298       bool any = false;
299       if (ds.pending) { wake = ds.nextPollMs; any = true; }
300       if (tds.pending && (!any || tds.nextPollMs < wake)) wake = tds.nextPollMs;
301       const uint32_t waitMs = wake > now ? (uint32_t)(wake - now) : 1;
302       const TickType_t ticks = pdMS_TO_TICKS(waitMs);
303       vTaskDelay(ticks > 0 ? ticks : 1);
304     }
305
306     printRomIfNew();
307     printTimingOnce(&ds, &dsTimed);
308     printTimingOnce(&tds, &tdsTimed);
309     Reading t;
310     Reading ppm;
311     sensorDriverReadResult(&s_ds.base, &t, 1);
312     sensorDriverReadResult(&s_tds.base, &ppm, 1);
```

The loop steps both pending drivers, applies a frame guard, chooses the earlier next-poll timestamp and delays at least one tick. It does not busy-wait for the slower conversion.

test/bringup/water_main.c / lines 313-316

```
313     if (rows++ % WATER_HEADER_EVERY == 0) printHeader();
314     printRow(monoNowMs(), &ds, &t, &tds, &ppm, &trend);
315   }
316 }
```

After work ends, it prints identity changes, first timings and one table row from copied results. The same task owns all these accesses. The test therefore teaches interleaving without introducing cross-task driver sharing.

Configure both physical interfaces before task creation

test/bringup/water_main.c / lines 317-336

```
317
318 void setup(void) {
319   ds18b20DriverConstruct(&s_ds, &s_dsConfig);
320   tdsDriverConstruct(&s_tds, &s_tdsConfig);
321   delay(200);
322   printf("\n# water bring-up: DS18B20 GPIO%d + TDS GPIO%d, %lu ms frame\n",
323          (int)BOARD_PIN_ONE_WIRE, (int)BOARD_PIN_TDS_ADC, (unsigned
  >   long)CFG_FRAME_PERIOD_MS);
324   if (!oneWireBusBegin(&s_oneWire, BOARD_PIN_ONE_WIRE)) {
325     printf("# GPIO%d cannot be used for 1-Wire\n", (int)BOARD_PIN_ONE_WIRE);
326   }
327   if (!adcChannelBegin(&s_adc, BOARD_PIN_TDS_ADC)) {
328     printf("# GPIO%d is not an ADC1 pin\n", (int)BOARD_PIN_TDS_ADC);
329   }
330
331   const TaskSpec* spec = cfgTaskSpec(TASK_ID_ACQUISITION_NORMAL);
332   xTaskCreatePinnedToCore(waterTask, "water", spec->stackBytes, NULL, spec->priority,
  >   NULL,
333                           spec->core);
334 }
335
336 void loop(void) { vTaskDelay(portMAX_DELAY); }
```

Setup constructs the DS18B20 and TDS objects, configures their distinct buses and creates the water task using the normal acquisition placement. The old loop blocks.

If the analog input works but the water dependency fails, the table can still expose raw TDS voltage with NO TEMP. That is more informative than either hiding the voltage or labelling the resulting nominal number fully valid.

Send words in the device's order

The SGP30 uses I2C but its command format differs from the BME280 register model. A command is a 16-bit word. Data words are accompanied by CRC bytes. The codec handles word packing and checking; the driver handles command timing.

Worked command packing - measure_iaq

```
uint16_t command = 0x2008;
uint8_t high = (uint8_t)(command >> 8);
uint8_t low = (uint8_t)command;
// Bytes sent: 0x20, then 0x08
```

The cast to uint8_t keeps the low eight bits. The right shift puts the high byte in the low position before that cast. Received words are reconstructed in the reverse direction, then their CRCs are checked before they are accepted.

The measure response places CO2eq before TVOC. The set-baseline command expects TVOC before CO2eq. A shared integer type does not protect you from swapping these two values. The driver explicitly names the order for each command.

After start sends a measurement command, poll waits until startMs + 13ms before reading in this implementation. The task can do other work during that interval. A failed command is represented by a stored step so poll can report the outcome consistently.

CO2eq is an equivalent estimate produced by this gas-sensing system, not a direct CO2 concentration measurement. The firmware names its metric CO2EQ for that reason. Preserve that distinction in the app and in your interpretation of telemetry.

Reference: Sensirion: SGP30 datasheet

COMMON PITFALL

An I2C helper that works for register-based devices is not automatically the complete protocol for every I2C chip.

Time can be part of measurement validity

This driver sends the air-quality measurement command on an approximately 1s cadence. Its baseline algorithm depends on regular operation. That is why a slow network task should not decide when the sensor happens to run.

Driver time condition	Meaning in this source
First 15s after init	WARMING: fixed startup outputs are not accepted as normal observations.
No restored baseline	Learning policy remains UNCALIBRATED for 12h.
Start gap greater than 1800ms	Count a missed-cadence condition.
Repeated gaps or failures	Request reinitialisation after the configured count.

These are separate states. A numerically plausible 400ppm reading does not prove warm-up is over. The driver uses elapsed time, not equality to 400, to label warming. After warming, an unrestored baseline can still keep the result uncalibrated under this project's policy.

A restored baseline is stored sensor state that can shorten the need to learn from scratch. The driver periodically fetches a new baseline, and Configuration decides when to save it. Fetching internal state and writing flash are different jobs; the sensor driver should not block itself on NVS.

The driver runs at most one housekeeping command after a successful measurement. Baseline retrieval and a humidity update both need protocol time. Sending every pending command immediately could ask the chip to process another command before the previous one completes.

This is the first driver where the history of valid operation is part of the meaning of the next number. The shared start/poll interface still works; the private state machine simply has more steps.

The 12h learning label and 1800ms gap tolerance are project choices visible in the archived source.

Reference: Sensirion: SGP30 datasheet

Use humidity without hiding its age

The acquisition owner passes BME280 air temperature and humidity into sgp30DriverSetHumidity(). That function derives the absolute-humidity input used by the gas sensor. Relative humidity alone is a percentage relative to the air's temperature-dependent capacity; the compensation input needs a different quantity.

You do not have to memorise the vapour-pressure formula to understand the architecture. You do have to know its inputs, units and acceptance conditions. The driver checks the supplied temperature and humidity records, uses their timing, and marks the dependency invalid when they are unsuitable.

Worked fixed-point representation - simplified positive input

```
float absoluteHumidity = 10.0f;  // g per cubic metre
uint16_t packed = (uint16_t)(absoluteHumidity * 256.0f);
// packed = 2560 = 0x0A00
```

An 8.8 fixed-point word uses eight fractional bits, so one physical unit corresponds to 256 stored units. This packing is an encoding step after the physical calculation. A raw 2560 is not 2560 grams per cubic metre.

If the BME280 stops updating, the gas driver must not keep applying an old humidity correction indefinitely. The source clears the compensation input when its age exceeds the climate limit and reports the dependency status. A good CRC on a gas response cannot make a stale compensation input fresh.

You now have two cross-sensor paths: DS18B20 to TDS and BME280 to SGP30. Both travel through copied Reading records supplied by the acquisition owner. Drivers do not independently reach into one another's hardware or create competing sensor tasks.

Reference: Sensirion: SGP30 datasheet

REMEMBER

Dependency freshness belongs to the moment of use. A previously accepted input can become unsuitable while retained.

Name byte packing and physical conversion separately

The gas protocol uses big-endian words with a CRC per word. This codec keeps that byte contract separate from the driver's cadence.

The humidity helpers operate on physical values supplied by validated Reading records.

src/domain/sgp30_codec.h / lines 1-28

```
// SGP30 word framing and humidity conversion (WP6). Pure C, host-tested.
// Datasheet: Sensirion SGP30 v1.0, §6.3 (humidity compensation), §6.6 (CRC).
//
// Every 16-bit word on the wire is MSB, LSB, CRC-8. The CRC is mandatory in
// the write direction; the driver also checks it on every word it reads.
#pragma once
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

// CRC-8: polynomial 0x31, init 0xFF, no reflection, no final XOR.
// Datasheet test vector: CRC(0xBE, 0xEF) = 0x92.
uint8_t sgp30Crc8(const uint8_t* data, size_t len);

// out = {MSB, LSB, CRC}.
void sgp30PackWord(uint16_t word, uint8_t out[3]);

// Decode nWords 3-byte groups from buf into out[]. false on any CRC mismatch
// (out is then not fully written). An all-0xFF floating-bus read fails here.
bool sgp30UnpackWords(const uint8_t* buf, uint8_t nWords, uint16_t* out);

// g/m3 as the sensor's 8.8 fixed-point value, rounded and saturated to
// 0x0001..0xFFFF. 0 for a non-positive or NaN input (0 disables compensation).
uint16_t sgp30GramsToFixed8p8(float gramsPerM3);

// Absolute humidity (g/m3) from air temperature and relative humidity, using
// the datasheet's Magnus formula. Returns a negative value for invalid input.
float sgp30AbsoluteHumidity(float tempC, float rhPercent);
```

The header declares CRC, word packing, word checking, absolute humidity and fixed-point encoding. Those functions do not select an I2C address or initiate a measurement.

A payload word and a compensation input can both be uint16_t after encoding, but their meanings remain different. Follow the function names and units instead of assuming the shared C type makes them interchangeable.

Protect each two-byte word with its own check byte

Read the codec with the worked command-word and 8.8 examples from the lesson.

Notice that this CRC has a different initial value and bit direction from the 1-Wire CRC.

src/domain/sgp30_codec.c / lines 1-27

```
#include "domain/sgp30_codec.h"

#include <math.h>

uint8_t sgp30Crc8(const uint8_t* data, size_t len) {
  uint8_t crc = 0xFF;
  for (size_t i = 0; i < len; ++i) {
    crc ^= data[i];
    for (uint8_t bit = 0; bit < 8; ++bit) {
      crc = (crc & 0x80) ? (uint8_t)((crc << 1) ^ 0x31) : (uint8_t)(crc << 1);
    }
  }
  return crc;
}

void sgp30PackWord(uint16_t word, uint8_t out[3]) {
  out[0] = (uint8_t)(word >> 8);
  out[1] = (uint8_t)(word & 0xFF);
  out[2] = sgp30Crc8(out, 2);
}

bool sgp30UnpackWords(const uint8_t* buf, uint8_t nWords, uint16_t* out) {
  for (uint8_t i = 0; i < nWords; ++i) {
    const uint8_t* w = &buf[3 * i];
    if (sgp30Crc8(w, 2) != w[2]) return false;
    out[i] = (uint16_t)((w[0] << 8) | w[1]);
  }
```

The CRC starts at 0xFF and processes the most-significant bit with polynomial 0x31. PackWord emits high byte, low byte and their CRC. UnpackWords advances in three-byte groups and validates each pair before reconstructing its integer.

src/domain/sgp30_codec.c / lines 28-29

```
  return true;
}
```

A failed later word can leave earlier output elements assigned. The boolean controls whether the whole requested result set is accepted. The caller should not use a partially decoded set as a successful measurement.

Encode a bounded humidity value without hiding invalid input

src/domain/sgp30_codec.c / lines 30-46

```
30
31 uint16_t sgp30GramsToFixed8p8(float gramsPerM3) {
32   if (!(gramsPerM3 > 0.0f)) return 0;  // also catches NaN
33   const float scaled = gramsPerM3 * 256.0f + 0.5f;
34   if (scaled >= 65535.0f) return 0xFFFF;
35   if (scaled < 1.0f) return 0x0001;  // smallest value that keeps compensation on
36   return (uint16_t)scaled;
37 }
38
39 // dv = 216.7 * (RH/100 * 6.112 * exp(17.62 T / (243.12 + T))) / (273.15 + T)
40 float sgp30AbsoluteHumidity(float tempC, float rhPercent) {
41   if (!(rhPercent >= 0.0f && rhPercent <= 100.0f) || !(tempC > -40.0f && tempC < 85.0f))
 >    {
42     return -1.0f;
43   }
44   const float saturation = 6.112f * expf(17.62f * tempC / (243.12f + tempC));  // hPa
45   return 216.7f * (rhPercent / 100.0f * saturation) / (273.15f + tempC);
46 }
```

GramsToFixed8p8 rejects nonpositive input, including NaN under the chosen comparison, rounds after multiplying by 256 and saturates the maximum representable word. A small positive value maps to at least one because zero explicitly disables compensation.

AbsoluteHumidity validates relative humidity and temperature, evaluates a saturation-pressure model and converts to grams per cubic metre. The returned negative sentinel means rejected input. The driver then decides whether to write zero and label a dependency failure.

Separate warm-up, baseline learning and a single fault

This driver must remember recent cadence, baseline history and humidity input as well as one pending command. The extra fields each answer a different timing question.

Do not compress all of these conditions into one ready boolean.

src/drivers/sgp30.h / lines 1-28

```
// sgp30 driver. Work package WP6 (Stage 2). Architecture §9, §10, §16 (D13, D18).
// Datasheet: Sensirion SGP30 v1.0, May 2020 (E13; local copy in
// Electrical System Study/Datasheets/Sensirion_Gas_Sensors_Datasheet_SGP30.pdf).
//
// Retained 3.3 V supply, shared I2C bus with the BME280 (address 0x58). The
// Adafruit breakout regulates the chip's 1.8 V and shifts the I2C levels.
// Produces Tvoc (ppb) and Co2eq (ppm, an estimate - never a CO2 measurement).
//
// init() checks the feature set (product type 0), sends iaq_init and restores
// a baseline if one is known. Then one measure_iaq per second from the
// absolute frame: start() sends the command, poll() reads 6 bytes once the
// 12 ms conversion is over and checks the CRC-8 of each word. The sensor NACKs
// while it converts, so it is read once, never polled.
//
// Health, highest priority first:
//   FAULT              CRC or bus error on this measurement (also while warming)
//   WARMING            first 15 s after iaq_init (fixed 400 ppm / 0 ppb)
//   DEPENDENCY_INVALID humidity compensation enabled but its input is stale
//   UNCALIBRATED       baseline still LEARNING (no restore, < 12 h) - D13
//   OK                 otherwise; kQualityUncompensated if no humidity input
//
// Unlike the BME280, a single fault does NOT force a re-init: re-init means a
// new iaq_init and 15 s of WARMING. CRC/bus errors and missed seconds (start()
// later than 1.8 s after the previous one) count as gaps; after 3 consecutive
// gaps start() returns Fault and the caller re-runs init(), which re-applies
// the last known baseline.
//
// Baseline persistence belongs to Configuration (WP8, NVS). This driver only
```

The introductory contract prioritises a current bus or CRC fault, then warm-up, then dependency validity and baseline learning. A transient failed measurement does not automatically reset the chip, because reinitialisation would restart warm-up.

Separate warm-up, baseline learning and a single fault / continued

src/drivers/sgp30.h / lines 29-36

```
// keeps a RAM copy: it is fetched hourly once VALID or RESTORED, read with
// sgp30DriverGetBaseline() and restored at init from sgp30DriverSetBaseline().
//
// Worst-case blocking (I2C_BUS_TIMEOUT_MS = T per transaction):
//   init()    ~30 ms + 4 T  (feature set, iaq_init, set_iaq_baseline, each
//                             followed by its 10 ms maximum duration)
//   start()   1 T           (one command write)
//   poll()    2 T           (6-byte read + humidity write or baseline command);
```

src/drivers/sgp30.h / lines 37-40

```
//                            Pending until start+13 ms; once an hour Pending
//                            for 11 ms more to read the baseline (1 T)
//   others    no bus access
//
```

The baseline is retained internally by the driver in RAM. Configuration owns persistent storage and write rates. This lets the sensor keep its required cadence while another service handles flash operations.

Give baseline state a name

src/drivers/sgp30.h / lines 41-68

```
41 // Implements the SensorDriver contract in drivers/sensor_driver.h.
42 #pragma once
43 #include "drivers/sensor_driver.h"
44 #include "hal/i2c_bus.h"
45
46 typedef struct Sgp30Config {
47   I2cBus* bus;
48   uint8_t address;  // BOARD_SGP30_I2C_ADDRESS
49 } Sgp30Config;
50
51 #define SGP30_METRICS 2  // results_ index: 0 TVOC, 1 CO2eq
52
53 // Baseline compensation state, reported separately from the readings (§10).
54 typedef uint8_t Sgp30BaselineStatus;
55 enum {
56   SGP30_BASELINE_NONE = 0,  // not initialised yet
57   SGP30_BASELINE_LEARNING,  // no baseline restored; learning for 12 h
58   SGP30_BASELINE_RESTORED,  // stored baseline applied after iaq_init
59   SGP30_BASELINE_VALID      // learned for 12 h without a restart
60 };
61
62 // Members after `base` are private to sgp30.c.
63 typedef struct Sgp30Driver {
64   SensorDriver base;  // must stay first
65   Sgp30Config config_;
66   bool initialised_;
67   uint8_t step_;             // measurement step (sgp30.c)
68   uint8_t consecutiveGaps_;
```

The baseline enum distinguishes none, learning, restored and valid. These labels describe history, not the instantaneous TVOC concentration. SGP30_METRICS fixes the result order as TVOC then CO2eq even though some wire commands use another word order.

src/drivers/sgp30.h / lines 69-77

```
69   uint16_t featureSet_;
70   MonoMs startMs_;
71   MonoMs initMs_;
72   MonoMs lastStartMs_;
73   MonoMs baselineReadyMs_;   // when the baseline answer can be read
74   MonoMs baselineFetchDueMs_;
75   Sgp30BaselineStatus baselineStatus_;
76   MonoMs learningStartMs_;
77   bool hasBaseline_;         // baselineCo2eq_/baselineTvoc_ hold a usable baseline
```

Keep those two orders separate while reading start and read helpers. A correct byte count with swapped meanings still creates wrong observations.

Read the retained state in groups

src/drivers/sgp30.h / lines 78-105

```
  uint16_t baselineCo2eq_;
  uint16_t baselineTvoc_;
  bool hasPendingBaseline_;  // set by sgp30DriverSetBaseline(), applied at init
  uint16_t pendingCo2eq_;
  uint16_t pendingTvoc_;
  bool humidityEnabled_;     // sgp30DriverSetHumidity() has been called
  bool humidityValid_;       // its last input was fresh
  bool humidityDirty_;       // humidity8p8_ not yet written to the chip
  uint16_t humidity8p8_;     // g/m3 in 8.8 fixed point, 0 = compensation off
  MonoMs humidityInputMs_;   // intervalEndMs of the older of the two inputs
  Reading results_[SGP30_METRICS];
} Sgp30Driver;

// Constructor. Use &driver.base wherever a SensorDriver* is expected.
void sgp30DriverConstruct(Sgp30Driver* self, const Sgp30Config* config);

// Humidity compensation input from the latest BME280 temperature and humidity
// readings (copied, not shared). Both must be OK and fresh at `now`
// (CFG_STALE_BME280_MS); otherwise compensation is switched off on the chip
// (written as 0) and the readings report DependencyInvalid. The same happens
// if the caller stops calling this and the last input ages past that limit.
// Written after the next measurement (changes under ~0.03 g/m3 are ignored);
// no bus access here.
void sgp30DriverSetHumidity(Sgp30Driver* self, const Reading* airTemperature,
                            const Reading* airHumidity, MonoMs now);

// Stored baseline (from NVS) to restore at the next init(). Values in the
// order get_iaq_baseline returns them.
```

First identify operation state and timestamps. Next identify baseline words and pending restore state. Finally identify humidity enabled/valid/dirty state and the retained older-input time. Each group drives a different later branch.

humidityDirty means a chip write is required; humidityValid means the source evidence was accepted. A value can be valid but unchanged enough to avoid another write. That distinction reduces protocol traffic without silently losing freshness information.

Update inputs without taking over the bus

src/drivers/sgp30.h / lines 106-116

```
106 void sgp30DriverSetBaseline(Sgp30Driver* self, uint16_t co2eq, uint16_t tvoc);
107
108 // Latest baseline worth storing. false while LEARNING (Sensirion: do not
109 // store a baseline before 12 h unless one was restored).
110 bool sgp30DriverGetBaseline(const Sgp30Driver* self, uint16_t* co2eq, uint16_t* tvoc);
111
112 Sgp30BaselineStatus sgp30DriverBaselineStatus(const Sgp30Driver* self);
113 const char* sgp30BaselineStatusName(Sgp30BaselineStatus s);
114
115 // Feature set word read in init() (0 until then), e.g. 0x0022.
116 uint16_t sgp30DriverFeatureSet(const Sgp30Driver* self);
```

The setters receive supplied records or words and update retained state. GetBaseline provides a copy only under the declared baseline conditions. FeatureSet and status getters expose diagnostic values without causing transactions.

These APIs remain safe only under their ownership rule. A const getter prevents edits through its pointer; it does not create a cross-task snapshot or lock. Acquisition and its same-task bring-up callbacks use them deliberately.

Make protocol constants and private states visible

We will read the command lifecycle and its housekeeping in source order. Keep the one-second measurement cadence separate from the 13ms response wait and the much longer baseline policy.

The driver can be busy with a baseline read after this second's gas values were already stored.

src/drivers/sgp30.c / lines 1-28

```
#include "drivers/sgp30.h"

#include <Arduino.h>
#include <string.h>

#include "app_config.h"
#include "domain/sgp30_codec.h"
#include "hal/clock.h"

// Commands (datasheet Table 10).
static const uint16_t kCmdIaqInit = 0x2003;
static const uint16_t kCmdMeasureIaq = 0x2008;
static const uint16_t kCmdGetIaqBaseline = 0x2015;
static const uint16_t kCmdSetIaqBaseline = 0x201E;
static const uint16_t kCmdSetAbsoluteHumidity = 0x2061;
static const uint16_t kCmdGetFeatureSet = 0x202F;

static const uint16_t kProductTypeMask = 0xF000;  // Table 9: SGP30 = 0

// Timing. Driver-local because app_config.h is frozen.
static const uint32_t kCmdMaxMs = 10;           // iaq_init, baseline, humidity, feature
    set
static const uint32_t kFirstReadMs = 13;        // > measure_iaq max 12 ms, 1 ms clock
  margin
static const uint32_t kWarmupMs = 15000;        // fixed 400 ppm / 0 ppb after iaq_init
static const uint32_t kGapMs = 1800;            // start() later than this = missed
  second
static const uint8_t kMaxGaps = 3;              // consecutive gaps before a re-init
static const uint32_t kLearningMs = 12UL * 60 * 60 * 1000;  // no stored baseline
static const uint32_t kBaselineFetchPeriodMs = 60UL * 60 * 1000;
static const uint16_t kHumidityDeadband8p8 = 8;  // ~0.03 g/m3: ignore BME280 noise
```

The command words name init, measurement, baseline and humidity operations. Timing constants describe command wait, warm-up, gap tolerance and baseline maintenance. The private step enum distinguishes a normal measurement, a failed write and a pending baseline response.

Make protocol constants and private states visible / continued

src/drivers/sgp30.c / lines 29-49

```
29
30 enum {
31   STEP_IDLE = 0,
32   STEP_MEASURING,
33   STEP_WRITE_FAILED,  // start() could not send the command; poll() reports it
34   STEP_BASELINE_READ
35 };
36
37 static inline Sgp30Driver* asSgp30(SensorDriver* d) { return (Sgp30Driver*)d; }
38
39 static bool sendCommand(Sgp30Driver* self, uint16_t cmd);
40 static bool sendCommandWords(Sgp30Driver* self, uint16_t cmd, const uint16_t* words,
41                              uint8_t nWords);
42 static bool readWords(Sgp30Driver* self, uint16_t* out, uint8_t nWords);
43 static PollResult fail(Sgp30Driver* self, MonoMs now);
44 static PollResult finish(Sgp30Driver* self, MonoMs now);
45 static void storeResults(Sgp30Driver* self, uint16_t co2eq, uint16_t tvoc, MonoMs now);
46
47 static SensorId sgp30Id(const SensorDriver* d) { (void)d; return SENSOR_SGP30; }
48 static const char* sgp30Name(const SensorDriver* d) { (void)d; return "sgp30"; }
49 static uint8_t sgp30MetricCount(const SensorDriver* d) { (void)d; return SGP30_METRICS;
 >   }
```

The fixed metadata functions still use the generic interface. More internal states do not require a new acquisition API as long as start, poll and result contracts remain intact.

Identify and initialise before restoring history

src/drivers/sgp30.c / lines 50-76

```
50
51 static DriverStatus sgp30Init(SensorDriver* d) {
52   Sgp30Driver* self = asSgp30(d);
53   self->initialised_ = false;
54   self->step_ = STEP_IDLE;
55   self->baselineStatus_ = SGP30_BASELINE_NONE;
56
57   // Identity: feature set, product type 0. measure_test is not used (it must
58   // not follow iaq_init and takes 220 ms), nor the General Call reset (it
59   // would reset every device on the shared bus).
60   uint16_t featureSet = 0;
61   if (!sendCommand(self, kCmdGetFeatureSet)) {
62     i2cBusRecover(self->config_.bus);
63     return DRIVER_FAULT;
64   }
65   delay(kCmdMaxMs + 1);  // delay(n) may return up to 1 tick early
66   if (!readWords(self, &featureSet, 1)) {
67     i2cBusRecover(self->config_.bus);
68     return DRIVER_FAULT;
69   }
70   if ((featureSet & kProductTypeMask) != 0) return DRIVER_NOT_PRESENT;
71   self->featureSet_ = featureSet;
72
73   if (!sendCommand(self, kCmdIaqInit)) return DRIVER_FAULT;
74   delay(kCmdMaxMs + 1);  // delay(n) may return up to 1 tick early
75   const MonoMs now = monoNowMs();
76
```

Init clears active state and baseline status, requests the feature set and validates product type. It then issues iaq_init and waits the bounded command durations. Bus recovery occurs on the relevant failed read/write path.

The deliberate setup waits occupy elapsed time inside init. The acquisition scheduler must treat this call as a longer blocking step. PENDING conversion design elsewhere does not make every setup operation nonblocking.

Choose a restored baseline or a new learning period

src/drivers/sgp30.c / lines 77-102

```
  // A stored baseline from Configuration wins over the RAM copy.
  if (self->hasPendingBaseline_) {
    self->baselineCo2eq_ = self->pendingCo2eq_;
    self->baselineTvoc_ = self->pendingTvoc_;
    self->hasBaseline_ = true;
    self->hasPendingBaseline_ = false;
  }
  if (self->hasBaseline_) {
    // set_iaq_baseline takes TVOC first, then CO2eq (datasheet §6.3).
    const uint16_t words[2] = {self->baselineTvoc_, self->baselineCo2eq_};
    if (!sendCommandWords(self, kCmdSetIaqBaseline, words, 2)) return DRIVER_FAULT;
    delay(kCmdMaxMs + 1);  // the harness may send measure_iaq right after init()
    self->baselineStatus_ = SGP30_BASELINE_RESTORED;
  } else {
    self->baselineStatus_ = SGP30_BASELINE_LEARNING;
    self->learningStartMs_ = now;
  }

  self->initMs_ = now;
  self->lastStartMs_ = now;
  self->baselineFetchDueMs_ = now + kBaselineFetchPeriodMs;
  self->consecutiveGaps_ = 0;
  self->humidityDirty_ = self->humidityEnabled_;  // re-send after a restart
  self->initialised_ = true;
  return DRIVER_OK;
}
```

A pending baseline offered by Configuration takes precedence over the existing RAM copy. The set-baseline command sends TVOC before CO2eq, so the temporary words array is deliberately reordered.

Without a baseline, the driver starts the learning interval. It then records init and last-start times, schedules the next baseline fetch, resets gap state and marks initialised. The retained result identities themselves are not reconstructed on every recovery.

Count cadence gaps without restarting after every bad command

src/drivers/sgp30.c / lines 103-126

```
103
104 static DriverStatus sgp30Start(SensorDriver* d, MonoMs now) {
105   Sgp30Driver* self = asSgp30(d);
106   if (!self->initialised_) return DRIVER_FAULT;
107
108   if (now - self->lastStartMs_ > kGapMs && self->consecutiveGaps_ < 0xFF) {
109     ++self->consecutiveGaps_;
110   }
111   self->lastStartMs_ = now;
112   if (self->consecutiveGaps_ >= kMaxGaps) {
113     self->initialised_ = false;  // caller re-runs init(): iaq_init + baseline
114     return DRIVER_FAULT;
115   }
116
117   self->startMs_ = now;
118   if (sendCommand(self, kCmdMeasureIaq)) {
119     self->step_ = STEP_MEASURING;
120   } else {
121     // Reported by poll() as one gap, not as a re-init (that would cost 15 s).
122     i2cBusRecover(self->config_.bus);
123     self->step_ = STEP_WRITE_FAILED;
124   }
125   return DRIVER_OK;
126 }
```

Start compares now with lastStartMs_ and records excessive gaps. Once the configured consecutive-gap count is reached, it rejects work and asks for reinit. Otherwise it sends the measurement command and records either MEASURING or WRITE_FAILED.

Even a failed write returns OK from this start path so poll can report one coherent failed outcome and gap. Read the implementation's status convention, rather than assuming every low-level failure must return immediately from start.

Read measurement words only after their command wait

src/drivers/sgp30.c / lines 127-151

```
127
128 static PollResult sgp30Poll(SensorDriver* d, MonoMs now) {
129   Sgp30Driver* self = asSgp30(d);
130
131   switch (self->step_) {
132     case STEP_MEASURING: {
133       if (now < self->startMs_ + kFirstReadMs) {
134         PollResult r = {DRIVER_PENDING, self->startMs_ + kFirstReadMs};
135         return r;
136       }
137       uint16_t words[2] = {0, 0};
138       if (!readWords(self, words, 2)) {
139         i2cBusRecover(self->config_.bus);
140         return fail(self, now);
141       }
142       // measure_iaq answers CO2eq first, then TVOC (datasheet §6.3).
143       storeResults(self, words[0], words[1], now);
144       return finish(self, now);
145     }
146
147     case STEP_BASELINE_READ: {
148       if (now < self->baselineReadyMs_) {
149         PollResult r = {DRIVER_PENDING, self->baselineReadyMs_};
150         return r;
151       }
```

MEASURING first returns PENDING until start+13ms. The actual read then validates both words. A failed read recovers the bus and goes through fail; a successful read supplies CO2eq and TVOC in response order to storeResults.

finish may immediately select one allowed housekeeping action. The measurement values are already retained before that next private step begins. The acquisition caller still receives the complete operation outcome through poll.

Complete a baseline response without losing this second's result

src/drivers/sgp30.c / lines 152-175

```
      // get_iaq_baseline answers CO2eq first, then TVOC. A failure only
      // delays the next copy: this second's readings are already stored.
      uint16_t words[2] = {0, 0};
      if (readWords(self, words, 2)) {
        self->baselineCo2eq_ = words[0];
        self->baselineTvoc_ = words[1];
        self->hasBaseline_ = true;
      }
      self->baselineFetchDueMs_ = now + kBaselineFetchPeriodMs;
      self->step_ = STEP_IDLE;
      PollResult r = {DRIVER_READY, now};
      return r;
    }

    case STEP_WRITE_FAILED:
      return fail(self, now);

    default: {
      // poll() without a start(): nothing in flight.
      PollResult r = {DRIVER_FAULT, now};
      return r;
    }
  }
}
```

BASELINE_READ waits until its response time, then updates RAM baseline words only if the read is valid. A failed baseline fetch postpones a later copy; it does not erase the already stored gas values.

WRITE_FAILED takes the common failure path. The default branch rejects polling without a meaningful active step. Private step and observation health remain distinct throughout the switch.

Perform at most one maintenance command

src/drivers/sgp30.c / lines 176-198

```
176
177 // After a good measurement the bus is free and the next measure_iaq is ~1 s
178 // away: do at most one housekeeping command, so the chip is never asked for a
179 // second command while it is still processing the first (10 ms each). The
180 // hourly baseline copy goes first; a pending humidity write waits 1 s.
181 static PollResult finish(Sgp30Driver* self, MonoMs now) {
182   self->step_ = STEP_IDLE;
183
184   if (self->baselineStatus_ == SGP30_BASELINE_LEARNING &&
185       now - self->learningStartMs_ >= kLearningMs) {
186     self->baselineStatus_ = SGP30_BASELINE_VALID;
187     self->baselineFetchDueMs_ = now;  // first copy right away
188   }
189
190   if ((self->baselineStatus_ == SGP30_BASELINE_VALID ||
191        self->baselineStatus_ == SGP30_BASELINE_RESTORED) &&
192       monoReached(now, self->baselineFetchDueMs_)) {
193     if (sendCommand(self, kCmdGetIaqBaseline)) {
194       self->step_ = STEP_BASELINE_READ;
195       self->baselineReadyMs_ = now + kCmdMaxMs + 1;
196       PollResult r = {DRIVER_PENDING, self->baselineReadyMs_};
197       return r;
198     }
```

finish moves the private step to idle and advances baseline learning when its interval is satisfied. A due baseline fetch gets priority and returns PENDING for its later response. Otherwise a dirty humidity value can be sent.

src/drivers/sgp30.c / lines 199-209

```
199     self->baselineFetchDueMs_ = now + kBaselineFetchPeriodMs;
200   } else if (self->humidityDirty_) {
201     const uint16_t word = self->humidity8p8_;
202     if (sendCommandWords(self, kCmdSetAbsoluteHumidity, &word, 1)) {
203       self->humidityDirty_ = false;
204     }
205   }
206
207   PollResult r = {DRIVER_READY, now};
208   return r;
209 }
```

Doing every pending housekeeping request at once would ignore the chip's command-processing time. The bounded choice keeps one measurement cycle from growing into an unbounded maintenance loop.

Expire a compensation input even if nobody calls the setter

src/drivers/sgp30.c / lines 210-237

```
210
211 static void storeResults(Sgp30Driver* self, uint16_t co2eq, uint16_t tvoc, MonoMs now) {
212   self->consecutiveGaps_ = 0;
213
214   // The caller stopped refreshing the humidity input: switch compensation off
215   // on the chip (written at the next finish()) instead of keeping it silently.
216   if (self->humidityEnabled_ && self->humidityValid_ &&
217       now - self->humidityInputMs_ > CFG_STALE_BME280_MS) {
218     self->humidityValid_ = false;
219     self->humidity8p8_ = 0;
220     self->humidityDirty_ = true;
221   }
222
223   Health health = HEALTH_OK;
224   if (now - self->initMs_ < kWarmupMs) {
225     health = HEALTH_WARMING;  // by time, never by the 400/0 values
226   } else if (self->humidityEnabled_ && !self->humidityValid_) {
227     health = HEALTH_DEPENDENCY_INVALID;
228   } else if (self->baselineStatus_ == SGP30_BASELINE_LEARNING) {
229     health = HEALTH_UNCALIBRATED;
230   }
231   const uint16_t quality = (self->humidityEnabled_ && self->humidityValid_)
232                                ? kQualityNone
233                                : kQualityUncompensated;
234
235   const uint16_t raws[SGP30_METRICS] = {tvoc, co2eq};
236   for (uint8_t i = 0; i < SGP30_METRICS; ++i) {
237     Reading* r = &self->results_[i];
```

storeResults resets the consecutive-gap count after a successful gas read. It also checks the age of the retained humidity input and schedules a zero write when that evidence has aged out.

Health then follows a deliberate priority: warming, required-but-invalid dependency, baseline learning, otherwise OK. A fixed startup 400/0 response is judged by elapsed init time, not by assuming those numbers are always invalid in the real world.

Write each metric with the verdict that belongs to this observation

src/drivers/sgp30.c / lines 238-264

```
    r->intervalStartMs = self->startMs_;
    r->intervalEndMs = now;
    r->hasRaw = true;
    r->raw = raws[i];
    r->qualityFlags = quality;
    r->health = health;
    if (health == HEALTH_OK || health == HEALTH_UNCALIBRATED ||
        health == HEALTH_DEPENDENCY_INVALID) {
      r->value = (float)raws[i];
    }
    if (health == HEALTH_OK || health == HEALTH_UNCALIBRATED) {
      r->lastGoodMs = now;
      ++r->sequence;
    }
  }
}

// One rejected measurement: count the gap and mark both metrics FAULT. The
// driver stays initialised; start() re-inits only after kMaxGaps in a row.
static PollResult fail(Sgp30Driver* self, MonoMs now) {
  self->step_ = STEP_IDLE;
  if (self->consecutiveGaps_ < 0xFF) ++self->consecutiveGaps_;
  for (uint8_t i = 0; i < SGP30_METRICS; ++i) {
    self->results_[i].health = HEALTH_FAULT;
    self->results_[i].intervalStartMs = self->startMs_;
    self->results_[i].intervalEndMs = now;
  }
```

The driver fills identity-preserving result slots with interval, raw values, quality and health. Only declared usable outcomes advance lastGoodMs and the driver's usable sequence. Some diagnostic outcomes retain a numeric value without making it actionable.

src/drivers/sgp30.c / lines 265-267

```
  PollResult r = {DRIVER_FAULT, now};
  return r;
}
```

fail marks both metrics FAULT and increments the gap counter instead of immediately reinitialising. That preserves the chance to recover on the next scheduled command without paying a new warm-up every time.

Pack bounded command buffers and validate complete responses

src/drivers/sgp30.c / lines 268-291

```
268
269 static bool sendCommand(Sgp30Driver* self, uint16_t cmd) {
270   const uint8_t tx[2] = {(uint8_t)(cmd >> 8), (uint8_t)(cmd & 0xFF)};
271   return i2cBusWrite(self->config_.bus, self->config_.address, tx, sizeof(tx));
272 }
273
274 // Command followed by nWords (max 2) parameter words, each with its CRC.
275 static bool sendCommandWords(Sgp30Driver* self, uint16_t cmd, const uint16_t* words,
276                              uint8_t nWords) {
277   uint8_t tx[2 + 2 * 3];
278   if (nWords > 2) return false;
279   tx[0] = (uint8_t)(cmd >> 8);
280   tx[1] = (uint8_t)(cmd & 0xFF);
281   for (uint8_t i = 0; i < nWords; ++i) sgp30PackWord(words[i], &tx[2 + 3 * i]);
282   return i2cBusWrite(self->config_.bus, self->config_.address, tx, 2 + 3 *
  >   (size_t)nWords);
283 }
284
285 static bool readWords(Sgp30Driver* self, uint16_t* out, uint8_t nWords) {
286   uint8_t rx[2 * 3];
287   if (nWords > 2) return false;
288   const size_t len = 3 * (size_t)nWords;
289   return i2cBusRead(self->config_.bus, self->config_.address, rx, len) &&
290          sgp30UnpackWords(rx, nWords, out);
291 }
```

SendCommand emits two command bytes. SendCommandWords bounds nWords before filling its fixed buffer, then appends three-byte protected words. ReadWords requests exactly the expected byte count and calls the pure CRC decoder.

The byte count is derived from the number of parameter words. This is a concrete example of checking capacity before indexing or copying, rather than noticing overflow after the transaction was already built.

Copy results and stop crediting abandoned steps

src/drivers/sgp30.c / lines 292-310

```
292
293 static uint8_t sgp30ReadResult(SensorDriver* d, Reading* out, uint8_t maxOut) {
294   Sgp30Driver* self = asSgp30(d);
295   if (out == NULL) return 0;
296   const uint8_t n = maxOut < SGP30_METRICS ? maxOut : SGP30_METRICS;
297   for (uint8_t i = 0; i < n; ++i) out[i] = self->results_[i];
298   return n;
299 }
300
301 // A started measurement finishes by itself inside the chip.
302 static void sgp30Cancel(SensorDriver* d) { asSgp30(d)->step_ = STEP_IDLE; }
303
304 // The 3.3 V rail stays up in low power (§3); just require a fresh init(),
305 // which re-applies the RAM baseline.
306 static void sgp30PreparePowerOff(SensorDriver* d) {
307   Sgp30Driver* self = asSgp30(d);
308   self->step_ = STEP_IDLE;
309   self->initialised_ = false;
310 }
```

ReadResult copies up to the smaller of caller capacity and two metrics. Cancel drops the private step. PreparePowerOff also clears initialisation, allowing a future init to restore the RAM baseline and reestablish timing.

The sensor may finish an internal conversion after cancellation. The software contract simply refuses to treat it as the pending operation that was abandoned. The bus is not read speculatively during cleanup.

Separate a changed correction from fresh evidence

src/drivers/sgp30.c / lines 311-338

```
311
312 static const SensorDriverOps kSgp30Ops = {
313     .id = sgp30Id,
314     .name = sgp30Name,
315     .metricCount = sgp30MetricCount,
316     .init = sgp30Init,
317     .start = sgp30Start,
318     .poll = sgp30Poll,
319     .readResult = sgp30ReadResult,
320     .cancel = sgp30Cancel,
321     .preparePowerOff = sgp30PreparePowerOff,
322 };
323
324 void sgp30DriverSetHumidity(Sgp30Driver* self, const
  >   Reading* airTemperature,
325                             const Reading* airHumidity,
  >   MonoMs now) {
326   self->humidityEnabled_ = true;
327   float grams = -1.0f;
328   if (readingIsActionableAt(airTemperature, now,
  >   CFG_STALE_BME280_MS) &&
329       readingIsActionableAt(airHumidity, now,
  >   CFG_STALE_BME280_MS)) {
330     grams = sgp30AbsoluteHumidity(airTemperature->value,
  >   airHumidity->value);
331   }
332   // Stale or invalid input: write 0, which switches
  >   compensation off, rather
333   // than let the chip keep an old value silently (§10).
334   const uint16_t value = grams > 0.0f ?
  >   sgp30GramsToFixed8p8(grams) : 0;
335   self->humidityValid_ = value != 0;
336   if (self->humidityValid_) {
337     const MonoMs t = airTemperature->intervalEndMs;
338     const MonoMs h = airHumidity->intervalEndMs;
```

COMMON PITFALL

The source uses strict positive absolute humidity to enable correction. A zero correction word means off under this protocol.

The operations table precedes the public setter. SetHumidity marks compensation requested, accepts only fresh temperature and humidity, and converts their values to a fixed-point word. The retained timestamp is the older input's end time.

Separate a changed correction from fresh evidence / continued

src/drivers/sgp30.c / lines 339-347

```
    self->humidityInputMs_ = t < h ? t : h;  // age of the older input
  }
  const uint16_t old = self->humidity8p8_;
  const uint16_t diff = value > old ? (uint16_t)(value - old) : (uint16_t)(old - value);
  if ((value == 0) != (old == 0) || diff >= kHumidityDeadband8p8) {
    self->humidity8p8_ = value;
    self->humidityDirty_ = true;
  }
}
```

A deadband suppresses tiny numerical changes, but transitions between zero and nonzero always count as meaningful. humidityDirty requests a later write; the setter itself performs no I2C traffic.

Stage a restore and expose only a usable baseline copy

src/drivers/sgp30.c / lines 348-364

```
348
349 void sgp30DriverSetBaseline(Sgp30Driver* self, uint16_t co2eq, uint16_t tvoc) {
350   self->pendingCo2eq_ = co2eq;
351   self->pendingTvoc_ = tvoc;
352   self->hasPendingBaseline_ = true;
353 }
354
355 bool sgp30DriverGetBaseline(const Sgp30Driver* self, uint16_t* co2eq, uint16_t* tvoc) {
356   const bool usable = self->hasBaseline_ &&
357                       (self->baselineStatus_ == SGP30_BASELINE_VALID ||
358                        self->baselineStatus_ == SGP30_BASELINE_RESTORED);
359   if (usable) {
360     if (co2eq != NULL) *co2eq = self->baselineCo2eq_;
361     if (tvoc != NULL) *tvoc = self->baselineTvoc_;
362   }
363   return usable;
364 }
```

SetBaseline records a pending pair for the next init rather than issuing a command immediately. GetBaseline checks hasBaseline and the restored/valid status before writing optional outputs.

A caller that wants persistence should use the boolean, not merely nonzero retained words. The learning state deliberately withholds a baseline under this project's storage policy.

Construct stable result identities once

src/drivers/sgp30.c / lines 365-392

```
365
366 Sgp30BaselineStatus sgp30DriverBaselineStatus(const Sgp30Driver* self) {
367   return self->baselineStatus_;
368 }
369
370 const char* sgp30BaselineStatusName(Sgp30BaselineStatus s) {
371   switch (s) {
372     case SGP30_BASELINE_NONE: return "NONE";
373     case SGP30_BASELINE_LEARNING: return "LEARNING";
374     case SGP30_BASELINE_RESTORED: return "RESTORED";
375     case SGP30_BASELINE_VALID: return "VALID";
376     default: return "?";
377   }
378 }
379
380 uint16_t sgp30DriverFeatureSet(const Sgp30Driver* self) { return self->featureSet_; }
381
382 // Readings, baseline and humidity state are reset once here so they survive
383 // re-inits.
384 void sgp30DriverConstruct(Sgp30Driver* self, const Sgp30Config* config) {
385   memset(self, 0, sizeof(*self));
386   self->base.ops = &kSgp30Ops;
387   self->config_ = *config;
388   self->step_ = STEP_IDLE;
389   self->baselineStatus_ = SGP30_BASELINE_NONE;
390   readingReset(&self->results_[0], SENSOR_SGP30, METRIC_TVOC, UNIT_PPB);
391   readingReset(&self->results_[1], SENSOR_SGP30, METRIC_CO2EQ, UNIT_PPM);
392 }
```

The getters provide state and feature-set names. Construction zeroes the object, installs the table, copies configuration and resets the TVOC and CO2eq result identities with different units.

The initial baseline status is NONE. Later init chooses restore or learning. That sequence keeps construction of a C object distinct from communication with an actual gas sensor.

Report changed state and changed baseline words

The isolated gas test exposes warm-up, baseline state and optional restoration without a climate dependency.

An absent humidity setter means uncompensated operation, not a stale required humidity input.

test/bringup/sgp30_main.c / lines 1-28

```
// Bring-up entry point for sgp30 (WP6).  pio run -e test_sgp30 -t upload -t monitor
// Retained 3.3 V supply, shared I2C bus. Expect 2 readings every 1 s, ~13 ms
// each. The first 15 s after init are WARMING (fixed 400 ppm / 0 ppb), never
// published. Without a restored baseline the values then print "(uncal)"
// (baseline LEARNING, D13) for 12 h; after that the baseline is VALID, the
// values turn OK and the baseline words are printed once an hour.
// Humidity compensation is not used here (one driver per harness), so OK
// values are uncompensated.
// Replaces src/main.c in this env only; links the production driver.
#include <Arduino.h>
#include <stdio.h>

#include "app_config.h"
#include "board_profile.h"
#include "bringup_harness.h"
#include "hal/i2c_bus.h"
#include "drivers/sgp30.h"

// Baseline to restore at init, as printed by an earlier run ("baseline co2eq
// 0x.... tvoc 0x...."). Both 0 = none: the sensor learns from scratch.
// Sensirion: a stored baseline is valid for up to 7 days with the sensor off.
#define SGP30_RESTORE_CO2EQ 0x0000
#define SGP30_RESTORE_TVOC 0x0000

static I2cBus s_i2c = I2C_BUS_INIT;
static Sgp30Config s_config = {&s_i2c, BOARD_SGP30_I2C_ADDRESS};
static Sgp30Driver s_driver;

```

The static variables remember the last reported status and words. The observer prints a state transition and only prints an available baseline when the copied words changed.

test/bringup/sgp30_main.c / lines 29-37

```
static Sgp30BaselineStatus s_lastStatus = SGP30_BASELINE_NONE;
static uint16_t s_lastCo2eq = 0;
static uint16_t s_lastTvoc = 0;

static void printBaseline(SensorDriver* d) {
  (void)d;
  const Sgp30BaselineStatus st = sgp30DriverBaselineStatus(&s_driver);
  if (st != s_lastStatus) {
    printf("[sgp30] baseline %s (feature set 0x%04X)\n", sgp30BaselineStatusName(st),
```

Report changed state and changed baseline words / continued

test/bringup/sgp30_main.c / lines 38-49

```
38            (unsigned)sgp30DriverFeatureSet(&s_driver));
39      s_lastStatus = st;
40    }
41    uint16_t co2eq = 0;
42    uint16_t tvoc = 0;
43    if (sgp30DriverGetBaseline(&s_driver, &co2eq, &tvoc) &&
44        (co2eq != s_lastCo2eq || tvoc != s_lastTvoc)) {
45      printf("[sgp30] baseline co2eq 0x%04X tvoc 0x%04X\n", (unsigned)co2eq,
 >    (unsigned)tvoc);
46      s_lastCo2eq = co2eq;
47      s_lastTvoc = tvoc;
48    }
49  }
```

GetBaseline's boolean prevents a learning-state value from being printed as a baseline worth saving. The callback runs in the harness owner, so it does not compete for internal driver state.

Stage restoration before init begins

test/bringup/sgp30_main.c / lines 50-71

```
50
51 static BringupContext s_ctx = {&s_driver.base, CFG_FRAME_PERIOD_MS, printBaseline};
52
53 void setup(void) {
54   sgp30DriverConstruct(&s_driver, &s_config);
55   if (SGP30_RESTORE_CO2EQ != 0 || SGP30_RESTORE_TVOC != 0) {
56     sgp30DriverSetBaseline(&s_driver, SGP30_RESTORE_CO2EQ, SGP30_RESTORE_TVOC);
57   }
58   delay(200);
59   printf("\n=== bring-up: %s (SDA %d, SCL %d) ===\n", sensorDriverName(&s_driver.base),
60          BOARD_PIN_I2C_SDA, BOARD_PIN_I2C_SCL);
61   i2cBusBegin(&s_i2c, BOARD_PIN_I2C_SDA, BOARD_PIN_I2C_SCL, BOARD_I2C_CLOCK_HZ);
62   printf("probe 0x%02X -> %s\n", BOARD_SGP30_I2C_ADDRESS,
63          i2cBusProbe(&s_i2c, BOARD_SGP30_I2C_ADDRESS) ? "ACK" : "-");
64   if (SGP30_RESTORE_CO2EQ != 0 || SGP30_RESTORE_TVOC != 0) {
65     printf("restoring baseline co2eq 0x%04X tvoc 0x%04X\n",
 >   (unsigned)SGP30_RESTORE_CO2EQ,
66            (unsigned)SGP30_RESTORE_TVOC);
67   }
68   bringupStart(&s_ctx);
69 }
70
71 void loop(void) { vTaskDelay(portMAX_DELAY); }
```

Setup constructs the driver, optionally offers restore words, starts the shared bus and prints the address probe before starting the harness. Init later consumes the pending baseline and sends the appropriate command sequence.

The source includes a retention-age comment for manually reused baselines. The constants themselves contain no timestamp, so this test cannot enforce that age. A user copying old words must keep their collection conditions and age separately; the numeric pair does not establish its own history.

Give each device its own run record on one shared bus

This test joins the BME280 and SGP30 on one bus. It shows both a physical sharing relationship and a data dependency.

One task owns the bus and releases four climate ticks per gas cycle.

test/bringup/climate_main.c / lines 1-28

```
// Combined bring-up for the shared I2C bus: BME280 (WP1) + SGP30 (WP6).
//   pio run -e test_climate -t upload -t monitor
// Both on the retained 3.3 V supply, SDA 21 / SCL 47, addresses 0x76/0x77 and
// 0x58. Not the production scheduler (WP7): one task owns the bus and runs a
// 250 ms tick. Each tick runs one BME280 measurement; every 4th tick (1 s) it
// passes the fresh BME280 T/RH to the SGP30 as humidity compensation, then
// runs one SGP30 measurement. One labelled line per second:
//   t=   17s  BME280  T  25.62 C  RH  58.28 %  P 1022.96 hPa  AH 13.87 g/m3  |
//     SGP30  TVOC     5 ppb  CO2eq   413 ppm  LEARNING          (one line)
// A value that is not usable shows its health instead (FAULT, warm, ...).
// state = SGP30 health: WARMING, LEARNING (baseline not yet learned, D13),
// OK, DEP_INVALID (humidity input stale), FAULT; "nocomp" = no humidity
// compensation. Events (init, faults, baseline and compensation changes,
// a driver over its time budget) print as their own "# ..." lines.
// Replaces src/main.c in this env only; links the production drivers.
#include <Arduino.h>
#include <stdio.h>
#include <string.h>

#include "app_config.h"
#include "board_profile.h"
#include "bringup_harness.h"
#include "domain/sgp30_codec.h"
#include "drivers/bme280.h"
#include "drivers/sgp30.h"
#include "hal/clock.h"
#include "hal/i2c_bus.h"

```

The opening contract explains a 250ms climate tick and a gas measurement every fourth tick. The objects share s_i2c but keep separate concrete driver state. DriverRun retains copied outputs and timing for presentation.

test/bringup/climate_main.c / lines 29-36

```
#define CLIMATE_SGP30_EVERY_TICKS 4  // 4 x 250 ms = the SGP30's 1 s cadence
#define CLIMATE_SGP30_BUDGET_MS 15   // architecture §9 host budget

static I2cBus s_i2c = I2C_BUS_INIT;
static Bme280Config s_bmeConfig;  // address chosen at runtime in setup()
static Bme280Driver s_bme;
static Sgp30Config s_sgpConfig = {&s_i2c, BOARD_SGP30_I2C_ADDRESS};
static Sgp30Driver s_sgp;
```

Give each device its own run record on one shared bus / continued

test/bringup/climate_main.c / lines 37-47

```
37
38 typedef struct DriverRun {
39   SensorDriver* driver;
40   bool initialised;
41   uint32_t guardMs;  // harness-level guard, not the driver budget
42   Reading out[4];
43   uint8_t n;
44   uint32_t elapsedMs;
45 } DriverRun;
46
47 // One init (if needed) -> start -> poll until READY/error -> readResult.
```

Sharing a bus does not mean sharing a driver object. The one task chooses the transaction order and supplies humidity after climate values become available.

Reuse init and start with explicit failure exits

test/bringup/climate_main.c / lines 48-72

```
// Same logic as bringupTask(), for one measurement. true if readings arrived.
static bool runOnce(DriverRun* run) {
  SensorDriver* drv = run->driver;
  run->n = 0;
  if (!run->initialised) {
    const DriverStatus st = sensorDriverInit(drv);
    printf("# %s init -> %s\n", sensorDriverName(drv), driverStatusName(st));
    if (st != DRIVER_OK) return false;
    run->initialised = true;
  }

  const MonoMs t0 = monoNowMs();
  const DriverStatus st = sensorDriverStart(drv, t0);
  if (st != DRIVER_OK) {
    printf("# %s start -> %s\n", sensorDriverName(drv), driverStatusName(st));
    if (st != DRIVER_PENDING) run->initialised = false;
    return false;
  }

  PollResult pr = sensorDriverPoll(drv, monoNowMs());
  while (pr.status == DRIVER_PENDING) {
    const MonoMs now = monoNowMs();
    if (now - t0 >= run->guardMs) {
      sensorDriverCancel(drv);
      pr.status = DRIVER_TIMEOUT;
```

runOnce clears its output count and initialises the device only when required. A failed setup or start returns false before output collection. The boolean tells the test caller whether this run yielded copied records.

t0 is captured after init, so the reported operation duration concerns start through completion rather than the full setup cost. Know which interval a timing measurement actually covers before comparing it with a budget.

Wait between polls and apply the harness guard

test/bringup/climate_main.c / lines 73-91

```
      break;
    }
    uint32_t waitMs = 1;
    if (pr.nextPollMs > now) waitMs = (uint32_t)(pr.nextPollMs - now);
    const TickType_t ticks = pdMS_TO_TICKS(waitMs);
    vTaskDelay(ticks > 0 ? ticks : 1);
    pr = sensorDriverPoll(drv, monoNowMs());
  }

  run->elapsedMs = (uint32_t)(monoNowMs() - t0);
  if (pr.status != DRIVER_READY) {
    printf("# %s poll -> %s after %lu ms\n", sensorDriverName(drv),
           driverStatusName(pr.status), (unsigned long)run->elapsedMs);
    if (pr.status == DRIVER_NOT_PRESENT) run->initialised = false;
    return false;
  }
  run->n = sensorDriverReadResult(drv, run->out, 4);
  return true;
}
```

The pending loop respects nextPollMs and forces at least one tick. Its outer guard cancels overlong work. A non-READY end prints status and may clear initialisation; a READY end copies up to four records into the run's own output array.

The caller then owns those copies. Printing or comparing them need not perform new bus transactions or borrow a pointer to a mutable driver result.

Print transitions rather than bury them in repeated lines

test/bringup/climate_main.c / lines 92-119

```
92
93 static void printSgp30Baseline(void) {
94   static Sgp30BaselineStatus lastStatus = SGP30_BASELINE_NONE;
95   static uint16_t lastCo2eq = 0;
96   static uint16_t lastTvoc = 0;
97   const Sgp30BaselineStatus st = sgp30DriverBaselineStatus(&s_sgp);
98   if (st != lastStatus) {
99     printf("# sgp30 baseline %s (feature set 0x%04X)\n", sgp30BaselineStatusName(st),
100           (unsigned)sgp30DriverFeatureSet(&s_sgp));
101    lastStatus = st;
102  }
103  uint16_t co2eq = 0;
104  uint16_t tvoc = 0;
105  if (sgp30DriverGetBaseline(&s_sgp, &co2eq, &tvoc) && (co2eq != lastCo2eq || tvoc !=
  >  lastTvoc)) {
106    printf("# sgp30 baseline co2eq 0x%04X tvoc 0x%04X (store these)\n", (unsigned)co2eq,
107           (unsigned)tvoc);
108    lastCo2eq = co2eq;
109    lastTvoc = tvoc;
110  }
111 }
112
113 // First timing once, then only a run over its budget.
114 static void checkTiming(const char* name, const DriverRun* run, uint32_t budgetMs,
115                         bool* reported) {
116   if (!*reported) {
117     printf("# %s took %lu ms (budget %lu ms)\n", name, (unsigned long)run->elapsedMs,
118            (unsigned long)budgetMs);
119     *reported = true;
```

The baseline observer retains previous state and word values. checkTiming reports the first completed duration, then reports later overruns. This makes timing deviations visible without filling the console with identical budget declarations.

test/bringup/climate_main.c / lines 120-124

```
120   } else if (run->elapsedMs > budgetMs) {
121     printf("# %s took %lu ms, over its %lu ms budget\n", name, (unsigned
  >  long)run->elapsedMs,
122            (unsigned long)budgetMs);
123   }
124 }
```

A printed duration is still a sampled measurement under this experiment's load. It does not establish the worst case for production with radio, flash and power transitions active.

Keep display scaling distinct from sensor units

test/bringup/climate_main.c / lines 125-147

```
125
126 // One labelled value, e.g. "T 25.65 C". Not usable: its health, e.g. "T FAULT".
127 static void printCell(const char* label, const Reading* r, int width, int decimals,
128                        float scale, const char* unit) {
129   const Health h = r->health;
130   if (h == HEALTH_OK || h == HEALTH_UNCALIBRATED || h == HEALTH_DEPENDENCY_INVALID) {
131     printf("  %s %*.*f %s", label, width, decimals, (double)(r->value * scale), unit);
132     return;
133   }
134   const char* why = h == HEALTH_WARMING ? "warm" : h == HEALTH_STARTING ? "-" :
  >   healthName(h);
135   printf("  %s %*s %*s", label, width, why, (int)strlen(unit), "");
136 }
137
138 static const char* sgp30State(const Reading* r) {
139   switch (r->health) {
140     case HEALTH_OK: return "OK";
141     case HEALTH_UNCALIBRATED: return "LEARNING";
142     case HEALTH_DEPENDENCY_INVALID: return "DEP_INVALID";
143     case HEALTH_STARTING: return "-";
144     default: return healthName(r->health);
145   }
146 }
147
```

printCell selects a numeric display only for the test's declared diagnostic health states. printRow converts pressure pascals to displayed hectopascals with 0.01 while leaving the stored Reading unchanged.

Keep display scaling distinct from sensor units / continued

test/bringup/climate_main.c / lines 148-164

```
148 // e.g.  t=64s  BME280  T 25.65 C  RH 57.70 %  P 1022.89 hPa  AH 13.74 g/m3  |
149 //       SGP30  TVOC   22 ppb  CO2eq  413 ppm  LEARNING      (one line)
150 static void printRow(MonoMs now, const Reading* bme, bool ahValid, float ah,
151                       const Reading* sgp) {
152   printf("t=%5lus  BME280", (unsigned long)(now / 1000));
153   printCell("T", &bme[0], 6, 2, 1.0f, "C");
154   printCell("RH", &bme[1], 6, 2, 1.0f, "%");
155   printCell("P", &bme[2], 7, 2, 0.01f, "hPa");  // Pa -> hPa
156   if (ahValid) {
157     printf("  AH %5.2f g/m3", (double)ah);
158   } else {
159     printf("  AH %5s     ", "-");
160   }
161   printf("  |  SGP30");
162   printCell("TVOC", &sgp[0], 5, 0, 1.0f, "ppb");
163   printCell("CO2eq", &sgp[1], 5, 0, 1.0f, "ppm");
164   printf("  %s%s\n", sgp30State(&sgp[0]),
```

test/bringup/climate_main.c / lines 165-166

```
165           (sgp[0].qualityFlags & kQualityUncompensated) ? " nocomp" : "");
166 }
```

The gas state label and uncompensated tag remain visible beside its values. Showing a diagnostic number under DEPENDENCY_INVALID is not the same as permitting a control decision to use it.

Refresh climate four times and feed gas once per second

test/bringup/climate_main.c / lines 167-194

```
167
168 static void climateTask(void* arg) {
169   (void)arg;
170   DriverRun bme = {&s_bme.base, false, CFG_BME280_PERIOD_MS, {{0}}, 0, 0};
171   DriverRun sgp = {&s_sgp.base, false, CFG_BME280_PERIOD_MS, {{0}}, 0, 0};
172   bool bmeTimed = false;
173   bool sgpTimed = false;
174   int lastCompensated = -1;  // unknown
175   uint32_t tick = 0;
176   TickType_t lastWake = xTaskGetTickCount();
177
178   for (;;) {
179     vTaskDelayUntil(&lastWake, pdMS_TO_TICKS(CFG_BME280_PERIOD_MS));
180     const bool secondTick = (tick++ % CLIMATE_SGP30_EVERY_TICKS) == 0;
181
182     if (runOnce(&bme)) checkTiming("bme280", &bme, CFG_BME280_BUDGET_MS, &bmeTimed);
183     if (!secondTick) continue;
184
185     // Readings 0/1 are air temperature and humidity (bme280.h). A failed
186     // BME280 run leaves the last readings, which then age out as stale.
187     const MonoMs now = monoNowMs();
188     Reading latest[BME280_METRICS];
189     sensorDriverReadResult(&s_bme.base, latest, BME280_METRICS);
190     sgp30DriverSetHumidity(&s_sgp, &latest[0], &latest[1], now);
191     const bool fresh = readingIsActionableAt(&latest[0], now, CFG_STALE_BME280_MS) &&
192                        readingIsActionableAt(&latest[1], now, CFG_STALE_BME280_MS);
193     const float ah = fresh ? sgp30AbsoluteHumidity(latest[0].value, latest[1].value) :
  >   0.0f;
194     if ((int)fresh != lastCompensated) {
```

The task uses an absolute tick grid and a modulo-four counter. Every tick runs BME280; only the selected ticks continue into the gas path. It copies the latest climate records and checks their effective freshness before deriving absolute humidity.

Refresh climate four times and feed gas once per second / continued

test/bringup/climate_main.c / lines 195-206

```
195       printf(fresh ? "# sgp30 humidity compensation ON (from BME280)\n"
196                    : "# BME280 T/RH not fresh -> sgp30 humidity compensation OFF\n");
197       lastCompensated = fresh;
198     }
199
200     if (runOnce(&sgp)) checkTiming("sgp30", &sgp, CLIMATE_SGP30_BUDGET_MS, &sgpTimed);
201     Reading sgpOut[SGP30_METRICS];
202     sensorDriverReadResult(&s_sgp.base, sgpOut, SGP30_METRICS);
203     printRow(now, latest, fresh, ah, sgpOut);
204     printSgp30Baseline();
205   }
206 }
```

The same records are supplied to SetHumidity. If climate fails, retained values can age out instead of being stamped fresh merely because the gas task loop ran. The row and baseline diagnostics are then printed after the gas operation.

Choose both addresses before starting the shared owner

test/bringup/climate_main.c / lines 207-231

```
207
208 void setup(void) {
209   delay(200);
210   printf("\n=== bring-up: climate (bme280 + sgp30, SDA %d, SCL %d, %lu Hz) ===\n",
211          BOARD_PIN_I2C_SDA, BOARD_PIN_I2C_SCL, (unsigned long)BOARD_I2C_CLOCK_HZ);
212
213   const bool busOk = i2cBusBegin(&s_i2c, BOARD_PIN_I2C_SDA, BOARD_PIN_I2C_SCL,
214                                  BOARD_I2C_CLOCK_HZ);
215   printf("i2c begin -> %s\n", busOk ? "OK" : "FAILED");
216
217   // BME280: whichever address ACKs (SDO strap); default if neither does.
218   uint8_t bmeAddress = BOARD_BME280_I2C_ADDRESS;
219   const bool ack76 = i2cBusProbe(&s_i2c, BOARD_BME280_I2C_ADDRESS);
220   const bool ack77 = i2cBusProbe(&s_i2c, BOARD_BME280_I2C_ADDRESS_ALT);
221   const bool ack58 = i2cBusProbe(&s_i2c, BOARD_SGP30_I2C_ADDRESS);
222   printf("probe bme280 0x%02X -> %s, 0x%02X -> %s | sgp30 0x%02X -> %s\n",
223          BOARD_BME280_I2C_ADDRESS, ack76 ? "ACK" : "-", BOARD_BME280_I2C_ADDRESS_ALT,
224          ack77 ? "ACK" : "-", BOARD_SGP30_I2C_ADDRESS, ack58 ? "ACK" : "-");
225   if (!ack76 && ack77) bmeAddress = BOARD_BME280_I2C_ADDRESS_ALT;
226
227   s_bmeConfig.bus = &s_i2c;
228   s_bmeConfig.address = bmeAddress;
229   bme280DriverConstruct(&s_bme, &s_bmeConfig);
230   sgp30DriverConstruct(&s_sgp, &s_sgpConfig);
231
```

Setup probes both BME280 options and the fixed gas address. It copies the selected climate address into configuration, constructs both drivers and creates one climate task.

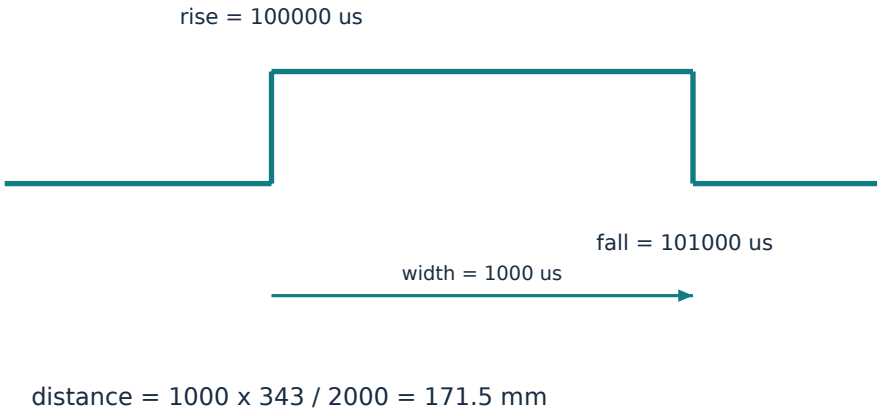
test/bringup/climate_main.c / lines 232-237

```
232   const TaskSpec* spec = cfgTaskSpec(TASK_ID_ACQUISITION_NORMAL);
233   xTaskCreatePinnedToCore(climateTask, "climate", spec->stackBytes, NULL,
  >   spec->priority, NULL,
234                           spec->core);
235 }
236
237 void loop(void) { vTaskDelay(portMAX_DELAY); }
```

Both protocols therefore use the same configured controller under one execution context. This is a smaller rehearsal for production acquisition, where more devices and timing exclusions make the same ownership rule more demanding.

The echo width gives a round trip

HC-SR04 exposes TRIG and ECHO. The firmware emits a short trigger pulse. The sensor then returns an echo pulse whose width represents the sound's outgoing and returning travel time. We need the difference between its rising and falling edges.



distance = 1000 x 343 / 2000 = 171.5 mm

Worked example using nominal speed. Task wake latency is a separate quantity.

COMMON PITFALL

Measuring when a task finally notices ECHO is not the same as measuring the edge itself.

Original firmware - src/domain/ultrasonic.c, lines 3-6

```
float ultrasonicEchoUsToMm(uint32_t widthUs, float speedMps) {
  // us * (m/s) = um; one way = half; um -> mm = /1000.
  return (float)widthUs * speedMps / 2000.0f;
}
```

The echo width gives a round trip / continued

For widthUs=1000 and speed=343m/s, the function returns 171.5mm. For 2000us it returns 343mm. Dividing by two is required because the pulse describes a round trip, not the one-way sensor-to-surface distance.

The project publishes distance, not tank volume. To obtain water height, you would need a measured mounting reference and a geometry model. To obtain volume, you would also need the tank shape. A sensor driver should not invent those application-specific facts.

The source uses a nominal sound speed and marks the Reading uncompensated. Changes in air conditions can change the distance estimate. Keeping this quality flag makes that assumption visible without mixing temperature compensation into the basic pulse-capture mechanism.

The board profile supplies the electrical interface and pins. This lesson follows the firmware measurement path, not a replacement wiring design.

Record edges in a short interrupt handler

Original firmware - src/hal/pulse_capture.c, lines 23-37

```
static void IRAM_ATTR onEchoEdge(void* arg) {
  PulseCapture* pc = (PulseCapture*)arg;
  const uint32_t t = nowUs();
  const CaptureState s = pc->state_;
  if (s != CAPTURE_WAITING && s != CAPTURE_HIGH) return;
  if (echoLevel(pc->echo_)) {
    if (s == CAPTURE_WAITING) {
      pc->riseUs_ = t;
      pc->state_ = CAPTURE_HIGH;
    }
  } else if (s == CAPTURE_HIGH) {
    pc->fallUs_ = t;
    pc->state_ = CAPTURE_DONE;
  }
}
```

COMMON PITFALL

Shared ISR state still needs a justified publication and access protocol. Adding volatile everywhere does not supply that protocol automatically.

An interrupt handler runs in response to the GPIO edge. arg points to the capture object registered with the handler. The cast lets the function access that object's fields. It gets a microsecond timestamp and inspects the current capture state.

While WAITING, a rising edge stores riseUs_ and changes the state to HIGH. While HIGH, a falling edge stores fallUs_ and then marks DONE. Edges outside the armed states are ignored. The order of the stores matters: DONE advertises that the completed timestamps have been written.

The handler does not calculate tank volume, print a line or run a queue wait. Its job is to capture the time-sensitive fact before returning. The task later converts the width and publishes an observation. That keeps the interrupt's execution short.

IRAM_ATTR requests placement of this handler in instruction RAM. It does not, by itself, prove that every function and data access in its call path is safe while flash cache is unavailable. That is why interrupt design needs a call-path review as well as an attribute.

This original driver uses timestamped state plus task polling. It does not send each echo through a FreeRTOS queue. The generic interrupt-notification example in the following lesson (p. 312) shows another handoff pattern; keep it separate from what this archived implementation actually does.

When an interrupt should wake a task

Teaching alternative - GPIO ISR; workerHandle set before enabling it

```
static void IRAM_ATTR onEvent(void *arg) {
    BaseType_t higherWoken = pdFALSE;
    vTaskNotifyGiveFromISR(workerHandle, &higherWoken);
    if (higherWoken == pdTRUE) {
        portYIELD_FROM_ISR(higherWoken);
    }
}
```

Use an ISR-safe API when a kernel operation is needed from an interrupt. vTaskNotifyGiveFromISR increments the worker's notification. higherWoken is an output flag: it records whether the operation unblocked a task that calls for the port's context-switch request. The yield request lets the scheduler react promptly when the interrupt exits.

The flag starts false for this invocation. Pass its address so the API can update it. Do not call ordinary blocking task APIs from this context. An interrupt cannot wait for a queue slot or a mutex holder to finish later.

The worker must still find the actual event information somewhere. For a timestamped pulse, store the timestamps in a properly protected or owned capture object, or send a small event record with an ISR-safe queue API. A notification alone contains no pulse width.

Why does the original HC-SR04 use polling? Its ISR already records precise edges, and acquisition revisits the state on a near-term deadline. A later poll changes when the result is noticed, not the stored edge width. A notification could reduce detection delay, but adding it is an architectural change to assess, not a syntax requirement.

The portYIELD_FROM_ISR form shown here matches the ESP32-style yield request used by the reference power-sense ISR. Timer-dispatch ISR facilities can have their own yield rules; follow the callback context's documentation instead of copying a GPIO handler into every interrupt-like callback.

Reference: Espressif: ESP-IDF FreeRTOS SMP, v4.4.2 (S3)

REMEMBER

An ISR is an interrupt context, not an unusually high-priority task.

Disarm a failed pulse before starting another

hcsr04Poll first asks pulseCapturePoll for state and width. If capture is DONE, it converts the timestamped width and checks the allowed range. If still WAITING or HIGH before the deadline, it returns PENDING. At the deadline, an unfinished pulse becomes TIMEOUT.

A timeout is not a distance of zero. Zero could be misread as a real nearby surface. The driver sets health and leaves the old value field uninterpretable as a fresh measurement. It distinguishes never-rose from rose-but-never-fell through raw-availability information.

Before reporting timeout, it cancels capture. Otherwise a very late falling edge could be mistaken for part of the next ping. Re-arming first sets an idle state, clears timestamps, checks ECHO is not already high, and then emits the trigger.

COMMON PITFALL

A 1ms poll request does not guarantee 1ms noticing latency. Tick rate, priorities and blocking calls still affect when the poll executes.

Unsigned subtraction across a wrap

Teaching example - C

```c
uint32_t rise = 0xFFFFFF00u;
uint32_t fall = 0x000002E8u;
uint32_t width = fall - rise;
// width = 1000, using unsigned modulo arithmetic
```

The 32-bit microsecond counter wraps about every 71.6 minutes. The subtraction works for this bounded interval even though the displayed fall timestamp is numerically smaller. It does not let you recover an interval spanning an unknown number of wraps.

A late task poll can accept a DONE pulse because the edges were timestamped earlier. In this source, DONE is checked before the pending-operation timeout. The task's poll time and the physical edge times therefore answer different questions. The acquisition frame guard can still abandon a run that exceeds the wider service schedule.

Give the raw width a physical interpretation

This header states the nominal distance model and acceptance range. It does not measure an edge.

Keeping the conversion pure lets the host tests check distance independently of interrupt timing.

src/domain/ultrasonic.h / lines 1-21

```
// HC-SR04 echo-time conversion (WP4). Pure C, host-tested.
// Source: HC-SR04.pdf (module guide) p.2 specifications, p.4 equations 2-3.
//
// ECHO stays high for the sound's round trip, so distance = width * c / 2.
// With c = 343 m/s (20 degC, datasheet) that is 58.3 us per cm. No temperature
// compensation yet: c changes ~0.17 %/degC.
#pragma once
#include <stdbool.h>
#include <stdint.h>

#define ULTRASONIC_NOMINAL_SPEED_MPS 343.0f

// Datasheet ranging distance 2-400 cm.
#define ULTRASONIC_MIN_MM 20.0f
#define ULTRASONIC_MAX_MM 4000.0f

// Echo pulse width (us) -> one-way distance (mm).
float ultrasonicEchoUsToMm(uint32_t widthUs, float speedMps);

// True inside the datasheet ranging distance (inclusive).
bool ultrasonicInRangeMm(float mm);
```

The nominal sound speed and minimum/maximum millimetres define this model. Conversion accepts width in microseconds and speed in metres per second. Range checking accepts or rejects the resulting distance.

Changing the tank mounting reference is a later application calculation, not a reason to change pulse-width decoding. The driver publishes sensor-to-target distance, not litres or water depth.

Divide the round trip and convert units

The complete arithmetic is small enough to verify by units.

A 1000us pulse should produce 171.5mm at 343m/s.

src/domain/ultrasonic.c / lines 1-10

```c
#include "domain/ultrasonic.h"

float ultrasonicEchoUsToMm(uint32_t widthUs, float speedMps) {
  // us * (m/s) = um; one way = half; um -> mm = /1000.
  return (float)widthUs * speedMps / 2000.0f;
}

bool ultrasonicInRangeMm(float mm) {
  return mm >= ULTRASONIC_MIN_MM && mm <= ULTRASONIC_MAX_MM;
}
```

Microseconds times metres per second yields micrometres. Divide by two for the one-way path and by 1000 for millimetres, combined as 2000.0f. The cast and float constants make this floating-point arithmetic.

The range helper uses inclusive limits. A measured distance beyond those limits can still be useful raw diagnostic evidence, but the driver will label it a fault rather than claim it is an accepted observation.

Keep the capture state machine separate from timeout policy

This capture object is shared between a GPIO handler and its owning task. Read every field in terms of who writes it and when the other side may use it.

The task's poll rate affects noticing latency, while edge timestamps determine the stored pulse width.

src/hal/pulse_capture.h / lines 1-28

```
// Echo-pulse capture (HAL). Work package WP4 (with HC-SR04).
// Use RMT or an audited edge-capture ISR - not pulseIn() (§2, §7).
//
// Implementation: CHANGE interrupt on ECHO + esp_timer microsecond timestamps
// (decision WP4, Oct 2026). The ISR only records the two edges; the width is
// read by pulseCapturePoll() from task context. Timestamps are the low 32 bits
// of esp_timer_get_time(), so the ISR -> task handoff is single-word writes
// (no lock) and widths are a modulo-2^32 subtraction (valid for ~71 min).
// Accuracy does not depend on how often poll() is called. ISR latency jitter
// is a few us (< 1 mm of distance).
//
// The timeout policy belongs to the caller (driver budget): this HAL never
// gives up by itself. Cancel after a timeout so a late falling edge is never
// credited to the next trigger.
#pragma once
#include <stdbool.h>
#include <stdint.h>

typedef uint8_t CaptureState;
enum {
  CAPTURE_IDLE = 0,  // not armed (begun, cancelled, or nothing triggered)
  CAPTURE_WAITING,   // triggered, ECHO has not risen yet
  CAPTURE_DONE,      // rising and falling edge seen, width available
  CAPTURE_TIMEOUT,   // unused by this HAL (timeouts are the driver's), kept for callers
  CAPTURE_ERROR,     // not begun
  CAPTURE_HIGH       // ECHO rose, falling edge not seen yet
};

```

The state enum distinguishes idle, waiting, high and done. TIMEOUT remains a named value for callers but this HAL does not autonomously declare it; the driver owns that deadline.

src/hal/pulse_capture.h / lines 29-29

```
// Treat the members as private to pulse_capture.c. Initialise with PULSE_CAPTURE_INIT.
```

The source comment describes expected interrupt accuracy. That is a timing claim to measure under the actual system load, especially while other code masks interrupts. A fine timestamp API does not guarantee an equally fine interrupt response.

Record single-word timestamps with a justified handoff

src/hal/pulse_capture.h / lines 30-57

```
30 typedef struct PulseCapture {
31   int8_t trig_;
32   int8_t echo_;
33   bool begun_;
34   volatile CaptureState state_;  // written last by the ISR
35   volatile uint32_t riseUs_;
36   volatile uint32_t fallUs_;
37   uint32_t trigUs_;
38 } PulseCapture;
39
40 #define PULSE_CAPTURE_INIT {-1, -1, false, CAPTURE_IDLE, 0,
 >   0, 0}
41
42 // TRIG output low, ECHO input with NO pull (a pull-up would
 >    back-feed the
43 // unpowered 5 V module through the divider), CHANGE
 >   interrupt attached.
44 // false if a pin is invalid.
45 bool pulseCaptureBegin(PulseCapture* pc, int8_t trigPin,
 >   int8_t echoPin);
46
47 // 10 us trigger pulse, arm capture. Busy-waits ~12 us (2 us
 >    low + 10 us high).
48 // false (nothing sent) if not begun or ECHO is already
 >   high: a stuck line or a
49 // late echo from the previous ping.
50 bool pulseCaptureTrigger(PulseCapture* pc);
51
52 // Non-blocking. *widthUs = pulse width when DONE, else 0.
53 CaptureState pulseCapturePoll(PulseCapture* pc, uint32_t*
 >   widthUs);
54
55 // Microseconds since the last trigger (0 if never
 >   triggered).
56 uint32_t pulseCaptureElapsedUs(const PulseCapture* pc);
57
```

COMMON PITFALL

Writing volatile on a field does not by itself prove a portable cross-core memory-ordering protocol.

Record single-word timestamps with a justified handoff / continued

The object stores volatile edge timestamps and a state published after relevant writes. These fields are a specialised ISR handoff, not a general substitute for queues or atomic record publication.

src/hal/pulse_capture.h / lines 58-62

```
58 // Current ECHO level (false if not begun).
59 bool pulseCaptureEchoHigh(const PulseCapture* pc);
60
61 void pulseCaptureCancel(PulseCapture* pc);                    // disarm, back to IDLE
62 void pulseCaptureSetSafe(PulseCapture* pc);                    // TRIG low, ECHO input,
 >    no pulls (§12)
```

Begin establishes pin modes and the handler; Trigger arms and emits a short pulse; Poll copies width only when DONE; Cancel disarms; SetSafe also leaves the interface suitable for a lost 5V supply. The caller must honour the capture ownership and cancellation sequence.

Read the correct GPIO register bank

We will read the edge handler, arming sequence and task-side inspection in full. Their ordering is the core of this measurement.

The driver above the HAL supplies the 30ms budget and converts accepted widths.

src/hal/pulse_capture.c / lines 1-18

```
#include "hal/pulse_capture.h"

#include <Arduino.h>
#include <esp_attr.h>
#include <esp_timer.h>
#include <soc/gpio_reg.h>
#include <stddef.h>

// ESP32-S3 GPIO numbers run 0-48.
static const int8_t kMaxGpio = 48;

static inline uint32_t nowUs(void) { return (uint32_t)esp_timer_get_time(); }

// Direct register read: IRAM-safe, unlike digitalRead().
static inline IRAM_ATTR bool echoLevel(int8_t pin) {
  if (pin < 32) return (REG_READ(GPIO_IN_REG) >> pin) & 1u;
  return (REG_READ(GPIO_IN1_REG) >> (pin - 32)) & 1u;
}
```

nowUs keeps the low 32 bits of the microsecond clock. echoLevel selects the register bank by pin number, then shifts and masks the desired bit. It avoids a higher-level digitalRead call in the edge path.

The declared IRAM placement of a helper is part of flash-interference reasoning. Review the whole call and data path before concluding that a handler can run during every cache-disabled interval.

Write a timestamp before advertising its state

src/hal/pulse_capture.c / lines 19-37

```
19
20 // Only records edges while armed. Rising edge -> HIGH; falling edge while HIGH
21 // -> DONE. The state is written after the timestamp, so a DONE seen by poll()
22 // always has a valid fallUs_.
23 static void IRAM_ATTR onEchoEdge(void* arg) {
24   PulseCapture* pc = (PulseCapture*)arg;
25   const uint32_t t = nowUs();
26   const CaptureState s = pc->state_;
27   if (s != CAPTURE_WAITING && s != CAPTURE_HIGH) return;
28   if (echoLevel(pc->echo_)) {
29     if (s == CAPTURE_WAITING) {
30       pc->riseUs_ = t;
31       pc->state_ = CAPTURE_HIGH;
32     }
33   } else if (s == CAPTURE_HIGH) {
34     pc->fallUs_ = t;
35     pc->state_ = CAPTURE_DONE;
36   }
37 }
```

The handler ignores edges outside WAITING or HIGH. An accepted rising edge writes riseUs_ before setting HIGH. An accepted falling edge writes fallUs_ before setting DONE.

The task can then interpret DONE as a completed pair under this implementation's access assumptions. Reversing those two writes would advertise a result before its second timestamp exists. The handler performs no distance calculation or serial output.

Put pins in known states before enabling captures

src/hal/pulse_capture.c / lines 38-55

```
38
39 bool pulseCaptureBegin(PulseCapture* pc, int8_t trigPin, int8_t echoPin) {
40   pc->begun_ = false;
41   pc->state_ = CAPTURE_IDLE;
42   pc->trig_ = trigPin;
43   pc->echo_ = echoPin;
44   if (trigPin < 0 || trigPin > kMaxGpio || echoPin < 0 || echoPin > kMaxGpio ||
45       trigPin == echoPin) {
46     return false;
47   }
48   digitalWrite((uint8_t)trigPin, LOW);
49   pinMode((uint8_t)trigPin, OUTPUT);
50   digitalWrite((uint8_t)trigPin, LOW);
51   pinMode((uint8_t)echoPin, INPUT);  // no pull: no back-feeding the 5 V module
52   attachInterruptArg((uint8_t)echoPin, onEchoEdge, pc, CHANGE);
53   pc->begun_ = true;
54   return true;
55 }
```

Begin clears software state, records pins, rejects invalid or identical assignments, sets TRIG low and configures ECHO as an input without pulls. It attaches a CHANGE handler that receives this object's address.

The object must persist after Begin returns because later interrupts use that address. A capture object local to a short setup helper would leave a dangling callback context.

Disarm, clear and rearm in an intentional order

src/hal/pulse_capture.c / lines 56-71

```
56
57 bool pulseCaptureTrigger(PulseCapture* pc) {
58   if (!pc->begun_) return false;
59   pc->state_ = CAPTURE_IDLE;  // ignore edges while re-arming
60   if (echoLevel(pc->echo_)) return false;
61   pc->riseUs_ = 0;
62   pc->fallUs_ = 0;
63   digitalWrite((uint8_t)pc->trig_, LOW);
64   delayMicroseconds(2);
65   pc->trigUs_ = nowUs();
66   pc->state_ = CAPTURE_WAITING;
67   digitalWrite((uint8_t)pc->trig_, HIGH);
68   delayMicroseconds(10);  // datasheet: >= 10 us TTL pulse
69   digitalWrite((uint8_t)pc->trig_, LOW);
70   return true;
71 }
```

Trigger first ignores edges while rearming and rejects ECHO already high. It clears old edge times, ensures a low preamble, stores trigger time, marks WAITING and produces the high trigger pulse.

The short busy waits define the trigger waveform. They are not a wait for the echo. The task can leave after those microseconds while the edge handler records the later response.

Read completed width without taking another measurement

src/hal/pulse_capture.c / lines 72-88

```
72
73 CaptureState pulseCapturePoll(PulseCapture* pc, uint32_t* widthUs) {
74   if (widthUs) *widthUs = 0;
75   if (!pc->begun_) return CAPTURE_ERROR;
76   const CaptureState s = pc->state_;
77   if (s == CAPTURE_DONE && widthUs) *widthUs = pc->fallUs_ - pc->riseUs_;
78   return s;
79 }
80
81 uint32_t pulseCaptureElapsedUs(const PulseCapture* pc) {
82   if (pc->trigUs_ == 0) return 0;
83   return nowUs() - pc->trigUs_;
84 }
85
86 bool pulseCaptureEchoHigh(const PulseCapture* pc) {
87   return pc->begun_ && echoLevel(pc->echo_);
88 }
```

Poll initialises the optional width output to zero, rejects an unbegun capture and subtracts timestamps only for DONE. ElapsedUs measures time since trigger for timeout diagnostics. EchoHigh reads the current level when configured.

Unsigned subtraction supports the bounded pulse interval across a 32-bit wrap. It does not identify how many wraps occurred over an arbitrarily long unattended interval. The driver's short timeout makes the intended interval bounded.

Cancel first and make the output suitable for an absent rail

src/hal/pulse_capture.c / lines 89-101

```
89
90 void pulseCaptureCancel(PulseCapture* pc) { pc->state_ = CAPTURE_IDLE; }
91
92 // The interrupt stays attached (pinMode() keeps the edge type): while IDLE the
93 // ISR ignores edges, and the 2 kOhm divider leg holds ECHO low with the module
94 // unpowered. The capture is usable again as soon as the rail returns.
95 void pulseCaptureSetSafe(PulseCapture* pc) {
96   pc->state_ = CAPTURE_IDLE;
97   if (!pc->begun_) return;
98   digitalWrite((uint8_t)pc->trig_, LOW);  // never drive TRIG high into an unpowered
 >   module
99   pinMode((uint8_t)pc->trig_, OUTPUT);
100  pinMode((uint8_t)pc->echo_, INPUT);  // no pulls
101 }
```

Cancel sets IDLE so a late falling edge cannot complete the abandoned ping. SetSafe does the same, then holds TRIG low and leaves ECHO as an input without an internal pull.

The handler stays attached and ignores idle edges. A later driver init can use the configured capture again after power returns. Logical cancellation, pin safety and physical voltage availability are three related but different facts.

Distinguish missing edges from an out-of-range width

The driver gives the pulse mechanism a measurement deadline and a Reading policy.

The raw field may carry a width or an elapsed-high diagnostic; health and hasRaw identify which outcome occurred.

src/drivers/hcsr04.h / lines 1-28

```
// hcsr04 driver. Work package WP4 (Stage 3). Architecture §7, §12, §14.
// Source: HC-SR04.pdf (module guide, pages 2-5).
//
// Switched 5 V rail. TRIG (GPIO17) driven at 3.3 V logic; ECHO (GPIO18)
// through a 1 kOhm / 2 kOhm divider (5 V -> ~3.33 V). A 10 us trigger starts an
// 8-cycle 40 kHz burst; ECHO stays high for the round trip. Distance =
// width * 343 m/s / 2 (domain/ultrasonic.h), no temperature compensation.
// Datasheet range 2-400 cm, resolution 0.3 cm. One ping per frame (1 s) leaves
// old echoes time to die out (datasheet §5: echoes give false short readings).
//
// Results (poll() returns READY for all of them; the verdict is in health,
// raw = echo width or elapsed time in us). A missing distance is never 0:
//   echo 20-4000 mm          -> value mm, UNIT_MILLIMETRE, HEALTH_OK
//   echo outside 20-4000 mm  -> HEALTH_FAULT, raw = width
//   no rising edge by timeout -> HEALTH_TIMEOUT, no raw (stuck low, ECHO
//                                unplugged or 5 V rail off: indistinguishable,
//                                there is no rail feedback pin)
//   rose, no fall by timeout -> HEALTH_TIMEOUT, raw = elapsed (no object in
//                                range; clones hold ECHO ~38 ms or longer)
// ECHO already high at init()/start() (stuck high, or a late echo still
// running) -> DRIVER_FAULT, stored reading HEALTH_FAULT, re-init required.
// The timeout is Hcsr04Config.timeoutMs (CFG_HCSR04_TIMEOUT_MS = 30 ms covers
// the 23.3 ms round trip at 4 m plus the ~0.5 ms burst lead-in).
//
// Worst-case blocking:
//   init()            none (one GPIO read)
//   start()           ~12 us busy-wait (2 us TRIG low + 10 us TRIG high)
//   poll()            none (reads the ISR's timestamps); Pending, 1 ms steps,
```

The contract lists successful range, completed-but-invalid distance, never-rising timeout and never-falling timeout. It also rejects an already-high ECHO before a new trigger. None of the missing-distance cases should be represented as a fresh zero distance.

src/drivers/hcsr04.h / lines 29-31

```
//                     until the echo ends or timeoutMs
//   cancel(), preparePowerOff()  a few GPIO writes
// The ECHO edge ISR (hal/pulse_capture.c) only stores a timestamp.
```

The nominal 30ms allowance is larger than the expected maximum round-trip time in the accepted range. The actual capture completion and service noticing still depend on handler and scheduler timing, so the configured timeout is not a precision guarantee.

Add a deadline around a reusable capture object

src/drivers/hcsr04.h / lines 32-57

```
32 //
33 // preparePowerOff(): TRIG low, ECHO input without pulls (no driving into the
34 // unpowered module).
35 //
36 // Implements the SensorDriver contract in drivers/sensor_driver.h.
37 #pragma once
38 #include "drivers/sensor_driver.h"
39 #include "hal/pulse_capture.h"
40
41 typedef struct Hcsr04Config {
42   PulseCapture* capture;  // already begun on BOARD_PIN_HCSR04_TRIG / _ECHO
43   uint32_t timeoutMs;     // CFG_HCSR04_TIMEOUT_MS
44 } Hcsr04Config;
45
46 // Members after `base` are private to hcsr04.c.
47 typedef struct Hcsr04Driver {
48   SensorDriver base;  // must stay first
49   Hcsr04Config config_;
50   bool initialised_;
51   bool busy_;
52   MonoMs startMs_;
53   Reading result_;
54 } Hcsr04Driver;
55
56 // Constructor. Use &driver.base wherever a SensorDriver* is expected.
57 void hcsr04DriverConstruct(Hcsr04Driver* self, const Hcsr04Config* config);
```

Hcsr04Config points to an already configured PulseCapture and stores timeoutMs. The driver owns lifecycle flags, start time and one result; it does not duplicate the ISR fields.

Construction installs the generic base. The HAL records edges, the driver judges one operation, and acquisition later tags the wider rail history. This is one physical event interpreted at three distinct boundaries.

Reject impossible starting conditions before arming

Follow one ping through its normal and incomplete outcomes. Watch the order of DONE and timeout checks.

The capture continues independently of when acquisition next calls poll.

src/drivers/hcsr04.c / lines 1-28

```
#include "drivers/hcsr04.h"

#include <stddef.h>

#include "domain/ultrasonic.h"

// Poll step while the echo is in flight. The width comes from the ISR's us
// timestamps, so this only sets how soon the result is noticed.
static const uint32_t kPollStepMs = 1;

static inline Hcsr04Driver* asHcsr04(SensorDriver* d) { return (Hcsr04Driver*)d; }

static PollResult finish(Hcsr04Driver* self, MonoMs now, Health health, bool hasRaw,
                         uint32_t rawUs, float mm);

static SensorId hcsr04Id(const SensorDriver* d) { (void)d; return SENSOR_HCSR04; }
static const char* hcsr04Name(const SensorDriver* d) { (void)d; return "hcsr04"; }
static uint8_t hcsr04MetricCount(const SensorDriver* d) { (void)d; return 1; }

static DriverStatus hcsr04Init(SensorDriver* d) {
  Hcsr04Driver* self = asHcsr04(d);
  self->initialised_ = false;
  self->busy_ = false;
  if (self->config_.capture == NULL || self->config_.timeoutMs == 0) return
  DRIVER_FAULT;
  pulseCaptureCancel(self->config_.capture);
  uint32_t width = 0;
  if (pulseCapturePoll(self->config_.capture, &width) == CAPTURE_ERROR) return
  DRIVER_FAULT;
  // Idle ECHO must be low; high means a stuck line or an echo still running.
```

Reject impossible starting conditions before arming / continued

Init clears active state and requires a capture pointer and nonzero timeout. It cancels any old capture, verifies the HAL is configured and rejects an already-high ECHO.

src/drivers/hcsr04.c / lines 29-32

```
29   if (pulseCaptureEchoHigh(self->config_.capture)) return DRIVER_FAULT;
30   self->initialised_ = true;
31   return DRIVER_OK;
32 }
```

A normal idle-low level lets initialised become true. That does not prove a target exists; it establishes a suitable starting condition for a new operation. The generic metadata still reports one distance metric.

Accept the operation only after a trigger succeeds

src/drivers/hcsr04.c / lines 33-46

```
33
34 static DriverStatus hcsr04Start(SensorDriver* d, MonoMs now) {
35   Hcsr04Driver* self = asHcsr04(d);
36   if (!self->initialised_) return DRIVER_FAULT;
37   self->startMs_ = now;
38   if (!pulseCaptureTrigger(self->config_.capture)) {
39     // ECHO high before the trigger: nothing was sent.
40     self->initialised_ = false;
41     finish(self, now, HEALTH_FAULT, false, 0, 0.0f);
42     return DRIVER_FAULT;
43   }
44   self->busy_ = true;
45   return DRIVER_OK;
46 }
```

Start records now, calls the HAL trigger and handles refusal by clearing initialisation and recording a fault outcome. A successful trigger sets busy_ and returns OK.

This path reports a start failure as an observation fault rather than silently keeping an earlier good distance. Acquisition can collect and publish the new failure with its own observation sequence.

Prefer a timestamped complete pulse over poll-time guesses

src/drivers/hcsr04.c / lines 47-64

```
47
48 static PollResult hcsr04Poll(SensorDriver* d, MonoMs now) {
49   Hcsr04Driver* self = asHcsr04(d);
50   PollResult r = {DRIVER_FAULT, now};
51   if (!self->busy_) return r;
52   PulseCapture* pc = self->config_.capture;
53
54   uint32_t width = 0;
55   const CaptureState s = pulseCapturePoll(pc, &width);
56   if (s == CAPTURE_DONE) {
57     // Accepted even on a late poll: the width is timestamped by the ISR.
58     const float mm = ultrasonicEchoUsToMm(width, ULTRASONIC_NOMINAL_SPEED_MPS);
59     return finish(self, now, ultrasonicInRangeMm(mm) ? HEALTH_OK : HEALTH_FAULT, true,
60                   width, mm);
61   }
62   if (s != CAPTURE_WAITING && s != CAPTURE_HIGH) {  // capture lost its arming
63     pulseCaptureCancel(pc);
64     return finish(self, now, HEALTH_FAULT, false, 0, 0.0f);
```

Poll asks the HAL for state and width. DONE is tested before the pending-operation deadline. A late task poll can therefore accept an already completed timestamp pair and convert its physical width.

An unexpected idle/error state is a different failure from a still-pending pulse. The code cancels it and returns a fault. Timing of notice is not substituted for timing of the echo edges.

End an unfinished operation at the deadline

src/drivers/hcsr04.c / lines 65-81

```
  }

  const MonoMs deadline = self->startMs_ + self->config_.timeoutMs;
  if (!monoReached(now, deadline)) {
    MonoMs next = now + kPollStepMs;
    if (next > deadline) next = deadline;
    r.status = DRIVER_PENDING;
    r.nextPollMs = next;
    return r;
  }

  // Disarm first so a late falling edge is never credited to the next ping.
  const uint32_t elapsedUs = pulseCaptureElapsedUs(pc);
  pulseCaptureCancel(pc);
  if (s == CAPTURE_HIGH) return finish(self, now, HEALTH_TIMEOUT, true, elapsedUs,
  0.0f);
  return finish(self, now, HEALTH_TIMEOUT, false, 0, 0.0f);  // never rose
}
```

WAITING or HIGH before the deadline returns PENDING with a bounded next poll time. At the deadline, the driver obtains elapsed time and disarms before reporting TIMEOUT.

HIGH carries elapsed raw diagnostic time; never-risen does not claim a raw width. That distinction helps separate observed incomplete pulses from complete measurements, even though several electrical causes can still share the same timeout result.

Retain failure health without inventing a zero measurement

src/drivers/hcsr04.c / lines 82-109

```
82
83 // Store one outcome. Only HEALTH_OK carries a value; every other verdict keeps
84 // the previous value untouched and is reported through health, never as 0.
85 static PollResult finish(Hcsr04Driver* self, MonoMs now, Health health, bool hasRaw,
86                          uint32_t rawUs, float mm) {
87   self->busy_ = false;
88   Reading* r = &self->result_;
89   r->intervalStartMs = self->startMs_;
90   r->intervalEndMs = now;
91   r->hasRaw = hasRaw;
92   r->raw = hasRaw ? (int32_t)rawUs : 0;
93   r->qualityFlags = kQualityUncompensated;  // nominal 343 m/s, no air temperature
94   r->unit = UNIT_MILLIMETRE;
95   r->health = health;
96   if (health == HEALTH_OK) {
97     r->value = mm;
98     r->lastGoodMs = now;
99     ++r->sequence;
100  }
101  PollResult pr = {DRIVER_READY, now};
102  return pr;
103 }
104
105 static uint8_t hcsr04ReadResult(SensorDriver* d, Reading* out, uint8_t maxOut) {
106   if (out == NULL || maxOut < 1) return 0;
107   out[0] = asHcsr04(d)->result_;
108   return 1;
109 }
```

finish stamps interval, raw availability, nominal-speed quality and health. Only OK changes the physical distance and usable history. An old number can remain in memory under a new failure verdict.

ReadResult copies one retained record if destination and capacity are valid. The returned count, not the existence of an old value, controls collection. A consumer must interpret the copy's health before using its value.

Cancel software work and then make the pins safe

src/drivers/hcsr04.c / lines 110-124

```
110
111 static void hcsr04Cancel(SensorDriver* d) {
112   Hcsr04Driver* self = asHcsr04(d);
113   self->busy_ = false;
114   if (self->config_.capture != NULL) pulseCaptureCancel(self->config_.capture);
115 }
116
117 // TRIG low and ECHO a plain input before the 5 V rail goes (§12). init()
118 // again once the rail is back.
119 static void hcsr04PreparePowerOff(SensorDriver* d) {
120   Hcsr04Driver* self = asHcsr04(d);
121   hcsr04Cancel(d);
122   if (self->config_.capture != NULL) pulseCaptureSetSafe(self->config_.capture);
123   self->initialised_ = false;
124 }
```

Cancel ends busy state and disarms a configured capture. PreparePowerOff calls that cleanup, requests the HAL's safe pin state and clears initialisation.

The service can now release ownership without leaving a trigger or late echo logically active. After a rail return, a fresh init must check idle ECHO before another measurement is accepted.

Construct one complete generic driver instance

src/drivers/hcsr04.c / lines 125-145

```
125
126 static const SensorDriverOps kHcsr04Ops = {
127     .id = hcsr04Id,
128     .name = hcsr04Name,
129     .metricCount = hcsr04MetricCount,
130     .init = hcsr04Init,
131     .start = hcsr04Start,
132     .poll = hcsr04Poll,
133     .readResult = hcsr04ReadResult,
134     .cancel = hcsr04Cancel,
135     .preparePowerOff = hcsr04PreparePowerOff,
136 };
137
138 void hcsr04DriverConstruct(Hcsr04Driver* self, const Hcsr04Config* config) {
139   self->base.ops = &kHcsr04Ops;
140   self->config_ = *config;
141   self->initialised_ = false;
142   self->busy_ = false;
143   self->startMs_ = 0;
144   readingReset(&self->result_, SENSOR_HCSR04, METRIC_WATER_DISTANCE, UNIT_MILLIMETRE);
145 }
```

The operations table supplies all lifecycle entries, including the power-off cleanup. Construction installs it, copies timeout/capture configuration and resets the distance identity and unit.

A constructor does not configure GPIOs in this pattern. PulseCaptureBegin must already establish the hardware before init can succeed. This ordering becomes visible again in production beginHals.

Choose test cases that produce distinct capture states

The isolated echo test exposes raw timing and the incomplete-edge cases. Read its electrical procedure as part of the test's assumptions.

This test lacks production rail gating, so it is not an appropriate program for experimenting with an unpowered module while TRIG continues.

test/bringup/hcsr04_main.c / lines 1-26

```
// Bring-up entry point for hcsr04 (WP4).  pio run -e test_hcsr04 -t upload -t monitor
// Switched 5 V rail must be ON. Verify the 1k/2k ECHO divider first.
// Replaces src/main.c in this env only; links the production driver.
//
// Procedure (README "HC-SR04 bring-up"), note the line printed for each step:
//  0. Divider gate, before connecting GPIO18: with the module's ECHO wire
//     disconnected, put 5 V on the module side of the 1 kOhm and measure ~3.3 V
//     at the divider junction (or scope the junction on a running module).
//     Never force the module's ECHO output to 5 V: it is a driven output.
//  1. Flat target at 10, 50 and 100 cm: "[hcsr04] 1000.xx mm"; compare with a tape.
//  2. Point at open space (> 4 m): "water_distance=TIMEOUT (raw <us>)".
//  3. Unplug ECHO at the MODULE side (the 2 kOhm keeps GPIO18 low):
//     "water_distance=TIMEOUT", no raw. Not at the GPIO18 side: the pin has no
//     pull on purpose and would float. Do not test with the 5 V rail off while
//     this harness runs - it keeps driving TRIG high into the unpowered module.
//  4. Hold ECHO high (to 3.3 V on the GPIO side): "init -> FAULT".
//  5. Target closer than 2 cm: FAULT (raw = width) or an unstable value.
#include <Arduino.h>
#include <stdio.h>

#include "app_config.h"
#include "board_profile.h"
#include "bringup_harness.h"
#include "hal/pulse_capture.h"
#include "drivers/hcsr04.h"

```

Choose test cases that produce distinct capture states / continued

The source procedure separates known distances, open space, an unplugged module-side ECHO and a stuck-high line. These conditions exercise completed width, no rise, no fall and rejected start.

test/bringup/hcsr04_main.c / lines 27-31

```
27 static PulseCapture s_capture = PULSE_CAPTURE_INIT;
28 static Hcsr04Config s_config = {&s_capture, CFG_HCSR04_TIMEOUT_MS};
29 static Hcsr04Driver s_driver;
30 static BringupContext s_ctx = {&s_driver.base, CFG_FRAME_PERIOD_MS, NULL};
31
```

The static capture, configuration, driver and harness context keep callback addresses valid. The divider checks establish a prerequisite for connecting the signal; firmware cannot repair an inappropriate pin voltage after the fact.

Start the same driver used by production

test/bringup/hcsr04_main.c / lines 32-46

```
32 void setup(void) {
33   hcsr04DriverConstruct(&s_driver, &s_config);
34   delay(200);
35   printf("\n=== bring-up: %s ===\n", sensorDriverName(&s_driver.base));
36   if (!pulseCaptureBegin(&s_capture, BOARD_PIN_HCSR04_TRIG, BOARD_PIN_HCSR04_ECHO)) {
37     printf("pulse capture: invalid pins TRIG=%d ECHO=%d\n", (int)BOARD_PIN_HCSR04_TRIG,
38            (int)BOARD_PIN_HCSR04_ECHO);
39   }
40   printf("TRIG GPIO%d, ECHO GPIO%d, timeout %lu ms, nominal 343 m/s\n",
41          (int)BOARD_PIN_HCSR04_TRIG, (int)BOARD_PIN_HCSR04_ECHO,
42          (unsigned long)CFG_HCSR04_TIMEOUT_MS);
43   bringupStart(&s_ctx);
44 }
45
46 void loop(void) { vTaskDelay(portMAX_DELAY); }
```

Setup constructs the object, configures the capture pins, prints the timeout and nominal speed, then starts the generic harness. loop blocks.

Compare accepted distances with measured target positions, but also inspect health and raw microseconds when a case fails. A complete driver test includes cancellation and refusal, not only a good 1000mm line.

One active worker owns the drivers

You can now read acquisition.c as a scheduler for ordinary function calls. Its static objects include one I2C bus, one 1-Wire bus, ADC channels, a pulse-capture object and seven concrete drivers. A DriverRun record stores scheduling state beside each driver pointer.

Original firmware - src/services/acquisition.c, lines 64-79

```
enum { RUN_BME280 = 0, RUN_DS18B20, RUN_SGP30, RUN_HCSR04,
>   RUN_TDS, RUN_PH, RUN_LIGHT, RUN_COUNT };

typedef struct DriverRun {
  SensorDriver* driver;
  uint32_t initBudgetMs;   // worst-case blocking of init()
  uint32_t stepBudgetMs;   // worst-case blocking of one
>   start()/poll(); > 2 = "blocking"
  bool initialised;
  bool pending;            // started, not yet READY or failed
  bool wantStart;          // due this frame, not started yet
  MonoMs nextPollMs;
  MonoMs retryAtMs;        // earliest init() retry after a
>   failure
  DriverStatus lastFailure;  // DRIVER_OK = none; logged on
>   change only
  bool disabled;           // off by the configuration (WP8) or
>    the lease (WP9): never scheduled
  uint32_t startEpoch;     // 5 V sensors: rail epoch when the
>   measurement started
} DriverRun;
```

initialised says whether device setup succeeded. pending says an operation has started but not finished. wantStart says this frame wants a new operation. nextPollMs names its next useful step. retryAtMs stops a missing device from triggering an immediate endless reinitialisation loop.

COMMON PITFALL

Two tasks with separate stacks can still share the same file-scope driver objects. The lease, not stack separation, controls their access.

One active worker owns the drivers / continued

The budget fields describe the maximum intended blocking duration of a call, not the sensor's whole conversion time. A DS18B20 conversion lasts hundreds of milliseconds, but its start and read transactions occupy much shorter intervals. Acquisition needs both facts to schedule around other work.

The firmware creates two workers for its normal and battery modes. Only the worker holding the application's acquisition lease may touch the shared drivers. A lease here is an ownership protocol stored in application state, not a FreeRTOS primitive that every driver secretly understands.

For now, follow the currently active worker. The second worker remains waiting without permission. In the power chapter we will show how one owner stops before the other gets permission. Nothing about static storage or different core numbers alone would enforce that rule.

Advance releases without inventing old samples

Original firmware - src/domain/schedule.c, lines 13-26

```
void scheduleJobMarkRun(ScheduleJob* job, MonoMs now) {
  const MonoMs lateness = now > job->nextDueMs ? now -
>   job->nextDueMs : 0;
  const MonoMs missed = lateness / job->periodMs;  // grid
>   points already passed
  const uint32_t lateMs = lateness > UINT32_MAX ? UINT32_MAX :
>   (uint32_t)lateness;

  ++job->runs;
  job->lastLateMs = lateMs;
  if (lateMs > job->maxLateMs) job->maxLateMs = lateMs;
  if (lateMs > SCHEDULE_LATE_TOLERANCE_MS) ++job->late;
  job->skipped += missed > UINT32_MAX ? UINT32_MAX :
>   (uint32_t)missed;
  job->nextDueMs += (missed + 1) * job->periodMs;
}
```

This function marks one scheduled job as run. lateness is zero when the caller is on time or early; otherwise it is now minus the intended due time. missed divides that lateness by the period to count additional release points already passed.

Use the earlier example: nextDue=250, period=250, now=820. lateness=570; missed=2. The function counts one actual run, two skipped releases and advances nextDue by (2+1)*250, from 250 to 1000. It keeps the original grid rather than setting the next due time to 1070.

lateMs is clamped to UINT32_MAX before being stored in a 32-bit diagnostic. The true scheduling timestamps remain MonoMs. A cast alone could truncate a large lateness and make a very delayed job appear almost on time.

The engine has a 1000ms ordinary frame job and a 250ms BME280 job. The ordinary frame requests DS18B20, SGP30, HC-SR04, TDS, pH and light in a chosen order. The BME280 gets its own faster release grid.

A frame is a scheduling boundary, not a batch that must remain unpublished until all sensors finish. Acquisition publishes after each completed driver result as well as at frame boundaries. That lets consumers see a new temperature promptly instead of waiting for the slowest sensor.

REMEMBER

A budget is an intended bound. Use the late, skipped and jitter counters to check whether the running system actually respects it.

Read one pass in the order it executes

Original firmware - src/services/acquisition.c, lines 464-482

```
static void runPass(MonoMs now) {
  if (scheduleJobDue(&s_jobs[JOB_FRAME], now)) startFrame(now);
  if (scheduleJobDue(&s_jobs[JOB_BME280], now))
>   startBmeSlot(now);

  for (uint8_t iter = 0; iter < 8; ++iter) {
    bool progress = false;
    now = monoNowMs();
    if (s_runs[RUN_BME280].wantStart) progress |=
>   tryStart(RUN_BME280, now);
    for (uint8_t r = 0; r < RUN_COUNT; ++r) progress |=
>   tryPoll(r, monoNowMs());
    for (uint8_t i = 0; i < FRAME_ORDER_COUNT; ++i) {
      const uint8_t r = kFrameOrder[i];
      if (!s_runs[r].wantStart) continue;
      if (!tryStart(r, monoNowMs())) break;  // keep the order:
>    later ones wait too
      progress = true;
    }
    if (!progress) break;
  }
}
```

The pass first notices due frame and BME280 releases. It then performs up to eight bounded iterations. BME280 gets a chance to start; due polls advance; ordinary starts are attempted in their fixed order. If no operation makes progress, the loop stops early.

The |= expressions accumulate whether any operation progressed while still evaluating the called functions. Do not confuse bitwise | with short-circuit || here: a short-circuit expression could skip a later function call once progress was already true.

COMMON PITFALL

A loop bound limits iterations, not necessarily execution time. Every called step still needs a real blocking bound.

Read one pass in the order it executes / continued

tryStart supplies dependencies before a measurement begins. For TDS it copies the latest water temperature; for SGP30 it supplies air temperature and humidity. tryPoll leaves a pending run pending with a future nextPollMs, or finishes it and collects the outcome.

Why restrict overlap? A 1-Wire transaction can mask interrupts and delay echo timestamps. blockingAllowed therefore rejects long calls while HC-SR04 is pending. pingAllowed postpones a ping when an upcoming DS18B20 call could collide with its echo window. A flash gate also excludes NVS writes during that window.

Finally, nextWake chooses the earliest relevant schedule release or permitted poll. The worker can block on a notification with that timeout. A configuration or power event can wake it early, so it checks state again before starting new work. You have already learned each piece; this function coordinates their order.

Follow one result into the retained Snapshot

Original firmware - src/services/acquisition.c, lines 183-193

```
static void storeResults(DriverRun* run, MonoMs now, bool
>   guardTimeout) {
  Reading out[4];
  const uint8_t n = sensorDriverReadResult(run->driver, out, 4);
  const RailState rail = railStateRead();
  const uint32_t epoch = rail.epoch;
  // The rail went down (or down and up) during the
>   measurement: the number
  // may come from a collapsing supply, so it is never
>   published (§14).
  const bool railLost = onFiveVoltRail(run) && (!rail.valid5v
>   || rail.epoch != run->startEpoch);
  for (uint8_t k = 0; k < n; ++k) {
    Reading* r = &out[k];
    if (r->metric >= METRIC_COUNT) continue;
```

Original firmware - src/services/acquisition.c, lines 203-212

```
    r->sequence = s_work.readings[r->metric].sequence + 1u;
    r->powerEpoch = epoch;
    r->configRevision = s_appliedRevision;
    s_work.readings[r->metric] = *r;
    evaluateRules(r, now);
  }
  s_work.publishedMs = now;
  s_work.powerEpoch = epoch;
  snapshotPublish(&s_work);
}
```

storeResults asks the driver to copy up to four Reading records into a local array. This covers BME280's three metrics and SGP30's two without dynamic allocation. The returned count says how many records may be visited.

Before accepting a 5V sensor result, acquisition compares the current rail state with startEpoch. A rail loss or changed epoch turns that observation UNPOWERED. A frame guard can instead mark an abandoned run TIMEOUT and set the late quality bit.

REMEMBER

Local out[] is safe here because the records are copied into s_work and the queue before the function returns.

Follow one result into the retained Snapshot / continued

For each returned metric, acquisition assigns the next published sequence, stamps the epoch and configuration revision, copies the record into s_work, then evaluates matching alarm rules. It publishes the updated working Snapshot after processing the results.

Suppose a driver previously returned a good sample with sequence 8, then fails without incrementing its internal good-sample count. Acquisition still publishes the failure as sequence 9. A remote consumer can now recognise the new fault rather than discarding it as an old resend.

The driver does not call Safety or BLE. It produces a result. Acquisition connects that result to policy evaluation and retained state. Safety owns actuator decisions; the transport owns message encoding. You can now follow this path by responsibility rather than by memorising every filename.

Return a timestamp rather than a delay

The acquisition scheduler needs one agreed notion of now. This small interface hides the platform clock call.

Drivers receive timestamps from callers; they do not each invent a calendar-time convention.

src/hal/clock.h / lines 1-5

```
1 // Monotonic clock (HAL). The only source of "now" for ages and deadlines.
2 #pragma once
3 #include "domain/types.h"
4
5 MonoMs monoNowMs(void);
```

monoNowMs takes no arguments and returns monotonic milliseconds. The declaration does not change the clock or wait for a deadline. Keeping it in the HAL lets domain functions accept supplied now values during host tests.

When reading acquisition, distinguish this call that observes time from functions such as ulTaskNotifyTake that can block execution.

Keep the fine clock at the boundary and expose milliseconds

The complete wrapper makes the clock's scale visible in one line.

The time restarts with the relevant boot/timer lifetime, so observations across resets need session context.

src/hal/clock.c / lines 1-5

```
#include "hal/clock.h"

#include <esp_timer.h>

MonoMs monoNowMs(void) { return (MonoMs)(esp_timer_get_time() / 1000); }
```

esp_timer_get_time returns a microsecond count. Integer division by 1000 discards the sub-millisecond remainder, and the cast provides MonoMs. A returned 1234 represents whole milliseconds, not a promise of a 1ms scheduling response.

The pulse HAL retains microseconds for widths; this scheduler wrapper intentionally uses the coarser scale appropriate to its jobs and freshness budgets.

Define due, budget and timing evidence separately

This header describes a periodic release grid without knowing tasks or sensors.

The service decides which job is a frame and which job is a climate slot.

src/domain/schedule.h / lines 1-28

```
// Absolute-deadline job table for the acquisition engine (§9). Pure C.
// Work package WP7 (Stage 4). Host-tested; the acquisition task only calls it.
//
// Each job has a period and an absolute next deadline on the monotonic clock.
// A job that runs late keeps its grid: the next deadline is the first grid
// point after `now`. Missed grid points are counted as skipped and are never
// replayed in a burst ("skip, never catch up").
#pragma once
#include <stdbool.h>
#include <stdint.h>
#include "domain/types.h"

// Lateness up to this is release jitter, not a late run (§19: <= 5 ms).
#define SCHEDULE_LATE_TOLERANCE_MS ((uint32_t)5)

typedef struct ScheduleJob {
  uint32_t periodMs;
  uint32_t budgetMs;   // worst-case worker time for one step of this job
  MonoMs nextDueMs;
  uint32_t runs;
  uint32_t late;       // runs started more than SCHEDULE_LATE_TOLERANCE_MS late
  uint32_t skipped;    // whole periods that passed without a run
  uint32_t maxLateMs;  // largest lateness seen (jitter for on-time runs)
  uint32_t lastLateMs;
} ScheduleJob;

void scheduleJobInit(ScheduleJob* job, uint32_t periodMs, uint32_t budgetMs, MonoMs firstDueMs);

```

COMMON PITFALL

A budget is not reserved CPU time. Higher-priority work and interrupt masking can still affect the execution interval.

Define due, budget and timing evidence separately / continued

ScheduleJob stores period, execution budget, next intended release and counters for actual runs, late releases and skipped slots. scheduleJobDue only compares now with nextDue.

src/domain/schedule.h / lines 29-44

```
29 static inline bool scheduleJobDue(const ScheduleJob* job, MonoMs now) {
30   return monoReached(now, job->nextDueMs);
31 }
32
33 // Record a run that starts at `now` (call only when due) and move the
34 // deadline to the next grid point strictly after `now`.
35 void scheduleJobMarkRun(ScheduleJob* job, MonoMs now);
36
37 // Earliest deadline over `count` jobs (count >= 1).
38 MonoMs scheduleEarliestDue(const ScheduleJob* jobs, uint8_t count);
39
40 // true if a step of `budgetMs` started at `now` ends no later than `deadline`
41 // (§9: defer an ordinary step that would cross the next BME280 slot).
42 static inline bool scheduleFitsBefore(MonoMs now, uint32_t budgetMs, MonoMs deadline) {
43   return now + budgetMs <= deadline;
44 }
```

scheduleFitsBefore checks whether a proposed blocking interval fits before another release, including the specified guard semantics. It is a prediction based on a budget; a measured call that exceeds that budget can still violate the schedule.

Establish a future grid before the first pass

Read this with an overdue 250ms job at 820ms. The goal is one useful present observation, not a burst pretending to measure the past.

The counters allow the Supervisor to distinguish delay from a fully stopped worker.

src/domain/schedule.c / lines 1-12

```
#include "domain/schedule.h"

void scheduleJobInit(ScheduleJob* job, uint32_t periodMs, uint32_t budgetMs, MonoMs
  firstDueMs) {
  job->periodMs = periodMs > 0 ? periodMs : 1;
  job->budgetMs = budgetMs;
  job->nextDueMs = firstDueMs;
  job->runs = 0;
  job->late = 0;
  job->skipped = 0;
  job->maxLateMs = 0;
  job->lastLateMs = 0;
}
```

Init clamps a zero period to one, records budget and first release, and clears counters. Supplying now as firstDue makes the first pass immediately eligible. Supplying a future timestamp deliberately delays the first job.

No kernel object is involved. This is ordinary state that the acquisition owner consults inside its task.

Count releases already missed and advance from the old grid

src/domain/schedule.c / lines 13-25

```
13
14 void scheduleJobMarkRun(ScheduleJob* job, MonoMs now) {
15   const MonoMs lateness = now > job->nextDueMs ? now -
 >   job->nextDueMs : 0;
16   const MonoMs missed = lateness / job->periodMs;  // grid
 >   points already passed
17   const uint32_t lateMs = lateness > UINT32_MAX ? UINT32_MAX
 >    : (uint32_t)lateness;
18
19   ++job->runs;
20   job->lastLateMs = lateMs;
21   if (lateMs > job->maxLateMs) job->maxLateMs = lateMs;
22   if (lateMs > SCHEDULE_LATE_TOLERANCE_MS) ++job->late;
23   job->skipped += missed > UINT32_MAX ? UINT32_MAX :
 >   (uint32_t)missed;
24   job->nextDueMs += (missed + 1) * job->periodMs;
25 }
```

Lateness is measured against nextDue, and integer division by period counts further release points already passed. The nextDue update uses missed+1 periods from the previous grid point.

At 250/250/820, one current run accounts for the overdue 250 slot, records two skipped releases and advances to 1000. Diagnostic lateness is clamped for its 32-bit storage; the scheduling timestamp remains wide.

COMMON PITFALL

The skipped counter is a long-running 32-bit diagnostic. Saturating one increment does not by itself make every future accumulated total saturate.

Choose the earliest release without changing it

src/domain/schedule.c / lines 26-33

```
26
27 MonoMs scheduleEarliestDue(const ScheduleJob* jobs, uint8_t count) {
28   MonoMs earliest = jobs[0].nextDueMs;
29   for (uint8_t i = 1; i < count; ++i) {
30     if (jobs[i].nextDueMs < earliest) earliest = jobs[i].nextDueMs;
31   }
32   return earliest;
33 }
```

EarliestDue starts from the first job and scans the rest for a smaller nextDue. It returns the timestamp and does not mark any job run.

The function assumes count is nonzero because it immediately reads jobs[0]. Acquisition supplies a fixed nonempty job array. An external interface accepting arbitrary arrays would need to validate that precondition.

Understand the move from one worker to two contexts

This is the public boundary of the complete sensor owner. Read the calls as permissions and observations; other tasks do not receive a pointer to an internal driver.

The PowerCoordinator alone controls the acquisition lease through grant and quiesce requests.

src/services/acquisition.h / lines 1-28

```
// Acquisition engine: the one serial owner of all sensor buses.
// Work package WP7 (Stage 4). Architecture §6, §7, §9, §12.
//
// Stage 4 (decision D15) had ONE worker, AcqNormal (core 1, priority 4), owning
// every driver and bus. It runs an absolute-deadline schedule (1 s frame +
// BME280 every 250 ms, domain/schedule), stamps each completed reading,
// evaluates the alarm rules for it, submits TRIP/CLEAR to Safety at once and
// publishes the snapshot after every completed reading and at each frame.
//
// WP9 (§14): two workers share the drivers and the schedule, and only the
// lease holder runs. AcqNormal (core 1) holds it with external power and runs
// every sensor; AcqLowPower (core 0) holds it in low power and runs only the
// 3.3 V sensors (ACQ_LOW_POWER_SENSORS). PowerCoordinator moves the lease:
//   acquisitionRequestQuiesce(gen)  the holder stops at its next pass
//                                   boundary, cancels pending steps, makes the
//                                   5 V interfaces safe if the rail is down
//                                   (preparePowerOff: HC-SR04 TRIG low), leaves
//                                   the task watchdog and releases;
//   acquisitionReleasedGeneration() the acknowledgement;
//   acquisitionGrant(worker, gen, allowed)  the next holder joins the
//                                   watchdog, restarts the schedule and applies
//                                   the configuration limited to `allowed`.
// A sensor outside `allowed` publishes UNPOWERED; when it comes back it is
// initialised again and shows WARMING until its first result.
// Rail gating (§14): a 5 V sensor (HC-SR04, pH, TDS) starts only while the
// rail is valid and its result is kept only if the rail is still valid with
// the same epoch at the end; otherwise it is published UNPOWERED.
// HALs begin in AcqNormal on core 1 (the GPIO ISR service is installed there).
```

The history comment first describes the original normal worker and then the low-power worker added later. The current code creates both, but only the lease holder advances the shared objects.

Understand the move from one worker to two contexts / continued

src/services/acquisition.h / lines 29-35

```
#pragma once
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>
#include <stdbool.h>
#include <stdint.h>

#include "app_config.h"
```

A worker not holding the lease waits. A handover request asks the current owner to cancel pending steps and release permission. We will read that cleanup in this file before examining the coordinator that requests it.

Declare the allowed low-power set and the acknowledgement path

src/services/acquisition.h / lines 36-58

```
#include "domain/power_mode.h"
#include "domain/sensor_set.h"
#include "domain/types.h"

// Retained 3.3 V supply: the sensors that keep running in low power (§15).
#define ACQ_LOW_POWER_SENSORS                                  \
  ((SensorMask)((1u << SENSOR_BME280) | (1u << SENSOR_DS18B20) | (1u << SENSOR_LIGHT) | \
                (1u << SENSOR_SGP30)))

bool acquisitionInit(void);  // construct drivers, create both workers; no lease yet

// Lease protocol, called only by PowerCoordinator (§14).
// Grant refuses (false) while another worker still holds the lease: never two owners.
bool acquisitionGrant(PowerWorker worker, uint32_t generation, SensorMask allowed);
bool acquisitionRequestQuiesce(uint32_t generation);
uint32_t acquisitionReleasedGeneration(void);  // 0 = nothing released yet

// True once AcqNormal has begun the buses (and the GPIO ISR service on core 1).
bool acquisitionHalsReady(void);

// Task id of the lease holder for the progress check; TASK_ID_COUNT if none.
TaskId acquisitionActiveTaskId(void);

```

The low-power mask keeps the retained 3.3V sensors. Grant receives worker identity, generation and allowed set. ReleasedGeneration lets the coordinator observe that a particular ownership interval ended.

COMMON PITFALL

A returned task handle is a notification destination, not ownership of the driver state.

Declare the allowed low-power set and the acknowledgement path / continued

src/services/acquisition.h / lines 59-65

```
59 // Counters for Supervisor (copied under a lock; zero-wait).
60 typedef struct AcquisitionStats {
61   uint32_t frames;
62   uint32_t frameLate;      // frame releases more than SCHEDULE_LATE_TOLERANCE_MS late
63   uint32_t frameSkipped;
64   uint32_t frameJitterMaxMs;
65   uint32_t bmeLate;
```

src/services/acquisition.h / lines 66-68

```
66   uint32_t bmeSkipped;
67   uint32_t guardTimeouts;  // steps cancelled by the frame / slot guard
68   uint32_t driverFailures; // start/poll results other than OK/PENDING/READY
```

HalsReady reports when core 1 setup exists, including the shared GPIO handler service. A generation number prevents an old release record from being mistaken for completion of a later handover.

Expose copies of diagnostics rather than live mutable internals

src/services/acquisition.h / lines 69-83

```
69   uint32_t ruleEvents;     // TRIP + CLEAR submitted to Safety
70 } AcquisitionStats;
71
72 void acquisitionGetStats(AcquisitionStats* out);
73
74 // SGP30 baseline status name ("none", "learning", "restored", "valid"),
75 // refreshed every frame. Zero wait, any task.
76 const char* acquisitionSgp30BaselineName(void);
77
78 // The lease holder's task (AcqNormal if none), e.g. to wake it for a new
79 // configuration. NULL before acquisitionInit().
80 TaskHandle_t acquisitionTaskHandle(void);
81 // For Supervisor's stack check.
82 TaskHandle_t acquisitionNormalTaskHandle(void);
83 TaskHandle_t acquisitionLowPowerTaskHandle(void);
```

AcquisitionStats counts completed frames, late and skipped releases, guard timeouts, failures and rule events. GetStats copies the protected record. Other accessors expose active task identity and baseline status.

These APIs support supervision and telemetry without allowing a radio task to step a driver. The header therefore documents both execution ownership and the information other services may inspect.

Construct one shared hardware object graph

You have read all seven drivers. This service now schedules their existing functions, supplies dependencies, publishes outcomes and preserves one active owner across power modes.

Treat the file's static storage as shared between the two worker contexts, except while the lease gives one of them exclusive permission.

src/services/acquisition.c / lines 1-28

```c
#include "services/acquisition.h"

#include <esp_task_wdt.h>
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>
#include <stdint.h>
#include <string.h>

#include "app_config.h"
#include "board_profile.h"
#include "diagnostics/logger.h"
#include "diagnostics/progress.h"
#include "domain/alarm.h"
#include "domain/schedule.h"
#include "drivers/bme280.h"
#include "drivers/ds18b20.h"
#include "drivers/hcsr04.h"
#include "drivers/light.h"
#include "drivers/ph.h"
#include "drivers/sgp30.h"
#include "drivers/tds.h"
#include "hal/adc_channel.h"
#include "hal/clock.h"
#include "hal/i2c_bus.h"
#include "hal/onewire_bus.h"
#include "hal/pulse_capture.h"
#include "policy_config.h"
#include "services/configuration.h"
```

src/services/acquisition.c / lines 29-39

```c
#include "services/flash_gate.h"
#include "services/rail_state.h"
#include "services/safety.h"
#include "services/snapshot_store.h"

// ---- Buses and drivers: owned by the lease holder only (§11, §14) ------------
static I2cBus s_i2c = I2C_BUS_INIT;
static OneWireBus s_oneWire = ONE_WIRE_BUS_INIT;
static AdcChannel s_adcLight = ADC_CHANNEL_INIT;
static AdcChannel s_adcTds = ADC_CHANNEL_INIT;
static AdcChannel s_adcPh = ADC_CHANNEL_INIT;
```

Construct one shared hardware object graph / continued

src/services/acquisition.c / lines 40-54

```
40 static PulseCapture s_capture = PULSE_CAPTURE_INIT;
41
42 static Bme280Config s_bmeConfig = {&s_i2c, BOARD_BME280_I2C_ADDRESS};
43 static Sgp30Config s_sgpConfig = {&s_i2c, BOARD_SGP30_I2C_ADDRESS};
44 static Ds18b20Config s_dsConfig = {&s_oneWire};
45 // Calibrations come from the applied configuration: copied in
46 // acquisitionInit(), then pushed to the drivers whenever a new revision is
47 // applied (on-device calibration, applyConfiguration()).
48 static LightConfig s_lightConfig = {&s_adcLight, {1.0f, 0.0f, 0, false}};
49 static TdsConfig s_tdsConfig = {&s_adcTds, {1.0f, 0.0f, 0, false}};
50 static PhConfig s_phConfig = {&s_adcPh, BOARD_PH_DIVIDER_TOP_OHM,
 >   BOARD_PH_DIVIDER_BOTTOM_OHM,
51                               {0.0f, 0.0f, 0, false},
52                               true};  // compiled nominal fit: published UNCALIBRATED
53 static Hcsr04Config s_hcConfig = {&s_capture, CFG_HCSR04_TIMEOUT_MS};
54
```

The includes name the collaborators; they do not execute startup. Static buses and capture storage are followed by configurations holding their addresses, then seven concrete driver objects.

src/services/acquisition.c / lines 55-62

```
55 static Bme280Driver s_bme;
56 static Sgp30Driver s_sgp;
57 static Ds18b20Driver s_ds;
58 static LightDriver s_light;
59 static TdsDriver s_tds;
60 static PhDriver s_ph;
61 static Hcsr04Driver s_hc;
62
```

The analog fits in these initialisers are starting placeholders. acquisitionInit replaces them from applied configuration before construction. The same s_i2c pointer is intentionally supplied to both I2C sensors.

Keep scheduling state beside each generic pointer

src/services/acquisition.c / lines 63-90

```
63 // ---- Per-driver stepping (init -> start -> poll), as test/bringup/water_main.c
64 enum { RUN_BME280 = 0, RUN_DS18B20, RUN_SGP30, RUN_HCSR04, RUN_TDS, RUN_PH, RUN_LIGHT,
 >   RUN_COUNT };
65
66 typedef struct DriverRun {
67   SensorDriver* driver;
68   uint32_t initBudgetMs;   // worst-case blocking of init()
69   uint32_t stepBudgetMs;   // worst-case blocking of one start()/poll(); > 2 =
 >   "blocking"
70   bool initialised;
71   bool pending;            // started, not yet READY or failed
72   bool wantStart;          // due this frame, not started yet
73   MonoMs nextPollMs;
74   MonoMs retryAtMs;        // earliest init() retry after a failure
75   DriverStatus lastFailure;  // DRIVER_OK = none; logged on change only
76   bool disabled;           // off by the configuration (WP8) or the lease (WP9): never
 >   scheduled
77   uint32_t startEpoch;     // 5 V sensors: rail epoch when the measurement started
78 } DriverRun;
79
80 // Worst-case blocking from each driver header (§16 budgets).
81 static DriverRun s_runs[RUN_COUNT] = {
82     [RUN_BME280] = {&s_bme.base, 30, 1, false, false, false, 0, 0, DRIVER_OK},
83     [RUN_DS18B20] = {&s_ds.base, 50, 13, false, false, false, 0, 0, DRIVER_OK},
84     [RUN_SGP30] = {&s_sgp.base, 45, 1, false, false, false, 0, 0, DRIVER_OK},
85     [RUN_HCSR04] = {&s_hc.base, 1, 1, false, false, false, 0, 0, DRIVER_OK},
86     [RUN_TDS] = {&s_tds.base, 1, 1, false, false, false, 0, 0, DRIVER_OK},
87     [RUN_PH] = {&s_ph.base, 1, 1, false, false, false, 0, 0, DRIVER_OK},
88     [RUN_LIGHT] = {&s_light.base, 1, 1, false, false, false, 0, 0, DRIVER_OK},
89 };
90
```

DriverRun records init and step budgets, lifecycle flags, next poll, retry, failure, disabled state and start epoch. The indexed table pairs each concrete base pointer with its intended blocking costs.

Keep scheduling state beside each generic pointer / continued

src/services/acquisition.c / lines 91-99

```
// One-second jobs, started in this order at each frame (§9): the 1-Wire work
// comes before the HC-SR04 ping because 1-Wire slots mask interrupts and
// would delay the echo timestamps (onewire_bus.h).
static const uint8_t kFrameOrder[] = {RUN_DS18B20, RUN_SGP30, RUN_HCSR04, RUN_TDS,
  RUN_PH, RUN_LIGHT};
#define FRAME_ORDER_COUNT ((uint8_t)(sizeof(kFrameOrder) / sizeof(kFrameOrder[0])))

enum { JOB_FRAME = 0, JOB_BME280, JOB_COUNT };
static ScheduleJob s_jobs[JOB_COUNT];

```

src/services/acquisition.c / lines 100-103

```
// ---- Applied configuration (WP8): read at frame boundaries only ---------------
static uint16_t s_appliedRevision;  // stamped into every Reading as configRevision
static ConfigBlob s_cfg;            // task-private copy (static: not on the stack)
#define BASELINE_OFFER_PERIOD_MS ((MonoMs)60 * 60 * 1000)  // Configuration rate-limits
  the write too
```

The ordinary frame order starts 1-Wire work before the ultrasonic ping. The array does not imply every conversion completes before the next starts. It orders attempts while the later guards decide which overlaps are permitted.

Separate owner-only state from shared protected metadata

src/services/acquisition.c / lines 104-131

```
104 static MonoMs s_nextBaselineOfferMs;
105 static volatile Sgp30BaselineStatus s_sgpBaseline =
  >   SGP30_BASELINE_NONE;  // one writer, word-sized
106
107 // ---- Working snapshot and rule state (task-private)
  >   --------------------------
108 static Snapshot s_work;
109 static AlarmRuleState s_ruleState[POLICY_RULE_COUNT];
110
111 static portMUX_TYPE s_statsMux =
  >   portMUX_INITIALIZER_UNLOCKED;
112 static AcquisitionStats s_stats;
113
114 static StaticTask_t s_tcb;
115 static StackType_t s_stack[4 * 1024];  //
  >   cfgTaskSpec(TASK_ID_ACQUISITION_NORMAL)->stackBytes
116 static TaskHandle_t s_task = NULL;
117 static StaticTask_t s_tcbLow;
118 static StackType_t s_stackLow[4 * 1024];  //
  >   cfgTaskSpec(TASK_ID_ACQUISITION_LOW_POWER)->stackBytes
119 static TaskHandle_t s_taskLow = NULL;
120
121 // ---- Lease (WP9, §14)
  >   ------------------------------------------------------------
  >   --
122 // Everything above is shared by the two workers but touched
  >    only by the lease
123 // holder; the lock below orders the hand-over between the
  >   cores.
124 static portMUX_TYPE s_leaseMux =
  >   portMUX_INITIALIZER_UNLOCKED;
125 static PowerWorker s_owner = POWER_WORKER_NONE;
126 static uint32_t s_leaseGen;
127 static SensorMask s_allowed = SENSOR_MASK_ALL;
128 static uint32_t s_quiesceGen;   // requested, for the
  >   current lease
129 static uint32_t s_releasedGen;
130 static volatile bool s_halsReady;
131 static bool s_forceApply;       // a new holder applies the
  >   configuration at once
```

s_cfg, s_work and rule states belong to the active acquisition owner. Statistics have their own portMUX. Lease identity, generation, allowed mask and quiesce/release records use another lock.

COMMON PITFALL

static and a private C name do not prevent the other worker from reaching these same objects. The lease protocol is the access rule.

Separate owner-only state from shared protected metadata / continued

src/services/acquisition.c / lines 132-139

```
132
133 #define STATS_ADD(field, n)        \
134   do {                             \
135     portENTER_CRITICAL(&s_statsMux); \
136     s_stats.field += (n);          \
137     portEXIT_CRITICAL(&s_statsMux);  \
138   } while (0)
139
```

The STATS_ADD macro wraps its statements in do/while(0) so a call behaves like one statement in surrounding control flow. It protects only the tiny update, not the sensor operation that produced the statistic.

Evaluate only matching enabled rules

src/services/acquisition.c / lines 140-164

```
// ---- Bookkeeping after a result
  > ----------------------------------------------

static void evaluateRules(const Reading* r, MonoMs now) {
  for (uint8_t i = 0; i < POLICY_RULE_COUNT; ++i) {
    const AlarmRule* rule = &kPolicyRules[i];
    if (!rule->enabled || rule->metric != r->metric)
  > continue;
    const RuleDecision d = evaluateRule(rule, r, now,
  > alarmStaleLimitMs(r->metric), &s_ruleState[i]);
    if (d != RULE_DECISION_TRIP && d != RULE_DECISION_CLEAR)
  >  continue;
    AlarmEvent e;
    e.ruleId = i;
    e.decision = d;
    e.sample = *r;
#ifdef FW_ALARM_LATENCY_TEST
    safetyLatencyMarkSubmit();
    s_ruleState[i].tripped = false;  // re-arm: the next
  > sample trips again
#endif
    safetySubmitEvent(&e);  // zero wait; wakes Safety now
  > (§4)
    STATS_ADD(ruleEvents, 1);
  }
}

// Sensors on the external, switched 5 V rail (§14): gated
  > on the rail epoch.
static bool onFiveVoltRail(const DriverRun* run) {
  return sensorMaskHas(SENSOR_MASK_RAIL_5V,
  > sensorDriverId(run->driver));
}
```

evaluateRules skips disabled rules and nonmatching metrics, calls the pure rule function with its persistent state and submits only TRIP or CLEAR events. The event contains a copy of the triggering Reading.

The latency-test build changes rearming so each eligible test sample can produce another trip. That conditional code is a test behaviour, not the ordinary production rule policy. onFiveVoltRail uses sensor membership rather than a hardcoded GPIO comparison.

REMEMBER

The call through the driver table is ordinary C dispatch. Review the operations table (p. 82) when d->ops is unfamiliar.

Publish a new absence or state observation

src/services/acquisition.c / lines 165-180

```
165
166 // Every metric of the run's sensor shows `health` now. No new value (telemetry
167 // sends null), but a new observation: a new sequence, so the app records the
168 // transition and every repeated failure (contract: one row per seq).
169 static void markSensor(const DriverRun* run, Health health, MonoMs now) {
170   const SensorId s = sensorDriverId(run->driver);
171   for (uint8_t m = 0; m < METRIC_COUNT; ++m) {
172     if (sensorOfMetric((MetricId)m) != s) continue;
173     Reading* r = &s_work.readings[m];
174     r->health = health;
175     r->intervalStartMs = now;
176     r->intervalEndMs = now;
177     r->powerEpoch = railStateRead().epoch;
178     ++r->sequence;
179   }
180 }
```

markSensor walks every metric belonging to the affected sensor. It changes health and interval, stamps the current rail epoch and advances the published sequence even without a new physical value.

This makes a new repeated failure or power-state observation visible to consumers that suppress duplicate sequences. The function does not claim that an old stored number was measured again; health carries the new meaning.

Collect outcomes and reject a changed rail history

src/services/acquisition.c / lines 181-200

```
181
182 // Stamp, store, check rules, for every reading the driver returned.
183 static void storeResults(DriverRun* run, MonoMs now, bool guardTimeout) {
184   Reading out[4];
185   const uint8_t n = sensorDriverReadResult(run->driver, out, 4);
186   const RailState rail = railStateRead();
187   const uint32_t epoch = rail.epoch;
188   // The rail went down (or down and up) during the measurement: the number
189   // may come from a collapsing supply, so it is never published (§14).
190   const bool railLost = onFiveVoltRail(run) && (!rail.valid5v || rail.epoch !=
  >   run->startEpoch);
191   for (uint8_t k = 0; k < n; ++k) {
192     Reading* r = &out[k];
193     if (r->metric >= METRIC_COUNT) continue;
194     if (guardTimeout) {  // abandoned at its deadline: a new, failed result
195       r->health = HEALTH_TIMEOUT;
196       r->qualityFlags = (uint16_t)(r->qualityFlags | kQualityLate);
197       r->intervalEndMs = now;
198     }
199     if (railLost) r->health = HEALTH_UNPOWERED;
200     // Every stored result is a new observation, failures included: the
```

storeResults obtains up to four copied records and one coherent rail state. For a 5V sensor, an invalid rail or changed epoch rejects the in-flight observation. An outer guard can instead add TIMEOUT and the late qualification.

The metric range guard prevents an invalid driver result from indexing outside s_work. This check belongs at a trust boundary even though the current concrete drivers are expected to supply valid identities.

Stamp a new observation before publishing its collection

src/services/acquisition.c / lines 201-212

```
    // drivers count only usable samples, which would give a FAULT after an OK
    // the OK's sequence (and the app would drop it as a re-send).
    r->sequence = s_work.readings[r->metric].sequence + 1u;
    r->powerEpoch = epoch;
    r->configRevision = s_appliedRevision;
    s_work.readings[r->metric] = *r;
    evaluateRules(r, now);
  }
  s_work.publishedMs = now;
  s_work.powerEpoch = epoch;
  snapshotPublish(&s_work);
}
```

The published sequence comes from the previous working slot plus one, so driver-level usable-sample counters cannot hide a new fault. Epoch and configuration revision are assigned before the structure copy and rule evaluation.

After all returned metrics are processed, the service updates Snapshot metadata and overwrites the retained store. It does not wait for every unrelated sensor in the frame to finish.

Log a change in failure state instead of repeating it

src/services/acquisition.c / lines 213-225

```
213
214 static void noteFailure(DriverRun* run, DriverStatus st, const char* what) {
215   if (st == run->lastFailure) return;  // log transitions only
216   run->lastFailure = st;
217   logPrintf(LOG_LEVEL_WARN, "acq: %s %s -> %s", sensorDriverName(run->driver), what,
218             driverStatusName(st));
219 }
220
221 static void noteRecovered(DriverRun* run) {
222   if (run->lastFailure == DRIVER_OK) return;
223   run->lastFailure = DRIVER_OK;
224   logPrintf(LOG_LEVEL_INFO, "acq: %s OK", sensorDriverName(run->driver));
225 }
```

noteFailure compares the new status with lastFailure and prints only a change. noteRecovered similarly reports a return to success. These helpers improve visibility without placing an identical message on every poll.

Logging is still a bounded side path. A log transition describes what the worker observed; it does not control whether the driver is scheduled again.

End every run through its resource cleanup

src/services/acquisition.c / lines 226-253

```
226
227 // The HC-SR04 ping holds the flash gate from start to
  >   result (flash_gate.h).
228 static void releasePingGate(const DriverRun* run) {
229   if (run == &s_runs[RUN_HCSR04]) flashGateGiveAfterPing();
230 }
231
232 static void finishRun(DriverRun* run, MonoMs now,
  >   DriverStatus st) {
233   run->pending = false;
234   releasePingGate(run);
235   if (st == DRIVER_READY) {
236     noteRecovered(run);
237   } else {
238     noteFailure(run, st, "result");
239     STATS_ADD(driverFailures, 1);
240     if (st == DRIVER_NOT_PRESENT || st ==
  >   DRIVER_NOT_IMPLEMENTED) {
241       run->initialised = false;
242       run->retryAtMs = now;  // re-init at its next due time
243     }
244   }
245   storeResults(run, now, false);
246 }
247
248 static void guardTimeout(DriverRun* run, MonoMs now) {
249   sensorDriverCancel(run->driver);
250   run->pending = false;
251   releasePingGate(run);
252   noteFailure(run, DRIVER_TIMEOUT, "guard");
253   STATS_ADD(guardTimeouts, 1);
```

finishRun clears pending and releases a held ping gate before reporting recovery or failure and collecting results. Selected missing/unimplemented statuses force later init.

src/services/acquisition.c / lines 254-255

```
254   storeResults(run, now, true);
255 }
```

guardTimeout cancels the driver, clears pending, releases the same gate and collects a timeout-tagged outcome. Normal completion and outer cancellation therefore do not leave the flash-exclusion resource held.

COMMON PITFALL

A cleanup path that forgets one acquired gate can stall another service long after the original measurement failed.

Protect two distinct timing conflicts

src/services/acquisition.c / lines 256-278

```
256
257 // ---- Blocking rules (§9) ------------------------------------------------------
258
259 static bool isBlocking(uint32_t budgetMs) { return budgetMs > 2; }
260
261 // May a step blocking the worker for budgetMs run now? Not while a BME280
262 // conversion is pending (its poll would come too late and time out), not
263 // while an HC-SR04 echo is in flight (1-Wire masks interrupts), and only if
264 // it ends before the next BME280 slot.
265 static bool blockingAllowed(uint8_t runId, uint32_t budgetMs, MonoMs now) {
266   if (!isBlocking(budgetMs)) return true;
267   if (runId != RUN_BME280 && s_runs[RUN_BME280].pending) return false;
268   if (s_runs[RUN_HCSR04].pending) return false;
269   if (runId == RUN_BME280) return true;
270   return scheduleFitsBefore(now, budgetMs, s_jobs[JOB_BME280].nextDueMs);
271 }
272
273 // The ping must not overlap a DS18B20 call: defer it if one is due soon.
274 static bool pingAllowed(MonoMs now) {
275   const DriverRun* ds = &s_runs[RUN_DS18B20];
276   if (ds->wantStart) return false;
277   return !ds->pending || ds->nextPollMs > now + CFG_HCSR04_TIMEOUT_MS + 2;
278 }
```

blockingAllowed rejects long calls while a BME280 conversion needs timely polling or while an echo is active, then checks fit before the next climate slot. pingAllowed postpones an ultrasonic trigger if a pending or wanted 1-Wire action could collide with its window.

The inequalities use documented budgets. They are not a real-time scheduler proof: if an actual call exceeds its declared cost, the exclusion reasoning must be revisited.

Supply cross-sensor dependencies from one retained collection

src/services/acquisition.c / lines 279-289

```
279
280 // ---- Start and poll ------------------------------------------------------
281
282 static void prepareInputs(uint8_t runId, MonoMs now) {
283   if (runId == RUN_SGP30) {
284     sgp30DriverSetHumidity(&s_sgp, &s_work.readings[METRIC_AIR_TEMPERATURE],
285                            &s_work.readings[METRIC_AIR_HUMIDITY], now);
286   } else if (runId == RUN_TDS) {
287     tdsDriverSetWaterTemperature(&s_tds, &s_work.readings[METRIC_WATER_TEMPERATURE]);
288   }
289 }
```

Before gas starts, acquisition passes the latest temperature and humidity records. Before TDS starts, it passes water temperature. The setters copy or derive retained input without taking another bus.

The dependency can be from a previous conversion. The receiving driver checks health and age at use. This is why a working Snapshot needs per-reading timestamps rather than only a frame number.

Refuse work before touching an unavailable device

src/services/acquisition.c / lines 290-317

```
290
291 // Returns false if the start must wait (blocking rule); the job stays wanted.
292 static bool tryStart(uint8_t runId, MonoMs now) {
293   DriverRun* run = &s_runs[runId];
294   if (runId == RUN_HCSR04 && !pingAllowed(now)) return false;
295   if (onFiveVoltRail(run)) {
296     const RailState rail = railStateRead();
297     if (!rail.valid5v) {  // no 5 V: no init, no conversion of a dead output
298       markSensor(run, HEALTH_UNPOWERED, now);
299       run->wantStart = false;
300       return true;
301     }
302     run->startEpoch = rail.epoch;
303   }
304   if (!run->initialised) {
305     if (!monoReached(now, run->retryAtMs)) {  // back-off: skip this period
306       run->wantStart = false;
307       return true;
308     }
309     if (!blockingAllowed(runId, run->initBudgetMs, now)) return false;
310     const DriverStatus st = sensorDriverInit(run->driver);
311     if (st != DRIVER_OK) {
312       noteFailure(run, st, "init");
313       run->retryAtMs = now + POLICY_INIT_RETRY_MS;
314       run->wantStart = false;
315       return true;
316     }
317     run->initialised = true;
```

tryStart first protects the echo/1-Wire relation and 5V rail. It stores the starting epoch before accepting a rail-dependent operation. A failed init is retried only after retryAtMs, with a later scheduled attempt rather than an endless immediate loop.

After a potentially long init, the service refreshes now. Reusing the pre-init timestamp for every subsequent budget decision would ignore the time setup just consumed.

Take the ping gate before issuing its trigger

src/services/acquisition.c / lines 318-335

```
    logPrintf(LOG_LEVEL_INFO, "acq: %s init OK", sensorDriverName(run->driver));
    now = monoNowMs();  // init may have taken tens of ms
  }
  if (!blockingAllowed(runId, run->stepBudgetMs, now)) return false;
  // No NVS write may overlap the echo window: defer the ping while one runs.
  if (runId == RUN_HCSR04 && !flashGateTryTakeForPing()) return false;
  prepareInputs(runId, now);
  run->wantStart = false;
  const DriverStatus st = sensorDriverStart(run->driver, now);
  if (st == DRIVER_OK) {
    run->pending = true;
    run->nextPollMs = now;
  } else {
    if (st != DRIVER_PENDING) run->initialised = false;
    finishRun(run, now, st);
  }
  return true;
}
```

The next blocking check uses the refreshed time. HC-SR04 must obtain the flash gate before prepareInputs and start. The helper then clears wantStart and either establishes pending state or finishes a rejected operation.

A refusal to obtain the gate leaves the work wanted, allowing a later pass to try again. That is different from a hardware failure, which produces a collected outcome and diagnostic state.

Advance only a due, permitted step

src/services/acquisition.c / lines 336-349

```
336
337 // Returns true if the run made progress.
338 static bool tryPoll(uint8_t runId, MonoMs now) {
339   DriverRun* run = &s_runs[runId];
340   if (!run->pending || now < run->nextPollMs) return false;
341   if (!blockingAllowed(runId, run->stepBudgetMs, now)) return false;
342   const PollResult pr = sensorDriverPoll(run->driver, now);
343   if (pr.status == DRIVER_PENDING) {
344     run->nextPollMs = pr.nextPollMs > now ? pr.nextPollMs : now + 1;
345     return true;
346   }
347   finishRun(run, monoNowMs(), pr.status);
348   return true;
349 }
```

tryPoll rejects a nonpending run or a future poll. It also applies the same blocking exclusions. PENDING stores a useful future time, clamping an immediate-or-past suggestion to now+1.

Any completed status routes through finishRun, which centralises cleanup and publication. Returning true means the pass made scheduling progress; it does not mean the physical result was OK.

Copy configuration inputs into the actual drivers

src/services/acquisition.c / lines 350-372

```
350
351 // ---- Frame and slot boundaries ------------------------------------------------
352
353 // pH is provisional (UNCALIBRATED in pH units) until the user fits it on the
354 // device; the compiled nominal fit is never buffer-verified (D13).
355 static bool phProvisional(const ConfigBlob* cfg) { return (cfg->calOverridden &
  >   CAL_BIT_PH) == 0; }
356
357 // Drivers keep a private copy of their config: hand them the applied fits.
358 // A window in progress finishes with the old fit (its x is already taken).
359 static void applyCalibrations(const ConfigBlob* cfg) {
360   phDriverSetCalibration(&s_ph, &cfg->ph, phProvisional(cfg));
361   tdsDriverSetCalibration(&s_tds, &cfg->tds);
362   lightDriverSetCalibration(&s_light, &cfg->light);
363 }
364
365 // Sensor on/off and calibration changes take effect here, between frames, by
366 // this task only: no other task ever touches a driver (§17).
367 static SensorMask allowedSensors(void) {
368   portENTER_CRITICAL(&s_leaseMux);
369   const SensorMask allowed = s_allowed;
370   portEXIT_CRITICAL(&s_leaseMux);
371   return allowed;
372 }
```

phProvisional derives trust from the override bit. applyCalibrations calls the three setters with a coherent configuration copy. allowedSensors reads the current lease mask under its lock.

A valid fitted record and permission to operate a sensor answer different questions. The next helper combines requested enable state with what this worker may actually access.

Apply a revision at an ownership-safe boundary

src/services/acquisition.c / lines 373-400

```
373
374 static void applyConfiguration(MonoMs now) {
375   configurationPeekApplied(&s_cfg);
376   if (s_cfg.revision == s_appliedRevision && !s_forceApply)
  >   return;
377   s_forceApply = false;
378   applyCalibrations(&s_cfg);
379   const SensorMask allowed = allowedSensors();
380   for (uint8_t i = 0; i < RUN_COUNT; ++i) {
381     DriverRun* run = &s_runs[i];
382     const SensorId id = sensorDriverId(run->driver);
383     const bool wanted = sensorMaskHas(s_cfg.sensorEnabled,
  >   id);
384     const bool enabled = wanted && sensorMaskHas(allowed,
  >   id);
385     if (wanted && !enabled) markSensor(run,
  >   HEALTH_UNPOWERED, now);  // low power: its rail is off
386     if (!enabled && !run->disabled) {
387       if (run->pending) {
388         sensorDriverCancel(run->driver);  // no
  >   preparePowerOff: the shared I2C bus stays in use
389         run->pending = false;
390         releasePingGate(run);
391       }
392       run->disabled = true;
393       run->wantStart = false;
394       logPrintf(LOG_LEVEL_INFO, "acq: %s disabled",
  >   sensorDriverName(run->driver));
395     } else if (enabled && run->disabled) {
396       run->disabled = false;
397       run->initialised = false;  // init again at its next
  >   due time
398       run->retryAtMs = now;
399       run->lastFailure = DRIVER_OK;
400       markSensor(run, HEALTH_WARMING, now);  // until its
  >   first result (§13 restore)
```

COMMON PITFALL

The source comment says an in-progress window uses its previous fit; frame guards and ownership boundaries must preserve that claim when settings are applied.

Apply a revision at an ownership-safe boundary / continued

The helper peeks applied configuration and exits if neither revision nor forced reapply changed. It updates calibration copies, then compares each requested sensor with the allowed set. Disabled in-flight work is cancelled and releases any ping gate; newly enabled work requires fresh init and begins WARMING.

src/services/acquisition.c / lines 401-406

```
401         logPrintf(LOG_LEVEL_INFO, "acq: %s enabled", sensorDriverName(run->driver));
402       }
403     }
404     s_appliedRevision = s_cfg.revision;
405     configurationAckApplied(s_appliedRevision);
406 }
```

Only after the loop does acquisition record and acknowledge the revision. The acknowledgement means this owner processed that revision at its boundary, not merely that Configuration wrote it to flash.

Release the ordinary jobs and end leftover old-frame work

src/services/acquisition.c / lines 407-433

```
407
408 static void offerBaseline(MonoMs now) {
409   if (now < s_nextBaselineOfferMs) return;
410   s_nextBaselineOfferMs = now + BASELINE_OFFER_PERIOD_MS;
411   uint16_t co2eq, tvoc;
412   if (!s_runs[RUN_SGP30].disabled && sgp30DriverGetBaseline(&s_sgp, &co2eq, &tvoc)) {
413     configurationOfferBaseline(co2eq, tvoc);
414   }
415 }
416
417 static void startFrame(MonoMs now) {
418   applyConfiguration(now);
419   offerBaseline(now);
420   s_sgpBaseline = sgp30DriverBaselineStatus(&s_sgp);
421   ScheduleJob* job = &s_jobs[JOB_FRAME];
422   const uint32_t lateBefore = job->late;
423   const uint32_t skippedBefore = job->skipped;
424   scheduleJobMarkRun(job, now);
425
426   // Frame guard: a one-second job still running from the last frame is
427   // abandoned, never retried inside this frame (§18).
428   for (uint8_t i = 0; i < FRAME_ORDER_COUNT; ++i) {
429     DriverRun* run = &s_runs[kFrameOrder[i]];
430     if (run->pending) guardTimeout(run, now);
431     run->wantStart = !run->disabled;
432   }
433
```

offerBaseline rate-limits offers to Configuration without performing a flash write. startFrame applies configuration, updates gas status and marks the frame release. Any ordinary operation still pending from the preceding frame is abandoned through guardTimeout.

Release the ordinary jobs and end leftover old-frame work / continued

src/services/acquisition.c / lines 434-444

```
434     ++s_work.frameSequence;
435     s_work.publishedMs = now;
436     snapshotPublish(&s_work);
437
438     portENTER_CRITICAL(&s_statsMux);
439     ++s_stats.frames;
440     s_stats.frameLate += job->late - lateBefore;
441     s_stats.frameSkipped += job->skipped - skippedBefore;
442     s_stats.frameJitterMaxMs = job->maxLateMs;
443     portEXIT_CRITICAL(&s_statsMux);
444 }
```

It then marks enabled jobs wanted, advances frame metadata and publishes. The frame publication can include old-but-retained readings while newly released physical operations have not yet completed.

Keep the fast release grid independent

src/services/acquisition.c / lines 445-459

```
445
446 static void startBmeSlot(MonoMs now) {
447   ScheduleJob* job = &s_jobs[JOB_BME280];
448   const uint32_t lateBefore = job->late;
449   const uint32_t skippedBefore = job->skipped;
450   scheduleJobMarkRun(job, now);
451   if (s_runs[RUN_BME280].pending) guardTimeout(&s_runs[RUN_BME280], now);
452   // A disabled BME280 keeps its slot grid: blocking steps still respect it
453   // (scheduleFitsBefore), which is conservative but never wrong.
454   s_runs[RUN_BME280].wantStart = !s_runs[RUN_BME280].disabled;
455   portENTER_CRITICAL(&s_statsMux);
456   s_stats.bmeLate += job->late - lateBefore;
457   s_stats.bmeSkipped += job->skipped - skippedBefore;
458   portEXIT_CRITICAL(&s_statsMux);
459 }
```

The climate slot marks timing evidence, cancels an overlong preceding BME280 run and requests another only when enabled. The grid remains present even when the sensor is disabled, making blocking-fit checks conservative.

This choice trades some potentially unused capacity for simpler scheduling assumptions. A disabled device does not automatically remove every timing reservation from this implementation.

Advance several short operations without an unbounded loop

src/services/acquisition.c / lines 460-481

```
460
461 // One pass: boundaries first, the BME280 fast lane next, then every due poll
462 // and the frame starts in order. Repeats while something progressed (each
463 // step is bounded, so the pass is too).
464 static void runPass(MonoMs now) {
465   if (scheduleJobDue(&s_jobs[JOB_FRAME], now)) startFrame(now);
466   if (scheduleJobDue(&s_jobs[JOB_BME280], now)) startBmeSlot(now);
467
468   for (uint8_t iter = 0; iter < 8; ++iter) {
469     bool progress = false;
470     now = monoNowMs();
471     if (s_runs[RUN_BME280].wantStart) progress |= tryStart(RUN_BME280, now);
472     for (uint8_t r = 0; r < RUN_COUNT; ++r) progress |= tryPoll(r, monoNowMs());
473     for (uint8_t i = 0; i < FRAME_ORDER_COUNT; ++i) {
474       const uint8_t r = kFrameOrder[i];
475       if (!s_runs[r].wantStart) continue;
476       if (!tryStart(r, monoNowMs())) break;  // keep the order: later ones wait too
477       progress = true;
478     }
479     if (!progress) break;
480   }
481 }
```

runPass notices boundaries, gives the fast job an opportunity, polls due operations and attempts ordinary starts in order. Up to eight internal iterations allow newly unblocked steps to progress without an open-ended loop.

The bitwise |= keeps evaluating calls while accumulating progress. Replacing it with a short-circuit expression could skip later steps once progress was already true. Break on no progress prevents repeatedly calling a driver before time or permissions change.

Choose a useful wake and configure hardware on its intended core

src/services/acquisition.c / lines 482-509

```
482
483 // Next wake: the earliest job deadline or poll time that is not blocked
484 // (a blocked step is woken by whatever blocks it, which is in this set).
485 static MonoMs nextWake(MonoMs now) {
486   MonoMs due = scheduleEarliestDue(s_jobs, JOB_COUNT);
487   for (uint8_t r = 0; r < RUN_COUNT; ++r) {
488     const DriverRun* run = &s_runs[r];
489     if (!run->pending) continue;
490     const MonoMs t = run->nextPollMs > now ? run->nextPollMs : now + 1;
491     if (blockingAllowed(r, run->stepBudgetMs, t) && t < due) due = t;
492   }
493   return due;
494 }
495
496 // HALs begin in AcqNormal, on core 1: the echo ISR (and with it the GPIO ISR
497 // service the power-sense ISR shares) is installed on this core.
498 static void beginHals(void) {
499   if (!i2cBusBegin(&s_i2c, BOARD_PIN_I2C_SDA, BOARD_PIN_I2C_SCL, BOARD_I2C_CLOCK_HZ)) {
500     logPrintf(LOG_LEVEL_ERROR, "acq: I2C begin failed");
501   }
502   if (!oneWireBusBegin(&s_oneWire, BOARD_PIN_ONE_WIRE)) logPrintf(LOG_LEVEL_ERROR, "acq:
  >    1-Wire begin failed");
503   if (!adcChannelBegin(&s_adcLight, BOARD_PIN_LIGHT_ADC)) logPrintf(LOG_LEVEL_ERROR,
  >   "acq: light ADC failed");
504   if (!adcChannelBegin(&s_adcTds, BOARD_PIN_TDS_ADC)) logPrintf(LOG_LEVEL_ERROR, "acq:
  >   TDS ADC failed");
505   if (!adcChannelBegin(&s_adcPh, BOARD_PIN_PH_ADC)) logPrintf(LOG_LEVEL_ERROR, "acq: pH
  >   ADC failed");
506   if (!pulseCaptureBegin(&s_capture, BOARD_PIN_HCSR04_TRIG, BOARD_PIN_HCSR04_ECHO)) {
507     logPrintf(LOG_LEVEL_ERROR, "acq: HC-SR04 capture failed");
508   }
509 }
```

nextWake considers the earliest release and permitted pending polls. beginHals configures the shared I2C bus,1-Wire line, three ADC channels and pulse capture. Its logs report individual setup failures.

Normal acquisition runs this initial HAL setup on core 1 so the GPIO handler service is installed there. That placement is a dependency for the later power-sense handler; setup is not duplicated by the low-power worker.

Inspect lease permission under the same lock used by writers

src/services/acquisition.c / lines 510-525

```
510
511 static TaskHandle_t workerTask(PowerWorker w) { return w == POWER_WORKER_LOW ? s_taskLow
  >     : s_task; }
512
513 static bool holdsLease(PowerWorker me) {
514   portENTER_CRITICAL(&s_leaseMux);
515   const bool mine = s_owner == me;
516   portEXIT_CRITICAL(&s_leaseMux);
517   return mine;
518 }
519
520 static bool quiesceRequested(void) {
521   portENTER_CRITICAL(&s_leaseMux);
522   const bool requested = s_quiesceGen != 0 && s_quiesceGen == s_leaseGen;
523   portEXIT_CRITICAL(&s_leaseMux);
524   return requested;
525 }
```

workerTask selects a context handle. holdsLease and quiesceRequested read their related fields while holding the lease portMUX. The returned booleans are snapshots of permission, not permanent guarantees independent of the protocol.

The worker acts on quiescence at its own boundary. No other task forcibly edits a driver half way through a transaction. Short lock holds protect metadata; they do not cover the whole sensor pass.

Restart the schedule when a new owner begins

src/services/acquisition.c / lines 526-543

```
526
527 // New holder: join the task watchdog, restart the schedule from now (the
528 // previous holder's deadlines are not ours to count as late) and apply the
529 // configuration within the allowed sensors at the first frame.
530 static void acquireLease(PowerWorker me, MonoMs now) {
531   if (esp_task_wdt_add(NULL) != ESP_OK) logPrintf(LOG_LEVEL_ERROR, "acq: task watchdog
  >   add failed");
532   scheduleJobInit(&s_jobs[JOB_FRAME], CFG_FRAME_PERIOD_MS, CFG_FRAME_RESERVATION_MS,
  >   now);
533   scheduleJobInit(&s_jobs[JOB_BME280], CFG_BME280_PERIOD_MS, CFG_BME280_BUDGET_MS, now);
534   for (uint8_t i = 0; i < RUN_COUNT; ++i) {
535     s_runs[i].pending = false;
536     s_runs[i].wantStart = false;
537   }
538   s_forceApply = true;
539   progressMark(me == POWER_WORKER_NORMAL ? TASK_ID_ACQUISITION_NORMAL :
  >   TASK_ID_ACQUISITION_LOW_POWER);
540   logPrintf(LOG_LEVEL_INFO, "acq: %s worker holds the lease (gen %lu, sensors 0x%02x)",
541             me == POWER_WORKER_NORMAL ? "normal" : "low-power", (unsigned
  >   long)s_leaseGen,
542             (unsigned)allowedSensors());
543 }
```

The new holder subscribes itself to the task watchdog, initialises both release grids from now, clears pending/wanted scheduling flags and forces configuration application. It marks meaningful startup progress and logs which allowed set it received.

Restarting from now avoids a burst of obsolete releases from a worker that had waited without ownership. Driver configuration and availability are then reconciled before new observations are accepted.

Finish cleanup before publishing release

src/services/acquisition.c / lines 544-571

```
544
545 // QUIESCE -> RELEASED (§14): nothing is left running, the 5
  >    V interfaces are
546 // safe if their rail is down, and the worker never touches
  >   a driver again
547 // until the next grant.
548 static void releaseLease(PowerWorker me) {
549   const bool railDown = !railStateRead().valid5v;
550   for (uint8_t i = 0; i < RUN_COUNT; ++i) {
551     DriverRun* run = &s_runs[i];
552     if (run->pending) {
553       sensorDriverCancel(run->driver);
554       run->pending = false;
555       releasePingGate(run);
556     }
557     run->wantStart = false;
558     if (railDown && onFiveVoltRail(run)) {
559       sensorDriverPreparePowerOff(run->driver);  // HC-SR04:
  >    TRIG held low, no drive into a dead module
560       run->initialised = false;
561     }
562   }
563   snapshotPublish(&s_work);
564   esp_task_wdt_delete(NULL);  // a blocked worker owes the
  >   watchdog nothing
565   portENTER_CRITICAL(&s_leaseMux);
566   s_owner = POWER_WORKER_NONE;
567   s_releasedGen = s_leaseGen;
568   s_quiesceGen = 0;
569   portEXIT_CRITICAL(&s_leaseMux);
570   logPrintf(LOG_LEVEL_INFO, "acq: %s worker released (gen
  >   %lu)", me == POWER_WORKER_NORMAL ? "normal" : "low-power",
571             (unsigned long)s_releasedGen);
```

COMMON PITFALL

Publishing NONE before cancellation would let a new owner enter while the previous owner still touches shared drivers.

The holder cancels pending operations, gives the ping gate and makes 5V interfaces safe when the rail is down. It publishes retained state and leaves its watchdog subscription before changing owner to NONE under the lock.

src/services/acquisition.c / lines 572-572

```
572 }
```

Only then does releasedGen advertise the finished ownership interval. The coordinator can use that record to grant another worker without simultaneous access. The stored generation must be the one actually released.

Wait for permission before entering the active loop

src/services/acquisition.c / lines 573-598

```
573
574 static void acqTask(void* arg) {
575   const PowerWorker me = (PowerWorker)(uintptr_t)arg;
576   const TaskId id = me == POWER_WORKER_NORMAL ? TASK_ID_ACQUISITION_NORMAL :
  >   TASK_ID_ACQUISITION_LOW_POWER;
577   if (me == POWER_WORKER_NORMAL) {
578     beginHals();
579     s_halsReady = true;
580     portENTER_CRITICAL(&s_leaseMux);
581     const PowerWorker owner = s_owner;
582     portEXIT_CRITICAL(&s_leaseMux);
583     if (owner == POWER_WORKER_LOW && s_taskLow != NULL) xTaskNotifyGive(s_taskLow);  //
  >   granted before the HALs
584   }
585   const MonoMs t0 = monoNowMs();
586   (void)t0;
587   bool holding = false;
588
589   // Task watchdog (§18): the holder resets it after each pass. A pass happens
590   // at least every 250 ms (BME280 slot), so 5 s of silence means the task is
591   // stuck inside a call. A worker without the lease is not on the watchdog.
592   for (;;) {
593     if (!holding) {
594       ulTaskNotifyTake(pdTRUE, portMAX_DELAY);  // a grant (or a stale wake-up)
595       if (!s_halsReady || !holdsLease(me)) continue;
596       holding = true;
597       acquireLease(me, monoNowMs());
598     }
```

acqTask decodes the small worker identity carried through void*. Normal performs HAL setup; low-power waits for that readiness. A worker without holding state blocks on notifications and then verifies actual lease permission, rejecting stale wake-ups.

Acquiring permission is a state check, not the notification itself. A notification can prompt inspection while the recorded owner still belongs to somebody else.

Recheck urgent state after every wake

src/services/acquisition.c / lines 599-619

```
599     if (quiesceRequested()) {
600       releaseLease(me);
601       holding = false;
602       continue;
603     }
604     const MonoMs now = monoNowMs();
605     const MonoMs due = nextWake(now);
606     ulTaskNotifyTake(pdTRUE, due > now ? pdMS_TO_TICKS((uint32_t)(due - now)) : 0);
607     if (quiesceRequested()) continue;  // released at the top, before any new step
608     runPass(monoNowMs());
609 #ifdef FW_INJECT_STALL_ACQ
610     // Acceptance test (§19 "Stall injection"): block forever after N ms.
611     if (monoNowMs() - t0 >= (MonoMs)(FW_INJECT_STALL_ACQ)) {
612       logPrintf(LOG_LEVEL_WARN, "acq: INJECTED STALL - blocking forever");
613       vTaskDelay(portMAX_DELAY);
614     }
615 #endif
616     progressMark(id);
617     esp_task_wdt_reset();
618   }
619 }
```

An active worker handles quiescence before new work, derives a due timeout and waits for a notification or deadline. After waking it checks quiescence again before running a pass.

Progress and watchdog reset follow the completed pass. The optional injected stall deliberately prevents that completion for an acceptance-test build. A task without a lease is intentionally not watched as though it owed active acquisition progress.

Keep context lookup and static task creation distinct

src/services/acquisition.c / lines 620-645

```
620
621 TaskHandle_t acquisitionTaskHandle(void) {
622   portENTER_CRITICAL(&s_leaseMux);
623   const PowerWorker owner = s_owner;
624   portEXIT_CRITICAL(&s_leaseMux);
625   return owner == POWER_WORKER_LOW ? s_taskLow : s_task;
626 }
627
628 TaskHandle_t acquisitionNormalTaskHandle(void) { return s_task; }
629 TaskHandle_t acquisitionLowPowerTaskHandle(void) { return s_taskLow; }
630
631 static TaskHandle_t createWorker(TaskId id, PowerWorker w, StackType_t* stack, uint32_t
  >   stackBytes,
632                                  StaticTask_t* tcb) {
633   const TaskSpec* spec = cfgTaskSpec(id);
634   if (spec->stackBytes != stackBytes) {
635     logPrintf(LOG_LEVEL_WARN, "acq: %s stack %lu B, task table says %lu B", spec->name,
  >   (unsigned long)stackBytes,
636               (unsigned long)spec->stackBytes);
637   }
638   TaskHandle_t t = xTaskCreateStaticPinnedToCore(acqTask, spec->name, stackBytes,
  >   (void*)(uintptr_t)w,
639                                                  spec->priority, stack, tcb,
  >   spec->core);
640   if (t != NULL) {
641     logPrintf(LOG_LEVEL_INFO, "acq: %s started (core %u, prio %u)", spec->name,
  >   (unsigned)spec->core,
642               (unsigned)spec->priority);
643   }
644   return t;
645 }
```

The public handle helpers expose notification destinations. createWorker compares the actual supplied stack byte count with the task table, then supplies the persistent stack and control block to the static pinned API.

The worker identity is transported through uintptr_t and void* as a small identifier on this target. It is not an address to dereference. Successful creation returns a handle; a failed creation remains a separate state the caller must handle.

Construct all drivers before granting any owner

src/services/acquisition.c / lines 646-671

```
646
647 bool acquisitionInit(void) {
648   if (s_task != NULL) return true;
649   // Applied configuration: calibrations now, the sensor mask at the first frame.
650   configurationPeekApplied(&s_cfg);
651   s_phConfig.calibration = s_cfg.ph;
652   s_phConfig.calibrationProvisional = phProvisional(&s_cfg);
653   s_tdsConfig.calibration = s_cfg.tds;
654   s_lightConfig.calibration = s_cfg.light;
655   bme280DriverConstruct(&s_bme, &s_bmeConfig);
656   sgp30DriverConstruct(&s_sgp, &s_sgpConfig);
657   ds18b20DriverConstruct(&s_ds, &s_dsConfig);
658   lightDriverConstruct(&s_light, &s_lightConfig);
659   tdsDriverConstruct(&s_tds, &s_tdsConfig);
660   phDriverConstruct(&s_ph, &s_phConfig);
661   hcsr04DriverConstruct(&s_hc, &s_hcConfig);
662   uint16_t co2eq, tvoc;
663   if (configurationStoredBaseline(&co2eq, &tvoc)) {
664     sgp30DriverSetBaseline(&s_sgp, co2eq, tvoc);  // restored at the SGP30's init
665     logPrintf(LOG_LEVEL_INFO, "acq: SGP30 baseline restored from NVS");
666   }
667   s_appliedRevision = 0;  // revisions start at 1: forces applyConfiguration() at the
  >   first frame
668   s_nextBaselineOfferMs = BASELINE_OFFER_PERIOD_MS;
669   snapshotReset(&s_work);
670   memset(s_ruleState, 0, sizeof(s_ruleState));
671
```

Init loads the applied fits, constructs every driver, stages a stored gas baseline, resets the working Snapshot and rule state, then creates normal and low-power contexts without granting permission.

Construct all drivers before granting any owner / continued

src/services/acquisition.c / lines 672-678

```
672   // No lease yet: PowerCoordinator grants the first one (§14).
673   s_task = createWorker(TASK_ID_ACQUISITION_NORMAL, POWER_WORKER_NORMAL, s_stack,
  >   sizeof(s_stack), &s_tcb);
674   s_taskLow = createWorker(TASK_ID_ACQUISITION_LOW_POWER, POWER_WORKER_LOW, s_stackLow,
  >   sizeof(s_stackLow),
675                            &s_tcbLow);
676   if (s_taskLow == NULL) logPrintf(LOG_LEVEL_ERROR, "acq: low-power worker missing - no
  >   sensing in low power");
677   return s_task != NULL;
678 }
```

The coordinator chooses the initial holder later. A missing low-power context is logged; the return mainly reports whether the normal worker exists. Review that partial-start behaviour when reasoning about a later battery transition.

Record permission before sending the prompt

src/services/acquisition.c / lines 679-698

```
679
680 bool acquisitionGrant(PowerWorker worker, uint32_t generation, SensorMask allowed) {
681   if (worker != POWER_WORKER_NORMAL && worker != POWER_WORKER_LOW) return false;
682   portENTER_CRITICAL(&s_leaseMux);
683   const bool free = s_owner == POWER_WORKER_NONE;
684   if (free) {
685     s_owner = worker;
686     s_leaseGen = generation;
687     s_allowed = (SensorMask)(allowed & SENSOR_MASK_ALL);
688     s_quiesceGen = 0;
689   }
690   portEXIT_CRITICAL(&s_leaseMux);
691   if (!free) {
692     logPrintf(LOG_LEVEL_ERROR, "acq: grant gen %lu refused - the lease is still held",
  >   (unsigned long)generation);
693     return false;
694   }
695   TaskHandle_t t = workerTask(worker);
696   if (t != NULL) xTaskNotifyGive(t);
697   return true;
698 }
```

Grant accepts only recognised worker identities and changes owner/generation/allowed fields only when no owner currently exists. It releases the lock before logging or notifying.

The receiving worker verifies this recorded permission after waking. A true return establishes the recorded grant; it does not prove that the task has already run a pass or produced its first new sample.

Match quiescence to the current ownership interval

src/services/acquisition.c / lines 699-717

```
699
700 bool acquisitionRequestQuiesce(uint32_t generation) {
701   portENTER_CRITICAL(&s_leaseMux);
702   const bool current = s_owner != POWER_WORKER_NONE && s_leaseGen == generation;
703   const PowerWorker owner = s_owner;
704   if (current) s_quiesceGen = generation;
705   portEXIT_CRITICAL(&s_leaseMux);
706   if (!current) return false;
707   TaskHandle_t t = workerTask(owner);
708   if (t != NULL) xTaskNotifyGive(t);  // wakes it from a bounded wait
709   return true;
710 }
711
712 uint32_t acquisitionReleasedGeneration(void) {
713   portENTER_CRITICAL(&s_leaseMux);
714   const uint32_t gen = s_releasedGen;
715   portEXIT_CRITICAL(&s_leaseMux);
716   return gen;
717 }
```

RequestQuiesce accepts only the current generation, records the request under the lock and wakes the corresponding worker. ReleasedGeneration copies the last completed release under the same protection.

A stale caller cannot request cleanup of an unrelated later interval merely by holding an old worker handle. The protocol uses both identity and generation to connect the request with its acknowledgement.

Expose bounded observations of the active engine

src/services/acquisition.c / lines 718-743

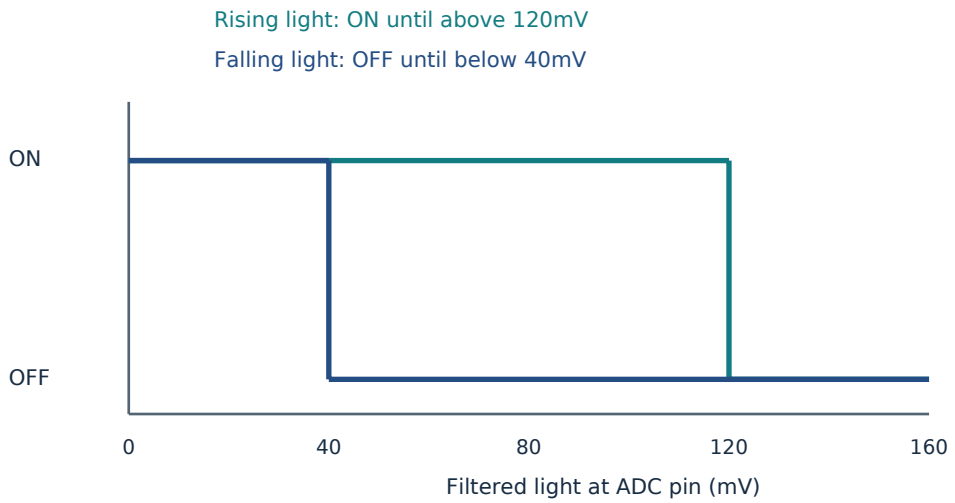
```
718
719 bool acquisitionHalsReady(void) { return s_halsReady; }
720
721 TaskId acquisitionActiveTaskId(void) {
722   portENTER_CRITICAL(&s_leaseMux);
723   const PowerWorker owner = s_owner;
724   portEXIT_CRITICAL(&s_leaseMux);
725   if (owner == POWER_WORKER_NORMAL) return TASK_ID_ACQUISITION_NORMAL;
726   if (owner == POWER_WORKER_LOW) return TASK_ID_ACQUISITION_LOW_POWER;
727   return TASK_ID_COUNT;
728 }
729
730 const char* acquisitionSgp30BaselineName(void) {
731   switch (s_sgpBaseline) {  // contract v2 spelling (lowercase)
732     case SGP30_BASELINE_LEARNING: return "learning";
733     case SGP30_BASELINE_RESTORED: return "restored";
734     case SGP30_BASELINE_VALID: return "valid";
735     default: return "none";
736   }
737 }
738
739 void acquisitionGetStats(AcquisitionStats* out) {
740   portENTER_CRITICAL(&s_statsMux);
741   *out = s_stats;
742   portEXIT_CRITICAL(&s_statsMux);
743 }
```

The remaining getters report HAL readiness, current active TaskId, baseline status spelling and a protected statistics copy. The active-id sentinel means no worker currently owns acquisition.

These functions support other services without allowing them to drive the buses. Notice which specialised fields still rely on volatile/single-writer assumptions; review those paths separately if the concurrency model changes.

Keep the previous command inside the band

The compiled lamp policy turns on below 40mV and off above 120mV. Between those levels it keeps its previous command. This is hysteresis: two thresholds prevent a noisy boundary from repeatedly toggling an output.



Threshold comparisons are strict in lamp_control.c; equality retains the previous desire.

Start OFF. A 30mV result asks for ON. A later 80mV result keeps ON. A 130mV result asks for OFF. Returning to 80mV keeps OFF. The same 80mV input can therefore produce either command depending on the prior state. One comparison without stored state cannot express that behaviour.

Dwell is a separate timer. With a 30000ms minimum-off interval, darkness at boot does not immediately energise the relay. Boot establishes OFF at its timestamp. A fresh dark result at or after the required held time can permit ON. An earlier result merely sets a waiting condition; later decisions still need current evidence.

The source uses raw pin millivolts after a lux calibration, so changing the display calibration does not move the lamp thresholds. Saturated light counts as bright. Repeated failed observations and stale data have their own paths that can force OFF.

40mV, 120mV and 30s are source policy values for this snapshot. They are not universal horticultural thresholds.

COMMON PITFALL

The lamp itself can illuminate the sensor. Thresholds must account for that optical feedback or the controller may switch itself back off.

Trip, clear and reset are different events

An alarm rule compares one metric with a trip threshold and a separate clear threshold. For an above-temperature teaching rule, trip at or above 40 degrees and clear at or below 37. The band between them retains the rule's current state.

Persistence adds another condition. Suppose persistMs=10000 and the first accepted high sample ends at 1000ms. Accepted high samples through 11000ms can satisfy the 10s interval. An intervening below-trip sample resets pending persistence. An invalid or stale input also cancels pending persistence rather than counting as proof that the high condition continued.

Call shape from the rule path - names shortened for teaching

```
RuleDecision d = evaluateRule(rule, sample, now,
                              staleLimitMs, state);
if (d == RULE_DECISION_TRIP) {
    // Produce an event for Safety.
}
```

A latched interlock stays latched after the physical condition clears. CLEAR means the condition is no longer active under the rule. RESET is an explicit request to remove the retained latch, and Safety refuses that reset while the condition is still active.

Why separate them? Without a latch, a brief critical condition could disappear before an operator notices it. Without a clear state, a reset might pretend that an ongoing condition ended. The firmware carries active and latched masks separately.

The ordinary temperature rules in this source snapshot are disabled pending commissioning. The separate alarm_latency build enables a test rule. Do not read the existence of the alarm machinery as proof that production temperature limits are already active.

COMMON PITFALL

Changing a threshold constant does nothing if the rule remains disabled. Read both configuration and execution path.

Retain the trip before waking Safety

Original firmware - src/services/safety.c, lines 127-143

```
bool safetySubmitEvent(const AlarmEvent* e) {
  if (e == NULL || e->ruleId >= POLICY_RULE_COUNT) return false;
  if (e->decision != RULE_DECISION_TRIP && e->decision !=
>   RULE_DECISION_CLEAR) return false;
  StickySlot* slot = &s_sticky[e->ruleId];

  portENTER_CRITICAL(&s_stickyMux);
  stickyBegin(slot, &kPolicyRules[e->ruleId], e);  // in
>   flight: Safety leaves the slot alone
  portEXIT_CRITICAL(&s_stickyMux);

  const bool queued = s_detail != NULL && xQueueSend(s_detail,
>   e, 0) == pdTRUE;  // never inside the portMUX
  portENTER_CRITICAL(&s_stickyMux);
  stickyEnd(slot, queued);
  if (!queued) ++s_detailOverflow;
  portEXIT_CRITICAL(&s_stickyMux);
  if (s_task != NULL) xTaskNotifyGive(s_task);  // after
>   stickyEnd(): the next pass sees the outcome
  return queued;
}
```

First validate the event. Then stickyBegin records the in-flight event under a small lock. Next, xQueueSend attempts the detailed FIFO without waiting. stickyEnd records whether that attempt succeeded. Finally, the function notifies Safety.

REMEMBER

Store, finalise, then wake. A higher-priority Safety task may run immediately after the notification.

Retain the trip before waking Safety / continued

Why not just send the queue item? A full detail queue could lose a trip. Why not just keep the newest event? A later CLEAR could overwrite the trip before Safety sees it. The sticky state preserves the required trip information while the queue carries normal ordered detail.

The queue send happens outside the portMUX. Even a zero-wait kernel call does not belong inside this application critical section. Only the small sticky-state updates are protected there. Safety later drains the detail queue and collects the fallback information without processing a successfully queued record twice.

Trace the full-queue case: Producer begins retaining TRIP, the send fails, stickyEnd marks detail loss, and the wake makes Safety inspect the fallback. The caller receives false because detailed queuing failed; that is not the same as saying every trace of the trip vanished.

This is a direct application of the contracts you learned earlier: a bounded FIFO for events, retained state for information that must survive overflow, and a notification for prompt scheduling. Each mechanism has one defined job.

Keep hardware writes outside the decision core

safetyTask collects events and commands, peeks a Snapshot, gates its 5V readings against current rail validity, reads configuration and calls safetyCoreStep(). The core is pure C: its inputs include the time and records it needs. It does not call FreeRTOS or touch a GPIO.

Original firmware - src/services/safety.c, lines 245-254

```
    const SensorMask disabled = (SensorMask)(SENSOR_MASK_ALL &
>   (SensorMask)~s_cfg.sensorEnabled);
    const SafetyInputs in = {haveSnap ? &s_snap : NULL,
>   s_events, n, now, s_cmds, nCmd, disabled,
                           !rail.valid5v};
    safetyCoreStep(&s_core, &in, &out);

    const bool relayChanged = out.relayOn != relayOn;
    if (relayChanged) {
      relayWrite(out.relayOn);
      relayOn = out.relayOn;
    }
```

The returned out.relayOn is the final decision. Only this task calls relayWrite when that decision changes. A lamp desire cannot bypass a latched interlock, missing required input or invalid rail. This avoids two tasks fighting over the same output pin with unrelated policies.

Safety waits for an event notification with a 100ms timeout, then checks age when scheduled. Work and scheduler delays also contribute to the interval between passes. If acquisition stops publishing, no new message will announce that an old reading just became stale. The periodic pass gives Safety a way to discover that absence.

The light-fault counter advances only for a newly observed light result in the core's current implementation. It does not count ten wakes on the same failed reading as ten separate sensor failures. For light, the source recognises a new result by its end timestamp. The published sequence still serves other consumers.

A relay command is not proof of physical relay position. This board path has no contact feedback in the reference code. Your software test can prove the requested GPIO state; a bench test must establish that the intended load responds. Do not label the status field as measured feedback if it only mirrors the command.

TIP

Pure decisions are easy to host-test: supply an invented now and Snapshot, then inspect the returned decision.

COMMON PITFALL

Do not feed the task watchdog from another task on Safety's behalf. That would conceal a stalled output owner.

Describe a rule without executing it

An alarm rule is data plus retained evaluation state, not a task of its own. The event it produces carries the exact observation that caused a transition.

Acquisition evaluates rules; Safety owns latching and final actuator policy.

src/domain/alarm.h / lines 1-28

```
// Alarm rules and events (§10). Pure C.
// Work package WP7 (Stage 4). Thresholds are NOT commissioned: the 40/50 degC
// values in the original request are examples only (§1).
#pragma once
#include <stdbool.h>
#include <stdint.h>
#include "domain/reading.h"

typedef uint8_t Severity;
enum { SEVERITY_INFO = 0, SEVERITY_WARNING, SEVERITY_CRITICAL };

typedef uint8_t Comparison;
enum { COMPARISON_ABOVE = 0, COMPARISON_BELOW };

typedef uint8_t ActuatorAction;
enum { ACTUATOR_ACTION_NONE = 0, ACTUATOR_ACTION_DEENERGIZE, ACTUATOR_ACTION_ENERGIZE };

// Actuator ids used by AlarmRule.actuatorId (board has one relay, §20).
enum { ACTUATOR_NONE = 0, ACTUATOR_RELAY1 = 1 };

typedef struct AlarmRule {
  uint8_t ruleId;         // index of the rule in its table
  bool enabled;           // false until the limit is commissioned
  MetricId metric;
  Comparison comparison;
  float tripThreshold;
  float clearThreshold;   // hysteresis
  uint32_t persistMs;     // 0 = trip on one valid sample
```

Severity, comparison and actuator-action names are compact constants. AlarmRule groups metric identity, thresholds, persistence, enable state and intended output action. Constructing such a record does not turn the rule on unless enabled is true and a caller evaluates it.

Describe a rule without executing it / continued

src/domain/alarm.h / lines 29-36

```
  Severity severity;
  bool latching;          // requires explicit reset after clearing
  uint8_t actuatorId;
  ActuatorAction action;
} AlarmRule;

typedef uint8_t RuleDecision;
enum {
```

src/domain/alarm.h / lines 37-43

```
  RULE_DECISION_NOT_ELIGIBLE = 0,
  RULE_DECISION_NO_CHANGE,
  RULE_DECISION_TRIP,
  RULE_DECISION_CLEAR
};

// Event submitted by acquisition to Safety immediately after a sample (§4).
```

Trip and clear thresholds define hysteresis. Latching records a further policy: Safety can retain a trip after the active condition clears. It is not implemented by moving the clear threshold.

Keep evidence, evaluation history and current time explicit

src/domain/alarm.h / lines 44-71

```
typedef struct AlarmEvent {
  uint8_t ruleId;
  RuleDecision decision;
  Reading sample;
} AlarmEvent;

// Per-rule evaluation state, owned by the caller (acquisition, one per rule).
typedef struct AlarmRuleState {
  bool tripped;
  bool pending;           // beyond the trip threshold, persistMs not yet elapsed
  MonoMs pendingSinceMs;
} AlarmRuleState;

// Freshness limit for a metric (§10): BME280 metrics 1 s, all others 3 s.
// Used by acquisition when it evaluates rules and by Safety's stale checks.
uint32_t alarmStaleLimitMs(MetricId metric);

// Pure threshold evaluation for one sample. Only `state` is modified.
// A sample counts only if readingIsActionableAt(sample, now, staleLimitMs):
// a stale or non-OK reading is NOT_ELIGIBLE, neither trips nor clears, and
// restarts any persistence window (a gap in valid data is not "sustained").
//
// ABOVE: trip when value >= trip for persistMs (0 = this sample), clear when
// value <= clear. BELOW mirrors it. trip/clear must leave a hysteresis band
// (ABOVE: clear < trip). Persistence is timed on the samples' intervalEndMs.
// Latching is Safety's job: CLEAR is reported even for a latching rule.
RuleDecision evaluateRule(const AlarmRule* rule, const Reading* sample, MonoMs now,
                          uint32_t staleLimitMs, AlarmRuleState* state);
```

AlarmEvent contains a Reading copy. AlarmRuleState remembers pending persistence and tripped state for one rule. evaluateRule receives both the current time and stale limit and modifies only the supplied state.

The contract times persistence from sample end timestamps. A gap of invalid observations resets pending persistence; it is not evidence that the physical condition remained beyond threshold during an unobserved interval.

Use a metric-specific age policy and mirrored comparisons

Follow the pure threshold algorithm with a high-temperature example. You can test every branch without a sensor or an RTOS task.

This function assumes the rule configuration was validated by its owner.

src/domain/alarm.c / lines 1-22

```c
#include "domain/alarm.h"

#include "app_config.h"

uint32_t alarmStaleLimitMs(MetricId metric) {
  switch (metric) {
    case METRIC_AIR_TEMPERATURE:
    case METRIC_AIR_HUMIDITY:
    case METRIC_AIR_PRESSURE: return CFG_STALE_BME280_MS;
    default: return CFG_STALE_ORDINARY_MS;
  }
}

static bool beyondTrip(const AlarmRule* rule, float v) {
  return rule->comparison == COMPARISON_ABOVE ? v >= rule->tripThreshold
                                              : v <= rule->tripThreshold;
}

static bool pastClear(const AlarmRule* rule, float v) {
  return rule->comparison == COMPARISON_ABOVE ? v <= rule->clearThreshold
                                              : v >= rule->clearThreshold;
}
```

Climate metrics get the shorter stale limit. beyondTrip uses >= for above and <= for below; pastClear reverses that direction at the recovery threshold.

The ternary expressions choose comparison orientation, not a remembered state. The stateful hysteresis comes from evaluating the clear condition only after state->tripped is true.

Accumulate persistence only across eligible observations

src/domain/alarm.c / lines 23-46

```
23
24 RuleDecision evaluateRule(const AlarmRule* rule, const Reading* sample, MonoMs now,
25                           uint32_t staleLimitMs, AlarmRuleState* state) {
26   if (!readingIsActionableAt(sample, now, staleLimitMs)) {
27     state->pending = false;
28     return RULE_DECISION_NOT_ELIGIBLE;
29   }
30   const float v = sample->value;
31   const MonoMs t = sample->intervalEndMs;
32
33   if (state->tripped) {
34     if (!pastClear(rule, v)) return RULE_DECISION_NO_CHANGE;
35     state->tripped = false;
36     return RULE_DECISION_CLEAR;
37   }
38
39   if (!beyondTrip(rule, v)) {
40     state->pending = false;
41     return RULE_DECISION_NO_CHANGE;
42   }
43   if (!state->pending) {
44     state->pending = true;
45     state->pendingSinceMs = t;
46   }
```

The first guard rejects a stale or non-OK observation and resets pending. If already tripped, only the clear condition changes the state. Otherwise a non-trip value cancels pending; a first beyond-trip sample stores its end timestamp.

src/domain/alarm.c / lines 47-51

```
47   if (t - state->pendingSinceMs < rule->persistMs) return RULE_DECISION_NO_CHANGE;
48   state->pending = false;
49   state->tripped = true;
50   return RULE_DECISION_TRIP;
51 }
```

The elapsed comparison decides whether persistence is satisfied. Then pending clears and tripped becomes true before TRIP returns. The caller must retain a distinct AlarmRuleState for each rule; sharing one would mix their histories.

Give the lamp a raw electrical control quantity

This file supplies concrete project policy values. It is useful precisely because the mechanism and its selected numbers are separate.

Read enabled flags along with thresholds before claiming a protection path is active.

include/policy_config.h / lines 1-26

```
1  // Control policy: grow-lamp thresholds and alarm rules (Stage 4, WP7).
2  // NOT a frozen contract: these are placeholders to be replaced by measured /
3  // commissioned values. Pure C (no Arduino headers).
4  //
5  // Lamp: values from the test_lamp bench (README "Lamp relay bring-up"). Set
6  // POLICY_LAMP_OFF_ABOVE_MV above the "night + lamp ON" reading measured at the
7  // final sensor position, or the lamp will switch itself off.
8  #pragma once
9  #include <stdbool.h>
10 #include <stdint.h>
11
12 #include "app_config.h"
13 #include "domain/alarm.h"
14
15 // 1 = Safety drives the relay from the lamp decision. 0 = Safety still
16 // computes and reports the decision but keeps the relay de-energised.
17 #define POLICY_LAMP_AUTO_ENABLED 1
18
19 #define POLICY_LAMP_ON_BELOW_MV 40.0f
20 #define POLICY_LAMP_OFF_ABOVE_MV 120.0f
21 #define POLICY_LAMP_MIN_ON_MS ((uint32_t)30000)   // bench; minutes in real use
22 #define POLICY_LAMP_MIN_OFF_MS ((uint32_t)30000)
23 #define POLICY_LAMP_FAULTS_TO_OFF ((uint8_t)5)
24 // Light results older than this force the lamp OFF (§10 ordinary limit).
25 #define POLICY_LIGHT_STALE_MS CFG_STALE_ORDINARY_MS
26
```

The lamp thresholds use pin millivolts and the minimum dwell uses milliseconds. The nominal pH fit remains a provisional starting point. Fit plausibility bounds constrain later commands but do not substitute for reference measurements.

Give the lamp a raw electrical control quantity / continued

include/policy_config.h / lines 27-36

```
27 // Analog calibration used by production acquisition (README "pH-4502C
28 // calibration", "SEN0244 TDS bring-up"). Nominal values: pH and TDS stay
29 // HEALTH_UNCALIBRATED until buffer / reference-solution fits replace them.
30 #define POLICY_PH_NOMINAL_MV_AT_PH7 2758.0f    // BNC shorted, 17 Oct 2026
31 #define POLICY_PH_NOMINAL_PH_PER_MV (-0.0057f)  // tutorial sketch slope
32
33 // On-device calibration (Calibration page) plausibility limits. A fit outside
34 // them is refused ("bad_fit"), never stored.
35 #define POLICY_PH_SLOPE_MIN_ABS 0.002f   // pH per module mV (nominal 0.0057)
36 #define POLICY_PH_SLOPE_MAX_ABS 0.02f
```

include/policy_config.h / lines 37-40

```
37 #define POLICY_TDS_K_MIN 0.5f            // K = reference ppm / nominal ppm
38 #define POLICY_TDS_K_MAX 2.0f
39 #define POLICY_LIGHT_LUX_PER_MV_MIN 0.01f
40 #define POLICY_LIGHT_LUX_PER_MV_MAX 100.0f
```

The calibration age limit allows a recent retained observation to be captured without creating another ADC owner. It is a maximum accepted age, not a promise that every command sees a sample exactly that old.

Choose bounded recovery and an explicit transition policy

include/policy_config.h / lines 41-60

```
41 // cal_capture uses a reading at most this old (2 frames + slack).
42 #define POLICY_CAL_CAPTURE_MAX_AGE_MS ((uint32_t)5000)
43
44 // WP9 low-power mode (services/power_coordinator.h). 1 = losing the external
45 // 5 V moves sensing to the low-power worker (3.3 V sensors only) and switches
46 // Wi-Fi off. 0 = kill switch: report the sense level only, change nothing.
47 // Not hardware-tested yet (written before the circuit existed): if a power
48 // transition misbehaves on the bench, set 0 and report it.
49 #define POLICY_POWER_TRANSITIONS_ENABLED 1
50
51 // Retry interval for a driver whose init() failed (§18: bounded, no loop).
52 #define POLICY_INIT_RETRY_MS ((uint32_t)5000)
53
54 // Alarm rules. ruleId = index. All DISABLED: no trip/clear limit has been
55 // commissioned (§20). Enable a rule only with measured, agreed limits.
56 #define POLICY_RULE_COUNT 2
57 static const AlarmRule kPolicyRules[POLICY_RULE_COUNT] = {
58 #ifdef FW_ALARM_LATENCY_TEST
59     // Test build only (env:alarm_latency): trips on every valid air sample,
60     // non-latching; acquisition re-arms it after each TRIP (§19 latency test).
```

The power-transition enable flag selects whether the coordinator changes operational mode. The init retry interval prevents immediate repeated setup work against a missing device.

These are source choices with consequences in their callers. A disabled transition policy changes which acquisition permissions and rail interpretation apply; changing the define alone does not redesign the physical UPS circuit.

Distinguish disabled production examples from a test build

include/policy_config.h / lines 61-77

```
    {.ruleId = 0, .enabled = true, .metric = METRIC_AIR_TEMPERATURE,
     .comparison = COMPARISON_ABOVE, .tripThreshold = -100.0f, .clearThreshold = -200.0f,
     .persistMs = 0, .severity = SEVERITY_CRITICAL, .latching = false,
     .actuatorId = ACTUATOR_RELAY1, .action = ACTUATOR_ACTION_DEENERGIZE},
#else
    // Example from §12: air over-temperature, latched, relay de-energised.
    {.ruleId = 0, .enabled = false, .metric = METRIC_AIR_TEMPERATURE,
     .comparison = COMPARISON_ABOVE, .tripThreshold = 40.0f, .clearThreshold = 37.0f,
     .persistMs = 0, .severity = SEVERITY_CRITICAL, .latching = true,
     .actuatorId = ACTUATOR_RELAY1, .action = ACTUATOR_ACTION_DEENERGIZE},
#endif
    // Water temperature high, warning only.
    {.ruleId = 1, .enabled = false, .metric = METRIC_WATER_TEMPERATURE,
     .comparison = COMPARISON_ABOVE, .tripThreshold = 30.0f, .clearThreshold = 28.0f,
     .persistMs = 10000, .severity = SEVERITY_WARNING, .latching = false,
     .actuatorId = ACTUATOR_NONE, .action = ACTUATOR_ACTION_NONE},
};
```

The ordinary temperature rules are present but disabled in this snapshot. Under FW_ALARM_LATENCY_TEST, the first rule is replaced with a deliberately easy-to-trigger test rule.

The water rule also specifies persistence and a warning-only action. The table therefore combines comparison behaviour with what Safety should do after a transition. Review every field, not only the tripThreshold number.

COMMON PITFALL

A test rule selected at compile time is not a commissioned plant-protection setting.

Retain enough to survive the send window

This fallback preserves a trip even when the bounded detail queue cannot accept its record. It is a protocol shared by producer and consumer, with locking supplied by Safety's service wrapper.

The pure helper itself contains no mutex or queue operation.

src/domain/alarm_sticky.h / lines 1-28

```
// Sticky per-rule fallback for the alarm detail queue (§11). Pure C, no
// locking: services/safety.c wraps every call in its portMUX and does the
// queue send between stickyBegin() and stickyEnd().
//
// Producer (acquisition), per event:
//   stickyBegin()   record the decision, mark the slot in flight
//   xQueueSend()    outside the critical section
//   stickyEnd()     mark the record lost if the send failed, end the flight
// Consumer (Safety), per pass: drain the queue, then stickyCollect().
//
// A slot in flight is left untouched by stickyCollect(): its send result is
// not known yet, so its flags must survive until stickyEnd(). The producer
// wakes Safety after stickyEnd(), so the next pass sees the outcome. Without
// this, a drain between a failed send and the "lost" mark erased the TRIP
// from both delivery paths.
#pragma once
#include <stdbool.h>
#include <stdint.h>

#include "domain/alarm.h"

typedef struct StickySlot {
  bool tripSinceDrain;  // a TRIP was submitted since Safety last collected
  bool lost;            // a detail record of this rule was dropped
  uint8_t inFlight;     // submits between stickyBegin() and stickyEnd()
  RuleDecision last;    // last decision submitted (TRIP or CLEAR)
  AlarmEvent trip;      // worst TRIP since the last collect
  AlarmEvent clear;     // the last CLEAR
```

StickySlot remembers whether a trip occurred, whether detail was lost, which submissions are still in flight, the last decision and copies of the important trip/clear evidence.

src/domain/alarm_sticky.h / lines 29-30

```
} StickySlot;

```

inFlight is necessary because the producer temporarily leaves the lock to call the queue API. A consumer visiting during that interval cannot yet know whether the detail record will be available from the queue.

Collect only after the outcome is known

src/domain/alarm_sticky.h / lines 31-38

```
31 void stickyBegin(StickySlot* slot, const AlarmRule* rule, const AlarmEvent* e);
32 void stickyEnd(StickySlot* slot, bool queued);
33
34 // After the queue was drained: for each slot (not in flight) whose detail
35 // record was dropped, append the TRIP if none came through the queue
36 // (sawTrip[rule]) and, if the rule's last decision was CLEAR, that CLEAR
37 // too. Writes at most 2 * count events at out[0..]; returns how many.
38 uint8_t stickyCollect(StickySlot* slots, uint8_t count, const bool* sawTrip, AlarmEvent*
 >     out);
```

stickyBegin precedes the send and stickyEnd records its result. Collect uses sawTrip to avoid replaying a trip already received through the detail queue. It can add a retained trip and a final clear when needed.

The output capacity must allow up to twice the rule count. That bound is part of the caller's buffer sizing, not dynamically enforced by an array object inside this helper.

Keep the worst trip and record the send outcome

Trace a full detail queue: begin TRIP, send fails, end marks loss, then collect supplies the retained evidence.

Also trace the successful-send case to see why the same event is not processed twice.

src/domain/alarm_sticky.c / lines 1-21

```c
#include "domain/alarm_sticky.h"

static bool moreExtreme(const AlarmRule* rule, float a,
  float b) {
  return rule->comparison == COMPARISON_ABOVE ? a > b : a <
  b;
}

void stickyBegin(StickySlot* slot, const AlarmRule* rule,
  const AlarmEvent* e) {
  slot->last = e->decision;
  if (e->decision == RULE_DECISION_TRIP) {
    if (!slot->tripSinceDrain || moreExtreme(rule,
  e->sample.value, slot->trip.sample.value)) slot->trip =
  *e;
    slot->tripSinceDrain = true;
  } else {
    slot->clear = *e;
  }
  if (slot->inFlight < UINT8_MAX) ++slot->inFlight;
}

void stickyEnd(StickySlot* slot, bool queued) {
  if (!queued) slot->lost = true;
  if (slot->inFlight > 0) --slot->inFlight;
}
```

moreExtreme follows the rule direction. Begin stores the latest decision, replaces trip evidence when more extreme and records a clear separately. The bounded in-flight count prevents wrapping into a misleading zero.

End marks lost on a failed queue attempt, then reduces the active-submission count. Waking Safety must happen after this step so the next pass can see the final outcome.

COMMON PITFALL

The helpers assume the surrounding producer/consumer protocol and lock discipline. They do not make an ordinary struct safe for concurrent calls by themselves.

Replay only the information missing from detail delivery

src/domain/alarm_sticky.c / lines 22-36

```
22
23 uint8_t stickyCollect(StickySlot* slots, uint8_t count, const bool* sawTrip, AlarmEvent*
 >    out) {
24   uint8_t n = 0;
25   for (uint8_t r = 0; r < count; ++r) {
26     StickySlot* slot = &slots[r];
27     if (slot->inFlight > 0) continue;  // outcome unknown: keep everything for the next
 >   pass
28     if (slot->lost) {
29       if (slot->tripSinceDrain && !sawTrip[r]) out[n++] = slot->trip;
30       if (slot->last == RULE_DECISION_CLEAR) out[n++] = slot->clear;
31     }
32     slot->tripSinceDrain = false;
33     slot->lost = false;
34   }
35   return n;
36 }
```

Collect skips an in-flight slot entirely. For a completed lost-detail slot, it appends a trip only when no matching trip arrived through the queue, then appends the last clear if appropriate.

After collection, it clears the retained bookkeeping. This deliberately preserves the critical transition information, not an unlimited chronological history of every event. Detailed history belongs to the bounded queue and journal contracts.

Give each reader its own next sequence

A retained event journal serves independent communication readers. It solves a different problem from immediate alarm delivery to Safety.

A slow reader cannot stop Safety appending a new event.

src/domain/event_journal.h / lines 1-28

```
// Alarm event journal with independent reader cursors (§11, RTOS §14.1).
// Pure C, no locking: services/safety.c wraps writes and reads in a portMUX.
// Work package WP8 (Stage 5).
//
// Safety appends; BLE and LAN each keep their own cursor, so one slow or
// disconnected link never holds events back from the other. When a reader
// falls more than EVENT_JOURNAL_LEN events behind, the oldest details are
// lost and the next read reports a gap (drop oldest, with gap indication).
#pragma once
#include <stdbool.h>
#include <stdint.h>

#include "domain/alarm.h"

#define EVENT_JOURNAL_LEN 64  // = CFG_ALARM_JOURNAL_LEN (checked in services/safety.c)

typedef uint8_t JournalKind;
enum { JOURNAL_KIND_TRIP = 0, JOURNAL_KIND_CLEAR, JOURNAL_KIND_RESET };

typedef struct JournalEvent {
  uint32_t n;          // 1, 2, 3, ... assigned on append
  uint8_t ruleId;
  Severity severity;
  JournalKind kind;
  MonoMs atMs;         // sample end time (trip/clear) or reset time
  float value;
} JournalEvent;

```

COMMON PITFALL

The implementation uses 32-bit journal sequence arithmetic. Very long-run wrap behaviour needs review if uninterrupted lifetime approaches that counter range.

Give each reader its own next sequence / continued

JournalEvent contains an assigned event number, rule, severity, kind, timestamp and value. EventJournal owns 64 slots and the next event number. Each JournalCursor records which sequence one reader wants next.

src/domain/event_journal.h / lines 29-51

```
29 typedef struct EventJournal {
30   JournalEvent ring[EVENT_JOURNAL_LEN];
31   uint32_t nextN;      // n the next append gets
32 } EventJournal;
33
34 typedef struct JournalCursor {
35   uint32_t nextN;      // next event this reader wants
36 } JournalCursor;
37
38 void journalInit(EventJournal* j);
39
40 // Stores a copy of *e with a new n; returns that n.
41 uint32_t journalAppend(EventJournal* j, const JournalEvent* e);
42
43 // A new reader starts at the next event (it does not replay history).
44 void journalCursorInit(const EventJournal* j, JournalCursor* c);
45
46 // Next event for this reader, false if none. *gap is true when events were
47 // lost before this one (the cursor jumps to the oldest one still stored).
48 bool journalNext(const EventJournal* j, JournalCursor* c, JournalEvent* out, bool* gap);
49
50 const char* journalKindName(JournalKind k);
51 const char* severityName(Severity s);
```

A new cursor begins at future events rather than replaying all retained history. When a reader falls behind the available ring, Next reports a gap. This makes loss visible instead of silently presenting a later event as consecutive history.

Separate the event number from the array index

Read append and next as a bounded circular history. The service will supply locking around these ordinary C operations.

The modulo operation chooses storage; the event sequence supplies logical order.

src/domain/event_journal.c / lines 1-22

```c
#include "domain/event_journal.h"

#include <string.h>

void journalInit(EventJournal* j) {
  memset(j, 0, sizeof(*j));
  j->nextN = 1;
}

uint32_t journalAppend(EventJournal* j, const JournalEvent* e) {
  const uint32_t n = j->nextN++;
  JournalEvent* slot = &j->ring[n % EVENT_JOURNAL_LEN];
  *slot = *e;
  slot->n = n;
  return n;
}

void journalCursorInit(const EventJournal* j, JournalCursor* c) { c->nextN = j->nextN; }

static uint32_t oldestStored(const EventJournal* j) {
  return j->nextN > EVENT_JOURNAL_LEN ? j->nextN - EVENT_JOURNAL_LEN : 1;
}
```

Init starts numbering at one. Append obtains the next number, selects number modulo 64, copies the supplied event and overwrites its n field with the assigned number. CursorInit begins at the journal's current next number.

The oldest retained logical event advances after capacity is exceeded. Slot zero can contain a later event even though the array begins there; never infer chronology from physical slot order alone.

Advance one cursor without consuming another reader's event

src/domain/event_journal.c / lines 23-45

```
23
24 bool journalNext(const EventJournal* j, JournalCursor* c, JournalEvent* out, bool* gap)
 >   {
25   *gap = false;
26   if (c->nextN >= j->nextN) return false;
27   const uint32_t oldest = oldestStored(j);
28   if (c->nextN < oldest) {
29     *gap = true;
30     c->nextN = oldest;
31   }
32   *out = j->ring[c->nextN % EVENT_JOURNAL_LEN];
33   ++c->nextN;
34   return true;
35 }
36
37 const char* journalKindName(JournalKind k) {
38   switch (k) {
39     case JOURNAL_KIND_TRIP: return "trip";
40     case JOURNAL_KIND_CLEAR: return "clear";
41     case JOURNAL_KIND_RESET: return "reset";
42   }
43   return "?";
44 }
45
```

Next first reports no future item when the cursor has caught up. If its desired event predates retention, it jumps to oldestStored and sets gap. It copies one record and advances only this cursor.

src/domain/event_journal.c / lines 46-53

```
46 const char* severityName(Severity s) {
47   switch (s) {
48     case SEVERITY_INFO: return "INFO";
49     case SEVERITY_WARNING: return "WARNING";
50     case SEVERITY_CRITICAL: return "CRITICAL";
51   }
52   return "?";
53 }
```

The naming helpers then convert kind and severity to stable strings. No event is removed by a read, so BLE and another reader can progress independently. A service lock is still required around concurrent appends and copies.

Make the helper's missing age check explicit

The lamp model combines two voltage thresholds, dwell and a finite sequence of unusable readings. It decides a desired state without knowing a relay GPIO.

Production Safety adds freshness and interlocks around this helper.

src/domain/lamp_control.h / lines 1-25

```
// Grow-lamp decision from the DFR0026 light level (bring-up, ahead of WP7).
// Pure C, host-tested. Decides only: the caller drives the relay.
//
// Hysteresis: ON below onBelowMv, OFF above offAboveMv, unchanged in between.
// Dwell: after a change the state is held for minOnMs / minOffMs, which also
// covers boot (init counts as a change to OFF), so a reset loop cannot cycle
// the lamp. The sensor may see the lamp it controls: offAboveMv must sit above
// the "night + lamp ON" reading measured at the final sensor position.
//
// Readings: OK and UNCALIBRATED (the light driver publishes uncalibrated mV)
// are used; a saturated one counts as bright. Anything else holds the state,
// and faultsToOff such readings in a row force OFF at once (safe state: relay
// de-energised, lamp on NO contact).
//
// Acts on UNCALIBRATED mV by explicit decision (relative light level; no lux
// calibration, D13). It does not check reading age itself: in production
// domain/safety_core feeds it each new light result and calls
// lampControlForceOff() when the light data is stale (audit B1).
#pragma once
#include <stdbool.h>
#include <stdint.h>

#include "domain/reading.h"
#include "domain/types.h"

```

src/domain/lamp_control.h / lines 26-40

```
typedef struct LampControlConfig {
  float onBelowMv;     // must be < offAboveMv
  float offAboveMv;
  uint32_t minOnMs;
  uint32_t minOffMs;
  uint8_t faultsToOff;  // >= 1
} LampControlConfig;

typedef struct LampControl {
  LampControlConfig config;
  bool on;
  bool waiting;  // the level asks for a change the dwell is still holding back
  MonoMs lastChangeMs;
  uint8_t faults;  // unusable readings in a row
} LampControl;
```

Make the helper's missing age check explicit / continued

src/domain/lamp_control.h / lines 41-51

```
41
42 // Starts OFF, as if it had just switched OFF at nowMs.
43 void lampControlInit(LampControl* self, const LampControlConfig* config, MonoMs nowMs);
44
45 // Feed one light reading; returns the lamp state to drive.
46 bool lampControlUpdate(LampControl* self, const Reading* light, MonoMs nowMs);
47
48 // Switch OFF now, ignoring the dwell (stale light data, interlock: §12). The
49 // OFF dwell starts here, so the lamp cannot come straight back on.
50 void lampControlForceOff(LampControl* self, MonoMs nowMs);
51
```

LampControlConfig states band, dwell and fault count. LampControl retains current state, waiting condition, last change time and consecutive faults. Init treats boot as a transition to OFF.

src/domain/lamp_control.h / lines 52-55

```
52 // Time in the current state.
53 static inline uint32_t lampControlHeldMs(const LampControl* self, MonoMs nowMs) {
54   return (uint32_t)(nowMs - self->lastChangeMs);
55 }
```

Update accepts the project's explicit raw-mV uncalibrated light case. It does not independently check age. ForceOff bypasses normal dwell for a higher-level safety condition and establishes a new OFF dwell. The caller must feed new results and enforce freshness correctly.

Record a real transition, not every repeated request

Read each return as an answer to whether the requested state may change now.

Use the lesson's30,80 and 130mV path while watching on, waiting and lastChangeMs.

src/domain/lamp_control.c / lines 1-19

```
#include "domain/lamp_control.h"

static void setState(LampControl* self, bool on, MonoMs nowMs) {
  if (on != self->on) {
    self->on = on;
    self->lastChangeMs = nowMs;
  }
  self->waiting = false;
}

void lampControlInit(LampControl* self, const LampControlConfig* config, MonoMs nowMs) {
  self->config = *config;
  self->on = false;
  self->waiting = false;
  self->lastChangeMs = nowMs;
  self->faults = 0;
}

void lampControlForceOff(LampControl* self, MonoMs nowMs) { setState(self, false,
  nowMs); }
```

setState changes lastChangeMs only when on actually changes, then clears waiting. Init stores configuration, establishes OFF at now and clears fault history. ForceOff uses the same transition helper.

A repeated force-off while already OFF does not restart the timer in setState. Distinguish that exact behaviour from a vague statement that any OFF request always resets dwell.

Classify the observation before asking for a level change

src/domain/lamp_control.c / lines 20-39

```
20
21 bool lampControlUpdate(LampControl* self, const Reading* light, MonoMs nowMs) {
22   const bool usable = light->health == HEALTH_OK || light->health ==
 >   HEALTH_UNCALIBRATED;
23   if (!usable) {
24     if (self->faults < UINT8_MAX) ++self->faults;
25     if (self->faults >= self->config.faultsToOff) setState(self, false, nowMs);
26     return self->on;
27   }
28   self->faults = 0;
29
30   // Thresholds are pin mV. After an on-device lux fit the value is lux, but
31   // raw still carries the filtered pin mV: calibration never moves the lamp.
32   const float levelMv = light->unit == UNIT_MILLIVOLT || !light->hasRaw ? light->value :
 >    (float)light->raw;
33   const bool bright = (light->qualityFlags & kQualitySaturated) != 0;
34   bool want = self->on;
35   if (bright || levelMv > self->config.offAboveMv) {
36     want = false;
37   } else if (levelMv < self->config.onBelowMv) {
38     want = true;
39   }
```

Unusable health increments the bounded fault count and forces OFF only when the configured count is reached. Usable input resets that count. After a lux fit, raw pin millivolts preserve the lamp's electrical thresholds.

Saturation is treated as bright, asking for OFF. Inside the threshold band, want begins as the previous on state. Equal-to-threshold values also retain the prior desire because the comparisons here are strict.

Apply the dwell appropriate to the current state

src/domain/lamp_control.c / lines 40-52

```
  if (want == self->on) {
    self->waiting = false;
    return self->on;
  }

  const uint32_t dwellMs = self->on ? self->config.minOnMs : self->config.minOffMs;
  if (lampControlHeldMs(self, nowMs) >= dwellMs) {
    setState(self, want, nowMs);
  } else {
    self->waiting = true;
  }
  return self->on;
}
```

If want already equals on, waiting clears. Otherwise the helper chooses minimum-on while currently ON and minimum-off while currently OFF. It changes state only after that held interval has elapsed.

Waiting records that a level requested a deferred change. It is not a timer callback scheduled to switch automatically later; another suitable caller pass and current input are needed to reconsider the request.

Separate active, latched and explanatory state

This interface gathers every fact needed for a final decision into explicit inputs. Its body can then run on a PC with an invented now.

The service owns queues and GPIOs; the core owns policy state.

src/domain/safety_core.h / lines 1-27

```
// Safety decision logic, pure C (§12). Work package WP7 (Stage 4).
// services/safety.c only waits, copies inputs in, calls safetyCoreStep() and
// writes the relay; every decision is here so it is host-tested with a fake
// clock.
//
// One step = one Safety wake, in the §12 order:
//   1 power validity      (always valid until WP9)
//   2 alarm events        TRIP latches (latching rules) / sets active; CLEAR
//                         releases only non-latching rules
//   3 freshness           new light result -> lamp_control; light result
//                         older than lightStaleMs -> lamp forced OFF; stale
//                         inputs of enabled rules are reported
//   4 commands            WP8: latch resets, refused when expired or while
//                         the rule is still active (§12 step 4)
//   5 output              relay = auto enabled AND lamp ON AND no interlock
//                         AND every relay-interlock input available
// Interlock: any enabled rule with action DEENERGIZE on ACTUATOR_RELAY1 that
// is latched or active, or that tripped in this step's events (so a TRIP
// followed by a CLEAR in the same batch still de-energises once).
// Unavailable input (§12): an enabled relay-interlock rule whose input is in
// staleMask cannot watch the relay, so the relay is held OFF (reason
// INPUT_MISSING) until its data is fresh again. A rule whose sensor was
// switched off is not monitored (staleMask), unless it is still latched.
#pragma once
#include <stdbool.h>
#include <stdint.h>

```

src/domain/safety_core.h / lines 28-40

```
#include "domain/alarm.h"
#include "domain/lamp_control.h"
#include "domain/reading.h"
#include "domain/sensor_set.h"

#define SAFETY_MAX_RULES 8
#define SAFETY_MAX_COMMANDS 4

typedef uint8_t LampReason;
enum {
  LAMP_REASON_NO_DATA = 0,  // no light result yet: lamp stays OFF
  LAMP_REASON_LEVEL,        // following the light level (ON or OFF)
  LAMP_REASON_DWELL,        // level asks for a change, dwell still holding
```

Separate active, latched and explanatory state / continued

src/domain/safety_core.h / lines 41-54

```
  LAMP_REASON_FAULT,        // unusable light results
  LAMP_REASON_STALE,        // light result too old: forced OFF
  LAMP_REASON_INTERLOCK,    // alarm interlock: relay de-energised
  LAMP_REASON_DISABLED,     // POLICY_LAMP_AUTO_ENABLED = 0
  LAMP_REASON_UNPOWERED,    // WP9: external 5 V down, the relay coil has no supply
  LAMP_REASON_INPUT_MISSING // a relay-interlock rule has no fresh input: relay held OFF
};

typedef struct SafetyConfig {
  bool lampAutoEnabled;
  LampControlConfig lamp;
  uint32_t lightStaleMs;
} SafetyConfig;

```

SafetyRuleState remembers active condition, retained latch and trip evidence. LampReason explains a final output policy choice; it is not a command that changes hardware. SafetyConfig selects lamp behaviour and freshness.

src/domain/safety_core.h / lines 55-64

```
typedef struct SafetyRuleState {
  bool active;        // last decision was TRIP and no CLEAR since
  bool latched;       // latching rule tripped; only safetyCoreResetLatch() clears
  uint32_t trips;     // TRIP events seen
  float worst;        // most extreme tripping value
  MonoMs firstTripMs; // sample time of the first TRIP (0 = never)
} SafetyRuleState;

typedef struct SafetyCore {
  SafetyConfig config;
```

SAFETY_MAX_RULES bounds the internal arrays. The chosen rule count must stay within that bound. Compact masks refer to rule indices, while sensor-disabled masks refer to SensorIds; keep those index spaces separate.

Carry commands with an expiry and a correlation tag

src/domain/safety_core.h / lines 65-92

```
  const AlarmRule* rules;
  uint8_t ruleCount;
  SafetyRuleState rule[SAFETY_MAX_RULES];
  LampControl lamp;
  bool haveLight;
  MonoMs lastLightEndMs;  // intervalEndMs of the last light result fed to lamp
} SafetyCore;

typedef uint8_t SafetyCommandKind;
enum { SAFETY_CMD_RESET_LATCH = 0 };

// A bounded remote request (WP8). The transport sets the expiry; a command
// that waited too long is refused, never applied late (§11, §12).
typedef struct SafetyCommand {
  SafetyCommandKind kind;
  uint8_t ruleId;
  MonoMs expiresMs;
  uint32_t tag;               // caller's request handle, echoed in the result
} SafetyCommand;

typedef uint8_t SafetyCommandOutcome;
enum {
  SAFETY_CMD_APPLIED = 0,
  SAFETY_CMD_EXPIRED,
  SAFETY_CMD_STILL_ACTIVE,    // condition persists: no valid recovery yet
  SAFETY_CMD_BAD_RULE
};

```

SafetyCommand records the intended latch reset, rule index, latest acceptable time and caller tag. Its outcome can be applied, expired, still active or bad rule.

Carry commands with an expiry and a correlation tag / continued

src/domain/safety_core.h / lines 93-104

```
typedef struct SafetyCommandResult {
  uint32_t tag;
  SafetyCommandOutcome outcome;
} SafetyCommandResult;

typedef struct SafetyInputs {
  const Snapshot* snap;       // NULL until the first publish
  const AlarmEvent* events;   // drained this wake, in arrival order
  uint8_t eventCount;
  MonoMs now;
  // WP8. Zero-initialised inputs keep the WP7 behaviour.
  const SafetyCommand* commands;
```

The core retains lamp state and the last processed light end time. That lets it avoid treating repeated peeks as multiple new light failures. The command arrays in the input are borrowed for one step; the core does not store pointers to a transport's temporary request buffer.

Return a decision and explicit result records

src/domain/safety_core.h / lines 105-131

```
  uint8_t commandCount;        // at most SAFETY_MAX_COMMANDS are handled
  SensorMask sensorDisabled;   // bit per SensorId switched off by Configuration
  // WP9. The external 5 V rail is not valid (rail_state): the relay coil has
  // no supply, so the relay output is driven OFF (unavailable-actuator
  // policy, §13) and the lamp restarts from OFF, with its dwell, when it returns.
  bool railDown;
} SafetyInputs;

typedef struct SafetyOutputs {
  bool relayOn;          // what relay 1 must be
  bool lampWantsOn;      // lamp policy decision before interlock / enable
  LampReason lampReason;
  bool interlock;
  uint16_t latchedMask;  // bit per rule index
  uint16_t activeMask;
  uint16_t staleMask;    // enabled rules whose input is not fresh and OK
                         // (a disabled sensor's rule only while latched)
  SafetyCommandResult results[SAFETY_MAX_COMMANDS];
  uint8_t resultCount;
} SafetyOutputs;

// rules may be NULL with ruleCount 0. At most SAFETY_MAX_RULES are used.
void safetyCoreInit(SafetyCore* self, const SafetyConfig* config, const AlarmRule*
  rules,
                    uint8_t ruleCount, MonoMs now);

void safetyCoreStep(SafetyCore* self, const SafetyInputs* in, SafetyOutputs* out);

```

SafetyInputs includes optional Snapshot, events, commands, disabled sensors, now and railDown. SafetyOutputs gives requested relay state, masks, reason and bounded command outcomes.

Return a decision and explicit result records / continued

src/domain/safety_core.h / lines 132-142

```
132 // Explicit reset of a latched rule (WP8 command). Refused (false) while the
133 // rule is still active: clearing needs a valid recovery observation first.
134 bool safetyCoreResetLatch(SafetyCore* self, uint8_t ruleId);
135
136 // True if any enabled rule on one of this sensor's metrics is latched or
137 // active in the given masks. Configuration refuses to disable such a sensor:
138 // switching an input off must never silence an alarm (§12).
139 bool safetyRulesBlockSensor(const AlarmRule* rules, uint8_t ruleCount, uint16_t
  >   latchedMask,
140                             uint16_t activeMask, SensorId sensor);
141
142 const char* lampReasonName(LampReason r);
```

Step does not touch a pin. ResetLatch refuses an active rule. RulesBlockSensor lets Configuration refuse a request that would remove a currently active or latched alarm input. Those declarations connect policy without transferring state ownership.

Initialise policy before reading events

Follow the complete decision order: events, freshness, commands and final output gating. These functions explain why a desired lamp ON may still produce a relay OFF.

Keep one state object per running Safety instance.

src/domain/safety_core.c / lines 1-22

```c
#include "domain/safety_core.h"

#include <string.h>

static bool isRelayInterlock(const AlarmRule* rule) {
  return rule->enabled && rule->actuatorId == ACTUATOR_RELAY1 &&
         rule->action == ACTUATOR_ACTION_DEENERGIZE;
}

static bool moreExtreme(const AlarmRule* rule, float candidate, float current) {
  return rule->comparison == COMPARISON_ABOVE ? candidate > current : candidate <
  current;
}

void safetyCoreInit(SafetyCore* self, const SafetyConfig* config, const AlarmRule*
  rules,
                    uint8_t ruleCount, MonoMs now) {
  memset(self, 0, sizeof(*self));
  self->config = *config;
  self->rules = rules;
  self->ruleCount = ruleCount > SAFETY_MAX_RULES ? SAFETY_MAX_RULES : ruleCount;
  if (self->rules == NULL) self->ruleCount = 0;
  lampControlInit(&self->lamp, &config->lamp, now);  // boot counts as a switch to OFF
}
```

isRelayInterlock recognises enabled de-energise rules for the relay. moreExtreme follows comparison direction. Init clears retained state, copies configuration, clamps rule count and initializes the lamp OFF.

The rules pointer is borrowed constant configuration with a suitable lifetime. If it were a local array that vanished after init, the core would later dereference invalid storage. The production static rule table satisfies that requirement.

Retain a trip even if a clear follows in the same batch

src/domain/safety_core.c / lines 23-46

```
23
24 // Step 2. Returns true if a relay-interlock rule tripped in this batch.
25 static bool applyEvents(SafetyCore* self, const SafetyInputs* in) {
26   bool trippedNow = false;
27   for (uint8_t i = 0; i < in->eventCount; ++i) {
28     const AlarmEvent* e = &in->events[i];
29     if (e->ruleId >= self->ruleCount) continue;
30     const AlarmRule* rule = &self->rules[e->ruleId];
31     SafetyRuleState* st = &self->rule[e->ruleId];
32     if (e->decision == RULE_DECISION_TRIP) {
33       st->active = true;
34       if (rule->latching) st->latched = true;
35       if (st->trips == 0 || moreExtreme(rule, e->sample.value, st->worst)) {
36         st->worst = e->sample.value;
37       }
38       if (st->trips == 0) st->firstTripMs = e->sample.intervalEndMs;
39       if (st->trips < UINT32_MAX) ++st->trips;
40       if (isRelayInterlock(rule)) trippedNow = true;
41     } else if (e->decision == RULE_DECISION_CLEAR) {
42       st->active = false;  // a latched rule stays latched until reset
43     }
44   }
45   return trippedNow;
46 }
```

TRIP marks active, optionally latches, retains worst and first evidence and counts trips. CLEAR removes active but does not remove a latch. trippedNow separately records whether a relay interlock tripped anywhere in this batch.

That extra boolean ensures a TRIP followed by CLEAR still de-energises for this step. Looking only at the final active bit would lose the transient condition before the output calculation.

Check age even when no new event arrived

src/domain/safety_core.c / lines 47-74

```
47
48 // Step 3 (lamp). Feeds lamp_control once per new light result, forces OFF on
49 // stale data even when no new result arrives.
50 static LampReason updateLamp(SafetyCore* self, const SafetyInputs* in) {
51   if (in->snap == NULL) return LAMP_REASON_NO_DATA;
52   const Reading* light = &in->snap->readings[METRIC_LIGHT_LEVEL];
53   if (light->health == HEALTH_STARTING || light->intervalEndMs == 0) {
54     if (!self->haveLight) return LAMP_REASON_NO_DATA;  // lamp is OFF since boot
55     if (self->lamp.on) lampControlForceOff(&self->lamp, in->now);
56     return LAMP_REASON_STALE;
57   }
58
59   // Age from intervalEndMs: readingHealthAt() does not age UNCALIBRATED
60   // readings, and the light driver publishes UNCALIBRATED mV.
61   const MonoMs end = light->intervalEndMs;
62   if (in->now > end && in->now - end > self->config.lightStaleMs) {
63     if (self->lamp.on) lampControlForceOff(&self->lamp, in->now);
64     return LAMP_REASON_STALE;
65   }
66
67   if (!self->haveLight || end != self->lastLightEndMs) {
68     self->haveLight = true;
69     self->lastLightEndMs = end;
70     lampControlUpdate(&self->lamp, light, in->now);
71   }
72   if (self->lamp.faults > 0) return LAMP_REASON_FAULT;
73   return self->lamp.waiting ? LAMP_REASON_DWELL : LAMP_REASON_LEVEL;
74 }
```

The helper handles no Snapshot, never-measured light and an over-age interval before applying ordinary level logic. It directly ages the light record because uncalibrated mV is a permitted lamp input and the general age helper ages only stored OK.

A new end timestamp feeds Update once; repeated wakes reuse its state without incrementing the fault count again. The returned reason explains stale, fault, dwell or level. It does not prove a relay contact moved.

Keep missing interlock evidence visible

src/domain/safety_core.c / lines 75-94

```
75
76 static uint16_t staleRules(const SafetyCore* self, const SafetyInputs* in) {
77   uint16_t mask = 0;
78   for (uint8_t i = 0; i < self->ruleCount; ++i) {
79     const AlarmRule* rule = &self->rules[i];
80     if (!rule->enabled) continue;
81     // A switched-off input is not monitored, unless its rule is still latched:
82     // then the missing data stays visible until the sensor is back.
83     if (rule->metric < METRIC_COUNT && sensorMaskHas(in->sensorDisabled,
 >   sensorOfMetric(rule->metric)) &&
84         !self->rule[i].latched) {
85       continue;
86     }
87     const uint32_t limit = alarmStaleLimitMs(rule->metric);
88     if (in->snap == NULL || rule->metric >= METRIC_COUNT ||
89         !readingIsActionableAt(&in->snap->readings[rule->metric], in->now, limit)) {
90       mask = (uint16_t)(mask | (1u << i));
91     }
92   }
93   return mask;
94 }
```

For each enabled rule, the helper checks whether its sensor is intentionally disabled and whether a latch still requires visibility. Otherwise it uses the appropriate stale limit and effective health.

The resulting rule-index mask says which monitored inputs are unavailable. A missing input for a relay interlock later holds the output OFF rather than assuming the physical condition cleared while unseen.

Refuse a late reset or an active condition

src/domain/safety_core.c / lines 95-111

```
95
96 static void applyCommands(SafetyCore* self, const SafetyInputs* in, SafetyOutputs* out)
 >   {
97   if (in->commands == NULL) return;
98   const uint8_t n = in->commandCount > SAFETY_MAX_COMMANDS ? SAFETY_MAX_COMMANDS :
 >   in->commandCount;
99   for (uint8_t i = 0; i < n; ++i) {
100    const SafetyCommand* c = &in->commands[i];
101    SafetyCommandResult* r = &out->results[out->resultCount++];
102    r->tag = c->tag;
103    if (in->now > c->expiresMs) {
104      r->outcome = SAFETY_CMD_EXPIRED;
105    } else if (c->kind != SAFETY_CMD_RESET_LATCH || c->ruleId >= self->ruleCount) {
106      r->outcome = SAFETY_CMD_BAD_RULE;
107    } else {
108      r->outcome = safetyCoreResetLatch(self, c->ruleId) ? SAFETY_CMD_APPLIED :
 >   SAFETY_CMD_STILL_ACTIVE;
109    }
110  }
111 }
```

The command count is bounded by the core capacity. Each result echoes its tag, then follows expiry, kind/rule validation and latch-reset logic. A still-active rule refuses reset.

A result of expired is not a failure to dequeue; the command reached the owner but was no longer permitted to apply. This is why external request acceptance and completed application are different milestones.

Build final interlock and missing-input masks

src/domain/safety_core.c / lines 112-139

```
112
113 void safetyCoreStep(SafetyCore* self, const SafetyInputs* in, SafetyOutputs* out) {
114   memset(out, 0, sizeof(*out));
115
116   // 1 Power validity (WP9): 5 V readings are already gated on the rail epoch
117   // by acquisition (published UNPOWERED, never actionable); here only the
118   // actuator on that rail is handled (step 5).
119
120   // 2 Alarm events.
121   const bool trippedNow = applyEvents(self, in);
122
123   // 3 Freshness.
124   LampReason reason = updateLamp(self, in);
125   out->staleMask = staleRules(self, in);
126
127   // 4 Commands.
128   applyCommands(self, in, out);
129
130   // 5 Output.
131   bool interlock = trippedNow;
132   bool inputMissing = false;
133   for (uint8_t i = 0; i < self->ruleCount; ++i) {
134     const SafetyRuleState* st = &self->rule[i];
135     if (st->latched) out->latchedMask = (uint16_t)(out->latchedMask | (1u << i));
136     if (st->active) out->activeMask = (uint16_t)(out->activeMask | (1u << i));
137     if (!isRelayInterlock(&self->rules[i])) continue;
138     if (st->latched || st->active) interlock = true;
139     if ((out->staleMask & (1u << i)) != 0) inputMissing = true;  // the rule cannot see
  >   a fault now
```

Step clears outputs, applies events, updates lamp/freshness and handles commands. It then scans retained rule state to assemble active and latched masks and decide whether an enabled relay interlock has an unavailable input.

The ordering is intentional. A reset command sees the active/clear state produced by the current event batch, and final output considers both newly tripped and retained interlocks.

Let higher-level rejection override ordinary lamp demand

src/domain/safety_core.c / lines 140-157

```
  }
  const bool hold = interlock || inputMissing || in->railDown;
  if (hold && self->lamp.on) lampControlForceOff(&self->lamp, in->now);

  out->interlock = interlock;
  out->lampWantsOn = self->lamp.on;
  if (interlock) {
    reason = LAMP_REASON_INTERLOCK;
  } else if (inputMissing) {
    reason = LAMP_REASON_INPUT_MISSING;
  } else if (in->railDown) {
    reason = LAMP_REASON_UNPOWERED;
  } else if (!self->config.lampAutoEnabled) {
    reason = LAMP_REASON_DISABLED;
  }
  out->lampReason = reason;
  out->relayOn = self->config.lampAutoEnabled && !hold && self->lamp.on;
}
```

hold combines interlock, missing required input and railDown. A hold forces the internal lamp state OFF so a later release respects its dwell. Reason priority then explains interlock, missing input, unpowered or disabled operation.

relayOn requires auto enable, no hold and the resulting lamp state. The returned expression is the one final decision the service may apply. No lower-level driver gets a bypass around it.

Permit recovery only after the necessary condition clears

src/domain/safety_core.c / lines 158-176

```
158
159 bool safetyCoreResetLatch(SafetyCore* self, uint8_t ruleId) {
160   if (ruleId >= self->ruleCount) return false;
161   SafetyRuleState* st = &self->rule[ruleId];
162   if (st->active) return false;
163   st->latched = false;
164   return true;
165 }
166
167 bool safetyRulesBlockSensor(const AlarmRule* rules, uint8_t ruleCount, uint16_t
  >   latchedMask,
168                             uint16_t activeMask, SensorId sensor) {
169   if (rules == NULL) return false;
170   const uint16_t held = (uint16_t)(latchedMask | activeMask);
171   for (uint8_t i = 0; i < ruleCount && i < 16; ++i) {
172     if (!rules[i].enabled || (held & (1u << i)) == 0) continue;
173     if (rules[i].metric < METRIC_COUNT && sensorOfMetric(rules[i].metric) == sensor)
  >   return true;
174   }
175   return false;
176 }
```

ResetLatch validates the rule index and refuses while active. RulesBlockSensor checks enabled rules whose metric belongs to the requested sensor and whose active/latched mask is held.

Configuration can use that answer before disabling an input. It is a copied view of current status, so the service still needs the defined race policy for a trip arriving concurrently with the request.

Name the reason without changing the decision

src/domain/safety_core.c / lines 177-191

```
177
178 const char* lampReasonName(LampReason r) {
179   switch (r) {
180     case LAMP_REASON_NO_DATA: return "no data";
181     case LAMP_REASON_LEVEL: return "level";
182     case LAMP_REASON_DWELL: return "dwell";
183     case LAMP_REASON_FAULT: return "fault";
184     case LAMP_REASON_STALE: return "stale";
185     case LAMP_REASON_INTERLOCK: return "interlock";
186     case LAMP_REASON_DISABLED: return "disabled";
187     case LAMP_REASON_UNPOWERED: return "no 5 V";
188     case LAMP_REASON_INPUT_MISSING: return "input missing";
189   }
190   return "?";
191 }
```

lampReasonName maps every declared reason to a diagnostic string. It makes a status understandable without asking the output task to duplicate policy comparisons.

If an app shows relay OFF, this reason distinguishes normal bright-light operation from a stale observation or interlock. A single boolean would conceal those different recovery requirements.

Establish output defaults before ownership begins

Before any task starts, outputs need declared initial levels. This tiny interface makes that boot action visible to setup.

It does not start Safety or enable automatic lamp control.

src/hal/gpio_safe.h / lines 1-5

```
1 // Boot-time safe output defaults (§5 "thin main", §10 actuator policy).
2 // Called first in setup(), before any network or sensor start.
3 #pragma once
4
5 void gpioApplySafeDefaults(void);
```

gpioApplySafeDefaults is called once at the beginning of production setup. Its responsibility is initial pin state; the ongoing output owner is Safety.

Keeping that transition explicit avoids several init functions independently guessing a relay level while other tasks may already be running.

Write the inactive latch before enabling the driver

The pin latch is written before output mode is enabled. That ordering reduces the chance of an unintended active pulse during setup.

The board profile determines actual assignment and polarity.

src/hal/gpio_safe.c / lines 1-25

```
#include "hal/gpio_safe.h"

#include <Arduino.h>

#include "board_profile.h"

// Level first, then output: the pin never drives the active level, even for
// the instant between the two calls.
static void driveInactive(int8_t pin, bool activeLow) {
  if (!boardPinAssigned(pin)) return;
  const uint8_t level = activeLow ? HIGH : LOW;
  digitalWrite(pin, level);
  pinMode(pin, OUTPUT);
  digitalWrite(pin, level);
}

void gpioApplySafeDefaults(void) {
  // Relay 1 (GPIO15, active-high): de-energised = lamp OFF, lamp on the NO
  // contact. The 10 kOhm gate pull-down holds it off until here; from now on
  // only Safety changes it.
  driveInactive(BOARD_PIN_RELAY1, BOARD_RELAY1_ACTIVE_LOW);
  // HC-SR04 TRIG (GPIO17) would float until pulseCaptureBegin(). LOW never
  // back-feeds the module when its 5 V rail is off (§14).
  driveInactive(BOARD_PIN_HCSR04_TRIG, false);
}
```

driveInactive first rejects an unassigned pin, chooses the inactive level from polarity, writes it, sets OUTPUT and writes it again. The second write makes the intended state explicit after mode change.

ApplySafeDefaults uses it for the relay and ultrasonic TRIG. This software action complements the board's passive pull-downs; it does not replace their behaviour before firmware begins executing.

Distinguish a command mirror from measured feedback

The relay adapter translates a logical energised request to pin polarity.

In production only Safety should issue ongoing writes through this interface.

src/hal/relay.h / lines 1-12

```
// Relay 1 output (GPIO15, IRLZ44N low side). HAL, Stage 4 (WP7).
// SOLE USER: services/safety.c (§11 invariant "Safety alone writes relay
// outputs"). Do not include this header anywhere else.
//
// gpioApplySafeDefaults() has already driven the pin to its de-energised
// level and made it an output; relayWrite() only changes the level.
#pragma once
#include <stdbool.h>

// on = coil energised. Polarity from BOARD_RELAY1_ACTIVE_LOW.
void relayWrite(bool on);
bool relayIsOn(void);
```

relayWrite accepts logical on. relayIsOn returns the stored software command. There is no contact-sense input in this interface.

A status obtained from relayIsOn cannot demonstrate that the coil has power, the contacts changed or the lamp received energy. Keep the name's software meaning clear while reviewing higher-level status.

Map logical on through the configured active level

Follow one logical request to the GPIO write and retained command state.

The mapping may invert the pin level according to board polarity.

src/hal/relay.c / lines 1-16

```
#include "hal/relay.h"

#include <Arduino.h>

#include "board_profile.h"

static bool s_on = false;  // gpioApplySafeDefaults() leaves it de-energised

void relayWrite(bool on) {
  if (!boardPinAssigned(BOARD_PIN_RELAY1)) return;
  const bool high = BOARD_RELAY1_ACTIVE_LOW ? !on : on;
  digitalWrite(BOARD_PIN_RELAY1, high ? HIGH : LOW);
  s_on = on;
}

bool relayIsOn(void) { return s_on; }
```

An unassigned pin causes the function to return without changing s_on. For an assigned pin, the ternary inverts on only when active-low is selected, writes HIGH or LOW and records the logical request.

The stored command is static program state, not a new observation of the output circuit. A driver fault outside this code can leave the real load different from s_on without this adapter knowing.

Define zero-wait event submission and retained status

The Safety service is the task boundary around the pure core. Other services submit copies and inspect published status through these functions.

A caller never receives a writable pointer to the core or relay decision state.

src/services/safety.h / lines 1-26

```
// Safety: latched alarms, interlocks, sole relay-output owner.
// Work package WP7 (Stage 4). Architecture §11, §12; decisions in
// domain/safety_core (host-tested). This file only waits, copies and writes.
//
// Task: core 1, priority 6 (cfgTaskSpec(TASK_ID_SAFETY); core 1 since the D14
// radio-load test, away from the Wi-Fi/BLE/TCP-IP tasks on core 0). Wakes on an alarm
// event or every CFG_SAFETY_HEALTH_CHECK_MS. Never waits on network I/O,
// NVS or a mutex. Grow-lamp auto control follows POLICY_LAMP_AUTO_ENABLED.
#pragma once
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>
#include <stdbool.h>
#include <stdint.h>

#include "domain/alarm.h"
#include "domain/event_journal.h"
#include "domain/safety_core.h"

// Creates the detail queue, status mailbox and the task. Relay stays OFF.
bool safetyInit(void);

// Zero-wait, safe from any task (not from an ISR). The per-rule sticky slot
// is always updated; the detail queue may be full, which is counted (§9).
// Wakes Safety. Returns false only if the detail record was dropped.
bool safetySubmitEvent(const AlarmEvent* event);

```

SafetyInit creates queues and task state with the relay OFF. SubmitEvent updates the fallback and attempts bounded detail delivery. SafetyStatus carries the final decision, reason, masks and pass/overflow diagnostics.

src/services/safety.h / lines 27-36

```
// Published after every Safety pass (length-1 mailbox, §8).
typedef struct SafetyStatus {
  MonoMs atMs;
  bool relayOn;
  bool lampWantsOn;
  LampReason lampReason;
  bool interlock;
  uint16_t latchedMask;
  uint16_t activeMask;
  uint16_t staleMask;
```

Define zero-wait event submission and retained status / continued

src/services/safety.h / lines 37-45

```
  uint32_t detailOverflow;  // alarm detail records dropped since boot
  uint32_t passes;
} SafetyStatus;

// Zero-wait copy of the latest status. false before the first pass.
bool safetyStatusPeek(SafetyStatus* out);

// For Supervisor's stack check. NULL before safetyInit().
TaskHandle_t safetyTaskHandle(void);
```

StatusPeek is a retained copy. It may be one pass old when another service reads it, so a consumer must not treat it as an instantaneous lock on Safety's current execution.

Keep callbacks short and requests tied to their session

src/services/safety.h / lines 46-69

```
46
47 // ---- WP8: bounded remote commands (§12 step 4) -----------------------------
48 // The outcome is reported from the Safety task after the step that handled
49 // the command: the callback must copy and return at once.
50 typedef void (*SafetyReplyFn)(uint32_t tag, SafetyCommandOutcome outcome, void* ctx);
51
52 // Zero wait. false = command queue full (reply "busy"). command->expiresMs
53 // is checked in Safety, never applied late. session is the transport's link
54 // generation at receive time: if sessionNow() differs when Safety handles the
55 // command, it is dropped unapplied and unanswered (NULL = no session check).
56 bool safetySubmitCommand(const SafetyCommand* command, uint32_t session, uint32_t
 >   (*sessionNow)(void),
57                          SafetyReplyFn reply, void* ctx);
58
59 // ---- WP8: alarm event journal (§11) ----------------------------------------
60 // TRIP / CLEAR / reset events with independent reader cursors (BLE, LAN).
61 // Zero wait, any task. A new cursor sees only events appended after it.
62 void safetyJournalCursorInit(JournalCursor* cursor);
63 bool safetyJournalNext(JournalCursor* cursor, JournalEvent* out, bool* gap);
64
65 #ifdef FW_ALARM_LATENCY_TEST
66 // Test build only: acquisition records the esp_timer time (us) at which it
67 // submitted an event; Safety measures until it has applied the action.
68 void safetyLatencyMarkSubmit(void);
69 #endif
```

The reply function reports a tag and owner-decided outcome. SubmitCommand includes an optional session-generation checker so an old connection's queued request can be dropped before application.

Journal cursors are independent per reader. The latency marker exists only under its test macro. These public operations are task-context APIs; the header explicitly does not permit SubmitEvent from an ISR.

Reserve queues, core state and scratch arrays with clear roles

Now read the task wrapper that stores alarms, drains commands, calls the core and owns relay writes.

Queues carry copies; portMUX regions protect only small shared bookkeeping.

src/services/safety.c / lines 1-23

```
#include "services/safety.h"

#include <esp_task_wdt.h>
#include <esp_timer.h>
#include <freertos/FreeRTOS.h>
#include <freertos/queue.h>
#include <freertos/task.h>
#include <string.h>

#include "app_config.h"
#include "diagnostics/logger.h"
#include "diagnostics/progress.h"
#include "hal/clock.h"
#include "hal/relay.h"
#include "domain/alarm_sticky.h"
#include "domain/sensor_set.h"
#include "policy_config.h"
#include "services/configuration.h"
#include "services/rail_state.h"
#include "services/snapshot_store.h"

_Static_assert(EVENT_JOURNAL_LEN == CFG_ALARM_JOURNAL_LEN,
  "journal length must match app_config.h");

```

src/services/safety.c / lines 24-32

```
// ---- Sticky per-rule store (§11): survives a full detail
  queue -------------
// Protocol in domain/alarm_sticky.h; every call is under
  s_stickyMux.
static portMUX_TYPE s_stickyMux =
  portMUX_INITIALIZER_UNLOCKED;
static StickySlot s_sticky[POLICY_RULE_COUNT];
static volatile uint32_t s_detailOverflow;  // single writer
   per increment site, read for status only

// ---- Queues and task (all static, §16)
  -------------------------------------
static StaticQueue_t s_detailControl;
static uint8_t s_detailStorage[CFG_ALARM_DETAIL_QUEUE_LEN *
  sizeof(AlarmEvent)];
```

COMMON PITFALL

Static scratch is shared if the function were called concurrently. This implementation relies on one Safety task owning its pass buffers.

Reserve queues, core state and scratch arrays with clear roles / continued

src/services/safety.c / lines 33-51

```
33 static QueueHandle_t s_detail = NULL;
34
35 static StaticQueue_t s_statusControl;
36 static uint8_t s_statusStorage[sizeof(SafetyStatus)];
37 static QueueHandle_t s_status = NULL;
38
39 static StaticTask_t s_tcb;
40 static StackType_t s_stack[3 * 1024];  // cfgTaskSpec(TASK_ID_SAFETY)->stackBytes
41 static TaskHandle_t s_task = NULL;
42
43 // ---- WP8: command queue and event journal ----------------------------------
44 typedef struct SafetyRequest {
45   SafetyCommand command;
46   uint32_t session;              // transport session generation at receive time
47   uint32_t (*sessionNow)(void);  // NULL = no session check
48   SafetyReplyFn reply;
49   void* ctx;
50 } SafetyRequest;
51
```

The file declares sticky slots, a 16-record detail queue, a one-item status mailbox, a command queue, a journal and task storage. Scratch arrays gather one pass's copied inputs without borrowing another task's buffers.

src/services/safety.c / lines 52-70

```
52 static StaticQueue_t s_commandControl;
53 static uint8_t s_commandStorage[SAFETY_MAX_COMMANDS * sizeof(SafetyRequest)];
54 static QueueHandle_t s_commands = NULL;
55
56 static portMUX_TYPE s_journalMux = portMUX_INITIALIZER_UNLOCKED;
57 static EventJournal s_journal;
58
59 // Task-private state: static, never on the 3 KiB stack.
60 static SafetyRequest s_requests[SAFETY_MAX_COMMANDS];
61 static SafetyCommand s_cmds[SAFETY_MAX_COMMANDS];
62 static ConfigBlob s_cfg;
63 static SafetyCore s_core;
64 static Snapshot s_snap;
65 static AlarmEvent s_events[CFG_ALARM_DETAIL_QUEUE_LEN + 2 * POLICY_RULE_COUNT];
66 static SafetyStatus s_statusOut;
67
68 #ifdef FW_ALARM_LATENCY_TEST
69 // ---- Alarm latency test (env:alarm_latency, §19) ----------------------------
70 // submit -> action: esp_timer us from acquisition's submit to the end of the
```

Their capacities are related: the event array leaves room for queue detail plus up to two fallback events per rule. The source assertions connect journal and rule capacities to configuration constants.

Measure the interval the test actually names

src/services/safety.c / lines 71-89

```
71 //                   Safety pass that applied the decision (relay written).
72 // sample -> action: ms from the sample's intervalEndMs (BME280 read done).
73 static portMUX_TYPE s_latMux = portMUX_INITIALIZER_UNLOCKED;
74 static int64_t s_submitUs;  // 0 = no event waiting
75
76 typedef struct LatencyStats {
77   uint32_t n;
78   int64_t minUs, maxUs, sumUs;
79   uint32_t minMs, maxMs, sumMs;
80 } LatencyStats;
81 static LatencyStats s_lat;
82 static uint32_t s_latReports;
83
84 void safetyLatencyMarkSubmit(void) {
85   const int64_t t = esp_timer_get_time();
86   portENTER_CRITICAL(&s_latMux);
87   s_submitUs = t;
88   portEXIT_CRITICAL(&s_latMux);
89 }
```

The optional test structures retain a submission timestamp and aggregate durations. MarkSubmit records microsecond time under a short lock.

These declarations do not measure the physical relay contact. The later stop point is the software output action in safetyTask. Name that interval accurately when interpreting the printed latency.

Report bounded latency statistics after actions

src/services/safety.c / lines 90-117

```
90
91 static void latencyRecord(const AlarmEvent* events, uint8_t n) {
92   const AlarmEvent* trip = NULL;
93   for (uint8_t i = 0; i < n; ++i) {
94     if (events[i].decision == RULE_DECISION_TRIP) trip = &events[i];
95   }
96   portENTER_CRITICAL(&s_latMux);
97   const int64_t submitted = s_submitUs;
98   s_submitUs = 0;
99   portEXIT_CRITICAL(&s_latMux);
100  if (trip == NULL || submitted == 0) return;
101
102  const int64_t us = esp_timer_get_time() - submitted;
103  const MonoMs now = monoNowMs();
104  const uint32_t ms = now > trip->sample.intervalEndMs ? (uint32_t)(now -
   >  trip->sample.intervalEndMs) : 0;
105  if (ms > 5 || us > 5000) {  // one slow trip: when exactly (the 20-trip summary hides
   >  it)
106    logPrintf(LOG_LEVEL_WARN, "latency SPIKE at t=%lu ms: submit->action %lld us,
   >  sample->action %lu ms",
107              (unsigned long)now, (long long)us, (unsigned long)ms);
108  }
109  if (s_lat.n == 0 || us < s_lat.minUs) s_lat.minUs = us;
110  if (s_lat.n == 0 || us > s_lat.maxUs) s_lat.maxUs = us;
111  if (s_lat.n == 0 || ms < s_lat.minMs) s_lat.minMs = ms;
112  if (s_lat.n == 0 || ms > s_lat.maxMs) s_lat.maxMs = ms;
113  s_lat.sumUs += us;
114  s_lat.sumMs += ms;
115  if (++s_lat.n == 20) {
116    logPrintf(LOG_LEVEL_INFO,
117              "latency #%lu (20 TRIPs): submit->action us min %lld avg %lld max %lld | "
```

The test records submit-to-action microseconds and sample-to-action milliseconds over a small batch, updates extrema and prints periodically. It then resets the aggregate.

Report bounded latency statistics after actions / continued

src/services/safety.c / lines 118-124

```
118               "sample->action ms min %lu avg %lu max %lu",
119               (unsigned long)++s_latReports, (long long)s_lat.minUs, (long
  >   long)(s_lat.sumUs / 20),
120               (long long)s_lat.maxUs, (unsigned long)s_lat.minMs, (unsigned
  >   long)(s_lat.sumMs / 20),
121               (unsigned long)s_lat.maxMs);
122     memset(&s_lat, 0, sizeof(s_lat));
123   }
124 }
```

The two origins answer different questions: scheduling/dispatch after submission versus total age since sample completion. Neither is automatically the sensor's trigger-to-physical-response interval.

Retain, attempt detail delivery, finalise and wake

src/services/safety.c / lines 125-143

```
#endif

bool safetySubmitEvent(const AlarmEvent* e) {
  if (e == NULL || e->ruleId >= POLICY_RULE_COUNT) return false;
  if (e->decision != RULE_DECISION_TRIP && e->decision != RULE_DECISION_CLEAR) return
  false;
  StickySlot* slot = &s_sticky[e->ruleId];

  portENTER_CRITICAL(&s_stickyMux);
  stickyBegin(slot, &kPolicyRules[e->ruleId], e);  // in flight: Safety leaves the slot
  alone
  portEXIT_CRITICAL(&s_stickyMux);

  const bool queued = s_detail != NULL && xQueueSend(s_detail, e, 0) == pdTRUE;  //
  never inside the portMUX
  portENTER_CRITICAL(&s_stickyMux);
  stickyEnd(slot, queued);
  if (!queued) ++s_detailOverflow;
  portEXIT_CRITICAL(&s_stickyMux);
  if (s_task != NULL) xTaskNotifyGive(s_task);  // after stickyEnd(): the next pass sees
   the outcome
  return queued;
}
```

The function validates event identity and transition kind. It calls stickyBegin under the lock, performs xQueueSend outside it, records the result under the lock and only then notifies the task.

False reports a dropped detail record, while sticky retention can still preserve the critical transition. The caller must understand that narrower return meaning rather than assuming all safety evidence vanished.

Drain detailed events before collecting fallback evidence

src/services/safety.c / lines 144-166

```
144
145 // Drain the detail queue, then, only for a rule whose detail record was
146 // dropped, let its sticky slot re-supply what may be missing: the TRIP (if
147 // none came through the queue) and, if the rule's last decision was CLEAR,
148 // that CLEAR too, so the rule cannot stay active forever. A record that was
149 // queued arrives through the queue, so nothing is processed twice. A slot
150 // whose send is still in flight is collected on the next pass.
151 static uint8_t drainEvents(void) {
152   uint8_t n = 0;
153   while (n < CFG_ALARM_DETAIL_QUEUE_LEN && xQueueReceive(s_detail, &s_events[n], 0) ==
  >   pdTRUE) ++n;
154
155   bool sawTrip[POLICY_RULE_COUNT] = {false};
156   for (uint8_t i = 0; i < n; ++i) {
157     if (s_events[i].ruleId < POLICY_RULE_COUNT && s_events[i].decision ==
  >   RULE_DECISION_TRIP) {
158       sawTrip[s_events[i].ruleId] = true;
159     }
160   }
161   portENTER_CRITICAL(&s_stickyMux);
162   n = (uint8_t)(n + stickyCollect(s_sticky, POLICY_RULE_COUNT, sawTrip, &s_events[n]));
163   portEXIT_CRITICAL(&s_stickyMux);
164   return n;
165 }
166
```

drainEvents receives bounded detail records, marks which rules supplied a trip and then asks stickyCollect for missing transitions. The output array covers the declared maximum.

src/services/safety.c / lines 167-174

```
167 bool safetySubmitCommand(const SafetyCommand* command, uint32_t session, uint32_t
  >   (*sessionNow)(void),
168                           SafetyReplyFn reply, void* ctx) {
169   if (s_commands == NULL || command == NULL) return false;
170   const SafetyRequest request = {*command, session, sessionNow, reply, ctx};
171   if (xQueueSend(s_commands, &request, 0) != pdTRUE) return false;
172   if (s_task != NULL) xTaskNotifyGive(s_task);
173   return true;
174 }
```

SubmitCommand copies the command, session checker, reply function and context into a queued request, then wakes Safety. Callback/context lifetimes must remain suitable until the owner processes the request.

Expose a journal copy under a short lock

src/services/safety.c / lines 175-200

```
175
176 void safetyJournalCursorInit(JournalCursor* cursor) {
177   portENTER_CRITICAL(&s_journalMux);
178   journalCursorInit(&s_journal, cursor);
179   portEXIT_CRITICAL(&s_journalMux);
180 }
181
182 bool safetyJournalNext(JournalCursor* cursor, JournalEvent* out, bool* gap) {
183   portENTER_CRITICAL(&s_journalMux);
184   const bool got = journalNext(&s_journal, cursor, out, gap);
185   portEXIT_CRITICAL(&s_journalMux);
186   return got;
187 }
188
189 static void journal(uint8_t ruleId, JournalKind kind, MonoMs atMs, float value) {
190   JournalEvent e;
191   memset(&e, 0, sizeof(e));
192   e.ruleId = ruleId;
193   e.severity = ruleId < POLICY_RULE_COUNT ? kPolicyRules[ruleId].severity :
  >   SEVERITY_INFO;
194   e.kind = kind;
195   e.atMs = atMs;
196   e.value = value;
197   portENTER_CRITICAL(&s_journalMux);
198   journalAppend(&s_journal, &e);
199   portEXIT_CRITICAL(&s_journalMux);
200 }
```

CursorInit and Next wrap the pure journal with a portMUX so a record copy cannot interleave with an append. The private journal helper builds one event and appends it under the same protection.

Timestamp choice depends on event kind: trip/clear use sample end, while reset uses the owner's action time. The event's n supplies journal order; atMs records the relevant semantic time.

Drop commands whose originating session no longer exists

src/services/safety.c / lines 201-216

```
201
202 // A request from a link that has since closed (or reconnected) is dropped
203 // unapplied and unanswered, as Configuration does (§11).
204 static uint8_t drainCommands(void) {
205   uint8_t n = 0;
206   while (n < SAFETY_MAX_COMMANDS && xQueueReceive(s_commands, &s_requests[n], 0) ==
  >   pdTRUE) {
207     const SafetyRequest* r = &s_requests[n];
208     if (r->sessionNow != NULL && r->sessionNow() != r->session) {
209       logPrintf(LOG_LEVEL_INFO, "safety: rule %u reset from a closed session dropped",
  >   (unsigned)r->command.ruleId);
210       continue;
211     }
212     s_cmds[n] = r->command;
213     ++n;
214   }
215   return n;
216 }
```

The helper receives at most the bounded command count and compares a non-null session checker with the request's recorded generation. Expired sessions are not forwarded to the pure core or replied to as a new connection's command.

Accepted requests are retained alongside their pure command copies so the task can correlate later results with the correct callback and context.

Wake for work or age, then assemble one coherent decision input

src/services/safety.c / lines 217-244

```
217
218 static void safetyTask(void* arg) {
219   (void)arg;
220   SafetyOutputs out;
221   bool relayOn = false;
222   LampReason lastReason = LAMP_REASON_NO_DATA;
223   uint32_t passes = 0;
224
225   // Task watchdog (§18): reset by this task only, after each completed pass
226   // (at least every 100 ms). Supervisor never resets it on Safety's behalf.
227   if (esp_task_wdt_add(NULL) != ESP_OK) logPrintf(LOG_LEVEL_ERROR, "safety: task
  >   watchdog add failed");
228
229   for (;;) {
230     ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(CFG_SAFETY_HEALTH_CHECK_MS));
231 #ifdef FW_INJECT_STALL_SAFETY
232     // Acceptance test (§19 "Stall injection"): block forever after N ms.
233     if (monoNowMs() >= (MonoMs)(FW_INJECT_STALL_SAFETY)) {
234       logPrintf(LOG_LEVEL_WARN, "safety: INJECTED STALL - blocking forever");
235       vTaskDelay(portMAX_DELAY);
236     }
237 #endif
238     const MonoMs now = monoNowMs();
239     const uint8_t n = drainEvents();
240     const uint8_t nCmd = drainCommands();
241     const bool haveSnap = snapshotPeek(&s_snap);
242     const RailState rail = railStateRead();
243     if (haveSnap) snapshotGateRail(&s_snap, rail.valid5v, rail.epoch);  // no 5 V value
  >   of an older epoch
244     configurationPeekApplied(&s_cfg);  // defaults (all on) if Configuration is not
  >   running
```

Wake for work or age, then assemble one coherent decision input / continued

Safety subscribes to its watchdog and waits on notifications with a 100ms health timeout. Each pass drains events/commands, peeks the Snapshot, gates rail-dependent copies with current rail state and peeks configuration.

src/services/safety.c / lines 245-248

```
245     const SensorMask disabled = (SensorMask)(SENSOR_MASK_ALL &
  >   (SensorMask)~s_cfg.sensorEnabled);
246     const SafetyInputs in = {haveSnap ? &s_snap : NULL, s_events, n, now, s_cmds, nCmd,
  >   disabled,
247                              !rail.valid5v};
248     safetyCoreStep(&s_core, &in, &out);
```

It derives the disabled-sensor mask and calls safetyCoreStep with now and railDown. The timed wake lets missing new data become stale even when acquisition sends no event announcing the absence.

Apply the one final output before publishing explanations

src/services/safety.c / lines 249-275

```
249
250     const bool relayChanged = out.relayOn != relayOn;
251     if (relayChanged) {
252       relayWrite(out.relayOn);
253       relayOn = out.relayOn;
254     }
255 #ifdef FW_ALARM_LATENCY_TEST
256     latencyRecord(s_events, n);  // the action is applied: stop the clock
257 #endif
258     if (relayChanged) {
259       logPrintf(LOG_LEVEL_INFO, "safety: relay %s (%s)", relayOn ? "ON" : "OFF",
260                 lampReasonName(out.lampReason));
261     } else if (out.lampReason != lastReason &&
262                (out.lampReason == LAMP_REASON_STALE || out.lampReason ==
  >   LAMP_REASON_FAULT ||
263                 out.lampReason == LAMP_REASON_INTERLOCK)) {
264       logPrintf(LOG_LEVEL_WARN, "safety: lamp %s", lampReasonName(out.lampReason));
265     }
266     lastReason = out.lampReason;
267     for (uint8_t i = 0; i < out.resultCount && i < nCmd; ++i) {
268       if (out.results[i].outcome == SAFETY_CMD_APPLIED) {
269         journal(s_cmds[i].ruleId, JOURNAL_KIND_RESET, now, 0.0f);
270         logPrintf(LOG_LEVEL_INFO, "safety: rule %u latch reset",
  >   (unsigned)s_cmds[i].ruleId);
271       }
272       if (s_requests[i].reply != NULL) {
273         s_requests[i].reply(out.results[i].tag, out.results[i].outcome,
  >   s_requests[i].ctx);
274       }
275     }
```

The task writes the relay only if the core's decision changed. The optional latency stop point follows that write. Logging reports changed state or selected changed failure reasons rather than repeating every pass.

src/services/safety.c / lines 276-278

```
276     for (uint8_t i = 0; i < n; ++i) {
277 #ifdef FW_ALARM_LATENCY_TEST
278       if (s_events[i].decision == RULE_DECISION_TRIP && s_events[i].ruleId == 0)
  >   continue;  // 4 per s
```

Command results are then journalled and replied to from the owner. A callback must copy and return promptly, because its execution is still part of Safety's pass.

Publish status and mark completed progress

src/services/safety.c / lines 279-302

```
279 #endif
280       journal(s_events[i].ruleId,
281               s_events[i].decision == RULE_DECISION_TRIP ? JOURNAL_KIND_TRIP :
  >   JOURNAL_KIND_CLEAR,
282               s_events[i].sample.intervalEndMs, s_events[i].sample.value);
283       logPrintf(LOG_LEVEL_WARN, "safety: rule %u %s (%s = %.2f)",
  >   (unsigned)s_events[i].ruleId,
284                 s_events[i].decision == RULE_DECISION_TRIP ? "TRIP" : "CLEAR",
285                 metricName(s_events[i].sample.metric),
  >   (double)s_events[i].sample.value);
286     }
287
288     s_statusOut.atMs = now;
289     s_statusOut.relayOn = relayOn;
290     s_statusOut.lampWantsOn = out.lampWantsOn;
291     s_statusOut.lampReason = out.lampReason;
292     s_statusOut.interlock = out.interlock;
293     s_statusOut.latchedMask = out.latchedMask;
294     s_statusOut.activeMask = out.activeMask;
295     s_statusOut.staleMask = out.staleMask;
296     s_statusOut.detailOverflow = s_detailOverflow;
297     s_statusOut.passes = ++passes;
298     xQueueOverwrite(s_status, &s_statusOut);
299     progressMark(TASK_ID_SAFETY);
300     esp_task_wdt_reset();
301   }
302 }
```

Trip/clear events are journalled with their evidence. The service fills status fields from this pass, overwrites the mailbox, marks progress and resets its own watchdog.

Publishing after output application means status describes the completed decision pass. Another task still reads a retained copy, not a permanent guarantee about later rail or input changes.

Create dependencies before the task can execute

src/services/safety.c / lines 303-330

```
303
304 TaskHandle_t safetyTaskHandle(void) { return s_task; }
305
306 bool safetyInit(void) {
307   if (s_task != NULL) return true;
308   s_detail = xQueueCreateStatic(CFG_ALARM_DETAIL_QUEUE_LEN, sizeof(AlarmEvent),
  >   s_detailStorage,
309                                 &s_detailControl);
310   s_status = xQueueCreateStatic(1, sizeof(SafetyStatus), s_statusStorage,
  >   &s_statusControl);
311   s_commands = xQueueCreateStatic(SAFETY_MAX_COMMANDS, sizeof(SafetyRequest),
  >   s_commandStorage,
312                                   &s_commandControl);
313   if (s_detail == NULL || s_status == NULL || s_commands == NULL) return false;
314   journalInit(&s_journal);
315
316   // Lamp policy from the applied configuration (compiled defaults in 8.2).
317   configurationPeekApplied(&s_cfg);
318   SafetyConfig config;
319   memset(&config, 0, sizeof(config));
320   config.lampAutoEnabled = s_cfg.lampAutoEnabled;
321   config.lamp = s_cfg.lamp;
322   config.lightStaleMs = POLICY_LIGHT_STALE_MS;
323   safetyCoreInit(&s_core, &config, kPolicyRules, POLICY_RULE_COUNT, monoNowMs());
324   relayWrite(false);
325
326   const TaskSpec* spec = cfgTaskSpec(TASK_ID_SAFETY);
327   if (spec->stackBytes != sizeof(s_stack)) {
328     logPrintf(LOG_LEVEL_WARN, "safety: stack %u B, task table says %lu B",
329               (unsigned)sizeof(s_stack), (unsigned long)spec->stackBytes);
330   }
```

Init creates detail, status and command queues, initialises the journal and pure core, writes OFF and creates a static pinned task using the selected TaskSpec. A queue failure returns before task creation.

Create dependencies before the task can execute / continued

src/services/safety.c / lines 331-338

```
331   const BaseType_t safetyCore = spec->core;
332   s_task = xTaskCreateStaticPinnedToCore(safetyTask, spec->name, sizeof(s_stack), NULL,
333                                          spec->priority, s_stack, &s_tcb, safetyCore);
334   if (s_task == NULL) return false;
335   logPrintf(LOG_LEVEL_INFO, "safety: started (core %u, prio %u), lamp auto %s, relay
  >   OFF",
336             (unsigned)safetyCore, (unsigned)spec->priority, s_cfg.lampAutoEnabled ? "ON"
  >    : "off");
337   return true;
338 }
```

Because a task may execute immediately, all data it uses is established first. The actual stack storage is compared with the configuration table so a mismatch is visible rather than silently assumed correct.

Return a status copy only when available

src/services/safety.c / lines 339-343

```
339
340 bool safetyStatusPeek(SafetyStatus* out) {
341   if (s_status == NULL || out == NULL) return false;
342   return xQueuePeek(s_status, out, 0) == pdTRUE;
343 }
```

StatusPeek rejects a missing mailbox or destination and performs an immediate peek. True means the caller obtained the most recent retained pass record.

It does not hold Safety paused while the caller examines that copy. Configuration and communication should use the record's semantics and age instead of trying to modify its masks as a backdoor command.

Read the actual polarity and optical assumptions

This isolated lamp test intentionally gives its own test task the relay. It is a separate entry point, not an additional production writer.

The shared pure lamp model lets the test exercise hysteresis and dwell without the full Safety service.

test/bringup/lamp_main.c / lines 1-28

```
// Bring-up: DFR0026 light level -> relay module -> grow lamp.
//   pio run -e test_lamp -t upload -t monitor
// Light as in test_light (S -> 10 kOhm -> GPIO1, 1 uF to GND). Relay: a
// TONGLING JQC-3FF-S-Z 5 V "Low Level Trigger" module, VCC 5 V, GND common.
// IN is driven through an N-MOSFET or NPN like the PCB stage (GPIO15 -> 220 Ohm
// gate / 1 kOhm base, 10 kOhm pull-down, IN on drain/collector), so GPIO15
// HIGH = relay ON: active-high, as BOARD_RELAY1_ACTIVE_LOW false. Direct drive
// (IN on GPIO15) failed on hardware 24 Oct 2026: 3.3 V cannot release the
// module's 5 V-referenced input, the relay stayed on with GPIO15 HIGH, and an
// unpowered module back-fed ~2.5 V onto VCC through IN.
// Lamp live on COM -> NO, NC empty: relay off = lamp off.
//
// Not the production path (WP7: Safety owns the relay). The decision is
// domain/lamp_control; this file only feeds it readings and drives the pin.
// One line per reading, e.g.
//   [lamp] 32 mV (uncal) -> ON  held 41 s
//   [lamp] 150 mV (uncal) -> ON  held 12 s, OFF waiting (dwell 30 s)
// LAMP_* thresholds are bench placeholders; set them from the lamp-off /
// lamp-on readings at the final sensor position (README "Lamp relay bring-up").
#include <Arduino.h>
#include <stdio.h>

#include "app_config.h"
#include "board_profile.h"
#include "bringup_harness.h"
#include "domain/lamp_control.h"
#include "drivers/light.h"
#include "hal/adc_channel.h"
```

The test documents the transistor interface, active-high logical control and normally-open lamp contact. It defines the same style of raw-mV thresholds and dwell as policy_config.

test/bringup/lamp_main.c / lines 29-35

```
#include "hal/clock.h"

#define LAMP_RELAY_ACTIVE_LOW 0  // transistor stage; 1 only for a module that releases
  at 3.3 V
#define LAMP_ON_BELOW_MV 40.0f
#define LAMP_OFF_ABOVE_MV 120.0f
#define LAMP_MIN_ON_MS ((uint32_t)30000)   // bench; minutes in real use
#define LAMP_MIN_OFF_MS ((uint32_t)30000)
```

Read the actual polarity and optical assumptions / continued

test/bringup/lamp_main.c / lines 36-43

```
#define LAMP_FAULTS_TO_OFF 5

static AdcChannel s_adc = ADC_CHANNEL_INIT;
static LightConfig s_config = {&s_adc};
static LightDriver s_driver;
static LampControl s_lamp;

static void relayWrite(bool on) {
```

The historical wiring observation explains why this setup uses an interface stage. It is not a reason to copy an arbitrary relay module's polarity to a different circuit. Establish which logical level energises the actual stage before interpreting the software trace.

Feed one light outcome into one lamp update

test/bringup/lamp_main.c / lines 44-65

```
  const bool high = LAMP_RELAY_ACTIVE_LOW ? !on : on;
  digitalWrite(BOARD_PIN_RELAY1, high ? HIGH : LOW);
}

static void afterLight(SensorDriver* driver) {
  Reading r;
  sensorDriverReadResult(driver, &r, 1);
  const MonoMs now = monoNowMs();
  const bool wasOn = s_lamp.on;
  const bool on = lampControlUpdate(&s_lamp, &r, now);
  relayWrite(on);

  printf("[lamp] ");
  if (r.health == HEALTH_OK || r.health == HEALTH_UNCALIBRATED) {
    printf("%.0f mV%s", (double)r.value, (r.qualityFlags & kQualitySaturated) ? " SAT" :
 > "");
  } else {
    printf("light=%s (%u in a row)", healthName(r.health), (unsigned)s_lamp.faults);
  }
  printf(" -> %s  held %lu s", on ? "ON " : "OFF",
         (unsigned long)(lampControlHeldMs(&s_lamp, now) / 1000));
  if (s_lamp.waiting) {
    printf(", %s waiting (dwell %lu s)", on ? "OFF" : "ON",
```

The test-local relayWrite maps polarity. afterLight obtains one retained result, remembers the previous lamp state, calls the pure update and writes the requested output.

This callback runs after READY in the harness owner. It therefore uses the filtered driver outcome and does not compete with another ADC task. The later log can distinguish a desired change held by dwell from a real state change.

Print the state and the reason a transition may still wait

test/bringup/lamp_main.c / lines 66-70

```
66              (unsigned long)((on ? LAMP_MIN_ON_MS : LAMP_MIN_OFF_MS) / 1000));
67    }
68    if (on != wasOn) printf("   <-- relay %s", on ? "ON" : "OFF");
69    printf("\n");
70  }
```

The diagnostic includes raw voltage or health, current state, held seconds and waiting direction. A changed state gets an explicit marker.

This bring-up callback does not implement the production core's periodic stale-data check and interlocks. Its useful scope is the level/dwell response to arriving readings, not proof of the complete production failure policy.

Establish OFF before creating repeated light work

test/bringup/lamp_main.c / lines 71-98

```
71
72 static BringupContext s_ctx = {&s_driver.base, CFG_FRAME_PERIOD_MS, afterLight};
73
74 void setup(void) {
75   // Relay off before the pin becomes an output, so the lamp never blinks.
76   relayWrite(false);
77   pinMode(BOARD_PIN_RELAY1, OUTPUT);
78   relayWrite(false);
79
80   static const LampControlConfig lampConfig = {LAMP_ON_BELOW_MV, LAMP_OFF_ABOVE_MV,
81                                                 LAMP_MIN_ON_MS, LAMP_MIN_OFF_MS,
82                                                 LAMP_FAULTS_TO_OFF};
83   lampControlInit(&s_lamp, &lampConfig, monoNowMs());
84   lightDriverConstruct(&s_driver, &s_config);
85   delay(200);
86   printf("\n=== bring-up: lamp (light GPIO%d -> relay GPIO%d, active-%s) ===\n",
87          (int)BOARD_PIN_LIGHT_ADC, (int)BOARD_PIN_RELAY1, LAMP_RELAY_ACTIVE_LOW ? "low"
 >   : "high");
88   printf("ON below %.0f mV, OFF above %.0f mV, min on/off %lu/%lu s, OFF after %d bad
 >   readings\n",
89          (double)LAMP_ON_BELOW_MV, (double)LAMP_OFF_ABOVE_MV,
90          (unsigned long)(LAMP_MIN_ON_MS / 1000), (unsigned long)(LAMP_MIN_OFF_MS /
 >   1000),
91          LAMP_FAULTS_TO_OFF);
92   if (!adcChannelBegin(&s_adc, BOARD_PIN_LIGHT_ADC)) {
93     printf("GPIO%d is not an ADC1 pin\n", (int)BOARD_PIN_LIGHT_ADC);
94   }
95   bringupStart(&s_ctx);
96 }
97
98 void loop(void) { vTaskDelay(portMAX_DELAY); }
```

Setup preloads OFF before output mode, initializes the pure model at the current time, constructs light and configures the ADC. It reports thresholds and creates the harness.

At boot the minimum-off interval begins, so fresh darkness should not energise immediately. Observe that delay and the opposite transition after a new bright reading before changing the policy values.

A rail can fail and recover between two reads

A boolean rail-valid flag tells you the current state, but not its history. Suppose pH starts while 5V is valid. The rail drops for 5ms and returns before pH finishes. A final valid=true test would miss the interruption. The measurement may have crossed a collapsing supply.

An epoch is a counter that changes when the rail is invalidated. Store startEpoch when a 5V operation starts. At completion, accept only if the rail is valid and its epoch still matches. A down-and-up transition changes the epoch even if the final level is high.

Moment	Rail valid	Epoch	Effect
Start	true	12	Store startEpoch=12.
Edge detected	false	13	Invalidate in-flight observations.
Stable return	true	13	Current supply may be usable.
Old result ends	true	13	Reject: 13 differs from 12.

The exact counter values are invented; the relationship is what matters. Restoration also names the epoch on which the decision was based. A newer edge prevents an older restore action from declaring the rail valid by mistake.

The ISR records urgent invalidation promptly. The coordinator later applies debounce and stability rules for mode changes. Those jobs have different timing: questionable measurements should be rejected immediately, while a stable mode transition should not chatter on every short edge.

Here valid5v is software permission to use a rail-dependent observation, not a firmware command that magically produces voltage. The physical supply switching belongs to the power circuit. Keep a logical validity action distinct from a measured electrical rail level.

REMEMBER

Epoch answers whether the relevant history changed, not only whether the input is high now.

Ask the old owner to stop, then grant the new one



Worked handover. The generation numbers identify different ownership periods.

COMMON PITFALL

A timeout waiting for release is not proof of release. Granting the second owner anyway creates concurrent access at the worst possible moment.

The coordinator requests quiescence from the current worker. Quiesce means stop starting new sensor work, cancel pending operations, release a held ping gate, make affected pins safe, and leave the active watchdog subscription. The worker performs this at its own pass boundary.

Only after completing that cleanup does it set owner=NONE and publish the released generation under the lease lock. The coordinator waits for that acknowledgement before granting the other worker. The new worker then restarts its schedule and applies the allowed sensor set.

A generation number distinguishes this acknowledgement from an old one still stored in memory. Released generation 7 is evidence that ownership period 7 ended. It is not automatically permission to finish a later request for generation 9.

Why not suspend the task immediately? It might be halfway through a bus transaction or holding the flash gate. Pausing it there leaves shared hardware in an unknown intermediate condition. Cooperative quiescence lets the owner complete its cleanup deliberately.

Both workers use the same static drivers. Their different cores and private task stacks do not duplicate those drivers. The handover protocol is therefore essential to the current design's one-owner rule.

Delay a mode decision, not the whole task

The coordinator repeatedly feeds current sense level, rail state, time and acknowledgements into powerModeStep(). That pure function updates its state and returns actions. The task then executes the requested quiesce, logical rail restore, grant and Wi-Fi actions.

Source timing	Purpose
50ms low debounce	Recognise sustained external-power loss.
500ms stable high	Accept a return after settling.
200ms release acknowledgement budget	Detect an unusually slow owner handover.
2000ms network-off budget	Detect delayed Wi-Fi shutdown.

Instead of one blocking 500ms delay, the state machine stores when high was first seen and revisits the condition. If another edge appears, it can restart the stability interval. If Wi-Fi acknowledges, it can advance. Time remains an input to a short step.

NORMAL can move toward LOW_POWER; LOW_POWER can move toward NORMAL. If a required acknowledgement misses its budget, the mode becomes DEGRADED while the coordinator keeps waiting. It does not grant a second owner merely to make the state name look healthy. A late valid acknowledgement can still allow progress.

In low-power mode the allowed set contains BME280, DS18B20, light and SGP30 on the retained 3.3V rail. The switched-5V sensors are marked unavailable. This source's low-power mode is an application policy; it is not a claim that the MCU automatically enters deep sleep.

The power-transition policy is enabled in this snapshot, while comments record that this path still needed hardware testing. Preserve that distinction when assessing readiness.

COMMON PITFALL

Debouncing mode changes must not leave already-invalid sensor results eligible for control.

Know what a simpler design would change

REMEMBER

Task count is a design consequence, not a measure of sophistication.

You could keep one acquisition task and change only its allowed sensor mask when external power disappears. That would remove the cross-task lease handover and one worker stack. It could be a reasonable redesign if the same task's affinity and operating policy suit both modes.

The archived design instead moves between two workers pinned to different cores. Reviewing that choice means comparing actual needs: core availability, radio interference, stack cost, transition complexity and measured response. The existence of two cores does not by itself justify migration.

One worker with mode	Two workers with a lease
Fewer task resources and no owner transfer.	Distinct affinity and worker policies.
Mode state must cancel invalid work correctly.	Mode state must also prove old-owner release.
Still needs rail epochs and safe pin handling.	Still needs rail epochs and safe pin handling.

This book keeps the original implementation intact so its code and explanations remain aligned. The simpler option is an architectural comparison, not a silent rewrite. If you implement it later, preserve the behaviours we have already justified: one owner, bounded steps, explicit cancellation, epoch checks and output gating.

That is how to change architecture responsibly as a learner. Identify which problem a mechanism solves. Replace it only when the new design preserves the required behaviour by another clear route. Reducing the number of files or tasks is useful only if the resulting reasoning remains correct.

Read validity and history as one record

This pair of fields distinguishes a currently valid 5V observation context from one interrupted by a rail event.

The epoch is a history tag. It is not the external voltage or a task generation.

src/services/rail_state.h / lines 1-22

```
// 5 V rail validity + epoch (§12). Work package WP9 (Stage 6).
// The ISR invalidates and bumps the epoch on every sense edge; only
// PowerCoordinator may restore validity. Copies are taken in a short
// portMUX critical section so both cores see a coherent {valid, epoch}.
#pragma once
#include <stdbool.h>
#include <stdint.h>

typedef struct RailState {
  bool valid5v;
  uint32_t epoch;
} RailState;

RailState railStateRead(void);
void railStateInvalidateFromIsr(void);          // ISR-safe
void railStateRestore(uint32_t expectedEpoch);  // PowerCoordinator only

// Accept a 5 V result only if the rail was valid before and after and the
// epoch did not change in between.
static inline bool railSampleStillValid(const RailState* before, const RailState* after)
    {
  return before->valid5v && after->valid5v && before->epoch == after->epoch;
}
```

RailState groups valid 5v and epoch so readers can obtain a coherent pair. The ISR invalidates and advances history; only the coordinator restores validity for the epoch it inspected.

railSampleStillValid requires valid before, valid after and equal epochs. A low-high interruption between the two observations therefore fails even when the final level is high. The helper does not switch the physical supply.

Copy both fields under one lock

The implementation uses a short cross-core critical section to protect that pair.

The begin-state policy differs deliberately between isolated bring-up and production.

src/services/rail_state.c / lines 1-20

```
#include "services/rail_state.h"

#include <freertos/FreeRTOS.h>

static portMUX_TYPE s_railMux = portMUX_INITIALIZER_UNLOCKED;
// Bench bring-up assumes the 5 V rail is present. Otherwise unknown power at
// boot is treated conservatively as unavailable (§11) until WP9 samples it.
#ifdef FW_BRINGUP
static RailState s_rail = {true, 1};
#else
static RailState s_rail = {false, 1};
#endif

RailState railStateRead(void) {
  RailState copy;
  portENTER_CRITICAL(&s_railMux);
  copy = s_rail;
  portEXIT_CRITICAL(&s_railMux);
  return copy;
}
```

Bring-up starts valid for bench experiments that assume the external rail is supplied. Production starts invalid until the coordinator qualifies it. Read takes a local copy under s_railMux and returns it after unlocking.

A reader therefore receives a pair from one protected moment, not a new epoch combined with an old validity flag. The copy remains historical once returned; a later edge can invalidate the current state.

Restore only the epoch that was actually qualified

src/services/rail_state.c / lines 21-33

```
21
22 void railStateInvalidateFromIsr(void) {
23   portENTER_CRITICAL_ISR(&s_railMux);
24   s_rail.valid5v = false;
25   s_rail.epoch++;
26   portEXIT_CRITICAL_ISR(&s_railMux);
27 }
28
29 void railStateRestore(uint32_t expectedEpoch) {
30   portENTER_CRITICAL(&s_railMux);
31   if (s_rail.epoch == expectedEpoch) s_rail.valid5v = true;
32   portEXIT_CRITICAL(&s_railMux);
33 }
```

InvalidateFromIsr uses the ISR-safe critical-section form, clears validity and increments epoch. Restore compares the expected epoch before setting true.

If another edge occurs after the coordinator's decision, its newer epoch prevents the stale restore from undoing that invalidation. This conditional write is the centre of the race protection, not merely the presence of a lock.

Read the handover contract before the state names

The pure power state machine returns requested actions instead of operating drivers or radios itself.

The coordinator supplies actual levels and acknowledgements and executes the returned actions in the required order.

src/domain/power_mode.h / lines 1-28

```
// Power-mode state machine and acquisition lease decisions (§13, §14).
// Pure C, host-tested. Work package WP9 (Stage 6).
//
// PowerCoordinator (services/power_coordinator.c) feeds it the external-power
// sense level, the rail validity and the acknowledgements, once per tick, and
// executes the actions it returns. It never touches hardware itself.
//
//   NORMAL --(power low for lossDebounceMs)--> TRANSITION_TO_LOW:
//       quiesce the normal worker, stop Wi-Fi;
//       on RELEASED(gen): grant the low-power worker (3.3 V sensors only);
//       when Wi-Fi reports its radio off: LOW_POWER.
//   LOW_POWER --(power high for returnStableMs)--> TRANSITION_TO_NORMAL:
//       quiesce the low-power worker;
//       on RELEASED(gen): restore the 5 V rail, grant the normal worker (all
//       sensors), start Wi-Fi: NORMAL.
//       Power lost again before the grant aborts the return (back to the
//       low-power path).
//   A missing RELEASED after ackTimeoutMs, or Wi-Fi not off after
//   networkAckTimeoutMs, reports DEGRADED. The machine keeps waiting: it never
//   grants a second owner while the first has not released (§14), and moves
//   on as soon as the late acknowledgement arrives.
//
// Rail validity: the ISR invalidates the rail on every sense edge (also a
// short glitch). In NORMAL the machine restores it after returnStableMs of
// stable power. Stability is measured from the newest rail epoch: an edge the
// ISR saw but this machine did not sample (a LOW-HIGH pulse between two ticks)
// still restarts the timer, and so does a muted ISR (edges unobservable). The
// restore names the epoch it was decided on, so a newer edge is never
```

The opening sequence explains normal-to-low and low-to-normal transitions. Both paths require the old worker's matching release before a new grant. A missing acknowledgement makes progress visibly degraded while preserving the one-owner rule.

Read the handover contract before the state names / continued

src/domain/power_mode.h / lines 29-34

```
29 // acknowledged by mistake. Boot: power present -> normal worker, rail restored once
30 // stable; power absent -> conservative low-power path.
31 //
32 // transitionsEnabled = false (POLICY_POWER_TRANSITIONS_ENABLED 0): the normal
33 // worker keeps every sensor, the rail is kept valid and the mode only reports
34 // the debounced sense level (NORMAL / LOW_POWER), nothing is switched.
```

src/domain/power_mode.h / lines 35-36

```
35 #pragma once
36 #include <stdbool.h>
```

The rail stability interval restarts when history changes, including an edge not directly sampled as low. This prevents a level-only check from accepting a short unseen interruption as uninterrupted stable power.

Distinguish mode, worker and configuration

src/domain/power_mode.h / lines 37-60

```
#include <stdint.h>

#include "domain/types.h"

typedef uint8_t PowerMode;
enum {
  POWER_MODE_UNKNOWN = 0,
  POWER_MODE_NORMAL,
  POWER_MODE_TRANSITION_TO_LOW,
  POWER_MODE_LOW_POWER,
  POWER_MODE_TRANSITION_TO_NORMAL,
  POWER_MODE_DEGRADED
};

typedef uint8_t PowerWorker;
enum { POWER_WORKER_NONE = 0, POWER_WORKER_NORMAL, POWER_WORKER_LOW };

typedef struct PowerModeConfig {
  uint32_t lossDebounceMs;       // power low this long = lost
  (CFG_POWER_LOSS_DEBOUNCE_MS)
  uint32_t returnStableMs;       // power high this long = back
  (CFG_POWER_RETURN_STABLE_MS)
  uint32_t ackTimeoutMs;         // worker RELEASED deadline (CFG_TRANSITION_ACK_MS)
  uint32_t networkAckTimeoutMs;  // Wi-Fi radio off deadline
  (CFG_NETWORK_SHUTDOWN_ACK_MS)
  bool transitionsEnabled;
} PowerModeConfig;
```

PowerMode names the reported operational condition. PowerWorker identifies which acquisition context may own the drivers. PowerModeConfig holds loss, return and acknowledgement budgets plus the transitions enable policy.

Those types are compact aliases but describe different categories. A normal reported mode can coexist briefly with an invalid rail while a glitch is being qualified; do not equate one state name with every electrical fact.

Supply facts and return actions as separate records

src/domain/power_mode.h / lines 61-83

```
61
62 typedef struct PowerModeInputs {
63   MonoMs now;
64   bool externalPower;     // sense level now (true = external 5 V present)
65   bool railValid;         // rail_state validity now
66   uint32_t releasedGen;   // newest generation the lease holder released (0 = none)
67   bool networkOff;        // Wi-Fi acknowledged: radio off
68   uint32_t railEpoch;     // rail_state epoch, read together with railValid
69   bool edgesMissed;       // the sense ISR is muted: edges are not counted now
70 } PowerModeInputs;
71
72 // What PowerCoordinator must do after a step, in this order: quiesce,
73 // restore the rail, grant, Wi-Fi.
74 typedef struct PowerModeActions {
75   PowerWorker quiesce;    // NONE = nothing
76   uint32_t quiesceGen;
77   bool restoreRail;
78   uint32_t restoreEpoch;  // restore only if the rail is still at this epoch
79   PowerWorker grant;      // NONE = nothing
80   uint32_t grantGen;
81   bool wifiStop;
82   bool wifiStart;
83 } PowerModeActions;
```

Inputs contain now, current power level, rail validity/history and completed acknowledgements. Actions contain quiesce, epoch-qualified restore, grant and network requests.

A pure step cannot know whether the requested GPIO or radio operation already occurred. That effect belongs to the service. The record boundary makes the state machine host-testable using invented acknowledgements.

Retain enough history to recognise a later acknowledgement

src/domain/power_mode.h / lines 84-103

```
84
85 typedef struct PowerModeFsm {
86   PowerModeConfig cfg;
87   PowerMode mode;
88   uint8_t phase;          // private
89   PowerWorker owner;      // lease holder (NONE while handing over)
90   PowerWorker target;     // worker to grant once the owner has released
91   uint32_t gen;           // generation of the current / pending lease
92   bool netPending;        // waiting for Wi-Fi radio off
93   bool haveLow, haveHigh;
94   uint32_t lastEpoch;     // rail epoch seen on the previous step
95   MonoMs lowSinceMs, highSinceMs;
96   MonoMs ackDeadlineMs, netDeadlineMs;
97 } PowerModeFsm;
98
99 void powerModeInit(PowerModeFsm* f, const PowerModeConfig* cfg);
100 void powerModeStep(PowerModeFsm* f, const PowerModeInputs* in, PowerModeActions* out);
101
102 // Uppercase name ("NORMAL", ...). The contract sends it lowercased.
103 const char* powerModeName(PowerMode m);
```

The FSM keeps current owner, target, generation, phase, high/low start times and deadlines. phase describes which protocol step is pending, while mode communicates the broader condition.

Separating those fields lets DEGRADED retain a pending quiescence rather than destroying the information needed to continue when the late release finally arrives. The next file implements that behaviour explicitly.

Start a stability interval only from observed history

Read this as a cooperative protocol, not as a chain of delays. Each helper handles a short part of one step and returns requested actions.

Use the earlier generation 7-to 8 diagram while following the ownership updates.

src/domain/power_mode.c / lines 1-28

```
#include "domain/power_mode.h"

#include <string.h>

enum {
  PHASE_BOOT = 0,
  PHASE_STEADY,     // one owner, nothing pending (Wi-Fi may still be pending in LOW:
  see NET_WAIT)
  PHASE_QUIESCING,  // waiting for the owner's RELEASED
  PHASE_NET_WAIT,   // low-power worker granted, waiting for Wi-Fi radio off
  PHASE_REPORT      // transitions disabled: report the level only
};

void powerModeInit(PowerModeFsm* f, const PowerModeConfig* cfg) {
  memset(f, 0, sizeof(*f));
  f->cfg = *cfg;
  f->mode = POWER_MODE_UNKNOWN;
  f->phase = PHASE_BOOT;
}

static void track(PowerModeFsm* f, const PowerModeInputs* in) {
  // A new epoch is an edge, even one this machine never sampled LOW.
  if (in->railEpoch != f->lastEpoch || in->edgesMissed) f->haveHigh = false;
  f->lastEpoch = in->railEpoch;
  if (in->externalPower) {
    f->haveLow = false;
    if (!f->haveHigh && !in->edgesMissed) {  // stability counts from the first watched
  tick
      f->haveHigh = true;
      f->highSinceMs = in->now;
```

Init clears the machine, copies budgets and begins in BOOT. track notices a changed epoch or missed-edge condition and cancels uninterrupted-high history. It then records the first watched high or low tick.

Start a stability interval only from observed history / continued

src/domain/power_mode.c / lines 29-34

```
    }
  } else {
    f->haveHigh = false;
    if (!f->haveLow) {
      f->haveLow = true;
      f->lowSinceMs = in->now;
```

src/domain/power_mode.c / lines 35-37

```
    }
  }
}
```

A repeatedly high input does not restart highSince every pass. A genuinely new edge does. That difference makes elapsed stability measurable instead of permanently postponed or falsely accumulated.

Record an intended action and its matching deadline

src/domain/power_mode.c / lines 38-65

```
38
39 static bool powerLost(const PowerModeFsm* f, MonoMs now) {
40   return f->haveLow && now - f->lowSinceMs >= f->cfg.lossDebounceMs;
41 }
42
43 static bool powerStable(const PowerModeFsm* f, MonoMs now) {
44   return f->haveHigh && now - f->highSinceMs >= f->cfg.returnStableMs;
45 }
46
47 static void restore(const PowerModeInputs* in, PowerModeActions* out) {
48   out->restoreRail = true;
49   out->restoreEpoch = in->railEpoch;
50 }
51
52 static void grant(PowerModeFsm* f, PowerWorker w, PowerModeActions* out) {
53   ++f->gen;
54   f->owner = w;
55   out->grant = w;
56   out->grantGen = f->gen;
57 }
58
59 static void startQuiesce(PowerModeFsm* f, PowerWorker target, MonoMs now,
 >   PowerModeActions* out) {
60   out->quiesce = f->owner;
61   out->quiesceGen = f->gen;
62   f->target = target;
63   f->ackDeadlineMs = now + f->cfg.ackTimeoutMs;
64   f->phase = PHASE_QUIESCING;
65 }
```

powerLost and powerStable compare elapsed intervals with their separate limits. restore names the current epoch. grant advances generation and names the new worker in both state and output.

startQuiesce records the current owner/generation, target and acknowledgement deadline, then enters the waiting phase. It does not call acquisitionRequestQuiesce itself. The task executes that returned action afterward.

Wait for radio shutdown without revoking the active low worker

src/domain/power_mode.c / lines 66-84

```
66
67 static void enterLowPath(PowerModeFsm* f, MonoMs now, PowerModeActions* out) {
68   out->wifiStop = true;
69   f->netPending = true;
70   f->netDeadlineMs = now + f->cfg.networkAckTimeoutMs;
71   f->mode = POWER_MODE_TRANSITION_TO_LOW;
72 }
73
74 // The low-power worker holds the lease: LOW_POWER once Wi-Fi is off.
75 static void settleLow(PowerModeFsm* f, const PowerModeInputs* in) {
76   if (f->netPending && in->networkOff) f->netPending = false;
77   if (!f->netPending) {
78     f->mode = POWER_MODE_LOW_POWER;
79     f->phase = PHASE_STEADY;
80   } else {
81     f->phase = PHASE_NET_WAIT;
82     if (in->now > f->netDeadlineMs) f->mode = POWER_MODE_DEGRADED;  // radio not
 >   confirmed off
83   }
84 }
```

enterLowPath requests Wi-Fi stop and starts its acknowledgement deadline. settleLow clears the pending network condition once networkOff is observed; otherwise it remains NET_WAIT and eventually reports DEGRADED.

The low-power acquisition worker can already own its permitted sensors while network shutdown is pending. One action's slow acknowledgement does not justify giving another worker concurrent bus access.

Choose an explicit boot and disabled-transition policy

src/domain/power_mode.c / lines 85-112

```
85
86 static void stepBoot(PowerModeFsm* f, const PowerModeInputs* in, PowerModeActions* out)
 > {
87   if (!f->cfg.transitionsEnabled) {
88     grant(f, POWER_WORKER_NORMAL, out);
89     if (!in->railValid) restore(in, out);
90     f->mode = in->externalPower ? POWER_MODE_NORMAL : POWER_MODE_LOW_POWER;
91     f->phase = PHASE_REPORT;
92     return;
93   }
94   if (in->externalPower) {
95     grant(f, POWER_WORKER_NORMAL, out);  // the rail is restored once power is stable
96     f->mode = POWER_MODE_NORMAL;
97     f->phase = PHASE_STEADY;
98     return;
99   }
100  // Unknown or absent power at boot: conservative low-power policy (§13).
101  grant(f, POWER_WORKER_LOW, out);
102  enterLowPath(f, in->now, out);
103  settleLow(f, in);
104 }
105
106 static void stepReport(PowerModeFsm* f, const PowerModeInputs* in, PowerModeActions*
 >  out) {
107  if (!in->railValid) restore(in, out);  // no ISR in this mode; keep 5 V sensing on
108  if (powerLost(f, in->now)) {
109    f->mode = POWER_MODE_LOW_POWER;
110  } else if (in->externalPower) {
111    f->mode = POWER_MODE_NORMAL;
112  }
```

With transitions disabled, boot grants normal acquisition and enters report-only behaviour. With transitions enabled, present power grants normal while absent/unknown power takes the low path.

src/domain/power_mode.c / lines 113-113

```
113 }
```

Report mode updates the reported level without handing acquisition between workers. It keeps logical rail validity according to that policy. This is a configuration alternative, not a test of physical battery switching.

Begin a handover only after the relevant qualification

src/domain/power_mode.c / lines 114-129

```
114
115 static void stepSteady(PowerModeFsm* f, const PowerModeInputs* in, PowerModeActions*
  >   out) {
116   if (f->owner == POWER_WORKER_NORMAL) {
117     if (powerLost(f, in->now)) {
118       enterLowPath(f, in->now, out);
119       startQuiesce(f, POWER_WORKER_LOW, in->now, out);
120     } else if (!in->railValid && powerStable(f, in->now)) {
121       restore(in, out);  // a glitch the ISR saw, now stable again
122     }
123     return;
124   }
125   if (powerStable(f, in->now)) {  // owner LOW
126     f->mode = POWER_MODE_TRANSITION_TO_NORMAL;
127     startQuiesce(f, POWER_WORKER_NORMAL, in->now, out);
128   }
129 }
```

A normal owner sees a debounced loss, requests network stop and starts quiescence toward low power. A transient invalid rail can instead be restored after stable return without changing the owner.

A low owner begins normal return only after high stability. The machine records its intended target and waits; it does not assume the currently sleeping normal worker has permission merely because external power returned.

A late release remains a pending release

src/domain/power_mode.c / lines 130-157

```
130
131 static void stepQuiescing(PowerModeFsm* f, const
  >   PowerModeInputs* in, PowerModeActions* out) {
132   // A new loss before the normal worker is granted aborts
  >   the return (§13).
133   if (f->target == POWER_WORKER_NORMAL &&
  >   !in->externalPower) {
134     f->target = POWER_WORKER_LOW;
135     f->mode = POWER_MODE_TRANSITION_TO_LOW;
136     // Wi-Fi was never restarted on this path: it is still
  >   off.
137   }
138   if (in->releasedGen != f->gen) {
139     if (in->now > f->ackDeadlineMs) f->mode =
  >   POWER_MODE_DEGRADED;  // keep waiting: never two owners
140     return;
141   }
142   f->owner = POWER_WORKER_NONE;
143   if (f->target == POWER_WORKER_NORMAL) {
144     // A glitch during the hand-over restarted the timer:
  >   stepSteady restores
145     // the rail once power has been stable again.
146     if (powerStable(f, in->now)) restore(in, out);
147     grant(f, POWER_WORKER_NORMAL, out);
148     out->wifiStart = true;
149     f->netPending = false;
150     f->mode = POWER_MODE_NORMAL;
151     f->phase = PHASE_STEADY;
152     return;
153   }
154   grant(f, POWER_WORKER_LOW, out);
155   f->mode = POWER_MODE_TRANSITION_TO_LOW;
156   settleLow(f, in);
157 }
```

A renewed loss before normal grant changes the target back to low. A releasedGen mismatch keeps waiting and can mark the mode degraded after the deadline. It never treats timeout as completed cleanup.

After a matching release, the machine records no owner, optionally restores a still-qualified epoch and grants the selected worker with a new generation. Normal return requests Wi-Fi start; low return settles its outstanding network state.

COMMON PITFALL

A matching number is evidence only under the declared generation protocol. Keep its counter range and wrap assumptions in mind for extreme runtimes.

Dispatch one phase and expose a mode name

src/domain/power_mode.c / lines 158-181

```
158
159 static void stepNetWait(PowerModeFsm* f, const PowerModeInputs* in, PowerModeActions*
  >   out) {
160   if (powerStable(f, in->now)) {  // power back before Wi-Fi confirmed: return anyway
161     f->netPending = false;
162     f->mode = POWER_MODE_TRANSITION_TO_NORMAL;
163     startQuiesce(f, POWER_WORKER_NORMAL, in->now, out);
164     return;
165   }
166   settleLow(f, in);
167 }
168
169 void powerModeStep(PowerModeFsm* f, const PowerModeInputs* in, PowerModeActions* out) {
170   memset(out, 0, sizeof(*out));
171   track(f, in);
172   switch (f->phase) {
173     case PHASE_BOOT: stepBoot(f, in, out); break;
174     case PHASE_REPORT: stepReport(f, in, out); break;
175     case PHASE_STEADY: stepSteady(f, in, out); break;
176     case PHASE_QUIESCING: stepQuiescing(f, in, out); break;
177     case PHASE_NET_WAIT: stepNetWait(f, in, out); break;
178     default: break;
179   }
180 }
181
```

NET_WAIT can begin normal return if power stabilises before shutdown is confirmed. Step clears the action record, updates level/history and dispatches exactly the current phase. No previous action bits leak into this pass.

src/domain/power_mode.c / lines 182-191

```
182 const char* powerModeName(PowerMode m) {
183   switch (m) {
184     case POWER_MODE_NORMAL: return "NORMAL";
185     case POWER_MODE_TRANSITION_TO_LOW: return "TRANSITION_TO_LOW";
186     case POWER_MODE_LOW_POWER: return "LOW_POWER";
187     case POWER_MODE_TRANSITION_TO_NORMAL: return "TRANSITION_TO_NORMAL";
188     case POWER_MODE_DEGRADED: return "DEGRADED";
189     default: return "UNKNOWN";
190   }
191 }
```

The name helper is display-only. A mode string should be accompanied by the actual rail and transition diagnostics when assessing why a particular observation or actuator is unavailable.

Expose initialisation and observation rather than transition internals

The coordinator is the RTOS and hardware wrapper around the pure machine.

Other services inspect a copied mode or its task handle; they do not edit the FSM.

src/services/power_coordinator.h / lines 1-28

```
// PowerCoordinator: power-mode state machine and acquisition lease owner.
// Work package WP9 (Stage 6). Architecture §13, §14, §15.
//
// Task "PowerCoord" (cfgTaskSpec: core 0, priority 5, 4 KiB). The decisions
// live in domain/power_mode (host-tested); this service only feeds it and
// carries out its actions:
//   sense    GPIO8 (external 5 V through 220 / 2 x 220 Ohm, measured 25 Oct
//            2026, BOARD_POWER_SENSE_VERIFIED). The edge ISR runs on core 1
//            (the GPIO ISR service is installed there by AcqNormal for the
//            HC-SR04 echo): it invalidates the rail (rail_state, new epoch),
//            disables its own interrupt (a loss chatters 479-928 edges) and
//            notifies this task. The task samples the level every
//            POWER_TICK_MS and re-arms the interrupt once the level has been
//            quiet for CFG_POWER_LOSS_DEBOUNCE_MS.
//   lease    acquisitionRequestQuiesce / acquisitionReleased / acquisitionGrant
//   network  wifiMqttRequestStop / wifiMqttRadioOff / wifiMqttRequestStart
//   rail     railStateRestore (only this task restores validity)
// Debounce: CFG_POWER_LOSS_DEBOUNCE_MS (50 ms, app_config.h) is the mode
// debounce; BOARD_POWER_SENSE_QUALIFY_MS (20 ms) was the bench qualification
// window and is not used here. The measured bounce is 5-14 ms.
//
// Safety stays on core 1 in every mode (D14, 28 Oct 2026): unlike §15, core 1
// is not idle in low power (Safety and SerialLogger keep running there; only
// AcqNormal is blocked). CPU frequency and light sleep are unchanged (§15:
// later decision, measure first).
//
// POLICY_POWER_TRANSITIONS_ENABLED 0 (policy_config.h) is the kill switch:
// no ISR, every sensor on the normal worker, the rail kept valid, and the
```

Init creates the task and input setup. PowerModeCurrent returns a protected mode snapshot. The task handle supports supervision.

Expose initialisation and observation rather than transition internals / continued

src/services/power_coordinator.h / lines 29-38

```
29 // mode only reports the debounced sense level.
30 #pragma once
31 #include <stdbool.h>
32 #include <stdint.h>
33
34 #include "domain/power_mode.h"
35
36 // Starts the task. On failure the normal worker is granted every sensor and
37 // the rail is restored, so sensing runs as before WP9.
38 bool powerCoordinatorInit(void);
```

src/services/power_coordinator.h / lines 39-43

```
39
40 PowerMode powerModeCurrent(void);  // zero wait, any task
41
42 // For Supervisor. NULL before init.
43 void* powerCoordinatorTaskHandle(void);
```

The header links this service to the acquisition lease and radio acknowledgements. Those are internal collaborators of the coordinator, not permission for arbitrary application tasks to invent a grant.

Invalidate immediately and defer slow policy

Follow the urgent edge path first, then the task that qualifies it and executes actions.

GPIO service placement is inherited from normal acquisition's HAL setup.

src/services/power_coordinator.c / lines 1-26

```
#include "services/power_coordinator.h"

#include <Arduino.h>
#include <driver/gpio.h>
#include <esp_task_wdt.h>
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>

#include "app_config.h"
#include "board_profile.h"
#include "diagnostics/logger.h"
#include "diagnostics/progress.h"
#include "hal/clock.h"
#include "policy_config.h"
#include "services/acquisition.h"
#include "services/rail_state.h"
#include "transports/wifi_mqtt.h"

#define POWER_TICK_MS 10u          // level sampling while anything is pending
#define POWER_IDLE_TICK_MS 100u    // steady and armed: the ISR wakes us
#define POWER_HAL_WAIT_MS 2000u    // AcqNormal installs the GPIO ISR service on core 1

static StaticTask_t s_tcb;
static StackType_t s_stack[4 * 1024];  //
  cfgTaskSpec(TASK_ID_POWER_COORDINATOR)->stackBytes
static TaskHandle_t s_task = NULL;

```

The declarations provide task storage, protected reported mode and private FSM. onSenseEdge invalidates rail history, disables further sense interrupts temporarily, updates diagnostic state and notifies the coordinator with the ISR API.

src/services/power_coordinator.c / lines 27-35

```
static portMUX_TYPE s_modeMux = portMUX_INITIALIZER_UNLOCKED;
static PowerMode s_mode = POWER_MODE_UNKNOWN;

static PowerModeFsm s_fsm;  // task-private
static volatile bool s_isrArmed;
static volatile uint32_t s_edges;  // ISR count since boot (diagnostics)

static inline bool senseHigh(void) { return digitalRead(BOARD_PIN_EXTERNAL_POWER_SENSE)
  == HIGH; }

```

Invalidate immediately and defer slow policy / continued

src/services/power_coordinator.c / lines 36-47

```
// Core 1 (the GPIO ISR service lives there). The Arduino service is installed
// without ESP_INTR_FLAG_IRAM, so this never runs while the flash cache is off.
// It does the minimum: invalidate the 5 V epoch, mute itself, wake the task.
static void onSenseEdge(void) {
  railStateInvalidateFromIsr();
  gpio_intr_disable((gpio_num_t)BOARD_PIN_EXTERNAL_POWER_SENSE);
  s_isrArmed = false;
  s_edges = s_edges + 1u;
  BaseType_t woken = pdFALSE;
  if (s_task != NULL) vTaskNotifyGiveFromISR(s_task, &woken);
  portYIELD_FROM_ISR(woken);
}
```

The handler does not perform a lease handover or wait 500ms for stable power. Those operations require repeated task-level decisions and acknowledgements. The source notes that this handler is not installed as universally IRAM-safe.

Apply returned actions in protocol order

src/services/power_coordinator.c / lines 48-72

```
48
49 static void setMode(PowerMode m) {
50   portENTER_CRITICAL(&s_modeMux);
51   const PowerMode old = s_mode;
52   s_mode = m;
53   portEXIT_CRITICAL(&s_modeMux);
54   if (old != m) logPrintf(LOG_LEVEL_INFO, "power: %s -> %s", powerModeName(old),
 >   powerModeName(m));
55 }
56
57 static void execute(const PowerModeActions* a) {
58   if (a->quiesce != POWER_WORKER_NONE) {
59     acquisitionRequestQuiesce(a->quiesceGen);
60     logPrintf(LOG_LEVEL_INFO, "power: quiesce %s worker (gen %lu)", a->quiesce ==
 >   POWER_WORKER_NORMAL ? "normal" : "low",
61               (unsigned long)a->quiesceGen);
62   }
63   if (a->restoreRail) {
64     railStateRestore(a->restoreEpoch);  // a newer edge keeps the rail invalid
65     const RailState rail = railStateRead();
66     if (rail.valid5v) {
67       logPrintf(LOG_LEVEL_INFO, "power: 5 V rail valid (epoch %lu)", (unsigned
 >   long)rail.epoch);
68     } else {
69       logPrintf(LOG_LEVEL_INFO, "power: 5 V restore skipped, epoch %lu -> %lu",
 >   (unsigned long)a->restoreEpoch,
70                 (unsigned long)rail.epoch);
71     }
72   }
```

setMode copies the reported mode under its lock and logs a change after unlocking. execute requests quiescence, performs an epoch-qualified logical restore, grants the selected allowed set and then requests radio changes.

Apply returned actions in protocol order / continued

src/services/power_coordinator.c / lines 73-82

```
  if (a->grant != POWER_WORKER_NONE) {
    const bool normal = a->grant == POWER_WORKER_NORMAL;
    const bool all = normal || !POLICY_POWER_TRANSITIONS_ENABLED;
    acquisitionGrant(a->grant, a->grantGen, all ? SENSOR_MASK_ALL :
  ACQ_LOW_POWER_SENSORS);
    logPrintf(LOG_LEVEL_INFO, "power: grant %s worker (gen %lu)", normal ? "normal" :
  "low",
              (unsigned long)a->grantGen);
  }
  if (a->wifiStop) wifiMqttRequestStop();
  if (a->wifiStart) wifiMqttRequestStart();
}
```

A restore attempt can be refused if a newer epoch appeared. The helper reads rail state again for its diagnostic rather than logging every request as a successful restore. The grant return is not used to drive a separate recovery policy here, so its preconditions matter.

Rearm only after quiet and establish the intended handler core

src/services/power_coordinator.c / lines 83-107

```
83
84 // After a mode change the edge storm is over once the level has held for the
85 // debounce time; only then may the interrupt fire again.
86 static void maybeRearm(bool high, bool* lastHigh, MonoMs* sinceMs, MonoMs now) {
87   if (high != *lastHigh) {
88     *lastHigh = high;
89     *sinceMs = now;
90     return;
91   }
92   if (s_isrArmed || now - *sinceMs < CFG_POWER_LOSS_DEBOUNCE_MS) return;
93   s_isrArmed = true;
94   gpio_intr_enable((gpio_num_t)BOARD_PIN_EXTERNAL_POWER_SENSE);  // on the service's
 >   core (1)
95 }
96
97 static bool armIsr(void) {
98   // The service must already exist on core 1, or attachInterrupt() would
99   // install it on this core (0) and move the HC-SR04 echo ISR next to the radios.
100   for (uint32_t waited = 0; !acquisitionHalsReady(); waited += POWER_TICK_MS) {
101     if (waited >= POWER_HAL_WAIT_MS) return false;
102     vTaskDelay(pdMS_TO_TICKS(POWER_TICK_MS));
103   }
104   attachInterrupt(digitalPinToInterrupt(BOARD_PIN_EXTERNAL_POWER_SENSE), onSenseEdge,
 >   CHANGE);
105   s_isrArmed = true;
106   return true;
107 }
```

maybeRearm watches level stability before enabling another interrupt. armIsr first waits with a bound for normal acquisition to install the shared GPIO service, then attaches the sense handler.

That avoids accidentally installing the shared service on the coordinator's core 0. The bounded wait can fail; the task then falls back to polling, with reduced edge observability that its state-machine inputs must acknowledge.

Step from copied facts, then choose a bounded wake interval

src/services/power_coordinator.c / lines 108-130

```
108
109 static void powerTask(void* arg) {
110   (void)arg;
111   if (esp_task_wdt_add(NULL) != ESP_OK) logPrintf(LOG_LEVEL_ERROR, "power: task watchdog
  >    add failed");
112   bool isr = false;
113   if (POLICY_POWER_TRANSITIONS_ENABLED) {
114     isr = armIsr();
115     if (!isr) logPrintf(LOG_LEVEL_ERROR, "power: GPIO ISR service not ready - polling
  >   the sense pin only");
116   }
117   bool lastHigh = senseHigh();
118   MonoMs sinceMs = monoNowMs();
119   for (;;) {
120     const MonoMs now = monoNowMs();
121     const RailState rail = railStateRead();  // one coherent {valid, epoch} per tick
122     const bool high = senseHigh();
123     const PowerModeInputs in = {now, high, rail.valid5v,
  >   acquisitionReleasedGeneration(), wifiMqttRadioOff(),
124                                 rail.epoch, isr && !s_isrArmed};
125     PowerModeActions a;
126     powerModeStep(&s_fsm, &in, &a);
127     execute(&a);
128     setMode(s_fsm.mode);
129     if (isr) maybeRearm(high, &lastHigh, &sinceMs, now);
130
```

The task subscribes to its watchdog, optionally arms the handler and repeatedly gathers time, rail pair, sense level, release generation and network-off status. It passes missed-edge visibility while the handler is muted.

src/services/power_coordinator.c / lines 131-137

```
131     progressMark(TASK_ID_POWER_COORDINATOR);
132     esp_task_wdt_reset();
133     const bool busy = s_fsm.mode != POWER_MODE_NORMAL && s_fsm.mode !=
  >   POWER_MODE_LOW_POWER;
134     const bool settling = !high || !in.railValid || (isr && !s_isrArmed);
135     ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(busy || settling ? POWER_TICK_MS :
  >   POWER_IDLE_TICK_MS));
136   }
137 }
```

After executing actions and publishing mode, it rearms as appropriate, marks progress and waits. Pending/settling paths get 10ms polling; settled operation can wait longer because an edge notification prompts earlier inspection.

Configure and create before exposing a running coordinator

src/services/power_coordinator.c / lines 138-164

```
138
139 bool powerCoordinatorInit(void) {
140   if (s_task != NULL) return true;
141   pinMode(BOARD_PIN_EXTERNAL_POWER_SENSE, INPUT);  // no pull: the 440 Ohm leg defines
  >   LOW
142   const PowerModeConfig cfg = {CFG_POWER_LOSS_DEBOUNCE_MS, CFG_POWER_RETURN_STABLE_MS,
  >   CFG_TRANSITION_ACK_MS,
143                                CFG_NETWORK_SHUTDOWN_ACK_MS,
  >   POLICY_POWER_TRANSITIONS_ENABLED != 0};
144   powerModeInit(&s_fsm, &cfg);
145
146   const TaskSpec* spec = cfgTaskSpec(TASK_ID_POWER_COORDINATOR);
147   if (spec->stackBytes != sizeof(s_stack)) {
148     logPrintf(LOG_LEVEL_WARN, "power: stack %u B, task table says %lu B",
  >   (unsigned)sizeof(s_stack),
149               (unsigned long)spec->stackBytes);
150   }
151   s_task = xTaskCreateStaticPinnedToCore(powerTask, spec->name, sizeof(s_stack), NULL,
  >   spec->priority, s_stack,
152                                          &s_tcb, spec->core);
153   if (s_task == NULL) {
154     // Never leave sensing without an owner: run as before WP9.
155     railStateRestore(railStateRead().epoch);
156     acquisitionGrant(POWER_WORKER_NORMAL, 1, SENSOR_MASK_ALL);
157     return false;
158   }
159   logPrintf(LOG_LEVEL_INFO, "power: started (core %u, prio %u), external power %s,
  >   transitions %s",
160             (unsigned)spec->core, (unsigned)spec->priority, senseHigh() ? "present" :
  >   "ABSENT",
161             POLICY_POWER_TRANSITIONS_ENABLED ? "on" : "OFF (kill switch)");
162   return true;
163 }
164
```

Configure and create before exposing a running coordinator / continued

Init sets the sense pin as a plain input, builds the pure configuration from policy and timing constants and creates a static pinned task. ModeCurrent copies under the mode portMUX.

src/services/power_coordinator.c / lines 165-172

```
165 PowerMode powerModeCurrent(void) {
166   portENTER_CRITICAL(&s_modeMux);
167   const PowerMode m = s_mode;
168   portEXIT_CRITICAL(&s_modeMux);
169   return m;
170 }
171
172 void* powerCoordinatorTaskHandle(void) { return s_task; }
```

The software input qualification does not add analog hysteresis to the divider. The hardware sensing behaviour still needs the separate measurement tool we will read next.

Translate the measurement procedure into explicit limits

This larger experiment measures a shared digital/analog sense pin and records transitions. It is a complete diagnostic program with its own samplers and history, not a production sensor driver.

Read it as an example of collecting evidence before trusting a board-profile verification flag.

test/bringup/power_sense_main.c / lines 1-24

```
// Bring-up entry point for the external-power sense (WP9 gate, §13, §20).
//   pio run -e test_power_sense -t upload -t monitor
// Ext. 5 V + rail -> 220 Ohm -> GPIO8 (ADC1_CH7) -> 2 x 220 Ohm -> GND.
// Nominal 3.33 V at 5.00 V, 0 V on battery. No hysteresis on the divider.
// Replaces src/main.c in this env only. No production driver is involved: this
// is the commissioning tool that decides BOARD_POWER_SENSE_VERIFIED.
//
// GPIO8 is read two ways at the same time:
//  - digital: CHANGE interrupt, every edge timestamped in us (esp_timer);
//  - analog: ADC1_CH7 sampled every 1 ms into an 8 s ring buffer.
// The ADC is configured once through the IDF driver and read with
// adc1_get_raw(). Arduino's analogRead*() is NOT used: each call does
// pinMode(pin, ANALOG), which disables the digital input buffer and leaves the
// interrupt deaf. The status line checks that both paths still agree
// ("isr" and "mismatch" counters); if they do not, press 'a' for
// digital-only mode and repeat the steps.
//
// The ADC clips at ~3050 mV (11 dB), so it cannot measure the 3.33 V high
// level. It only shows ">= VIH (clipped)". The exact high level needs a
// multimeter at GPIO8.
//
// Output: a MODE banner (NORMAL / LOW-POWER) each time the level has been
// stable for BOARD_POWER_SENSE_QUALIFY_MS, as the WP9 power task will decide,
// then indented result / timing / bounce lines once the pin has settled.
```

COMMON PITFALL

The 1ms sample request depends on the actual tick configuration and scheduling. Treat 1kHz as a target to verify, not a property of the source comment.

Translate the measurement procedure into explicit limits / continued

test/bringup/power_sense_main.c / lines 25-28

```
25 // A heartbeat repeats the mode every 30 s; problems print as "WARNING".
26 //
27 // Procedure:
28 //  1. External 5 V plugged in: banner "MODE: NORMAL". With a multimeter,
```

test/bringup/power_sense_main.c / lines 29-54

```
29 //     GPIO8 must read >= 2.475 V and <= 3.6 V (VDD + 0.3 V).
30 //  2. Unplug the external 5 V: banner "MODE: LOW-POWER", then result PASS.
31 //  3. Plug it back in: banner "MODE: NORMAL", then result PASS.
32 //  4. Repeat 2-3 at least 5 times, once with the sensors working (pH, TDS,
33 //     HC-SR04 loaded) and once by pulling the cable slowly / wiggling it.
34 //  5. Press 'd' for the table of all transitions and paste it back.
35 // Keys: v = edge details on/off, d = transition table, a = ADC on/off,
36 // r = reset counters, h = help.
37 // The monitor opens with DTR/RTS released (platformio.ini), so reconnecting
38 // after an unplug does not reset the board and the RAM history survives.
39 #include <Arduino.h>
40 #include <driver/adc.h>
41 #include <esp_adc_cal.h>
42 #include <esp_timer.h>
43 #include <fcntl.h>
44 #include <soc/gpio_reg.h>
45 #include <stdio.h>
46 #include <string.h>
47 #include <unistd.h>
48
49 #include "board_profile.h"
50 #include "hal/arduino_entry.h"
51
52 #define PS_PIN BOARD_PIN_EXTERNAL_POWER_SENSE
53 #define PS_ADC_CHANNEL ADC1_CHANNEL_7
54
```

The introductory procedure compares digital edges, calibrated ADC samples and an external meter. The constants define logical bands, clipping, ring capacity, pre-trigger history, quiet time and settling thresholds.

test/bringup/power_sense_main.c / lines 55-64

```
55 // ESP32-S3 input thresholds at VDD = 3.3 V (architecture §5, [L14]).
56 #define PS_VIH_MV 2475u
57 #define PS_VIL_MV 825u
58 #define PS_VMAX_MV 3600u      // VDD + 0.3 V
59 #define PS_CLIP_MV 3050u      // same ceiling as hal/adc_channel.c
60 #define PS_ZERO_MV 50u        // "fully discharged"
61
62 #define PS_SAMPLE_PERIOD_MS 1
63 #define PS_SAMPLE_RING 8192u      // 8.2 s at 1 kHz, power of two
64 #define PS_EDGE_RING 1024u        // power of two
```

Translate the measurement procedure into explicit limits / continued

test/bringup/power_sense_main.c / lines 65-73

```
#define PS_PRE_TRIGGER_MS 300u    // ADC history kept before the trigger
#define PS_EDGE_QUIET_MS 300u     // no edge for this long = digital settled
#define PS_STABLE_WINDOW_MS 200u  // ADC span below PS_STABLE_SPAN_MV = settled
#define PS_STABLE_SPAN_MV 30u
#define PS_TRIGGER_DELTA_MV 200u  // ADC move that opens a transition
#define PS_TRANSITION_TIMEOUT_MS 7000u
#define PS_STATUS_PERIOD_MS 30000u  // heartbeat
#define PS_EDGES_LISTED 12u
#define PS_HISTORY 48u
```

A clipped ADC high is only a lower bound. It cannot measure the exact 3.33V level described by the nominal divider. Keep the meter observation separate from the digital and ADC evidence.

Store sample evidence before deriving a transition

test/bringup/power_sense_main.c / lines 74-101

```
#define PS_QUALIFY_US (BOARD_POWER_SENSE_QUALIFY_MS * 1000u)

typedef struct Sample {
  uint32_t tUs;    // esp_timer low 32 bits (wrap-safe for intervals < 71 min)
  uint16_t mv;     // 0 when the ADC is off
  uint8_t before;  // GPIO level just before the conversion
  uint8_t after;   // GPIO level just after the conversion
} Sample;

typedef struct Edge {
  uint32_t tUs;
  uint8_t level;  // level read at ISR entry
} Edge;

typedef struct Transition {
  bool rising;
  uint32_t tStartUs;      // first edge or ADC departure
  uint32_t edges;         // ISR edges in the transition
  uint32_t bounceUs;      // first edge -> last edge
  uint32_t minGapUs;      // shortest interval between two edges
  uint8_t endLevel;
  bool adcValid;
  uint16_t mvBefore, mvAfter;
  int32_t tVihMs, tVilMs, tEndMs;  // from ADC departure; -1 = not reached
  uint16_t flipMinMv, flipMaxMv;   // ADC value where the sampler saw GPIO flip
  bool timedOut;
  bool dip;     // ADC excursion that recovered with no GPIO edge
  bool outage;  // GPIO left its level and came back before the pin settled
```

Sample records time, voltage and digital levels immediately before/after conversion. Edge records an ISR timestamp and level. Transition retains derived direction, timing, bounce and classification.

test/bringup/power_sense_main.c / lines 102-114

```
} Transition;

static Sample s_samples[PS_SAMPLE_RING];
static volatile uint32_t s_sampleCount;
static volatile bool s_adcOn = true;

static Edge s_edges[PS_EDGE_RING];
static volatile uint32_t s_edgeHead;   // written by the ISR only
static volatile uint32_t s_edgeTotal;  // never wraps the ring
static volatile uint32_t s_lastEdgeUs;
static uint32_t s_edgeTail;
static bool s_verbose;

```

Store sample evidence before deriving a transition / continued

test/bringup/power_sense_main.c / lines 115-123

```
115 static esp_adc_cal_characteristics_t s_cal;
116
117 // Health of the shared-pin setup.
118 static volatile uint32_t s_samplerFlips;  // level changes seen by the sampler
119 static volatile uint32_t s_adcDisturbed;  // level differs before/after a conversion
120 static uint32_t s_mismatch;               // steady GPIO level disagrees with mV
121
122 static Transition s_history[PS_HISTORY];
123 static uint32_t s_historyCount;
```

The sample and edge rings have different writers. Head/count fields publish progress, while the analyzer maintains its own tail and history. These are specialised concurrent diagnostic assumptions; volatile alone is not a general portable ring-buffer proof.

Capture a short edge record and calibrate an analog sample

test/bringup/power_sense_main.c / lines 124-142

```
124
125 static inline uint8_t gpioLevel(void) {
126   return (uint8_t)((REG_READ(GPIO_IN_REG) >> PS_PIN) & 1u);
127 }
128
129 static void IRAM_ATTR onEdge(void) {
130   const uint32_t head = s_edgeHead;
131   s_edges[head & (PS_EDGE_RING - 1u)].tUs = (uint32_t)esp_timer_get_time();
132   s_edges[head & (PS_EDGE_RING - 1u)].level = gpioLevel();
133   s_lastEdgeUs = s_edges[head & (PS_EDGE_RING - 1u)].tUs;
134   s_edgeHead = head + 1u;
135   s_edgeTotal++;
136 }
137
138 static uint16_t adcReadMv(void) {
139   const int raw = adc1_get_raw(PS_ADC_CHANNEL);
140   if (raw < 0) return 0;
141   return (uint16_t)esp_adc_cal_raw_to_voltage((uint32_t)raw, &s_cal);
142 }
```

gpioLevel uses a register read. The handler fills one power-of-two ring slot, records last time and advances the head after writing the item. adcReadMv uses the configured IDF ADC path and calibration record.

This experiment avoids the Arduino analog helper because its pad-mode changes could disturb the simultaneous digital input. The self-check later verifies that required coexistence rather than assuming two API calls leave the pin unchanged.

Observe the digital input around each conversion

test/bringup/power_sense_main.c / lines 143-162

```
143
144 // ---- sampler (core 0): 1 kHz ADC + GPIO level -------------------------------
145 static void samplerTask(void* arg) {
146   (void)arg;
147   TickType_t lastWake = xTaskGetTickCount();
148   uint8_t prev = gpioLevel();
149   for (;;) {
150     vTaskDelayUntil(&lastWake, pdMS_TO_TICKS(PS_SAMPLE_PERIOD_MS));
151     Sample s;
152     s.tUs = (uint32_t)esp_timer_get_time();
153     s.before = gpioLevel();
154     s.mv = s_adcOn ? adcReadMv() : 0;
155     s.after = gpioLevel();
156     if (s.before != s.after) s_adcDisturbed++;
157     if (s.after != prev) s_samplerFlips++;
158     prev = s.after;
159     s_samples[s_sampleCount & (PS_SAMPLE_RING - 1u)] = s;
160     s_sampleCount = s_sampleCount + 1u;
161   }
162 }
```

The sampler waits on an intended periodic grid, records time and before/after GPIO levels, takes an ADC value if enabled and updates disturbance/flip counts. It writes the sample before advancing sampleCount.

A changed digital level during conversion may be a real transition or interference to investigate; the counters make that relationship observable. They do not automatically identify its physical cause.

Retrieve bounded history and summarise a time window

test/bringup/power_sense_main.c / lines 163-190

```
163
164 // ---- analysis helpers (analyzer task only) -----------------------------------
165 static const Sample* sampleAt(uint32_t index) {
166   return &s_samples[index & (PS_SAMPLE_RING - 1u)];
167 }
168
169 // Index of the first sample taken at or after tUs, searching back from 'last'.
170 static uint32_t sampleIndexAt(uint32_t tUs, uint32_t last) {
171   uint32_t i = last;
172   const uint32_t oldest = (last >= PS_SAMPLE_RING - 16u) ? last - (PS_SAMPLE_RING - 16u)
  >    : 0u;
173   while (i > oldest && (int32_t)(sampleAt(i - 1u)->tUs - tUs) >= 0) --i;
174   return i;
175 }
176
177 static void spanMv(uint32_t from, uint32_t to, uint16_t* lo, uint16_t* hi, uint32_t*
  >   mean) {
178   uint16_t mn = 0xFFFF, mx = 0;
179   uint64_t sum = 0;
180   for (uint32_t i = from; i < to; ++i) {
181     const uint16_t v = sampleAt(i)->mv;
182     if (v < mn) mn = v;
183     if (v > mx) mx = v;
184     sum += v;
185   }
186   const uint32_t n = to > from ? to - from : 1u;
187   if (lo) *lo = (to > from) ? mn : 0;
188   if (hi) *hi = mx;
189   if (mean) *mean = (uint32_t)(sum / n);
190 }
```

sampleAt masks a logical index into the ring. sampleIndexAt searches backward within retained history using bounded wrap-aware time differences. spanMv computes minimum, maximum and mean for an index interval.

The helper uses a 64-bit sum and avoids division by zero for an empty range. The caller must ensure the logical samples still belong to retained history while reading; modulo indexing alone does not prove a slot has not been overwritten.

Label voltage bands and elapsed intervals without changing evidence

test/bringup/power_sense_main.c / lines 191-216

```
191
192 static const char* bandName(uint32_t mv) {
193   if (mv >= PS_VIH_MV) return "HIGH";
194   if (mv <= PS_VIL_MV) return "LOW";
195   return "UNDEFINED";
196 }
197
198 static int32_t msSince(uint32_t fromUs, uint32_t toUs) {
199   return (int32_t)((toUs - fromUs) / 1000u);
200 }
201
202 // "mm:ss.s" since boot, for the banners (the monitor may not add timestamps).
203 static void printUptime(void) {
204   const uint32_t ds = (uint32_t)(esp_timer_get_time() / 100000);
205   printf("%02lu:%02lu.%lu", (unsigned long)(ds / 600u), (unsigned long)(ds / 10u % 60u),
206          (unsigned long)(ds % 10u));
207 }
208
209 static void printModeBanner(uint8_t level) {
210   printf("\n+-------------------------------------------------------------+\n");
211   printf("|  MODE: %-10s %-27s ", level ? "NORMAL" : "LOW-POWER",
212          level ? "external 5 V present" : "external 5 V LOST");
213   printUptime();
214   printf("  |\n+-------------------------------------------------------------+\n");
215 }
216
```

bandName classifies voltage against the selected logical thresholds. msSince converts a bounded microsecond difference. The printing helpers display uptime, qualified mode and either a reached time or the negative not-reached sentinel.

test/bringup/power_sense_main.c / lines 217-224

```
217 // "after N ms", or "not reached" for -1.
218 static void printAfter(int32_t ms) {
219   if (ms < 0) {
220     printf("not reached");
221   } else {
222     printf("after %ld ms", (long)ms);
223   }
224 }
```

These helpers do not decide the production FSM's current owner. They format this experiment's observations. Keep a diagnostic banner distinct from a completed production handover.

State exactly which checks earn the printed result

test/bringup/power_sense_main.c / lines 225-252

```
225
226 static void printSummary(const Transition* t) {
227   bool pass = t->endLevel == (t->rising ? 1 : 0);
228   if (t->adcValid) pass = pass && (t->rising ? t->mvAfter >= PS_VIH_MV : t->mvAfter <=
  >   PS_VIL_MV);
229
230   printf("  result  ");
231   if (!t->adcValid) {
232     printf("ADC off");
233   } else if (t->mvAfter >= PS_CLIP_MV) {
234     printf("pin %u -> >=%u mV (clipped)", t->mvBefore, PS_CLIP_MV);
235   } else if (t->mvBefore >= PS_CLIP_MV) {
236     printf("pin >=%u -> %u mV", PS_CLIP_MV, t->mvAfter);
237   } else {
238     printf("pin %u -> %u mV", t->mvBefore, t->mvAfter);
239   }
240   printf(", GPIO %u   %s\n", t->endLevel, pass ? "PASS" : "FAIL");
241   if (!pass) {
242     printf("  WARNING: expected GPIO %u and %s\n", t->rising ? 1 : 0,
243            t->rising ? "pin >= 2475 mV" : "pin <= 825 mV (rail back-fed or not switched
  >   off?)");
244   }
245
246   if (t->adcValid && t->rising) {
247     printf("  timing  > %u mV ", PS_VIH_MV);
248     printAfter(t->tVihMs);
249     printf("\n");
250   } else if (t->adcValid) {
251     printf("  timing  < %u mV ", PS_VIH_MV);
252     printAfter(t->tVihMs);
```

The summary compares final digital level and, when available, its ADC band. It reports clipping honestly, prints threshold crossing times and compares bounce duration with the qualification window.

test/bringup/power_sense_main.c / lines 253-263

```
253     printf(", < %u mV ", PS_VIL_MV);
254     printAfter(t->tVilMs);
255     if (t->tEndMs >= 0) {
256       printf(", ~0 V ");
257       printAfter(t->tEndMs);
258     } else {
259       printf(", settled at %u mV", t->mvAfter);
260     }
261     printf("\n");
262   }
263
```

State exactly which checks earn the printed result / continued

test/bringup/power_sense_main.c / lines 264-279

```
264   printf("  bounce  %lu edge(s) in ", (unsigned long)t->edges);
265   if (t->bounceUs < 1000u) {
266     printf("%lu us", (unsigned long)t->bounceUs);
267   } else {
268     printf("%lu.%lu ms", (unsigned long)(t->bounceUs / 1000u),
269            (unsigned long)(t->bounceUs % 1000u / 100u));
270   }
271   if (t->bounceUs >= PS_QUALIFY_US) {
272     printf("\n  WARNING: longer than the %lu ms qualify window\n",
273            (unsigned long)BOARD_POWER_SENSE_QUALIFY_MS);
274   } else {
275     printf(" -> filtered by the %lu ms qualify window\n",
276            (unsigned long)BOARD_POWER_SENSE_QUALIFY_MS);
277   }
278   if (t->timedOut) printf("  WARNING: pin did not settle within 7 s\n");
279 }
```

PASS here means these selected sense-input checks passed. It does not validate every power rail, battery switchover or the full controller's load behaviour. The bounded transition timeout remains visible as a separate condition.

Derive departure and threshold crossings from retained samples

test/bringup/power_sense_main.c / lines 280-306

```
280
281 // Analyse samples [first, last) and the edges collected for this transition.
282 static void reportTransition(Transition* t, const Edge* edges, uint32_t nListed,
  >   uint32_t first,
283                              uint32_t last) {
284   t->adcValid = s_adcOn;
285   t->tVihMs = t->tVilMs = t->tEndMs = -1;
286   t->flipMinMv = 0xFFFF;
287   t->flipMaxMv = 0;
288
289   uint32_t departUs = t->tStartUs;
290   if (t->adcValid && last > first + 2u * PS_STABLE_WINDOW_MS) {
291     uint32_t meanBefore, meanAfter;
292     spanMv(first, first + PS_PRE_TRIGGER_MS / 2u, NULL, NULL, &meanBefore);
293     spanMv(last - PS_STABLE_WINDOW_MS, last, NULL, NULL, &meanAfter);
294     t->mvBefore = (uint16_t)meanBefore;
295     t->mvAfter = (uint16_t)meanAfter;
296     t->rising = meanAfter > meanBefore;
297
298     // Departure: first sample more than 50 mV away from the level before.
299     uint32_t depart = first;
300     for (uint32_t i = first; i < last; ++i) {
301       const int32_t d = (int32_t)sampleAt(i)->mv - (int32_t)meanBefore;
302       if ((t->rising && d > 50) || (!t->rising && d < -50)) {
303         depart = i;
304         break;
305       }
306     }
```

reportTransition begins with unknown timing sentinels. With sufficient ADC history it averages before/after levels, infers direction, finds the first significant departure and searches later samples for threshold crossings.

test/bringup/power_sense_main.c / lines 307-319

```
307     departUs = sampleAt(depart)->tUs;
308     for (uint32_t i = depart; i < last; ++i) {
309       const Sample* s = sampleAt(i);
310       const int32_t ms = msSince(departUs, s->tUs);
311       if (t->rising) {
312         if (t->tVilMs < 0 && s->mv > PS_VIL_MV) t->tVilMs = ms;
313         if (t->tVihMs < 0 && s->mv >= PS_VIH_MV) t->tVihMs = ms;
314         if (t->tEndMs < 0 && s->mv >= PS_CLIP_MV) t->tEndMs = ms;
315       } else {
316         if (t->tVihMs < 0 && s->mv < PS_VIH_MV) t->tVihMs = ms;
317         if (t->tVilMs < 0 && s->mv <= PS_VIL_MV) t->tVilMs = ms;
318         if (t->tEndMs < 0 && s->mv <= PS_ZERO_MV) t->tEndMs = ms;
319       }
```

Derive departure and threshold crossings from retained samples / continued

test/bringup/power_sense_main.c / lines 320-333

```
      if (i > first && s->before != sampleAt(i - 1u)->after) {
        if (s->mv < t->flipMinMv) t->flipMinMv = s->mv;
        if (s->mv > t->flipMaxMv) t->flipMaxMv = s->mv;
      }
    }
  } else if (t->edges > 0) {
    t->rising = t->endLevel == 1;
  }

  // ADC moved >= PS_TRIGGER_DELTA_MV and came back without a GPIO edge, e.g.
  // contact bounce of the plug. Not a power transition: no verdict.
  if (t->adcValid && t->edges == 0 && !t->timedOut &&
      strcmp(bandName(t->mvBefore), bandName(t->mvAfter)) == 0) {
    uint16_t dipLo, dipHi;
```

It also records voltage near observed digital flips. These times are limited by sampling resolution and available history. The code does not interpolate a precision edge voltage that was never measured.

Distinguish an analog dip from a completed level transition

test/bringup/power_sense_main.c / lines 334-358

```
    spanMv(first, last, &dipLo, &dipHi, NULL);
    printf("  note    pin dipped to %u mV and recovered, GPIO stayed %u -> ignored\n",
           t->endLevel ? dipLo : dipHi, t->endLevel);
    t->tVihMs = t->tVilMs = t->tEndMs = -1;
    t->dip = true;
    s_history[s_historyCount % PS_HISTORY] = *t;
    s_historyCount++;
    return;
  }

  // GPIO edges but the pin ended in the band it started in: power went away
  // and came back (or the reverse) before the pin settled. The mode banners
  // already showed both changes; report it as one short outage, no verdict.
  if (t->adcValid && t->edges > 0 && !t->timedOut &&
      strcmp(bandName(t->mvBefore), bandName(t->mvAfter)) == 0) {
    const bool wasHigh = t->mvBefore >= PS_VIH_MV;
    uint32_t extreme = wasHigh ? 0xFFFFu : 0u;
    int32_t leftMs = -1, backMs = -1;
    for (uint32_t i = sampleIndexAt(departUs, last); i < last; ++i) {
      const Sample* s = sampleAt(i);
      const bool inBand = wasHigh ? s->mv >= PS_VIH_MV : s->mv <= PS_VIL_MV;
      if (wasHigh ? s->mv < extreme : s->mv > extreme) extreme = s->mv;
      if (leftMs < 0 && !inBand) leftMs = msSince(departUs, s->tUs);
      if (leftMs >= 0 && backMs < 0 && inBand) backMs = msSince(departUs, s->tUs);
    }
```

An ADC excursion without an edge that returns to the same band is classified as a dip. Edges followed by return to the original band can instead be reported as a short outage or blip.

Distinguish an analog dip from a completed level transition / continued

test/bringup/power_sense_main.c / lines 359-372

```
359     printf("  result  short %s: pin %s for %ld ms (down to %lu mV), GPIO back to %u
  >   OK\n",
360             wasHigh ? "outage" : "power blip", wasHigh ? "left HIGH" : "left LOW",
361             (leftMs >= 0 && backMs >= 0) ? (long)(backMs - leftMs) : -1L,
362             (unsigned long)extreme, t->endLevel);
363     printf("  bounce  %lu edge(s) over both changes\n", (unsigned long)t->edges);
364     t->outage = true;
365     t->tEndMs = backMs;
366     s_history[s_historyCount % PS_HISTORY] = *t;
367     s_historyCount++;
368     return;
369   }
370
371   printSummary(t);
372   if (s_verbose) {
```

Both branches retain their evidence in bounded history and return before the ordinary summary path. A final same-band value therefore does not erase a brief recorded interruption, echoing the production epoch idea at a diagnostic level.

Attach edge details to the same retained interval

test/bringup/power_sense_main.c / lines 373-391

```
    if (t->adcValid && t->flipMaxMv > 0) {
      printf("  detail  GPIO flipped at %u-%u mV (1 ms sampler)\n", t->flipMinMv,
  >   t->flipMaxMv);
    }
    for (uint32_t i = 0; i < nListed; ++i) {
      const int32_t dt = (int32_t)(edges[i].tUs - departUs);
      const uint32_t si = sampleIndexAt(edges[i].tUs, last);
      printf("  detail  edge %2lu  %+9.3f ms  level %u", (unsigned long)(i + 1u),
             (double)dt / 1000.0, edges[i].level);
      if (t->adcValid && si < last) printf("  ~%u mV", sampleAt(si)->mv);
      printf("\n");
    }
    if (t->edges > nListed) {
      printf("  detail  ... %lu more edge(s), shortest gap %lu us\n",
             (unsigned long)(t->edges - nListed), (unsigned long)t->minGapUs);
    }
  }
  s_history[s_historyCount % PS_HISTORY] = *t;
  s_historyCount++;
}
```

The ordinary path prints the summary and optional edge list relative to departure. It associates each edge with a nearby retained sample when available, reports truncated detail and stores the resulting Transition.

The listed edge array has a finite capacity. The total count can exceed printed entries, so the truncation message prevents a short list from being mistaken for the complete physical history.

Keep bounded history and interactive controls explicit

test/bringup/power_sense_main.c / lines 392-414

```
392
393 static void printTable(void) {
394   printf("\n%lu transition(s); last %u shown\n", (unsigned long)s_historyCount,
395          (unsigned)(s_historyCount < PS_HISTORY ? s_historyCount : PS_HISTORY));
396   printf("  #  dir   edges  bounce_ms  mV_before  mV_after  t_VIH  t_VIL  t_end  gpio
  >   flip_mV\n");
397   const uint32_t from = s_historyCount > PS_HISTORY ? s_historyCount - PS_HISTORY : 0u;
398   for (uint32_t k = from; k < s_historyCount; ++k) {
399     const Transition* t = &s_history[k % PS_HISTORY];
400     printf("%3lu  %s  %5lu  %9.3f  %9u  %8u  %5ld  %5ld  %5ld  %4u  %u-%u%s\n",
401            (unsigned long)(k + 1u), t->dip ? "DIP " : t->outage ? "OUT " : t->rising ?
  >   "RISE" : "FALL",
402            (unsigned long)t->edges,
403            (double)t->bounceUs / 1000.0, t->mvBefore, t->mvAfter, (long)t->tVihMs,
404            (long)t->tVilMs, (long)t->tEndMs, t->endLevel,
405            t->flipMaxMv ? t->flipMinMv : 0, t->flipMaxMv, t->timedOut ? "  TIMEOUT" :
  >   "");
406   }
407   printf("  (t_* in ms from ADC departure; -1 = not reached; mV = 0 when the ADC was
  >   off)\n\n");
408 }
409
410 static void printHelp(void) {
411   printf("keys: v = edge details (%s), d = transition table, a = ADC (%s), r = reset, "
412          "h = help\n", s_verbose ? "on" : "off", s_adcOn ? "on" : "off");
413 }
414
```

PrintTable starts from the oldest retained transition and walks logical history indices. Help describes detail, ADC, table and reset controls. handleKeys uses nonblocking reads so the analyzer does not wait indefinitely for a keystroke.

test/bringup/power_sense_main.c / lines 415-428

```
415 static void handleKeys(void) {
416   char c;
417   while (read(STDIN_FILENO, &c, 1) == 1) {
418     switch (c) {
419       case 'd': printTable(); break;
420       case 'v':
421         s_verbose = !s_verbose;
422         printf("edge details %s\n", s_verbose ? "on" : "off");
423         break;
424       case 'a':
425         s_adcOn = !s_adcOn;
426         printf("ADC %s\n", s_adcOn ? "on (combined mode)" : "off (digital-only mode)");
427         break;
428       case 'r':
```

Keep bounded history and interactive controls explicit / continued

test/bringup/power_sense_main.c / lines 429-440

```
429          s_historyCount = 0;
430          s_mismatch = 0;
431          s_adcDisturbed = 0;
432          s_samplerFlips = 0;
433          s_edgeTotal = 0;
434          printf("counters reset\n");
435          break;
436        case 'h': printHelp(); break;
437        default: break;
438      }
439    }
440  }
```

Resetting shared diagnostic counters is a convenient bench action, not a synchronised production statistics transaction. A concurrent increment may be lost around a reset; interpret totals with that experimental operation in mind.

Qualify a mode while draining new edge evidence

test/bringup/power_sense_main.c / lines 441-463

```
441
442 // ---- analyzer (core 1): groups edges + ADC into transitions -------------------
443 static void analyzerTask(void* arg) {
444   (void)arg;
445   bool active = false;
446   Transition cur;
447   Edge listed[PS_EDGES_LISTED];
448   uint32_t nListed = 0;
449   uint32_t firstSample = 0;
450   uint32_t lastEdgeUs = 0;
451   uint32_t settledMv = 0;
452   bool settledKnown = false;
453   uint32_t lastStatusUs = (uint32_t)esp_timer_get_time();
454   uint32_t edgesAtStatus = 0, flipsAtStatus = 0;
455   int mode = -1;  // qualified GPIO level: 1 = NORMAL, 0 = LOW-POWER, -1 = not yet
456
457   for (;;) {
458     vTaskDelay(pdMS_TO_TICKS(10));
459     handleKeys();
460     const uint32_t nowUs = (uint32_t)esp_timer_get_time();
461     const uint32_t last = s_sampleCount;
462     if (last < PS_PRE_TRIGGER_MS + PS_STABLE_WINDOW_MS) continue;
463
```

The analyzer starts with no qualified mode, waits for sufficient history and checks that the last observed edge has been quiet for the selected interval. It then drains the edge ring, handling overrun by skipping to retained entries.

Qualify a mode while draining new edge evidence / continued

test/bringup/power_sense_main.c / lines 464-480

```
464     // Mode decision as WP9 will make it: act on the level only once no edge
465     // has arrived for BOARD_POWER_SENSE_QUALIFY_MS.
466     const uint8_t pinLevel = gpioLevel();
467     if ((s_edgeTotal == 0 || nowUs - s_lastEdgeUs >= PS_QUALIFY_US) && (int)pinLevel !=
  >   mode) {
468       mode = pinLevel;
469       printModeBanner(pinLevel);
470       lastStatusUs = nowUs;
471     }
472
473     // Drain new edges.
474     const uint32_t head = s_edgeHead;
475     if (head - s_edgeTail > PS_EDGE_RING) s_edgeTail = head - PS_EDGE_RING;  // overrun
476     while (s_edgeTail != head) {
477       const Edge e = s_edges[s_edgeTail & (PS_EDGE_RING - 1u)];
478       s_edgeTail++;
479       if (!active) {
480         memset(&cur, 0, sizeof(cur));
```

test/bringup/power_sense_main.c / lines 481-485

```
481         cur.tStartUs = e.tUs;
482         cur.minGapUs = UINT32_MAX;
483         nListed = 0;
484         firstSample = sampleIndexAt(e.tUs - PS_PRE_TRIGGER_MS * 1000u, last);
485         active = true;
```

Starting a transition captures pre-trigger samples and a bounded list of edges. The analyzer tracks total edges and shortest gaps independently from how many detail records it can show.

Wait for settling or a bounded timeout

test/bringup/power_sense_main.c / lines 486-513

```
      } else if (cur.edges > 0 && e.tUs - lastEdgeUs < cur.minGapUs) {
        cur.minGapUs = e.tUs - lastEdgeUs;
      }
      if (cur.edges == 0) cur.tStartUs = e.tUs;
      if (nListed < PS_EDGES_LISTED) listed[nListed++] = e;
      cur.edges++;
      cur.bounceUs = e.tUs - cur.tStartUs;
      lastEdgeUs = e.tUs;
    }

    uint16_t lo, hi;
    uint32_t mean;
    spanMv(last - PS_STABLE_WINDOW_MS, last, &lo, &hi, &mean);
    const bool adcStable = !s_adcOn || (uint32_t)(hi - lo) < PS_STABLE_SPAN_MV;

    // ADC departure without any edge (e.g. rail drops but stays above VIH).
    if (!active && s_adcOn && settledKnown) {
      const uint32_t now = sampleAt(last - 1u)->mv;
      const uint32_t delta = now > settledMv ? now - settledMv : settledMv - now;
      if (delta >= PS_TRIGGER_DELTA_MV) {
        memset(&cur, 0, sizeof(cur));
        cur.tStartUs = sampleAt(last - 1u)->tUs;
        cur.minGapUs = UINT32_MAX;
        nListed = 0;
        firstSample = sampleIndexAt(cur.tStartUs - PS_PRE_TRIGGER_MS * 1000u, last);
        active = true;
      }
    }
```

The analyzer summarises a recent voltage window and can open a transition from analog departure even without a GPIO edge. An active transition ends only after sufficient quiet/stability or its timeout.

Wait for settling or a bounded timeout / continued

test/bringup/power_sense_main.c / lines 514-525

```
514
515     if (active) {
516       const bool edgesQuiet = cur.edges == 0 || nowUs - lastEdgeUs >= PS_EDGE_QUIET_MS *
  >    1000u;
517       const bool longEnough = nowUs - cur.tStartUs >= PS_EDGE_QUIET_MS * 1000u;
518       const bool timedOut = nowUs - cur.tStartUs >= PS_TRANSITION_TIMEOUT_MS * 1000u;
519       if ((edgesQuiet && adcStable && longEnough) || timedOut) {
520         cur.timedOut = timedOut && !(edgesQuiet && adcStable);
521         cur.endLevel = gpioLevel();
522         if (cur.minGapUs == UINT32_MAX) cur.minGapUs = 0;
523         reportTransition(&cur, listed, nListed, firstSample, last);
524         active = false;
525         settledMv = mean;
```

test/bringup/power_sense_main.c / lines 526-530

```
526         settledKnown = true;
527         lastStatusUs = nowUs;
528       }
529       continue;
530     }
```

It then derives and prints the result, stores the newly settled level and resumes watching. This is a state machine embedded in a diagnostic task: remembered state plus conditions, not one long fixed delay that misses intervening events.

Check whether both observation paths remain alive

test/bringup/power_sense_main.c / lines 531-558

```
531
532     if (!settledKnown && adcStable) {
533       settledMv = mean;
534       settledKnown = true;
535     }
536
537     // Steady state: GPIO level must agree with the ADC band.
538     const uint8_t level = gpioLevel();
539     if (s_adcOn && ((level == 1 && mean <= PS_VIL_MV) || (level == 0 && mean >=
  >   PS_VIH_MV))) {
540       s_mismatch++;
541     }
542
543     if (nowUs - lastStatusUs >= PS_STATUS_PERIOD_MS * 1000u) {
544       lastStatusUs = nowUs;
545       uint16_t sLo, sHi;
546       uint32_t sMean;
547       spanMv(last - 1000u, last, &sLo, &sHi, &sMean);
548       const uint32_t edges = s_edgeTotal, flips = s_samplerFlips;
549       // The sampler saw the level change but the ISR did not: interrupt is deaf.
550       const bool isrOk = !(flips != flipsAtStatus && edges == edgesAtStatus);
551       edgesAtStatus = edges;
552       flipsAtStatus = flips;
553       printf("  ");
554       printUptime();
555       printf("  still %s | pin ", mode == 1 ? "NORMAL" : mode == 0 ? "LOW-POWER" : "?");
556       if (!s_adcOn) {
557         printf("ADC off");
558       } else if (sHi >= PS_CLIP_MV) {
```

In steady state, the task compares digital level with the ADC band. Periodic status compares sampled flips with interrupt counts to expose a handler that stopped observing changes.

Check whether both observation paths remain alive / continued

test/bringup/power_sense_main.c / lines 559-570

```
559         printf(">=%u mV", PS_CLIP_MV);
560       } else {
561         printf("%lu mV", (unsigned long)sMean);
562       }
563       printf(" | %lu transition(s)\n", (unsigned long)s_historyCount);
564       if (!isrOk) printf("  WARNING: GPIO changed but the interrupt did not fire (ISR
  >   deaf)\n");
565       if (s_mismatch > 0) {
566         printf("  WARNING: GPIO level disagreed with the ADC for %lu check(s)\n",
567                (unsigned long)s_mismatch);
568         s_mismatch = 0;
569       }
570     }
```

test/bringup/power_sense_main.c / lines 571-572

```
571   }
572 }
```

These checks address a specific shared-pad risk: an analog configuration path can leave the digital interrupt ineffective. More print output alone would not expose that without independent counters and comparison logic.

Configure the shared pad and test one conversion before attaching work

test/bringup/power_sense_main.c / lines 573-600

```
573
574 void setup(void) {
575   delay(200);
576   printf("\n=== power-sense test: GPIO%d (ADC1_CH7), qualify %lu ms ===\n", (int)PS_PIN,
577          (unsigned long)BOARD_POWER_SENSE_QUALIFY_MS);
578
579   gpio_num_t io = GPIO_NUM_NC;
580   if (adc1_pad_get_io_num(PS_ADC_CHANNEL, &io) != ESP_OK || (int)io != PS_PIN) {
581     printf("ADC1_CH7 is GPIO%d, not GPIO%d: fix PS_ADC_CHANNEL\n", (int)io,
  >   (int)PS_PIN);
582     return;
583   }
584
585   // ADC first (this routes the pad to the RTC mux), then the digital input:
586   // gpio_config() hands the pad back to the digital mux. The ADC still samples
587   // the pad; the counters below confirm it on this board.
588   adc1_config_width(ADC_WIDTH_BIT_12);
589   adc1_config_channel_atten(PS_ADC_CHANNEL, ADC_ATTEN_DB_12);
590   const esp_adc_cal_value_t calType =
591       esp_adc_cal_characterize(ADC_UNIT_1, ADC_ATTEN_DB_12, ADC_WIDTH_BIT_12, 1100,
  >   &s_cal);
592   pinMode(PS_PIN, INPUT);  // no pull: the 440 Ohm bottom leg defines LOW
593
594   // Self-check: one conversion must not change the digital level.
595   const uint8_t l0 = gpioLevel();
596   const uint16_t mv = adcReadMv();
597   const uint8_t l1 = gpioLevel();
598   const bool agree = (l1 == 1 && mv >= PS_VIH_MV) || (l1 == 0 && mv <= PS_VIL_MV);
599   if (calType != ESP_ADC_CAL_VAL_EFUSE_TP_FIT && calType != ESP_ADC_CAL_VAL_EFUSE_TP) {
600     printf("WARNING: no eFuse ADC calibration, mV values are approximate\n");
```

Setup verifies channel-to-pin mapping, configures ADC width/attenuation and calibration, then restores the digital input mode. It samples the digital level before and after one analog conversion and checks band agreement.

The order reflects the pad-multiplexer interaction being investigated. Replacing the raw-IDF read with a convenience call without checking pin-mode effects could invalidate the experiment's two-observer premise.

Create a fast sampler and a slower analyzer

test/bringup/power_sense_main.c / lines 601-618

```
  }
  if (l0 == l1 && agree) {
    printf("self-check OK: ADC and GPIO agree (%u mV, GPIO %u)\n", mv, l1);
  } else if (l0 != l1) {
    printf("WARNING: self-check failed, an ADC conversion changes the GPIO level; press
  'a'\n");
  } else {
    printf("WARNING: self-check: GPIO %u but ADC %u mV (pin in the undefined band?)\n",
  l1, mv);
  }

  attachInterrupt(digitalPinToInterrupt(PS_PIN), onEdge, CHANGE);
  fcntl(STDIN_FILENO, F_SETFL, fcntl(STDIN_FILENO, F_GETFL, 0) | O_NONBLOCK);
  printHelp();

  xTaskCreatePinnedToCore(samplerTask, "ps_sample", 3072, NULL, 5, NULL, 0);
  xTaskCreatePinnedToCore(analyzerTask, "ps_analyze", 6144, NULL, 2, NULL, 1);
}

void loop(void) { vTaskDelay(portMAX_DELAY); }
```

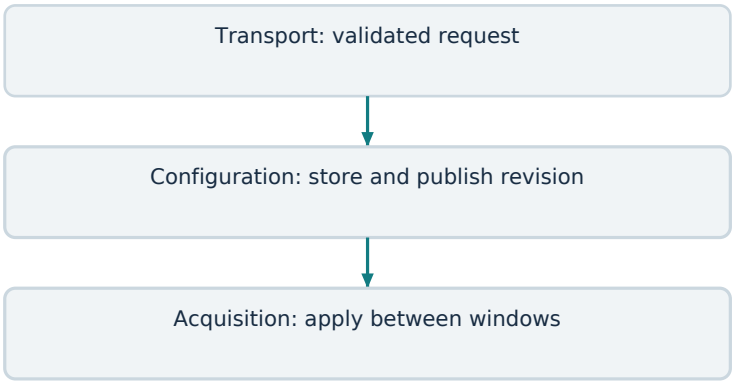
The remaining setup prints self-check results, attaches the edge ISR, makes standard input nonblocking and creates the sampler on core 0 and analyzer on core 1. loop blocks.

Separate tasks keep reporting work away from the intended fast acquisition path, but the shared rings still need their stated bounded-access assumptions. The test is useful because it exposes those assumptions and measurement limits rather than pretending a printed PASS proves every possible power transition.

Give settings one writer

A BLE command should not edit a driver's calibration fields while a measurement is in progress. It also should not perform a flash commit in a short radio callback. Configuration owns the settings and persistent writes; acquisition owns when driver settings are applied.

The transport parses a command into a ConfigRequest and attempts a bounded queue submission. Configuration validates the request, builds a new ConfigBlob and stores it. It publishes the retained applied-configuration record so consumers can copy it. Acquisition notices the revision and updates its drivers at a frame boundary.



This command path crosses owners by copied records and an acknowledgement.

The word applied needs care. A settings record can be stored before acquisition has actually used it. The service waits for an explicit configuration-revision acknowledgement when a command needs that stronger outcome. A timeout can therefore mean stored but not confirmed applied in time, rather than definitely not stored.

Why choose a frame boundary? The pH driver collects raw taps, filters them and applies its retained fit when finishing the window. Changing that fit while a window is pending would make the setting change occur inside an observation's lifetime. The chosen boundary keeps one completed window under one declared configuration, and each Reading records the revision used. Acquisition also cancels old-frame work that has overrun before proceeding with the new frame.

COMMON PITFALL

Accepted into a command queue, saved to flash and applied by a worker are three distinct milestones. Report the one that actually completed.

Keep the previous valid record

Original firmware - src/services/configuration.c, lines 140-149

```
static bool storeConfig(const ConfigBlob* next) {
  const uint8_t slot = s_haveSlot ? (uint8_t)(1u - s_currentSlot) : 0u;
  if (!gatedSetBlob(kSlotKey[slot], next, sizeof(*next))) return false;
  memcpy(&s_current, next, sizeof(s_current));
  s_currentSlot = slot;
  s_haveSlot = true;
  return true;
}

static void publish(void) { xQueueOverwrite(s_applied, &s_current); }
```

NVS is ESP-IDF's flash-backed key/value storage. This application uses two keys, cfg_a and cfg_b. storeConfig chooses the slot that does not currently hold the active record. Only after a successful write and commit does it copy next into s_current and switch the current-slot identity.

Suppose slot A contains valid revision 4. Write revision 5 to B. If that write fails, A still provides the previous accepted configuration. At boot, the application validates stored records, including their schema and checksum, and chooses according to its revision policy.

A schema states the record format, not the user's current settings. A revision identifies a change within that format. A checksum helps detect accidental corruption of the bytes the application expects. Those are different checks; a checksum does not establish that a calibration value is physically sensible.

The storage helper calls nvs_commit after a successful set. If it reports failure, Configuration must not answer as though the new record is durably accepted. Flash persistence also needs a power-interruption test; host tests of record validation cannot reproduce every storage failure.

The source has migration support for an older schema. Migration reads the old format, validates it, and constructs the new format with appropriate defaults. It is not safe to reinterpret an old byte blob as a larger new C structure and assume the extra fields exist.

NVS provides its own recovery mechanisms. The application's A/B scheme adds a record-selection policy on top.

Reference: Espressif: NVS, v4.4.7 (S3)

Protect timing while flash is busy

The ultrasonic result depends on timely GPIO edge handling. A flash operation can interfere with execution paths that depend on cached flash code. The project therefore excludes Configuration writes during an active echo window using the flash gate mutex.

Original firmware - src/services/flash_gate.c, lines 15-27

```
bool flashGateTryTakeForPing(void) {
  if (s_gate == NULL) return true;
  if (s_pingHeld) return true;
  s_pingHeld = xSemaphoreTake(s_gate, 0) == pdTRUE;
  return s_pingHeld;
}

void flashGateGiveAfterPing(void) {
  if (s_gate == NULL || !s_pingHeld) return;
  s_pingHeld = false;
  xSemaphoreGive(s_gate);
}
```

Before starting a ping, acquisition tries to take the gate without waiting. If Configuration is already writing, the ping is deferred. Once acquired, the gate stays held until the ping finishes or is cancelled. Configuration takes the same mutex before its flash operation and gives it afterward.

This lock spans elapsed echo time even though acquisition may block between polls. Holding a mutex while blocked is permitted, but it must be intentional: here the protected condition is no overlapping flash write, not continuous CPU ownership. A bounded ping timeout and cleanup path prevent an abandoned ping from keeping the gate forever.

The mutex does not prove that every possible timing disturbance has vanished. Interrupt masking from 1-Wire calls is handled by acquisition's separate overlap rules. Radio and framework behaviour still need measurement. One gate protects one declared conflict.

A calibration capture also respects ownership: it uses a recent retained observation rather than asking Configuration to operate the ADC independently. That keeps one acquisition owner even while an external app requests calibration.

COMMON PITFALL

Cancelling a pending measurement must release every resource acquired for it. Review timeout, power-loss and normal-completion paths together.

Give capture limits and outcomes their own names

Calibration should use a recent retained observation without creating another hardware owner. This interface describes that capture policy.

The returned x is the input quantity expected by the chosen fit, not always the displayed value.

src/domain/cal_capture.h / lines 1-23

```
// Raw calibration input from the latest snapshot (WP8, Calibration page).
// Pure C, host-tested.
//
// Acquisition stays the only ADC owner: a capture never touches a driver. It
// reads the raw level every calibratable driver already publishes in
// Reading.raw (filtered pin mV) and converts it to the x the stored fit uses:
//   pH     module mV   (pin mV through the 10k/15k divider)
//   TDS    ppm referred to 25 degC with the nominal datasheet curve, before K
//          (needs a fresh water temperature, as the driver does)
//   light  pin mV
#pragma once
#include <stdint.h>

#include "domain/reading.h"

typedef uint8_t CaptureStatus;
enum {
  CAPTURE_OK = 0,
  CAPTURE_STALE,           // no usable reading newer than maxAgeMs (also FAULT /
  UNPOWERED)
  CAPTURE_NO_TEMPERATURE,  // TDS only: water temperature missing, old or out of range
  CAPTURE_UNSUPPORTED      // not a calibratable sensor
};

```

CaptureLimits supplies maximum sample age, temperature age and pH divider values. CaptureStatus distinguishes a usable point from stale input, missing temperature and unsupported sensor.

Give capture limits and outcomes their own names / continued

src/domain/cal_capture.h / lines 24-32

```
24 typedef struct CaptureLimits {
25   uint32_t maxAgeMs;            // the sensor's reading must be at most this old
26   uint32_t temperatureMaxAgeMs; // TDS compensation input
27   uint32_t phDividerTopOhm;
28   uint32_t phDividerBottomOhm;
29 } CaptureLimits;
30
31 CaptureStatus calCaptureFromSnapshot(SensorId sensor, const Snapshot* snap, MonoMs now,
 >   const CaptureLimits* limits,
32                                      float* xOut, uint32_t* ageMsOut);
```

The function receives a Snapshot copy and now. It writes the captured x and its age through output pointers. It never reads an ADC or starts a new sensor conversion, so the request stays outside the acquisition timing path.

Accept raw data only with a usable history

Follow why pH, light and TDS capture different x values even though all three retain millivolts.

The calibration model determines the appropriate input axis.

src/domain/cal_capture.c / lines 1-13

```
#include "domain/cal_capture.h"

#include "domain/calibration.h"
#include "domain/tds_codec.h"

static bool usableRaw(const Reading* r, MonoMs now, uint32_t maxAgeMs, uint32_t* ageMs)
  {
  if (!r->hasRaw || r->intervalEndMs == 0) return false;
  if (r->health != HEALTH_OK && r->health != HEALTH_UNCALIBRATED) return false;
  const MonoMs age = now > r->intervalEndMs ? now - r->intervalEndMs : 0;
  if (age > maxAgeMs) return false;
  *ageMs = (uint32_t)age;
  return true;
}
```

The helper requires hasRaw, a measured interval and health OK or UNCALIBRATED. It calculates bounded age and rejects a point older than the capture limit.

Accepting uncalibrated raw input is deliberate: reference calibration is how that raw relationship becomes calibrated. It does not mean every health state with a populated raw field is suitable for fitting.

Capture the model input rather than its already fitted output

src/domain/cal_capture.c / lines 14-37

```
14
15 CaptureStatus calCaptureFromSnapshot(SensorId sensor, const Snapshot* snap, MonoMs now,
 >   const CaptureLimits* limits,
16                                      float* xOut, uint32_t* ageMsOut) {
17   *xOut = 0.0f;
18   *ageMsOut = 0;
19   switch (sensor) {
20     case SENSOR_PH: {
21       const Reading* r = &snap->readings[METRIC_PH];
22       if (!usableRaw(r, now, limits->maxAgeMs, ageMsOut)) return CAPTURE_STALE;
23       *xOut = dividerRecoverMv((float)r->raw, limits->phDividerTopOhm,
 >   limits->phDividerBottomOhm);
24       return CAPTURE_OK;
25     }
26     case SENSOR_LIGHT: {
27       const Reading* r = &snap->readings[METRIC_LIGHT_LEVEL];
28       if (!usableRaw(r, now, limits->maxAgeMs, ageMsOut)) return CAPTURE_STALE;
29       *xOut = (float)r->raw;
30       return CAPTURE_OK;
31     }
32     case SENSOR_TDS: {
33       const Reading* t = &snap->readings[METRIC_WATER_TEMPERATURE];
34       if (!readingIsActionableAt(t, now, limits->temperatureMaxAgeMs) || t->value <
 >   TDS_MIN_WATER_TEMP_C ||
35           t->value > TDS_MAX_WATER_TEMP_C) {
36         return CAPTURE_NO_TEMPERATURE;
37       }
```

pH recovers module millivolts from pin raw. Light uses pin millivolts directly. TDS first requires fresh acceptable water temperature, then evaluates the nominal compensated model from raw voltage.

src/domain/cal_capture.c / lines 38-46

```
38       const Reading* r = &snap->readings[METRIC_TDS];
39       // DEPENDENCY_INVALID (no temperature at the driver) is not usable either.
40       if (!usableRaw(r, now, limits->maxAgeMs, ageMsOut)) return CAPTURE_STALE;
41       *xOut = tdsPpmFromModuleMv((float)r->raw, tdsCompensationCoefficient(t->value));
42       return CAPTURE_OK;
43     }
44     default: return CAPTURE_UNSUPPORTED;
45   }
46 }
```

Using TDS's already gain-calibrated displayed value as x would compound a new gain on the previous fit. This helper avoids that by reconstructing the defined nominal input. Unsupported sensors return a distinct outcome rather than pretending zero is a reference point.

Separate identity, format version and user revision

A stored configuration needs a format, validation and update rules in addition to the settings themselves.

This record is local persistence for this firmware, not a portable on-wire C-struct format.

src/domain/config_blob.h / lines 1-25

```
// Versioned configuration record persisted by services/configuration (§17).
// Pure C. Work package WP8 (Stage 5).
//
// Stored in two NVS slots (A/B). A change writes the inactive slot with the
// next revision; boot loads the newest slot whose magic, schema and CRC are
// valid, else the compiled defaults (policy_config.h). The revision is what
// every Reading carries as configRevision.
//
// Wi-Fi credentials and the SGP30 baseline are NOT in here: they have their
// own NVS keys and write rates.
//
// The sensor mask and the pH / TDS / light calibrations are set by commands.
// The lamp policy has no command, so it always comes from the compiled
// defaults (policy_config.h). configBlobRefreshDefaults() copies the compiled
// values on every load, so a re-measured value in the source is never shadowed
// by an old copy in flash - except for a calibration whose CAL_BIT_* is set in
// calOverridden: the user fitted it on the device and it is kept.
//
// Schema 2 (calibration commands) added `light` and `calOverridden`. A schema 1
// slot is migrated at boot (configBlobMigrateV1) so the sensor mask survives
// the firmware update.
#pragma once
#include <stdbool.h>
#include <stdint.h>

```

Magic identifies the record family. Schema identifies layout. Revision identifies an accepted settings change. The sensor mask and calibration-override bits describe current settings and their origin.

src/domain/config_blob.h / lines 26-36

```
#include "domain/calibration.h"
#include "domain/lamp_control.h"
#include "domain/sensor_set.h"

#define CONFIG_BLOB_MAGIC 0x42445948u  // "HYDB" little-endian
#define CONFIG_BLOB_SCHEMA 2u

typedef struct ConfigBlob {
  uint32_t magic;
  uint16_t schema;
  uint16_t revision;          // 1.., wraps past 0xFFFF to 1
```

Separate identity, format version and user revision / continued

src/domain/config_blob.h / lines 37-46

```
  SensorMask sensorEnabled;
  bool lampAutoEnabled;
  uint8_t calOverridden;      // CAL_BIT_* (calibration.h): fitted by the user
  uint8_t reserved;           // keep zero (CRC covers every byte)
  LampControlConfig lamp;
  LinearCalibration ph;
  LinearCalibration tds;
  LinearCalibration light;    // x = pin mV, y = lux
  uint32_t crc;               // CRC-32 (IEEE) of every byte before this field
} ConfigBlob;
```

The final CRC covers bytes preceding the crc member, including padding in this selected representation. That is why the implementation zeroes and byte-copies records deliberately instead of treating arbitrary padding as irrelevant.

Retain an old layout only for migration

src/domain/config_blob.h / lines 47-74

```
47
48 // Schema 1 layout (firmware up to Stage 8.4), only for migration.
49 typedef struct ConfigBlobV1 {
50   uint32_t magic;
51   uint16_t schema;
52   uint16_t revision;
53   SensorMask sensorEnabled;
54   bool lampAutoEnabled;
55   uint8_t reserved[2];
56   LampControlConfig lamp;
57   LinearCalibration ph;
58   LinearCalibration tds;
59   uint32_t crc;
60 } ConfigBlobV1;
61
62 // Compiled defaults of every field except the sensor mask.
63 typedef struct ConfigDefaults {
64   bool lampAutoEnabled;
65   LampControlConfig lamp;
66   LinearCalibration ph;
67   LinearCalibration tds;
68   LinearCalibration light;
69 } ConfigDefaults;
70 // The CRC covers padding bytes too: copy a sealed blob with memcpy (struct
71 // assignment may not preserve padding), or re-seal after assigning.
72
73 uint32_t crc32Ieee(const void* data, uint32_t len);
74
```

ConfigBlobV1 records the previous schema shape. ConfigDefaults contains compiled policy rather than user enable state. The header explains which defaults refresh on boot and which calibrated overrides survive.

src/domain/config_blob.h / lines 75-76

```
75 // Revision 1, all sensors enabled, nothing overridden, the given defaults.
76 void configBlobInit(ConfigBlob* b, const ConfigDefaults* d);
```

An older blob cannot be interpreted by simply casting its address to the larger new struct. Migration must validate the old byte count and construct the current layout intentionally.

Expose updates as new sealed records

src/domain/config_blob.h / lines 77-104

```
77
78 void configBlobSeal(ConfigBlob* b);
79
80 // Magic, schema, CRC and value ranges (lamp band, faultsToOff, sensor mask).
81 bool configBlobValid(const ConfigBlob* b);
82
83 // Newest valid of two slots (either may be NULL or invalid); NULL if none.
84 // Revisions compare with wrap-around (serial-number arithmetic).
85 const ConfigBlob* configBlobPickNewest(const ConfigBlob* a, const ConfigBlob* b);
86
87 // next = cur with the sensor switched, a new revision and a fresh CRC.
88 // Returns false (next untouched) if nothing would change.
89 bool configBlobWithSensor(const ConfigBlob* cur, SensorId sensor, bool enabled,
 >   ConfigBlob* next);
90
91 uint16_t configRevisionNext(uint16_t revision);
92
93 // Copies the compiled lamp policy and every calibration NOT marked in
94 // calOverridden into b. If anything differed, b gets a new revision and a
95 // fresh CRC and the result is true (the caller stores it); otherwise b is
96 // untouched.
97 bool configBlobRefreshDefaults(ConfigBlob* b, const ConfigDefaults* d);
98
99 // next = cur with the sensor's calibration replaced, a new revision and a
100 // fresh CRC. overridden = true for a user fit (its revision becomes the new
101 // config revision, reported as cal_rev), false for a reset to the compiled
102 // default. false (next untouched) for a sensor without a calibration.
103 bool configBlobWithCalibration(const ConfigBlob* cur, SensorId sensor, const
 >   LinearCalibration* cal,
104                                bool overridden, ConfigBlob* next);
```

The API offers init, seal, validate, choose newest, sensor update, calibration update, default refresh and schema migration. Update helpers receive current and next records so callers can keep the old accepted state until persistence succeeds.

Expose updates as new sealed records / continued

src/domain/config_blob.h / lines 105-113

```
105
106 // The stored calibration of a calibratable sensor (NULL otherwise).
107 const LinearCalibration* configBlobCalibration(const ConfigBlob* b, SensorId sensor);
108 uint8_t configCalibrationBit(SensorId sensor);  // CAL_BIT_*, 0 if none
109
110 // A valid schema 1 blob (len must be sizeof(ConfigBlobV1)) becomes a schema 2
111 // blob with the same revision and sensor mask, the defaults for everything
112 // else and nothing overridden. false if v1 is not valid.
113 bool configBlobMigrateV1(const void* v1, uint32_t len, const ConfigDefaults* d,
  >   ConfigBlob* out);
```

A false result may mean unsupported input or no actual change depending on the helper. Read each contract before turning a boolean into a generic storage error. The service will decide when to publish and acknowledge an accepted revision.

Calculate a checksum over an explicit byte prefix

Read persistence as a sequence of checked byte transformations. This file never writes flash; it prepares and validates the records the service will store.

Use different names for a checksum-valid record and a physically sensible calibrated system.

src/domain/config_blob.c / lines 1-20

```c
#include "domain/config_blob.h"

#include <stddef.h>
#include <string.h>

uint32_t crc32Ieee(const void* data, uint32_t len) {
  const uint8_t* p = (const uint8_t*)data;
  uint32_t crc = 0xFFFFFFFFu;
  for (uint32_t i = 0; i < len; ++i) {
    crc ^= p[i];
    for (int bit = 0; bit < 8; ++bit) {
      crc = (crc & 1u) ? (crc >> 1) ^ 0xEDB88320u : crc >> 1;
    }
  }
  return ~crc;
}

static uint32_t blobCrc(const ConfigBlob* b) {
  return crc32Ieee(b, (uint32_t)offsetof(ConfigBlob, crc));
}
```

The CRC loop processes len bytes from a byte pointer with a reflected polynomial and final complement. blobCrc uses offsetof(ConfigBlob, crc), so it excludes the checksum member itself.

offsetof asks the compiler for the selected layout's member offset. It does not serialize a portable format or eliminate padding. The record's construction must make those covered bytes deterministic.

Initialise covered bytes before sealing and validate meaning afterward

src/domain/config_blob.c / lines 21-47

```
21
22 void configBlobInit(ConfigBlob* b, const ConfigDefaults* d) {
23   memset(b, 0, sizeof(*b));  // padding bytes are part of the CRC
24   b->magic = CONFIG_BLOB_MAGIC;
25   b->schema = (uint16_t)CONFIG_BLOB_SCHEMA;
26   b->revision = 1;
27   b->sensorEnabled = SENSOR_MASK_ALL;
28   b->lampAutoEnabled = d->lampAutoEnabled;
29   memcpy(&b->lamp, &d->lamp, sizeof(b->lamp));
30   memcpy(&b->ph, &d->ph, sizeof(b->ph));
31   memcpy(&b->tds, &d->tds, sizeof(b->tds));
32   memcpy(&b->light, &d->light, sizeof(b->light));
33   configBlobSeal(b);
34 }
35
36 void configBlobSeal(ConfigBlob* b) { b->crc = blobCrc(b); }
37
38 bool configBlobValid(const ConfigBlob* b) {
39   if (b == NULL || b->magic != CONFIG_BLOB_MAGIC || b->schema != CONFIG_BLOB_SCHEMA)
 >   return false;
40   if (b->crc != blobCrc(b)) return false;
41   if (b->revision == 0) return false;
42   if ((b->sensorEnabled & (SensorMask)~SENSOR_MASK_ALL) != 0) return false;
43   if ((b->calOverridden & (uint8_t)~CAL_BITS_ALL) != 0) return false;
44   if (!(b->lamp.onBelowMv < b->lamp.offAboveMv) || b->lamp.faultsToOff == 0) return
 >   false;
45   if (!linearCalibrationFinite(&b->ph) || !linearCalibrationFinite(&b->tds) ||
 >   !linearCalibrationFinite(&b->light)) {
46     return false;
47   }
```

Init zeroes the entire object, writes identity/version/revision, copies defaults and seals. Valid first checks pointer, magic, schema and CRC, then rejects invalid revisions, unsupported mask bits, an inverted lamp band and non-finite fits.

src/domain/config_blob.c / lines 48-49

```
48   return true;
49 }
```

A matching CRC proves consistency with those bytes, not that their chosen thresholds are appropriate for a physical installation. Range and commissioning decisions still belong to the configuration model and measured setup.

Compare revisions across a bounded serial-number wrap

src/domain/config_blob.c / lines 50-66

```
50
51 // true if a is newer than b (RFC 1982 style on 16 bits).
52 static bool revisionNewer(uint16_t a, uint16_t b) { return (int16_t)(uint16_t)(a - b) >
 >   0; }
53
54 const ConfigBlob* configBlobPickNewest(const ConfigBlob* a, const ConfigBlob* b) {
55   const bool va = configBlobValid(a);
56   const bool vb = configBlobValid(b);
57   if (va && vb) return revisionNewer(b->revision, a->revision) ? b : a;
58   if (va) return a;
59   if (vb) return b;
60   return NULL;
61 }
62
63 uint16_t configRevisionNext(uint16_t revision) {
64   const uint16_t next = (uint16_t)(revision + 1u);
65   return next == 0 ? 1 : next;
66 }
```

revisionNewer interprets a 16-bit modular difference as a signed ordering. PickNewest first validates both candidates, then chooses a valid newer one or returns NULL. Next skips zero when the revision counter wraps.

This ordering assumes the compared revisions are within the serial-number comparison window, not arbitrarily many laps apart. The two alternating slots normally represent nearby updates, which is the intended use.

Prepare a changed mask without editing current state

src/domain/config_blob.c / lines 67-79

```
67
68 bool configBlobWithSensor(const ConfigBlob* cur, SensorId sensor, bool enabled,
 >   ConfigBlob* next) {
69   if (sensor >= SENSOR_COUNT) return false;
70   const SensorMask bit = (SensorMask)(1u << sensor);
71   const SensorMask mask = enabled ? (SensorMask)(cur->sensorEnabled | bit)
72                                   : (SensorMask)(cur->sensorEnabled & (SensorMask)~bit);
73   if (mask == cur->sensorEnabled) return false;
74   memcpy(next, cur, sizeof(*next));
75   next->sensorEnabled = mask;
76   next->revision = configRevisionNext(cur->revision);
77   configBlobSeal(next);
78   return true;
79 }
```

The helper validates the sensor index before shifting, computes the new enable mask and returns false if nothing changes. It byte-copies cur to next, updates mask and revision and reseals.

The caller can now attempt storage while cur remains the accepted record. Returning true means a new record was prepared, not that any driver has already stopped or started.

Refresh only fields that still belong to compiled policy

src/domain/config_blob.c / lines 80-106

```
static bool sameCal(const LinearCalibration* a, const LinearCalibration* b) {
  return a->slope == b->slope && a->offset == b->offset && a->revision == b->revision &&
   a->valid == b->valid;
}

static void copyCal(LinearCalibration* dst, const LinearCalibration* src) { memcpy(dst,
  src, sizeof(*dst)); }

bool configBlobRefreshDefaults(ConfigBlob* b, const ConfigDefaults* d) {
  ConfigBlob fresh;
  memcpy(&fresh, b, sizeof(fresh));
  fresh.lampAutoEnabled = d->lampAutoEnabled;
  memcpy(&fresh.lamp, &d->lamp, sizeof(fresh.lamp));
  if ((b->calOverridden & CAL_BIT_PH) == 0) copyCal(&fresh.ph, &d->ph);
  if ((b->calOverridden & CAL_BIT_TDS) == 0) copyCal(&fresh.tds, &d->tds);
  if ((b->calOverridden & CAL_BIT_LIGHT) == 0) copyCal(&fresh.light, &d->light);
  // Compare field by field: the inputs may carry padding garbage.
  const bool same = fresh.lampAutoEnabled == b->lampAutoEnabled &&
                    d->lamp.onBelowMv == b->lamp.onBelowMv && d->lamp.offAboveMv ==
  b->lamp.offAboveMv &&
                    d->lamp.minOnMs == b->lamp.minOnMs && d->lamp.minOffMs ==
  b->lamp.minOffMs &&
                    d->lamp.faultsToOff == b->lamp.faultsToOff && sameCal(&fresh.ph,
  &b->ph) &&
                    sameCal(&fresh.tds, &b->tds) && sameCal(&fresh.light, &b->light);
  if (same) return false;
  fresh.revision = configRevisionNext(b->revision);
  configBlobSeal(&fresh);
  memcpy(b, &fresh, sizeof(*b));
  return true;
}
```

A temporary copy receives the current compiled lamp policy and any calibration not marked overridden. The function compares semantic fields rather than padding bytes, then advances revision and reseals only if something changed.

This prevents an old stored default from permanently hiding a newly compiled policy, while preserving a user's reference fit. sameCal compares exact stored coefficients because the question is whether the selected settings changed, not whether two noisy measurements are approximately equal.

Locate only supported calibration records

src/domain/config_blob.c / lines 107-124

```
uint8_t configCalibrationBit(SensorId sensor) {
  switch (sensor) {
    case SENSOR_PH: return CAL_BIT_PH;
    case SENSOR_TDS: return CAL_BIT_TDS;
    case SENSOR_LIGHT: return CAL_BIT_LIGHT;
    default: return 0;
  }
}

const LinearCalibration* configBlobCalibration(const ConfigBlob* b, SensorId sensor) {
  switch (sensor) {
    case SENSOR_PH: return &b->ph;
    case SENSOR_TDS: return &b->tds;
    case SENSOR_LIGHT: return &b->light;
    default: return NULL;
  }
}
```

CalibrationBit maps pH, TDS and light to their override bit. BlobCalibration returns the address of the corresponding stored line or NULL. Both explicitly reject non-calibratable sensors.

The const pointer provides a read-only view through this interface. It does not allocate a fit or imply that every SensorId has one. The later update helper first checks the bit before using the selected record.

Copy the chosen fit and its origin as one update

src/domain/config_blob.c / lines 125-142

```
125
126 bool configBlobWithCalibration(const ConfigBlob* cur, SensorId sensor, const
  >   LinearCalibration* cal,
127                                 bool overridden, ConfigBlob* next) {
128   const uint8_t bit = configCalibrationBit(sensor);
129   if (bit == 0) return false;
130   memcpy(next, cur, sizeof(*next));
131   next->revision = configRevisionNext(cur->revision);
132   LinearCalibration* dst = (LinearCalibration*)configBlobCalibration(next, sensor);
133   copyCal(dst, cal);
134   if (overridden) {
135     dst->revision = next->revision;
136     next->calOverridden = (uint8_t)(next->calOverridden | bit);
137   } else {
138     next->calOverridden = (uint8_t)(next->calOverridden & (uint8_t)~bit);
139   }
140   configBlobSeal(next);
141   return true;
142 }
```

The helper byte-copies the current record, advances revision and obtains the selected destination. Its cast removes const from a view of an underlying writable next object; it would not be legitimate for genuinely const storage.

A user override receives the new revision and sets its bit. Resetting to default clears that bit. Sealing follows all changes, so origin, coefficient and revision bytes belong to one checked candidate.

Validate the old layout before constructing the new one

src/domain/config_blob.c / lines 143-156

```
143
144 bool configBlobMigrateV1(const void* v1, uint32_t len, const ConfigDefaults* d,
  >   ConfigBlob* out) {
145   if (v1 == NULL || len != sizeof(ConfigBlobV1)) return false;
146   ConfigBlobV1 old;
147   memcpy(&old, v1, sizeof(old));
148   if (old.magic != CONFIG_BLOB_MAGIC || old.schema != 1u || old.revision == 0) return
  >   false;
149   if (old.crc != crc32Ieee(&old, (uint32_t)offsetof(ConfigBlobV1, crc))) return false;
150   if ((old.sensorEnabled & (SensorMask)~SENSOR_MASK_ALL) != 0) return false;
151   configBlobInit(out, d);
152   out->revision = old.revision;
153   out->sensorEnabled = old.sensorEnabled;
154   configBlobSeal(out);
155   return true;
156 }
```

Migration checks exact old size, identity, schema, revision, CRC and supported mask. It initializes a fresh current blob from current defaults, copies the old revision and sensor mask, then reseals.

Old calibration fields are not blindly retained as current user overrides under this migration policy. The preserved setting is the validated sensor selection; the new layout explicitly establishes current default calibration and policy behaviour.

Give acquisition a try and Configuration a bounded wait

The gate excludes flash work from an ultrasonic echo interval. It is a timing resource shared by acquisition and the sole NVS writer.

The mutex must be taken and released by its owning task.

src/services/flash_gate.h / lines 1-27

```
// Flash-write gate: keeps NVS commits out of the HC-SR04 echo window.
// Work package WP8 (Stage 5). RTOS Architecture content_2 §227.
//
// A flash write or erase disables the cache on both cores, which delays the
// echo-edge interrupt on core 1 and corrupts the distance. Acquisition holds
// the gate for the length of one ping; Configuration holds it around each
// NVS commit.
//
// Rules:
// - Acquisition only ever TRIES (zero wait) and defers the ping if the gate
//   is busy. It never blocks on Configuration.
// - Configuration waits at most a bounded time, then retries later.
// - Created by flashGateInit() at the very start of boot, independent of
//   Configuration. Before that (bring-up builds never call it), every take
//   succeeds: there is no writer to keep out.
// Mutex owned by one task at a time; take and give from the same task.
#pragma once
#include <stdbool.h>
#include <stdint.h>

bool flashGateInit(void);

bool flashGateTryTakeForPing(void);       // zero wait (acquisition)
void flashGateGiveAfterPing(void);        // safe to call when not held

bool flashGateTakeForWrite(uint32_t waitMs);  // bounded wait (Configuration)
void flashGateGiveAfterWrite(void);
```

The ping side tries with zero wait and defers its trigger if writing already owns the gate. The write side accepts a bounded wait and retries under service policy.

Before initialisation, the helpers deliberately allow work, supporting isolated builds with no Configuration writer. In production a failed gate init therefore weakens this exclusion; boot logging of failure is meaningful, not merely cosmetic.

Create one persistent mutex and remember the ping hold

Read the complete mutex wrapper and its task-private ping-held tag.

A released or cancelled ping must use the same give path as a normal completed one.

src/services/flash_gate.c / lines 1-20

```c
#include "services/flash_gate.h"

#include <freertos/FreeRTOS.h>
#include <freertos/semphr.h>

static StaticSemaphore_t s_gateControl;
static SemaphoreHandle_t s_gate = NULL;
static bool s_pingHeld;  // touched by acquisition only

bool flashGateInit(void) {
  if (s_gate == NULL) s_gate = xSemaphoreCreateMutexStatic(&s_gateControl);
  return s_gate != NULL;
}

bool flashGateTryTakeForPing(void) {
  if (s_gate == NULL) return true;
  if (s_pingHeld) return true;
  s_pingHeld = xSemaphoreTake(s_gate, 0) == pdTRUE;
  return s_pingHeld;
}
```

Init supplies static semaphore control storage. TryTake returns true if no gate exists or if acquisition already holds its ping interval; otherwise it attempts an immediate mutex take and records the result.

s_pingHeld is touched by acquisition only. It is not a general cross-task ownership flag. The underlying mutex coordinates the Configuration side, while that small tag avoids repeated takes by the same ping path.

Make repeated cleanup harmless and keep writes bounded

src/services/flash_gate.c / lines 21-35

```
21
22 void flashGateGiveAfterPing(void) {
23   if (s_gate == NULL || !s_pingHeld) return;
24   s_pingHeld = false;
25   xSemaphoreGive(s_gate);
26 }
27
28 bool flashGateTakeForWrite(uint32_t waitMs) {
29   if (s_gate == NULL) return true;
30   return xSemaphoreTake(s_gate, pdMS_TO_TICKS(waitMs)) == pdTRUE;
31 }
32
33 void flashGateGiveAfterWrite(void) {
34   if (s_gate != NULL) xSemaphoreGive(s_gate);
35 }
```

GiveAfterPing returns when no hold exists, otherwise clears the tag and gives the mutex. TakeForWrite converts a bounded wait and GiveAfterWrite releases a configured gate.

The wrapper contains no flash transaction itself. Configuration must hold the gate across both setting or erasing a key and its commit, then release it on every returned outcome.

Read the namespace as several persistence policies

This service accepts copied requests, owns NVS and publishes a retained configuration. It does not own the sensor buses.

Acquisition acknowledges application of a revision at a frame boundary.

src/services/configuration.h / lines 1-28

```
// Configuration: validated commands, versioned config, sole NVS writer.
// Work package WP8 (Stage 5). Architecture §17; RTOS §14.1.
//
// NVS namespace "hydro":
//   cfg_a / cfg_b  ConfigBlob A/B slots; boot loads the newest valid one
//                  (schema 1 slots are migrated: the sensor mask survives)
//   wifi           ConfigWifi (credentials; never logged, never sent back)
//   sgp_base       SGP30 baseline (co2eq, tvoc), written at most hourly
//   rst_hist       last 8 reset reasons, written once ~30 s after boot
//
// configurationInit() loads and PUBLISHES the applied configuration before it
// returns, so Safety and acquisition (started next) read it on their first
// pass. If NVS fails, the compiled defaults (all sensors on) are published.
//
// The LAN link and its "lan_tok" key are gone (the app link is Bluetooth
// only); a left-over lan_tok is erased once at boot.
//
// Calibration (Calibration page): set_calibration fits the points sent by the
// app (domain/calibration), refuses an implausible fit (bad_fit) or a sensor
// whose alarm rule is latched / active, stores the fit with its CAL_BIT_* set
// so the boot refresh keeps it, and waits until acquisition has applied it.
// reset_calibration restores the compiled default and clears the bit.
//
// Requests come through an 8-entry queue (CFG_COMMAND_QUEUE_LEN) from the
// transports; a full queue is reported as busy, never as success (§11). The
// Configuration task checks expiry and session before touching flash, writes
// the inactive slot, publishes, waits (bounded) until acquisition has applied
// the new revision, then replies with the actual outcome.
```

The header lists A/B configuration slots, credentials, gas baseline and reset history as separate keys. They have different update rates and interpretation. A fast sensor result is not written to flash on every sample.

src/services/configuration.h / lines 29-36

```
#pragma once
#include <stdbool.h>
#include <stdint.h>

#include "domain/config_blob.h"
#include "domain/protocol_codec.h"
#include "domain/types.h"

```

Read the namespace as several persistence policies / continued

src/services/configuration.h / lines 37-45

```
typedef uint8_t ConfigRequestKind;
enum {
  CONFIG_REQ_SET_SENSOR_ENABLED = 0,
  CONFIG_REQ_SET_WIFI,
  CONFIG_REQ_FORGET_WIFI,
  CONFIG_REQ_SET_CALIBRATION,
  CONFIG_REQ_RESET_CALIBRATION,
};

```

ConfigurationInit publishes defaults or loaded settings before returning, so subsequently created Safety and acquisition contexts can inspect a record immediately. The task then owns later changes.

Carry expiry, session and reply identity with the request

src/services/configuration.h / lines 46-69

```
typedef struct ConfigResult {
  uint32_t tag;
  ProtocolError error;              // PROTOCOL_OK on success
  uint16_t cfgRev;                  // sensor / calibration requests: the applied
  revision
} ConfigResult;

// Called from the Configuration task: copy the result and return at once.
typedef void (*ConfigReplyFn)(const ConfigResult* result, void* ctx);

typedef struct ConfigRequest {
  ConfigRequestKind kind;
  uint32_t tag;            // caller's handle, echoed in the result
  uint32_t session;        // transport session generation at receive time
  uint32_t (*sessionNow)(void);  // current generation; NULL = no session check
  MonoMs expiresMs;        // not processed after this
  ConfigReplyFn reply;
  void* replyCtx;
  SensorId sensor;
  bool enabled;
  char ssid[PROTOCOL_SSID_MAX + 1];
  char password[PROTOCOL_PASSWORD_MAX + 1];
  int32_t calX1, calY1, calX2, calY2;  // SET_CALIBRATION, scaled as in protocol_codec.h
} ConfigRequest;

```

ConfigRequest includes kind, tag, session checker, latest processing time, reply function/context and bounded payload storage. The queue copies that record; it does not borrow an app's temporary JSON buffer.

src/services/configuration.h / lines 70-80

```
typedef struct ConfigWifi {
  char ssid[PROTOCOL_SSID_MAX + 1];
  char password[PROTOCOL_PASSWORD_MAX + 1];
} ConfigWifi;

bool configurationInit(void);

// Zero wait. false = queue full (reply "busy") or not initialised.
bool configurationSubmit(const ConfigRequest* request);

// Latest applied configuration (zero wait). Before init or on failure it
```

The session function pointer still refers to external code with a suitable lifetime. A copied callback address is not a copy of the component it calls. The transport remains responsible for its stable callback implementation.

Expose copied settings and explicit acknowledgement

src/services/configuration.h / lines 81-99

```
// fills the compiled defaults and returns false.
bool configurationPeekApplied(ConfigBlob* out);

// Stored SGP30 baseline from the last boot, for acquisition before init.
bool configurationStoredBaseline(uint16_t* co2eq, uint16_t* tvoc);

// Acquisition offers the current baseline (zero wait); written at most hourly
// and only when it changed.
void configurationOfferBaseline(uint16_t co2eq, uint16_t tvoc);

// Acquisition reports the revision it applied at a frame boundary.
void configurationAckApplied(uint16_t revision);

// Wi-Fi credentials for the Wi-Fi owner (Stage 8.4). changeCount increases on
// every set/forget, so the reader knows when to reconnect.
bool configurationGetWifi(ConfigWifi* out, uint32_t* changeCount);

// For Supervisor. NULL before init.
void* configurationTaskHandle(void);
```

Submit reports an unavailable/full queue. PeekApplied supplies a retained configuration or compiled fallback. Baseline offers and application acknowledgements are short cross-task updates. GetWifi copies credentials and a change counter to the Wi-Fi owner.

A task receiving credentials should not print them in ordinary logs. The interface exists for connection setup, while other transports can report whether credentials are configured without returning the password.

Reserve persistent service state and bounded scratch

Read how one owner turns a request into validated state, persistent bytes, published revision and an eventual reply.

Keep received, stored and applied as three different milestones.

src/services/configuration.c / lines 1-28

```
#include "services/configuration.h"

#include <esp_system.h>
#include <esp_task_wdt.h>
#include <freertos/FreeRTOS.h>
#include <freertos/queue.h>
#include <freertos/task.h>
#include <nvs.h>
#include <stdio.h>
#include <string.h>

#include "app_config.h"
#include "diagnostics/logger.h"
#include "diagnostics/progress.h"
#include "domain/safety_core.h"
#include "hal/clock.h"
#include "policy_config.h"
#include "services/acquisition.h"
#include "services/flash_gate.h"
#include "services/safety.h"

#define CONFIG_NAMESPACE "hydro"
#define CONFIG_WAKE_MS 500u                 // idle wake: housekeeping, progress
#define CONFIG_ACK_TIMEOUT_MS 3000u         // acquisition applies at a frame boundary
  (<= 1 s)
#define CONFIG_ACK_POLL_MS 50u
#define CONFIG_GATE_WAIT_MS 100u            // one HC-SR04 ping is <= ~35 ms
#define CONFIG_GATE_TRIES 5
#define CONFIG_BASELINE_MIN_INTERVAL_MS (60u * 60u * 1000u)  // §16: rate-limited
```

src/services/configuration.c / lines 29-38

```
#define CONFIG_RESET_HISTORY_AFTER_MS 30000u // not during a reset loop (unstable
  supply)
#define CONFIG_RESET_HISTORY_LEN 8

static const char* const kSlotKey[2] = {"cfg_a", "cfg_b"};

// ---- Mailboxes and queues (static) ------------------------------------------
static StaticQueue_t s_appliedControl;
static uint8_t s_appliedStorage[sizeof(ConfigBlob)];
static QueueHandle_t s_applied = NULL;

```

Reserve persistent service state and bounded scratch / continued

src/services/configuration.c / lines 39-53

```
39 static StaticQueue_t s_requestControl;
40 static uint8_t s_requestStorage[CFG_COMMAND_QUEUE_LEN * sizeof(ConfigRequest)];
41 static QueueHandle_t s_requests = NULL;
42
43 static StaticTask_t s_tcb;
44 static StackType_t s_stack[4 * 1024];  // cfgTaskSpec(TASK_ID_CONFIGURATION)->stackBytes
45 static TaskHandle_t s_task = NULL;
46
47 // ---- State ------------------------------------------------------------------
48 static nvs_handle_t s_nvs;
49 static bool s_nvsOk;
50 static ConfigBlob s_current;   // task-owned after init
51 static uint8_t s_currentSlot;  // slot holding s_current (the other one is written next)
52 static bool s_haveSlot;        // s_current is stored somewhere
53
```

The constants define namespace, idle wake, acknowledgement wait, gate retries and slow housekeeping rates. Queue storage holds one applied blob and eight copied requests. s_current and its slot identity are owned by Configuration after boot.

src/services/configuration.c / lines 54-67

```
54 static portMUX_TYPE s_mux = portMUX_INITIALIZER_UNLOCKED;
55 static uint16_t s_ackRevision;
56 static ConfigWifi s_wifi;
57 static uint32_t s_wifiChanges;
58 static bool s_haveStoredBaseline;
59 static uint16_t s_storedCo2, s_storedTvoc;
60 static bool s_offerPending;
61 static uint16_t s_offerCo2, s_offerTvoc;
62
63 static MonoMs s_lastBaselineWriteMs;
64 static bool s_baselineWritten;
65 static bool s_historyWritten;
66 static ConfigRequest s_req;  // task-private: keeps the stack small
67
```

The small portMUX protects shared acknowledgements, copied Wi-Fi state and pending baseline offers. It does not cover NVS operations. Long writes and waits happen after those protected copies are taken.

Build explicit compiled defaults

src/services/configuration.c / lines 68-91

```
68 // ---- Compiled defaults ------------------------------------------------------
69 static void compiledDefaults(ConfigDefaults* d) {
70   const LampControlConfig l = {POLICY_LAMP_ON_BELOW_MV, POLICY_LAMP_OFF_ABOVE_MV,
 >   POLICY_LAMP_MIN_ON_MS,
71                                POLICY_LAMP_MIN_OFF_MS, POLICY_LAMP_FAULTS_TO_OFF};
72   memset(d, 0, sizeof(*d));
73   d->lampAutoEnabled = POLICY_LAMP_AUTO_ENABLED != 0;
74   d->lamp = l;
75   // pH: nominal fit from policy_config (published UNCALIBRATED as provisional).
76   d->ph.slope = POLICY_PH_NOMINAL_PH_PER_MV;
77   d->ph.offset = 7.0f - POLICY_PH_NOMINAL_PH_PER_MV * POLICY_PH_NOMINAL_MV_AT_PH7;
78   d->ph.revision = 0;
79   d->ph.valid = true;
80   // TDS: nominal datasheet curve, K = 1 (D13), not calibrated.
81   d->tds.slope = 1.0f;
82   d->tds.offset = 0.0f;
83   d->tds.revision = 0;
84   d->tds.valid = false;
85   // Light: no lux mapping until a reference-meter fit; published as mV.
86   d->light.slope = 1.0f;
87   d->light.offset = 0.0f;
88   d->light.revision = 0;
89   d->light.valid = false;
90 }
91
```

The lamp record comes from policy constants. pH receives the nominal line but remains provisional through the override bit. TDS and light keep invalid reference fits so their default output is explicitly uncalibrated.

src/services/configuration.c / lines 92-96

```
92 static void defaultBlob(ConfigBlob* b) {
93   ConfigDefaults d;
94   compiledDefaults(&d);
95   configBlobInit(b, &d);
96 }
```

defaultBlob delegates to the pure initializer and seal. Before any flash record is accepted, these defaults give downstream tasks a coherent, known starting record rather than uninitialised settings.

Hold the flash exclusion across the commit

src/services/configuration.c / lines 97-122

```
97
98  // ---- NVS helpers (every write goes through the flash gate) ---------------------
99  static void keepAlive(void) {
100   if (s_task != NULL && xTaskGetCurrentTaskHandle() == s_task) {
101     progressMark(TASK_ID_CONFIGURATION);
102     esp_task_wdt_reset();
103   }
104 }
105
106 static bool gatedSetBlob(const char* key, const void* data, size_t len) {
107   if (!s_nvsOk) return false;
108   for (int tries = 0; tries < CONFIG_GATE_TRIES; ++tries) {
109     if (!flashGateTakeForWrite(CONFIG_GATE_WAIT_MS)) {
110       keepAlive();
111       continue;
112     }
113     esp_err_t err = data != NULL ? nvs_set_blob(s_nvs, key, data, len) :
  >   nvs_erase_key(s_nvs, key);
114     if (err == ESP_ERR_NVS_NOT_FOUND && data == NULL) err = ESP_OK;  // already gone
115     if (err == ESP_OK) err = nvs_commit(s_nvs);
116     flashGateGiveAfterWrite();
117     if (err != ESP_OK) logPrintf(LOG_LEVEL_ERROR, "cfg: NVS write %s failed (%d)", key,
  >   (int)err);
118     return err == ESP_OK;
119   }
120   logPrintf(LOG_LEVEL_WARN, "cfg: flash gate busy, %s not written", key);
121   return false;
122 }
```

keepAlive marks progress only when called from the actual Configuration task, avoiding accidental reset of another task's watchdog subscription during boot. gatedSetBlob rejects unavailable NVS, takes the gate with bounded retries and performs set or erase plus commit.

The gate is given before logging and return. Erasing an already missing key counts as success. Failure to obtain the gate ends with a visible false outcome instead of an unbounded wait inside a timing-sensitive service.

Choose the other slot only after a successful write

src/services/configuration.c / lines 123-149

```
123
124 static bool getBlob(const char* key, void* out, size_t len) {
125   if (!s_nvsOk) return false;
126   size_t size = len;
127   return nvs_get_blob(s_nvs, key, out, &size) == ESP_OK && size == len;
128 }
129
130 // A schema 1 slot (firmware up to Stage 8.4) as a schema 2 blob, same revision.
131 static bool loadV1(const char* key, const ConfigDefaults* d, ConfigBlob* out) {
132   ConfigBlobV1 old;
133   if (!getBlob(key, &old, sizeof(old))) return false;
134   if (!configBlobMigrateV1(&old, sizeof(old), d, out)) return false;
135   logPrintf(LOG_LEVEL_INFO, "cfg: %s migrated from schema 1 (rev %u)", key,
  >   (unsigned)out->revision);
136   return true;
137 }
138
139 // Writes the slot not holding the current config; on success it becomes current.
140 static bool storeConfig(const ConfigBlob* next) {
141   const uint8_t slot = s_haveSlot ? (uint8_t)(1u - s_currentSlot) : 0u;
142   if (!gatedSetBlob(kSlotKey[slot], next, sizeof(*next))) return false;
143   memcpy(&s_current, next, sizeof(s_current));
144   s_currentSlot = slot;
145   s_haveSlot = true;
146   return true;
147 }
148
149 static void publish(void) { xQueueOverwrite(s_applied, &s_current); }
```

GetBlob checks both API success and exact byte count. LoadV1 validates and migrates the old format. StoreConfig selects the inactive key and updates s_current/slot identity only after gated persistence succeeds.

Publish overwrites the retained mailbox from that accepted current record. It is a RAM visibility operation after the storage milestone, not the flash write itself.

Reply with the owner-observed milestone

src/services/configuration.c / lines 150-176

```
150
151 // ---- Requests ------------------------------------------
  >   ------------------------
152 static void reply(const ConfigRequest* req, ProtocolError
  >   error, uint16_t cfgRev) {
153   if (req->reply == NULL) return;
154   ConfigResult r;
155   memset(&r, 0, sizeof(r));
156   r.tag = req->tag;
157   r.error = error;
158   r.cfgRev = cfgRev;
159   req->reply(&r, req->replyCtx);
160 }
161
162 static uint16_t ackedRevision(void) {
163   portENTER_CRITICAL(&s_mux);
164   const uint16_t rev = s_ackRevision;
165   portEXIT_CRITICAL(&s_mux);
166   return rev;
167 }
168
169 static bool waitApplied(uint16_t revision) {
170   if (acquisitionTaskHandle() == NULL) return false;
171   xTaskNotifyGive(acquisitionTaskHandle());  // apply at the
  >    next frame boundary
172   for (uint32_t waited = 0; waited < CONFIG_ACK_TIMEOUT_MS;
  >   waited += CONFIG_ACK_POLL_MS) {
173     if (ackedRevision() == revision) return true;
174     vTaskDelay(pdMS_TO_TICKS(CONFIG_ACK_POLL_MS));
175     keepAlive();  // the wait is normal work, not a stall
176   }
```

COMMON PITFALL

A callback executes synchronously in its caller here. Blocking inside it delays Configuration's next request and housekeeping.

reply builds a result with the caller tag and revision and invokes the supplied callback in this task. ackedRevision is read under the shared lock. WaitApplied wakes acquisition and checks for its matching acknowledgement with bounded delays.

src/services/configuration.c / lines 177-178

```
177   return ackedRevision() == revision;
178 }
```

A timeout can leave a new record stored and published but not confirmed applied within the budget. The reply must describe that outcome honestly instead of claiming the flash update never happened.

Check alarm state before removing an input

src/services/configuration.c / lines 179-203

```
179
180 static void handleSetSensor(const ConfigRequest* req) {
181   if (req->sensor >= SENSOR_COUNT) {
182     reply(req, PROTOCOL_ERR_BAD_FIELD, s_current.revision);
183     return;
184   }
185   if (!req->enabled) {
186     // Switching an input off must never silence an alarm (§12). The status can
187     // be up to one Safety pass (100 ms) old; a TRIP in flight still latches,
188     // and a latched rule stays visible as stale (safety_core), so the race is
189     // fail-safe. Accepted gap: a rule that has not tripped yet cannot trip
190     // once its sensor is off.
191     SafetyStatus st;
192     memset(&st, 0, sizeof(st));
193     if (safetyStatusPeek(&st) &&
194         safetyRulesBlockSensor(kPolicyRules, POLICY_RULE_COUNT, st.latchedMask,
  >   st.activeMask, req->sensor)) {
195       reply(req, PROTOCOL_ERR_RULE_LATCHED, s_current.revision);
196       return;
197     }
198   }
199   ConfigBlob next;
200   if (!configBlobWithSensor(&s_current, req->sensor, req->enabled, &next)) {
201     reply(req, PROTOCOL_OK, s_current.revision);  // already so
202     return;
203   }
```

The handler validates SensorId, consults copied Safety masks before disabling a sensor and returns an error when a held rule blocks it. A no-change request can return success with the existing revision.

Check alarm state before removing an input / continued

src/services/configuration.c / lines 204-214

```
  if (!storeConfig(&next)) {
    reply(req, PROTOCOL_ERR_STORAGE, s_current.revision);
    return;
  }
  publish();
  logPrintf(LOG_LEVEL_INFO, "cfg: rev %u: %s %s", (unsigned)s_current.revision,
  >  sensorKey(req->sensor),
            req->enabled ? "enabled" : "disabled");
  const bool applied = waitApplied(s_current.revision);
  if (!applied) logPrintf(LOG_LEVEL_WARN, "cfg: rev %u stored but not applied in time",
  >  (unsigned)s_current.revision);
  reply(req, applied ? PROTOCOL_OK : PROTOCOL_ERR_TIMEOUT, s_current.revision);
}
```

A changed candidate is stored, published and then awaited for application. The source comments explicitly describe the race policy for a trip arriving while a disable is being considered. A status copy is not a lock on future Safety events.

Persist credentials before publishing their change counter

src/services/configuration.c / lines 215-233

```
215
216 static void handleWifi(const ConfigRequest* req, bool forget) {
217   ConfigWifi wifi;
218   memset(&wifi, 0, sizeof(wifi));
219   if (!forget) {
220     snprintf(wifi.ssid, sizeof(wifi.ssid), "%s", req->ssid);
221     snprintf(wifi.password, sizeof(wifi.password), "%s", req->password);
222   }
223   const bool ok = gatedSetBlob("wifi", forget ? NULL : &wifi, sizeof(wifi));
224   if (ok) {
225     portENTER_CRITICAL(&s_mux);
226     memcpy(&s_wifi, &wifi, sizeof(s_wifi));
227     ++s_wifiChanges;
228     portEXIT_CRITICAL(&s_mux);
229   }
230   memset(&wifi, 0, sizeof(wifi));  // credentials never linger on the stack
231   logPrintf(LOG_LEVEL_INFO, "cfg: wifi %s %s", forget ? "forget" : "set", ok ? "stored"
  >   : "FAILED");
232   reply(req, ok ? PROTOCOL_OK : PROTOCOL_ERR_STORAGE, s_current.revision);
233 }
```

The temporary record begins zeroed. A set request copies bounded strings; forget asks the persistence helper to erase the key. Only success replaces shared credentials and advances changeCount under the lock.

The temporary is cleared afterward and logs report operation/result rather than credential text. Ordinary memset expresses cleanup intent, but do not infer a universal secure-erasure guarantee from an optimisation-sensitive C call.

Reject blocked sensors and undo the protocol scales

src/services/configuration.c / lines 234-261

```
234
235 // A calibration must never hide an alarm the sensor is part of (§12), as for
236 // switching it off: refused while one of its rules is latched or active.
237 static ProtocolError calibrationBlocked(SensorId sensor) {
238   SafetyStatus st;
239   memset(&st, 0, sizeof(st));
240   if (!safetyStatusPeek(&st)) return PROTOCOL_OK;
241   if (safetyRulesBlockSensor(kPolicyRules, POLICY_RULE_COUNT, st.latchedMask, 0,
  >   sensor)) {
242     return PROTOCOL_ERR_RULE_LATCHED;
243   }
244   if (safetyRulesBlockSensor(kPolicyRules, POLICY_RULE_COUNT, 0, st.activeMask, sensor))
  >    {
245     return PROTOCOL_ERR_RULE_ACTIVE;
246   }
247   return PROTOCOL_OK;
248 }
249
250 static bool fitCalibration(const ConfigRequest* req, LinearCalibration* out) {
251   const float x1 = (float)req->calX1 / CAL_X_SCALE;
252   switch (req->sensor) {
253     case SENSOR_PH:
254       return phFitFromBuffers(x1, (float)req->calY1 / CAL_PH_Y_SCALE, (float)req->calX2
  >   / CAL_X_SCALE,
255                               (float)req->calY2 / CAL_PH_Y_SCALE,
  >   POLICY_PH_SLOPE_MIN_ABS, POLICY_PH_SLOPE_MAX_ABS,
256                               0, out);
257     case SENSOR_TDS:
258       return linearGainFromPoint(x1, (float)req->calY1 / CAL_Y_SCALE, POLICY_TDS_K_MIN,
  >   POLICY_TDS_K_MAX, 0, out);
259     case SENSOR_LIGHT:
260       return linearGainFromPoint(x1, (float)req->calY1 / CAL_Y_SCALE,
  >   POLICY_LIGHT_LUX_PER_MV_MIN,
261                                  POLICY_LIGHT_LUX_PER_MV_MAX, 0, out);
```

calibrationBlocked distinguishes latched and active rules. FitCalibration divides fixed-point command integers by the declared x/y scales, then selects two-point pH or one-point gain for TDS/light with policy bounds.

src/services/configuration.c / lines 262-264

```
262     default: return false;
263   }
264 }
```

The scaled integer format belongs to the protocol; the math helpers use physical floats. Reusing an integer without the matching scale would create a plausible-looking but wrong fit.

Prepare, store and acknowledge one coherent fit change

src/services/configuration.c / lines 265-290

```
265
266 static void handleCalibration(const ConfigRequest* req, bool reset) {
267   if (configCalibrationBit(req->sensor) == 0) {
268     reply(req, PROTOCOL_ERR_BAD_FIELD, s_current.revision);
269     return;
270   }
271   const ProtocolError blocked = calibrationBlocked(req->sensor);
272   if (blocked != PROTOCOL_OK) {
273     reply(req, blocked, s_current.revision);
274     return;
275   }
276   LinearCalibration cal;
277   if (reset) {
278     ConfigDefaults d;
279     compiledDefaults(&d);
280     cal = req->sensor == SENSOR_PH ? d.ph : req->sensor == SENSOR_TDS ? d.tds : d.light;
281   } else if (!fitCalibration(req, &cal)) {
282     reply(req, PROTOCOL_ERR_BAD_FIT, s_current.revision);
283     return;
284   }
285   ConfigBlob next;
286   configBlobWithCalibration(&s_current, req->sensor, &cal, !reset, &next);
287   if (!storeConfig(&next)) {
288     reply(req, PROTOCOL_ERR_STORAGE, s_current.revision);
289     return;
290   }
```

The handler checks calibratability and alarm restrictions. Reset chooses compiled defaults; set computes and validates a fit. It builds a new sealed blob, persists it, publishes it and awaits acquisition's acknowledgement.

Prepare, store and acknowledge one coherent fit change / continued

src/services/configuration.c / lines 291-302

```
  publish();
  if (reset) {
    logPrintf(LOG_LEVEL_INFO, "cfg: rev %u: %s calibration reset to default",
  (unsigned)s_current.revision,
              sensorKey(req->sensor));
  } else {
    logPrintf(LOG_LEVEL_INFO, "cfg: rev %u: %s calibrated, slope %.6f offset %.4f",
  (unsigned)s_current.revision,
              sensorKey(req->sensor), (double)cal.slope, (double)cal.offset);
  }
  const bool applied = waitApplied(s_current.revision);
  if (!applied) logPrintf(LOG_LEVEL_WARN, "cfg: rev %u stored but not applied in time",
  (unsigned)s_current.revision);
  reply(req, applied ? PROTOCOL_OK : PROTOCOL_ERR_TIMEOUT, s_current.revision);
}
```

The calibration-origin bit tells later boot refresh whether the fitted line should survive. The result revision is therefore more than a UI label: it identifies which settings future readings may report using.

Discard stale authority before touching flash

src/services/configuration.c / lines 303-319

```
303
304 static void handle(const ConfigRequest* req) {
305   // Stale requests never change flash (§11): wrong session = no reply at all.
306   if (req->sessionNow != NULL && req->sessionNow() != req->session) return;
307   if (monoNowMs() > req->expiresMs) {
308     reply(req, PROTOCOL_ERR_EXPIRED, s_current.revision);
309     return;
310   }
311   switch (req->kind) {
312     case CONFIG_REQ_SET_SENSOR_ENABLED: handleSetSensor(req); break;
313     case CONFIG_REQ_SET_WIFI: handleWifi(req, false); break;
314     case CONFIG_REQ_FORGET_WIFI: handleWifi(req, true); break;
315     case CONFIG_REQ_SET_CALIBRATION: handleCalibration(req, false); break;
316     case CONFIG_REQ_RESET_CALIBRATION: handleCalibration(req, true); break;
317     default: reply(req, PROTOCOL_ERR_UNKNOWN_OP, s_current.revision); break;
318   }
319 }
```

A session mismatch returns without applying or answering an old connection. An expired request receives an explicit expired result. Only then does kind dispatch reach sensor, credential or calibration handling.

This placement keeps every handler behind the same authority/time boundary. Repeating checks only in the BLE parser would be insufficient because the request could wait in the queue after parsing.

Copy an offered baseline and apply a slow write policy

src/services/configuration.c / lines 320-340

```
320
321 // ---- Housekeeping ------------------------------------------------------------
322 static void maybeWriteBaseline(MonoMs now) {
323   portENTER_CRITICAL(&s_mux);
324   const bool pending = s_offerPending;
325   const uint16_t co2 = s_offerCo2, tvoc = s_offerTvoc;
326   s_offerPending = false;
327   portEXIT_CRITICAL(&s_mux);
328   if (!pending) return;
329   if (s_haveStoredBaseline && co2 == s_storedCo2 && tvoc == s_storedTvoc) return;
330   if (s_baselineWritten && now - s_lastBaselineWriteMs <
  >   CONFIG_BASELINE_MIN_INTERVAL_MS) return;
331   const uint16_t pair[2] = {co2, tvoc};
332   if (gatedSetBlob("sgp_base", pair, sizeof(pair))) {
333     s_storedCo2 = co2;
334     s_storedTvoc = tvoc;
335     s_haveStoredBaseline = true;
336     s_baselineWritten = true;
337     s_lastBaselineWriteMs = now;
338     logPrintf(LOG_LEVEL_INFO, "cfg: SGP30 baseline stored (co2eq 0x%04x, tvoc 0x%04x)",
  >   co2, tvoc);
339   }
340 }
```

The pending offer is copied and cleared under the lock. The task then ignores unchanged words and rate-limits successful writes. A successful commit updates retained words, availability and last-write time.

Offers discarded during the minimum interval are not an unbounded backlog; acquisition offers again under its own slow cadence. Understand that chosen coalescing policy before expecting every sensor-internal baseline change to be saved.

Record reset history later and process one copied request

src/services/configuration.c / lines 341-368

```
341
342 static void maybeWriteResetHistory(MonoMs now) {
343   if (s_historyWritten || now < CONFIG_RESET_HISTORY_AFTER_MS) return;
344   s_historyWritten = true;
345   uint8_t hist[CONFIG_RESET_HISTORY_LEN];
346   if (!getBlob("rst_hist", hist, sizeof(hist))) memset(hist, 0, sizeof(hist));  // 0 =
    >   ESP_RST_UNKNOWN
347   memmove(&hist[1], &hist[0], sizeof(hist) - 1);
348   hist[0] = (uint8_t)esp_reset_reason();
349   if (gatedSetBlob("rst_hist", hist, sizeof(hist))) {
350     logPrintf(LOG_LEVEL_INFO, "cfg: reset history (newest first, esp_reset_reason_t): %u
    >    %u %u %u %u %u %u %u",
351               hist[0], hist[1], hist[2], hist[3], hist[4], hist[5], hist[6], hist[7]);
352   }
353 }
354
355 static void configTask(void* arg) {
356   (void)arg;
357   if (esp_task_wdt_add(NULL) != ESP_OK) logPrintf(LOG_LEVEL_ERROR, "cfg: task watchdog
    >   add failed");
358   for (;;) {
359     if (xQueueReceive(s_requests, &s_req, pdMS_TO_TICKS(CONFIG_WAKE_MS)) == pdTRUE) {
360       handle(&s_req);
361       memset(s_req.password, 0, sizeof(s_req.password));
362     }
363     const MonoMs now = monoNowMs();
364     maybeWriteBaseline(now);
365     maybeWriteResetHistory(now);
366     progressMark(TASK_ID_CONFIGURATION);
367     esp_task_wdt_reset();
368   }
```

The reset-history helper waits until boot has remained up long enough, shifts a bounded byte history with memmove and attempts a write. configTask waits up to 500ms for a request, handles a successful receive and clears its password scratch.

src/services/configuration.c / lines 369-369

```
369 }
```

Idle wake time also allows housekeeping and meaningful progress/watchdog updates. A task waiting normally on the queue is different from a function that never returns while holding an internal resource.

Load both records before publishing the selected configuration

src/services/configuration.c / lines 370-397

```
370
371 // ---- Public API ------------------------------------------------------------
372 bool configurationInit(void) {
373   if (s_task != NULL) return true;
374   s_applied = xQueueCreateStatic(1, sizeof(ConfigBlob), s_appliedStorage,
  >   &s_appliedControl);
375   s_requests = xQueueCreateStatic(CFG_COMMAND_QUEUE_LEN, sizeof(ConfigRequest),
  >   s_requestStorage,
376                                   &s_requestControl);
377   if (s_applied == NULL || s_requests == NULL) return false;
378
379   // NVS itself was initialised by the Arduino core (initArduino()).
380   s_nvsOk = nvs_open(CONFIG_NAMESPACE, NVS_READWRITE, &s_nvs) == ESP_OK;
381   if (!s_nvsOk) logPrintf(LOG_LEVEL_ERROR, "cfg: NVS open failed - compiled defaults,
  >   nothing stored");
382
383   ConfigDefaults defaults;
384   compiledDefaults(&defaults);
385   ConfigBlob slots[2];
386   bool have[2];
387   for (uint8_t i = 0; i < 2; ++i) {
388     have[i] = getBlob(kSlotKey[i], &slots[i], sizeof(ConfigBlob)) || loadV1(kSlotKey[i],
  >    &defaults, &slots[i]);
389   }
390   const ConfigBlob* newest = configBlobPickNewest(have[0] ? &slots[0] : NULL, have[1] ?
  >   &slots[1] : NULL);
391   bool store = false;
392   if (newest != NULL) {
393     memcpy(&s_current, newest, sizeof(s_current));
394     s_currentSlot = newest == &slots[0] ? 0 : 1;
395     s_haveSlot = true;
396   } else {
397     defaultBlob(&s_current);
```

Init creates queues and opens NVS, then loads current or migratable slot candidates. PickNewest validates them. If neither is usable it constructs defaults; compiled-default refresh can also require a new stored revision.

src/services/configuration.c / lines 398-404

```
398     store = s_nvsOk;  // first boot or both slots corrupt
399     if (have[0] || have[1]) logPrintf(LOG_LEVEL_WARN, "cfg: stored config invalid -
  >   defaults");
400   }
401   if (configBlobRefreshDefaults(&s_current, &defaults)) store = s_nvsOk;
402   if (store) {
403     ConfigBlob next;
404     memcpy(&next, &s_current, sizeof(next));
```

Load both records before publishing the selected configuration / continued

src/services/configuration.c / lines 405-413

```
405     if (!storeConfig(&next)) logPrintf(LOG_LEVEL_WARN, "cfg: could not store rev %u",
  >   (unsigned)next.revision);
406   }
407   s_ackRevision = 0;
408   publish();  // before Safety and acquisition start (they peek it on their first pass)
409
410   ConfigWifi wifi;
411   if (getBlob("wifi", &wifi, sizeof(wifi))) {
412     wifi.ssid[PROTOCOL_SSID_MAX] = '\0';
413     wifi.password[PROTOCOL_PASSWORD_MAX] = '\0';
```

The accepted current record is published before Safety and acquisition start. Failure to open NVS still leaves a RAM fallback available, while writes remain disabled. This preserves explicit startup dependencies rather than making every consumer wait for flash recovery.

Treat stored credentials and gas baseline history differently

src/services/configuration.c / lines 414-440

```
414     memcpy(&s_wifi, &wifi, sizeof(s_wifi));
415     memset(&wifi, 0, sizeof(wifi));
416   }
417   size_t tokenSize = 0;  // LAN link removed: drop its token once
418   if (s_nvsOk && nvs_get_blob(s_nvs, "lan_tok", NULL, &tokenSize) == ESP_OK) {
419     gatedSetBlob("lan_tok", NULL, 0);
420     logPrintf(LOG_LEVEL_INFO, "cfg: old LAN token erased");
421   }
422   // SGP30 baseline (Sensirion integration guide §3.8): valid only if the
423   // sensor was off for less than 7 days. There is no wall clock, so the
424   // downtime is known only for resets where the chip kept its supply (software,
425   // panic, watchdog, reset pin: seconds). After a power-on, brown-out or
426   // unknown reset it may have been off any time: relearn, and erase the old
427   // baseline so a later warm reset cannot bring it back.
428   uint16_t pair[2];
429   if (getBlob("sgp_base", pair, sizeof(pair))) {
430     const esp_reset_reason_t why = esp_reset_reason();
431     const bool supplyKept = why == ESP_RST_SW || why == ESP_RST_PANIC || why ==
  >   ESP_RST_INT_WDT ||
432                             why == ESP_RST_TASK_WDT || why == ESP_RST_WDT || why ==
  >   ESP_RST_EXT;
433     if (supplyKept) {
434       s_storedCo2 = pair[0];
435       s_storedTvoc = pair[1];
436       s_haveStoredBaseline = true;
437     } else {
438       gatedSetBlob("sgp_base", NULL, 0);
439       logPrintf(LOG_LEVEL_INFO, "cfg: SGP30 baseline dropped (reset %d: off-time
  >   unknown, relearning)", (int)why);
440     }
```

The boot path bounds credential string terminators and removes a retired LAN token. Baseline restoration depends on reset reason because the program lacks a calendar clock proving how long the gas chip was unpowered.

Warm reset categories are treated as retained supply; unknown/power/brownout cases discard the old pair under this policy. That inference has stated limits: a numeric baseline alone contains no reliable off-time timestamp.

Create the owner only after its first publication exists

src/services/configuration.c / lines 441-454

```
  }

  const TaskSpec* spec = cfgTaskSpec(TASK_ID_CONFIGURATION);
  if (spec->stackBytes != sizeof(s_stack)) {
    logPrintf(LOG_LEVEL_WARN, "cfg: stack %u B, task table says %lu B",
  (unsigned)sizeof(s_stack),
              (unsigned long)spec->stackBytes);
  }
  s_task = xTaskCreateStaticPinnedToCore(configTask, spec->name, sizeof(s_stack), NULL,
  spec->priority,
                                         s_stack, &s_tcb, spec->core);
  logPrintf(LOG_LEVEL_INFO, "cfg: rev %u, sensors 0x%02x, calibrated 0x%02x, wifi %s,
  SGP30 baseline %s",
            (unsigned)s_current.revision, (unsigned)s_current.sensorEnabled,
  (unsigned)s_current.calOverridden,
            s_wifi.ssid[0] != '\0' ? "set" : "none", s_haveStoredBaseline ? "stored" :
  "none");
  return s_task != NULL;
}
```

The remaining init compares stack reservation with TaskSpec, creates the static Configuration task and reports revision, masks and configured-state summaries. It does not log credential contents.

Task creation can schedule immediately, so queues, NVS handle and current state have already been established. A failed creation remains visible in the returned boolean even though the initial RAM publication may exist.

Offer copied records with explicit fallback semantics

src/services/configuration.c / lines 455-472

```
455
456 bool configurationSubmit(const ConfigRequest* request) {
457   if (s_requests == NULL || request == NULL) return false;
458   return xQueueSend(s_requests, request, 0) == pdTRUE;
459 }
460
461 bool configurationPeekApplied(ConfigBlob* out) {
462   if (s_applied != NULL && xQueuePeek(s_applied, out, 0) == pdTRUE) return true;
463   defaultBlob(out);
464   return false;
465 }
466
467 bool configurationStoredBaseline(uint16_t* co2eq, uint16_t* tvoc) {
468   if (!s_haveStoredBaseline) return false;
469   *co2eq = s_storedCo2;
470   *tvoc = s_storedTvoc;
471   return true;
472 }
```

Submit rejects a missing queue/request and uses a zero-wait send. PeekApplied returns a mailbox copy when present; otherwise it fills defaults and returns false. StoredBaseline copies only when boot accepted one.

The caller must read each boolean in its own contract. A false PeekApplied still provides defined defaults, whereas a false arbitrary queue receive does not supply a new message.

Keep shared offers, acknowledgements and credentials brief

src/services/configuration.c / lines 473-496

```
473
474 void configurationOfferBaseline(uint16_t co2eq, uint16_t tvoc) {
475   portENTER_CRITICAL(&s_mux);
476   s_offerPending = true;
477   s_offerCo2 = co2eq;
478   s_offerTvoc = tvoc;
479   portEXIT_CRITICAL(&s_mux);
480 }
481
482 void configurationAckApplied(uint16_t revision) {
483   portENTER_CRITICAL(&s_mux);
484   s_ackRevision = revision;
485   portEXIT_CRITICAL(&s_mux);
486 }
487
488 bool configurationGetWifi(ConfigWifi* out, uint32_t* changeCount) {
489   portENTER_CRITICAL(&s_mux);
490   memcpy(out, &s_wifi, sizeof(*out));
491   if (changeCount != NULL) *changeCount = s_wifiChanges;
492   portEXIT_CRITICAL(&s_mux);
493   return out->ssid[0] != '\0';
494 }
495
496 void* configurationTaskHandle(void) { return s_task; }
```

OfferBaseline records one coalesced pair under the shared lock. AckApplied stores a revision. GetWifi byte-copies credentials and optional change count under that same protection, then reports whether an SSID exists.

None of these getters writes flash or steps a driver. Their small protected operations let task owners exchange the facts needed to coordinate without moving long work into a cross-core critical section.

Read the command before reading its parser

Wire text - JSON, not a C statement

```
{"v":2,"id":"g7","op":"get_sensors"}
```

JSON is a text representation for structured values. Braces surround an object: a collection of named fields. A colon separates each field name from its value; commas separate fields. Field names are quoted strings. Here v has the number 2, id has the string g7 and op has the string get_sensors.

The protocol assigns meaning to those names. v selects message version 2. id lets the client recognise the eventual reply. op selects the requested operation. JSON by itself gives those names no device-control behaviour; this firmware's parser and service owners supply it.

Wire text - one command object with a nested data object

```
{"v":2,"id":"d7","op":"set_sensor_enabled",
 "data":{"sensor":"light","enabled":false}}
```

data is another object. Its sensor string selects light, while enabled is the JSON boolean false. Writing "false" would create a string instead. The document could remain valid JSON while failing this operation's type checks. Arrays use square brackets, as in [0,2]; null explicitly represents no value, as when telemetry cannot supply a usable measurement.

Encoding turns selected application values into these bytes. Parsing checks bytes and constructs a Command. Executing the command is a later operation performed by its owner. We will keep those three verbs separate while reading the complete codec.

REMEMBER

Legal JSON is only the first test. The operation still needs the expected keys, types, lengths and ranges.

Count payload bytes separately from buffer capacity

C source - backslashes quote characters inside a string literal

```
char text[] = "{\"v\":2}";
// Stored content: {"v":2} followed by one zero byte.
// strlen(text) is 7. sizeof(text) is 8.
```

C uses the outer quotes to mark a string literal. A backslash before an inner quote includes that quote in the stored content. Those escape backslashes are C source notation; they are not sent as extra bytes for this message. The final zero byte terminates the C string in RAM.

The JSON payload above occupies seven bytes. This protocol sends a two-byte length header followed by those seven bytes: nine bytes in total. Its header is 07 00 in hexadecimal. The C string terminator is excluded from the wire payload. sizeof and strlen answer different questions here because sizeof includes storage for that terminator.

Teaching fragment using the production writer interface

```
char out[8];
JsonWriter w;
jsonWriterInit(&w, out, sizeof(out));
jsonRaw(&w, "{\"v\":2}");
size_t n = jsonWriterLength(&w);  // 7
```

Capacity eight admits seven content bytes and the terminator. Changing only out to a seven-byte array makes this complete fragment too large. The writer latches overflow and jsonWriterLength returns zero. The caller must stop that transmission; sending the retained prefix would produce a damaged message.

The command parser instead receives an address and an explicit payload length. Incoming bytes need not carry a zero terminator. Parser.p advances through that bounded range; Parser.end marks the limit. Never replace that explicit length with strlen on untrusted received bytes.

COMMON PITFALL

UTF-8 can use several bytes for one displayed character. Protocol buffer limits and frame lengths count bytes.

Packets are pieces of a message

The protocol uses a two-byte little-endian payload length followed by that many UTF-8 JSON bytes. A BLE delivery may contain half a header, part of a payload, or several frame fragments. The decoder must retain partial state until a complete frame exists.

Worked framing bytes - HELLO is not a valid project JSON command

```
05 00  48 45 4C 4C 4F
// Length = 5; payload bytes spell HELLO.
// Delivery 1 might be: 05
// Delivery 2 might be: 00 48 45
// Delivery 3 might be: 4C 4C 4F
```

The first length byte is low, so 05 00 means 5, not 1280. The assembler counts stored bytes, checks the declared length against its fixed maximum, and reports READY only when the payload is complete. An input chunk ending is not a frame-ending event.

frameAssemblerFeed reports how many bytes it consumed and returns a pointer into its own buffer for the complete payload. That pointer stays valid only until the next feed. If you need the message afterward, decode or copy it before advancing the assembler.

On disconnect, discard a partial frame so bytes from a new connection cannot finish a message begun by the old one. The transport also tracks session generations for pending commands. A late completion from an old connection must not be delivered as the answer to a new session's request.

The bounded JSON codec then validates fields and lengths. A length-valid frame is only a transport success; an unknown command or invalid sensor identifier can still be an application error.

COMMON PITFALL

Do not treat a payload as a zero-terminated string unless a buffer and terminator have actually been supplied. Pass its explicit length.

Own the assembly buffer and expose borrowed payload bytes

Before parsing a command, the receiver must know where that command ends. This header defines the length-prefixed frame independently of BLE and JSON.

Two little-endian bytes describe a payload of 1 through 4096 bytes. A BLE write is only a delivery chunk; it is not a frame boundary.

COMMON PITFALL

A pointer into an object is valid only while that object and the relevant contents remain valid.

src/domain/frame_codec.h / lines 1-23

```
// Length-prefixed frame reassembly (FirmwareContract.md v2, §17). Pure C.
// Work package WP8 (Stage 5). Shared by the BLE and LAN transports.
//
// Every frame is a 2-byte little-endian payload length followed by that many
// bytes of UTF-8 JSON. Transport packets may split or merge frames, so the
// receiver feeds whatever arrives and takes complete payloads out one by one.
// A length of 0 or above FRAME_MAX_PAYLOAD is a protocol error: the buffer is
// dropped (the caller decides whether to disconnect).
#pragma once
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

#define FRAME_HEADER_BYTES ((size_t)2)
#define FRAME_MAX_PAYLOAD ((size_t)4096)

typedef uint8_t FrameStatus;
enum {
  FRAME_NEED_MORE = 0,  // all input consumed, no complete frame yet
  FRAME_READY,          // one payload available; feed the rest of the input again
  FRAME_ERROR           // bad length; assembler reset
};

```

FrameAssembler retains partly received bytes between calls. FrameStatus distinguishes incomplete input, one complete frame and a rejected length. Feed reports consumed input separately from the payload length.

Own the assembly buffer and expose borrowed payload bytes / continued

src/domain/frame_codec.h / lines 24-45

```
24 typedef struct FrameAssembler {
25   uint8_t buf[FRAME_HEADER_BYTES + FRAME_MAX_PAYLOAD];
26   size_t used;
27   bool ready;          // buf holds a complete frame returned by the last feed
28   uint32_t rejected;   // FRAME_ERROR count
29 } FrameAssembler;
30
31 void frameAssemblerInit(FrameAssembler* self);
32
33 // Drop any partial frame (e.g. on disconnect). Keeps the rejected counter.
34 void frameAssemblerReset(FrameAssembler* self);
35
36 // Consume bytes from data[0..len) until one frame completes or the input ends.
37 // *consumed says how many bytes were used; call again with the remainder.
38 // On FRAME_READY, *payload / *payloadLen point into the assembler and stay
39 // valid until the next feed.
40 FrameStatus frameAssemblerFeed(FrameAssembler* self, const uint8_t* data, size_t len,
41                                size_t* consumed, const uint8_t** payload, size_t*
 >   payloadLen);
42
43 // Write the 2-byte header for a payload of payloadLen bytes. false if the
44 // length is 0 or above FRAME_MAX_PAYLOAD.
45 bool frameWriteHeader(uint8_t out[2], size_t payloadLen);
```

The payload pointer points into the assembler, not into newly allocated memory. Finish using or copy that payload before feeding another frame. The next feed may reset and reuse this same storage.

Reset progress without allocating another buffer

Trace a frame whose payload is 300 bytes. Its header is 0x2C, 0x01 because 44 + 256 = 300. Deliver those two header bytes in separate calls.

The implementation must preserve the first byte while waiting for the second.

src/domain/frame_codec.c / lines 1-14

```
#include "domain/frame_codec.h"

#include <string.h>

void frameAssemblerInit(FrameAssembler* self) { memset(self, 0, sizeof(*self)); }

void frameAssemblerReset(FrameAssembler* self) {
  self->used = 0;
  self->ready = false;
}

static size_t declaredLength(const FrameAssembler* self) {
  return (size_t)self->buf[0] | ((size_t)self->buf[1] << 8);
}
```

Initialisation zeroes the assembler. Reset clears progress and rejection flags while retaining the fixed storage. declaredLength reconstructs the integer with a shift and OR.

The uint16_t cast makes the high-byte operation explicit. These are numerical operations on unsigned bytes; the host processor's memory byte order does not determine the wire interpretation.

Collect a complete header before trusting its length

src/domain/frame_codec.c / lines 15-31

```
15
16 FrameStatus frameAssemblerFeed(FrameAssembler* self, const uint8_t* data, size_t len,
17                                size_t* consumed, const uint8_t** payload, size_t*
 >   payloadLen) {
18   *consumed = 0;
19   *payload = NULL;
20   *payloadLen = 0;
21   if (self->ready) frameAssemblerReset(self);  // the previous payload is released
22
23   while (*consumed < len) {
24     if (self->used < FRAME_HEADER_BYTES) {
25       self->buf[self->used++] = data[(*consumed)++];
26       if (self->used == FRAME_HEADER_BYTES) {
27         const size_t declared = declaredLength(self);
28         if (declared == 0 || declared > FRAME_MAX_PAYLOAD) {
29           frameAssemblerReset(self);
30           if (self->rejected < UINT32_MAX) ++self->rejected;
31           return FRAME_ERROR;
```

Feed clears a previously completed frame before accepting more input. It first copies enough bytes to obtain the header, then validates its declared length against both limits.

With only 0x2C available, the function cannot yet distinguish a 44-byte frame from a 300-byte one. It must return NEED_MORE rather than guessing. A rejected zero or oversized length clears assembly state for later input.

Copy only what this frame still needs

src/domain/frame_codec.c / lines 32-50

```
        }
      }
      continue;
    }
    const size_t total = FRAME_HEADER_BYTES +
  declaredLength(self);
    size_t take = total - self->used;
    if (take > len - *consumed) take = len - *consumed;
    memcpy(&self->buf[self->used], &data[*consumed], take);
    self->used += take;
    *consumed += take;
    if (self->used == total) {
      self->ready = true;
      *payload = &self->buf[FRAME_HEADER_BYTES];
      *payloadLen = total - FRAME_HEADER_BYTES;
      return FRAME_READY;
    }
  }
  return FRAME_NEED_MORE;
}
```

The minimum of remaining input and remaining frame capacity decides the copy length. consumed tells the caller where to resume if several frames arrived together. READY returns the complete payload and marks the retained frame ready.

Imagine the current frame needs 12 more bytes and a 40-byte input chunk arrives. The call consumes 12; the caller feeds the remaining 28 again. This loop is what makes coalesced messages work.

COMMON PITFALL

Never assume consumed always equals the supplied chunk length.

Write the same header format on transmission

src/domain/frame_codec.c / lines 51-57

```
51
52 bool frameWriteHeader(uint8_t out[2], size_t payloadLen) {
53   if (payloadLen == 0 || payloadLen > FRAME_MAX_PAYLOAD) return false;
54   out[0] = (uint8_t)(payloadLen & 0xFFu);
55   out[1] = (uint8_t)(payloadLen >> 8);
56   return true;
57 }
```

frameWriteHeader rejects the same invalid length range and writes the low byte before the high byte. No JSON is generated here.

The sender and receiver share one framing contract. The higher transport code reserves two bytes before its JSON payload so this function can fill the header without moving the whole payload.

Keep capacity beside the output address

Now the boundary is known. This header defines the message contents: bounded JSON output, views over application state and typed command input.

Read each group as a different direction of data movement. Views feed encoders; Command receives validated input.

COMMON PITFALL

Capacity includes the terminating NUL byte. A 40-byte array cannot hold 40 bytes of JSON plus its terminator.

src/domain/protocol_codec.h / lines 1-28

```
// JSON encoders and command parser for contract v2 (FirmwareContract.md).
// Pure C, no heap, no cJSON (the native test env has no ESP SDK). WP8.
//
// Encoders write the JSON payload only (no length header) into a JsonWriter
// over a caller buffer. On overflow the writer stops and reports it; the
// caller must not send a truncated frame.
//
// The parser accepts one flat command object with an optional flat "data"
// object, validates every field the operation needs and ignores unknown keys.
// Anything malformed is rejected before it can reach Configuration or Safety.
#pragma once
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

#include "domain/calibration.h"
#include "domain/reading.h"
#include "domain/sensor_set.h"

#define PROTOCOL_VERSION 2
#define PROTOCOL_ID_MAX 40
#define PROTOCOL_SSID_MAX 32        // bytes (802.11)
#define PROTOCOL_PASSWORD_MIN 8     // WPA2 passphrase, printable ASCII
#define PROTOCOL_PASSWORD_MAX 63
#define PROTOCOL_MAX_RULE_ID 15     // masks are 16 bits; Safety checks ruleCount

// ------------------------------------------------------------- writer
```

Keep capacity beside the output address / continued

JsonWriter stores a caller-owned buffer, capacity, length and overflow flag. Its final length becomes zero after overflow, so the caller cannot mistake a truncated document for a complete payload.

src/domain/protocol_codec.h / lines 29-55

```
typedef struct JsonWriter {
  char* buf;
  size_t cap;
  size_t len;
  bool overflow;
} JsonWriter;

void jsonWriterInit(JsonWriter* w, char* buf, size_t cap);
void jsonRaw(JsonWriter* w, const char* text);
void jsonString(JsonWriter* w, const char* s);  // quoted and escaped
void jsonKey(JsonWriter* w, const char* key);   // "key": (no comma handling)
void jsonU32(JsonWriter* w, uint32_t v);
void jsonI32(JsonWriter* w, int32_t v);
void jsonU64(JsonWriter* w, uint64_t v);
void jsonFloat(JsonWriter* w, float v);         // null for NaN / inf
void jsonBool(JsonWriter* w, bool v);
// Writes [i, j, ...] for the bits set in mask below `count`.
void jsonBitList(JsonWriter* w, uint16_t mask, uint8_t count);

// Payload length, or 0 if the writer overflowed.
static inline size_t jsonWriterLength(const JsonWriter* w) { return w->overflow ? 0 :
  w->len; }

// ------------------------------------------------------------------ encoders

typedef struct StateView {
  const char* node;
  const char* fw;
```

The small json functions handle strings, numbers and punctuation. jsonRaw deliberately bypasses escaping and expects trusted JSON fragments. A user-supplied string belongs in jsonString instead.

Describe state without exporting service objects

src/domain/protocol_codec.h / lines 56-78

```
  uint32_t boot;
  MonoMs uptimeMs;
  const char* link;        // "ble" (the only data link)
  const char* mode;        // lowercase power mode name
  uint32_t powerEpoch;
  uint16_t cfgRev;
  const char* wifiState;   // unconfigured / connecting / connected / failed
  const char* wifiSsid;    // NULL or "" when unconfigured
  const char* wifiIp;      // only sent when connected
  int32_t wifiRssi;
  bool relayOn;
  const char* lampReason;
  bool interlock;
  uint16_t latchedMask;
  uint16_t activeMask;
  uint16_t staleMask;
  uint8_t ruleCount;
  const char* sgp30Baseline;
  SensorMask enabled;
} StateView;

size_t protocolEncodeState(const StateView* s, char* buf, size_t cap);

```

src/domain/protocol_codec.h / lines 79-97

```
// All METRIC_COUNT entries keyed by metricName(). A value is sent only when
// the sensor is enabled, the stored health is OK or UNCALIBRATED and the
// sample is not older than alarmStaleLimitMs(); otherwise value is null and
// an aged OK/UNCALIBRATED reading is reported as STALE.
size_t protocolEncodeTelemetry(const Snapshot* snap, MonoMs now, uint32_t boot, uint16_t
    cfgRev,
                               SensorMask enabled, char* buf, size_t cap);

// status "ok" with dataJson (a complete JSON object, NULL means {}).
size_t protocolEncodeResultOk(const char* id, const char* dataJson, char* buf, size_t
  cap);
// status "error" with a short error code (see protocolErrorName()).
size_t protocolEncodeResultError(const char* id, const char* error, char* buf, size_t
  cap);
size_t protocolEncodeResultBusy(const char* id, char* buf, size_t cap);

typedef struct EventView {
  uint32_t n;
  uint8_t rule;
  const char* level;  // INFO / WARNING / CRITICAL
  const char* kind;   // trip / clear / reset
  uint32_t ageMs;
```

Describe state without exporting service objects / continued

src/domain/protocol_codec.h / lines 98-105

```
98    bool gap;
99  } EventView;
100
101 size_t protocolEncodeEvent(const EventView* e, char* buf, size_t cap);
102
103 // {"sensors":[{"key","label","metrics":[...],"enabled","depends_note"},...]}
104 size_t protocolEncodeSensorList(SensorMask enabled, char* buf, size_t cap);
105
```

StateView contains the specific values and string pointers needed for one synchronous encode call. It does not retain those pointers after returning. Telemetry receives a Snapshot plus now so it can calculate effective health.

src/domain/protocol_codec.h / lines 106-119

```
106 // Calibratable sensors (pH, TDS, light) and their stored fits.
107 typedef struct CalibrationView {
108   LinearCalibration ph;
109   LinearCalibration tds;
110   LinearCalibration light;
111   uint8_t overriddenMask;  // CAL_BIT_* : fitted on the device by the user
112 } CalibrationView;
113
114 // {"calibration":{"ph":{"overridden","rev","valid","slope","offset"},"tds":...,"light":
  >   ...}}
115 size_t protocolEncodeCalibration(const CalibrationView* v, char* buf, size_t cap);
116
117 // {"sensor":"ph","x":<x scaled>,"age_ms":n}
118 size_t protocolEncodeCapture(SensorId sensor, int32_t xScaled, uint32_t ageMs, char*
  >   buf, size_t cap);
119
```

Result and event encoders share the protocol envelope but carry different data. A gap flag reports missing event continuity; it is not a sensor-health code. CalibrationView exposes fits and revision information, not driver memory.

Assign explicit scales to calibration commands

src/domain/protocol_codec.h / lines 120-144

```
// ------------------------------------------------------------
  >   ------- commands

typedef uint8_t CommandOp;
enum {
  CMD_OP_NONE = 0,
  CMD_OP_GET_SENSORS,
  CMD_OP_SET_SENSOR_ENABLED,
  CMD_OP_RESET_LATCH,
  CMD_OP_SET_WIFI,
  CMD_OP_FORGET_WIFI,
  CMD_OP_GET_CALIBRATION,
  CMD_OP_CAL_CAPTURE,
  CMD_OP_SET_CALIBRATION,
  CMD_OP_RESET_CALIBRATION
};

// Calibration commands carry scaled integers (the parser
  >   keeps no floats):
//   ph     x = module mV x 10,           y = pH x 1000
  >   (two points)
//   tds    x = uncalibrated ppm x 10,    y = ppm x 10
  >   (one point, K)
//   light  x = pin mV x 10,              y = lux x 10
  >   (one point, gain)
// cal_capture returns x in the same scale.
#define CAL_X_SCALE 10.0f
#define CAL_PH_Y_SCALE 1000.0f
#define CAL_Y_SCALE 10.0f

```

COMMON PITFALL

An integer without its scale is incomplete information. Keep the scale with the field contract.

Assign explicit scales to calibration commands / continued

CommandOp lists the operations this firmware actually accepts. The command carries scaled integers: pH 7.000 becomes 7000; a module reading of 2500.0mV becomes 25000. This avoids accepting arbitrary floating-point syntax for fit inputs.

src/domain/protocol_codec.h / lines 145-151

```
145 typedef uint8_t ProtocolError;
146 enum {
147   PROTOCOL_OK = 0,
148   PROTOCOL_ERR_BAD_REQUEST,   // not JSON / not an object / wrong version / no id
149   PROTOCOL_ERR_UNKNOWN_OP,
150   PROTOCOL_ERR_BAD_FIELD,     // missing, wrong type or out of range
151   PROTOCOL_ERR_RULE_LATCHED,
```

The raw x scale and target y scale are different. The pH target uses 1000; TDS and lux targets use 10. The configuration owner converts them back before fitting.

Separate parse success from later execution success

src/domain/protocol_codec.h / lines 152-179

```
  PROTOCOL_ERR_RULE_ACTIVE,
  PROTOCOL_ERR_EXPIRED,
  PROTOCOL_ERR_TIMEOUT,
  PROTOCOL_ERR_STORAGE,
  PROTOCOL_ERR_STALE,         // cal_capture: no fresh usable reading (or TDS without
  >   water temperature)
  PROTOCOL_ERR_BAD_FIT        // set_calibration: the points give an implausible fit
};

typedef struct Command {
  CommandOp op;
  char id[PROTOCOL_ID_MAX + 1];  // set as soon as it parses, even on later errors
  SensorId sensor;
  bool enabled;
  uint8_t rule;
  char ssid[PROTOCOL_SSID_MAX + 1];
  char password[PROTOCOL_PASSWORD_MAX + 1];
  int32_t calX1, calY1, calX2, calY2;  // set_calibration, scaled (above)
  bool calTwoPoints;
} Command;

// Sensors that take a calibration command: SENSOR_PH, SENSOR_TDS, SENSOR_LIGHT.
bool protocolSensorCalibratable(SensorId s);

ProtocolError protocolParseCommand(const uint8_t* payload, size_t len, Command* out);

// Contract error string ("bad_request", "rule_latched", ...).
const char* protocolErrorName(ProtocolError e);

```

Command owns fixed-size copies of id, credentials and calibration numbers. It can outlive the input buffer after parsing. The error enum includes both parse failures and service outcomes such as storage failure or expired work.

src/domain/protocol_codec.h / lines 180-180

```
const char* commandOpName(CommandOp op);
```

protocolParseCommand proves that the requested fields have accepted shapes and ranges. It cannot prove the rail stayed valid, a rule has cleared or NVS will accept a write. Those decisions remain with their owners.

Keep one place responsible for output bounds

This is the largest pure C file in the project. Read it in three passes: writer mechanics, application encoders, then parser and operation validation.

You do not need to memorise all JSON punctuation. Follow capacity, pointer position and acceptance conditions.

REMEMBER

Review payload length and capacity (p. 568) before following put.

src/domain/protocol_codec.c / lines 1-28

```
#include "domain/protocol_codec.h"

#include <math.h>
#include <stdio.h>
#include <string.h>

#include "domain/alarm.h"

// ============================================================
  ======= writer

void jsonWriterInit(JsonWriter* w, char* buf, size_t cap) {
  w->buf = buf;
  w->cap = cap;
  w->len = 0;
  w->overflow = cap == 0;
  if (cap > 0) buf[0] = '\0';
}

static void put(JsonWriter* w, const char* text, size_t n) {
  if (w->overflow) return;
  if (w->len + n + 1 > w->cap) {  // keep room for the
  terminator
    w->overflow = true;
    return;
  }
  memcpy(&w->buf[w->len], text, n);
  w->len += n;
  w->buf[w->len] = '\0';
}
```

jsonWriterInit establishes an empty string. put appends only while space remains for a final NUL; otherwise it latches overflow. Later writes cannot turn that failed document into a valid partial reply.

For capacity 8, at most 7 content bytes fit. The distinction is visible here before any state encoder uses the helper. A private low-level helper makes that rule consistent across every message.

Escape the string content, then close the quotes

src/domain/protocol_codec.c / lines 29-54

```
29
30 void jsonRaw(JsonWriter* w, const char* text) { put(w, text, strlen(text)); }
31
32 void jsonString(JsonWriter* w, const char* s) {
33   put(w, "\"", 1);
34   if (s != NULL) {
35     for (const unsigned char* p = (const unsigned char*)s; *p != '\0'; ++p) {
36       switch (*p) {
37         case '"': put(w, "\\\"", 2); break;
38         case '\\': put(w, "\\\\", 2); break;
39         case '\n': put(w, "\\n", 2); break;
40         case '\r': put(w, "\\r", 2); break;
41         case '\t': put(w, "\\t", 2); break;
42         default:
43           if (*p < 0x20) {
44             char esc[8];
45             snprintf(esc, sizeof(esc), "\\u%04x", (unsigned)*p);
46             put(w, esc, 6);
47           } else {
48             put(w, (const char*)p, 1);  // UTF-8 bytes pass through
49           }
50       }
51     }
52   }
53   put(w, "\"", 1);
54 }
```

jsonRaw copies trusted text. jsonString adds the quotes and escapes quotation marks, backslashes and control bytes. Otherwise a device name containing a quote could accidentally become JSON structure.

The loop reads unsigned bytes so control-byte comparisons do not depend on whether plain char is signed. This is byte-oriented escaping; it does not validate all incoming UTF-8 text.

Convert numeric values into bounded temporary strings

src/domain/protocol_codec.c / lines 55-78

```
55
56 void jsonKey(JsonWriter* w, const char* key) {
57   jsonString(w, key);
58   put(w, ":", 1);
59 }
60
61 void jsonU32(JsonWriter* w, uint32_t v) {
62   char tmp[12];
63   const int n = snprintf(tmp, sizeof(tmp), "%lu", (unsigned long)v);
64   put(w, tmp, (size_t)n);
65 }
66
67 void jsonI32(JsonWriter* w, int32_t v) {
68   char tmp[12];
69   const int n = snprintf(tmp, sizeof(tmp), "%ld", (long)v);
70   put(w, tmp, (size_t)n);
71 }
72
73 void jsonU64(JsonWriter* w, uint64_t v) {
74   char tmp[24];
75   const int n = snprintf(tmp, sizeof(tmp), "%llu", (unsigned long long)v);
76   put(w, tmp, (size_t)n);
77 }
78
```

The integer helpers format into small local arrays, then pass those arrays to the bounded writer. PRI macros choose a format suitable for the fixed-width type. A 64-bit uptime needs a 64-bit format.

src/domain/protocol_codec.c / lines 79-87

```
79 void jsonFloat(JsonWriter* w, float v) {
80   if (isnan(v) || isinf(v)) {
81     put(w, "null", 4);
82     return;
83   }
84   char tmp[24];
85   const int n = snprintf(tmp, sizeof(tmp), "%.7g", (double)v);  // float has ~7 digits
86   put(w, tmp, (size_t)n);
87 }
```

jsonFloat emits null for NaN or infinity, since those C floating-point values are not JSON numbers. A sensor fault should already be expressed through health, but this last boundary also prevents invalid JSON.

Encode booleans and bit positions explicitly

src/domain/protocol_codec.c / lines 88-114

```
88
89 void jsonBool(JsonWriter* w, bool v) {
90   if (v) {
91     put(w, "true", 4);
92   } else {
93     put(w, "false", 5);
94   }
95 }
96
97 void jsonBitList(JsonWriter* w, uint16_t mask, uint8_t count) {
98   put(w, "[", 1);
99   bool first = true;
100  for (uint8_t i = 0; i < count && i < 16; ++i) {
101    if ((mask & (1u << i)) == 0) continue;
102    if (!first) put(w, ",", 1);
103    jsonU32(w, i);
104    first = false;
105  }
106  put(w, "]", 1);
107 }
108
109 // ================================================================ encoders
110
111 static void header(JsonWriter* w, const char* type) {
112   jsonRaw(w, "{\"v\":2,\"type\":");
113   jsonString(w, type);
114 }
```

jsonBool emits JSON true or false without quotes. jsonBitList visits each supported bit and writes the indices that are set, inserting commas only between entries.

header opens the shared versioned envelope. The encoders that follow must close the structure they opened. Keeping common envelope text together reduces accidental differences between State, Telemetry and Result.

Begin the State object with identity and timing

src/domain/protocol_codec.c / lines 115-142

```
115
116 size_t protocolEncodeState(const StateView* s, char* buf, size_t cap) {
117   JsonWriter w;
118   jsonWriterInit(&w, buf, cap);
119   header(&w, "state");
120   jsonRaw(&w, ",");
121   jsonKey(&w, "node");
122   jsonString(&w, s->node);
123   jsonRaw(&w, ",");
124   jsonKey(&w, "fw");
125   jsonString(&w, s->fw);
126   jsonRaw(&w, ",");
127   jsonKey(&w, "boot");
128   jsonU32(&w, s->boot);
129   jsonRaw(&w, ",");
130   jsonKey(&w, "uptime_ms");
131   jsonU64(&w, s->uptimeMs);
132   jsonRaw(&w, ",");
133   jsonKey(&w, "link");
134   jsonString(&w, s->link);
135   jsonRaw(&w, ",");
136   jsonKey(&w, "mode");
137   jsonString(&w, s->mode);
138   jsonRaw(&w, ",");
139   jsonKey(&w, "power_epoch");
140   jsonU32(&w, s->powerEpoch);
141   jsonRaw(&w, ",");
142   jsonKey(&w, "cfg_rev");
```

protocolEncodeState writes node, firmware, boot identity, uptime, transport and power mode from its view. The boot number distinguishes repeated sequence numbers after a reboot.

The writer copies string content during this call. It does not store StateView or borrow its pointers for later network work. Encoding and sending remain separate operations.

Represent optional network state deliberately

src/domain/protocol_codec.c / lines 143-169

```
  jsonU32(&w, s->cfgRev);

  jsonRaw(&w, ",\"wifi\":{");
  jsonKey(&w, "state");
  jsonString(&w, s->wifiState);
  if (s->wifiSsid != NULL && s->wifiSsid[0] != '\0') {
    jsonRaw(&w, ",");
    jsonKey(&w, "ssid");
    jsonString(&w, s->wifiSsid);
  }
  if (s->wifiIp != NULL && s->wifiState != NULL && strcmp(s->wifiState, "connected") ==
  0) {
    jsonRaw(&w, ",");
    jsonKey(&w, "ip");
    jsonString(&w, s->wifiIp);
    jsonRaw(&w, ",");
    jsonKey(&w, "rssi");
    jsonI32(&w, s->wifiRssi);
  }
  jsonRaw(&w, "}");

  jsonRaw(&w, ",\"safety\":{");
  jsonKey(&w, "relay");
  jsonBool(&w, s->relayOn);
  jsonRaw(&w, ",");
  jsonKey(&w, "lamp_reason");
  jsonString(&w, s->lampReason);
  jsonRaw(&w, ",");
```

The Wi-Fi fields depend on what is known. A missing SSID or IP is encoded through the chosen null/conditional representation rather than using an old buffer accidentally.

SSID is descriptive state; the password is absent from StateView. Notice that omission in the interface itself: the encoder never needs the secret to report connection status.

Finish the control and sensor status fields

src/domain/protocol_codec.c / lines 170-196

```
  jsonKey(&w, "interlock");
  jsonBool(&w, s->interlock);
  jsonRaw(&w, ",");
  jsonKey(&w, "latched");
  jsonBitList(&w, s->latchedMask, s->ruleCount);
  jsonRaw(&w, ",");
  jsonKey(&w, "active");
  jsonBitList(&w, s->activeMask, s->ruleCount);
  jsonRaw(&w, ",");
  jsonKey(&w, "stale");
  jsonBitList(&w, s->staleMask, s->ruleCount);
  jsonRaw(&w, "}");

  jsonRaw(&w, ",");
  jsonKey(&w, "sgp30_baseline");
  jsonString(&w, s->sgp30Baseline);
  jsonRaw(&w, ",\"enabled\":[");
  bool first = true;
  for (uint8_t i = 0; i < SENSOR_COUNT; ++i) {
    if (!sensorMaskHas(s->enabled, (SensorId)i)) continue;
    if (!first) jsonRaw(&w, ",");
    jsonString(&w, sensorKey((SensorId)i));
    first = false;
  }
  jsonRaw(&w, "]}");
  return jsonWriterLength(&w);
}
```

Relay state, lamp reason, masks, baseline status and the enabled set describe controller decisions. They are different from the latest numeric readings. The final return asks the writer whether the whole document fitted.

A zero length means do not transmit this output. It does not mean an empty, valid State message. The caller checks that outcome before building a frame.

Decide whether a retained number may leave the device

src/domain/protocol_codec.c / lines 197-224

```
197
198 static void encodeMetric(JsonWriter* w, const Reading* r,
  >   MonoMs now, bool enabled) {
199   const bool measured = r->intervalEndMs != 0 && r->health
  >   != HEALTH_STARTING;
200   const MonoMs age = measured && now > r->intervalEndMs ?
  >   now - r->intervalEndMs : 0;
201   Health health = r->health;
202   bool usable = health == HEALTH_OK || health ==
  >   HEALTH_UNCALIBRATED;
203   if (usable && age > alarmStaleLimitMs(r->metric)) {
204     health = HEALTH_STALE;  // never send a retained value
  >   as a new observation (§10)
205     usable = false;
206   }
207
208   jsonKey(w, metricName(r->metric));
209   jsonRaw(w, "{\"value\":");
210   if (usable && enabled) {
211     jsonFloat(w, r->value);
212   } else {
213     jsonRaw(w, "null");
214   }
215   jsonRaw(w, ",");
216   jsonKey(w, "unit");
217   jsonString(w, unitName(r->unit));
218   jsonRaw(w, ",");
219   jsonKey(w, "health");
220   jsonString(w, healthName(health));
221   jsonRaw(w, ",");
222   jsonKey(w, "age_ms");
223   if (measured) {
224     jsonU64(w, age);
```

encodeMetric calculates age from the measurement endpoint. Both OK and UNCALIBRATED can become STALE for this output policy; disabled sensors or unusable health produce a null value.

COMMON PITFALL

Telemetry display eligibility does not imply permission to drive the relay.

Decide whether a retained number may leave the device / continued

src/domain/protocol_codec.c / lines 225-232

```
  } else {
    jsonRaw(w, "null");
  }
  jsonRaw(w, ",");
  jsonKey(w, "seq");
  jsonU32(w, r->sequence);
  jsonRaw(w, ",");
  jsonKey(w, "q");
```

src/domain/protocol_codec.c / lines 233-238

```
  jsonU32(w, r->qualityFlags);
  jsonRaw(w, ",");
  jsonKey(w, "cal_rev");
  jsonU32(w, r->calibrationRevision);
  jsonRaw(w, "}");
}
```

This policy is deliberately broader than Safety's actionable-OK check: a user may need to see an uncalibrated raw observation while commissioning a sensor. Read unit, health and quality beside the number.

Walk the fixed metric array once

src/domain/protocol_codec.c / lines 239-259

```
239
240 size_t protocolEncodeTelemetry(const Snapshot* snap, MonoMs now, uint32_t boot, uint16_t
  >     cfgRev,
241                                 SensorMask enabled, char* buf, size_t cap) {
242   JsonWriter w;
243   jsonWriterInit(&w, buf, cap);
244   header(&w, "telemetry");
245   jsonRaw(&w, ",");
246   jsonKey(&w, "boot");
247   jsonU32(&w, boot);
248   jsonRaw(&w, ",");
249   jsonKey(&w, "cfg_rev");
250   jsonU32(&w, cfgRev);
251   jsonRaw(&w, ",\"m\":{");
252   for (uint8_t i = 0; i < METRIC_COUNT; ++i) {
253     if (i > 0) jsonRaw(&w, ",");
254     const Reading* r = &snap->readings[i];
255     encodeMetric(&w, r, now, sensorMaskHas(enabled, sensorOfMetric(r->metric)));
256   }
257   jsonRaw(&w, "}}");
258   return jsonWriterLength(&w);
259 }
```

The telemetry encoder visits every MetricId and delegates its object to encodeMetric. The shared boot and configuration revision describe the message context, while each record retains its own sequence and measurement metadata.

A Snapshot is a collection of retained observations. Iterating it does not resample sensors or make all its values simultaneous.

Give results a shared correlation identifier

src/domain/protocol_codec.c / lines 260-287

```
260
261 static void resultHead(JsonWriter* w, const char* id, const char* status) {
262   header(w, "result");
263   jsonRaw(w, ",");
264   jsonKey(w, "id");
265   jsonString(w, id);
266   jsonRaw(w, ",");
267   jsonKey(w, "status");
268   jsonString(w, status);
269 }
270
271 size_t protocolEncodeResultOk(const char* id, const char* dataJson, char* buf, size_t
  >   cap) {
272   JsonWriter w;
273   jsonWriterInit(&w, buf, cap);
274   resultHead(&w, id, "ok");
275   jsonRaw(&w, ",\"data\":");
276   jsonRaw(&w, dataJson != NULL ? dataJson : "{}");
277   jsonRaw(&w, "}");
278   return jsonWriterLength(&w);
279 }
280
281 size_t protocolEncodeResultError(const char* id, const char* error, char* buf, size_t
  >   cap) {
282   JsonWriter w;
283   jsonWriterInit(&w, buf, cap);
284   resultHead(&w, id, "error");
285   jsonRaw(&w, ",");
286   jsonKey(&w, "error");
287   jsonString(&w, error);
```

resultHead starts a result with the supplied request id. The three public variants then emit accepted data, a named error or busy. A client uses id to associate this asynchronous message with its request.

Give results a shared correlation identifier / continued

src/domain/protocol_codec.c / lines 288-298

```
288   jsonRaw(&w, "}");
289   return jsonWriterLength(&w);
290 }
291
292 size_t protocolEncodeResultBusy(const char* id, char* buf, size_t cap) {
293   JsonWriter w;
294   jsonWriterInit(&w, buf, cap);
295   resultHead(&w, id, "busy");
296   jsonRaw(&w, "}");
297   return jsonWriterLength(&w);
298 }
```

dataJson is inserted as already-built JSON. Only an encoder-owned fragment belongs there. Sending arbitrary unescaped user text through this parameter would break the structure.

Encode events and the fixed sensor catalogue

src/domain/protocol_codec.c / lines 299-325

```
299
300 size_t protocolEncodeEvent(const EventView* e, char* buf, size_t cap) {
301   JsonWriter w;
302   jsonWriterInit(&w, buf, cap);
303   header(&w, "event");
304   jsonRaw(&w, ",");
305   jsonKey(&w, "n");
306   jsonU32(&w, e->n);
307   jsonRaw(&w, ",");
308   jsonKey(&w, "rule");
309   jsonU32(&w, e->rule);
310   jsonRaw(&w, ",");
311   jsonKey(&w, "level");
312   jsonString(&w, e->level);
313   jsonRaw(&w, ",");
314   jsonKey(&w, "kind");
315   jsonString(&w, e->kind);
316   jsonRaw(&w, ",");
317   jsonKey(&w, "age_ms");
318   jsonU32(&w, e->ageMs);
319   jsonRaw(&w, ",");
320   jsonKey(&w, "gap");
321   jsonBool(&w, e->gap);
322   jsonRaw(&w, "}");
323   return jsonWriterLength(&w);
324 }
325
```

src/domain/protocol_codec.c / lines 326-343

```
326 size_t protocolEncodeSensorList(SensorMask enabled, char* buf, size_t cap) {
327   JsonWriter w;
328   jsonWriterInit(&w, buf, cap);
329   jsonRaw(&w, "{\"sensors\":[");
330   for (uint8_t s = 0; s < SENSOR_COUNT; ++s) {
331     if (s > 0) jsonRaw(&w, ",");
332     jsonRaw(&w, "{");
333     jsonKey(&w, "key");
334     jsonString(&w, sensorKey((SensorId)s));
335     jsonRaw(&w, ",");
336     jsonKey(&w, "label");
337     jsonString(&w, sensorLabel((SensorId)s));
338     jsonRaw(&w, ",\"metrics\":[");
339     bool first = true;
340     for (uint8_t m = 0; m < METRIC_COUNT; ++m) {
341       if (sensorOfMetric((MetricId)m) != s) continue;
342       if (!first) jsonRaw(&w, ",");
343       jsonString(&w, metricName((MetricId)m));
```

Encode events and the fixed sensor catalogue / continued

src/domain/protocol_codec.c / lines 344-353

```
344         first = false;
345       }
346       jsonRaw(&w, "],");
347       jsonKey(&w, "enabled");
348       jsonBool(&w, sensorMaskHas(enabled, (SensorId)s));
349       jsonRaw(&w, ",");
350       jsonKey(&w, "depends_note");
351       jsonString(&w, sensorDependsNote((SensorId)s));
352       jsonRaw(&w, "}");
353     }
```

The event encoder reports sequence, rule, severity, kind, age and continuity. The sensor-list loop emits the known board sensor keys and enumerates their metrics through sensorOfMetric.

src/domain/protocol_codec.c / lines 354-356

```
354     jsonRaw(&w, "]}");
355     return jsonWriterLength(&w);
356 }
```

A list request is a description of supported hardware and enable state; it does not probe the bus. Dependency notes explain why disabling water temperature affects TDS and disabling air measurements affects gas compensation.

Report fitted coefficients and captured raw inputs

src/domain/protocol_codec.c / lines 357-384

```
357
358 static void encodeFit(JsonWriter* w, const char* key, const LinearCalibration* c, bool
  >   overridden) {
359   jsonKey(w, key);
360   jsonRaw(w, "{");
361   jsonKey(w, "overridden");
362   jsonBool(w, overridden);
363   jsonRaw(w, ",");
364   jsonKey(w, "rev");
365   jsonU32(w, c->revision);
366   jsonRaw(w, ",");
367   jsonKey(w, "valid");
368   jsonBool(w, c->valid);
369   jsonRaw(w, ",");
370   jsonKey(w, "slope");
371   jsonFloat(w, c->slope);
372   jsonRaw(w, ",");
373   jsonKey(w, "offset");
374   jsonFloat(w, c->offset);
375   jsonRaw(w, "}");
376 }
377
378 size_t protocolEncodeCalibration(const CalibrationView* v, char* buf, size_t cap) {
379   JsonWriter w;
380   jsonWriterInit(&w, buf, cap);
381   jsonRaw(&w, "{\"calibration\":{");
382   encodeFit(&w, "ph", &v->ph, (v->overriddenMask & CAL_BIT_PH) != 0);
383   jsonRaw(&w, ",");
384   encodeFit(&w, "tds", &v->tds, (v->overriddenMask & CAL_BIT_TDS) != 0);
```

encodeFit writes the stored fit with validity, revision and override status. protocolEncodeCapture reports a raw x with its declared scale and age so a later calibration command can use the same numerical convention.

src/domain/protocol_codec.c / lines 385-395

```
385   jsonRaw(&w, ",");
386   encodeFit(&w, "light", &v->light, (v->overriddenMask & CAL_BIT_LIGHT) != 0);
387   jsonRaw(&w, "}}");
388   return jsonWriterLength(&w);
389 }
390
391 size_t protocolEncodeCapture(SensorId sensor, int32_t xScaled, uint32_t ageMs, char*
  >   buf, size_t cap) {
392   JsonWriter w;
393   jsonWriterInit(&w, buf, cap);
394   jsonRaw(&w, "{");
395   jsonKey(&w, "sensor");
```

Report fitted coefficients and captured raw inputs / continued

src/domain/protocol_codec.c / lines 396-405

```
396    jsonString(&w, sensorKey(sensor));
397    jsonRaw(&w, ",");
398    jsonKey(&w, "x");
399    jsonI32(&w, xScaled);
400    jsonRaw(&w, ",");
401    jsonKey(&w, "age_ms");
402    jsonU32(&w, ageMs);
403    jsonRaw(&w, "}");
404    return jsonWriterLength(&w);
405  }
```

A capture is not a new physical sample. It is a checked retained observation. The BLE owner will supply that observation through snapshotPeek instead of calling the sensor driver.

Reserve parser limits before reading input

src/domain/protocol_codec.c / lines 406-432

```
406
407 // ============================================================== parser
408
409 #define FIELD_KEY_MAX 15
410 #define FIELD_STR_MAX 64
411 #define DATA_FIELDS_MAX 8
412 #define SKIP_DEPTH_MAX 4
413
414 typedef uint8_t JsonType;
415 enum { JT_NONE = 0, JT_STRING, JT_NUMBER, JT_BOOL, JT_NULL, JT_OBJECT, JT_ARRAY };
416
417 typedef struct Field {
418   char key[FIELD_KEY_MAX + 1];
419   JsonType type;
420   char str[FIELD_STR_MAX + 1];  // decoded UTF-8 for JT_STRING
421   size_t strLen;
422   bool strTooLong;
423   bool isInteger;               // JT_NUMBER without fraction/exponent, in int32 range
424   int32_t integer;
425   bool boolean;
426 } Field;
427
428 typedef struct Parser {
429   const uint8_t* p;
430   const uint8_t* end;
431 } Parser;
432
```

Field uses fixed arrays for key and decoded string plus explicit type flags. Parser stores a current pointer and an end pointer; the input need not end in a NUL byte. The field count and nesting limits bound retained work.

strLen tracks the decoded logical length even if storage fills. That lets later checks reject an overlong password rather than silently accepting its truncated prefix.

Advance the cursor only through small checked helpers

src/domain/protocol_codec.c / lines 433-459

```
static void skipWs(Parser* ps) {
  while (ps->p < ps->end && (*ps->p == ' ' || *ps->p == '\t' || *ps->p == '\n' || *ps->p
   == '\r')) {
    ++ps->p;
  }
}

static bool expect(Parser* ps, char c) {
  skipWs(ps);
  if (ps->p >= ps->end || *ps->p != (uint8_t)c) return false;
  ++ps->p;
  return true;
}

static bool hexValue(uint8_t c, uint32_t* out) {
  if (c >= '0' && c <= '9') *out = (uint32_t)(c - '0');
  else if (c >= 'a' && c <= 'f') *out = (uint32_t)(c - 'a' + 10);
  else if (c >= 'A' && c <= 'F') *out = (uint32_t)(c - 'A' + 10);
  else return false;
  return true;
}

static void appendByte(char* out, size_t cap, size_t* len, bool* tooLong, uint8_t b) {
  if (*len < cap) {
    out[*len] = (char)b;
  } else {
    *tooLong = true;
  }
```

skipWs moves over permitted whitespace. expect checks the next structural character. hexValue interprets one hexadecimal digit. appendByte records decoded length while restricting writes to the destination capacity.

src/domain/protocol_codec.c / lines 460-461

```
  ++*len;
}
```

These helpers separate reading bounds from higher-level meaning. Reaching the end pointer while expecting a quote is a syntax failure, even if the bytes copied so far look plausible.

Decode escapes while preserving length evidence

src/domain/protocol_codec.c / lines 462-489

```
462
463 // Parses a JSON string at ps->p (opening quote included) into out (cap bytes,
464 // always NUL-terminated within cap+1). Surrogate pairs are not supported (no
465 // field the firmware reads needs characters outside the BMP).
466 static bool parseString(Parser* ps, char* out, size_t cap, size_t* lenOut, bool*
  >   tooLong) {
467   size_t len = 0;
468   *tooLong = false;
469   if (!expect(ps, '"')) return false;
470   while (ps->p < ps->end) {
471     const uint8_t c = *ps->p++;
472     if (c == '"') {
473       out[len < cap ? len : cap] = '\0';
474       *lenOut = len;
475       return true;
476     }
477     if (c < 0x20) return false;
478     if (c != '\\') {
479       appendByte(out, cap, &len, tooLong, c);
480       continue;
481     }
482     if (ps->p >= ps->end) return false;
483     const uint8_t e = *ps->p++;
484     switch (e) {
485       case '"': appendByte(out, cap, &len, tooLong, '"'); break;
486       case '\\': appendByte(out, cap, &len, tooLong, '\\'); break;
487       case '/': appendByte(out, cap, &len, tooLong, '/'); break;
488       case 'b': appendByte(out, cap, &len, tooLong, '\b'); break;
489       case 'f': appendByte(out, cap, &len, tooLong, '\f'); break;
```

src/domain/protocol_codec.c / lines 490-505

```
490       case 'n': appendByte(out, cap, &len, tooLong, '\n'); break;
491       case 'r': appendByte(out, cap, &len, tooLong, '\r'); break;
492       case 't': appendByte(out, cap, &len, tooLong, '\t'); break;
493       case 'u': {
494         if (ps->end - ps->p < 4) return false;
495         uint32_t cp = 0;
496         for (int i = 0; i < 4; ++i) {
497           uint32_t nib;
498           if (!hexValue(*ps->p++, &nib)) return false;
499           cp = (cp << 4) | nib;
500         }
501         if (cp == 0 || (cp >= 0xD800 && cp <= 0xDFFF)) return false;
502         if (cp < 0x80) {
503           appendByte(out, cap, &len, tooLong, (uint8_t)cp);
504         } else if (cp < 0x800) {
505           appendByte(out, cap, &len, tooLong, (uint8_t)(0xC0 | (cp >> 6)));
```

Decode escapes while preserving length evidence / continued

src/domain/protocol_codec.c / lines 506-516

```
506             appendByte(out, cap, &len, tooLong, (uint8_t)(0x80 | (cp & 0x3F)));
507           } else {
508             appendByte(out, cap, &len, tooLong, (uint8_t)(0xE0 | (cp >> 12)));
509             appendByte(out, cap, &len, tooLong, (uint8_t)(0x80 | ((cp >> 6) & 0x3F)));
510             appendByte(out, cap, &len, tooLong, (uint8_t)(0x80 | (cp & 0x3F)));
511           }
512           break;
513         }
514         default: return false;
515       }
516     }
```

parseString requires opening and closing quotes, rejects unescaped control bytes and handles JSON escape sequences. A Unicode escape uses four checked hexadecimal digits; unsupported NUL and surrogate values are rejected.

src/domain/protocol_codec.c / lines 517-518

```
517   return false;  // unterminated
518 }
```

Notice the difference between escaped source bytes and decoded bytes: the six characters backslash-u-0-0-4-1 represent one decoded A. Length limits apply to what the operation will actually store. This implementation supports the written escape rules; it is not a general Unicode normalisation library.

Validate JSON number grammar before accepting an integer

src/domain/protocol_codec.c / lines 519-544

```
519
520 static bool isDigit(uint8_t c) { return c >= '0' && c <=
  >   '9'; }
521
522 static bool parseNumber(Parser* ps, Field* f) {
523   const uint8_t* p = ps->p;
524   bool negative = false;
525   if (p < ps->end && *p == '-') {
526     negative = true;
527     ++p;
528   }
529   if (p >= ps->end || !isDigit(*p)) return false;
530   int64_t value = 0;
531   bool overflow = false;
532   if (*p == '0') {
533     ++p;
534   } else {
535     while (p < ps->end && isDigit(*p)) {
536       // Past the supported range: stop accumulating (no
  >   signed overflow on a
537       // long number), keep validating the digits.
538       if (!overflow) {
539         value = value * 10 + (*p - '0');
540         if (value > 2147483648LL) overflow = true;
541       }
542       ++p;
543     }
544   }
```

The parser recognises optional minus, integer digits, optional fraction and optional exponent. It separately records whether the result is a representable int32_t with no fractional or exponent form.

src/domain/protocol_codec.c / lines 545-557

```
545   bool integer = true;
546   if (p < ps->end && *p == '.') {
547     integer = false;
548     ++p;
549     if (p >= ps->end || !isDigit(*p)) return false;
550     while (p < ps->end && isDigit(*p)) ++p;
551   }
552   if (p < ps->end && (*p == 'e' || *p == 'E')) {
553     integer = false;
554     ++p;
555     if (p < ps->end && (*p == '+' || *p == '-')) ++p;
556     if (p >= ps->end || !isDigit(*p)) return false;
557     while (p < ps->end && isDigit(*p)) ++p;
```

COMMON PITFALL

Do not cast a parsed floating value into an integer and silently accept a changed command.

Validate JSON number grammar before accepting an integer / continued

src/domain/protocol_codec.c / lines 558-565

```
  }
  ps->p = p;
  f->type = JT_NUMBER;
  if (negative) value = -value;
  f->isInteger = integer && !overflow && value >= INT32_MIN && value <= INT32_MAX;
  f->integer = f->isInteger ? (int32_t)value : 0;
  return true;
}
```

The command fields require integers even though JSON permits other numbers. A syntactically legal 1.5 can therefore pass number parsing and still fail field validation. Those are two different decisions.

Skip unknown structures without accepting malformed JSON

src/domain/protocol_codec.c / lines 566-592

```
static bool matchLiteral(Parser* ps, const char* lit) {
  const size_t n = strlen(lit);
  if ((size_t)(ps->end - ps->p) < n || memcmp(ps->p, lit, n) != 0) return false;
  ps->p += n;
  return true;
}

static bool skipValue(Parser* ps, int depth);

static bool skipContainer(Parser* ps, int depth, char close, bool object) {
  if (depth > SKIP_DEPTH_MAX) return false;
  ++ps->p;  // opening bracket
  skipWs(ps);
  if (ps->p < ps->end && *ps->p == (uint8_t)close) {
    ++ps->p;
    return true;
  }
  for (;;) {
    if (object) {
      char key[FIELD_KEY_MAX + 1];
      size_t keyLen;
      bool tooLong;
      skipWs(ps);
      if (!parseString(ps, key, FIELD_KEY_MAX, &keyLen, &tooLong)) return false;
      if (!expect(ps, ':')) return false;
    }
```

matchLiteral advances only after matching the complete expected word. skipContainer checks braces, brackets, separators and nested values up to the configured depth.

Skip unknown structures without accepting malformed JSON / continued

src/domain/protocol_codec.c / lines 593-606

```
    if (!skipValue(ps, depth + 1)) return false;
    skipWs(ps);
    if (ps->p >= ps->end) return false;
    if (*ps->p == ',') {
      ++ps->p;
      continue;
    }
    if (*ps->p == (uint8_t)close) {
      ++ps->p;
      return true;
    }
    return false;
  }
}
```

Ignoring an unknown field means ignoring its meaning. It does not mean ignoring broken syntax. A missing comma inside an unused object still rejects the document.

Retain scalar types rather than only their text

src/domain/protocol_codec.c / lines 607-634

```
607
608 // Parses any scalar into f, or skips a container (type JT_OBJECT/JT_ARRAY).
609 static bool parseScalarOrSkip(Parser* ps, Field* f, int depth) {
610   skipWs(ps);
611   if (ps->p >= ps->end) return false;
612   const uint8_t c = *ps->p;
613   if (c == '"') {
614     f->type = JT_STRING;
615     return parseString(ps, f->str, FIELD_STR_MAX, &f->strLen, &f->strTooLong);
616   }
617   if (c == '{') {
618     f->type = JT_OBJECT;
619     return skipContainer(ps, depth, '}', true);
620   }
621   if (c == '[') {
622     f->type = JT_ARRAY;
623     return skipContainer(ps, depth, ']', false);
624   }
625   if (c == 't' && matchLiteral(ps, "true")) {
626     f->type = JT_BOOL;
627     f->boolean = true;
628     return true;
629   }
630   if (c == 'f' && matchLiteral(ps, "false")) {
631     f->type = JT_BOOL;
632     f->boolean = false;
633     return true;
634   }
```

parseScalarOrSkip distinguishes strings, booleans, null, integer-capable numbers and containers. The scratch Field in skipValue provides the same grammar checks when no application field needs the content.

Retain scalar types rather than only their text / continued

src/domain/protocol_codec.c / lines 635-646

```
  if (c == 'n' && matchLiteral(ps, "null")) {
    f->type = JT_NULL;
    return true;
  }
  return parseNumber(ps, f);
}

static bool skipValue(Parser* ps, int depth) {
  Field scratch;
  memset(&scratch, 0, sizeof(scratch));
  return parseScalarOrSkip(ps, &scratch, depth);
}
```

This is why enabled:true is accepted but enabled:"true" is not. The visible word is similar; the parsed type is different.

Collect a bounded flat data object

src/domain/protocol_codec.c / lines 647-672

```
647
648 // Parses the flat "data" object; nested values are skipped (type kept).
649 static bool parseData(Parser* ps, Field* fields, uint8_t* count) {
650   *count = 0;
651   if (!expect(ps, '{')) return false;
652   skipWs(ps);
653   if (ps->p < ps->end && *ps->p == '}') {
654     ++ps->p;
655     return true;
656   }
657   for (;;) {
658     Field f;
659     memset(&f, 0, sizeof(f));
660     size_t keyLen;
661     bool keyTooLong;
662     skipWs(ps);
663     if (!parseString(ps, f.key, FIELD_KEY_MAX, &keyLen, &keyTooLong)) return false;
664     if (!expect(ps, ':')) return false;
665     if (!parseScalarOrSkip(ps, &f, 2)) return false;
666     if (!keyTooLong && *count < DATA_FIELDS_MAX) fields[(*count)++] = f;
667     skipWs(ps);
668     if (ps->p >= ps->end) return false;
669     if (*ps->p == ',') {
670       ++ps->p;
671       continue;
672     }
```

parseData retains only the bounded number of usable fields, yet continues checking the whole object. Nested objects are structurally skipped rather than interpreted as command data.

src/domain/protocol_codec.c / lines 673-679

```
673     if (*ps->p == '}') {
674       ++ps->p;
675       return true;
676     }
677     return false;
678   }
679 }
```

An input can be well-formed JSON and still fail because the required field was absent, mistyped, too long or beyond retained limits. This design favours a small fixed command vocabulary over a general JSON tree.

Map accepted names and check text policy

src/domain/protocol_codec.c / lines 680-704

```
680
681 static const Field* findField(const Field* fields, uint8_t count, const char* key) {
682   for (uint8_t i = 0; i < count; ++i) {
683     if (strcmp(fields[i].key, key) == 0) return &fields[i];
684   }
685   return NULL;
686 }
687
688 typedef struct OpName {
689   const char* name;
690   CommandOp op;
691 } OpName;
692
693 static const OpName kOps[] = {
694     {"get_sensors", CMD_OP_GET_SENSORS},
695     {"set_sensor_enabled", CMD_OP_SET_SENSOR_ENABLED},
696     {"reset_latch", CMD_OP_RESET_LATCH},
697     {"set_wifi", CMD_OP_SET_WIFI},
698     {"forget_wifi", CMD_OP_FORGET_WIFI},
699     {"get_calibration", CMD_OP_GET_CALIBRATION},
700     {"cal_capture", CMD_OP_CAL_CAPTURE},
701     {"set_calibration", CMD_OP_SET_CALIBRATION},
702     {"reset_calibration", CMD_OP_RESET_CALIBRATION},
703 };
704
```

findField selects a stored key. kOps associates wire spellings with CommandOp. commandOpName uses the same table in the other direction. printableAscii enforces the command text policy byte by byte.

Map accepted names and check text policy / continued

src/domain/protocol_codec.c / lines 705-717

```
const char* commandOpName(CommandOp op) {
  for (size_t i = 0; i < sizeof(kOps) / sizeof(kOps[0]); ++i) {
    if (kOps[i].op == op) return kOps[i].name;
  }
  return "?";
}

static bool printableAscii(const char* s, size_t len) {
  for (size_t i = 0; i < len; ++i) {
    if ((uint8_t)s[i] < 0x20 || (uint8_t)s[i] > 0x7E) return false;
  }
  return true;
}
```

The search returns the first matching stored data field. Do not design a client that relies on duplicate-key ambiguity. Send each expected key once, with its intended type.

Turn typed fields into domain values

src/domain/protocol_codec.c / lines 718-735

```
718
719 bool protocolSensorCalibratable(SensorId s) {
720   return s == SENSOR_PH || s == SENSOR_TDS || s == SENSOR_LIGHT;
721 }
722
723 static bool sensorField(const Field* data, uint8_t count, Command* out) {
724   const Field* s = findField(data, count, "sensor");
725   if (s == NULL || s->type != JT_STRING || s->strTooLong) return false;
726   out->sensor = sensorFromKey(s->str, s->strLen);
727   return out->sensor < SENSOR_COUNT;
728 }
729
730 static bool intField(const Field* data, uint8_t count, const char* key, int32_t min,
  >   int32_t max, int32_t* out) {
731   const Field* f = findField(data, count, key);
732   if (f == NULL || f->type != JT_NUMBER || !f->isInteger || f->integer < min ||
  >   f->integer > max) return false;
733   *out = f->integer;
734   return true;
735 }
```

protocolSensorCalibratable allows only pH, TDS and light. sensorField checks the decoded sensor key through sensorFromKey and rejects its sentinel. intField requires both integer type and the declared range.

The helpers produce useful types only after checking input. For example, an arbitrary rule integer is not yet proof that the active policy has that many rules; Safety will check the actual rule count.

Check calibration coordinates in their wire scales

src/domain/protocol_codec.c / lines 736-756

```
736
737 // Ranges in the scaled units of protocol_codec.h (plausibility only: the fit
738 // itself is checked by Configuration).
739 static ProtocolError validateCalibration(const Field* data, uint8_t count, Command* out)
  >    {
740   if (!sensorField(data, count, out) || !protocolSensorCalibratable(out->sensor)) return
  >    PROTOCOL_ERR_BAD_FIELD;
741   bool ok;
742   switch (out->sensor) {
743     case SENSOR_PH:  // module 0-5 V, pH 0-14, two buffers
744       out->calTwoPoints = true;
745       ok = intField(data, count, "x1", 0, 50000, &out->calX1) && intField(data, count,
  >   "y1", 0, 14000, &out->calY1) &&
746            intField(data, count, "x2", 0, 50000, &out->calX2) && intField(data, count,
  >   "y2", 0, 14000, &out->calY2);
747       break;
748     case SENSOR_TDS:  // 0.1-2000 ppm both ways
749       ok = intField(data, count, "x1", 1, 20000, &out->calX1) && intField(data, count,
  >   "y1", 1, 20000, &out->calY1);
750       break;
751     default:  // light: pin 0.1-3300 mV, 0.1-100000 lux
752       ok = intField(data, count, "x1", 1, 33000, &out->calX1) && intField(data, count,
  >   "y1", 1, 1000000, &out->calY1);
753       break;
754   }
755   return ok ? PROTOCOL_OK : PROTOCOL_ERR_BAD_FIELD;
756 }
```

The pH operation requires two x/y points, while TDS and light require the one-point form. Each sensor uses its documented integer limits and target scale.

This validates coordinates, not the resulting slope. Configuration still rejects coincident points or an implausible fit after converting these integers. Keep input shape and physical fit plausibility as separate checks.

Require the fields for the selected operation

src/domain/protocol_codec.c / lines 757-783

```
757
758 static ProtocolError validateData(const Field* data, uint8_t
  >    count, bool haveData, Command* out) {
759   switch (out->op) {
760     case CMD_OP_GET_SENSORS:
761     case CMD_OP_FORGET_WIFI:
762     case CMD_OP_GET_CALIBRATION:
763       return PROTOCOL_OK;  // data ignored
764     default: break;
765   }
766   if (!haveData) return PROTOCOL_ERR_BAD_FIELD;
767
768   switch (out->op) {
769     case CMD_OP_CAL_CAPTURE:
770     case CMD_OP_RESET_CALIBRATION:
771       return sensorField(data, count, out) &&
  >   protocolSensorCalibratable(out->sensor) ? PROTOCOL_OK
772
  >                            : PROTOCOL_ERR_BAD_FIELD;
773     case CMD_OP_SET_CALIBRATION: return
  >   validateCalibration(data, count, out);
774     case CMD_OP_SET_SENSOR_ENABLED: {
775       const Field* s = findField(data, count, "sensor");
776       const Field* e = findField(data, count, "enabled");
777       if (s == NULL || s->type != JT_STRING ||
  >   s->strTooLong) return PROTOCOL_ERR_BAD_FIELD;
778       if (e == NULL || e->type != JT_BOOL) return
  >   PROTOCOL_ERR_BAD_FIELD;
779       out->sensor = sensorFromKey(s->str, s->strLen);
780       if (out->sensor >= SENSOR_COUNT) return
  >   PROTOCOL_ERR_BAD_FIELD;
781       out->enabled = e->boolean;
782       return PROTOCOL_OK;
783     }
```

The switch chooses each operation's requirements. Set-enabled needs a known sensor and boolean. Reset-latch needs a bounded integer. Set-Wi-Fi checks SSID length, passphrase length and printable bytes before copying terminated arrays.

COMMON PITFALL

Successful field validation does not reserve queue capacity or storage space.

Require the fields for the selected operation / continued

src/domain/protocol_codec.c / lines 784-810

```
    case CMD_OP_RESET_LATCH: {
      const Field* r = findField(data, count, "rule");
      if (r == NULL || r->type != JT_NUMBER || !r->isInteger || r->integer < 0 ||
          r->integer > PROTOCOL_MAX_RULE_ID) {
        return PROTOCOL_ERR_BAD_FIELD;
      }
      out->rule = (uint8_t)r->integer;
      return PROTOCOL_OK;
    }
    case CMD_OP_SET_WIFI: {
      const Field* s = findField(data, count, "ssid");
      const Field* p = findField(data, count, "password");
      if (s == NULL || s->type != JT_STRING || s->strTooLong || s->strLen == 0 ||
          s->strLen > PROTOCOL_SSID_MAX) {
        return PROTOCOL_ERR_BAD_FIELD;
      }
      if (p == NULL || p->type != JT_STRING || p->strTooLong || p->strLen <
  > PROTOCOL_PASSWORD_MIN ||
          p->strLen > PROTOCOL_PASSWORD_MAX || !printableAscii(p->str, p->strLen)) {
        return PROTOCOL_ERR_BAD_FIELD;
      }
      memcpy(out->ssid, s->str, s->strLen + 1);
      memcpy(out->password, p->str, p->strLen + 1);
      return PROTOCOL_OK;
    }
    default: return PROTOCOL_ERR_UNKNOWN_OP;
  }
}
```

Read one operation and predict its accepted Command. A get operation needs no driver call; a set operation carries only data that its owner can safely copy into a request.

Accept exactly one complete versioned request

src/domain/protocol_codec.c / lines 811-838

```
811
812 ProtocolError protocolParseCommand(const uint8_t* payload, size_t len, Command* out) {
813   memset(out, 0, sizeof(*out));
814   out->sensor = SENSOR_COUNT;
815   if (payload == NULL || len == 0) return PROTOCOL_ERR_BAD_REQUEST;
816
817   Parser ps = {payload, payload + len};
818   Field data[DATA_FIELDS_MAX];
819   uint8_t dataCount = 0;
820   bool haveData = false, haveVersion = false, haveId = false, idValid = false;
821   int32_t version = -1;
822   char op[24] = {0};
823   size_t opLen = 0;
824   bool opSeen = false, opTooLong = false;
825
826   if (!expect(&ps, '{')) return PROTOCOL_ERR_BAD_REQUEST;
827   skipWs(&ps);
828   if (ps.p < ps.end && *ps.p == '}') {
829     ++ps.p;
830   } else {
831     for (;;) {
832       char key[FIELD_KEY_MAX + 1];
833       size_t keyLen;
834       bool keyTooLong;
835       skipWs(&ps);
836       if (!parseString(&ps, key, FIELD_KEY_MAX, &keyLen, &keyTooLong)) return
  >   PROTOCOL_ERR_BAD_REQUEST;
837       if (!expect(&ps, ':')) return PROTOCOL_ERR_BAD_REQUEST;
838       skipWs(&ps);
```

src/domain/protocol_codec.c / lines 839-852

```
839
840       if (!keyTooLong && strcmp(key, "data") == 0 && ps.p < ps.end && *ps.p == '{') {
841         if (!parseData(&ps, data, &dataCount)) return PROTOCOL_ERR_BAD_REQUEST;
842         haveData = true;
843       } else if (!keyTooLong && strcmp(key, "op") == 0 && ps.p < ps.end && *ps.p == '"')
  >    {
844         if (!parseString(&ps, op, sizeof(op) - 1, &opLen, &opTooLong)) return
  >   PROTOCOL_ERR_BAD_REQUEST;
845         opSeen = true;
846       } else {
847         Field f;
848         memset(&f, 0, sizeof(f));
849         if (!parseScalarOrSkip(&ps, &f, 1)) return PROTOCOL_ERR_BAD_REQUEST;
850         if (!keyTooLong && strcmp(key, "v") == 0) {
851           haveVersion = f.type == JT_NUMBER && f.isInteger;
852           version = f.integer;
```

Accept exactly one complete versioned request / continued

src/domain/protocol_codec.c / lines 853-865

```
853         } else if (!keyTooLong && strcmp(key, "id") == 0) {
854           haveId = true;
855           idValid = f.type == JT_STRING && !f.strTooLong && f.strLen > 0 &&
856                     f.strLen <= PROTOCOL_ID_MAX && printableAscii(f.str, f.strLen);
857           if (idValid) memcpy(out->id, f.str, f.strLen + 1);
858         }
859       }
860       skipWs(&ps);
861       if (ps.p >= ps.end) return PROTOCOL_ERR_BAD_REQUEST;
862       if (*ps.p == ',') {
863         ++ps.p;
864         continue;
865       }
```

The top-level parser clears the output, rejects trailing non-whitespace bytes, requires version 2 and a bounded printable id, resolves op, then validates data. It copies a valid id early so an error reply can still identify the request.

src/domain/protocol_codec.c / lines 866-887

```
866       if (*ps.p == '}') {
867         ++ps.p;
868         break;
869       }
870       return PROTOCOL_ERR_BAD_REQUEST;
871     }
872   }
873   skipWs(&ps);
874   if (ps.p != ps.end) return PROTOCOL_ERR_BAD_REQUEST;  // trailing bytes
875   if (!haveVersion || version != PROTOCOL_VERSION || !haveId || !idValid) {
876     return PROTOCOL_ERR_BAD_REQUEST;
877   }
878   if (!opSeen || opTooLong) return PROTOCOL_ERR_UNKNOWN_OP;
879   for (size_t i = 0; i < sizeof(kOps) / sizeof(kOps[0]); ++i) {
880     if (strlen(kOps[i].name) == opLen && memcmp(kOps[i].name, op, opLen) == 0) {
881       out->op = kOps[i].op;
882       break;
883     }
884   }
885   if (out->op == CMD_OP_NONE) return PROTOCOL_ERR_UNKNOWN_OP;
886   return validateData(data, dataCount, haveData, out);
887 }
```

Walk the failure returns with {"v":2,"op":"set_sensor_enabled","id":"a 1","data":{"sensor":"light","enabled":true}} in mind. Then change enabled to "true": JSON remains legal, but typed validation rejects it. This is the boundary between untrusted bytes and a typed request.

Translate outcome codes into stable protocol words

src/domain/protocol_codec.c / lines 888-904

```
888
889 const char* protocolErrorName(ProtocolError e) {
890   switch (e) {
891     case PROTOCOL_OK: return "ok";
892     case PROTOCOL_ERR_BAD_REQUEST: return "bad_request";
893     case PROTOCOL_ERR_UNKNOWN_OP: return "unknown_op";
894     case PROTOCOL_ERR_BAD_FIELD: return "bad_field";
895     case PROTOCOL_ERR_RULE_LATCHED: return "rule_latched";
896     case PROTOCOL_ERR_RULE_ACTIVE: return "rule_active";
897     case PROTOCOL_ERR_EXPIRED: return "expired";
898     case PROTOCOL_ERR_TIMEOUT: return "timeout";
899     case PROTOCOL_ERR_STORAGE: return "storage";
900     case PROTOCOL_ERR_STALE: return "stale";
901     case PROTOCOL_ERR_BAD_FIT: return "bad_fit";
902   }
903   return "?";
904 }
```

protocolErrorName maps internal error values to contract strings. The default gives an explicit fallback. The operation names and error names are part of the client contract, so changing them needs coordinated client work.

We have reached the end of the pure codec. Nothing in this file starts a task, acquires a sensor or writes NVS. The next chapter supplies execution context and routes accepted commands to those owners.

Keep transport constants visible at the boundary

This small transport header holds framing and firmware-version constants. The implementation of JSON lives in domain/protocol_codec.c, which you just read.

A short file can define a shared contract without owning runtime work.

src/transports/protocol.h / lines 1-14

```
// Protocol adapters: snapshot -> JSON state/telemetry frames.
// Work package WP8 (Stage 5). Architecture §17.
//
// The JSON itself is built in domain/protocol_codec (host-tested) and the
// frames in transports/frames; never from acquisition or ISRs. Coordinate
// schema changes with app/Protocol.py, app/Catalog.py and app/FirmwareContract.md.
#pragma once
#include <stddef.h>

#define PROTOCOL_FRAME_HEADER_BYTES ((size_t)2)  // little-endian payload length
#define PROTOCOL_MAX_PAYLOAD_BYTES ((size_t)4096)

// Reported as "fw" in every State frame.
#define PROTOCOL_FIRMWARE_VERSION "0.9.0"
```

The two-byte header and maximum payload must match the frame codec. The firmware version labels this source's output. It does not select an SDK or verify that the attached hardware has that firmware loaded.

When checking a received message, distinguish protocol version from firmware version: one describes message shape, the other identifies application software.

Recognise a retained project boundary

This translation unit is intentionally only an include and a comment. It adds no second parser.

The full implementation is already in the pure domain codec.

src/transports/protocol.c / lines 1-4

```c
#include "transports/protocol.h"

// Nothing to implement here: the contract v2 encoders live in
// domain/protocol_codec.c and the shared frame builders in transports/frames.c.
```

An otherwise empty .c file can remain after code was moved to a cleaner layer. It is useful to identify this during a full review so you do not search here for hidden behaviour.

Follow actual definitions rather than assuming every filename owns the similarly named feature. There is no runtime work to trace in this translation unit.

Allocate scratch per calling owner

The encoder needs current service values but must not borrow their private mutable buffers. TransportScratch provides caller-owned copies.

This header bridges service peeks to the pure encoders without adding sensor access.

src/transports/frames.h / lines 1-28

```
// State / Telemetry / event frames for the BLE link (WP8).
// FirmwareContract.md v2. The app keeps one row per (node, boot, metric, seq).
//
// Each calling task passes its own scratch (no shared buffers, no locks);
// every input is a zero-wait peek (snapshot, SafetyStatus, applied config,
// Wi-Fi status). Encoders write the JSON payload only (no length header).
#pragma once
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

#include "domain/config_blob.h"
#include "domain/event_journal.h"
#include "domain/reading.h"
#include "services/configuration.h"

#define TRANSPORT_NODE_NAME "Hydro-ESP1"

typedef struct TransportScratch {
  Snapshot snap;
  ConfigBlob cfg;
  char ssid[PROTOCOL_SSID_MAX + 1];
  char ip[16];
} TransportScratch;

// Random per boot, drawn on first use (after the radio starts: true RNG).
uint32_t transportBootId(void);

```

Snapshot and ConfigBlob are relatively large, so grouping them in a task-owned static scratch object avoids repeated stack pressure. SSID and IP arrays keep copied strings alive throughout the encode call.

Allocate scratch per calling owner / continued

src/transports/frames.h / lines 29-35

```
29 // link: "ble" (the LAN link was removed after Stage 8.4).
30 size_t transportEncodeState(TransportScratch* s, const char* link, char* buf, size_t
 >   cap);
31
32 // 0 if no snapshot has been published yet.
33 size_t transportEncodeTelemetry(TransportScratch* s, char* buf, size_t cap);
34
35 size_t transportEncodeEvent(const JournalEvent* e, bool gap, char* buf, size_t cap);
```

The functions return a payload length, without its length header. A caller must still check zero, add framing and decide what to do when sending fails. Scratch is reusable only after that synchronous encode finishes.

Keep one boot identifier across all message types

Read this as a data-gathering adapter. Each function gathers safe copies, builds a view and invokes a pure encoder.

No function here drives a bus or initiates a new measurement.

src/transports/frames.c / lines 1-23

```
#include "transports/frames.h"

#include <ctype.h>
#include <esp_system.h>
#include <freertos/FreeRTOS.h>
#include <string.h>

#include "domain/protocol_codec.h"
#include "domain/sensor_set.h"
#include "hal/clock.h"
#include "policy_config.h"
#include "services/acquisition.h"
#include "services/power_coordinator.h"
#include "services/rail_state.h"
#include "services/safety.h"
#include "services/snapshot_store.h"
#include "transports/protocol.h"
#include "transports/wifi_mqtt.h"

static portMUX_TYPE s_bootMux = portMUX_INITIALIZER_UNLOCKED;
static uint32_t s_boot;
static bool s_bootSet;

```

A random candidate is drawn outside the short critical section. Only the first caller installs it; later callers return the retained identifier. The boolean distinguishes an unset value from a valid random zero.

Keep one boot identifier across all message types / continued

src/transports/frames.c / lines 24-34

```
24 uint32_t transportBootId(void) {
25   const uint32_t candidate = esp_random();  // drawn outside the critical section
26   portENTER_CRITICAL(&s_bootMux);
27   if (!s_bootSet) {
28     s_boot = candidate;
29     s_bootSet = true;
30   }
31   const uint32_t boot = s_boot;
32   portEXIT_CRITICAL(&s_bootMux);
33   return boot;
34 }
```

The lock protects the check-and-set pair. It is not held while generating randomness or encoding text. A boot identifier is a correlation aid, not proof that no two devices can ever collide.

Gather the controller description through service interfaces

src/transports/frames.c / lines 35-62

```
size_t transportEncodeState(TransportScratch* s, const char* link, char* buf, size_t cap) {
  char mode[24];
  const char* name = powerModeName(powerModeCurrent());
  size_t i = 0;
  for (; name[i] != '\0' && i < sizeof(mode) - 1; ++i) mode[i] = (char)tolower((unsigned char)name[i]);
  mode[i] = '\0';

  SafetyStatus st;
  memset(&st, 0, sizeof(st));
  const bool haveSafety = safetyStatusPeek(&st);
  configurationPeekApplied(&s->cfg);
  WifiStatus wifi;
  wifiMqttGetStatus(&wifi);
  memcpy(s->ssid, wifi.ssid, sizeof(s->ssid));
  memcpy(s->ip, wifi.ip, sizeof(s->ip));
  const bool connected = wifi.state == WIFI_STATE_CONNECTED;
  const StateView v = {
      .node = TRANSPORT_NODE_NAME, .fw = PROTOCOL_FIRMWARE_VERSION, .boot = transportBootId(),
      .uptimeMs = monoNowMs(), .link = link, .mode = mode,
      .powerEpoch = railStateRead().epoch, .cfgRev = s->cfg.revision,  // live 5 V epoch (WP9)
      .wifiState = wifiStateName(wifi.state), .wifiSsid = s->ssid[0] != '\0' ? s->ssid : NULL,
      .wifiIp = connected ? s->ip : NULL, .wifiRssi = wifi.rssi,
      .relayOn = st.relayOn, .lampReason = haveSafety ? lampReasonName(st.lampReason) : "no data",
      .interlock = st.interlock, .latchedMask = st.latchedMask, .activeMask = st.activeMask,
      .staleMask = st.staleMask, .ruleCount = POLICY_RULE_COUNT,
      .sgp30Baseline = acquisitionSgp30BaselineName(), .enabled = s->cfg.sensorEnabled,
  };
```

State gathers mode, Safety status, applied config, Wi-Fi status, rail epoch and gas baseline status. The local lowercase mode buffer survives until the synchronous encoder returns.

src/transports/frames.c / lines 63-64

```
  return protocolEncodeState(&v, buf, cap);
}
```

Each service copy is internally protected, but these separate peeks are not one global atomic transaction. A power transition may occur between them. Interpret State as current reported status, not as a simultaneous oscilloscope capture of every task.

Recheck the rail at the consumer boundary

src/transports/frames.c / lines 65-73

```
65
66 size_t transportEncodeTelemetry(TransportScratch* s, char* buf, size_t cap) {
67   if (!snapshotPeek(&s->snap)) return 0;
68   const RailState rail = railStateRead();
69   snapshotGateRail(&s->snap, rail.valid5v, rail.epoch);  // the rail may have failed
 >   since acquisition
70   configurationPeekApplied(&s->cfg);
71   return protocolEncodeTelemetry(&s->snap, monoNowMs(), transportBootId(),
 >   s->cfg.revision, s->cfg.sensorEnabled,
72                                  buf, cap);
73 }
```

Telemetry first peeks the retained Snapshot, then gates its private copy against the current rail validity and epoch. It also obtains the applied enabled set.

A sample accepted by acquisition can become unusable after a rail failure. This extra check prevents an old 5V observation from being displayed merely because acquisition has not published its next copy yet.

Describe an event without changing the journal

src/transports/frames.c / lines 74-80

```
74
75 size_t transportEncodeEvent(const JournalEvent* e, bool gap, char* buf, size_t cap) {
76   const MonoMs now = monoNowMs();
77   const EventView v = {e->n, e->ruleId, severityName(e->severity),
 >   journalKindName(e->kind),
78                        now > e->atMs ? (uint32_t)(now - e->atMs) : 0u, gap};
79   return protocolEncodeEvent(&v, buf, cap);
80 }
```

Event age is now minus the event timestamp, clamped at zero if now is earlier. Names come from domain mappings. The gap bit is supplied by the reader's cursor history.

Encoding does not advance the journal cursor. The BLE task will commit that advance only after a successful send; keeping that decision outside this helper makes retry possible.

Give each BLE operation a concrete destination

The ESP32 exposes a GATT server: a set of values and permitted operations that the client can discover. GATT means Generic Attribute Profile. This firmware groups its related values into one service. Each named characteristic below is an addressed value with allowed read, write or notification operations.

Characteristic in this controller	What the client does
Command	Writes command bytes to the ESP32.
State	Reads the retained State frame or receives an update notification.
Telemetry	Receives periodic observation notifications.
Result	Receives command outcomes and alarm-event frames.

A UUID is the identifier used to distinguish a service or characteristic. The long constants in ble_gatt.cpp supply those identifiers. They are separate from the request id inside JSON: the characteristic identifies the route, while id identifies one application request sent through it.

A notification sends characteristic bytes to a subscribed client. A successful local notify call means the library accepted that chunk under its API contract. It does not prove that the remote application decoded and accepted the whole message. The application Result message has its own meaning, such as settings stored or a latch reset applied.

The server callbacks execute in NimBLE's host task. The next lesson follows the short copy-and-wake handoff to the application BLE task. Keep those two tasks separate while reading onWrite and onRx.

REMEMBER

GATT chooses where bytes go. The application frame and JSON choose what those bytes mean.

A callback runs in somebody's task

Original firmware - src/transports/ble_transport.c, lines 111-117

```
// NimBLE host task: copy and return (ble_gatt.h threading rule).
static void onRx(const uint8_t* data, size_t len, uint32_t
>   generation, void* ctx) {
  (void)generation;
  (void)ctx;
  if (xStreamBufferSend(s_rx, data, len, 0) != len)
>   STATS_ADD(rxOverflow, 1);
  if (s_task != NULL) xTaskNotifyGive(s_task);
}
```

onRx is called by the BLE host context. It copies received bytes into a stream buffer and wakes the transport task. It does not parse a long command, write NVS, read a sensor or switch the relay there. The callback returns promptly so the host can continue its work.

A callback is an ordinary function invoked by another component at an event. Its execution context matters. A BLE callback usually runs in a host task, not a GPIO ISR. Therefore this code uses task-context stream-buffer and notification APIs, not the FromISR variants.

ctx is a caller-supplied context pointer in many callback designs. It lets a generic callback find its own application state without assuming one global instance. Here the callback does not need it and explicitly ignores it. The function-pointer syntax is the same idea you already met in SensorDriverOps.

A stream buffer carries bytes; it does not attach a message boundary to each write. Its FreeRTOS contract is designed around one writer and one reader unless you add the required serialisation. The BLE receive path's producer and transport task must respect that assumption.

The two .cpp adapters expose C-callable interfaces to C application modules. C++ library objects stay in those adapters. Most policy and protocol code stays C. A comment saying all source is C would therefore be too broad for this snapshot.

REMEMBER

Before calling a wait or lock from a callback, identify whether it runs in a task, an interrupt or a library's constrained context.

Split a frame without changing its meaning

MTU means maximum transmission unit. Here ATT MTU limits the size of one attribute-protocol packet. A notification uses three of those bytes for its operation and attribute handle. With the default MTU of 23 used by this source, bleGattChunkSize therefore returns 20 available bytes for notification content.

Part of an invented 50-byte JSON payload	Bytes sent
Length header plus JSON	2 + 50 = 52 total
First notification	20
Second notification	20
Third notification	12

These chunks belong to one application frame. The receiver keeps the unfinished prefix until all 50 payload bytes have arrived. The next notification is not automatically a new command or a new JSON document. If the negotiated MTU is 247, this source allows 244 content bytes per notification instead.

Now connect this arithmetic to sendFrame. offset begins at zero. After a successful 20-byte chunk it becomes 20, then 40, then 52. If the first chunk cannot be accepted, an unsent telemetry frame can be dropped. If a later chunk repeatedly fails, a partial frame already exists at the client; this source disconnects to force a clean stream on reconnect.

The receive direction also accepts delivery in arbitrary chunks. Its assembler uses length and consumed count rather than assuming one characteristic write contains one complete command. The two paths share a framing contract even though their library calls differ.

COMMON PITFALL

The negotiated chunk limit is not the application's maximum JSON length. A long frame can use several chunks.

Read the C++ adapter with the C you already know

C++ teaching fragment - library declarations supplied by its header

```
NimBLEServer* server = nullptr;
// A pointer to a library object; nullptr means no object.

NimBLEDevice::init(deviceName);
// Call the operation named init in the NimBLEDevice class.
```

The .cpp extension selects a C++ compiler. Most control flow here is familiar: if, return, pointers and ordinary calls. A class is a type that groups state and operations. A method is one of those operations. objectPointer->method(...) selects a method through a pointer; the :: in NimBLEDevice::init names a class-scoped operation instead.

class ServerCallbacks : public NimBLEServerCallbacks declares a new callback type based on the library's callback interface. A body marked override supplies the operation that NimBLE calls at that event. The compiler checks that its signature matches. Defining the class does not start the callback or create a task.

C++ callback fragment from the adapter pattern

```
stats([](BleGattStats& s) { ++s.connects; });
```

The expression beginning [] is a lambda: an unnamed function object passed as an argument. Its parameter s is a C++ reference, an alias for the supplied BleGattStats object. That is why s.connects uses a dot while still changing the original stats. Here [] captures nothing from the surrounding local scope. A capture such as [reason] stores a copy of that value for the lambda body.

The stats template accepts different lambda types through its type parameter F. At runtime it enters the short critical section, calls the supplied update and exits. std::atomic<bool> similarly names a template with bool as its stored type; its load and store operations protect that individual field. Neither feature turns several fields into one atomic record or changes which task executes the call.

REMEMBER

In the headers, extern "C" exposes a C-callable linkage boundary. The C++ objects and methods remain inside these adapters.

Separate pairing, a secure session and request identity

A radio connection is only the beginning of the application link. Encryption protects traffic with session keys. Authentication adds evidence about the peer; this source requests passkey-based protection against an unauthenticated intermediary. Bonding retains keys so an accepted peer can establish later secure connections.

The source's linkIsSecure checks encryption, authentication, a bond and key size. Its single-owner policy also refuses a second bonded owner. Read each check as one condition for accepting commands. The six-digit passkey is displayed on serial; it is separate from a Wi-Fi password and separate from the request id in JSON.

A session generation solves another problem. Suppose a request was queued on generation 10, the link disconnects, and the client reconnects on a later generation. A result from that old request must not appear as a new session's reply. Configuration and Safety also check session authority before executing queued work. Bond identity, current secure-link state and request generation therefore preserve different facts.

Hold those meanings beside the callback code. onConnect establishes a new session and requests security. onAuthenticationComplete decides whether the connection qualifies. onWrite accepts command bytes only through its security checks. The C transport later applies the independent operation, expiry and queue-capacity checks.

COMMON PITFALL

A secure connection does not prove that a queued command is still fresh or that its requested operation has completed.

Describe callback bytes and channel identity

The application is written mostly in C, while NimBLE-Arduino exposes C++ classes. This header is the narrow C-callable boundary between them.

Its threading rule matters as much as its function signatures: receive callbacks execute in the NimBLE host task.

src/transports/ble_gatt.h / lines 1-28

```
1  // GATT server shim over NimBLE-Arduino 2.5.1 (C++), callable from C.
2  // Work package WP8 (Stage 5). Architecture §17; FirmwareContract.md v2.
3  //
4  // The shim is the only code that touches NimBLE. It owns the service map,
5  // LE Secure Connections pairing with a displayed passkey (MITM), the
6  // single-owner rule and the link "session generation". It does no protocol
7  // work: received Command bytes go to one callback, and the caller notifies
8  // already-framed bytes one chunk at a time.
9  //
10 // Threading: the RX callback runs in the NimBLE host task (core 0). It must
11 // only copy the bytes and return (e.g. xStreamBufferSend with zero wait).
12 // All other functions may be called from one application task
13 // (BleTransport). Notifications are sent only on a secure link.
14 #pragma once
15 #include <stdbool.h>
16 #include <stddef.h>
17 #include <stdint.h>
18
19 #ifdef __cplusplus
20 extern "C" {
21 #endif
22
23 typedef uint8_t BleChannel;
24 enum { BLE_CHANNEL_STATE = 0, BLE_CHANNEL_TELEMETRY, BLE_CHANNEL_RESULT,
 >   BLE_CHANNEL_COUNT };
25
26 // Command characteristic write, raw bytes as received (any chunking).
27 typedef void (*BleRxFn)(const uint8_t* data, size_t len, uint32_t generation, void*
 >   ctx);
28
```

BleRxFn receives an address, length, generation and context pointer. The address is borrowed callback data: the receiver must copy what it needs before returning. The stats structure records link activity and rejected work.

Describe callback bytes and channel identity / continued

src/transports/ble_gatt.h / lines 29-39

```
typedef struct BleGattStats {
  uint32_t connects;
  uint32_t disconnects;
  uint32_t pairingsAccepted;
  uint32_t pairingsRejected;   // not authenticated/bonded, or a second owner
  uint32_t notifyFailures;     // chunks NimBLE could not queue
  uint32_t rxDropped;          // writes received while the link was not secure
  int lastDisconnectReason;
} BleGattStats;

// Starts NimBLE, the GATT service and advertising. The six-digit passkey is
```

extern "C" changes linkage when a C++ compiler reads this header. It makes the exported names callable from C; it does not change where a function executes or make C++ classes available inside a C file.

Separate connection security from frame delivery

src/transports/ble_gatt.h / lines 40-65

```
40 // drawn after NimBLE has started the radio (esp_random() is then a true RNG)
41 // and before advertising; the caller prints bleGattPasskey() on serial.
42 // Returns false if NimBLE failed to start.
43 bool bleGattStart(const char* deviceName, BleRxFn rx, void* rxCtx);
44
45 uint32_t bleGattPasskey(void);
46
47 // True while one client is connected, encrypted, authenticated (MITM) and
48 // bonded, and it is the bonded owner.
49 bool bleGattSecure(void);
50
51 // Incremented on every connect and disconnect: requests from an earlier
52 // session must not be answered on a newer one.
53 uint32_t bleGattGeneration(void);
54
55 // Largest notification payload on the current link (ATT MTU - 3), >= 20.
56 size_t bleGattChunkSize(void);
57
58 // Sends one notification of at most bleGattChunkSize() bytes. false if the
59 // link is not secure, the chunk is too big, or NimBLE is out of buffers
60 // (the caller may retry shortly or drop the frame).
61 bool bleGattNotify(BleChannel channel, const uint8_t* data, size_t len);
62
63 // Value returned by a read of the State characteristic (one whole frame,
64 // header included; long reads are supported up to BLE_GATT_STATE_MAX).
65 #define BLE_GATT_STATE_MAX 1024
```

A connected link is not automatically a secure owner link. The shim exposes a checked secure state and generation. Chunk size reflects the negotiated ATT MTU minus protocol overhead.

A frame can be much larger than one notification. The shim sends one chunk; the application task owns the complete-frame loop. This keeps byte transport separate from JSON and command dispatch.

Provide explicit recovery and inspection operations

src/transports/ble_gatt.h / lines 66-81

```
66 bool bleGattSetStateValue(const uint8_t* frame, size_t len);
67
68 // Number of stored bonds, and the recovery path (BOOT held 10 s at runtime):
69 // removes every bond and drops the current client.
70 int bleGattBondCount(void);
71 void bleGattClearBonds(void);
72
73 // Drops the current client (bonds are kept). Used when a notification frame
74 // was cut after its first chunk: the client's stream is out of step.
75 void bleGattDisconnect(void);
76
77 void bleGattGetStats(BleGattStats* out);
78
79 #ifdef __cplusplus
80 }
81 #endif
```

The readable State value has its own bounded capacity. Bond clearing removes ownership credentials; disconnect drops only the current connection. The two recovery actions have different consequences.

The application does not reach into NimBLE globals. It asks this shim for stats or a recovery action, preserving a single integration point for the C++ library.

Keep library objects private to this translation unit

You need only a small amount of C++ to read this adapter: namespace-local objects, class inheritance, override, atomics and lambdas. The application state machines still remain in C.

The original opening comment says this is the only C++ file; wifi_lan.cpp is also present in this snapshot. Follow the actual file set rather than that stale comment.

REMEMBER

Review the characteristic map (p. 628) and notification chunk arithmetic (p. 630).

src/transports/ble_gatt.cpp / lines 1-28

```
1  // NimBLE-Arduino implementation of ble_gatt.h. The only C++
   >    file in the
2  // project: NimBLE-Arduino is a C++ library, and ESP-Demo
   >   already proved this
3  // version on espressif32@6.12.0 / ESP32-S3. Everything it
   >   exports is C.
4  //
5  // Compiled only with -D FW_ENABLE_BLE (the production
   >   envs), so the bring-up
6  // images do not link NimBLE.
7  #ifdef FW_ENABLE_BLE
8  #include "transports/ble_gatt.h"
9
10 #include <NimBLEDevice.h>
11
12 #include <esp_system.h>
13
14 #include <atomic>
15 #include <string.h>
16
17 namespace {
18
19 constexpr const char* kServiceUuid =
   >   "98a90f00-346e-4e65-9cbd-7687685bdc01";
20 constexpr const char* kStateUuid =
   >   "98a90f01-346e-4e65-9cbd-7687685bdc01";
21 constexpr const char* kTelemetryUuid =
   >   "98a90f02-346e-4e65-9cbd-7687685bdc01";
22 constexpr const char* kCommandUuid =
   >   "98a90f03-346e-4e65-9cbd-7687685bdc01";
23 constexpr const char* kResultUuid =
   >   "98a90f04-346e-4e65-9cbd-7687685bdc01";
24 constexpr const char* kAuthUuid =
   >   "98a90f05-346e-4e65-9cbd-7687685bdc01";
25 constexpr const char* kAuthValue = "HYDRO_SECURE_V1";
26 constexpr uint16_t kPreferredMtu = 247;
27 constexpr size_t kMinChunk = 20;
28
```

Keep library objects private to this translation unit / continued

The unnamed namespace hides UUIDs, server pointers, callback context and connection state from other source files. constexpr supplies fixed values. nullptr is the C++ null-pointer value.

src/transports/ble_gatt.cpp / lines 29-43

```
29 NimBLEServer* s_server = nullptr;
30 NimBLECharacteristic* s_channel[BLE_CHANNEL_COUNT] = {};
31 NimBLECharacteristic* s_command = nullptr;
32 BleRxFn s_rx = nullptr;
33 void* s_rxCtx = nullptr;
34 uint32_t s_passkey = 0;
35
36 std::atomic<bool> s_secure{false};
37 std::atomic<uint32_t> s_generation{0};
38 std::atomic<uint16_t> s_conn{BLE_HS_CONN_HANDLE_NONE};
39 std::atomic<uint16_t> s_mtu{23};
40
41 portMUX_TYPE s_statsMux = portMUX_INITIALIZER_UNLOCKED;
42 BleGattStats s_stats = {};
43
```

std::atomic protects individual secure/generation/connection/MTU accesses across tasks. It does not make a group of separate loads one combined snapshot. The stats structure uses a different mechanism: a short critical section.

Use a small callback to update guarded statistics

src/transports/ble_gatt.cpp / lines 44-53

```
44 template <typename F>
45 void stats(F update) {
46   portENTER_CRITICAL(&s_statsMux);
47   update(s_stats);
48   portEXIT_CRITICAL(&s_statsMux);
49 }
50
51 bool linkIsSecure(const NimBLEConnInfo& info) {
52   return info.isEncrypted() && info.isAuthenticated() && info.isBonded() &&
 >   info.getSecKeySize() >= 16;
53 }
```

stats is a function template taking an update operation. The lambda supplied by a caller edits the guarded structure between critical-section entry and exit. No network call belongs inside that update.

linkIsSecure checks encryption, authentication, bonding and key size. These are explicit conditions, not an assumption derived from seeing a connected client. Their result gates later command access.

Treat each connection as a new session

src/transports/ble_gatt.cpp / lines 54-81

```
54
55 class ServerCallbacks final : public NimBLEServerCallbacks {
56   void onConnect(NimBLEServer* server, NimBLEConnInfo& info) override {
57     if (server->getConnectedCount() > 1) {  // one client at a time
58       server->disconnect(info.getConnHandle());
59       return;
60     }
61     s_secure.store(false);
62     s_conn.store(info.getConnHandle());
63     s_mtu.store(23);
64     s_generation.fetch_add(1);
65     stats([](BleGattStats& s) { ++s.connects; });
66     // Ask for security at once: Windows and BlueZ then pair before discovery.
67     NimBLEDevice::startSecurity(info.getConnHandle());
68   }
69
70   void onDisconnect(NimBLEServer*, NimBLEConnInfo& info, int reason) override {
71     if (info.getConnHandle() == s_conn.load()) {
72       s_secure.store(false);
73       s_conn.store(BLE_HS_CONN_HANDLE_NONE);
74       s_generation.fetch_add(1);
75     }
76     stats([reason](BleGattStats& s) {
77       ++s.disconnects;
78       s.lastDisconnectReason = reason;
79     });
80     NimBLEDevice::startAdvertising();
81   }
```

The ServerCallbacks class derives from the NimBLE callback interface. override asks the compiler to verify the callback signature. onConnect rejects a second simultaneous client, resets link state, increments generation and requests security.

src/transports/ble_gatt.cpp / lines 82-83

```
82
83   void onMTUChange(uint16_t mtu, NimBLEConnInfo&) override { s_mtu.store(mtu); }
```

onDisconnect invalidates the current connection and increments generation again, then resumes advertising. An old command can carry the previous generation even when a new connection uses the same physical phone.

Accept only the authenticated bonded owner

src/transports/ble_gatt.cpp / lines 84-104

```
84
85    uint32_t onPassKeyDisplay() override { return s_passkey; }
86
87    void onAuthenticationComplete(NimBLEConnInfo& info)
 >    override {
88      bool secure = linkIsSecure(info);
89      // Single owner: the first bond is the owner. A second
 >    bond means another
90      // computer just paired: remove it and drop that client.
91      if (secure && NimBLEDevice::getNumBonds() > 1) {
92        NimBLEDevice::deleteBond(info.getIdAddress());
93        secure = false;
94      }
95      s_secure.store(secure);
96      if (secure) {
97        s_mtu.store(info.getMTU());
98        stats([](BleGattStats& s) { ++s.pairingsAccepted; });
99      } else {
100       stats([](BleGattStats& s) { ++s.pairingsRejected; });
101       s_server->disconnect(info.getConnHandle());
102     }
103   }
104 };
```

Authentication completion evaluates the full security predicate. If another owner bond would be created, this policy deletes that new bond and rejects the connection. Successful pairing records the negotiated MTU.

A six-digit passkey display callback returns the configured value; it does not parse commands. This class handles link policy, while the C task will handle application requests.

COMMON PITFALL

A callback method runs in the library's calling context. Inheritance does not create a private task for each object.

Copy command bytes out of the library callback

src/transports/ble_gatt.cpp / lines 105-121

```
105
106 class CommandCallbacks final : public NimBLECharacteristicCallbacks {
107   void onWrite(NimBLECharacteristic* characteristic, NimBLEConnInfo& info) override {
108     if (!s_secure.load() || !linkIsSecure(info)) {
109       stats([](BleGattStats& s) { ++s.rxDropped; });
110       s_server->disconnect(info.getConnHandle());
111       return;
112     }
113     const NimBLEAttValue value = characteristic->getValue();
114     if (s_rx != nullptr && value.length() > 0) s_rx(value.data(), value.length(),
  >   s_generation.load(), s_rxCtx);
115   }
116 };
117
118 ServerCallbacks s_serverCallbacks;
119 CommandCallbacks s_commandCallbacks;
120
121 }  // namespace
```

onWrite verifies the secure state again, obtains the characteristic value and forwards its bytes synchronously to s_rx. The callback objects themselves have static lifetime.

The next file's onRx uses a stream-buffer copy and a notification. It must not wait for flash, a calibration fit or a sensor conversion here, because doing so would occupy the NimBLE host task.

Start the library before advertising the application

src/transports/ble_gatt.cpp / lines 122-143

```
122
123 extern "C" bool bleGattStart(const char* deviceName, BleRxFn rx, void* rxCtx) {
124   if (s_server != nullptr) return true;
125   s_rx = rx;
126   s_rxCtx = rxCtx;
127   if (!NimBLEDevice::init(deviceName)) return false;
128   // The controller is running now, so the hardware RNG is seeded from RF noise.
129   s_passkey = 100000u + esp_random() % 900000u;
130   NimBLEDevice::setSecurityAuth(true, true, true);  // bonding, MITM, LE Secure
  >   Connections
131   NimBLEDevice::setSecurityIOCap(BLE_HS_IO_DISPLAY_ONLY);
132   NimBLEDevice::setSecurityPasskey(s_passkey);
133   NimBLEDevice::setMTU(kPreferredMtu);
134
135   s_server = NimBLEDevice::createServer();
136   s_server->setCallbacks(&s_serverCallbacks);
137   NimBLEService* service = s_server->createService(kServiceUuid);
138   // *_AUTHEN: the stack itself refuses access without an authenticated (MITM) link.
139   s_channel[BLE_CHANNEL_STATE] = service->createCharacteristic(
140       kStateUuid, NIMBLE_PROPERTY::READ | NIMBLE_PROPERTY::READ_AUTHEN |
  >   NIMBLE_PROPERTY::NOTIFY,
141       BLE_GATT_STATE_MAX);
142   s_channel[BLE_CHANNEL_TELEMETRY] = service->createCharacteristic(
143       kTelemetryUuid, NIMBLE_PROPERTY::NOTIFY | NIMBLE_PROPERTY::READ |
  >   NIMBLE_PROPERTY::READ_AUTHEN, 20);
```

bleGattStart records the callback and initialises NimBLE, then chooses pairing settings and creates the server and service. READ_AUTHEN and WRITE_AUTHEN request authenticated access at the stack boundary.

A characteristic has a UUID and properties; it is not a FreeRTOS queue. The application still needs explicit buffering when callback bytes arrive in pieces. Failure to initialise or start returns false to startup.

Build the characteristic map and advertise it

src/transports/ble_gatt.cpp / lines 144-162

```
144   s_command = service->createCharacteristic(kCommandUuid,
145                                              NIMBLE_PROPERTY::WRITE |
  >   NIMBLE_PROPERTY::WRITE_AUTHEN, 512);
146   s_command->setCallbacks(&s_commandCallbacks);
147   s_channel[BLE_CHANNEL_RESULT] = service->createCharacteristic(
148       kResultUuid, NIMBLE_PROPERTY::NOTIFY | NIMBLE_PROPERTY::READ |
  >   NIMBLE_PROPERTY::READ_AUTHEN, 20);
149   NimBLECharacteristic* auth =
150       service->createCharacteristic(kAuthUuid, NIMBLE_PROPERTY::READ |
  >   NIMBLE_PROPERTY::READ_AUTHEN, 32);
151   auth->setValue(kAuthValue);
152   if (!s_server->start()) return false;
153
154   // 31-byte advertising packet: flags + 128-bit service UUID (21 B) leave no room
155   // for the name, so the name goes in the scan response. Order matters:
156   // setName() only targets the scan response once it is enabled.
157   NimBLEAdvertising* advertising = NimBLEDevice::getAdvertising();
158   advertising->enableScanResponse(true);
159   advertising->setName(deviceName);
160   advertising->addServiceUUID(kServiceUuid);
161   return advertising->start();
162 }
```

The remaining characteristics separate commands, results and authentication text. Advertising has a limited packet budget, so this source enables a scan response before assigning the device name.

State supports a larger readable value than ordinary notification chunks. Clients can read it as a characteristic value, while periodic notifications still travel through the chunking path. These are two delivery mechanisms for related data.

Read atomic link fields and derive chunk capacity

src/transports/ble_gatt.cpp / lines 163-173

```
163
164 extern "C" uint32_t bleGattPasskey(void) { return s_passkey; }
165
166 extern "C" bool bleGattSecure(void) { return s_secure.load(); }
167
168 extern "C" uint32_t bleGattGeneration(void) { return s_generation.load(); }
169
170 extern "C" size_t bleGattChunkSize(void) {
171   const size_t mtu = s_mtu.load();
172   return mtu > kMinChunk + 3 ? mtu - 3 : kMinChunk;
173 }
```

The short exported accessors load retained link values. bleGattChunkSize returns MTU minus three bytes, with the minimum notification payload used here set to 20. For MTU 247, the application may offer 244 payload bytes per chunk.

The preferred MTU is a request, not a guarantee that every client negotiates it. Calculate chunk size at sending time rather than assuming the preferred value.

Reject unsuitable chunks before calling NimBLE

src/transports/ble_gatt.cpp / lines 174-186

```
174
175 extern "C" bool bleGattNotify(BleChannel channel, const uint8_t* data, size_t len) {
176   const uint16_t conn = s_conn.load();
177   if (channel >= BLE_CHANNEL_COUNT || !s_secure.load() || conn ==
  >   BLE_HS_CONN_HANDLE_NONE ||
178       len == 0 || len > bleGattChunkSize()) {
179     return false;
180   }
181   if (!s_channel[channel]->notify(data, len, conn)) {
182     stats([](BleGattStats& s) { ++s.notifyFailures; });
183     return false;
184   }
185   return true;
186 }
```

Notify checks channel, security, connection handle and chunk length. A false library result increments failure statistics and returns false so the task can choose retry or drop.

Success means the chunk was accepted by this notification API. It is not an application-level acknowledgement that the remote program parsed and stored the full JSON message.

Keep State reads and owner recovery explicit

src/transports/ble_gatt.cpp / lines 187-201

```
187
188 extern "C" bool bleGattSetStateValue(const uint8_t* frame, size_t len) {
189   if (s_channel[BLE_CHANNEL_STATE] == nullptr || len > BLE_GATT_STATE_MAX) return false;
190   s_channel[BLE_CHANNEL_STATE]->setValue(frame, len);
191   return true;
192 }
193
194 extern "C" int bleGattBondCount(void) { return s_server != nullptr ?
  >   NimBLEDevice::getNumBonds() : 0; }
195
196 extern "C" void bleGattClearBonds(void) {
197   if (s_server == nullptr) return;
198   const uint16_t conn = s_conn.load();
199   if (conn != BLE_HS_CONN_HANDLE_NONE) s_server->disconnect(conn);
200   NimBLEDevice::deleteAllBonds();
201 }
```

SetStateValue enforces its capacity before assigning the characteristic value. BondCount allows diagnostics. ClearBonds disconnects the current client and deletes all stored bonds.

The runtime BOOT-button policy lives in the C transport task, not in this adapter. Keeping the trigger outside makes it possible to reason separately about a long button hold and the resulting library action.

Disconnect a torn stream and copy stats safely

src/transports/ble_gatt.cpp / lines 202-215

```
202
203 extern "C" void bleGattDisconnect(void) {
204   if (s_server == nullptr) return;
205   const uint16_t conn = s_conn.load();
206   if (conn != BLE_HS_CONN_HANDLE_NONE) s_server->disconnect(conn);
207 }
208
209 extern "C" void bleGattGetStats(BleGattStats* out) {
210   portENTER_CRITICAL(&s_statsMux);
211   *out = s_stats;
212   portEXIT_CRITICAL(&s_statsMux);
213 }
214
215 #endif  // FW_ENABLE_BLE
```

bleGattDisconnect preserves bonds but drops the active link. The sender uses it when a frame has started and cannot be finished, so a future reconnect can begin with an empty decoder.

GetStats copies under the stats critical section. The caller owns its returned copy and can format it after leaving the lock, avoiding a slow log operation inside the guarded region.

Expose lifecycle and diagnostics without exporting buffers

This is the public surface of the application BLE owner. It owns framing, command dispatch and outgoing ordering; it never acquires a sensor directly.

The large implementation follows the same pattern you used in the small notification lab: store data, wake the owner, let the owner process it.

src/transports/ble_transport.h / lines 1-28

```
// BleTransport: the one application GATT sender (Architecture §17).
// Work package WP8 (Stage 8.3). FirmwareContract.md v2.
//
// Task "BleTransport" (cfgTaskSpec: core 0, priority 2, 6 KB). It owns the
// GATT shim (ble_gatt.h), reassembles command frames, routes them (get_sensors
// here, and get_calibration / cal_capture from zero-wait peeks;
// set_sensor_enabled / set_wifi / forget_wifi / set_calibration /
// reset_calibration to Configuration; reset_latch to Safety) and sends:
//   results and alarm events first, then State (every 2 s and on link-up),
//   then Telemetry (every 3 s). A telemetry frame that cannot be queued is
//   dropped and superseded by the next one; results are retried.
// It peeks the snapshot, SafetyStatus and the applied config with zero wait
// and never calls a driver. Requests carry the link's session generation and
// an expiry: a reply is never sent on a newer link, a command never applied
// late.
//
// Pairing: LE Secure Connections + bonding + MITM, passkey drawn after the
// radio starts and printed once on serial; single bonded owner. Holding BOOT
// (GPIO0) for 10 s while running clears all bonds.
//
// Built only with -D FW_ENABLE_BLE (production envs). Without it
// bleTransportInit() returns false and nothing links NimBLE.
#pragma once
#include <stdbool.h>
#include <stdint.h>

bool bleTransportInit(void);

```

Initialisation starts the service. Statistics report frames, drops, command failures and maximum send time. TaskHandle lets Supervisor inspect the owner without borrowing its private frame buffer.

src/transports/ble_transport.h / lines 29-34

```
typedef struct BleTransportStats {
  uint32_t framesSent;
  uint32_t telemetryDropped;   // superseded: link congested
  uint32_t resultsDropped;     // link closed or buffers stayed full
  uint32_t commands;
  uint32_t commandErrors;      // malformed frames / fields
```

Expose lifecycle and diagnostics without exporting buffers / continued

src/transports/ble_transport.h / lines 35-43

```
35   uint32_t busy;               // replies "busy" (queues full)
36   uint32_t rxOverflow;         // command bytes lost before parsing
37   uint32_t maxSendMs;          // longest frame send (all chunks)
38 } BleTransportStats;
39
40 void bleTransportGetStats(BleTransportStats* out);
41
42 // For Supervisor. NULL before init or without FW_ENABLE_BLE.
43 void* bleTransportTaskHandle(void);
```

The header comments describe command routes. Keep the distinction between immediate read-only replies and queued state-changing requests: only the latter need another owner's completion callback.

Select a build and state the resource budgets

This file joins several concepts you already learned: stream buffers, queues, notifications, deadlines, generations and fixed-memory ownership. Read the actual flow before the task loop.

The callbacks near the top only transfer work; bleTask near the bottom owns the protocol decisions.

src/transports/ble_transport.c / lines 1-28

```
#include "transports/ble_transport.h"

#ifndef FW_ENABLE_BLE

bool bleTransportInit(void) { return false; }
void bleTransportGetStats(BleTransportStats* out) { *out = (BleTransportStats){0}; }
void* bleTransportTaskHandle(void) { return 0; }

#else

#include <Arduino.h>
#include <ctype.h>
#include <esp_system.h>
#include <esp_task_wdt.h>
#include <freertos/FreeRTOS.h>
#include <freertos/queue.h>
#include <freertos/stream_buffer.h>
#include <freertos/task.h>
#include <stdio.h>
#include <string.h>

#include "app_config.h"
#include "board_profile.h"
#include "diagnostics/logger.h"
#include "diagnostics/progress.h"
#include "domain/cal_capture.h"
#include "domain/frame_codec.h"
#include "domain/protocol_codec.h"
```

Without FW_ENABLE_BLE the file exports inert functions and creates no transport task. The enabled branch defines wake period, RX capacity, retry count, command lifetime and pending-slot count.

src/transports/ble_transport.c / lines 29-34

```
#include "domain/sensor_set.h"
#include "hal/clock.h"
#include "policy_config.h"
#include "services/acquisition.h"
#include "services/configuration.h"
#include "services/rail_state.h"
```

Select a build and state the resource budgets / continued

src/transports/ble_transport.c / lines 35-51

```
#include "services/safety.h"
#include "services/snapshot_store.h"
#include "transports/ble_gatt.h"
#include "transports/frames.h"
#include "transports/protocol.h"

#define BLE_NODE_NAME TRANSPORT_NODE_NAME
#define BLE_BOOT_PIN 0
#define BLE_BOND_CLEAR_MS 10000u
#define BLE_WAKE_MS 50u
#define BLE_RX_BYTES 2048
#define BLE_NOTIFY_RETRIES 40     // x 5 ms: NimBLE buffers free up as packets leave
#define BLE_COMMAND_TTL_MS 5000u  // not handled within this: refused, never applied
  late
#define BLE_PENDING 8             // = CFG_COMMAND_QUEUE_LEN: one slot per queued
  command
#define BLE_PENDING_LIMIT_MS (BLE_COMMAND_TTL_MS + 5000u)  // a reply that never came
  frees its slot
#define BLE_STATS_LOG_MS 60000u

```

Forty 5ms retry waits are a per-chunk retry allowance in this loop, not a hard 200ms bound on a complete multi-chunk frame. Separate constants from the timing scope of the code that uses them.

Give the owner durable buffers

src/transports/ble_transport.c / lines 52-78

```
52 // ---- RX: filled by the NimBLE host callback, drained by this task ----------------
53 static StaticStreamBuffer_t s_rxStruct;
54 static uint8_t s_rxStorage[BLE_RX_BYTES + 1];
55 static StreamBufferHandle_t s_rx;
56
57 // ---- Task and buffers (static: never on the 6 KB stack) --------------------------
58 static StaticTask_t s_tcb;
59 static StackType_t s_stack[6 * 1024];  // cfgTaskSpec(TASK_ID_BLE_TRANSPORT)->stackBytes
60 static TaskHandle_t s_task = NULL;
61
62 static FrameAssembler s_assembler;
63 static uint8_t s_command[FRAME_MAX_PAYLOAD];
64 static uint8_t s_frame[FRAME_HEADER_BYTES + FRAME_MAX_PAYLOAD];
65 static char s_data[1536];
66 static TransportScratch s_scratch;
67 static ConfigBlob s_cfg;
68 static ConfigRequest s_req;  // static: a Wi-Fi password never sits on the stack
69
70 static portMUX_TYPE s_statsMux = portMUX_INITIALIZER_UNLOCKED;
71 static BleTransportStats s_stats;
72 #define STATS_ADD(field, n)          \
73   do {                               \
74     portENTER_CRITICAL(&s_statsMux); \
75     s_stats.field += (n);            \
76     portEXIT_CRITICAL(&s_statsMux);  \
77   } while (0)
78
```

Static stream storage, task stack, assembler, command copy, frame and scratch storage survive every iteration. The task has one reusable outgoing frame, so it cannot send from it while simultaneously overwriting it for another reply.

src/transports/ble_transport.c / lines 79-80

```
79 // ---- Commands in flight ----------------------------------------------------------
80 // Configuration and Safety answer from their own tasks; their callbacks only
```

The stack-size comment describes the intended byte budget. The creation call later compares sizeof(s_stack) with the task table, exposing a disagreement rather than silently assuming they match.

Tag pending work with both slot and history

src/transports/ble_transport.c / lines 81-108

```
81 // copy the outcome into s_results. The tag is slot | sequence << 8, so a late
82 // reply can never be matched to a slot that was freed and reused.
83 typedef struct Pending {
84   bool used;
85   CommandOp op;
86   uint32_t generation;
87   uint32_t seq;
88   MonoMs sinceMs;
89   char id[PROTOCOL_ID_MAX + 1];
90 } Pending;
91
92 typedef struct BleResult {
93   uint32_t tag;
94   ProtocolError error;
95   uint16_t cfgRev;
96 } BleResult;
97
98 static Pending s_pending[BLE_PENDING];
99 static uint32_t s_seq;
100 static StaticQueue_t s_resultControl;
101 static uint8_t s_resultStorage[BLE_PENDING * sizeof(BleResult)];
102 static QueueHandle_t s_results;
103 static JournalCursor s_journalCursor;
104 static bool s_eventPending;    // an event failed to send and waits at the cursor
105 static bool s_eventLost;       // ... and its link closed: the next event reports
  >   gap:true
106 static uint32_t s_tornGeneration = UINT32_MAX;  // link whose stream was cut mid-frame
107 #ifdef FW_INJECT_BLE_TEAR
108 static uint32_t s_tearCount;  // env:ble_tear: telemetry frames sent
```

Pending holds the request id, operation, session generation, sequence and start time. BleResult carries a compact completion tag and outcome. The bounded result queue transfers outcomes back to the BLE owner.

src/transports/ble_transport.c / lines 109-110

```
109 #endif
110
```

Slot alone is insufficient. Slot 2 may be freed and reused before an old callback returns. Packing a sequence beside the slot allows drainResults to reject that stale completion. The sequence is finite, so its protection assumes it does not wrap through the same value while old work remains outstanding.

Keep foreign-task callbacks short

src/transports/ble_transport.c / lines 111-133

```
// NimBLE host task: copy and return (ble_gatt.h threading rule).
static void onRx(const uint8_t* data, size_t len, uint32_t generation, void* ctx) {
  (void)generation;
  (void)ctx;
  if (xStreamBufferSend(s_rx, data, len, 0) != len) STATS_ADD(rxOverflow, 1);
  if (s_task != NULL) xTaskNotifyGive(s_task);
}

static void postResult(const BleResult* r) {
  xQueueSend(s_results, r, 0);  // never full: one entry per pending slot
  if (s_task != NULL) xTaskNotifyGive(s_task);
}

static void onConfigResult(const ConfigResult* r, void* ctx) {
  (void)ctx;
  BleResult out;
  memset(&out, 0, sizeof(out));
  out.tag = r->tag;
  out.error = r->error;
  out.cfgRev = r->cfgRev;
  postResult(&out);
}

```

onRx copies into the stream buffer with zero wait, then wakes the BLE task. Configuration and Safety callbacks similarly copy compact results into s_results. They do not send a notification frame directly.

COMMON PITFALL

Counting overflow does not repair the lost command stream. Review recovery behaviour when testing overload.

Keep foreign-task callbacks short / continued

src/transports/ble_transport.c / lines 134-141

```
134 static void onSafetyResult(uint32_t tag, SafetyCommandOutcome outcome, void* ctx) {
135   (void)ctx;
136   BleResult out;
137   memset(&out, 0, sizeof(out));
138   out.tag = tag;
139   out.error = outcome == SAFETY_CMD_APPLIED         ? PROTOCOL_OK
140               : outcome == SAFETY_CMD_EXPIRED       ? PROTOCOL_ERR_EXPIRED
141               : outcome == SAFETY_CMD_STILL_ACTIVE  ? PROTOCOL_ERR_RULE_ACTIVE
```

src/transports/ble_transport.c / lines 142-144

```
142                                                   : PROTOCOL_ERR_BAD_FIELD;
143   postResult(&out);
144 }
```

The queue is the data channel; xTaskNotifyGive is the wake-up hint. The receiving task drains available data regardless of how several wakes were combined. Notice that RX overflow is counted rather than prevented: a bounded buffer can lose bytes under sustained overload.

Start a frame only on the intended secure generation

src/transports/ble_transport.c / lines 145-172

```
145
146 static uint32_t sessionNow(void) { return bleGattGeneration(); }
147
148 // ---- Sending
  >   ------------------------------------------------------------------------
149 // Header + payload already in s_frame. Sent in MTU-sized chunks. A frame not
150 // started yet may be dropped (telemetry does not wait: the next frame
151 // replaces it). Once its first chunk is out, the frame must finish: the
152 // client reads each characteristic as one length-prefixed stream, so a cut
153 // frame would turn the next one into its continuation. If NimBLE buffers stay
154 // full mid-frame, the link is dropped instead (the app clears its decoders on
155 // reconnect) and nothing more is sent on it.
156 static bool sendFrame(BleChannel channel, size_t payloadLen, uint32_t generation) {
157   if (payloadLen == 0 || !frameWriteHeader(s_frame, payloadLen)) return false;
158   if (generation == s_tornGeneration) return false;  // being dropped
159   const MonoMs start = monoNowMs();
160   const size_t total = FRAME_HEADER_BYTES + payloadLen;
161   bool tear = false;  // env:ble_tear: buffers "stay full" after the first chunk
162 #ifdef FW_INJECT_BLE_TEAR
163   if (channel == BLE_CHANNEL_TELEMETRY && ++s_tearCount % (FW_INJECT_BLE_TEAR) == 0)
  >   tear = true;
164 #endif
165   for (size_t offset = 0; offset < total;) {
166     if (!bleGattSecure() || bleGattGeneration() != generation) return false;  // link
  >   gone: nothing to resync
167     size_t chunk = bleGattChunkSize();
168     if (chunk > total - offset) chunk = total - offset;
169     int tries = 0;
170     while ((tear && offset > 0) || !bleGattNotify(channel, &s_frame[offset], chunk)) {
171       if (offset == 0 && channel == BLE_CHANNEL_TELEMETRY) return false;
172       if (!bleGattSecure() || bleGattGeneration() != generation) return false;
```

sendFrame writes the header and rejects a generation already marked torn. Before each chunk it rechecks security and session identity, then limits the chunk to both negotiated capacity and remaining bytes.

src/transports/ble_transport.c / lines 173-176

```
173       if (++tries >= BLE_NOTIFY_RETRIES) {
174         if (offset > 0) {
175           s_tornGeneration = generation;
176           bleGattDisconnect();
```

A frame that has sent no bytes can be dropped without corrupting the receiver's current framing. Once its first chunk is sent, abandoning it would leave the receiver waiting for bytes from the next frame.

Finish a started frame or end its stream

src/transports/ble_transport.c / lines 177-192

```
177           logPrintf(LOG_LEVEL_WARN, "ble: frame cut after %u of %u bytes - link dropped
  >   to resync",
178                     (unsigned)offset, (unsigned)total);
179         }
180         return false;
181       }
182       vTaskDelay(pdMS_TO_TICKS(5));
183     }
184     offset += chunk;
185   }
186   const uint32_t took = (uint32_t)(monoNowMs() - start);
187   portENTER_CRITICAL(&s_statsMux);
188   ++s_stats.framesSent;
189   if (took > s_stats.maxSendMs) s_stats.maxSendMs = took;
190   portEXIT_CRITICAL(&s_statsMux);
191   return true;
192 }
```

The retry loop delays for 5ms when buffers are busy. Telemetry may be dropped immediately before its first chunk. A repeatedly failing partially sent frame marks the generation torn and disconnects.

The disconnect is part of parser recovery: it forces a new session with cleared decoders. Statistics measure successful complete-frame sends and observed duration. They do not prove an upper bound for all library calls.

Build payloads before choosing their delivery policy

src/transports/ble_transport.c / lines 193-216

```
193
194 static char* payload(void) { return (char*)&s_frame[FRAME_HEADER_BYTES]; }
195
196 static void sendResultFrame(size_t len, uint32_t generation) {
197   if (!sendFrame(BLE_CHANNEL_RESULT, len, generation)) STATS_ADD(resultsDropped, 1);
198 }
199
200 static void sendError(const char* id, ProtocolError error, uint32_t generation) {
201   sendResultFrame(protocolEncodeResultError(id, protocolErrorName(error), payload(),
  >   FRAME_MAX_PAYLOAD), generation);
202 }
203
204 static void sendBusy(const char* id, uint32_t generation) {
205   STATS_ADD(busy, 1);
206   sendResultFrame(protocolEncodeResultBusy(id, payload(), FRAME_MAX_PAYLOAD),
  >   generation);
207 }
208
209 // ---- State and telemetry -----------------------------------------------------
210 static void publishState(bool notify, uint32_t generation) {
211   const size_t len = transportEncodeState(&s_scratch, "ble", payload(),
  >   FRAME_MAX_PAYLOAD);
212   if (len == 0 || !frameWriteHeader(s_frame, len)) return;
213   bleGattSetStateValue(s_frame, FRAME_HEADER_BYTES + len);  // value for a plain read
214   if (notify) sendFrame(BLE_CHANNEL_STATE, len, generation);
215 }
216
```

payload points past the two reserved header bytes. Result helpers encode into that space and count a send failure. State also updates the readable characteristic value; Telemetry can be superseded by a later frame.

src/transports/ble_transport.c / lines 217-221

```
217 static void publishTelemetry(uint32_t generation) {
218   const size_t len = transportEncodeTelemetry(&s_scratch, payload(), FRAME_MAX_PAYLOAD);
219   if (len == 0) return;
220   if (!sendFrame(BLE_CHANNEL_TELEMETRY, len, generation)) STATS_ADD(telemetryDropped,
  >   1);
221 }
```

The code is serialised by one owner, so s_frame is reused only after the current synchronous send returns. Copying a pointer to s_frame into another asynchronous sender would break that ownership argument.

Reserve a bounded pending request slot

src/transports/ble_transport.c / lines 222-239

```
222
223 // ---- Commands
  >   --------------------------------------------------------------------------
224 static int claimSlot(const Command* cmd, uint32_t generation) {
225   for (int i = 0; i < BLE_PENDING; ++i) {
226     if (s_pending[i].used) continue;
227     Pending* p = &s_pending[i];
228     p->used = true;
229     p->op = cmd->op;
230     p->generation = generation;
231     p->seq = ++s_seq & 0x00FFFFFFu;
232     p->sinceMs = monoNowMs();
233     memcpy(p->id, cmd->id, sizeof(p->id));
234     return i;
235   }
236   return -1;
237 }
238
239 static uint32_t tagOf(int slot) { return (uint32_t)slot | (s_pending[slot].seq << 8); }
```

claimSlot finds an unused entry, copies id, remembers generation and assigns a sequence. tagOf packs slot into the low byte and sequence into the upper bits. If all eight slots are used, the caller must return busy.

For slot 3 and sequence 12, the tag is 3 | (12 << 8). drainResults will recover both values. This is a compact identity for work, not a pointer into the pending array.

Capture through retained data rather than driver access

src/transports/ble_transport.c / lines 240-264

```
240
241 // Answered here, like get_sensors: a capture only peeks the snapshot (the raw
242 // level every calibratable driver publishes), it never touches a driver.
243 static void handleCapture(const Command* cmd, uint32_t generation) {
244   configurationPeekApplied(&s_cfg);
245   if (!sensorMaskHas(s_cfg.sensorEnabled, cmd->sensor) ||
  >   !snapshotPeek(&s_scratch.snap)) {
246     sendError(cmd->id, PROTOCOL_ERR_STALE, generation);
247     return;
248   }
249   const RailState rail = railStateRead();
250   snapshotGateRail(&s_scratch.snap, rail.valid5v, rail.epoch);  // never capture from a
  >   failed rail
251   const CaptureLimits limits = {POLICY_CAL_CAPTURE_MAX_AGE_MS, CFG_STALE_ORDINARY_MS,
  >   BOARD_PH_DIVIDER_TOP_OHM,
252                                 BOARD_PH_DIVIDER_BOTTOM_OHM};
253   float x;
254   uint32_t ageMs;
255   const CaptureStatus st = calCaptureFromSnapshot(cmd->sensor, &s_scratch.snap,
  >   monoNowMs(), &limits, &x, &ageMs);
256   if (st != CAPTURE_OK) {
257     sendError(cmd->id, st == CAPTURE_UNSUPPORTED ? PROTOCOL_ERR_BAD_FIELD :
  >   PROTOCOL_ERR_STALE, generation);
258     return;
259   }
260   const int32_t scaled = (int32_t)(x * CAL_X_SCALE + (x >= 0.0f ? 0.5f : -0.5f));
261   if (protocolEncodeCapture(cmd->sensor, scaled, ageMs, s_data, sizeof(s_data)) > 0) {
262     sendResultFrame(protocolEncodeResultOk(cmd->id, s_data, payload(),
  >   FRAME_MAX_PAYLOAD), generation);
263   }
264 }
```

The capture path checks enabled state, obtains a Snapshot, applies live rail gating and asks calCaptureFromSnapshot for a usable raw quantity. The result is rounded into the command x scale.

This is the benefit of publishing hasRaw with each Reading: calibration tooling can inspect an owner-published observation without reaching across tasks into the sensor driver. A stale or unsupported capture produces a typed error.

Answer immediate queries and validate every command first

src/transports/ble_transport.c / lines 265-291

```
265
266 static void handleCommand(size_t len, uint32_t generation) {
267   Command cmd;
268   const ProtocolError error = protocolParseCommand(s_command, len, &cmd);
269   STATS_ADD(commands, 1);
270   if (error != PROTOCOL_OK) {
271     STATS_ADD(commandErrors, 1);
272     logPrintf(LOG_LEVEL_INFO, "ble: command ? -> %s", protocolErrorName(error));
273     sendError(cmd.id, error, generation);
274     return;
275   }
276   logPrintf(LOG_LEVEL_INFO, "ble: command %s", commandOpName(cmd.op));  // never the
  >   fields (set_wifi)
277   if (cmd.op == CMD_OP_GET_SENSORS) {
278     configurationPeekApplied(&s_cfg);
279     if (protocolEncodeSensorList(s_cfg.sensorEnabled, s_data, sizeof(s_data)) > 0) {
280       sendResultFrame(protocolEncodeResultOk(cmd.id, s_data, payload(),
  >   FRAME_MAX_PAYLOAD), generation);
281     }
282     return;
283   }
284   if (cmd.op == CMD_OP_GET_CALIBRATION) {
285     configurationPeekApplied(&s_cfg);
286     const CalibrationView v = {s_cfg.ph, s_cfg.tds, s_cfg.light, s_cfg.calOverridden};
287     if (protocolEncodeCalibration(&v, s_data, sizeof(s_data)) > 0) {
288       sendResultFrame(protocolEncodeResultOk(cmd.id, s_data, payload(),
  >   FRAME_MAX_PAYLOAD), generation);
289     }
290     return;
291   }
```

handleCommand decodes s_command into a local Command. Parse errors produce error replies; only operation names are logged, avoiding credential-field logging. Sensor list, calibration list and capture can be answered from retained copies.

Answer immediate queries and validate every command first / continued

src/transports/ble_transport.c / lines 292-300

```
292   if (cmd.op == CMD_OP_CAL_CAPTURE) {
293     handleCapture(&cmd, generation);
294     return;
295   }
296   const int slot = claimSlot(&cmd, generation);
297   if (slot < 0) {
298     sendBusy(cmd.id, generation);
299     return;
300   }
```

These read-only operations need no pending slot because they complete in the current call. A state-changing operation reaches claimSlot and creates an expiry for the owning service.

Queue mutations to the responsible service

src/transports/ble_transport.c / lines 301-328

```
  const MonoMs expires = monoNowMs() + BLE_COMMAND_TTL_MS;
  bool queued;
  if (cmd.op == CMD_OP_RESET_LATCH) {
    const SafetyCommand sc = {SAFETY_CMD_RESET_LATCH, cmd.rule, expires, tagOf(slot)};
    queued = safetySubmitCommand(&sc, generation, sessionNow, onSafetyResult, NULL);  //
   dropped if the link closes
  } else {
    memset(&s_req, 0, sizeof(s_req));
    s_req.tag = tagOf(slot);
    s_req.session = generation;
    s_req.sessionNow = sessionNow;
    s_req.expiresMs = expires;
    s_req.reply = onConfigResult;
    switch (cmd.op) {
      case CMD_OP_SET_SENSOR_ENABLED:
        s_req.kind = CONFIG_REQ_SET_SENSOR_ENABLED;
        s_req.sensor = cmd.sensor;
        s_req.enabled = cmd.enabled;
        break;
      case CMD_OP_SET_WIFI:
        s_req.kind = CONFIG_REQ_SET_WIFI;
        memcpy(s_req.ssid, cmd.ssid, sizeof(s_req.ssid));
        memcpy(s_req.password, cmd.password, sizeof(s_req.password));
        break;
      case CMD_OP_FORGET_WIFI: s_req.kind = CONFIG_REQ_FORGET_WIFI; break;
      case CMD_OP_SET_CALIBRATION:
        s_req.kind = CONFIG_REQ_SET_CALIBRATION;
        s_req.sensor = cmd.sensor;
        s_req.calX1 = cmd.calX1;
```

Reset-latch becomes a SafetyCommand. Other mutations fill ConfigRequest with session, expiry, tag and callback. The service receives a copy through its queue. A rejected enqueue frees the pending slot and replies busy.

Queue mutations to the responsible service / continued

src/transports/ble_transport.c / lines 329-340

```
329           s_req.calY1 = cmd.calY1;
330           s_req.calX2 = cmd.calX2;
331           s_req.calY2 = cmd.calY2;
332           break;
333         default:
334           s_req.kind = CONFIG_REQ_RESET_CALIBRATION;
335           s_req.sensor = cmd.sensor;
336           break;
337       }
338       queued = configurationSubmit(&s_req);
339       memset(s_req.password, 0, sizeof(s_req.password));
340     }
```

src/transports/ble_transport.c / lines 341-346

```
341     memset(cmd.password, 0, sizeof(cmd.password));
342     if (!queued) {
343       s_pending[slot].used = false;
344       sendBusy(cmd.id, generation);
345     }
346   }
```

The code clears password arrays after submission. That reduces residual copies in these objects; it does not prove every earlier stack or library copy has vanished. Most importantly for architecture, the BLE task never writes NVS or changes a driver directly.

Process fragmented and coalesced commands

src/transports/ble_transport.c / lines 347-369

```
347
348 static void drainRx(uint32_t generation) {
349   uint8_t chunk[128];
350   size_t got;
351   while ((got = xStreamBufferReceive(s_rx, chunk, sizeof(chunk), 0)) > 0) {
352     size_t offset = 0;
353     while (offset < got) {
354       size_t used;
355       const uint8_t* framePayload;
356       size_t payloadLen;
357       const FrameStatus status =
358           frameAssemblerFeed(&s_assembler, &chunk[offset], got - offset, &used,
  >   &framePayload, &payloadLen);
359       offset += used;
360       if (status == FRAME_READY) {
361         memcpy(s_command, framePayload, payloadLen);  // replies reuse s_frame
362         handleCommand(payloadLen, generation);
363       } else if (status == FRAME_ERROR) {
364         STATS_ADD(commandErrors, 1);
365         logPrintf(LOG_LEVEL_WARN, "ble: bad frame length from client, buffer dropped");
366       }
367     }
368   }
369 }
```

The owner pulls up to 128 bytes at a time, feeds the assembler repeatedly and advances by consumed. A complete payload is copied to s_command before handleCommand can reuse s_frame for replies.

That extra copy is intentional ownership management. Borrowed assembly bytes should not be assumed safe while other protocol work reuses shared transport buffers. A bad frame length is counted and logged.

Match a reply to both request identity and session

src/transports/ble_transport.c / lines 370-396

```
370
371 static void drainResults(uint32_t generation) {
372   BleResult r;
373   while (xQueueReceive(s_results, &r, 0) == pdTRUE) {
374     const uint32_t slot = r.tag & 0xFFu;
375     if (slot >= BLE_PENDING) continue;
376     Pending* p = &s_pending[slot];
377     if (!p->used || p->seq != (r.tag >> 8)) continue;  // freed already: a late reply
378     p->used = false;
379     if (p->generation != generation) continue;  // asked on an earlier link: never
  >   answered on this one
380     if (r.error != PROTOCOL_OK) {
381       sendError(p->id, r.error, generation);
382       logPrintf(LOG_LEVEL_INFO, "ble: %s -> %s", commandOpName(p->op),
  >   protocolErrorName(r.error));
383       continue;
384     }
385     if (p->op == CMD_OP_SET_SENSOR_ENABLED || p->op == CMD_OP_SET_CALIBRATION ||
386         p->op == CMD_OP_RESET_CALIBRATION) {
387       snprintf(s_data, sizeof(s_data), "{\"applied\":true,\"cfg_rev\":%u}",
  >   (unsigned)r.cfgRev);
388     } else if (p->op == CMD_OP_RESET_LATCH) {
389       snprintf(s_data, sizeof(s_data), "{\"applied\":true}");
390     } else {
391       snprintf(s_data, sizeof(s_data), "{\"stored\":true}");
392     }
393     logPrintf(LOG_LEVEL_INFO, "ble: %s -> ok", commandOpName(p->op));
394     sendResultFrame(protocolEncodeResultOk(p->id, s_data, payload(), FRAME_MAX_PAYLOAD),
  >    generation);
395   }
396 }
```

The low tag byte chooses a candidate slot; the upper bits must match its current sequence. The entry is freed before sending. A generation mismatch discards the reply, preventing an earlier client session's answer from appearing on a new one.

The payload distinguishes applied configuration from stored Wi-Fi credentials. A successful storage reply does not claim that the access point has accepted a connection. Read the result words as specific completed stages.

Advance an event cursor only after successful delivery

src/transports/ble_transport.c / lines 397-422

```
397
398 // A link change orphans every request of the old link (Configuration drops
399 // stale-session requests without replying); a reply that never comes frees
400 // its slot after BLE_PENDING_LIMIT_MS.
401 static void expirePending(uint32_t generation, MonoMs now) {
402   for (int i = 0; i < BLE_PENDING; ++i) {
403     Pending* p = &s_pending[i];
404     if (p->used && (p->generation != generation || now - p->sinceMs >
  >   BLE_PENDING_LIMIT_MS)) p->used = false;
405   }
406 }
407
408 // Peek, send, then commit: an event that could not be sent stays at the
409 // cursor and is retried on the next wake (the journal reports a gap if it
410 // overruns meanwhile). If the link closes first, the next session starts at
411 // the journal head, so its first event reports gap:true.
412 static void sendEvents(uint32_t generation) {
413   JournalEvent e;
414   bool gap;
415   JournalCursor next = s_journalCursor;
416   while (safetyJournalNext(&next, &e, &gap)) {
417     const size_t len = transportEncodeEvent(&e, gap || s_eventLost, payload(),
  >   FRAME_MAX_PAYLOAD);
418     if (!sendFrame(BLE_CHANNEL_RESULT, len, generation)) {
419       STATS_ADD(resultsDropped, 1);
420       s_eventPending = true;
421       break;
422     }
```

expirePending frees orphaned or over-age requests. sendEvents reads using a temporary cursor and commits s_journalCursor only when the frame send succeeds.

src/transports/ble_transport.c / lines 423-427

```
423     s_journalCursor = next;
424     s_eventPending = false;
425     s_eventLost = false;
426   }
427 }
```

Suppose event 40 cannot send. The retained cursor still precedes 40, so the next wake can retry it. If the journal overwrites old entries first, its gap indication exposes that lost continuity instead of inventing a complete history.

Read counters without holding locks during formatting

src/transports/ble_transport.c / lines 428-440

```
428
429 static void logStats(void) {
430   BleGattStats g;
431   BleTransportStats t;
432   bleGattGetStats(&g);
433   bleTransportGetStats(&t);
434   logPrintf(LOG_LEVEL_INFO, "ble: secure=%d bonds=%d conn=%lu disc=%lu pair ok=%lu
  >   rej=%lu | frames %lu tlmDrop %lu "
435             "resDrop %lu busy %lu maxSend %lums notifyFail %lu",
436             bleGattSecure(), bleGattBondCount(), (unsigned long)g.connects, (unsigned
  >   long)g.disconnects,
437             (unsigned long)g.pairingsAccepted, (unsigned long)g.pairingsRejected,
  >   (unsigned long)t.framesSent,
438             (unsigned long)t.telemetryDropped, (unsigned long)t.resultsDropped,
  >   (unsigned long)t.busy,
439             (unsigned long)t.maxSendMs, (unsigned long)g.notifyFailures);
440 }
```

The log gathers separate GATT and transport copies before formatting their fields. maxSendMs is an observation, while dropped counters expose workload the transport could not retain or deliver.

A healthy connection count alone says little about command service. Include queue saturation, parse errors and session changes when diagnosing a client report.

Wake for input or for periodic work

src/transports/ble_transport.c / lines 441-468

```
441
442 // ---- Task
  >   ------------------------------------------------------------------------------
443 static void bleTask(void* arg) {
444   (void)arg;
445   if (esp_task_wdt_add(NULL) != ESP_OK) logPrintf(LOG_LEVEL_ERROR, "ble: task watchdog
  >   add failed");
446   logPrintf(LOG_LEVEL_WARN, "ble: PAIRING PASSKEY %06lu (bonded owners: %d)", (unsigned
  >   long)bleGattPasskey(),
447             bleGattBondCount());
448
449   uint32_t generation = bleGattGeneration();
450   MonoMs nextState = 0, nextTelemetry = 0, nextStats = monoNowMs() + BLE_STATS_LOG_MS,
  >   bootPressedAt = 0;
451   bool wasSecure = false, bondsCleared = false;
452   for (;;) {
453     ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(BLE_WAKE_MS));
454     const MonoMs now = monoNowMs();
455
456     if (bleGattGeneration() != generation) {  // new session: drop half-received
  >   commands
457       generation = bleGattGeneration();
458       frameAssemblerReset(&s_assembler);
459       xStreamBufferReset(s_rx);
460       if (s_eventPending) s_eventLost = true;  // never delivered: the next link is told
461       s_eventPending = false;
462     }
463     expirePending(generation, now);
464     const bool secure = bleGattSecure();
465     if (secure != wasSecure) {
466       logPrintf(LOG_LEVEL_INFO, "ble: %s", secure ? "owner connected (secure)" : "link
  >   closed");
467       wasSecure = secure;
468       if (secure) {
```

Wake for input or for periodic work / continued

bleTask subscribes to the watchdog and starts a bounded wait. A changed generation clears partial assembly and RX bytes; a newly secure link schedules State immediately and starts an event cursor at the current journal head.

src/transports/ble_transport.c / lines 469-473

```
469          nextState = now;  // State at once
470          safetyJournalCursorInit(&s_journalCursor);  // events from now on, not history
471        }
472      }
473
```

The source intentionally begins a new subscriber at current events rather than replaying all history. A generation change and an event gap have different meanings: one is session identity, the other is delivery continuity.

Prioritise commands before periodic telemetry

src/transports/ble_transport.c / lines 474-499

```
474     // Priority: commands and their results, alarm events, State, Telemetry.
475     if (secure) drainRx(generation);
476     drainResults(generation);
477     if (secure) sendEvents(generation);
478     if (now >= nextState) {
479       publishState(secure, generation);
480       nextState = now + CFG_BLE_STATE_PERIOD_MS;
481     }
482     if (secure && now >= nextTelemetry) {
483       publishTelemetry(generation);
484       nextTelemetry = now + CFG_BLE_TELEMETRY_PERIOD_MS;
485     }
486
487     // Owner recovery: BOOT held 10 s while running (not at reset: strap pin).
488     if (digitalRead(BLE_BOOT_PIN) == LOW) {
489       if (bootPressedAt == 0) bootPressedAt = now;
490       if (!bondsCleared && now - bootPressedAt >= BLE_BOND_CLEAR_MS) {
491         bleGattClearBonds();
492         logPrintf(LOG_LEVEL_WARN, "ble: BOOT held 10 s - all bonds cleared, pair again
  >   with the passkey");
493         bondsCleared = true;
494       }
495     } else {
496       bootPressedAt = 0;
497       bondsCleared = false;
498     }
499
```

The loop drains commands and results, sends alarm events, then State and Telemetry when due. A runtime 10s BOOT hold clears bonds once per hold. Finally it marks progress and resets its watchdog subscription.

src/transports/ble_transport.c / lines 500-507

```
500     if (now >= nextStats) {
501       logStats();
502       nextStats = now + BLE_STATS_LOG_MS;
503     }
504     progressMark(TASK_ID_BLE_TRANSPORT);
505     esp_task_wdt_reset();
506   }
507 }
```

The periodic next timestamps are now plus period, so a delayed iteration shifts the next send. They are not the phase-preserving release schedule used for sensor acquisition. Different contracts justify different timing choices.

Create resources before allowing callbacks to use them

src/transports/ble_transport.c / lines 508-535

```
508
509 bool bleTransportInit(void) {
510   if (s_task != NULL) return true;
511   s_rx = xStreamBufferCreateStatic(BLE_RX_BYTES, 1, s_rxStorage, &s_rxStruct);
512   s_results = xQueueCreateStatic(BLE_PENDING, sizeof(BleResult), s_resultStorage,
  >   &s_resultControl);
513   if (s_rx == NULL || s_results == NULL) return false;
514   frameAssemblerInit(&s_assembler);
515   pinMode(BLE_BOOT_PIN, INPUT_PULLUP);
516   if (!bleGattStart(BLE_NODE_NAME, onRx, NULL)) {
517     logPrintf(LOG_LEVEL_ERROR, "ble: NimBLE failed to start");
518     return false;
519   }
520   publishState(false, bleGattGeneration());  // also fixes the shared boot id (radio
  >   running)  // readable State before the first client
521
522   const TaskSpec* spec = cfgTaskSpec(TASK_ID_BLE_TRANSPORT);
523   if (spec->stackBytes != sizeof(s_stack)) {
524     logPrintf(LOG_LEVEL_WARN, "ble: stack %u B, task table says %lu B",
  >   (unsigned)sizeof(s_stack),
525               (unsigned long)spec->stackBytes);
526   }
527   s_task = xTaskCreateStaticPinnedToCore(bleTask, spec->name, sizeof(s_stack), NULL,
  >   spec->priority, s_stack,
528                                          &s_tcb, spec->core);
529   if (s_task == NULL) return false;
530   TaskHandle_t host = xTaskGetHandle("nimble_host");
531   logPrintf(LOG_LEVEL_INFO, "ble: started (core %u, prio %u), NimBLE host core %d prio
  >   %u",
532             (unsigned)spec->core, (unsigned)spec->priority, host != NULL ?
  >   (int)xTaskGetAffinity(host) : -1,
533             host != NULL ? (unsigned)uxTaskPriorityGet(host) : 0u);
534   return true;
535 }
```

Init creates RX and result buffers, resets framing, configures BOOT input and starts GATT. Only then does it create the application task with its configured priority and affinity.

The first State encode occurs after radio startup and fixes the shared boot id. The logged NimBLE host task details let you compare intended context with the framework task that actually receives commands.

Return copies and an opaque owner handle

src/transports/ble_transport.c / lines 536-545

```
536
537 void bleTransportGetStats(BleTransportStats* out) {
538   portENTER_CRITICAL(&s_statsMux);
539   *out = s_stats;
540   portEXIT_CRITICAL(&s_statsMux);
541 }
542
543 void* bleTransportTaskHandle(void) { return s_task; }
544
545 #endif  // FW_ENABLE_BLE
```

GetStats copies under a short lock. TaskHandle supplies Supervisor with the owner identity. These functions expose observation, not permission for another task to reuse private frame storage.

You have now read the complete BLE path, including the cases where it refuses, drops, retries or disconnects. Keep those outcomes separate when following a full configuration event in the last chapter.

Know what connected means in this file

The ESP32 joins an access point as a Wi-Fi station. SSID is the network name it tries to join. The password is a credential for that join. Configuration supplies both to the sole Wi-Fi owner; it does not expose the password in WifiStatus or State messages.

Association connects the station to the access point. The network also needs an address for ordinary IPv4 traffic; DHCP is the usual protocol for obtaining one automatically. In this adapter, wifiLanConnected requires both the library's connected status and a nonzero local IPv4 address. Starting an attempt is earlier than satisfying that test.

REMEMBER

Configured, attempting, connected and serving an application request describe different completed stages.

Observation	Meaning in this implementation
CONNECTING	An attempt or reconnection path is in progress.
CONNECTED	The adapter sees connection status and a nonzero IPv4 address.
FAILED	An attempt reached its failure/timeout path; retry can follow.
RSSI	A received-signal-strength indicator, reported in dBm while connected.

A stored SSID proves only that credentials are configured. An IP address does not prove that a remote application server is reachable. This snapshot has no MQTT publisher despite its historical WifiMqtt task name. Read the implemented connection milestone before interpreting a status label.

The next lesson adds attempt deadlines and retry backoff. Their purpose is to let this owner revisit status without holding acquisition or Safety inside a networking wait.

Reconnect in steps and preserve local control

wifi_mqtt.c retains a historical name. In this source snapshot its body manages Wi-Fi association and retry; it does not contain an MQTT publisher. A filename is not evidence that an advertised feature exists.

The Wi-Fi task starts an association attempt, stores a 15000ms deadline and returns to its loop. Later passes inspect connection status. On failure it schedules a retry with bounded exponential backoff and jitter. It can also notice a power-mode stop request while an attempt or backoff is pending.

For an illustrative backoff without jitter, delays can grow 1s, 2s, 4s, 8s and so on until the cap. Jitter perturbs the delay so many devices do not all retry on exactly the same schedule. The exact archived constants and implementation are in domain/backoff.c and app_config.h.

Local measurement and Safety must continue when no access point is available. Their tasks do not synchronously call a network publish from inside a driver. Transport failure can reduce remote visibility; it should not require the sensor worker to wait through a reconnect attempt.

State frames describe current retained configuration and status. Telemetry frames carry observations. Event history needs a bounded journal and a cursor with a way to report gaps when older entries have been overwritten. These are separate information contracts, just as a latest Snapshot and an alarm FIFO are separate contracts.

You can now trace a command in both directions: incoming bytes become a decoded request, the owning service performs it, and a result travels back through the transport. The radio callback never has to own the sensor bus or relay.

COMMON PITFALL

A transport acknowledgement only proves the defined transport milestone. It does not automatically prove a requested setting reached the hardware.

Return a delay instead of performing a delay

A failed connection should leave time for other work and avoid retrying at a constant aggressive rate. Backoff stores the next base delay and its bounds.

Randomness is passed in, making the same helper testable without the ESP32 RNG.

src/domain/backoff.h / lines 1-20

```
1  // Exponential retry backoff with jitter (§17: 1, 2, 4, 8 s ... up to 60 s).
2  // Pure C; the caller supplies the random number. WP8 (Wi-Fi reconnect).
3  #pragma once
4  #include <stdint.h>
5
6  typedef struct Backoff {
7    uint32_t initialMs;
8    uint32_t maxMs;
9    uint32_t currentMs;   // base delay of the next attempt
10   uint8_t jitterPct;    // +/- percentage applied to each delay
11 } Backoff;
12
13 void backoffInit(Backoff* b, uint32_t initialMs, uint32_t maxMs, uint8_t jitterPct);
14
15 // Back to initialMs (after a successful connection).
16 void backoffReset(Backoff* b);
17
18 // Delay before the next attempt: currentMs +/- jitter, never above maxMs and
19 // never 0. Then the base doubles, capped at maxMs.
20 uint32_t backoffNext(Backoff* b, uint32_t random);
```

backoffNext computes how long the caller should wait; it never blocks a task itself. A successful connection calls reset to restore the initial base.

Jitter varies nearby retry times. With a 1000ms base and 20 percent span, this implementation can produce delays from 800 through 1200ms before maximum clamping. The owner turns that result into a monotonic deadline.

Normalise parameters and reset retry history

Use initial 1000ms, maximum 60000ms and jitter 20 to trace the arithmetic.

The returned delay and the next stored base are separate values.

src/domain/backoff.c / lines 1-10

```
#include "domain/backoff.h"

void backoffInit(Backoff* b, uint32_t initialMs, uint32_t maxMs, uint8_t jitterPct) {
  b->initialMs = initialMs == 0 ? 1 : initialMs;
  b->maxMs = maxMs < b->initialMs ? b->initialMs : maxMs;
  b->jitterPct = jitterPct > 100 ? 100 : jitterPct;
  b->currentMs = b->initialMs;
}

void backoffReset(Backoff* b) { b->currentMs = b->initialMs; }
```

Init replaces a zero initial delay with one, ensures maximum is at least initial and caps jitter at 100 percent. Reset restores currentMs from the retained initial setting.

This pure helper needs no queue or task handle. Its output becomes useful when the Wi-Fi owner checks now against a deadline in each iteration.

Widen arithmetic before doubling

src/domain/backoff.c / lines 11-22

```
11
12 uint32_t backoffNext(Backoff* b, uint32_t random) {
13   const uint32_t base = b->currentMs;
14   const uint32_t span = (uint32_t)(((uint64_t)base * b->jitterPct) / 100u);
15   uint64_t delay = (uint64_t)base - span + (span > 0 ? random % (2u * (uint64_t)span +
 >   1u) : 0u);
16   if (delay > b->maxMs) delay = b->maxMs;
17   if (delay == 0) delay = 1;
18
19   const uint64_t doubled = (uint64_t)base * 2u;
20   b->currentMs = doubled > b->maxMs ? b->maxMs : (uint32_t)doubled;
21   return (uint32_t)delay;
22 }
```

The span uses uint64_t multiplication, then the random remainder chooses an offset within the inclusive range. The final delay is clamped to maximum and at least one. The next base doubles with wide arithmetic before being capped.

For random 0, a 1000ms base with 20 percent jitter returns 800ms and stores 2000ms for next time. At the cap, clamping makes the returned distribution asymmetric. This is bounded retry jitter, not an exact uniform distribution over all final delays.

Expose operations the owner can poll

This is the second C-callable C++ adapter. It isolates Arduino WiFi integration from the C owner state machine.

Connection initiation and connection completion are deliberately different operations.

src/transports/wifi_lan.h / lines 1-24

```
// Wi-Fi station shim over the Arduino WiFi library (C++),
// callable from C. Work package WP8 (Stage 8.4). Used only by the WifiMqtt
// task (transports/wifi_mqtt.c), which is the sole Wi-Fi owner.
//
// The Arduino libraries keep netif and the default event loop initialised
// exactly once (initialising them again from IDF calls would fail under
// arduino-esp32 2.0.x). Credentials stay in RAM: WiFi.persistent(false) sets
// WIFI_STORAGE_RAM, so the Wi-Fi driver never writes them to NVS.
// Compiled only with -D FW_ENABLE_WIFI.
#pragma once
#include <stdbool.h>
#include <stdint.h>

#ifdef __cplusplus
extern "C" {
#endif

bool wifiLanBegin(const char* hostname);  // station mode, RAM storage, no
  auto-reconnect
void wifiLanConnect(const char* ssid, const char* password);  // non-blocking
void wifiLanDisconnect(void);
bool wifiLanConnected(void);              // associated and has an IPv4 address
bool wifiLanIp(char out[16]);
int32_t wifiLanRssi(void);

```

Begin establishes station policy; Connect starts an attempt; Connected also requires an IPv4 address. RadioOff and RadioOn support the power coordinator's handshake.

src/transports/wifi_lan.h / lines 25-32

```
// WP9 low power: WIFI_OFF (radio and its RF calibration stopped), and back to
// station mode. Both return false if the driver refused.
bool wifiLanRadioOff(void);
bool wifiLanRadioOn(void);

#ifdef __cplusplus
}
#endif
```

The adapter keeps credentials in RAM rather than letting the Wi-Fi library persist its own copy. Configuration remains the application's single persistence owner.

Let one application state machine own reconnection

Read the library calls as integration policy. This source disables competing reconnect and persistence behaviour so the application owner can account for them.

A wrapper can be short and still determine ownership of a complex library.

src/transports/wifi_lan.cpp / lines 1-27

```
// Arduino WiFi implementation of wifi_lan.h. C++ only because the
// Arduino libraries are; everything it exports is C.
#ifdef FW_ENABLE_WIFI
#include "transports/wifi_lan.h"

#include <WiFi.h>
#include <stdio.h>

extern "C" bool wifiLanBegin(const char* hostname) {
  WiFi.persistent(false);         // WIFI_STORAGE_RAM: credentials never reach NVS from
  here
  WiFi.setAutoReconnect(false);   // retries are WifiMqtt's backoff, not the driver's
  if (!WiFi.mode(WIFI_STA)) return false;
  WiFi.setHostname(hostname);
  // Modem sleep stays on (the default): required while BLE shares the radio.
  return true;
}

extern "C" void wifiLanConnect(const char* ssid, const char* password) {
  WiFi.disconnect(false, false);
  WiFi.begin(ssid, password);
}

extern "C" void wifiLanDisconnect(void) { WiFi.disconnect(false, false); }

extern "C" bool wifiLanConnected(void) {
  return WiFi.status() == WL_CONNECTED && static_cast<uint32_t>(WiFi.localIP()) != 0;
}
```

Begin selects RAM persistence and disables automatic reconnect. Connect first disconnects the existing station then starts WiFi.begin. Connected requires both WL_CONNECTED and a nonzero local address.

The adapter does not wait in a while loop for DHCP. It returns to WifiMqtt, which checks later. This is how the firmware avoids occupying the owner task for an entire 15s association allowance.

Copy IP text and acknowledge radio mode changes

src/transports/wifi_lan.cpp / lines 28-48

```
28
29 extern "C" bool wifiLanIp(char out[16]) {
30   if (!wifiLanConnected()) {
31     out[0] = '\0';
32     return false;
33   }
34   const IPAddress ip = WiFi.localIP();
35   snprintf(out, 16, "%u.%u.%u.%u", ip[0], ip[1], ip[2], ip[3]);
36   return true;
37 }
38
39 extern "C" int32_t wifiLanRssi(void) { return wifiLanConnected() ? WiFi.RSSI() : 0; }
40
41 extern "C" bool wifiLanRadioOff(void) {
42   WiFi.disconnect(true, false);  // wifioff = true
43   return WiFi.mode(WIFI_OFF);
44 }
45
46 extern "C" bool wifiLanRadioOn(void) { return WiFi.mode(WIFI_STA); }
47
48 #endif  // FW_ENABLE_WIFI
```

wifiLanIp writes at most the supplied 16-byte IPv4 array. RSSI is meaningful only while connected. RadioOff disconnects and selects WIFI_OFF; RadioOn returns to station mode.

The returned bool reports the library's mode-setting result. PowerCoord waits on the owner's acknowledgement rather than assuming that posting a stop request already switched the radio off.

Publish connection state without credentials

The historical filename contains MQTT, but this snapshot implements a Wi-Fi station owner and status reporting. There is no MQTT publisher here.

Read implemented behaviour rather than inferring features from a task name.

src/transports/wifi_mqtt.h / lines 1-26

```
// WifiMqtt: sole Wi-Fi owner (station), never restarts sensing. Work package
// WP8 (Stage 8.4). Architecture §17; FirmwareContract.md v2.
//
// Task "WifiMqtt" (cfgTaskSpec: core 0, priority 2, 6 KB):
//   station   credentials from Configuration; connect, and on failure retry
//             with exponential backoff 1, 2, 4 ... 60 s +/- 20 % jitter
//             (domain/backoff). A credential change or forget restarts it at
//             once. The Wi-Fi driver keeps nothing in NVS (WIFI_STORAGE_RAM).
//   status    state / SSID / IP / RSSI for the State frame (the app shows it
//             on its Wi-Fi page)
//   WP9       wifiMqttRequestStop() disconnects and puts the radio in
//             WIFI_OFF (low power); wifiMqttRadioOff() is the acknowledgement
//             PowerCoordinator waits for. RequestStart() turns it back on and
//             rejoins.
// The app talks to the controller over Bluetooth only: the read-only LAN link
// (TCP 47800 + mDNS) was removed after Stage 8.4. MQTT (server publishing) is
// not in scope yet; the name is kept from the architecture's task table.
//
// Built only with -D FW_ENABLE_WIFI (production envs). Without it the task
// does not exist and the status is always "unconfigured".
#pragma once
#include <stdbool.h>
#include <stdint.h>

#include "domain/protocol_codec.h"  // PROTOCOL_SSID_MAX

```

WifiStatus contains state, SSID, IP and RSSI. The password stays out of this public status record. Stop and start are requests; RadioOff is the acknowledgement query.

src/transports/wifi_mqtt.h / lines 27-36

```
typedef uint8_t WifiState;
enum { WIFI_STATE_UNCONFIGURED = 0, WIFI_STATE_CONNECTING, WIFI_STATE_CONNECTED,
  WIFI_STATE_FAILED };

typedef struct WifiStatus {
  WifiState state;
  char ssid[PROTOCOL_SSID_MAX + 1];
  char ip[16];             // dotted IPv4 when connected, else ""
  int32_t rssi;            // dBm when connected
} WifiStatus;

```

Publish connection state without credentials / continued

src/transports/wifi_mqtt.h / lines 37-54

```
37 bool wifiMqttInit(void);          // starts the task; never waits for an access point
38 void wifiMqttRequestStop(void);   // WP9 power transition: drop Wi-Fi, radio off,
 >   preempt retry waits
39 void wifiMqttRequestStart(void);
40 bool wifiMqttRadioOff(void);      // true once a stop request has the radio in WIFI_OFF
41
42 void wifiMqttGetStatus(WifiStatus* out);  // zero wait, any task
43 const char* wifiStateName(WifiState s);   // contract spelling
44
45 typedef struct WifiMqttStats {
46   uint32_t connects;          // station connections established
47   uint32_t connectFailures;   // attempts that timed out or were refused
48   uint32_t lastBackoffMs;
49 } WifiMqttStats;
50
51 void wifiMqttGetStats(WifiMqttStats* out);
52
53 // For Supervisor. NULL before init or without FW_ENABLE_WIFI.
54 void* wifiMqttTaskHandle(void);
```

The optional build can omit this task entirely. Supervisor accepts a null task handle in that build, while the disabled implementation reports an unconfigured status.

Supply status even when Wi-Fi is omitted

The owner advances one of four phases each time it wakes: idle, attempting, waiting or connected. It never restarts sensing to repair Wi-Fi.

Keep one 15s attempt deadline and one retry deadline in mind; the same stored field has a different meaning in each phase.

src/transports/wifi_mqtt.c / lines 1-25

```
#include "transports/wifi_mqtt.h"

#include <freertos/FreeRTOS.h>
#include <string.h>

static portMUX_TYPE s_statusMux = portMUX_INITIALIZER_UNLOCKED;
static WifiStatus s_status;  // all zero = unconfigured

const char* wifiStateName(WifiState s) {
  switch (s) {
    case WIFI_STATE_CONNECTING: return "connecting";
    case WIFI_STATE_CONNECTED: return "connected";
    case WIFI_STATE_FAILED: return "failed";
    default: return "unconfigured";
  }
}

void wifiMqttGetStatus(WifiStatus* out) {
  portENTER_CRITICAL(&s_statusMux);
  *out = s_status;
  portEXIT_CRITICAL(&s_statusMux);
}

#ifndef FW_ENABLE_WIFI

```

Supply status even when Wi-Fi is omitted / continued

The guarded status copy and state names exist outside the feature branch. Disabled builds return false from init and report the radio already off because no Wi-Fi owner exists.

src/transports/wifi_mqtt.c / lines 26-32

```
26 bool wifiMqttInit(void) { return false; }
27 void wifiMqttRequestStop(void) {}
28 void wifiMqttRequestStart(void) {}
29 bool wifiMqttRadioOff(void) { return true; }  // no Wi-Fi in this build: nothing to
 >   switch off
30 void wifiMqttGetStats(WifiMqttStats* out) { memset(out, 0, sizeof(*out)); }
31 void* wifiMqttTaskHandle(void) { return NULL; }
32
```

Conditional compilation removes work from the image. It is different from a task that exists but currently happens to be idle.

Store owner state separately from shared observations

src/transports/wifi_mqtt.c / lines 33-57

```
#else

#include <esp_system.h>
#include <esp_task_wdt.h>
#include <freertos/task.h>
#include <stdio.h>

#include "app_config.h"
#include "diagnostics/logger.h"
#include "diagnostics/progress.h"
#include "domain/backoff.h"
#include "hal/clock.h"
#include "services/configuration.h"
#include "transports/wifi_lan.h"

#define WIFI_HOSTNAME "hydro-esp1"
#define WIFI_WAKE_MS 50u
#define WIFI_ATTEMPT_TIMEOUT_MS 15000u  // one association +
   DHCP attempt
#define WIFI_JITTER_PCT 20u
#define WIFI_STATUS_REFRESH_MS 2000u    // RSSI / IP copy
  for State
#define WIFI_STATS_LOG_MS 60000u

typedef uint8_t Phase;
enum { PHASE_IDLE = 0, PHASE_ATTEMPT, PHASE_WAIT,
  PHASE_CONNECTED };

```

COMMON PITFALL

When redesigning this boundary, use an explicit atomic or guarded message contract for shared control state.

The enabled branch reserves task storage, stop/radio flags, stats, private credentials, backoff, phase and deadline. Credentials changes are identified by a counter.

src/transports/wifi_mqtt.c / lines 58-67

```
static StaticTask_t s_tcb;
static StackType_t s_stack[6 * 1024];  //
  cfgTaskSpec(TASK_ID_WIFI_MQTT)->stackBytes
static TaskHandle_t s_task = NULL;

static volatile bool s_stopRequested;
static volatile bool s_radioOff;  // WP9 acknowledgement:
  the radio is in WIFI_OFF
static portMUX_TYPE s_statsMux =
  portMUX_INITIALIZER_UNLOCKED;
static WifiMqttStats s_stats;
#define STATS_ADD(field, n)          \
  do {                               \
```

Store owner state separately from shared observations / continued

src/transports/wifi_mqtt.c / lines 68-79

```
    portENTER_CRITICAL(&s_statsMux); \
    s_stats.field += (n);            \
    portEXIT_CRITICAL(&s_statsMux);  \
  } while (0)

// Task-private (static: not on the 6 KB stack).
static ConfigWifi s_creds;
static uint32_t s_credsChanges = UINT32_MAX;
static Backoff s_backoff;
static Phase s_phase;
static MonoMs s_deadlineMs;

```

The shared stop flags are declared volatile in this original source. Volatile forces accesses under the compiler's rules; it does not provide a C atomic or a general cross-core synchronisation proof. The notifications wake the owner but do not change that language-level distinction.

Publish a complete bounded status copy

src/transports/wifi_mqtt.c / lines 80-99

```
80 static void setStatus(WifiState state) {
81   WifiStatus st;
82   memset(&st, 0, sizeof(st));
83   st.state = state;
84   memcpy(st.ssid, s_creds.ssid, sizeof(st.ssid));
85   if (state == WIFI_STATE_CONNECTED) {
86     wifiLanIp(st.ip);
87     st.rssi = wifiLanRssi();
88   }
89   portENTER_CRITICAL(&s_statusMux);
90   s_status = st;
91   portEXIT_CRITICAL(&s_statusMux);
92 }
93
94 static WifiState currentState(void) {
95   portENTER_CRITICAL(&s_statusMux);
96   const WifiState state = s_status.state;
97   portEXIT_CRITICAL(&s_statusMux);
98   return state;
99 }
```

setStatus starts with a cleared local record, copies SSID and only fills IP/RSSI when connected. It then replaces s_status under the short critical section. currentState reads under the same guard.

Library queries happen before the lock. Other tasks never receive a pointer into s_creds or an unfinished status update.

Convert attempt outcomes into phase transitions

src/transports/wifi_mqtt.c / lines 100-125

```
100
101 // ---- Station
  >   ------------------------------------------------------------------------------
102 static void teardown(void) { wifiLanDisconnect(); }
103
104 static void startAttempt(MonoMs now) {
105   wifiLanConnect(s_creds.ssid, s_creds.password);
106   s_deadlineMs = now + WIFI_ATTEMPT_TIMEOUT_MS;
107   s_phase = PHASE_ATTEMPT;
108 }
109
110 static void scheduleRetry(MonoMs now) {
111   const uint32_t delay = backoffNext(&s_backoff, esp_random());
112   portENTER_CRITICAL(&s_statsMux);
113   s_stats.lastBackoffMs = delay;
114   portEXIT_CRITICAL(&s_statsMux);
115   s_deadlineMs = now + delay;
116   s_phase = PHASE_WAIT;
117 }
118
119 static void onConnected(void) {
120   s_phase = PHASE_CONNECTED;
121   backoffReset(&s_backoff);
122   STATS_ADD(connects, 1);
123   setStatus(WIFI_STATE_CONNECTED);
124   logPrintf(LOG_LEVEL_INFO, "wifi: connected, %s (RSSI %ld dBm)", s_status.ip,
  >   (long)s_status.rssi);
125 }
```

startAttempt calls the adapter and sets a 15s deadline. scheduleRetry obtains a backoff delay and enters WAIT. onConnected resets the retry base, counts success and publishes CONNECTED.

The deadline represents when the task should reconsider the attempt. FreeRTOS does not know that deadline until the loop translates its periodic or notified wake into a now comparison.

React to credentials through their change counter

src/transports/wifi_mqtt.c / lines 126-153

```
126
127 // Credentials come from Configuration; a change takes effect at once.
128 static void followConfiguration(MonoMs now) {
129   uint32_t changes;
130   configurationGetWifi(&s_creds, &changes);
131   if (changes != s_credsChanges) {
132     s_credsChanges = changes;
133     teardown();
134     backoffReset(&s_backoff);
135     if (s_creds.ssid[0] != '\0' && !s_stopRequested) {
136       logPrintf(LOG_LEVEL_INFO, "wifi: joining \"%s\"", s_creds.ssid);
137       startAttempt(now);
138       setStatus(WIFI_STATE_CONNECTING);
139     } else {
140       s_phase = PHASE_IDLE;
141       setStatus(WIFI_STATE_UNCONFIGURED);
142     }
143   }
144 }
145
146 static void logStats(void) {
147   WifiMqttStats t;
148   wifiMqttGetStats(&t);
149   WifiStatus st;
150   wifiMqttGetStatus(&st);
151   logPrintf(LOG_LEVEL_INFO, "wifi: %s %s rssi %ld | joins %lu fails %lu backoff %lums",
  >   wifiStateName(st.state), st.ip,
152             (long)st.rssi, (unsigned long)t.connects, (unsigned long)t.connectFailures,
  >   (unsigned long)t.lastBackoffMs);
153 }
```

followConfiguration copies current credentials and compares their change count. A change tears down the previous link and either starts a new attempt or becomes idle. logStats prints status copies and counters.

Forgetting credentials therefore interrupts an old retry sequence; it does not wait until its former access-point attempt succeeds. Credentials are never included in the printed log format.

Handle power requests before the phase switch

src/transports/wifi_mqtt.c / lines 154-178

```
154
155 static void wifiTask(void* arg) {
156   (void)arg;
157   if (esp_task_wdt_add(NULL) != ESP_OK) logPrintf(LOG_LEVEL_ERROR, "wifi: task watchdog
  >   add failed");
158   MonoMs nextStatus = 0, nextStats = monoNowMs() + WIFI_STATS_LOG_MS;
159   bool stopped = false;
160   for (;;) {
161     ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(WIFI_WAKE_MS));
162     const MonoMs now = monoNowMs();
163
164     if (s_stopRequested != stopped) {  // WP9 power transitions
165       stopped = s_stopRequested;
166       s_credsChanges = UINT32_MAX;     // re-evaluated below: idle when stopped, rejoin
  >   when started
167       if (stopped) {
168         teardown();
169         s_radioOff = wifiLanRadioOff();  // the PowerCoord acknowledgement
170         logPrintf(LOG_LEVEL_INFO, "wifi: stopped on request, radio %s", s_radioOff ?
  >   "off" : "OFF FAILED");
171       } else {
172         s_radioOff = false;
173         if (!wifiLanRadioOn()) logPrintf(LOG_LEVEL_ERROR, "wifi: radio restart failed");
174         logPrintf(LOG_LEVEL_INFO, "wifi: started on request");
175       }
176     }
177     followConfiguration(now);
178
```

Each iteration requests a notification wait of up to 50ms. An earlier notification can end that wait; the actual interval between passes also includes work and scheduling delay. A changed stop request disconnects and asks the adapter for WIFI_OFF, retaining its acknowledgement. A restart returns to station mode and forces credentials to be reconsidered.

src/transports/wifi_mqtt.c / lines 179-182

```
179     switch (s_phase) {
180       case PHASE_ATTEMPT:
181         if (wifiLanConnected()) {
182           onConnected();
```

RequestStop is not completion. The caller reads wifiMqttRadioOff while this owner performs the mode change. That ordering is the same request/acknowledgement pattern used for acquisition quiescence.

Advance connection work without a long wait loop

src/transports/wifi_mqtt.c / lines 183-208

```
        } else if (now >= s_deadlineMs) {
          wifiLanDisconnect();
          STATS_ADD(connectFailures, 1);
          scheduleRetry(now);
          setStatus(WIFI_STATE_FAILED);
          logPrintf(LOG_LEVEL_WARN, "wifi: join failed, retry in %lu ms", (unsigned
  > long)s_stats.lastBackoffMs);
        }
        break;
      case PHASE_WAIT:
        if (now >= s_deadlineMs) {
          startAttempt(now);
          setStatus(WIFI_STATE_CONNECTING);
        }
        break;
      case PHASE_CONNECTED:
        if (!wifiLanConnected()) {
          logPrintf(LOG_LEVEL_WARN, "wifi: connection lost");
          teardown();
          scheduleRetry(now);  // from 1 s: the backoff was reset on connect
          setStatus(WIFI_STATE_CONNECTING);
          break;
        }
        if (now >= nextStatus) {  // RSSI, IP for State
          setStatus(WIFI_STATE_CONNECTED);
          nextStatus = now + WIFI_STATUS_REFRESH_MS;
        }
```

ATTEMPT checks connection or timeout; WAIT checks retry deadline; CONNECTED detects loss and refreshes status. Each pass returns to the bounded wait and marks progress.

Advance connection work without a long wait loop / continued

src/transports/wifi_mqtt.c / lines 209-220

```
209           break;
210         default: break;
211       }
212
213       if (now >= nextStats && currentState() != WIFI_STATE_UNCONFIGURED) {
214         logStats();
215         nextStats = now + WIFI_STATS_LOG_MS;
216       }
217       progressMark(TASK_ID_WIFI_MQTT);
218       esp_task_wdt_reset();
219     }
220 }
```

After a lost established connection, backoff starts from the initial base because onConnected reset it. A task can be making healthy progress while the network remains unavailable; connection success and task liveness are different observations.

Start the owner without waiting for an access point

src/transports/wifi_mqtt.c / lines 221-239

```
221
222 bool wifiMqttInit(void) {
223   if (s_task != NULL) return true;
224   backoffInit(&s_backoff, CFG_MQTT_BACKOFF_INITIAL_MS, CFG_MQTT_BACKOFF_MAX_MS,
  >   WIFI_JITTER_PCT);
225   if (!wifiLanBegin(WIFI_HOSTNAME)) {
226     logPrintf(LOG_LEVEL_ERROR, "wifi: station mode failed");
227     return false;
228   }
229   const TaskSpec* spec = cfgTaskSpec(TASK_ID_WIFI_MQTT);
230   if (spec->stackBytes != sizeof(s_stack)) {
231     logPrintf(LOG_LEVEL_WARN, "wifi: stack %u B, task table says %lu B",
  >   (unsigned)sizeof(s_stack),
232               (unsigned long)spec->stackBytes);
233   }
234   s_task = xTaskCreateStaticPinnedToCore(wifiTask, spec->name, sizeof(s_stack), NULL,
  >   spec->priority, s_stack, &s_tcb,
235                                          spec->core);
236   if (s_task == NULL) return false;
237   logPrintf(LOG_LEVEL_INFO, "wifi: started (core %u, prio %u)", (unsigned)spec->core,
  >   (unsigned)spec->priority);
238   return true;
239 }
```

Init configures backoff, starts station mode and creates the task. It does not wait for association or DHCP. Failure to set station mode or create the task returns false to the supervisor/startup path.

The task table supplies priority and core. The actual static stack size is checked against that table just as in the BLE owner, so configuration and allocation can be compared directly.

Request mode changes and inspect copies

src/transports/wifi_mqtt.c / lines 240-261

```
240
241 void wifiMqttRequestStop(void) {
242   s_stopRequested = true;
243   if (s_task != NULL) xTaskNotifyGive(s_task);
244 }
245
246 void wifiMqttRequestStart(void) {
247   s_stopRequested = false;
248   if (s_task != NULL) xTaskNotifyGive(s_task);
249 }
250
251 bool wifiMqttRadioOff(void) { return s_task == NULL || (s_stopRequested && s_radioOff);
  >   }
252
253 void wifiMqttGetStats(WifiMqttStats* out) {
254   portENTER_CRITICAL(&s_statsMux);
255   *out = s_stats;
256   portEXIT_CRITICAL(&s_statsMux);
257 }
258
259 void* wifiMqttTaskHandle(void) { return s_task; }
260
261 #endif  // FW_ENABLE_WIFI
```

Stop/Start update the requested state and wake the owner. RadioOff requires the stop request and completed acknowledgement, unless no task exists. Stats copies are guarded.

This finishes the network implementation present in the archive. No broker URL, topic encoder or publish operation is hidden behind the old MQTT name.

Read boot as a dependency order

Production setup() first applies safe GPIO defaults, then starts logging and the flash gate. It validates board assignments, creates the snapshot store, loads and publishes configuration, starts Safety, creates acquisition workers, starts the coordinator, then starts transports and supervision.

Each order choice has a reason. Safe outputs come before tasks can fail. The queue exists before a producer publishes. Configuration exists before consumers read their first settings. Safety exists before acquisition can submit its first alarm. The coordinator grants permission only after the worker and required HAL setup exist.

Original firmware - src/main.c, lines 60-63

```
void loop(void) {
  // Compatibility stub: the Arduino loop task blocks forever.
  vTaskDelay(portMAX_DELAY);
}
```

loop() is a compatibility stub that blocks. The application's ongoing work happens in its tasks, not in a large Arduino loop. Arduino already runs FreeRTOS in this environment; the application does not call vTaskStartScheduler itself.

The project defines setup and loop in C. hal/arduino_entry.h gives those symbols the C++ names expected by the Arduino runtime using GNU assembler-label declarations. That bridge solves a linkage detail. It does not turn the C bodies into C++ or create another scheduler.

Boot failures are logged while the relay begins de-energised. Review each failure path rather than assuming a single board-ready message proves every service started. For example, a radio failure should still leave local sensing and Safety available; a missing acquisition owner has a different consequence.

REMEMBER

Task creation may schedule the new task immediately. Build its dependencies before giving it an execution context.

Choose a target that answers one question

Project commands - run from the firmware project directory

```
pio run -e esp32dev
pio run -e test_light -t upload -t monitor
pio test -e native
```

The production environment builds src/main.c and enables the BLE and Wi-Fi paths. A sensor bring-up environment replaces that entry point with a small program under test/bringup/. It uses production modules while reducing the number of simultaneously active behaviours.

Start with one sensor when establishing its electrical and timing behaviour. For light, inspect millivolts, clipping and the response to a changed source. For DS18B20, inspect discovery, CRC failures and the delayed result. For HC-SR04, inspect raw microseconds and timeout cases before interpreting tank geometry.

The native target compiles domain code on your PC. It can test a stale boundary, polynomial calculation, latch policy or frame decoder using supplied inputs. It cannot prove an ADC pin is wired correctly, a relay actually moves, or a bus transaction meets its timing on the board.

Return to the preprocessing lesson (p. 35) if the feature flags need a reminder. In these environments, -D FW_ENABLE_BLE=1 defines a macro while compiling. The source filter then determines which entry-point file is built. Flags and filters answer different build questions.

Keep platform versions pinned while learning from this snapshot. Its platformio.ini selects espressif32@6.12.0 and an Arduino framework. A newer ESP-IDF major version may have changed API names or signatures. Upgrading and learning the existing architecture are separate tasks.

The serial capture helper in tools/serial_log.py records output for later inspection. Logs should include timestamps, result identity and relevant counters, not just repeated OK lines. A capture becomes useful evidence when you can match it to a specific build, input condition and expected behaviour.

COMMON PITFALL

Flashing an alarm-latency or stall-injection target deliberately changes behaviour. Read its build flags before treating it as production firmware.

A task stack is private storage, not a firewall

Each task has a stack for its call frames, automatic locals and saved context. Another task normally uses its own stack. That separation does not stop a bad C pointer from writing into the wrong address. Stack privacy is a programming model, not automatic memory isolation between ordinary tasks.

A large local array and deeply nested calls can exhaust a small stack. Moving an array to static storage changes where it lives but also makes it shared across calls. Do not make that move without checking who may call the function concurrently.

The stack high-water mark records the least free stack observed so far. It is evidence from executed paths, not proof that every future path fits. Exercise formatting, error handling, maximum message sizes and uncommon branches before reducing a stack reservation.

Always check the units returned by the actual ESP-IDF port and SDK version. The application's configured stack table uses bytes. Generic FreeRTOS documentation and newer ports may discuss words elsewhere. A fourfold sizing mistake can hide behind a perfectly reasonable-looking number.

COMMON PITFALL

More stack can postpone a crash without repairing the pointer error that caused it.

Debug the first invalid access

If a crash appears after a function returns, do not assume the returning function is the original cause. An earlier out-of-bounds write may have damaged the saved return address or another task's state. Check buffer bounds, pointer lifetimes and item sizes before increasing every stack.

A queue created for a large record but given a pointer to a smaller object will copy beyond that object. The queue can be internally thread-safe while the caller supplies invalid storage. Correct RTOS use still requires correct C memory use.

Reference: Espressif: ESP-IDF FreeRTOS SMP, v4.4.2 (S3)

Measure completed progress, then supervise it

The acquisition worker marks progress and resets its task-watchdog subscription after a completed pass. Safety does the same after its own work. If a function never returns, those updates stop. Resetting a watchdog at the top of a loop would be weaker evidence about the work that follows.

The Supervisor reads progress ages and diagnostic counters. It can warn that a service has stopped making expected progress. The task watchdog separately detects a watched task failing to reset within its configured interval. A log warning and a reset mechanism are different layers of observation and recovery.

An intentionally inactive acquisition worker leaves its watchdog subscription before waiting indefinitely without the lease. Otherwise the watchdog would interpret a legitimate idle mode as a stalled active worker. The active worker subscribes again when it acquires ownership.

A timestamp proves only the event used to update it. A worker spinning quickly and calling progressMark without completing useful work can look alive. Choose progress markers around meaningful completed operations, then use lateness, failure counts and retained data age to detect other failure types.

For a controlled stall test, the project has separate builds that stop acquisition or Safety after a delay. Observe the warning, output policy and reset behaviour with the intended setup. A host simulation of timeouts does not demonstrate the board watchdog's real reset path.

Logging itself must be bounded. A producer should enqueue a log record and let the logger own the slow output path. If the log queue fills, count or classify the loss rather than blocking a sensor operation indefinitely to preserve a diagnostic sentence.

Reference: Espressif: watchdogs, v4.4.7 (S3)

REMEMBER

Watchdog recovery does not establish that the last output state was physically safe during the delay before recovery.

Separate retained power from the switched branches

You have now read every hardware access path. Return to the board header and connect each named pin and rail to those drivers.

This is the hardware contract compiled into this source snapshot. It is not an automatic measurement of the currently attached board.

include/board_profile.h / lines 1-28

```
// Board profile: pins, rails and divider values for the ESP32-S3-WROOM-1 N16R8
// board (decision D16, supersedes D02).
// Source: Firmware Architecture Rev 3 §5 (Sensor <-> ESP32-S3 Signal Pinout,
// 22 Oct 2026, + Power Circuit Diagram). FROZEN CONTRACT - change only with a
// hardware-verified reason.
//
// Pure C (no Arduino headers) so domain code and native tests can use it.
// Anything not on the drawing stays BOARD_PIN_UNASSIGNED, and a pin on the
// drawing whose behaviour is not yet measured keeps its feature disabled
// (see boardProfileValidate()).
// Reserved on the N16R8: GPIO35-37 (octal PSRAM), GPIO19/20 (native USB),
// strapping pins GPIO0/3/45/46.
#pragma once
#include <stdbool.h>
#include <stdint.h>

#define BOARD_PIN_UNASSIGNED ((int8_t)-1)

typedef uint8_t Rail;
enum {
  RAIL_3V3 = 0,         // retained 3.3 V supply, kept on battery
  RAIL_5V_SWITCHED = 1  // external 5 V branches, lost on battery
};

// Bit mask used by the acquisition lease (allowed rails for a worker).
#define BOARD_RAIL_MASK_3V3 ((uint8_t)(1u << 0))
#define BOARD_RAIL_MASK_5V ((uint8_t)(1u << 1))

```

Rail and its masks describe which hardware remains available after external power is lost. The I2C pin names, clock and addresses select the shared bus you traced in the BME280 and SGP30 chapters.

Separate retained power from the switched branches / continued

include/board_profile.h / lines 29-34

```
29 // ---- Shared I2C bus (retained 3.3 V; 4.7 kOhm pull-ups to 3.3 V) ------------
30 #define BOARD_PIN_I2C_SDA ((int8_t)21)
31 #define BOARD_PIN_I2C_SCL ((int8_t)47)
32 #define BOARD_I2C_CLOCK_HZ ((uint32_t)400000)
33
34 // ---- BME280 (retained 3.3 V, I2C) ------------------------------------------
```

include/board_profile.h / lines 35-36

```
35 #define BOARD_BME280_I2C_ADDRESS ((uint8_t)0x76)      // SDO = GND
36 #define BOARD_BME280_I2C_ADDRESS_ALT ((uint8_t)0x77)  // SDO = VDDIO
```

The reserved-pin comments explain constraints for this N16R8 board. A different ESP32 variant needs a separately checked profile; its GPIO numbers and memory wiring cannot be inferred from this one.

Recover the physical meaning of raw ADC numbers

include/board_profile.h / lines 37-61

```
37
38 // ---- SGP30 module (retained 3.3 V, I2C, TVOC / CO2eq) -----------------------
39 #define BOARD_SGP30_I2C_ADDRESS ((uint8_t)0x58)  // fixed
40
41 // ---- DS18B20 (retained 3.3 V, 1-Wire, external supply, 4.7 kOhm pull-up) ----
42 #define BOARD_PIN_ONE_WIRE ((int8_t)14)
43
44 // ---- DFR0026 ambient light (retained 3.3 V, ADC1_CH0) -----------------------
45 // S -> 10 kOhm series -> GPIO1, 1 uF to GND (tau ~10 ms).
46 #define BOARD_PIN_LIGHT_ADC ((int8_t)1)
47
48 // ---- SEN0244 TDS (filtered external 5 V, ADC1_CH5, 0-2.3 V nominal) ---------
49 // A -> GPIO6, 100 nF to GND.
50 #define BOARD_PIN_TDS_ADC ((int8_t)6)
51
52 // ---- pH-4502C (filtered external 5 V, ADC1_CH6, divider 10k top / 15k bottom)
53 // PO -> 10k/15k -> GPIO7 + 1 uF to GND. DO and TO not connected.
54 #define BOARD_PIN_PH_ADC ((int8_t)7)
55 #define BOARD_PH_DIVIDER_TOP_OHM ((uint32_t)10000)
56 #define BOARD_PH_DIVIDER_BOTTOM_OHM ((uint32_t)15000)
57
58 // ---- HC-SR04 (external 5 V, ECHO through 1k/2k divider) ---------------------
59 #define BOARD_PIN_HCSR04_TRIG ((int8_t)17)
60 #define BOARD_PIN_HCSR04_ECHO ((int8_t)18)
61
```

The analog pins and divider values connect ADC millivolts to module voltage. For the pH divider, 10k above and 15k below give pin = module times 15/25. Acquisition uses the same constants when rebuilding calibration inputs.

include/board_profile.h / lines 62-67

```
62 // ---- External-power sense (ADC1_CH7; 220 Ohm over 2 x 220 Ohm from ext. 5 V) -
63 // ~3.33 V at 5.00 V. Used as a digital edge input; the ADC can check levels.
64 #define BOARD_PIN_EXTERNAL_POWER_SENSE ((int8_t)8)
65 #define BOARD_POWER_SENSE_DIVIDER_TOP_OHM ((uint32_t)220)
66 #define BOARD_POWER_SENSE_DIVIDER_BOTTOM_OHM ((uint32_t)440)
67 // Measured 25 Oct 2026 with test_power_sense (breadboard, 6 losses / 5 power-ons,
```

A correct software formula can still describe the wrong circuit if the assembled resistor values differ. Keep the hardware profile and the actual board revision together when reviewing measurements.

Read measured sense behaviour as an input to policy

include/board_profile.h / lines 68-89

```
68 // README "External-power sense bring-up"): LOW 0-53 mV, HIGH >= 3.05 V (ADC
69 // clipped; <= 3.6 V by calculation only, 3.17-3.50 V for a 4.75-5.25 V supply).
70 // Switches at ~1.65 V falling / ~1.95 V rising. Power loss chatters 479-928
71 // edges over 5-14 ms; power-on 3-16 edges over < 0.2 ms. First falling edge
72 // 59-220 ms after the rail starts to drop (+ rail already ~2.5 V).
73 // Requirements on WP9: the ISR disables itself on the first edge and the power
74 // task re-reads the level after BOARD_POWER_SENSE_QUALIFY_MS before acting and
75 // re-arming; the rail epoch also rejects 5 V readings that ended just before it.
76 #define BOARD_POWER_SENSE_VERIFIED true
77 #define BOARD_POWER_SENSE_QUALIFY_MS ((uint32_t)20)
78
79 // ---- Relay 1 (SRD-05VDC, coil on external 5 V) ------------------------------
80 // GPIO15 -> 220 Ohm -> IRLZ44N gate (low side), 10 kOhm gate pull-down,
81 // 1N5822 flyback. High = coil energised. Contact use and load still open (§12).
82 #define BOARD_PIN_RELAY1 ((int8_t)15)
83 #define BOARD_RELAY1_ACTIVE_LOW false
84
85 // ---- Open hardware gates (§20) - NOT on the drawing --------------------------
86 #define BOARD_PIN_RAIL_5V_FEEDBACK BOARD_PIN_UNASSIGNED
87
88 static inline bool boardPinAssigned(int8_t pin) { return pin != BOARD_PIN_UNASSIGNED; }
89
```

The sense notes record delayed, chattering edges observed on that prototype. The 20ms qualification is why the ISR disables itself at the first edge and the task later rereads the level.

BOARD_POWER_SENSE_VERIFIED is a compiled declaration that this commissioning gate was completed for the referenced setup. It does not measure anything at boot. Rail validity and epoch handling remain necessary after that gate is true.

Understand exactly what the board gate proves

include/board_profile.h / lines 90-105

```
90  typedef struct BoardValidation {
91    bool powerSenseAssigned;
92    bool railFeedbackAssigned;
93    bool relayAssigned;
94    bool productionReady;  // false => report a non-production build status
95  } BoardValidation;
96
97  static inline BoardValidation boardProfileValidate(void) {
98    BoardValidation v;
99    v.powerSenseAssigned = boardPinAssigned(BOARD_PIN_EXTERNAL_POWER_SENSE);
100   v.railFeedbackAssigned = boardPinAssigned(BOARD_PIN_RAIL_5V_FEEDBACK);
101   v.relayAssigned = boardPinAssigned(BOARD_PIN_RELAY1);
102   // Minimum gate for power modes: the sense pin exists AND has been measured.
103   v.productionReady = v.powerSenseAssigned && BOARD_POWER_SENSE_VERIFIED;
104   return v;
105 }
```

boardPinAssigned only compares against the unassigned sentinel. boardProfileValidate reports assignments, then productionReady requires assigned sense plus the verified constant. Relay assignment is reported separately; it is not included in that final expression.

Read the expression rather than giving the name productionReady a broader meaning. This helper does not test wiring, contact behaviour, ADC accuracy or every failure path. It returns the narrow software gate written here.

Pin the framework and board assumptions

The source is only one part of a reproducible firmware image. PlatformIO selects framework, board, libraries, feature flags and source files.

Read the environment you intend to build before assuming which setup function or radio code will enter the image.

platformio.ini / lines 1-28

```
; Hydroponics Monitoring System - new firmware (ESP32-S3-WROOM-1 N16R8)
; Env names kept from the WROOM-32D era so existing commands still work.
; Architecture: "Firmware Architecture.pdf" (Rev 3). Team guide: README.md.
; Language: all project sources are C (gnu11 on the host). The Arduino-ESP32
; framework stays the platform; its C HAL (esp32-hal-*) and FreeRTOS are used.
;
;   pio run -e esp32dev            production image (src/main.c)
;   pio run -e test_bme280 -t upload -t monitor
;                                  one sensor driver + its bring-up main
;   pio test -e native             host unit tests for src/domain (needs gcc)
;
; Versions are pinned to the ESP-Demo baseline (§13). Do not bump them
; without a compile + API check and a note in README.md.

[platformio]
default_envs = esp32dev

; ------------------------------------------------------------------- shared ----
[esp32_base]
platform = espressif32@6.12.0
framework = arduino
board = esp32-s3-devkitc-1
; N16R8: 16 MB QIO flash + 8 MB octal PSRAM (GPIO35-37 reserved, as board_profile.h).
board_build.arduino.memory_type = qio_opi
board_build.flash_size = 16MB
board_upload.flash_size = 16MB
board_build.partitions = default_16MB.csv
monitor_speed = 115200
```

The base selects espressif32 6.12.0, Arduino and the S3 board with 16MB flash and octal PSRAM settings. NimBLE-Arduino is pinned to 2.5.1. ArduinoJson is declared as a dependency but project JSON uses its own pure codec.

platformio.ini / lines 29-34

```
; NimBLE-Arduino (C++) is used through the C shim src/transports/ble_gatt.cpp.
; ArduinoJson is not used by project code (JSON is built in domain/protocol_codec).
lib_deps =
    h2zero/NimBLE-Arduino@2.5.1
    bblanchon/ArduinoJson@6.21.5
build_flags =
```

Pin the framework and board assumptions / continued

platformio.ini / lines 35-43

```
    -I src
    -D CORE_DEBUG_LEVEL=1
    -D BOARD_HAS_PSRAM
test_ignore =
    bringup
    native

; ------------------------------------------------------------ production ----
; Production images run the BLE transport (NimBLE) and the Wi-Fi station.
```

The old environment name esp32dev does not change the board setting. The opening all-C comment also predates the two C++ adapters you read. Distinguish build directives from descriptive comments that may lag the source.

Build production and deliberately altered test images

platformio.ini / lines 44-71

```
; Bring-up envs define neither, so they never link the radios.
[production]
build_flags =
    ${esp32_base.build_flags}
    -D FW_ENABLE_BLE=1
    -D FW_ENABLE_WIFI=1

[env:esp32dev]
extends = esp32_base
build_flags = ${production.build_flags}
build_src_filter = +<*>

; Acceptance test (§19 "Stall injection"): production image whose task blocks
; forever after 60 s. Expect: Supervisor WARN at ~2 s (acq) / ~1 s (safety),
; lamp OFF within 3 s (acq only), task-watchdog reset at ~5 s, then
; "reset reason: TASK_WDT" on the next boot. Never deploy these.
[env:stall_acq]
extends = env:esp32dev
build_flags =
    ${production.build_flags}
    -D FW_INJECT_STALL_ACQ=60000

[env:stall_safety]
extends = env:esp32dev
build_flags =
    ${production.build_flags}
    -D FW_INJECT_STALL_SAFETY=60000

```

The production flags enable BLE and Wi-Fi. Fault-injection environments add stall, torn-frame or latency-test flags. These change the running program and its expected behaviour.

Build production and deliberately altered test images / continued

platformio.ini / lines 72-85

```
; Acceptance test (cut BLE frame): every 5th Telemetry frame stops after its
; first chunk, as if NimBLE buffers stayed full. Expect "ble: frame cut ...
; link dropped to resync", a clean reconnect and no decode error in the app.
; Never deploy.
[env:ble_tear]
extends = env:esp32dev
build_flags =
    ${production.build_flags}
    -D FW_INJECT_BLE_TEAR=5

; Acceptance test (§19 "Temperature alarm"): a test rule trips on every valid
; BME280 sample (4 per second, every frame phase) and de-energises the relay.
; Safety logs submit->action (us) and sample->action (ms) every 20 TRIPs.
; Target: sample -> Safety action <= 10 ms. Never deploy.
```

platformio.ini / lines 86-87

```
[env:alarm_latency]
extends = env:esp32dev
```

A stall image is intended to expose progress warnings and eventual watchdog action. The comments state expected timing; they are not fresh test results from your board. Keep normal deployment and these temporary experiments identifiable by environment.

Replace the entry point for one-device bring-up

platformio.ini / lines 88-113

```
build_flags =
    ${production.build_flags}
    -D FW_ALARM_LATENCY_TEST=1

; ------------------------------------------------------- sensor bring-up ----
; Each env links every production module (drivers, hal, domain, ...) but
; replaces src/main.c with a single test/bringup/<sensor>_main.c (R9).
[bringup_base]
extends = esp32_base
build_flags =
    ${esp32_base.build_flags}
    -I test/bringup
    -D FW_BRINGUP=1

[env:test_bme280]
extends = bringup_base
build_src_filter = +<*> -<main.c> +<../test/bringup/bme280_main.c>

[env:test_ds18b20]
extends = bringup_base
build_src_filter = +<*> -<main.c> +<../test/bringup/ds18b20_main.c>

[env:test_light]
extends = bringup_base
build_src_filter = +<*> -<main.c> +<../test/bringup/light_main.c>

```

platformio.ini / lines 114-132

```
; Light level -> relay module (GPIO15) -> grow lamp, hysteresis + dwell.
[env:test_lamp]
extends = bringup_base
build_src_filter = +<*> -<main.c> +<../test/bringup/lamp_main.c>

[env:test_hcsr04]
extends = bringup_base
build_src_filter = +<*> -<main.c> +<../test/bringup/hcsr04_main.c>

[env:test_ph]
extends = bringup_base
build_src_filter = +<*> -<main.c> +<../test/bringup/ph_main.c>

[env:test_tds]
extends = bringup_base
build_src_filter = +<*> -<main.c> +<../test/bringup/tds_main.c>

[env:test_sgp30]
extends = bringup_base
```

Replace the entry point for one-device bring-up / continued

platformio.ini / lines 133-139

```
build_src_filter = +<*> -<main.c> +<../test/bringup/sgp30_main.c>

; Shared I2C bus: BME280 + SGP30 together, BME280 humidity -> SGP30 compensation.
[env:test_climate]
extends = bringup_base
build_src_filter = +<*> -<main.c> +<../test/bringup/climate_main.c>

```

Each bring-up environment excludes main.c and includes one test main while reusing the production modules. No radio feature flags are defined in the base bring-up configuration.

platformio.ini / lines 140-144

```
; Water probes: DS18B20 + TDS together, DS18B20 water temperature -> TDS compensation.
[env:test_water]
extends = bringup_base
build_src_filter = +<*> -<main.c> +<../test/bringup/water_main.c>

```

The source filter is the reason two setup functions do not collide. Adding a test file without excluding the production entry would create duplicate entry definitions instead of an isolated experiment.

Compile pure behaviour on the host

platformio.ini / lines 145-165

```
; External-power sense on GPIO8: edges (ISR, us) + ADC trace (1 kHz) on plug/unplug.
; Commissioning gate for BOARD_POWER_SENSE_VERIFIED. DTR/RTS released so that
; reopening the monitor after an unplug does not reset the board.
[env:test_power_sense]
extends = bringup_base
build_src_filter = +<*> -<main.c> +<../test/bringup/power_sense_main.c>
monitor_dtr = 0
monitor_rts = 0

; ------------------------------------------------------------ host unit tests ----
; Only src/domain is compiled: it must stay free of Arduino/FreeRTOS headers.
[env:native]
platform = native
build_flags =
    -I src
    -I include
    -std=gnu11
build_src_filter = -<*> +<domain/>
test_build_src = yes
test_filter = native/*
test_ignore = bringup
```

The native environment admits only domain sources, adds include paths and selects gnu11. Native tests do not acquire GPIOs, run the dual-core scheduler or exercise NimBLE.

The power-sense environment also releases monitor DTR/RTS to avoid confusing a serial-open reset with a power event. Build configuration participates in the experiment; it is not merely an editor preference.

Assign the symbol names expected by the framework

The Arduino core calls C++ entry symbols, while this project defines setup and loop in C. This GNU declaration bridges that particular build convention.

It is different from the extern "C" boundary in the radio adapters.

src/hal/arduino_entry.h / lines 1-9

```
1 // Arduino entry points defined in C.
2 // The Arduino-ESP32 core (cores/esp32/main.cpp) calls setup() and loop() with
3 // C++ linkage, i.e. the mangled symbols _Z5setupv and _Z4loopv. These
4 // declarations give the C definitions those symbol names. Include this header
5 // in every file that defines setup()/loop() (src/main.c, test/bringup/*.c).
6 #pragma once
7
8 void setup(void) __asm__("_Z5setupv");
9 void loop(void) __asm__("_Z4loopv");
```

The __asm__ label gives the C definition the linker symbol that the Arduino C++ caller expects. _Z5setupv and _Z4loopv encode those function names and void parameter lists in this toolchain's convention.

This is a platform-specific integration choice, not portable ISO C syntax. A pure ESP-IDF application would normally enter through app_main and would not copy this header simply because it also uses FreeRTOS.

Establish safe outputs and early diagnostics first

At last, read the production entry with every service already familiar. setup establishes dependencies; it does not become the sensor loop.

Trace what each initialiser makes available before the next one starts.

src/main.c / lines 1-28

```
// Production entry point - bootstrap ONLY (Firmware Architecture §5).
// No sensing, reconnect or policy work belongs here or in loop().
// Order (RTOS Architecture §5): safe outputs, logger, flash gate, board gate,
// queues, Configuration (publishes the applied config that Safety and
// acquisition read on their first pass), Safety BEFORE acquisition (the
// first alarm event always has a receiver), PowerCoordinator (grants the
// acquisition lease), the BLE transport, Wi-Fi, then Supervisor. A failed step is
>   logged and boot continues; the relay is
// already de-energised and only Safety can change it.
#include <Arduino.h>

#include "board_profile.h"
#include "diagnostics/logger.h"
#include "hal/arduino_entry.h"
#include "hal/gpio_safe.h"
#include "services/acquisition.h"
#include "services/configuration.h"
#include "services/flash_gate.h"
#include "services/power_coordinator.h"
#include "services/safety.h"
#include "services/snapshot_store.h"
#include "services/supervisor.h"
#include "transports/ble_transport.h"
#include "transports/wifi_mqtt.h"

void setup(void) {
  // 1. Safe output defaults before anything else.
  gpioApplySafeDefaults();

```

gpioApplySafeDefaults executes before task creation or radio startup. Logger and flash gate follow, so later initialisation has diagnostics and a shared flash-operation exclusion boundary.

Establish safe outputs and early diagnostics first / continued

src/main.c / lines 29-33

```
29   // 2. Serial console: UART0 at 115200 (CONFIG_ESP_CONSOLE_UART_BAUDRATE).
30   logInit();
31   logPrintf(LOG_LEVEL_INFO, "hydroponics firmware - stage 6 (BLE app link, calibration,
 >   low-power mode)");
32   if (!flashGateInit()) logPrintf(LOG_LEVEL_ERROR, "flash gate init failed - pings
 >   unguarded");
33
```

Failure is logged and startup proceeds in this source. An error string is an observation, not an alternate implementation. For example, a failed flash-gate init cannot make that gate protect later operations.

Publish initial state before starting its readers

src/main.c / lines 34-46

```
34  // 3. Board gate: pins assigned and power sense measured.
35  BoardValidation board = boardProfileValidate();
36  logPrintf(LOG_LEVEL_INFO, "board gate: power sense %s, relay %s -> %s",
37            board.powerSenseAssigned ? "yes" : "NO",
38            board.relayAssigned ? "yes" : "NO",
39            board.productionReady ? "pass" : "FAIL");
40
41  // 4. Static queues.
42  if (!snapshotStoreInit()) logPrintf(LOG_LEVEL_ERROR, "snapshot store init failed");
43
44  // 5. Configuration: loads NVS and publishes before the tasks below start.
45  // On failure the compiled defaults apply (all sensors on).
46  if (!configurationInit()) logPrintf(LOG_LEVEL_ERROR, "configuration init failed -
 >  compiled defaults");
```

The board gate is reported, Snapshot storage is created, then Configuration loads or supplies defaults and publishes applied state. Safety and acquisition therefore have an intended initial configuration to read.

The order is part of the architecture: a task may run immediately after it is created. Do not assume every subsequent line of setup finishes before the new task executes.

Start owners in dependency order and block the compatibility loop

src/main.c / lines 47-63

```
47
48    // 6-10. Tasks. Both acquisition workers start without the
 >     lease;
49    // PowerCoordinator grants the first one from the sense
 >    level (WP9).
50    if (!safetyInit()) logPrintf(LOG_LEVEL_ERROR, "safety init
 >     failed - relay stays OFF");
51    if (!acquisitionInit()) logPrintf(LOG_LEVEL_ERROR,
 >    "acquisition init failed");
52    if (!powerCoordinatorInit()) logPrintf(LOG_LEVEL_ERROR,
 >    "power coordinator init failed - normal mode, no
 >    transitions");
53    // Sensing and Safety run without it: a failed radio only
 >    costs the app link.
54    if (!bleTransportInit()) logPrintf(LOG_LEVEL_ERROR, "BLE
 >    transport init failed - no app link");
55    // Never waits for an access point; without credentials it
 >     stays idle.
56    if (!wifiMqttInit()) logPrintf(LOG_LEVEL_ERROR, "Wi-Fi
 >    init failed - Bluetooth only");
57    if (!supervisorInit()) logPrintf(LOG_LEVEL_ERROR,
 >    "supervisor init failed");
58  }
59
60  void loop(void) {
61    // Compatibility stub: the Arduino loop task blocks
 >    forever.
62    vTaskDelay(portMAX_DELAY);
63  }
```

Safety starts before acquisition so the alarm receiver exists before observations produce events. Acquisition workers are created before PowerCoord grants a lease. Radios start after sensing services; Supervisor comes last.

Notice the power-initialisation error text promising normal mode. Workers actually require a lease in the acquisition implementation you read; inspect which failure point grants one before relying on that message. The error text alone cannot establish continued sampling. loop blocks indefinitely because owner tasks now hold runtime work.

COMMON PITFALL

A fallback described in a log message must be supported by executable state transitions.

Use compiler help for printf-style calls

Logging must not turn a high-priority acquisition path into a serial-output wait. This interface makes formatting bounded and queueing nonblocking after logger startup.

The early-boot fallback is a separate, blocking path; the source makes that exception visible.

src/diagnostics/logger.h / lines 1-22

```
// Logger: bounded queue to the serial console, drop-and-count, never blocks
// callers. Work package WP10 (Stage 4). Architecture §9, §16.
//
// logPrintf() formats into a fixed record (LOG_TEXT_LEN, longer text is cut)
// and sends it with a zero timeout. The SerialLogger task (core 1, priority
// 1) prints it, so it only runs while acquisition is Blocked. Before the task
// exists (and in bring-up builds, which never call logInit()) logPrintf()
// prints directly. Never call it from an ISR.
#pragma once
#include <stdint.h>

#define LOG_TEXT_LEN 128

typedef unsigned char LogLevel;
enum { LOG_LEVEL_ERROR = 0, LOG_LEVEL_WARN, LOG_LEVEL_INFO, LOG_LEVEL_DEBUG };

void logInit(void);  // creates the queue and the SerialLogger task
void logPrintf(LogLevel level, const char* fmt, ...)
    __attribute__((format(printf, 2, 3)));
uint32_t logDroppedCount(void);  // records lost to a full queue since boot
uint32_t logQueueDepth(void);    // records waiting to be printed (0 before logInit)
void* logTaskHandle(void);       // SerialLogger TaskHandle_t, NULL before logInit()
```

The format attribute asks GCC to check the format and argument types for logPrintf. The numbered attribute arguments identify fmt and the first variadic argument. It catches many mismatches before they become corrupted output.

The public dropped count and queue depth make overload visible. The ISR restriction still applies: a normal task logging helper is not made ISR-safe by a zero queue timeout.

Reserve records, queue storage and a separate consumer stack

A complete log record is copied into a bounded queue, then one low-priority consumer writes the console.

This is an event queue rather than a latest-state mailbox: every retained record represents a distinct log message.

src/diagnostics/logger.c / lines 1-27

```
#include "diagnostics/logger.h"

#include <freertos/FreeRTOS.h>
#include <freertos/queue.h>
#include <freertos/task.h>
#include <stdarg.h>
#include <stdio.h>

#include "app_config.h"
#include "diagnostics/progress.h"

typedef struct LogRecord {
  LogLevel level;
  char text[LOG_TEXT_LEN];
} LogRecord;

static StaticQueue_t s_queueControl;
static uint8_t s_queueStorage[CFG_LOG_QUEUE_LEN * sizeof(LogRecord)];
static QueueHandle_t s_queue = NULL;

static StaticTask_t s_tcb;
static StackType_t s_stack[2 * 1024];  // cfgTaskSpec(TASK_ID_SERIAL_LOGGER)->stackBytes
static TaskHandle_t s_task = NULL;

static portMUX_TYPE s_dropMux = portMUX_INITIALIZER_UNLOCKED;
static uint32_t s_dropped = 0;

```

LogRecord owns a level and fixed text array. Static queue storage reserves capacity for complete records. tagOf guards the index before returning a string literal.

src/diagnostics/logger.c / lines 28-31

```
static const char* tagOf(LogLevel level) {
  static const char* const kTags[] = {"E", "W", "I", "D"};
  return level < sizeof(kTags) / sizeof(kTags[0]) ? kTags[level] : "?";
}
```

Formatting and queueing copy text, so a caller's local numeric variables need not stay alive until the UART prints. The consumer does not retain a pointer into the caller's stack.

Block while there is nothing to print

src/diagnostics/logger.c / lines 32-51

```
32
33 static void serialLoggerTask(void* arg) {
34   (void)arg;
35   static LogRecord rec;  // single consumer: static keeps it off the 2 KiB stack
36   uint32_t reportedDrops = 0;
37   for (;;) {
38     if (xQueueReceive(s_queue, &rec, portMAX_DELAY) != pdTRUE) continue;
39     fputs("[", stdout);
40     fputs(tagOf(rec.level), stdout);
41     fputs("] ", stdout);
42     fputs(rec.text, stdout);
43     fputs("\n", stdout);
44     const uint32_t dropped = logDroppedCount();
45     if (dropped != reportedDrops) {  // say so in the log itself, once per change
46       printf("[W] logger: %lu records dropped (queue full)\n", (unsigned long)dropped);
47       reportedDrops = dropped;
48     }
49     progressMark(TASK_ID_SERIAL_LOGGER);
50   }
51 }
```

The consumer waits indefinitely for a record, then writes its text and reports any changed drop count. It marks progress after that work, rather than waking merely to satisfy a timer.

A low-priority logger often runs when higher-priority tasks block, but the header's phrase only while acquisition is Blocked is too narrow: the scheduler can also choose it when acquisition is suspended, not ready on this core or otherwise ineligible. Use scheduler rules rather than treating that comment as an extra rule.

Create the destination before selecting queued logging

src/diagnostics/logger.c / lines 52-64

```
52
53 uint32_t logQueueDepth(void) { return s_queue != NULL ?
 >   (uint32_t)uxQueueMessagesWaiting(s_queue) : 0; }
54
55 void* logTaskHandle(void) { return s_task; }
56
57 void logInit(void) {
58   if (s_task != NULL) return;
59   s_queue = xQueueCreateStatic(CFG_LOG_QUEUE_LEN, sizeof(LogRecord), s_queueStorage,
 >   &s_queueControl);
60   if (s_queue == NULL) return;  // logPrintf keeps printing directly
61   const TaskSpec* spec = cfgTaskSpec(TASK_ID_SERIAL_LOGGER);
62   s_task = xTaskCreateStaticPinnedToCore(serialLoggerTask, spec->name, sizeof(s_stack),
 >   NULL,
63                                          spec->priority, s_stack, &s_tcb, spec->core);
64 }
```

QueueDepth and TaskHandle are observation helpers. Init creates the queue, then the task using its table entry. logPrintf chooses the queued path only when s_task is non-null.

A queue created without a successful consumer task does not cause all future calls to disappear into unread storage: the source's task-handle test keeps the direct fallback active.

Format within a fixed record, then copy or count loss

src/diagnostics/logger.c / lines 65-90

```
65
66 void logPrintf(LogLevel level, const char* fmt, ...) {
67   LogRecord rec;
68   rec.level = level;
69   va_list args;
70   va_start(args, fmt);
71   vsnprintf(rec.text, sizeof(rec.text), fmt, args);
72   va_end(args);
73
74   if (s_task == NULL) {  // early boot or bring-up: direct, blocking write
75     printf("[%s] %s\n", tagOf(level), rec.text);
76     return;
77   }
78   if (xQueueSend(s_queue, &rec, 0) != pdTRUE) {
79     portENTER_CRITICAL(&s_dropMux);
80     ++s_dropped;
81     portEXIT_CRITICAL(&s_dropMux);
82   }
83 }
84
85 uint32_t logDroppedCount(void) {
86   portENTER_CRITICAL(&s_dropMux);
87   const uint32_t n = s_dropped;
88   portEXIT_CRITICAL(&s_dropMux);
89   return n;
90 }
```

va_start and va_end delimit traversal of variadic arguments; vsnprintf bounds the local text. Before the consumer exists, printf writes directly. Afterwards xQueueSend with zero wait either retains the record or increments s_dropped.

Formatting still costs CPU time, even though queue insertion never waits. A long message can be truncated by the fixed text capacity, independently of queue-full loss. These are different diagnostic limits.

Report completion through one small function

A task that wakes is not necessarily completing its responsibility. Progress markers give Supervisor a timestamp after useful work.

The interface stores one timestamp per configured task identity.

src/diagnostics/progress.h / lines 1-8

```
// Per-task progress timestamps for the progress watchdog.
// Work package WP10 (Stage 4). Architecture §16.
#pragma once
#include "app_config.h"
#include "domain/types.h"

void progressMark(TaskId task);  // call after meaningful work
MonoMs progressLastMs(TaskId task);
```

progressMark records now for an id. progressLastMs returns a copy. Neither resets the hardware or task watchdog for another owner.

Choose the placement of the mark so the timestamp means something. Marking at the top of an endless retry loop could conceal that the intended operation never completes.

Read and write each timestamp under the same guard

A MonoMs is 64 bits on this 32-bit target. Another core must not see half of an old timestamp and half of a new one.

This is a short shared-memory copy, so the implementation uses a short critical section.

src/diagnostics/progress.c / lines 1-28

```
#include "diagnostics/progress.h"

#include <freertos/FreeRTOS.h>

#include "hal/clock.h"

// 64-bit stamps are written as two 32-bit halves: a reader on the other core
// could see a torn value without the lock (§11).
// The acquisition worker without the lease marks nothing; Supervisor checks
// only the lease holder (acquisitionActiveTaskId(), WP9).
static portMUX_TYPE s_mux = portMUX_INITIALIZER_UNLOCKED;
static MonoMs s_lastMs[TASK_ID_COUNT];

void progressMark(TaskId task) {
  if (task >= TASK_ID_COUNT) return;
  const MonoMs now = monoNowMs();
  portENTER_CRITICAL(&s_mux);
  s_lastMs[task] = now;
  portEXIT_CRITICAL(&s_mux);
}

MonoMs progressLastMs(TaskId task) {
  if (task >= TASK_ID_COUNT) return 0;
  portENTER_CRITICAL(&s_mux);
  const MonoMs t = s_lastMs[task];
  portEXIT_CRITICAL(&s_mux);
  return t;
}
```

Both functions check the TaskId range. Mark obtains the clock outside the lock, then assigns the timestamp under it. The reader copies under that same guard and returns after releasing it.

The inactive acquisition worker owes no sampling progress because it lacks the lease. Supervisor selects the active worker before judging timestamp age. Shared storage safety and correct monitoring policy are both needed.

Keep warning thresholds separate from watchdog action

Supervisor observes timing, resource headroom and loss counters. Its deadlines describe this application's expected progress rather than generic FreeRTOS requirements.

A warning can appear while the task is still scheduled; it means meaningful work is overdue.

src/services/supervisor.h / lines 1-28

```
// Supervisor: progress deadlines, counters, stack/heap health, every 1 s.
// Work package WP10 (Stage 4). Architecture §16, §18.
//
// Core 0, priority 1: it only observes and reports. It never resets the task
// watchdog for another task (AcqNormal and Safety reset it for themselves).
//   every 1 s   progress age per watched task (warn once per stall episode,
//               "progressing again" on recovery); loss/lateness counters
//               (frame/BME280 late and skipped, guard timeouts, alarm detail
//               overflow, log drops): one WARN line with the deltas
//   every 10 s  stack high-water marks and internal heap: WARN when low
//   every 60 s  the same as an INFO report
//   every 5 s   status line: readings, lamp, counters, "health OK" or
//               "DEGRADED <first cause>" for the last 5 s
//   boot        reset reason (a TASK_WDT here means a watched task stalled)
#pragma once
#include <stdbool.h>

#define SUPERVISOR_STATUS_EVERY_S 5
#define SUPERVISOR_MEMORY_EVERY_S 10
#define SUPERVISOR_MEMORY_REPORT_EVERY_S 60

// Progress deadlines. Safety passes every 100 ms, AcqNormal at least every
// 250 ms; the task watchdog (5 s) fires well after these warnings.
#define SUPERVISOR_SAFETY_DEADLINE_MS 1000u
#define SUPERVISOR_ACQ_DEADLINE_MS 2000u
#define SUPERVISOR_LOGGER_DEADLINE_MS 2000u
// Configuration wakes every 500 ms and marks progress while it waits for a
// flash-gate slot or for acquisition to apply a change (<= 3 s).
```

Safety has a 1000ms progress deadline; acquisition has 2000ms. A boot grace period avoids judging startup as a runtime stall. Logger is judged only when records are waiting.

Keep warning thresholds separate from watchdog action / continued

src/services/supervisor.h / lines 29-37

```
29 #define SUPERVISOR_CONFIG_DEADLINE_MS 3000u
30 // BleTransport wakes every 50 ms; one frame send retries for at most ~200 ms per chunk.
31 #define SUPERVISOR_BLE_DEADLINE_MS 2000u
32 // WifiMqtt wakes every 50 ms; every socket call is non-blocking.
33 #define SUPERVISOR_WIFI_DEADLINE_MS 2000u
34 // PowerCoord wakes at least every 100 ms (WP9).
35 #define SUPERVISOR_POWER_DEADLINE_MS 1000u
36 #define SUPERVISOR_BOOT_GRACE_MS 5000u
37
```

src/services/supervisor.h / lines 38-41

```
38 #define SUPERVISOR_STACK_WARN_BYTES 512u  // §16: keep >= 25 % headroom; warn below this
39 #define SUPERVISOR_HEAP_WARN_BYTES 20000u // internal heap minimum ever
40
41 bool supervisorInit(void);
```

The stack warning is a fixed 512-byte threshold here. Although a comment mentions percentage headroom, this code does not compute a per-task 25-percent threshold. Read the actual check in the implementation before interpreting a clean report.

Define the watched responsibilities

This task combines safe copies into diagnostic judgements. It never substitutes its own watchdog reset for a stalled owner's reset.

Read the monitoring conditions and their exceptions before trusting a health label.

src/services/supervisor.c / lines 1-27

```
#include "services/supervisor.h"

#include <esp_heap_caps.h>
#include <esp_system.h>
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>
#include <stdio.h>
#include <string.h>

#include "app_config.h"
#include "diagnostics/logger.h"
#include "diagnostics/progress.h"
#include "domain/reading.h"
#include "hal/clock.h"
#include "services/acquisition.h"
#include "services/configuration.h"
#include "services/power_coordinator.h"
#include "services/safety.h"
#include "services/snapshot_store.h"
#include "transports/ble_transport.h"
#include "transports/wifi_mqtt.h"

static StaticTask_t s_tcb;
static StackType_t s_stack[3 * 1024];  // cfgTaskSpec(TASK_ID_SUPERVISOR)->stackBytes
static TaskHandle_t s_task = NULL;
static Snapshot s_snap;  // static: 584 B does not belong on a 3 KiB stack

```

The Watched table associates a task identity, label, progress deadline and busy-only policy. s_degraded retains the first cause in one reporting period, so a status line has a bounded explanation.

src/services/supervisor.c / lines 28-36

```
// ---- Progress deadlines (§18) ----------------------------------------------
typedef struct Watched {
  TaskId id;
  const char* name;
  uint32_t deadlineMs;   // no progress for longer than this = stalled
  bool onlyWhenBusy;     // SerialLogger: stalled only if records are waiting
  bool stalled;          // current episode, warned once
} Watched;

```

Define the watched responsibilities / continued

src/services/supervisor.c / lines 37-54

```
37 static Watched s_watched[] = {
38     {TASK_ID_SAFETY, "safety", SUPERVISOR_SAFETY_DEADLINE_MS, false, false},
39     {TASK_ID_ACQUISITION_NORMAL, "acq", SUPERVISOR_ACQ_DEADLINE_MS, false, false},
40     {TASK_ID_SERIAL_LOGGER, "logger", SUPERVISOR_LOGGER_DEADLINE_MS, true, false},
41     {TASK_ID_CONFIGURATION, "cfg", SUPERVISOR_CONFIG_DEADLINE_MS, false, false},
42     {TASK_ID_BLE_TRANSPORT, "ble", SUPERVISOR_BLE_DEADLINE_MS, false, false},
43     {TASK_ID_WIFI_MQTT, "wifi", SUPERVISOR_WIFI_DEADLINE_MS, false, false},
44     {TASK_ID_POWER_COORDINATOR, "power", SUPERVISOR_POWER_DEADLINE_MS, false, false},
45 };
46 #define WATCHED_COUNT (sizeof(s_watched) / sizeof(s_watched[0]))
47
48 // Health summary shown in the status line: OK until something degraded in
49 // the last status period.
50 static char s_degraded[48];
51
52 static void markDegraded(const char* what) {
53   if (s_degraded[0] == '\0') snprintf(s_degraded, sizeof(s_degraded), "%s", what);
54 }
```

A first-cause string is not a complete list of all simultaneous problems. Individual warnings and counters preserve more detail than the short status label.

Judge the current lease holder and allow idle logging

src/services/supervisor.c / lines 55-82

```
55
56 static void checkProgress(MonoMs now) {
57   for (size_t i = 0; i < WATCHED_COUNT; ++i) {
58     Watched* w = &s_watched[i];
59     TaskId id = w->id;
60     if (id == TASK_ID_ACQUISITION_NORMAL) {
61       // WP9: whichever worker holds the lease owes progress; nobody during
62       // the hand-over (PowerCoordinator reports a stuck one as DEGRADED).
63       id = acquisitionActiveTaskId();
64       if (id == TASK_ID_COUNT) continue;
65     }
66     const MonoMs last = progressLastMs(id);  // 0 = never stamped: age from boot
67     const MonoMs age = now > last ? now - last : 0;
68     const bool idleLogger = w->onlyWhenBusy && logQueueDepth() == 0;
69     const bool late = !idleLogger && now > SUPERVISOR_BOOT_GRACE_MS && age >
 >   w->deadlineMs;
70     if (late) {
71       markDegraded(w->name);
72       if (!w->stalled) {
73         w->stalled = true;
74         logPrintf(LOG_LEVEL_WARN, "sup: %s no progress for %lu ms (deadline %lu)",
 >   w->name,
75                   (unsigned long)age, (unsigned long)w->deadlineMs);
76       }
77     } else if (w->stalled) {
78       w->stalled = false;
79       logPrintf(LOG_LEVEL_INFO, "sup: %s progressing again", w->name);
80     }
81   }
82 }
```

For acquisition, checkProgress substitutes acquisitionActiveTaskId; during handover there may be no holder. Logger is exempt while its queue is empty. Late episodes warn once and recovery emits a separate message.

The comparison uses now against the last meaningful timestamp after boot grace. An inactive worker's old timestamp should not become a false stall. PowerCoord separately reports a stuck transition.

Gather loss and lateness through public copies

src/services/supervisor.c / lines 83-104

```
83
84 // ---- Counters: warn when a loss / lateness counter goes up -----------------------
85 typedef struct Counters {
86   uint32_t frameLate, frameSkipped, bmeLate, bmeSkipped, guard, alarmOverflow, logDrops;
87 } Counters;
88
89 static Counters s_prev;
90
91 static void readCounters(Counters* c) {
92   AcquisitionStats a;
93   acquisitionGetStats(&a);
94   SafetyStatus st;
95   memset(&st, 0, sizeof(st));
96   safetyStatusPeek(&st);
97   c->frameLate = a.frameLate;
98   c->frameSkipped = a.frameSkipped;
99   c->bmeLate = a.bmeLate;
100  c->bmeSkipped = a.bmeSkipped;
101  c->guard = a.guardTimeouts;
102  c->alarmOverflow = st.detailOverflow;
103  c->logDrops = logDroppedCount();
104 }
```

Counters combines acquisition timing, guard timeouts, Safety detail overflow and logger drops. It does not reset those cumulative counters.

A cumulative value of 20 means twenty since its producer's reset, not twenty this second. checkCounters computes the change since its own previous observation before reporting current trouble.

Report deltas without flooding one line per event

src/services/supervisor.c / lines 105-132

```
105
106 static void checkCounters(void) {
107   Counters c;
108   readCounters(&c);
109   if (memcmp(&c, &s_prev, sizeof(c)) != 0) {
110     // One line per second at most; deltas since the last check.
111     static char line[LOG_TEXT_LEN];  // task-private: keeps the 3 KiB stack free
112     size_t used = (size_t)snprintf(line, sizeof(line), "sup: counters up:");
113     const struct { const char* name; uint32_t now, before; } items[] = {
114         {"frameLate", c.frameLate, s_prev.frameLate},
115         {"frameSkip", c.frameSkipped, s_prev.frameSkipped},
116         {"bmeLate", c.bmeLate, s_prev.bmeLate},
117         {"bmeSkip", c.bmeSkipped, s_prev.bmeSkipped},
118         {"guard", c.guard, s_prev.guard},
119         {"alarmOverflow", c.alarmOverflow, s_prev.alarmOverflow},
120         {"logDrops", c.logDrops, s_prev.logDrops},
121     };
122     for (size_t i = 0; i < sizeof(items) / sizeof(items[0]) && used < sizeof(line); ++i)
  >    {
123       if (items[i].now == items[i].before) continue;
124       const int n = snprintf(line + used, sizeof(line) - used, " %s +%lu",
  >   items[i].name,
125                              (unsigned long)(items[i].now - items[i].before));
126       if (n > 0) used += (size_t)n;
127       markDegraded(items[i].name);
128     }
129     logPrintf(LOG_LEVEL_WARN, "%s", line);
130     s_prev = c;
131   }
132 }
```

The helper compares the new group with its previous copy. Changed fields are appended to a bounded text buffer and mark the period degraded. It then retains the new group.

The line can be truncated, but the program does not write past its buffer merely because snprintf reports how many characters would have been needed. Follow the used < sizeof(line) guard before the next append.

Interpret stack high-water marks in this port's units

src/services/supervisor.c / lines 133-138

```
133
134 // ---- Stacks and heap
  >   ------------------------------------------------------------------------
135 static uint32_t stackFree(TaskHandle_t t) {
136   // ESP-IDF: StackType_t is a byte, so the high-water mark is in bytes.
137   return t != NULL ? (uint32_t)uxTaskGetStackHighWaterMark(t) : 0;
138 }
```

stackFree asks FreeRTOS for the smallest unused stack region observed for a task. This ESP-IDF port reports the byte-oriented convention used by the project. A missing task returns zero but is checked separately before a low-stack warning.

A high-water mark is historical observed headroom, not proof that a future deeper call path fits. Exercise error and configuration paths before treating the value as representative.

Observe internal heap and every application stack

src/services/supervisor.c / lines 139-166

```
139
140 static void checkMemory(bool report) {
141   const struct { const char* name; TaskHandle_t handle; } tasks[] = {
142       {"safety", safetyTaskHandle()},
143       {"acq", acquisitionNormalTaskHandle()},
144       {"log", (TaskHandle_t)logTaskHandle()},
145       {"sup", s_task},
146       {"cfg", (TaskHandle_t)configurationTaskHandle()},
147       {"ble", (TaskHandle_t)bleTransportTaskHandle()},
148       {"wifi", (TaskHandle_t)wifiMqttTaskHandle()},
149       {"acqlp", acquisitionLowPowerTaskHandle()},
150       {"pwr", (TaskHandle_t)powerCoordinatorTaskHandle()},
151   };
152   uint32_t freeB[9];
153   bool low = false;
154   for (size_t i = 0; i < 9; ++i) {
155     freeB[i] = stackFree(tasks[i].handle);
156     if (tasks[i].handle != NULL && freeB[i] < SUPERVISOR_STACK_WARN_BYTES) {
157       low = true;
158       markDegraded("stack");
159     }
160   }
161   const uint32_t heapFree = (uint32_t)heap_caps_get_free_size(MALLOC_CAP_INTERNAL);
162   const uint32_t heapMin =
  >   (uint32_t)heap_caps_get_minimum_free_size(MALLOC_CAP_INTERNAL);
163   const uint32_t largest =
  >   (uint32_t)heap_caps_get_largest_free_block(MALLOC_CAP_INTERNAL);
164   const bool heapLow = heapMin < SUPERVISOR_HEAP_WARN_BYTES;
165   if (heapLow) markDegraded("heap");
166   if (report || low || heapLow) {
```

The table gathers nine task handles. The loop compares existing stacks with the threshold; heap queries collect current free space, minimum ever free and the largest contiguous free block.

Observe internal heap and every application stack / continued

src/services/supervisor.c / lines 167-175

```
    logPrintf(low || heapLow ? LOG_LEVEL_WARN : LOG_LEVEL_INFO,
              // Fits LOG_TEXT_LEN (128) with every value at 6 digits.
              "sup: stack B saf %lu acq %lu log %lu sup %lu cfg %lu ble %lu wifi %lu | heap %lu min %lu big %lu",
              (unsigned long)freeB[0], (unsigned long)freeB[1], (unsigned long)freeB[2],
              (unsigned long)freeB[3], (unsigned long)freeB[4], (unsigned long)freeB[5], (unsigned long)freeB[6], (unsigned long)heapFree, (unsigned long)heapMin,
              (unsigned long)largest);
    logPrintf(low ? LOG_LEVEL_WARN : LOG_LEVEL_INFO, "sup: stack B acqlp %lu pwr %lu", (unsigned long)freeB[7],
              (unsigned long)freeB[8]);
}
```

src/services/supervisor.c / lines 176-176

```
}
```

Total free bytes and largest block answer different allocation questions. Fragmented free space may have enough total capacity but no sufficiently large single block. These observations complement the statically reserved application stacks.

Connect a reboot to a reported reset category

src/services/supervisor.c / lines 177-193

```
177
178 // ---- Status line
  >   --------------------------------------------------------------------------
179 static const char* resetReasonName(esp_reset_reason_t r) {
180   switch (r) {
181     case ESP_RST_POWERON: return "POWERON";
182     case ESP_RST_SW: return "SW";
183     case ESP_RST_PANIC: return "PANIC";
184     case ESP_RST_INT_WDT: return "INT_WDT";
185     case ESP_RST_TASK_WDT: return "TASK_WDT";
186     case ESP_RST_WDT: return "WDT";
187     case ESP_RST_BROWNOUT: return "BROWNOUT";
188     case ESP_RST_DEEPSLEEP: return "DEEPSLEEP";
189     case ESP_RST_EXT: return "EXT";
190     case ESP_RST_SDIO: return "SDIO";
191     default: return "UNKNOWN";
192   }
193 }
```

The switch maps the ESP reset reason to readable text. TASK_WDT, INT_WDT and BROWNOUT identify different reported reset classes.

A reset reason narrows the investigation; it is not a complete event trace. Keep the serial log leading up to the reset so warnings, sense transitions and the next boot can be compared.

Format stored observations with visible health

src/services/supervisor.c / lines 194-207

```
194
195 // "airT 23.1" for a usable value, "airT FAULT" otherwise.
196 static int cell(char* buf, size_t len, const char* label, const Reading* r, int
  >   decimals) {
197   if (r->health == HEALTH_OK || r->health == HEALTH_UNCALIBRATED) {
198     return snprintf(buf, len, " %s %.*f%s", label, decimals, (double)r->value,
199                     r->health == HEALTH_UNCALIBRATED ? "~" : "");
200   }
201   // Failures show their raw value when they have one, e.g. HC-SR04 TIMEOUT
202   // with raw = us elapsed (ECHO rose, no echo back) vs no raw (never rose).
203   if (r->hasRaw && (r->health == HEALTH_FAULT || r->health == HEALTH_TIMEOUT)) {
204     return snprintf(buf, len, " %s %s(raw %ld)", label, healthName(r->health),
  >   (long)r->raw);
205   }
206   return snprintf(buf, len, " %s %s", label, healthName(r->health));
207 }
```

cell prints numbers for stored OK or UNCALIBRATED and marks the latter with a tilde. Fault/timeout records may expose raw diagnostic evidence. Other states print their health name.

This helper does not call readingHealthAt or apply a fresh rail gate. A console number can therefore be older than its displayed stored health suggests. Safety and Telemetry have stronger acceptance checks; do not mistake this diagnostic row for their decision input.

Print a compact period report from retained copies

src/services/supervisor.c / lines 208-235

```
208
209 static void printStatus(MonoMs now) {
210   static char line[LOG_TEXT_LEN];  // task-private: keeps the 3 KiB stack free
211   size_t used = (size_t)snprintf(line, sizeof(line), "t=%lus", (unsigned long)(now /
  >   1000));
212   if (snapshotPeek(&s_snap)) {
213     const Reading* r = s_snap.readings;
214     struct { const char* label; MetricId m; int dec; } cols[] = {
215         {"airT", METRIC_AIR_TEMPERATURE, 1}, {"RH", METRIC_AIR_HUMIDITY, 0},
216         {"waterT", METRIC_WATER_TEMPERATURE, 1}, {"light", METRIC_LIGHT_LEVEL, 0},
217         {"pH", METRIC_PH, 2}, {"TDS", METRIC_TDS, 0}, {"dist", METRIC_WATER_DISTANCE,
  >   0},
218         {"CO2eq", METRIC_CO2EQ, 0}};
219     for (size_t i = 0; i < sizeof(cols) / sizeof(cols[0]) && used < sizeof(line); ++i) {
220       const int n = cell(line + used, sizeof(line) - used, cols[i].label, &r[cols[i].m],
  >    cols[i].dec);
221       if (n > 0) used += (size_t)n;
222     }
223   } else if (used < sizeof(line)) {
224     used += (size_t)snprintf(line + used, sizeof(line) - used, " no snapshot");
225   }
226   logPrintf(LOG_LEVEL_INFO, "%s", line);
227
228   SafetyStatus st;
229   AcquisitionStats a;
230   acquisitionGetStats(&a);
231   if (safetyStatusPeek(&st)) {
232     logPrintf(LOG_LEVEL_INFO,
233               "     lamp %s (%s)%s | latched 0x%04x | frames %lu late %lu skip %lu
  >   jitter %lu ms"
234               " | guard %lu fail %lu | %s%s",
235               st.relayOn ? "ON" : "OFF", lampReasonName(st.lampReason),
```

printStatus gathers Snapshot, acquisition stats and Safety status, then emits readings and controller decisions. The degraded string is cleared for the next reporting period.

Print a compact period report from retained copies / continued

src/services/supervisor.c / lines 236-242

```
236                 st.lampWantsOn && !st.relayOn ? " wants ON" : "",
  >   (unsigned)st.latchedMask,
237                 (unsigned long)a.frames, (unsigned long)a.frameLate, (unsigned
  >   long)a.frameSkipped,
238                 (unsigned long)a.frameJitterMaxMs, (unsigned long)a.guardTimeouts,
239                 (unsigned long)a.driverFailures, s_degraded[0] ? "DEGRADED " : "health
  >   OK",
240                 s_degraded);
241   }
242   s_degraded[0] = '\0';  // a new period starts
```

src/services/supervisor.c / lines 243-243

```
243 }
```

The console status is a bounded observation channel. Missing trailing text may be truncation rather than missing controller state; inspect specific counters or source-owned status when a diagnosis needs that detail.

Run periodic observation without taking ownership away

src/services/supervisor.c / lines 244-271

```
244
245 static void supervisorTask(void* arg) {
246   (void)arg;
247   logPrintf(LOG_LEVEL_INFO, "reset reason: %s", resetReasonName(esp_reset_reason()));
248   readCounters(&s_prev);
249   TickType_t last = xTaskGetTickCount();
250   uint32_t seconds = 0;
251   for (;;) {
252     vTaskDelayUntil(&last, pdMS_TO_TICKS(CFG_SUPERVISOR_PERIOD_MS));
253     const MonoMs now = monoNowMs();
254     ++seconds;
255     checkProgress(now);
256     checkCounters();
257     if (seconds % SUPERVISOR_MEMORY_EVERY_S == 0) {
258       checkMemory(seconds % SUPERVISOR_MEMORY_REPORT_EVERY_S == 0);
259     }
260     if (seconds % SUPERVISOR_STATUS_EVERY_S == 0) printStatus(now);
261     progressMark(TASK_ID_SUPERVISOR);
262   }
263 }
264
265 bool supervisorInit(void) {
266   if (s_task != NULL) return true;
267   const TaskSpec* spec = cfgTaskSpec(TASK_ID_SUPERVISOR);
268   s_task = xTaskCreateStaticPinnedToCore(supervisorTask, spec->name, sizeof(s_stack),
  >   NULL,
269                                          spec->priority, s_stack, &s_tcb, spec->core);
270   return s_task != NULL;
271 }
```

supervisorTask uses vTaskDelayUntil for its periodic release and schedules slower checks by the seconds counter. Init creates its statically allocated task.

Only owners can prove their own useful progress. Supervisor observes that evidence and warns; it cannot make a stalled acquisition healthy by writing its progress timestamp or feeding its watchdog on its behalf.

Make a controlled capture and preserve complete lines

A test log is useful only if opening the monitor does not introduce unexplained resets. This Python utility controls the serial handshake lines and records elapsed time.

It runs on your computer, not on the ESP32.

tools/serial_log.py / lines 1-28

```
"""Log the ESP serial port to a file without the monitor's DTR/RTS reset toggling.

usage: python tools/serial_log.py PORT OUTFILE SECONDS [--reset]

--reset   one controlled EN pulse (RTS) after opening, so exactly one boot is
          expected; any extra boot in the log is not caused by the monitor.
Run with PlatformIO's Python (it has pyserial):
  %USERPROFILE%\\.platformio\\penv\\Scripts\\python.exe tools/serial_log.py COM4
  boot.log 60 --reset
"""
import sys
import time

import serial


def main() -> None:
    port, out, seconds = sys.argv[1], sys.argv[2], float(sys.argv[3])
    ser = serial.Serial()
    ser.port, ser.baudrate, ser.timeout = port, 115200, 0.2
    ser.dtr = False  # IO0 high: normal boot, never download mode
    ser.rts = False  # EN released
    ser.open()
    start = time.monotonic()
    with open(out, "w", encoding="utf-8", errors="replace") as log:
        if "--reset" in sys.argv:
            log.write("[logger] EN reset pulse\n")
            ser.rts = True
            time.sleep(0.15)
```

The script sets DTR/RTS before opening the port, optionally applies one labelled reset pulse, then reads bytes until its monotonic duration expires. It splits newline-terminated records and flushes them to the file.

tools/serial_log.py / lines 29-34

```
            ser.rts = False
        buffer = b""
        while time.monotonic() - start < seconds:
            buffer += ser.read(4096)
            while b"\n" in buffer:
                line, buffer = buffer.split(b"\n", 1)
```

Make a controlled capture and preserve complete lines / continued

tools/serial_log.py / lines 35-41

```
                log.write(f"{time.monotonic() - start:8.2f} {line.decode('utf-8',
 >  'replace').rstrip()}\n")
                log.flush()
    ser.close()


if __name__ == "__main__":
    main()
```

The last unfinished line remains buffered when time expires, and arguments are read directly from argv without a friendly validator. Use the documented invocation and remember that this utility is a focused experiment aid, not a robust general serial terminal.

Preserve the interpreter's expected line endings

Version control can change line endings between systems. This rule preserves LF endings for shell scripts.

This file affects repository handling, not the running image.

.gitattributes / lines 1-2

```
# Shell scripts must keep LF endings, or bash on Linux / WSL rejects them.
*.sh text eol=lf
```

The pattern selects .sh files and requests LF. A stray carriage-return byte can become part of an interpreter name or shell token, so a text-format rule can prevent a build or tool invocation failure.

There are no application statements here. It belongs in the full source review because reproducible tooling includes how text files reach the build machine.

Distinguish source from local working output

Generated build products and local environments should not be mistaken for reviewed source. These patterns describe files Git ordinarily leaves untracked.

Ignoring a path does not remove a file that was already committed.

.gitignore / lines 1-13

```
.pio/
.vscode/
__pycache__/
tools/.ble_bench_state.json
# Desktop app (app/): environments and build work files. app/dist/ (the
# Windows .exe and the Linux binary) IS committed.
app/.venv/
app/.venv-linux/
app/build/
app/.pytest_cache/
app/*.spec
app/link.log
*.pyc
```

The rules exclude PlatformIO output, editor state, Python environments and app build caches. The comment explicitly keeps app distribution binaries under version control in the broader repository.

This reference archive contains the firmware file set, not every desktop app asset. The ignored patterns explain workflow; they do not implement networking or change firmware behaviour.

Read an assertion as an exact prediction

You have been making predictions throughout the book. A unit test asks the computer to repeat one of those predictions after a code change. Unit means the behaviour under test is small enough to supply its inputs directly. Here the pure domain layer accepts explicit timestamps, records and events, so a test needs no sensor connected.

Teaching example - Unity assertion

```
uint32_t runs = 0;
for (uint32_t i = 0; i < 4; ++i) {
    ++runs;
}
TEST_ASSERT_EQUAL_UINT32(4, runs);
```

The first assertion argument is the expected value, 4. The second is the actual computed value, runs. If they differ, Unity reports a failure and its source location. UINT32 tells the test framework how to compare and display that integer width. The assertion is a macro supplied by unity.h; it is not part of the C language itself.

Teaching example - distanceMm supplied by a preceding call

```
TEST_ASSERT_FLOAT_WITHIN(0.1f, 171.5f, distanceMm);
```

The floating-point form gives tolerance first, expected second and actual third. Here the accepted difference is at most 0.1mm around 171.5mm. Choose that tolerance from the numerical question being tested. A formula test tolerance is not automatically the physical accuracy of an HC-SR04.

setUp and tearDown are hooks Unity calls around each registered test; this source leaves them empty where each test creates its own state. RUN_TEST registers execution in main. Defining a test function alone does not run it. Read the final registration list after each file to see which predictions the suite actually checks.

COMMON PITFALL

A test that merely checks the same formula copied from the implementation can repeat the same mistake. Use a separately known vector, physical relation or state-order requirement.

REMEMBER

Arrange the input. Perform the operation. Check the promised outcome.

Choose the machine that can answer the question

Question	Useful target	Reason
Does the CRC reject a known corrupt reply?	Native domain test	Supply exact bytes and compare the result.
Does Safety refuse an expired reset?	Native domain test	Supply explicit now and expiry without waiting.
Does the ADC pin produce the expected millivolts?	Sensor bring-up image	The peripheral, wiring and calibration must be present.
Does a worker release after cancellation?	Production integration trace	The actual tasks, ownership and HAL cleanup interact.

Shell commands - run from the extracted project root

```
pio test -e native
pio run -e test_ds18b20 -t upload -t monitor
pio run -e esp32dev
```

The first command builds and runs pure domain suites on your computer. The second builds a hardware bring-up image and uploads it to the selected device. The third builds the production firmware image without uploading it. Check the selected serial port and environment before using an upload command; a test entry point intentionally replaces the controller entry.

The native domain test in this reference source calls memset without directly including string.h. A stricter modern host compiler can reject that missing declaration. Add the missing include in a working copy, or supply that header through a compiler option when reproducing the original unchanged source. This build issue is separate from a failed behaviour assertion.

Use a failing assertion to localise a deterministic decision. Use a timestamped hardware trace for scheduling and electrical questions. The full files that follow let you inspect the actual assertions rather than accepting a test count as an explanation of what was checked.

COMMON PITFALL

A successful host test does not execute GPIO timing, FreeRTOS core affinity, flash exclusion or radio callbacks.

REMEMBER

The build environment chooses both the source set and the machine on which it will run.

Test the exact freshness boundary

A test is a small executable argument: arrange a known state, call the function under controlled conditions, then assert the expected outcome. Read these tests as worked examples of the domain lessons.

Unity supplies TEST_ASSERT and RUN_TEST. An assertion names a property the implementation must satisfy; a passing suite establishes only the cases it actually checks.

test/native/test_domain/test_main.c / lines 1-24

```
1  // Host unit tests for src/domain (pio test -e native).
>    Architecture §17 stage 4.
2  // Domain code must stay free of Arduino/FreeRTOS so these
>    run on a PC.
3  #include <unity.h>
4
5  #include "app_config.h"
6  #include "board_profile.h"
7  #include <math.h>
8
9  #include "domain/alarm.h"
10 #include "domain/alarm_sticky.h"
11 #include "domain/calibration.h"
12 #include "domain/ds18b20_codec.h"
13 #include "domain/lamp_control.h"
14 #include "domain/reading.h"
15 #include "domain/safety_core.h"
16 #include "domain/schedule.h"
17 #include "domain/sensor_set.h"
18 #include "domain/sgp30_codec.h"
19 #include "domain/tds_codec.h"
20 #include "domain/ultrasonic.h"
21
22 void setUp(void) {}
23 void tearDown(void) {}
24
```

test/native/test_domain/test_main.c / lines 25-31

```
25 static void
>    test_reset_reading_is_starting_and_not_actionable(void) {
26   Reading r;
27   readingReset(&r, SENSOR_BME280, METRIC_AIR_TEMPERATURE,
>    UNIT_DEG_C);
28   TEST_ASSERT_TRUE(r.health == HEALTH_STARTING);
29   TEST_ASSERT_TRUE(r.sensor == SENSOR_BME280);
30   TEST_ASSERT_FALSE(readingIsActionable(&r));
31 }
```

COMMON PITFALL

This source uses memset later without directly including string.h. A strict host build needs that declaration supplied; keep the original listing visible rather than silently editing it.

Test the exact freshness boundary / continued

test/native/test_domain/test_main.c / lines 32-52

```
32
33 static void test_ok_reading_becomes_stale_after_limit(void) {
34   Reading r;
35   readingReset(&r, SENSOR_DS18B20, METRIC_WATER_TEMPERATURE, UNIT_DEG_C);
36   r.health = HEALTH_OK;
37   r.intervalEndMs = 1000;
38   TEST_ASSERT_TRUE(readingHealthAt(&r, 2000, 1000) == HEALTH_OK);
39   TEST_ASSERT_TRUE(readingHealthAt(&r, 2001, 1000) == HEALTH_STALE);
40 }
41
42 static void test_stale_reading_is_not_actionable(void) {
43   Reading r;
44   readingReset(&r, SENSOR_PH, METRIC_PH, UNIT_PH);
45   r.health = HEALTH_OK;
46   r.intervalEndMs = 1000;
47   TEST_ASSERT_TRUE(readingIsActionableAt(&r, 4000, 3000));   // exactly at the limit
48   TEST_ASSERT_FALSE(readingIsActionableAt(&r, 4001, 3000));  // stale
49   TEST_ASSERT_TRUE(readingIsActionable(&r));                 // health-only check is
 >   unchanged
50   r.health = HEALTH_UNCALIBRATED;
51   TEST_ASSERT_FALSE(readingIsActionableAt(&r, 1000, 3000));  // fresh but not OK
52 }
```

The first cases create a Reading through its public reset helper, then edit only the fields needed for the scenario. At age 1000ms with a 1000ms limit, health is still OK; at 1001ms it is STALE.

test/native/test_domain/test_main.c / lines 53-59

```
53
54 static void test_fault_is_not_masked_by_staleness(void) {
55   Reading r;
56   readingReset(&r, SENSOR_PH, METRIC_PH, UNIT_PH);
57   r.health = HEALTH_UNPOWERED;
58   TEST_ASSERT_TRUE(readingHealthAt(&r, 100000, 1000) == HEALTH_UNPOWERED);
59 }
```

The next checks distinguish stored health from time-aware actionability and preserve UNPOWERED rather than replacing it with a less informative age result. Read expected before actual in each Unity equality assertion.

Check numerical transforms with tolerances

test/native/test_domain/test_main.c / lines 60-83

```
60
61 static void test_divider_recovery_uses_nominal_ratios(void) {
62   // Power sense: 220/440 => x660/440. 3333 mV at GPIO8 -> 5000 mV external feed.
63   TEST_ASSERT_FLOAT_WITHIN(0.5f, 4999.5f,
64                            dividerRecoverMv(3333.0f, BOARD_POWER_SENSE_DIVIDER_TOP_OHM,
65                                             BOARD_POWER_SENSE_DIVIDER_BOTTOM_OHM));
66   // pH: 10k/15k => x25/15. 3000 mV at the pin -> 5000 mV at the module.
67   TEST_ASSERT_FLOAT_WITHIN(0.5f, 5000.0f,
68                            dividerRecoverMv(3000.0f, BOARD_PH_DIVIDER_TOP_OHM,
69                                             BOARD_PH_DIVIDER_BOTTOM_OHM));
70 }
71
72 static void test_two_point_fit_maps_buffers(void) {
73   // Module mV in pH 6.86 and 4.01 buffers, about -5.7 pH/V (tutorial).
74   LinearCalibration c;
75   TEST_ASSERT_TRUE(linearFromTwoPoints(2500.0f, 6.86f, 3000.0f, 4.01f, 1, &c));
76   TEST_ASSERT_TRUE(c.valid);
77   TEST_ASSERT_FLOAT_WITHIN(1e-6f, -0.0057f, c.slope);
78   TEST_ASSERT_FLOAT_WITHIN(0.001f, 6.86f, linearApply(&c, 2500.0f));
79   TEST_ASSERT_FLOAT_WITHIN(0.001f, 4.01f, linearApply(&c, 3000.0f));
80   TEST_ASSERT_FALSE(linearFromTwoPoints(2500.0f, 6.86f, 2500.0f, 4.01f, 2, &c));
81   TEST_ASSERT_FALSE(c.valid);
82 }
83
```

The divider checks recover module voltage from pin voltage. The two-point fit verifies both slope and mapped reference points, then rejects coincident x coordinates. The trimmed-mean case includes deliberate high and low spikes.

test/native/test_domain/test_main.c / lines 84-90

```
84 static void test_trimmed_mean_rejects_spikes(void) {
85   uint32_t s[10] = {1500, 4000, 1502, 1498, 0, 1501, 1499, 1500, 3100, 1500};
86   TEST_ASSERT_FLOAT_WITHIN(0.01f, 1500.333f, trimmedMeanU32(s, 10, 2));  // 1499..1502
87   TEST_ASSERT_TRUE(s[0] == 0 && s[9] == 4000);  // sorted in place
88   uint32_t few[4] = {1, 2, 3, 4};
89   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 0.0f, trimmedMeanU32(few, 4, 2));
90 }
```

TEST_ASSERT_FLOAT_WITHIN supplies a tolerance because floating-point calculations need a stated comparison allowance. The array is also checked after sorting: this helper intentionally changes its input.

Verify identity and eligibility before thresholds

test/native/test_domain/test_main.c / lines 91-118

```
91
92 static void test_snapshot_reset_indexes_by_metric(void) {
93   Snapshot s;
94   snapshotReset(&s);
95   for (uint8_t i = 0; i < METRIC_COUNT; ++i) {
96     TEST_ASSERT_TRUE(s.readings[i].metric == (MetricId)i);
97     TEST_ASSERT_TRUE(s.readings[i].health == HEALTH_STARTING);
98   }
99 }
100
101 static void test_production_ready_requires_power_sense(void) {
102   BoardValidation v = boardProfileValidate();
103   TEST_ASSERT_EQUAL(v.powerSenseAssigned && BOARD_POWER_SENSE_VERIFIED,
  >   v.productionReady);
104 }
105
106 static void test_snapshot_reset_sets_identity(void) {
107   Snapshot s;
108   snapshotReset(&s);
109   TEST_ASSERT_EQUAL_UINT8(SENSOR_BME280, s.readings[METRIC_AIR_HUMIDITY].sensor);
110   TEST_ASSERT_EQUAL_UINT8(UNIT_PERCENT_RH, s.readings[METRIC_AIR_HUMIDITY].unit);
111   TEST_ASSERT_EQUAL_UINT8(SENSOR_DS18B20, s.readings[METRIC_WATER_TEMPERATURE].sensor);
112   TEST_ASSERT_EQUAL_UINT8(SENSOR_LIGHT, s.readings[METRIC_LIGHT_LEVEL].sensor);
113   TEST_ASSERT_EQUAL_UINT8(SENSOR_SGP30, s.readings[METRIC_CO2EQ].sensor);
114   TEST_ASSERT_EQUAL_UINT8(UNIT_PPB, s.readings[METRIC_TVOC].unit);
115   for (uint8_t i = 0; i < METRIC_COUNT; ++i) {
116     TEST_ASSERT_TRUE(s.readings[i].sensor < SENSOR_COUNT);
117   }
118 }
```

Snapshot checks visit all indices and specific multi-metric mappings. The board test checks exactly the implemented readiness expression. The alarm case first presents an over-age sample, then a younger one.

Verify identity and eligibility before thresholds / continued

test/native/test_domain/test_main.c / lines 119-129

```
119
120 static void test_alarm_rule_ignores_stale_sample(void) {
121   AlarmRule rule = {0};
122   rule.metric = METRIC_AIR_TEMPERATURE;
123   AlarmRuleState state = {0};
124   Reading r;
125   readingReset(&r, SENSOR_BME280, METRIC_AIR_TEMPERATURE, UNIT_DEG_C);
126   r.health = HEALTH_OK;
127   r.value = 50.0f;
128   r.intervalEndMs = 1000;
129   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NOT_ELIGIBLE, evaluateRule(&rule, &r, 2001,
  >   1000, &state));
```

test/native/test_domain/test_main.c / lines 130-131

```
130   TEST_ASSERT_TRUE(evaluateRule(&rule, &r, 1500, 1000, &state) !=
  >   RULE_DECISION_NOT_ELIGIBLE);
131 }
```

A hot number is irrelevant if it is not eligible for comparison. These tests help you separate correct numerical threshold logic from the earlier question of whether a measurement may be used.

Use known CRC and encoding vectors

test/native/test_domain/test_main.c / lines 132-159

```
132
133 static void test_sgp30_crc_matches_datasheet_vector(void) {
134   const uint8_t beef[2] = {0xBE, 0xEF};
135   TEST_ASSERT_EQUAL_HEX8(0x92, sgp30Crc8(beef, 2));
136 }
137
138 static void test_sgp30_words_round_trip_and_reject_bad_crc(void) {
139   uint8_t buf[6];
140   sgp30PackWord(0x01A3, &buf[0]);  // CO2eq 419 ppm
141   sgp30PackWord(0x0007, &buf[3]);  // TVOC 7 ppb
142   uint16_t w[2] = {0, 0};
143   TEST_ASSERT_TRUE(sgp30UnpackWords(buf, 2, w));
144   TEST_ASSERT_EQUAL_HEX16(0x01A3, w[0]);
145   TEST_ASSERT_EQUAL_HEX16(0x0007, w[1]);
146   buf[4] ^= 0x01;  // one flipped bit in the second word
147   TEST_ASSERT_FALSE(sgp30UnpackWords(buf, 2, w));
148   const uint8_t floating[3] = {0xFF, 0xFF, 0xFF};  // released bus
149   TEST_ASSERT_FALSE(sgp30UnpackWords(floating, 1, w));
150 }
151
152 static void test_sgp30_humidity_fixed_point(void) {
153   TEST_ASSERT_EQUAL_HEX16(0x0F80, sgp30GramsToFixed8p8(15.5f));  // datasheet example
154   TEST_ASSERT_EQUAL_HEX16(0x0000, sgp30GramsToFixed8p8(0.0f));
155   TEST_ASSERT_EQUAL_HEX16(0x0001, sgp30GramsToFixed8p8(0.001f));
156   TEST_ASSERT_EQUAL_HEX16(0xFFFF, sgp30GramsToFixed8p8(300.0f));
157   // The datasheet formula gives 11.48 g/m3 at 25 degC / 50 %RH (its text rounds
158   // this example to 11.8).
159   TEST_ASSERT_FLOAT_WITHIN(0.02f, 11.48f, sgp30AbsoluteHumidity(25.0f, 50.0f));
```

The SGP30 tests check the documented BEEF CRC vector, pack/unpack two words, flip one bit and reject an all-ones response. The humidity cases cover exact 8.8 representation, tiny positive rounding, clamping and a physical input range.

test/native/test_domain/test_main.c / lines 160-161

```
160   TEST_ASSERT_TRUE(sgp30AbsoluteHumidity(25.0f, 120.0f) < 0.0f);
161 }
```

A round trip alone can hide matching encoder and decoder mistakes. The independently known CRC and fixed-point vectors provide a second reference for the implementation.

Test physical formulas at interpretable numbers

test/native/test_domain/test_main.c / lines 162-189

```
162
163 static void test_ultrasonic_echo_width_to_mm(void) {
164   // 58.3 us per cm at 343 m/s (HC-SR04 guide, equation 3).
165   TEST_ASSERT_FLOAT_WITHIN(0.1f, 1000.0f,
166                            ultrasonicEchoUsToMm(5831, ULTRASONIC_NOMINAL_SPEED_MPS));
167   TEST_ASSERT_FLOAT_WITHIN(0.1f, 20.0f,
168                            ultrasonicEchoUsToMm(117, ULTRASONIC_NOMINAL_SPEED_MPS));
169   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 0.0f, ultrasonicEchoUsToMm(0,
  >   ULTRASONIC_NOMINAL_SPEED_MPS));
170 }
171
172 static void test_ultrasonic_range_is_2_to_400_cm(void) {
173   TEST_ASSERT_TRUE(ultrasonicInRangeMm(20.0f));
174   TEST_ASSERT_TRUE(ultrasonicInRangeMm(4000.0f));
175   TEST_ASSERT_FALSE(ultrasonicInRangeMm(19.9f));
176   TEST_ASSERT_FALSE(ultrasonicInRangeMm(4000.1f));
177   TEST_ASSERT_FALSE(ultrasonicInRangeMm(0.0f));  // never a valid distance
178 }
179
180 static void test_tds_datasheet_curve(void) {
181   // (133.42 - 255.86 + 857.39) * 0.5 at 1.0 V, 25 degC (SEN0244 sample code).
182   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 1.0f, tdsCompensationCoefficient(25.0f));
183   TEST_ASSERT_FLOAT_WITHIN(0.01f, 367.475f, tdsPpmFromModuleMv(1000.0f, 1.0f));
184   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 0.0f, tdsPpmFromModuleMv(0.0f, 1.0f));
185   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 0.0f, tdsPpmFromModuleMv(1000.0f, 0.0f));
186   // 2.3 V (top of the output range) is above the rated 1000 ppm.
187   TEST_ASSERT_FLOAT_WITHIN(0.5f, 1120.9f, tdsPpmFromModuleMv(2300.0f, 1.0f));
188   TEST_ASSERT_TRUE(tdsPpmFromModuleMv(2300.0f, 1.0f) > TDS_FULL_SCALE_PPM);
189 }
```

The echo checks turn a known flight time into 1000mm and exercise the inclusive 20–4000mm range. TDS evaluates the polynomial at 1V and compares compensation at 25 and 35 degrees Celsius.

Test physical formulas at interpretable numbers / continued

test/native/test_domain/test_main.c / lines 190-200

```
190
191 static void test_tds_temperature_compensation(void) {
192   // The conductivity (ppm) is compensated, not the voltage (GravityTDS).
193   const float coefficient = tdsCompensationCoefficient(35.0f);
194   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 1.2f, coefficient);
195   TEST_ASSERT_FLOAT_WITHIN(0.01f, 367.475f / 1.2f, tdsPpmFromModuleMv(1000.0f,
  >   coefficient));
196   // Measured 24 Oct 2026, same tap water cold and warm: within ~2 % at 25 degC.
197   const float cold = tdsPpmFromModuleMv(1234.0f, tdsCompensationCoefficient(11.44f));
198   const float warm = tdsPpmFromModuleMv(1768.0f, tdsCompensationCoefficient(33.62f));
199   TEST_ASSERT_FLOAT_WITHIN(0.03f * cold, cold, warm);
200 }
```

The cold/warm constants are recorded historical observations; rerunning this pure test recalculates from them. It does not repeat a water experiment. Keep recorded measurements and live hardware evidence distinct.

Construct a valid reply before corrupting it

test/native/test_domain/test_main.c / lines 201-224

```
201
202 // Scratchpad with the given temperature register and 12-bit config, valid CRC.
203 static void makeScratchpad(int16_t raw, uint8_t sp[DS18B20_SCRATCHPAD_LEN]) {
204   const uint8_t base[8] = {(uint8_t)(raw & 0xFF), (uint8_t)((uint16_t)raw >> 8), 0x4B,
  >   0x46,
205                            DS18B20_CONFIG_12BIT, 0xFF, 0x0C, 0x10};
206   for (uint8_t i = 0; i < 8; ++i) sp[i] = base[i];
207   sp[8] = oneWireCrc8(sp, 8);
208 }
209
210 static void test_onewire_crc_matches_maxim_vector(void) {
211   // Maxim application note 27: ROM 02 1C B8 01 00 00 00 has CRC A2.
212   const uint8_t rom[8] = {0x02, 0x1C, 0xB8, 0x01, 0x00, 0x00, 0x00, 0xA2};
213   TEST_ASSERT_EQUAL_HEX8(0xA2, oneWireCrc8(rom, 7));
214   TEST_ASSERT_EQUAL_HEX8(0x00, oneWireCrc8(rom, 8));  // data + CRC -> 0
215   TEST_ASSERT_FALSE(ds18b20RomValid(rom));  // family 02h, not a DS18B20
216   uint8_t ds[8] = {0x28, 0xFF, 0x64, 0x1E, 0x0F, 0x4A, 0x3C, 0x00};
217   ds[7] = oneWireCrc8(ds, 7);
218   TEST_ASSERT_TRUE(ds18b20RomValid(ds));
219   ds[3] ^= 0x01;
220   TEST_ASSERT_FALSE(ds18b20RomValid(ds));
221   const uint8_t zero[8] = {0};  // bus stuck low: CRC 0 matches, family does not
222   TEST_ASSERT_FALSE(ds18b20RomValid(zero));
223 }
224
```

test/native/test_domain/test_main.c / lines 225-242

```
225 static void test_ds18b20_temperature_table(void) {
226   // Datasheet Table 1.
227   uint8_t sp[DS18B20_SCRATCHPAD_LEN];
228   makeScratchpad(0x0191, sp);
229   TEST_ASSERT_TRUE(ds18b20ScratchpadValid(sp));
230   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 25.0625f,
  >   ds18b20RawToDegC(ds18b20RawTemperature(sp)));
231   makeScratchpad((int16_t)0xFF5E, sp);
232   TEST_ASSERT_FLOAT_WITHIN(1e-6f, -10.125f,
  >   ds18b20RawToDegC(ds18b20RawTemperature(sp)));
233   makeScratchpad((int16_t)0xFC90, sp);
234   TEST_ASSERT_FLOAT_WITHIN(1e-6f, -55.0f, ds18b20RawToDegC(ds18b20RawTemperature(sp)));
235   makeScratchpad(0x07D0, sp);
236   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 125.0f, ds18b20RawToDegC(ds18b20RawTemperature(sp)));
237   makeScratchpad(DS18B20_POWER_ON_RAW, sp);
238   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 85.0f, ds18b20RawToDegC(ds18b20RawTemperature(sp)));
239 }
240
241 static void test_ds18b20_scratchpad_checks(void) {
242   uint8_t sp[DS18B20_SCRATCHPAD_LEN];
```

Construct a valid reply before corrupting it / continued

test/native/test_domain/test_main.c / lines 243-252

```
243   makeScratchpad(0x0191, sp);
244   sp[0] ^= 0x01;  // one bit flipped on the wire
245   TEST_ASSERT_FALSE(ds18b20ScratchpadValid(sp));
246   const uint8_t zero[DS18B20_SCRATCHPAD_LEN] = {0};  // CRC 0 matches, config does not
247   TEST_ASSERT_FALSE(ds18b20ScratchpadValid(zero));
248   uint8_t ff[DS18B20_SCRATCHPAD_LEN];  // floating / no device
249   for (uint8_t i = 0; i < DS18B20_SCRATCHPAD_LEN; ++i) ff[i] = 0xFF;
250   TEST_ASSERT_FALSE(ds18b20ScratchpadValid(ff));
251   TEST_ASSERT_TRUE(ds18b20Is12Bit(0x7F));
252   TEST_ASSERT_FALSE(ds18b20Is12Bit(0x5F));  // 11-bit
```

makeScratchpad assembles a realistic DS18B20 reply and computes its CRC. Tests then check signed temperature examples, wrong family, a flipped bit, all-zero and all-one patterns, and resolution configuration.

test/native/test_domain/test_main.c / lines 253-254

```
253   TEST_ASSERT_FALSE(ds18b20Is12Bit(0x1F));  // 9-bit
254 }
```

Generating a valid fixture first makes a later one-bit change meaningful. A zero CRC alone cannot establish that a disconnected bus supplied a plausible device record.

Keep the lamp threshold in its original unit

test/native/test_domain/test_main.c / lines 255-278

```
255
256 static const LampControlConfig kLampTest = {40.0f, 120.0f, 30000, 30000, 3};
257
258 static Reading lampLight(float mv, Health health) {
259   Reading r;
260   readingReset(&r, SENSOR_LIGHT, METRIC_LIGHT_LEVEL, UNIT_MILLIVOLT);
261   r.health = health;
262   r.value = mv;
263   return r;
264 }
265
266 static void test_lamp_hysteresis_band(void) {
267   LampControl c;
268   lampControlInit(&c, &kLampTest, 0);
269   Reading r = lampLight(80.0f, HEALTH_UNCALIBRATED);  // inside the band: stays OFF
270   TEST_ASSERT_FALSE(lampControlUpdate(&c, &r, 60000));
271   r.value = 39.0f;
272   TEST_ASSERT_TRUE(lampControlUpdate(&c, &r, 61000));
273   r.value = 119.0f;  // inside the band: stays ON
274   TEST_ASSERT_TRUE(lampControlUpdate(&c, &r, 200000));
275   r.value = 121.0f;
276   TEST_ASSERT_FALSE(lampControlUpdate(&c, &r, 201000));
277 }
278
```

lampLight builds a compact fixture. The hysteresis case visits the interior band in both ON and OFF states. The next test changes the displayed value to lux but supplies raw pin millivolts.

test/native/test_domain/test_main.c / lines 279-289

```
279 // After an on-device lux fit the value is lux; the thresholds stay pin mV (raw).
280 static void test_lamp_uses_pin_mv_after_lux_fit(void) {
281   LampControl c;
282   lampControlInit(&c, &kLampTest, 0);
283   Reading r = lampLight(10.0f, HEALTH_OK);  // 10 mV at the pin: dark
284   r.unit = UNIT_LUX;
285   r.value = 500.0f;  // would be "bright" if read as mV
286   r.hasRaw = true;
287   r.raw = 10;
288   TEST_ASSERT_TRUE(lampControlUpdate(&c, &r, 60000));
289 }
```

The expected relay request still follows raw pin voltage. Without this case, a new display calibration could accidentally change control behaviour while all raw-millivolt tests continued to pass.

Advance time across dwell and repeated-fault boundaries

test/native/test_domain/test_main.c / lines 290-317

```
290
291 static void test_lamp_dwell_holds_changes(void) {
292   LampControl c;
293   lampControlInit(&c, &kLampTest, 0);
294   Reading r = lampLight(10.0f, HEALTH_OK);
295   TEST_ASSERT_FALSE(lampControlUpdate(&c, &r, 29999));  // boot counts as OFF
296   TEST_ASSERT_TRUE(c.waiting);
297   TEST_ASSERT_TRUE(lampControlUpdate(&c, &r, 30000));
298   TEST_ASSERT_FALSE(c.waiting);
299   r.value = 500.0f;
300   TEST_ASSERT_TRUE(lampControlUpdate(&c, &r, 59999));
301   TEST_ASSERT_FALSE(lampControlUpdate(&c, &r, 60000));
302 }
303
304 static void test_lamp_faults_hold_then_force_off(void) {
305   LampControl c;
306   lampControlInit(&c, &kLampTest, 0);
307   Reading r = lampLight(10.0f, HEALTH_OK);
308   TEST_ASSERT_TRUE(lampControlUpdate(&c, &r, 30000));
309   Reading bad = lampLight(0.0f, HEALTH_FAULT);
310   TEST_ASSERT_TRUE(lampControlUpdate(&c, &bad, 31000));
311   TEST_ASSERT_TRUE(lampControlUpdate(&c, &bad, 32000));
312   TEST_ASSERT_TRUE(lampControlUpdate(&c, &r, 33000));  // a good reading resets the
  >   count
313   TEST_ASSERT_TRUE(lampControlUpdate(&c, &bad, 34000));
314   TEST_ASSERT_TRUE(lampControlUpdate(&c, &bad, 35000));
315   TEST_ASSERT_FALSE(lampControlUpdate(&c, &bad, 36000));  // 3rd in a row, no dwell
316 }
317
```

The dwell test checks one millisecond before and exactly at its deadline. The fault test introduces a good result between failures to prove that only consecutive bad results accumulate. Saturation is treated as bright according to policy.

Advance time across dwell and repeated-fault boundaries / continued

test/native/test_domain/test_main.c / lines 318-326

```
318 static void test_lamp_saturated_counts_as_bright(void) {
319   LampControl c;
320   lampControlInit(&c, &kLampTest, 0);
321   Reading r = lampLight(10.0f, HEALTH_OK);
322   TEST_ASSERT_TRUE(lampControlUpdate(&c, &r, 30000));
323   r.value = 20.0f;  // a clipped value is only a lower bound
324   r.qualityFlags = kQualitySaturated;
325   TEST_ASSERT_FALSE(lampControlUpdate(&c, &r, 60000));
326 }
```

These are deterministic calls with explicit now values. The test does not wait 30 real seconds; a pure state machine accepts the simulated timestamp as input.

Create readable fixtures for rule transitions

test/native/test_domain/test_main.c / lines 327-350

```
327
328 // ---- alarm rules (§12) ------------------------------------------------------
329
330 static const AlarmRule kHotRule = {.ruleId = 0, .enabled = true, .metric =
  >   METRIC_AIR_TEMPERATURE,
331                                    .comparison = COMPARISON_ABOVE, .tripThreshold =
  >   40.0f,
332                                    .clearThreshold = 37.0f, .persistMs = 0,
333                                    .severity = SEVERITY_CRITICAL, .latching = true,
334                                    .actuatorId = ACTUATOR_RELAY1,
335                                    .action = ACTUATOR_ACTION_DEENERGIZE};
336
337 static Reading airTemp(float degC, MonoMs endMs) {
338   Reading r;
339   readingReset(&r, SENSOR_BME280, METRIC_AIR_TEMPERATURE, UNIT_DEG_C);
340   r.health = HEALTH_OK;
341   r.value = degC;
342   r.intervalEndMs = endMs;
343   return r;
344 }
345
346 static RuleDecision evalAt(const AlarmRule* rule, float v, MonoMs t, AlarmRuleState* st)
  >    {
347   Reading r = airTemp(v, t);
348   return evaluateRule(rule, &r, t, 1000, st);
349 }
350
```

kHotRule defines a 40-degree trip and 37-degree clear threshold. airTemp makes a usable record; evalAt keeps each test statement focused on value, time and current state.

test/native/test_domain/test_main.c / lines 351-359

```
351 static void test_alarm_trips_on_one_sample_and_clears_with_hysteresis(void) {
352   AlarmRuleState st = {0};
353   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NO_CHANGE, evalAt(&kHotRule, 39.9f, 250, &st));
354   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_TRIP, evalAt(&kHotRule, 40.0f, 500, &st));
355   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NO_CHANGE, evalAt(&kHotRule, 45.0f, 750, &st));
356   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NO_CHANGE, evalAt(&kHotRule, 38.0f, 1000, &st));
  >     // in band
357   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_CLEAR, evalAt(&kHotRule, 37.0f, 1250, &st));
358   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NO_CHANGE, evalAt(&kHotRule, 39.0f, 1500, &st));
  >     // no chatter
359 }
```

The sequence checks the exact thresholds and the interior band. TRIP is an edge in rule state, so remaining hot returns NO_CHANGE rather than generating a new trip every call.

Interrupt persistence with an invalid observation

test/native/test_domain/test_main.c / lines 360-387

```
360
361 static void test_alarm_persistence_needs_sustained_valid_samples(void) {
362   AlarmRule rule = kHotRule;
363   rule.persistMs = 1000;
364   AlarmRuleState st = {0};
365   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NO_CHANGE, evalAt(&rule, 41.0f, 1000, &st));
366   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NO_CHANGE, evalAt(&rule, 41.0f, 1999, &st));
367   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_TRIP, evalAt(&rule, 41.0f, 2000, &st));
368
369   AlarmRuleState st2 = {0};
370   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NO_CHANGE, evalAt(&rule, 41.0f, 1000, &st2));
371   Reading bad = airTemp(41.0f, 1500);
372   bad.health = HEALTH_FAULT;  // a gap restarts the window
373   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NOT_ELIGIBLE, evaluateRule(&rule, &bad, 1500,
  >   1000, &st2));
374   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NO_CHANGE, evalAt(&rule, 41.0f, 2000, &st2));
375   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_TRIP, evalAt(&rule, 41.0f, 3000, &st2));
376 }
377
378 static void test_alarm_below_rule_mirrors_above(void) {
379   AlarmRule rule = kHotRule;
380   rule.comparison = COMPARISON_BELOW;
381   rule.tripThreshold = 5.0f;
382   rule.clearThreshold = 8.0f;
383   AlarmRuleState st = {0};
384   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_TRIP, evalAt(&rule, 4.0f, 100, &st));
385   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_NO_CHANGE, evalAt(&rule, 7.0f, 200, &st));
386   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_CLEAR, evalAt(&rule, 8.5f, 300, &st));
387 }
```

The persistence test first sustains heat for 1000ms. Its second run inserts a FAULT and proves the qualifying window must begin again. The below-rule case reverses the comparison direction.

test/native/test_domain/test_main.c / lines 388-393

```
388
389 static void test_alarm_stale_limits_per_metric(void) {
390   TEST_ASSERT_EQUAL_UINT32(CFG_STALE_BME280_MS, alarmStaleLimitMs(METRIC_AIR_HUMIDITY));
391   TEST_ASSERT_EQUAL_UINT32(CFG_STALE_ORDINARY_MS,
  >   alarmStaleLimitMs(METRIC_LIGHT_LEVEL));
392   TEST_ASSERT_EQUAL_UINT32(CFG_STALE_ORDINARY_MS, alarmStaleLimitMs(METRIC_TVOC));
393 }
```

Read both the output decision and the retained AlarmRuleState. A persistence timer is state carried between calls; it cannot be understood from one threshold comparison alone.

Run a simulated release grid

test/native/test_domain/test_main.c / lines 394-421

```
394
395 // ---- schedule (§9) ------------------------------------------------------
396
397 static void test_schedule_bme280_runs_four_times_per_frame(void) {
398   ScheduleJob jobs[2];
399   scheduleJobInit(&jobs[0], CFG_FRAME_PERIOD_MS, 20, 0);
400   scheduleJobInit(&jobs[1], CFG_BME280_PERIOD_MS, CFG_BME280_BUDGET_MS, 0);
401   uint32_t frameRuns = 0, bmeRuns = 0;
402   MonoMs now = 0;
403   while (now < 1000) {
404     if (scheduleJobDue(&jobs[0], now)) { scheduleJobMarkRun(&jobs[0], now); ++frameRuns;
  >    }
405     if (scheduleJobDue(&jobs[1], now)) { scheduleJobMarkRun(&jobs[1], now); ++bmeRuns; }
406     now = scheduleEarliestDue(jobs, 2);
407   }
408   TEST_ASSERT_EQUAL_UINT32(1, frameRuns);
409   TEST_ASSERT_EQUAL_UINT32(4, bmeRuns);
410   TEST_ASSERT_EQUAL_UINT32(0, jobs[1].late);
411   TEST_ASSERT_TRUE(jobs[1].nextDueMs == 1000);
412 }
413
414 static void test_schedule_skips_missed_slots_without_burst(void) {
415   ScheduleJob j;
416   scheduleJobInit(&j, 250, 12, 500);
417   scheduleJobMarkRun(&j, 530);  // 30 ms late: keeps the 250 ms grid
418   TEST_ASSERT_TRUE(j.nextDueMs == 750);
419   TEST_ASSERT_EQUAL_UINT32(1, j.late);
420   TEST_ASSERT_EQUAL_UINT32(0, j.skipped);
421   TEST_ASSERT_EQUAL_UINT32(30, j.maxLateMs);
```

The first schedule loop advances now directly to the earliest next release, producing four BME280 runs in one ordinary frame. The delayed-job case preserves the 250ms grid and skips missed slots rather than replaying them in a burst.

test/native/test_domain/test_main.c / lines 422-428

```
422
423   scheduleJobMarkRun(&j, 3760);  // stalled ~3 s: one run, the rest skipped
424   TEST_ASSERT_TRUE(j.nextDueMs == 4000);
425   TEST_ASSERT_EQUAL_UINT32(12, j.skipped);
426   TEST_ASSERT_FALSE(scheduleJobDue(&j, 3761));
427   TEST_ASSERT_EQUAL_UINT32(2, j.runs);
428 }
```

At 3760ms, one current run is recorded and the next release becomes 4000ms. That is a scheduling policy you can calculate on paper before reading the assertions.

Test timing tolerance and available slack

test/native/test_domain/test_main.c / lines 429-443

```
429
430 static void test_schedule_jitter_within_tolerance_is_not_late(void) {
431   ScheduleJob j;
432   scheduleJobInit(&j, 1000, 10, 1000);
433   scheduleJobMarkRun(&j, 1000 + SCHEDULE_LATE_TOLERANCE_MS);
434   TEST_ASSERT_EQUAL_UINT32(0, j.late);
435   TEST_ASSERT_EQUAL_UINT32(SCHEDULE_LATE_TOLERANCE_MS, j.lastLateMs);
436   TEST_ASSERT_TRUE(j.nextDueMs == 2000);
437 }
438
439 static void test_schedule_defers_step_crossing_bme280_slot(void) {
440   TEST_ASSERT_FALSE(scheduleFitsBefore(230, 50, 250));  // DS18B20 init ~50 ms
441   TEST_ASSERT_TRUE(scheduleFitsBefore(230, 19, 250));
442   TEST_ASSERT_TRUE(scheduleFitsBefore(262, 50, 500));
443 }
```

One case lands exactly within the allowed lateness tolerance. Another asks whether a 50ms step starting at 230ms fits before a 250ms climate slot; it does not. A 19ms step does.

These helpers calculate a plan. The tests do not establish that the physical operation actually takes the budget passed to them. Measure real call durations when integrating a new driver.

Build a reusable full-core history

test/native/test_domain/test_main.c / lines 444-471

```
444
445 // ---- safety core (§12) ------------------------------------------------------
446
447 static const SafetyConfig kSafetyTest = {true, {40.0f, 120.0f, 30000, 30000, 3}, 3000};
448
449 static void setLight(Snapshot* s, float mv, Health h, MonoMs endMs) {
450   Reading* r = &s->readings[METRIC_LIGHT_LEVEL];
451   r->health = h;
452   r->value = mv;
453   r->intervalEndMs = endMs;
454   ++r->sequence;
455 }
456
457 static SafetyOutputs step(SafetyCore* c, const Snapshot* s, MonoMs now, const
  >   AlarmEvent* ev,
458                           uint8_t n) {
459   SafetyInputs in = {s, ev, n, now};
460   SafetyOutputs out;
461   safetyCoreStep(c, &in, &out);
462   return out;
463 }
464
465 // Dark from t=1 s, one light result and one 25 degC air temperature per
466 // second (the rules' input stays fresh); returns the core at t=31 s with the
467 // lamp ON (boot dwell 30 s).
468 static void lampOnAt31s(SafetyCore* c, Snapshot* s, const AlarmRule* rules, uint8_t n) {
469   safetyCoreInit(c, &kSafetyTest, rules, n, 0);
470   snapshotReset(s);
471   for (MonoMs t = 1000; t <= 31000; t += 1000) {
```

setLight increments sequence each time it represents a new result. step wraps SafetyInputs around the supplied retained state. lampOnAt31s repeatedly supplies fresh dark and temperature records until boot dwell expires.

test/native/test_domain/test_main.c / lines 472-476

```
472     setLight(s, 10.0f, HEALTH_UNCALIBRATED, t);
473     s->readings[METRIC_AIR_TEMPERATURE] = airTemp(25.0f, t);
474     step(c, s, t, NULL, 0);
475   }
476 }
```

The setup matters: a single dark sample left unchanged for 31s would become stale. The helper models continuing acquisition, which makes its eventual lamp-ON state legitimate.

Combine startup data, rail state and dwell

test/native/test_domain/test_main.c / lines 477-504

```
477
478 static void test_safety_boot_off_until_data_and_dwell(void) {
479   SafetyCore c;
480   safetyCoreInit(&c, &kSafetyTest, NULL, 0, 0);
481   SafetyOutputs o = step(&c, NULL, 100, NULL, 0);
482   TEST_ASSERT_FALSE(o.relayOn);
483   TEST_ASSERT_EQUAL_UINT8(LAMP_REASON_NO_DATA, o.lampReason);
484
485   Snapshot s;
486   snapshotReset(&s);
487   o = step(&c, &s, 200, NULL, 0);  // published, but no light result yet
488   TEST_ASSERT_FALSE(o.relayOn);
489   TEST_ASSERT_EQUAL_UINT8(LAMP_REASON_NO_DATA, o.lampReason);
490
491   setLight(&s, 10.0f, HEALTH_UNCALIBRATED, 1000);  // dark, boot dwell running
492   o = step(&c, &s, 1000, NULL, 0);
493   TEST_ASSERT_FALSE(o.relayOn);
494   TEST_ASSERT_EQUAL_UINT8(LAMP_REASON_DWELL, o.lampReason);
495 }
496
497 // WP9: the relay coil is on the external 5 V. Rail down = relay OFF, and the
498 // lamp restarts from OFF (with its dwell) once the rail is back.
499 static void test_safety_rail_down_forces_relay_off(void) {
500   SafetyCore c;
501   Snapshot s;
502   lampOnAt31s(&c, &s, NULL, 0);
503   setLight(&s, 10.0f, HEALTH_UNCALIBRATED, 32000);
504   SafetyInputs in = {&s, NULL, 0, 32000, NULL, 0, 0, true};
```

The boot test separates no Snapshot, a reset Snapshot and a usable dark observation. The rail-down case forces OFF even when the lamp was already ON, then checks the restarted OFF dwell after return.

Combine startup data, rail state and dwell / continued

test/native/test_domain/test_main.c / lines 505-517

```
505   SafetyOutputs o;
506   safetyCoreStep(&c, &in, &o);
507   TEST_ASSERT_FALSE(o.relayOn);
508   TEST_ASSERT_EQUAL(LAMP_REASON_UNPOWERED, o.lampReason);
509   TEST_ASSERT_EQUAL_STRING("no 5 V", lampReasonName(o.lampReason));
510   setLight(&s, 10.0f, HEALTH_UNCALIBRATED, 33000);
511   o = step(&c, &s, 33000, NULL, 0);  // rail back: still dark, but the OFF dwell holds
512   TEST_ASSERT_FALSE(o.relayOn);
513 }
514
515 static void test_safety_dark_turns_lamp_on_after_dwell(void) {
516   SafetyCore c;
517   Snapshot s;
```

test/native/test_domain/test_main.c / lines 518-522

```
518   lampOnAt31s(&c, &s, NULL, 0);
519   SafetyOutputs o = step(&c, &s, 31100, NULL, 0);
520   TEST_ASSERT_TRUE(o.relayOn);
521   TEST_ASSERT_EQUAL_UINT8(LAMP_REASON_LEVEL, o.lampReason);
522 }
```

These are whole-core decisions. They connect several helpers you previously read in isolation and show which reason wins when multiple conditions are present.

Freeze data while time and task wakes continue

test/native/test_domain/test_main.c / lines 523-548

```
523
524 static void test_safety_stale_light_forces_off_without_new_sample(void) {  // audit B1
525   SafetyCore c;
526   Snapshot s;
527   lampOnAt31s(&c, &s, NULL, 0);
528   // Acquisition stalls: the snapshot keeps the reading that ended at 31 s.
529   TEST_ASSERT_TRUE(step(&c, &s, 33900, NULL, 0).relayOn);
530   TEST_ASSERT_TRUE(step(&c, &s, 34000, NULL, 0).relayOn);  // exactly 3 s: still fresh
531   SafetyOutputs o = step(&c, &s, 34100, NULL, 0);
532   TEST_ASSERT_FALSE(o.relayOn);
533   TEST_ASSERT_EQUAL_UINT8(LAMP_REASON_STALE, o.lampReason);
534   // Data resumes, still dark: the OFF dwell started at the forced OFF.
535   setLight(&s, 10.0f, HEALTH_UNCALIBRATED, 35000);
536   TEST_ASSERT_FALSE(step(&c, &s, 35000, NULL, 0).relayOn);
537   setLight(&s, 10.0f, HEALTH_UNCALIBRATED, 64100);
538   TEST_ASSERT_TRUE(step(&c, &s, 64100, NULL, 0).relayOn);
539 }
540
541 static void test_safety_faults_count_per_result_not_per_wake(void) {
542   SafetyCore c;
543   Snapshot s;
544   lampOnAt31s(&c, &s, NULL, 0);
545   setLight(&s, 0.0f, HEALTH_FAULT, 32000);
546   for (MonoMs t = 32000; t < 33000; t += 100) {  // ten wakes, one FAULT result
547     TEST_ASSERT_TRUE(step(&c, &s, t, NULL, 0).relayOn);
548   }
```

The stale-light test holds the same publication while now passes its age limit. The fault-count test wakes ten times over one FAULT result and verifies that those ten wakes do not become ten new failures.

test/native/test_domain/test_main.c / lines 549-554

```
549   TEST_ASSERT_EQUAL_UINT8(LAMP_REASON_FAULT, step(&c, &s, 33000, NULL, 0).lampReason);
550   setLight(&s, 0.0f, HEALTH_FAULT, 33000);
551   TEST_ASSERT_TRUE(step(&c, &s, 33000, NULL, 0).relayOn);
552   setLight(&s, 0.0f, HEALTH_FAULT, 34000);  // third in a row (faultsToOff = 3)
553   TEST_ASSERT_FALSE(step(&c, &s, 34000, NULL, 0).relayOn);
554 }
```

In this Safety implementation, updateLamp recognises a new light result by a changed intervalEndMs, stored in lastLightEndMs. It does not use sequence for that decision. The test fixture changes both fields for a new result, so inspect the implementation before attributing the behaviour to one of them. Repeated task wakes leave both unchanged.

Keep active, latched and missing-input states distinct

test/native/test_domain/test_main.c / lines 555-582

```
555
556 static AlarmEvent hotEvent(RuleDecision d, float v, MonoMs t) {
557   AlarmEvent e;
558   e.ruleId = 0;
559   e.decision = d;
560   e.sample = airTemp(v, t);
561   return e;
562 }
563
564 static void test_safety_latched_interlock_beats_lamp_until_reset(void) {
565   SafetyCore c;
566   Snapshot s;
567   lampOnAt31s(&c, &s, &kHotRule, 1);
568   const AlarmEvent trip = hotEvent(RULE_DECISION_TRIP, 41.0f, 31200);
569   SafetyOutputs o = step(&c, &s, 31200, &trip, 1);
570   TEST_ASSERT_FALSE(o.relayOn);
571   TEST_ASSERT_TRUE(o.interlock);
572   TEST_ASSERT_EQUAL_UINT8(LAMP_REASON_INTERLOCK, o.lampReason);
573   TEST_ASSERT_EQUAL_HEX16(0x0001, o.latchedMask);
574
575   TEST_ASSERT_FALSE(safetyCoreResetLatch(&c, 0));  // still active: refused
576   const AlarmEvent clear = hotEvent(RULE_DECISION_CLEAR, 36.0f, 40000);
577   setLight(&s, 10.0f, HEALTH_UNCALIBRATED, 40000);
578   o = step(&c, &s, 40000, &clear, 1);
579   TEST_ASSERT_FALSE(o.relayOn);  // cleared but latched
580   TEST_ASSERT_EQUAL_HEX16(0x0001, o.latchedMask);
581   TEST_ASSERT_EQUAL_HEX16(0x0000, o.activeMask);
582
```

A hot event sets the interlock and latch. A clear event removes active state but leaves the latch until a reset. The next test keeps light fresh while temperature ages beyond its limit.

test/native/test_domain/test_main.c / lines 583-595

```
583   TEST_ASSERT_TRUE(safetyCoreResetLatch(&c, 0));
584   setLight(&s, 10.0f, HEALTH_UNCALIBRATED, 41000);
585   s.readings[METRIC_AIR_TEMPERATURE] = airTemp(36.0f, 41000);
586   TEST_ASSERT_FALSE(step(&c, &s, 41000, NULL, 0).relayOn);  // OFF dwell from the
  >   interlock
587   setLight(&s, 10.0f, HEALTH_UNCALIBRATED, 61300);
588   s.readings[METRIC_AIR_TEMPERATURE] = airTemp(36.0f, 61300);
589   TEST_ASSERT_TRUE(step(&c, &s, 61300, NULL, 0).relayOn);
590 }
591
592 // Audit P2: an enabled relay-interlock rule without fresh input cannot see a
593 // fault, so the relay is held OFF (not an interlock: nothing tripped).
594 static void test_safety_missing_interlock_input_holds_relay_off(void) {
595   SafetyCore c;
```

Keep active, latched and missing-input states distinct / continued

test/native/test_domain/test_main.c / lines 596-607

```
  Snapshot s;
  lampOnAt31s(&c, &s, &kHotRule, 1);
  SafetyOutputs o = step(&c, &s, 31000, NULL, 0);
  TEST_ASSERT_TRUE(o.relayOn);
  setLight(&s, 10.0f, HEALTH_UNCALIBRATED, 32000);
  o = step(&c, &s, 32100, NULL, 0);  // temperature 1.1 s old, light fresh and dark
  TEST_ASSERT_EQUAL_HEX16(0x0001, o.staleMask);
  TEST_ASSERT_FALSE(o.interlock);
  TEST_ASSERT_FALSE(o.relayOn);
  TEST_ASSERT_EQUAL_UINT8(LAMP_REASON_INPUT_MISSING, o.lampReason);
  TEST_ASSERT_EQUAL_STRING("input missing", lampReasonName(o.lampReason));
}
```

Missing protective input forces OFF without claiming a threshold trip occurred. The separate stale mask and lamp reason preserve that distinction for diagnostics and the client.

Simulate the queue-overflow race in a fixed order

test/native/test_domain/test_main.c / lines 608-634

```
608
609 static AlarmEvent stickyTrip(float v) {
610   AlarmEvent e;
611   memset(&e, 0, sizeof(e));
612   e.ruleId = 0;
613   e.decision = RULE_DECISION_TRIP;
614   e.sample = airTemp(v, 1000);
615   return e;
616 }
617
618 // Audit P1: the send fails, Safety drains before the "lost" mark: the TRIP
619 // must still come out of the sticky slot on the next pass.
620 static void test_sticky_trip_survives_drain_during_failed_send(void) {
621   StickySlot slot;
622   memset(&slot, 0, sizeof(slot));
623   const bool sawTrip[1] = {false};
624   AlarmEvent out[2];
625   const AlarmEvent trip = stickyTrip(45.0f);
626   stickyBegin(&slot, &kHotRule, &trip);
627   TEST_ASSERT_EQUAL_UINT8(0, stickyCollect(&slot, 1, sawTrip, out));  // in flight:
  >   untouched
628   stickyEnd(&slot, false);
629   TEST_ASSERT_EQUAL_UINT8(1, stickyCollect(&slot, 1, sawTrip, out));
630   TEST_ASSERT_EQUAL_UINT8(RULE_DECISION_TRIP, out[0].decision);
631   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 45.0f, out[0].sample.value);
632   TEST_ASSERT_EQUAL_UINT8(0, stickyCollect(&slot, 1, sawTrip, out));  // delivered once
633 }
634
```

stickyBegin marks the in-flight trip before the send outcome is known. The first test drains during that interval, then reports send failure and confirms a later collection still recovers the trip once.

Simulate the queue-overflow race in a fixed order / continued

test/native/test_domain/test_main.c / lines 635-645

```
635 static void test_sticky_queued_record_is_not_duplicated(void) {
636   StickySlot slot;
637   memset(&slot, 0, sizeof(slot));
638   AlarmEvent out[2];
639   const AlarmEvent trip = stickyTrip(41.0f);
640   stickyBegin(&slot, &kHotRule, &trip);
641   stickyEnd(&slot, true);
642   const bool sawTrip[1] = {true};
643   TEST_ASSERT_EQUAL_UINT8(0, stickyCollect(&slot, 1, sawTrip, out));
644   TEST_ASSERT_FALSE(slot.tripSinceDrain);
645 }
```

The companion case confirms that a trip already seen in the queue is not emitted again from sticky storage. This controlled interleaving is more informative than hoping a race appears during a random long run.

Reject old power epochs and invalid arithmetic inputs

test/native/test_domain/test_main.c / lines 646-673

```
646
647 // Audit P2: consumers re-check 5 V readings against the live rail epoch.
648 static void test_snapshot_gate_rail_marks_old_epoch_unpowered(void) {
649   Snapshot s;
650   snapshotReset(&s);
651   s.readings[METRIC_PH].health = HEALTH_UNCALIBRATED;
652   s.readings[METRIC_PH].powerEpoch = 4;
653   s.readings[METRIC_TDS].health = HEALTH_OK;
654   s.readings[METRIC_TDS].powerEpoch = 5;
655   s.readings[METRIC_AIR_TEMPERATURE].health = HEALTH_OK;  // 3.3 V: never gated
656   s.readings[METRIC_AIR_TEMPERATURE].powerEpoch = 1;
657   snapshotGateRail(&s, true, 5);
658   TEST_ASSERT_EQUAL_UINT8(HEALTH_UNPOWERED, s.readings[METRIC_PH].health);
659   TEST_ASSERT_EQUAL_UINT8(HEALTH_OK, s.readings[METRIC_TDS].health);
660   TEST_ASSERT_EQUAL_UINT8(HEALTH_OK, s.readings[METRIC_AIR_TEMPERATURE].health);
661   TEST_ASSERT_EQUAL_UINT8(HEALTH_STARTING, s.readings[METRIC_WATER_DISTANCE].health);
662   snapshotGateRail(&s, false, 5);  // same epoch, rail invalid now
663   TEST_ASSERT_EQUAL_UINT8(HEALTH_UNPOWERED, s.readings[METRIC_TDS].health);
664 }
665
666 static void test_fits_reject_non_finite_values(void) {
667   LinearCalibration c;
668   TEST_ASSERT_FALSE(linearFromTwoPoints(NAN, 7.0f, 2000.0f, 4.0f, 1, &c));
669   TEST_ASSERT_FALSE(c.valid);
670   TEST_ASSERT_FALSE(linearFromTwoPoints(1000.0f, INFINITY, 2000.0f, 4.0f, 1, &c));
671   TEST_ASSERT_FALSE(linearGainFromPoint(INFINITY, 1.0f, 0.0f, 1e30f, 1, &c));
672   TEST_ASSERT_FALSE(phFitFromBuffers(NAN, 6.86f, 2000.0f, 4.01f, 0.001f, 0.02f, 1, &c));
673   c.slope = NAN;
```

Rail gating leaves current-epoch TDS and retained-rail air valid while invalidating old-epoch pH. It also preserves STARTING. Fit tests then reject NaN, infinity and a non-finite stored coefficient.

test/native/test_domain/test_main.c / lines 674-676

```
674   c.offset = 0.0f;
675   TEST_ASSERT_FALSE(linearCalibrationFinite(&c));
676 }
```

CRC checks byte integrity, while finite checks numerical plausibility. A blob can have a freshly computed CRC over NaN bytes and still be unsuitable for computation.

Preserve event history within one processed batch

test/native/test_domain/test_main.c / lines 677-702

```
677
678 static void test_safety_trip_and_clear_in_one_batch_still_deenergises(void) {
679   AlarmRule rule = kHotRule;
680   rule.latching = false;
681   SafetyCore c;
682   Snapshot s;
683   lampOnAt31s(&c, &s, &rule, 1);
684   const AlarmEvent batch[2] = {hotEvent(RULE_DECISION_TRIP, 50.0f, 31100),
685                                hotEvent(RULE_DECISION_CLEAR, 30.0f, 31150)};
686   SafetyOutputs o = step(&c, &s, 31200, batch, 2);
687   TEST_ASSERT_FALSE(o.relayOn);  // the unhandled TRIP is not erased by the CLEAR
688   TEST_ASSERT_TRUE(o.interlock);
689   TEST_ASSERT_EQUAL_HEX16(0x0000, o.activeMask);
690   TEST_ASSERT_EQUAL_UINT32(1, c.rule[0].trips);
691   TEST_ASSERT_FLOAT_WITHIN(1e-6f, 50.0f, c.rule[0].worst);
692 }
693
694 static void test_safety_disabled_rule_and_auto_switch(void) {
695   AlarmRule rule = kHotRule;
696   rule.enabled = false;
697   SafetyCore c;
698   Snapshot s;
699   lampOnAt31s(&c, &s, &rule, 1);
700   const AlarmEvent trip = hotEvent(RULE_DECISION_TRIP, 41.0f, 31200);
701   TEST_ASSERT_TRUE(step(&c, &s, 31200, &trip, 1).relayOn);  // disabled: no interlock
702
```

A nonlatching trip followed by clear in the same batch must still force an OFF action on that pass. The next tests distinguish disabled rules, disabled automatic control and stale protected inputs.

test/native/test_domain/test_main.c / lines 703-717

```
703   SafetyConfig off = kSafetyTest;
704   off.lampAutoEnabled = false;
705   safetyCoreInit(&c, &off, NULL, 0, 0);
706   snapshotReset(&s);
707   SafetyOutputs o = {0};
708   for (MonoMs t = 1000; t <= 31000; t += 1000) {
709     setLight(&s, 10.0f, HEALTH_UNCALIBRATED, t);
710     o = step(&c, &s, t, NULL, 0);
711   }
712   TEST_ASSERT_TRUE(o.lampWantsOn);
713   TEST_ASSERT_FALSE(o.relayOn);
714   TEST_ASSERT_EQUAL_UINT8(LAMP_REASON_DISABLED, o.lampReason);
715 }
716
717 static void test_safety_reports_stale_rule_inputs(void) {
```

Preserve event history within one processed batch / continued

test/native/test_domain/test_main.c / lines 718-726

```
  SafetyCore c;
  Snapshot s;
  safetyCoreInit(&c, &kSafetyTest, &kHotRule, 1, 0);
  snapshotReset(&s);
  TEST_ASSERT_EQUAL_HEX16(0x0001, step(&c, &s, 100, NULL, 0).staleMask);  // never
  > measured
  s.readings[METRIC_AIR_TEMPERATURE] = airTemp(25.0f, 1000);
  TEST_ASSERT_EQUAL_HEX16(0x0000, step(&c, &s, 1500, NULL, 0).staleMask);
  TEST_ASSERT_EQUAL_HEX16(0x0001, step(&c, &s, 2100, NULL, 0).staleMask);  // > 1 s
}
```

The final active mask alone would lose the fact that an unhandled trip occurred earlier in the batch. This is why Safety consumes event history as well as latest state.

Register every case with the test runner

test/native/test_domain/test_main.c / lines 727-754

```
727
728 int main(int argc, char** argv) {
729   (void)argc;
730   (void)argv;
731   UNITY_BEGIN();
732   RUN_TEST(test_reset_reading_is_starting_and_not_actionable);
733   RUN_TEST(test_ok_reading_becomes_stale_after_limit);
734   RUN_TEST(test_stale_reading_is_not_actionable);
735   RUN_TEST(test_fault_is_not_masked_by_staleness);
736   RUN_TEST(test_divider_recovery_uses_nominal_ratios);
737   RUN_TEST(test_two_point_fit_maps_buffers);
738   RUN_TEST(test_trimmed_mean_rejects_spikes);
739   RUN_TEST(test_snapshot_reset_indexes_by_metric);
740   RUN_TEST(test_production_ready_requires_power_sense);
741   RUN_TEST(test_snapshot_reset_sets_identity);
742   RUN_TEST(test_alarm_rule_ignores_stale_sample);
743   RUN_TEST(test_sgp30_crc_matches_datasheet_vector);
744   RUN_TEST(test_sgp30_words_round_trip_and_reject_bad_crc);
745   RUN_TEST(test_sgp30_humidity_fixed_point);
746   RUN_TEST(test_ultrasonic_echo_width_to_mm);
747   RUN_TEST(test_ultrasonic_range_is_2_to_400_cm);
748   RUN_TEST(test_tds_datasheet_curve);
749   RUN_TEST(test_tds_temperature_compensation);
750   RUN_TEST(test_onewire_crc_matches_maxim_vector);
751   RUN_TEST(test_ds18b20_temperature_table);
752   RUN_TEST(test_ds18b20_scratchpad_checks);
753   RUN_TEST(test_lamp_hysteresis_band);
754   RUN_TEST(test_lamp_dwell_holds_changes);
```

UNITY_BEGIN initialises the suite, each RUN_TEST invokes one named case, and UNITY_END supplies the result. argc and argv are explicitly unused.

test/native/test_domain/test_main.c / lines 755-768

```
755   RUN_TEST(test_lamp_uses_pin_mv_after_lux_fit);
756   RUN_TEST(test_lamp_faults_hold_then_force_off);
757   RUN_TEST(test_lamp_saturated_counts_as_bright);
758   RUN_TEST(test_alarm_trips_on_one_sample_and_clears_with_hysteresis);
759   RUN_TEST(test_alarm_persistence_needs_sustained_valid_samples);
760   RUN_TEST(test_alarm_below_rule_mirrors_above);
761   RUN_TEST(test_alarm_stale_limits_per_metric);
762   RUN_TEST(test_schedule_bme280_runs_four_times_per_frame);
763   RUN_TEST(test_schedule_skips_missed_slots_without_burst);
764   RUN_TEST(test_schedule_jitter_within_tolerance_is_not_late);
765   RUN_TEST(test_schedule_defers_step_crossing_bme280_slot);
766   RUN_TEST(test_safety_boot_off_until_data_and_dwell);
767   RUN_TEST(test_safety_dark_turns_lamp_on_after_dwell);
768   RUN_TEST(test_safety_rail_down_forces_relay_off);
```

Register every case with the test runner / continued

test/native/test_domain/test_main.c / lines 769-781

```
769    RUN_TEST(test_safety_stale_light_forces_off_without_new_sample);
770    RUN_TEST(test_safety_faults_count_per_result_not_per_wake);
771    RUN_TEST(test_safety_latched_interlock_beats_lamp_until_reset);
772    RUN_TEST(test_safety_trip_and_clear_in_one_batch_still_deenergises);
773    RUN_TEST(test_safety_disabled_rule_and_auto_switch);
774    RUN_TEST(test_safety_reports_stale_rule_inputs);
775    RUN_TEST(test_safety_missing_interlock_input_holds_relay_off);
776    RUN_TEST(test_sticky_trip_survives_drain_during_failed_send);
777    RUN_TEST(test_sticky_queued_record_is_not_duplicated);
778    RUN_TEST(test_snapshot_gate_rail_marks_old_epoch_unpowered);
779    RUN_TEST(test_fits_reject_non_finite_values);
780    return UNITY_END();
781 }
```

This registration list is part of coverage: defining a test function without adding RUN_TEST would leave it unexecuted. The native domain suite registers 48 cases; each is a worked property of the code you read, rather than a claim about all possible executions.

Choose shorter test timings without changing the algorithm

These tests run power transitions with an explicit clock and explicit acknowledgement inputs. The hardware ISR and task scheduler are replaced by controlled input changes.

Use the state-machine chapter beside this file and predict each action before reading its assertion.

test/native/test_power/test_main.c / lines 1-26

```
1  // Host unit tests for the WP9 power-mode state machine (pio test -e native):
2  // boot paths, debounce, the quiesce -> release -> grant lease order, Wi-Fi
3  // acknowledgement, DEGRADED on missing acknowledgements, abort on a new loss
4  // during the return, and the report-only mode of the kill switch.
5  #include <string.h>
6  #include <unity.h>
7
8  #include "domain/power_mode.h"
9
10 void setUp(void) {}
11 void tearDown(void) {}
12
13 static const PowerModeConfig kCfg = {50, 500, 200, 2000, true};
14
15 static PowerModeFsm s_f;
16 static PowerModeActions s_a;
17 static PowerModeInputs s_in;
18
19 static void begin(bool power, const PowerModeConfig* cfg) {
20   powerModeInit(&s_f, cfg);
21   memset(&s_in, 0, sizeof(s_in));
22   s_in.externalPower = power;
23   s_in.now = 1000;
24   powerModeStep(&s_f, &s_in, &s_a);
25 }
26
```

kCfg uses 50ms loss debounce, 500ms stable return, 200ms release wait and 2000ms network-off wait. begin resets state and performs the first step at 1000ms; at advances the supplied clock.

test/native/test_power/test_main.c / lines 27-30

```
27 static void at(MonoMs now) {
28   s_in.now = now;
29   powerModeStep(&s_f, &s_in, &s_a);
30 }
```

Test constants are intentionally different from some production constants. The invariant under test is ordering and boundary handling, not the exact wall-clock duration on your board.

Check both initial power conditions

test/native/test_power/test_main.c / lines 31-55

```
31
32 static void test_boot_with_power_grants_normal_and_restores_rail_when_stable(void) {
33   begin(true, &kCfg);
34   TEST_ASSERT_EQUAL(POWER_MODE_NORMAL, s_f.mode);
35   TEST_ASSERT_EQUAL(POWER_WORKER_NORMAL, s_a.grant);
36   TEST_ASSERT_EQUAL_UINT32(1, s_a.grantGen);
37   TEST_ASSERT_FALSE(s_a.restoreRail);  // not stable yet
38   at(1499);
39   TEST_ASSERT_FALSE(s_a.restoreRail);
40   at(1500);
41   TEST_ASSERT_TRUE(s_a.restoreRail);
42   s_in.railValid = true;
43   at(1600);
44   TEST_ASSERT_FALSE(s_a.restoreRail);
45 }
46
47 static void test_boot_without_power_takes_the_low_power_path(void) {
48   begin(false, &kCfg);
49   TEST_ASSERT_EQUAL(POWER_WORKER_LOW, s_a.grant);
50   TEST_ASSERT_TRUE(s_a.wifiStop);
51   TEST_ASSERT_EQUAL(POWER_MODE_TRANSITION_TO_LOW, s_f.mode);
52   s_in.networkOff = true;
53   at(1010);
54   TEST_ASSERT_EQUAL(POWER_MODE_LOW_POWER, s_f.mode);
55 }
```

With external power, the normal lease is granted but rail restoration waits for stable return timing. Without it, the low worker is granted and the network-off acknowledgement completes the transition.

Initialisation must choose an owner even though no previous worker can release a lease. Boot is therefore worth testing separately from runtime handover.

Trace the complete release-before-grant handshake

test/native/test_power/test_main.c / lines 56-83

```
56
57 // Full loss and return, every acknowledgement on time.
58 static void test_loss_and_return_hand_the_lease_over_in_order(void) {
59   begin(true, &kCfg);
60   s_in.railValid = true;
61   at(2000);
62   // The ISR invalidates the rail on the first edge; the level is low.
63   s_in.externalPower = false;
64   s_in.railValid = false;
65   at(2010);
66   TEST_ASSERT_EQUAL(POWER_MODE_NORMAL, s_f.mode);  // debouncing
67   TEST_ASSERT_EQUAL(POWER_WORKER_NONE, s_a.quiesce);
68   at(2060);
69   TEST_ASSERT_EQUAL(POWER_MODE_TRANSITION_TO_LOW, s_f.mode);
70   TEST_ASSERT_EQUAL(POWER_WORKER_NORMAL, s_a.quiesce);
71   TEST_ASSERT_EQUAL_UINT32(1, s_a.quiesceGen);
72   TEST_ASSERT_TRUE(s_a.wifiStop);
73   at(2100);  // not released yet: no grant
74   TEST_ASSERT_EQUAL(POWER_WORKER_NONE, s_a.grant);
75   s_in.releasedGen = 1;
76   at(2120);
77   TEST_ASSERT_EQUAL(POWER_WORKER_LOW, s_a.grant);
78   TEST_ASSERT_EQUAL_UINT32(2, s_a.grantGen);
79   TEST_ASSERT_EQUAL(POWER_MODE_TRANSITION_TO_LOW, s_f.mode);  // Wi-Fi not off yet
80   s_in.networkOff = true;
81   at(2200);
82   TEST_ASSERT_EQUAL(POWER_MODE_LOW_POWER, s_f.mode);
83
```

Loss first changes input, then passes debounce, then requests normal-worker quiescence. Before releasedGen matches, no new grant appears. After release, the low worker gets a new generation; network acknowledgement completes low-power entry.

Trace the complete release-before-grant handshake / continued

test/native/test_power/test_main.c / lines 84-96

```
84    // Power back: 500 ms stable before anything moves.
85    s_in.externalPower = true;
86    s_in.networkOff = false;
87    at(5000);
88    at(5499);
89    TEST_ASSERT_EQUAL(POWER_MODE_LOW_POWER, s_f.mode);
90    at(5500);
91    TEST_ASSERT_EQUAL(POWER_MODE_TRANSITION_TO_NORMAL, s_f.mode);
92    TEST_ASSERT_EQUAL(POWER_WORKER_LOW, s_a.quiesce);
93    TEST_ASSERT_EQUAL_UINT32(2, s_a.quiesceGen);
94    s_in.releasedGen = 2;
95    at(5520);
96    TEST_ASSERT_TRUE(s_a.restoreRail);
```

test/native/test_power/test_main.c / lines 97-101

```
97    TEST_ASSERT_EQUAL(POWER_WORKER_NORMAL, s_a.grant);
98    TEST_ASSERT_EQUAL_UINT32(3, s_a.grantGen);
99    TEST_ASSERT_TRUE(s_a.wifiStart);
100   TEST_ASSERT_EQUAL(POWER_MODE_NORMAL, s_f.mode);
101 }
```

Return waits 500ms, requests low-worker release and only then grants the normal worker and starts Wi-Fi. Read the generations 1, 2 and 3 beside the same worker names: identity and lease history are different.

Separate a mode glitch from invalid measurement history

test/native/test_power/test_main.c / lines 102-118

```
102
103 static void test_short_glitch_does_not_change_mode_but_rail_waits_for_stability(void) {
104   begin(true, &kCfg);
105   s_in.railValid = true;
106   at(2000);
107   s_in.externalPower = false;
108   s_in.railValid = false;  // the ISR saw it
109   at(2010);
110   s_in.externalPower = true;
111   at(2030);  // 20 ms low: below the 50 ms debounce
112   TEST_ASSERT_EQUAL(POWER_MODE_NORMAL, s_f.mode);
113   TEST_ASSERT_EQUAL(POWER_WORKER_NONE, s_a.quiesce);
114   at(2529);
115   TEST_ASSERT_FALSE(s_a.restoreRail);
116   at(2530);
117   TEST_ASSERT_TRUE(s_a.restoreRail);
118 }
```

A 20ms low pulse is shorter than the 50ms debounce, so no mode handover occurs. Rail restoration still waits for a fresh 500ms high period.

A mode remaining NORMAL does not imply every 5V sample stayed valid. The rail epoch and stability timer preserve evidence of the disturbance independently of the human-readable mode.

Hold back one acknowledgement at a time

test/native/test_power/test_main.c / lines 119-146

```
119
120 static void test_missing_release_reports_degraded_and_never_grants_early(void) {
121   begin(true, &kCfg);
122   s_in.externalPower = false;
123   at(2000);
124   at(2050);  // quiesce sent
125   at(2251);  // 200 ms later, still not released
126   TEST_ASSERT_EQUAL(POWER_MODE_DEGRADED, s_f.mode);
127   TEST_ASSERT_EQUAL(POWER_WORKER_NONE, s_a.grant);
128   at(4000);
129   TEST_ASSERT_EQUAL(POWER_WORKER_NONE, s_a.grant);
130   s_in.releasedGen = 1;  // late, but it came
131   s_in.networkOff = true;
132   at(4010);
133   TEST_ASSERT_EQUAL(POWER_WORKER_LOW, s_a.grant);
134   TEST_ASSERT_EQUAL(POWER_MODE_LOW_POWER, s_f.mode);
135 }
136
137 static void test_wifi_not_off_in_time_reports_degraded(void) {
138   begin(true, &kCfg);
139   s_in.externalPower = false;
140   at(2000);
141   at(2050);
142   s_in.releasedGen = 1;
143   at(2060);
144   TEST_ASSERT_EQUAL(POWER_MODE_TRANSITION_TO_LOW, s_f.mode);
145   at(4051);  // 2 s after the stop request
146   TEST_ASSERT_EQUAL(POWER_MODE_DEGRADED, s_f.mode);
```

The first test withholds worker release beyond 200ms and checks DEGRADED with no early grant. It then supplies late release and allows recovery. The next test supplies release but withholds network-off.

test/native/test_power/test_main.c / lines 147-150

```
147   s_in.networkOff = true;
148   at(4100);
149   TEST_ASSERT_EQUAL(POWER_MODE_LOW_POWER, s_f.mode);
150 }
```

A timeout exposes a failed timing promise; it must not become permission for two workers to access the same hardware. These assertions check that distinction directly.

Reverse direction during a transition

test/native/test_power/test_main.c / lines 151-173

```
151
152 static void test_new_loss_during_return_aborts_it(void) {
153   begin(false, &kCfg);
154   s_in.networkOff = true;
155   at(1010);
156   TEST_ASSERT_EQUAL(POWER_MODE_LOW_POWER, s_f.mode);
157   s_in.externalPower = true;
158   at(2000);
159   at(2500);
160   TEST_ASSERT_EQUAL(POWER_MODE_TRANSITION_TO_NORMAL, s_f.mode);
161   TEST_ASSERT_EQUAL_UINT32(1, s_a.quiesceGen);
162   s_in.externalPower = false;  // lost again before the release
163   at(2510);
164   TEST_ASSERT_EQUAL(POWER_MODE_TRANSITION_TO_LOW, s_f.mode);
165   s_in.releasedGen = 1;
166   at(2520);
167   TEST_ASSERT_EQUAL(POWER_WORKER_LOW, s_a.grant);  // the low-power worker again, new
  >   generation
168   TEST_ASSERT_EQUAL_UINT32(2, s_a.grantGen);
169   TEST_ASSERT_FALSE(s_a.restoreRail);
170   TEST_ASSERT_FALSE(s_a.wifiStart);
171   TEST_ASSERT_EQUAL(POWER_MODE_LOW_POWER, s_f.mode);
172 }
173
```

A new loss while returning to normal aborts that return. The eventual grant goes back to the low worker with a new generation, without restoring the rail or starting Wi-Fi.

Reverse direction during a transition / continued

test/native/test_power/test_main.c / lines 174-190

```
174 static void test_kill_switch_only_reports_the_level(void) {
175   const PowerModeConfig off = {50, 500, 200, 2000, false};
176   begin(true, &off);
177   TEST_ASSERT_EQUAL(POWER_WORKER_NORMAL, s_a.grant);
178   TEST_ASSERT_TRUE(s_a.restoreRail);
179   TEST_ASSERT_EQUAL(POWER_MODE_NORMAL, s_f.mode);
180   s_in.railValid = true;
181   s_in.externalPower = false;
182   at(2000);
183   at(2050);
184   TEST_ASSERT_EQUAL(POWER_MODE_LOW_POWER, s_f.mode);
185   TEST_ASSERT_EQUAL(POWER_WORKER_NONE, s_a.quiesce);
186   TEST_ASSERT_FALSE(s_a.wifiStop);
187   s_in.externalPower = true;
188   at(2060);
189   TEST_ASSERT_EQUAL(POWER_MODE_NORMAL, s_f.mode);
190   TEST_ASSERT_EQUAL_STRING("TRANSITION_TO_NORMAL",
  >   powerModeName(POWER_MODE_TRANSITION_TO_NORMAL));
```

test/native/test_power/test_main.c / lines 191-191

```
191 }
```

The following kill-switch case disables transition actions while still reporting input-related mode. Reporting and acting are separate outputs, so a mode label alone does not prove that a handover happened.

Use epochs to reveal an unsampled pulse

test/native/test_power/test_main.c / lines 192-219

```
192
193 // Audit P1: a LOW-HIGH pulse between two ticks is only visible as a new rail
194 // epoch. It must restart the stability timer, and the restore names the epoch.
195 static void test_unsampled_glitch_restarts_stability_from_the_new_epoch(void) {
196   begin(true, &kCfg);
197   s_in.railValid = true;
198   s_in.railEpoch = 1;
199   at(2000);
200   s_in.railValid = false;  // the ISR saw both edges; the level is HIGH again
201   s_in.railEpoch = 3;
202   at(2010);
203   TEST_ASSERT_FALSE(s_a.restoreRail);  // the old HIGH timer no longer counts
204   at(2509);
205   TEST_ASSERT_FALSE(s_a.restoreRail);
206   at(2510);
207   TEST_ASSERT_TRUE(s_a.restoreRail);
208   TEST_ASSERT_EQUAL_UINT32(3, s_a.restoreEpoch);
209 }
210
211 static void test_muted_isr_holds_the_rail_invalid(void) {
212   begin(true, &kCfg);
213   s_in.railValid = false;
214   s_in.edgesMissed = true;  // muted after the first edge: later edges are not counted
215   at(3000);
216   TEST_ASSERT_FALSE(s_a.restoreRail);
217   s_in.edgesMissed = false;  // re-armed
218   at(3010);
219   at(3509);
```

A low-high pulse between task samples can leave the level high at both observations. Changing railEpoch still restarts stability. edgesMissed prevents restoration while ISR muting means edge history may be incomplete.

test/native/test_power/test_main.c / lines 220-223

```
220   TEST_ASSERT_FALSE(s_a.restoreRail);
221   at(3510);
222   TEST_ASSERT_TRUE(s_a.restoreRail);
223 }
```

This is a concrete reason to retain history beside a boolean level. Two high readings cannot prove that the signal stayed high between them.

Allow owner restoration before measurement eligibility

test/native/test_power/test_main.c / lines 224-244

```
224
225 static void test_glitch_during_return_defers_the_rail_restore(void) {
226   begin(false, &kCfg);
227   s_in.networkOff = true;
228   at(1010);
229   s_in.externalPower = true;
230   at(2000);
231   at(2500);
232   TEST_ASSERT_EQUAL(POWER_MODE_TRANSITION_TO_NORMAL, s_f.mode);
233   s_in.railEpoch = 2;  // an unsampled glitch while quiescing
234   at(2505);
235   s_in.releasedGen = 1;
236   at(2510);
237   TEST_ASSERT_EQUAL(POWER_WORKER_NORMAL, s_a.grant);
238   TEST_ASSERT_FALSE(s_a.restoreRail);  // 5 ms of stability is not enough
239   at(3004);
240   TEST_ASSERT_FALSE(s_a.restoreRail);
241   at(3005);
242   TEST_ASSERT_TRUE(s_a.restoreRail);
243   TEST_ASSERT_EQUAL_UINT32(2, s_a.restoreEpoch);
244 }
```

A glitch while waiting for release changes the epoch. Normal ownership can return after release, but restoreRail remains false until the new stability interval completes.

The test checks restoreEpoch as well as the action bit. The eventual restoration must name the history it qualified, so an older acknowledgement cannot revive a newer failed rail.

Execute all power-transition properties

test/native/test_power/test_main.c / lines 245-262

```
245
246 int main(int argc, char** argv) {
247   (void)argc;
248   (void)argv;
249   UNITY_BEGIN();
250   RUN_TEST(test_boot_with_power_grants_normal_and_restores_rail_when_stable);
251   RUN_TEST(test_boot_without_power_takes_the_low_power_path);
252   RUN_TEST(test_loss_and_return_hand_the_lease_over_in_order);
253   RUN_TEST(test_short_glitch_does_not_change_mode_but_rail_waits_for_stability);
254   RUN_TEST(test_missing_release_reports_degraded_and_never_grants_early);
255   RUN_TEST(test_wifi_not_off_in_time_reports_degraded);
256   RUN_TEST(test_new_loss_during_return_aborts_it);
257   RUN_TEST(test_kill_switch_only_reports_the_level);
258   RUN_TEST(test_unsampled_glitch_restarts_stability_from_the_new_epoch);
259   RUN_TEST(test_muted_isr_holds_the_rail_invalid);
260   RUN_TEST(test_glitch_during_return_defers_the_rail_restore);
261   return UNITY_END();
262 }
```

The runner registers 11 cases. They cover boot, ordinary handover, short glitches, absent acknowledgements, reversal, disabled actions and epoch-based recovery.

These tests prove the pure transition decisions for the supplied traces. GPIO qualification, actual cancellation duration and radio-mode acknowledgement must still be observed through the hardware paths in their earlier chapters.

Exercise delivery boundaries independently of JSON

This suite combines byte-level examples with configuration and Safety contracts. The input strings and expected output fragments make it useful as a client-message reference.

Read long escaped C strings in two layers: C first constructs bytes; the JSON parser then interprets those bytes.

test/native/test_protocol/test_main.c / lines 1-27

```
// Host unit tests for the WP8 domain modules (pio test -e native):
// frame reassembly, contract v2 encoders and command parser, configuration
// blob (schema 2, calibration, v1 migration), calibration fits and capture,
// event journal, backoff, and the Safety command / disabled-sensor rules.
#include <math.h>
#include <stdio.h>
#include <string.h>
#include <unity.h>

#include <stddef.h>

#include "domain/backoff.h"
#include "domain/cal_capture.h"
#include "domain/calibration.h"
#include "domain/config_blob.h"
#include "domain/event_journal.h"
#include "domain/frame_codec.h"
#include "domain/protocol_codec.h"
#include "domain/safety_core.h"
#include "domain/sensor_set.h"

void setUp(void) {}
void tearDown(void) {}

static FrameAssembler s_fa;
static char s_out[4096];

```

test/native/test_protocol/test_main.c / lines 28-39

```
static ProtocolError parse(const char* json, Command* cmd) {
  return protocolParseCommand((const uint8_t*)json, strlen(json), cmd);
}

// ---------------------------------------------------------------- frames

static void test_frame_split_across_packets(void) {
  const uint8_t frame[] = {5, 0, 'h', 'e', 'l', 'l', 'o'};
  frameAssemblerInit(&s_fa);
  size_t used;
  const uint8_t* payload;
  size_t n;
```

Exercise delivery boundaries independently of JSON / continued

test/native/test_protocol/test_main.c / lines 40-55

```
  for (size_t i = 0; i < sizeof(frame) - 1; ++i) {
    TEST_ASSERT_EQUAL(FRAME_NEED_MORE, frameAssemblerFeed(&s_fa, &frame[i], 1, &used,
  &payload, &n));
    TEST_ASSERT_EQUAL(1, used);
  }
  TEST_ASSERT_EQUAL(FRAME_READY,
                    frameAssemblerFeed(&s_fa, &frame[sizeof(frame) - 1], 1, &used,
  &payload, &n));
  TEST_ASSERT_EQUAL(5, n);
  TEST_ASSERT_EQUAL_MEMORY("hello", payload, 5);
}

static void test_frame_two_in_one_packet(void) {
  const uint8_t data[] = {2, 0, 'a', 'b', 3, 0, 'c', 'd', 'e'};
  frameAssemblerInit(&s_fa);
  size_t used;
  const uint8_t* payload;
  size_t n;
```

The hello frame arrives one byte at a time. Two later frames arrive in one array and consumed selects the remaining bytes. Invalid-length checks include zero, 4097 and the maximum valid 4096 header.

Exercise delivery boundaries independently of JSON / continued

test/native/test_protocol/test_main.c / lines 56-76

```
56    TEST_ASSERT_EQUAL(FRAME_READY, frameAssemblerFeed(&s_fa, data, sizeof(data), &used,
 >    &payload, &n));
57    TEST_ASSERT_EQUAL(4, used);
58    TEST_ASSERT_EQUAL_MEMORY("ab", payload, 2);
59    TEST_ASSERT_EQUAL(FRAME_READY,
60                      frameAssemblerFeed(&s_fa, data + used, sizeof(data) - used, &used,
 >    &payload, &n));
61    TEST_ASSERT_EQUAL(5, used);
62    TEST_ASSERT_EQUAL_MEMORY("cde", payload, 3);
63  }
64
65  static void test_frame_rejects_zero_and_oversize(void) {
66    size_t used;
67    const uint8_t* payload;
68    size_t n;
69    const uint8_t zero[] = {0, 0};
70    const uint8_t big[] = {0x01, 0x10};  // 4097
71    frameAssemblerInit(&s_fa);
72    TEST_ASSERT_EQUAL(FRAME_ERROR, frameAssemblerFeed(&s_fa, zero, 2, &used, &payload,
 >    &n));
73    TEST_ASSERT_EQUAL(FRAME_ERROR, frameAssemblerFeed(&s_fa, big, 2, &used, &payload,
 >    &n));
74    TEST_ASSERT_EQUAL_UINT32(2, s_fa.rejected);
75    uint8_t hdr[2];
76    TEST_ASSERT_TRUE(frameWriteHeader(hdr, 4096));
```

test/native/test_protocol/test_main.c / lines 77-81

```
77    TEST_ASSERT_EQUAL_HEX8(0x00, hdr[0]);
78    TEST_ASSERT_EQUAL_HEX8(0x10, hdr[1]);
79    TEST_ASSERT_FALSE(frameWriteHeader(hdr, 4097));
80    TEST_ASSERT_FALSE(frameWriteHeader(hdr, 0));
81  }
```

These cases establish that a delivery packet boundary is irrelevant to application framing. No real BLE connection is required to test that property.

Verify value eligibility and bounded output

test/native/test_protocol/test_main.c / lines 82-107

```
82
83 // ------------------------------------------------------------- encoders
84
85 static Snapshot s_snap;
86
87 static void test_telemetry_value_rules(void) {
88   snapshotReset(&s_snap);
89   Reading* air = &s_snap.readings[METRIC_AIR_TEMPERATURE];
90   air->health = HEALTH_OK;
91   air->value = 23.5f;
92   air->intervalEndMs = 9900;
93   air->sequence = 7;
94   Reading* ph = &s_snap.readings[METRIC_PH];
95   ph->health = HEALTH_UNCALIBRATED;
96   ph->unit = UNIT_MILLIVOLT;
97   ph->value = 1532.0f;
98   ph->intervalEndMs = 5000;  // 5 s old: beyond the 3 s ordinary limit
99   Reading* tds = &s_snap.readings[METRIC_TDS];
100  tds->health = HEALTH_UNPOWERED;
101  tds->value = 999.0f;  // retained, must not be sent
102  tds->intervalEndMs = 9800;
103  Reading* light = &s_snap.readings[METRIC_LIGHT_LEVEL];
104  light->health = HEALTH_OK;
105  light->value = 80.0f;
106  light->intervalEndMs = 9900;
107
```

The retained Snapshot includes fresh air, stale uncalibrated pH, unpowered TDS and disabled light. Assertions inspect the expected health and null values and reject the retained 999 from output.

test/native/test_protocol/test_main.c / lines 108-117

```
108  const SensorMask enabled = (SensorMask)(SENSOR_MASK_ALL & ~(1u << SENSOR_LIGHT));
109  const size_t n = protocolEncodeTelemetry(&s_snap, 10000, 42, 3, enabled, s_out,
  >  sizeof(s_out));
110  TEST_ASSERT_TRUE(n > 0);
111  TEST_ASSERT_NOT_NULL(strstr(s_out,
  >  "\"air_temperature\":{\"value\":23.5,\"unit\":\"degC\","
112                                    "\"health\":\"OK\",\"age_ms\":100,\"seq\":7"));
113  TEST_ASSERT_NOT_NULL(strstr(s_out,
  >  "\"ph\":{\"value\":null,\"unit\":\"mV\",\"health\":\"STALE\""));
114  TEST_ASSERT_NOT_NULL(strstr(s_out,
  >  "\"tds\":{\"value\":null,\"unit\":\"ppm\",\"health\":\"UNPOWERED\""));
115  TEST_ASSERT_NOT_NULL(strstr(s_out, "\"light_level\":{\"value\":null"));  // disabled
116  TEST_ASSERT_NOT_NULL(strstr(s_out,
  >  "\"tvoc\":{\"value\":null,\"unit\":\"ppb\",\"health\":\"STARTING\","
117                                    "\"age_ms\":null"));
```

Verify value eligibility and bounded output / continued

test/native/test_protocol/test_main.c / lines 118-126

```
  TEST_ASSERT_NULL(strstr(s_out, "999"));
  TEST_ASSERT_TRUE(n < 1600);  // well inside one 4096-byte frame
}

static void test_telemetry_overflow_returns_zero(void) {
  snapshotReset(&s_snap);
  char small[64];
  TEST_ASSERT_EQUAL(0, protocolEncodeTelemetry(&s_snap, 0, 1, 1, SENSOR_MASK_ALL, small,
    sizeof(small)));
}
```

A tiny destination then verifies the zero-length overflow contract. This is how a sender avoids transmitting plausible-looking but incomplete JSON after its output buffer fills.

Check escaping and conditional status fields

test/native/test_protocol/test_main.c / lines 127-154

```
127
128 static void test_state_escapes_and_hides_ip_unless_connected(void) {
129   StateView v;
130   memset(&v, 0, sizeof(v));
131   v.node = "Hydro \"ESP1\"";
132   v.fw = "0.8.0";
133   v.link = "ble";
134   v.mode = "unknown";
135   v.wifiState = "connecting";
136   v.wifiSsid = "Green\\house";
137   v.wifiIp = "192.168.1.42";
138   v.lampReason = "level";
139   v.latchedMask = 0x0002;
140   v.ruleCount = 2;
141   v.sgp30Baseline = "learning";
142   v.enabled = (SensorMask)((1u << SENSOR_PH) | (1u << SENSOR_TDS));
143   TEST_ASSERT_TRUE(protocolEncodeState(&v, s_out, sizeof(s_out)) > 0);
144   TEST_ASSERT_NOT_NULL(strstr(s_out, "\"node\":\"Hydro \\\"ESP1\\\"\""));
145   TEST_ASSERT_NOT_NULL(strstr(s_out, "\"ssid\":\"Green\\\\house\""));
146   TEST_ASSERT_NULL(strstr(s_out, "192.168"));
147   TEST_ASSERT_NOT_NULL(strstr(s_out, "\"latched\":[1]"));
148   TEST_ASSERT_NOT_NULL(strstr(s_out, "\"enabled\":[\"ph\",\"tds\"]"));
149   v.wifiState = "connected";
150   v.wifiRssi = -61;
151   TEST_ASSERT_TRUE(protocolEncodeState(&v, s_out, sizeof(s_out)) > 0);
152   TEST_ASSERT_NOT_NULL(strstr(s_out, "\"ip\":\"192.168.1.42\",\"rssi\":-61"));
153 }
154
```

State strings deliberately contain a quote and a backslash. IP must be absent while connecting and present after the state becomes connected. Result tests check complete envelope strings, while the catalogue checks multi-metric membership.

Check escaping and conditional status fields / continued

test/native/test_protocol/test_main.c / lines 155-163

```
155 static void test_results_and_sensor_list(void) {
156   TEST_ASSERT_TRUE(protocolEncodeResultError("a1", "rule_latched", s_out, sizeof(s_out))
  >    > 0);
157   TEST_ASSERT_EQUAL_STRING(
158       "{\"v\":2,\"type\":\"result\",\"id\":\"a1\",\"status\":\"error\",\"error\":\"rule_
  >   latched\"}", s_out);
159   TEST_ASSERT_TRUE(protocolEncodeResultBusy("a2", s_out, sizeof(s_out)) > 0);
160   TEST_ASSERT_EQUAL_STRING("{\"v\":2,\"type\":\"result\",\"id\":\"a2\",\"status\":\"busy
  >   \"}", s_out);
161   TEST_ASSERT_TRUE(protocolEncodeSensorList(SENSOR_MASK_ALL, s_out, sizeof(s_out)) > 0);
162   TEST_ASSERT_NOT_NULL(strstr(s_out, "\"key\":\"bme280\""));
163   TEST_ASSERT_NOT_NULL(strstr(s_out,
  >   "\"metrics\":[\"air_temperature\",\"air_humidity\",\"air_pressure\"]"));
```

test/native/test_protocol/test_main.c / lines 164-165

```
164   TEST_ASSERT_NOT_NULL(strstr(s_out, "\"metrics\":[\"tvoc\",\"co2eq\"]"));
165 }
```

strstr checks selected fragments rather than parsing the whole JSON tree. These focused assertions are useful but do not imply every field combination has been exhaustively validated.

Read accepted command examples as typed inputs

test/native/test_protocol/test_main.c / lines 166-192

```
166
167 // ------------------------------------------------------------- parser
168
169 typedef struct BadCase {
170   const char* json;
171   ProtocolError expected;
172 } BadCase;
173
174 static void test_parse_valid_commands(void) {
175   Command c;
176   TEST_ASSERT_EQUAL(PROTOCOL_OK,
177                     parse("{\"v\":2,\"id\":\"x1\",\"op\":\"set_sensor_enabled\","
178                           "\"data\":{\"sensor\":\"tds\",\"enabled\":false}}",
179                           &c));
180   TEST_ASSERT_EQUAL(CMD_OP_SET_SENSOR_ENABLED, c.op);
181   TEST_ASSERT_EQUAL(SENSOR_TDS, c.sensor);
182   TEST_ASSERT_FALSE(c.enabled);
183   TEST_ASSERT_EQUAL_STRING("x1", c.id);
184
185   // Key order is free; unknown keys and nested values are ignored.
186   TEST_ASSERT_EQUAL(PROTOCOL_OK,
187                     parse(" { \"data\" : {\"rule\": 1, \"extra\": [1,{\"a\":2}]},
  >   \"op\":\"reset_latch\","
188                           " \"x\":null, \"id\":\"r\", \"v\":2 } ",
189                           &c));
190   TEST_ASSERT_EQUAL(CMD_OP_RESET_LATCH, c.op);
191   TEST_ASSERT_EQUAL(1, c.rule);
192
```

The valid cases show set-enabled, reset-latch, Wi-Fi and read-only operations. Key order varies, unknown nested content is ignored, and escaped text becomes decoded UTF-8 or printable password content.

Read accepted command examples as typed inputs / continued

test/native/test_protocol/test_main.c / lines 193-202

```
193   TEST_ASSERT_EQUAL(PROTOCOL_OK,
194                     parse("{\"v\":2,\"id\":\"w\",\"op\":\"set_wifi\","
195                           "\"data\":{\"ssid\":\"Caf\\u00e9\",\"password\":\"pa\\\"ss
  >   word\"}}",
196                           &c));
197   TEST_ASSERT_EQUAL_STRING("Caf\xC3\xA9", c.ssid);
198   TEST_ASSERT_EQUAL_STRING("pa\"ss word", c.password);
199
200   TEST_ASSERT_EQUAL(PROTOCOL_OK, parse("{\"v\":2,\"id\":\"g\",\"op\":\"get_sensors\"}",
  >   &c));
201   TEST_ASSERT_EQUAL(PROTOCOL_OK,
  >   parse("{\"v\":2,\"id\":\"g\",\"op\":\"get_calibration\"}", &c));
202   TEST_ASSERT_EQUAL(CMD_OP_GET_CALIBRATION, c.op);
```

test/native/test_protocol/test_main.c / lines 203-203

```
203 }
```

The C literal \" becomes a quote byte before the JSON parser sees it. When debugging an example, write the resulting JSON on paper first so C escaping does not obscure the protocol syntax.

Check scaled inputs and rejected operation shapes

test/native/test_protocol/test_main.c / lines 204-231

```
204
205 static void test_parse_calibration_commands(void) {
206   Command c;
207   TEST_ASSERT_EQUAL(PROTOCOL_OK,
208                     parse("{\"v\":2,\"id\":\"c\",\"op\":\"set_calibration\","
209
  >   "\"data\":{\"sensor\":\"ph\",\"x1\":25000,\"y1\":7000,\"x2\":30130,\"y2\":4010}}",
210                           &c));
211   TEST_ASSERT_EQUAL(CMD_OP_SET_CALIBRATION, c.op);
212   TEST_ASSERT_EQUAL(SENSOR_PH, c.sensor);
213   TEST_ASSERT_TRUE(c.calTwoPoints);
214   TEST_ASSERT_EQUAL_INT32(25000, c.calX1);
215   TEST_ASSERT_EQUAL_INT32(4010, c.calY2);
216   TEST_ASSERT_EQUAL(PROTOCOL_OK,
  >   parse("{\"v\":2,\"id\":\"c\",\"op\":\"set_calibration\","
217
  >   "\"data\":{\"sensor\":\"tds\",\"x1\":6500,\"y1\":7070}}",
218                                       &c));
219   TEST_ASSERT_EQUAL(SENSOR_TDS, c.sensor);
220   TEST_ASSERT_FALSE(c.calTwoPoints);
221   TEST_ASSERT_EQUAL(PROTOCOL_OK,
  >   parse("{\"v\":2,\"id\":\"c\",\"op\":\"set_calibration\","
222
  >   "\"data\":{\"sensor\":\"light\",\"x1\":12000,\"y1\":30000}}",
223                                       &c));
224   TEST_ASSERT_EQUAL(SENSOR_LIGHT, c.sensor);
225   TEST_ASSERT_EQUAL(PROTOCOL_OK,
226
  >   parse("{\"v\":2,\"id\":\"c\",\"op\":\"cal_capture\",\"data\":{\"sensor\":\"light\"}}",
  >    &c));
227   TEST_ASSERT_EQUAL(CMD_OP_CAL_CAPTURE, c.op);
228   TEST_ASSERT_EQUAL(PROTOCOL_OK,
229                     parse("{\"v\":2,\"id\":\"c\",\"op\":\"reset_calibration\",\"data\":{
  >   \"sensor\":\"tds\"}}", &c));
230   TEST_ASSERT_EQUAL(CMD_OP_RESET_CALIBRATION, c.op);
231
```

test/native/test_protocol/test_main.c / lines 232-238

```
232   static const BadCase kBad[] = {
233       // pH needs both buffers
234       {"{\"v\":2,\"id\":\"a\",\"op\":\"set_calibration\",\"data\":{\"sensor\":\"ph\",\"x
  >   1\":25000,\"y1\":7000}}",
235        PROTOCOL_ERR_BAD_FIELD},
236       // pH above 14
237       {"{\"v\":2,\"id\":\"a\",\"op\":\"set_calibration\",\"data\":{\"sensor\":\"ph\",\"x
  >   1\":1,\"y1\":15000,"
238        "\"x2\":2,\"y2\":4000}}",
```

Check scaled inputs and rejected operation shapes / continued

test/native/test_protocol/test_main.c / lines 239-259

```
        PROTOCOL_ERR_BAD_FIELD},
      // fractions are not accepted: values are scaled integers
      {"{\"v\":2,\"id\":\"a\",\"op\":\"set_calibration\",\"data\":{\"sensor\":\"tds\",\"
  x1\":650.5,\"y1\":707}}",
        PROTOCOL_ERR_BAD_FIELD},
      {"{\"v\":2,\"id\":\"a\",\"op\":\"set_calibration\",\"data\":{\"sensor\":\"tds\",\"
  x1\":0,\"y1\":707}}",
        PROTOCOL_ERR_BAD_FIELD},
      // sensors without a calibration
      {"{\"v\":2,\"id\":\"a\",\"op\":\"set_calibration\",\"data\":{\"sensor\":\"bme280\"
  ,\"x1\":1,\"y1\":1}}",
        PROTOCOL_ERR_BAD_FIELD},
      {"{\"v\":2,\"id\":\"a\",\"op\":\"cal_capture\",\"data\":{\"sensor\":\"hcsr04\"}}",
   PROTOCOL_ERR_BAD_FIELD},
      {"{\"v\":2,\"id\":\"a\",\"op\":\"reset_calibration\"}", PROTOCOL_ERR_BAD_FIELD},
      // the LAN ops are gone
      {"{\"v\":2,\"id\":\"a\",\"op\":\"get_lan_token\"}", PROTOCOL_ERR_UNKNOWN_OP},
      {"{\"v\":2,\"id\":\"a\",\"op\":\"hello\",\"data\":{\"token\":\"0123456789abcdef012
  3456789abcdef\"}}",
        PROTOCOL_ERR_UNKNOWN_OP},
  };
  for (size_t i = 0; i < sizeof(kBad) / sizeof(kBad[0]); ++i) {
    char msg[16];
    snprintf(msg, sizeof(msg), "case %u", (unsigned)i);
    TEST_ASSERT_EQUAL_MESSAGE(kBad[i].expected, parse(kBad[i].json, &c), msg);
  }
```

Accepted pH uses two scaled points; TDS and light use their one-point forms. The bad-case table rejects missing pH coordinates, out-of-range targets, fractions, unsupported sensors and removed LAN operations.

test/native/test_protocol/test_main.c / lines 260-262

```
  TEST_ASSERT_EQUAL_STRING("stale", protocolErrorName(PROTOCOL_ERR_STALE));
  TEST_ASSERT_EQUAL_STRING("bad_fit", protocolErrorName(PROTOCOL_ERR_BAD_FIT));
}
```

The loop adds a case index to assertion messages. A data table avoids repeating test machinery while keeping each rejected input and expected outcome visible.

Inspect revision and raw-capture representations

test/native/test_protocol/test_main.c / lines 263-272

```
263
264 static void test_encode_calibration_and_capture(void) {
265   const CalibrationView v = {{-0.0057f, 22.7f, 0, true}, {1.05f, 0.0f, 9, true}, {1.0f,
  >   0.0f, 0, false}, CAL_BIT_TDS};
266   TEST_ASSERT_TRUE(protocolEncodeCalibration(&v, s_out, sizeof(s_out)) > 0);
267   TEST_ASSERT_NOT_NULL(strstr(s_out,
  >   "\"ph\":{\"overridden\":false,\"rev\":0,\"valid\":true"));
268   TEST_ASSERT_NOT_NULL(strstr(s_out,
  >   "\"tds\":{\"overridden\":true,\"rev\":9,\"valid\":true,\"slope\":1.05"));
269   TEST_ASSERT_NOT_NULL(strstr(s_out,
  >   "\"light\":{\"overridden\":false,\"rev\":0,\"valid\":false"));
270   TEST_ASSERT_TRUE(protocolEncodeCapture(SENSOR_PH, 25012, 412, s_out, sizeof(s_out)) >
  >   0);
271   TEST_ASSERT_EQUAL_STRING("{\"sensor\":\"ph\",\"x\":25012,\"age_ms\":412}", s_out);
272 }
```

The CalibrationView combines valid and invalid fits with an explicit override mask. Assertions check that those independent facts survive encoding. Capture is compared with one exact JSON object.

A valid fit can be compiled rather than user-overridden. The override flag says where update policy should obtain future coefficients; it is not another name for validity.

Distinguish syntax errors from field errors

test/native/test_protocol/test_main.c / lines 273-300

```
273
274 static void test_parse_rejects_malformed_and_out_of_range(void) {
275   static const BadCase kCases[] = {
276       {"", PROTOCOL_ERR_BAD_REQUEST},
277       {"[]", PROTOCOL_ERR_BAD_REQUEST},
278       {"{\"v\":2,\"id\":\"a\",\"op\":\"get_sensors\"", PROTOCOL_ERR_BAD_REQUEST},  //
  >   truncated
279       {"{\"v\":2,\"id\":\"a\",\"op\":\"get_sensors\"}x", PROTOCOL_ERR_BAD_REQUEST},  //
  >   trailing
280       {"{\"v\":1,\"id\":\"a\",\"op\":\"get_sensors\"}", PROTOCOL_ERR_BAD_REQUEST},  //
  >   old version
281       {"{\"v\":2,\"op\":\"get_sensors\"}", PROTOCOL_ERR_BAD_REQUEST},              // no
  >    id
282       {"{\"v\":2,\"id\":\"a\"}", PROTOCOL_ERR_UNKNOWN_OP},
283       {"{\"v\":2,\"id\":\"a\",\"op\":\"add_device\"}", PROTOCOL_ERR_UNKNOWN_OP},  // v1
  >   op
284       {"{\"v\":2,\"id\":\"a\",\"op\":\"set_sensor_enabled\",\"data\":{\"sensor\":\"mq7\"
  >   ,\"enabled\":true}}",
285        PROTOCOL_ERR_BAD_FIELD},
286       {"{\"v\":2,\"id\":\"a\",\"op\":\"set_sensor_enabled\",\"data\":{\"sensor\":\"ph\",
  >   \"enabled\":1}}",
287        PROTOCOL_ERR_BAD_FIELD},
288       {"{\"v\":2,\"id\":\"a\",\"op\":\"set_sensor_enabled\"}", PROTOCOL_ERR_BAD_FIELD},
289       {"{\"v\":2,\"id\":\"a\",\"op\":\"reset_latch\",\"data\":{\"rule\":1.5}}",
  >   PROTOCOL_ERR_BAD_FIELD},
290       {"{\"v\":2,\"id\":\"a\",\"op\":\"reset_latch\",\"data\":{\"rule\":-1}}",
  >   PROTOCOL_ERR_BAD_FIELD},
291       {"{\"v\":2,\"id\":\"a\",\"op\":\"reset_latch\",\"data\":{\"rule\":16}}",
  >   PROTOCOL_ERR_BAD_FIELD},
292       {"{\"v\":2,\"id\":\"a\",\"op\":\"set_wifi\",\"data\":{\"ssid\":\"x\",\"password\":
  >   \"short\"}}",
293        PROTOCOL_ERR_BAD_FIELD},
294       {"{\"v\":2,\"id\":\"a\",\"op\":\"set_wifi\",\"data\":{\"ssid\":\"\",\"password\":\
  >   "longenough\"}}",
295        PROTOCOL_ERR_BAD_FIELD},
296       {"{\"v\":2,\"id\":\"a\",\"op\":\"set_wifi\",\"data\":{\"ssid\":\"12345678901234567
  >   8901234567890123\","
297        "\"password\":\"longenough\"}}",
298        PROTOCOL_ERR_BAD_FIELD},  // 33-byte SSID
299       {"{\"v\":2,\"id\":\"a\",\"op\":\"set_wifi\",\"data\":{\"ssid\":\"x\",\"password\":
  >   \"tab\\there!\"}}",
300        PROTOCOL_ERR_BAD_FIELD},  // control character
```

The table covers truncation, trailing bytes, wrong protocol version, absent id, unknown op, unknown sensor, mistyped enabled, short password, control text, long id and excessive nesting.

Distinguish syntax errors from field errors / continued

test/native/test_protocol/test_main.c / lines 301-320

```
301       {"{\"v\":2,\"id\":\"a\\u0000\",\"op\":\"get_sensors\"}",
  >   PROTOCOL_ERR_BAD_REQUEST},
302       {"{\"v\":2,\"id\":\"12345678901234567890123456789012345678901\",\"op\":\"get_senso
  >   rs\"}",
303        PROTOCOL_ERR_BAD_REQUEST},  // 41-char id
304       {"{\"v\":2,\"id\":\"a\",\"op\":\"get_sensors\",\"d\":[[[[[[1]]]]]]}",
  >   PROTOCOL_ERR_BAD_REQUEST},
305   };
306   for (size_t i = 0; i < sizeof(kCases) / sizeof(kCases[0]); ++i) {
307     Command c;
308     char msg[16];
309     snprintf(msg, sizeof(msg), "case %u", (unsigned)i);
310     TEST_ASSERT_EQUAL_MESSAGE(kCases[i].expected, parse(kCases[i].json, &c), msg);
311   }
312 }
313
314 static void test_parse_keeps_id_for_error_reply(void) {
315   Command c;
316   TEST_ASSERT_EQUAL(PROTOCOL_ERR_BAD_FIELD,
317
  >   parse("{\"v\":2,\"id\":\"keep-me\",\"op\":\"reset_latch\",\"data\":{}}", &c));
318   TEST_ASSERT_EQUAL_STRING("keep-me", c.id);
319   TEST_ASSERT_EQUAL_STRING("rule_latched",
  >   protocolErrorName(PROTOCOL_ERR_RULE_LATCHED));
320 }
```

A separately valid id survives a later bad-data rejection so the result can be correlated. An unknown or malformed id cannot be safely invented from arbitrary input bytes.

Check parser arithmetic and stored-record integrity

test/native/test_protocol/test_main.c / lines 321-348

```
321
322 // ------------------------------------------------------------- config blob
323
324 static const LampControlConfig kLamp = {40.0f, 120.0f, 30000, 30000, 5};
325 static const LinearCalibration kPh = {-0.0057f, 22.7f, 0, false};
326 static const LinearCalibration kTds = {1.0f, 0.0f, 1, true};
327 static const LinearCalibration kLight = {1.0f, 0.0f, 0, false};
328
329 static ConfigDefaults defaults(void) {
330   ConfigDefaults d;
331   memset(&d, 0, sizeof(d));
332   d.lampAutoEnabled = true;
333   d.lamp = kLamp;
334   d.ph = kPh;
335   d.tds = kTds;
336   d.light = kLight;
337   return d;
338 }
339
340 // Audit P2: a long integer must not overflow the accumulator (signed
341 // overflow is undefined), in a known field or in an ignored one.
342 static void test_parse_long_integers_without_overflow(void) {
343   Command c;
344   TEST_ASSERT_EQUAL(PROTOCOL_ERR_BAD_FIELD,
345                     parse("{\"v\":2,\"id\":\"a\",\"op\":\"reset_latch\","
346                           "\"data\":{\"rule\":12345678901234567890123456789012345678901234567890}}", &c));
347   TEST_ASSERT_EQUAL(PROTOCOL_OK,
348                     parse("{\"v\":2,\"id\":\"a\",\"op\":\"reset_latch\",\"pad\":-9999999
  >   99999999999999999999999,"
```

test/native/test_protocol/test_main.c / lines 349-364

```
349                           "\"data\":{\"rule\":1}}", &c));
350   TEST_ASSERT_EQUAL_UINT8(1, c.rule);
351 }
352
353 // A stored fit that is not a finite number is a corrupt blob.
354 static void test_blob_rejects_non_finite_fit(void) {
355   const ConfigDefaults d = defaults();
356   ConfigBlob b;
357   configBlobInit(&b, &d);
358   b.ph.slope = NAN;
359   configBlobSeal(&b);
360   TEST_ASSERT_FALSE(configBlobValid(&b));
361 }
362
363 static void test_blob_defaults_seal_and_corruption(void) {
364   const ConfigDefaults d = defaults();
```

Check parser arithmetic and stored-record integrity / continued

test/native/test_protocol/test_main.c / lines 365-376

```
365    ConfigBlob b;
366    configBlobInit(&b, &d);
367    TEST_ASSERT_TRUE(configBlobValid(&b));
368    TEST_ASSERT_EQUAL_UINT16(1, b.revision);
369    TEST_ASSERT_EQUAL_UINT16(2, b.schema);
370    TEST_ASSERT_EQUAL_HEX8(SENSOR_MASK_ALL, b.sensorEnabled);
371    TEST_ASSERT_EQUAL_HEX8(0, b.calOverridden);
372    ConfigBlob bad;
373    memcpy(&bad, &b, sizeof(bad));
374    ((uint8_t*)&bad)[12] ^= 0x01;  // one flipped bit in the lamp policy
375    TEST_ASSERT_FALSE(configBlobValid(&bad));
376    TEST_ASSERT_EQUAL_HEX32(0xCBF43926u, crc32Ieee("123456789", 9));  // standard check
  >    value
```

defaults builds a complete controlled configuration fixture. Long integers test that parsing can continue safely without overflowing a signed accumulator. A sealed blob containing NaN is rejected independently of CRC.

test/native/test_protocol/test_main.c / lines 377-377

```
377 }
```

The CRC standard check vector and a flipped byte test accidental-corruption detection. A checksum is not the same property as a valid schema or a finite coefficient; each has its own assertion.

Choose the newest valid slot and preserve user choices

test/native/test_protocol/test_main.c / lines 378-400

```
378
379 static void test_blob_sensor_change_and_slot_pick(void) {
380   const ConfigDefaults d = defaults();
381   ConfigBlob a, b;
382   configBlobInit(&a, &d);
383   TEST_ASSERT_FALSE(configBlobWithSensor(&a, SENSOR_TDS, true, &b));  // no change
384   TEST_ASSERT_TRUE(configBlobWithSensor(&a, SENSOR_TDS, false, &b));
385   TEST_ASSERT_TRUE(configBlobValid(&b));
386   TEST_ASSERT_EQUAL_UINT16(2, b.revision);
387   TEST_ASSERT_FALSE(sensorMaskHas(b.sensorEnabled, SENSOR_TDS));
388   TEST_ASSERT_TRUE(configBlobPickNewest(&a, &b) == &b);
389   TEST_ASSERT_TRUE(configBlobPickNewest(&b, &a) == &b);
390   b.crc ^= 1u;  // newest slot torn: fall back to the older one
391   TEST_ASSERT_TRUE(configBlobPickNewest(&a, &b) == &a);
392   TEST_ASSERT_NULL(configBlobPickNewest(NULL, &b));
393   // Revision wrap: 0xFFFF -> 1 is newer.
394   a.revision = 0xFFFF;
395   configBlobSeal(&a);
396   TEST_ASSERT_TRUE(configBlobWithSensor(&a, SENSOR_PH, false, &b));
397   TEST_ASSERT_EQUAL_UINT16(1, b.revision);
398   TEST_ASSERT_TRUE(configBlobPickNewest(&a, &b) == &b);
399 }
400
```

A no-op sensor change returns false. A real change increments revision and clears its enabled bit. If the newer slot is corrupted, the older valid slot wins; the wrap case checks revision 0xFFFF to 1.

Choose the newest valid slot and preserve user choices / continued

test/native/test_protocol/test_main.c / lines 401-418

```
401 static void test_blob_refresh_defaults_follows_compiled_values(void) {
402   const ConfigDefaults d = defaults();
403   ConfigBlob b;
404   configBlobInit(&b, &d);
405   ConfigBlob tdsOff;
406   TEST_ASSERT_TRUE(configBlobWithSensor(&b, SENSOR_TDS, false, &tdsOff));
407   // Same compiled values: untouched.
408   TEST_ASSERT_FALSE(configBlobRefreshDefaults(&tdsOff, &d));
409   TEST_ASSERT_EQUAL_UINT16(2, tdsOff.revision);
410   // A re-measured pH offset in the source wins over the stored copy, keeps the
411   // sensor mask, and gets a new revision.
412   ConfigDefaults d2 = d;
413   d2.ph.offset = 23.1f;
414   TEST_ASSERT_TRUE(configBlobRefreshDefaults(&tdsOff, &d2));
415   TEST_ASSERT_EQUAL_UINT16(3, tdsOff.revision);
416   TEST_ASSERT_EQUAL_FLOAT(23.1f, tdsOff.ph.offset);
417   TEST_ASSERT_FALSE(sensorMaskHas(tdsOff.sensorEnabled, SENSOR_TDS));
418   TEST_ASSERT_TRUE(configBlobValid(&tdsOff));
```

test/native/test_protocol/test_main.c / lines 419-419

```
419 }
```

Refreshing compiled defaults changes applicable coefficients while preserving the sensor mask. Read before/after revisions so a change in calibration history is visible rather than silently replacing numbers.

Respect user overrides across refresh and migration

test/native/test_protocol/test_main.c / lines 420-447

```
420
421 static void test_blob_user_calibration_survives_refresh_and_reset(void) {
422   const ConfigDefaults d = defaults();
423   ConfigBlob b, cal, reset;
424   configBlobInit(&b, &d);
425   const LinearCalibration fit = {-0.0061f, 23.9f, 0, true};
426   TEST_ASSERT_TRUE(configBlobWithCalibration(&b, SENSOR_PH, &fit, true, &cal));
427   TEST_ASSERT_TRUE(configBlobValid(&cal));
428   TEST_ASSERT_EQUAL_UINT16(2, cal.revision);
429   TEST_ASSERT_EQUAL_UINT16(2, cal.ph.revision);  // cal_rev = config revision of the fit
430   TEST_ASSERT_EQUAL_HEX8(CAL_BIT_PH, cal.calOverridden);
431   // A changed compiled pH default does NOT replace the user fit...
432   ConfigDefaults d2 = d;
433   d2.ph.offset = 99.0f;
434   d2.light.slope = 2.0f;
435   TEST_ASSERT_TRUE(configBlobRefreshDefaults(&cal, &d2));  // ...but the light default
  >   does change
436   TEST_ASSERT_EQUAL_FLOAT(23.9f, cal.ph.offset);
437   TEST_ASSERT_EQUAL_FLOAT(2.0f, cal.light.slope);
438   // Reset: compiled default back, bit cleared, refreshed from then on.
439   TEST_ASSERT_TRUE(configBlobWithCalibration(&cal, SENSOR_PH, &d2.ph, false, &reset));
440   TEST_ASSERT_EQUAL_HEX8(0, reset.calOverridden);
441   TEST_ASSERT_EQUAL_FLOAT(99.0f, reset.ph.offset);
442   TEST_ASSERT_EQUAL_UINT16(0, reset.ph.revision);
443   TEST_ASSERT_FALSE(configBlobWithCalibration(&cal, SENSOR_BME280, &fit, true, &reset));
444   TEST_ASSERT_TRUE(configBlobCalibration(&cal, SENSOR_LIGHT) == &cal.light);
445   TEST_ASSERT_NULL(configBlobCalibration(&cal, SENSOR_SGP30));
446 }
447
```

The user-fit test sets the override bit, refreshes other compiled defaults without replacing that fit, then resets it to the compiled default. The migration test constructs an old schema with its own checksum.

test/native/test_protocol/test_main.c / lines 448-459

```
448 static void test_blob_migrates_schema_1(void) {
449   ConfigBlobV1 old;
450   memset(&old, 0, sizeof(old));
451   old.magic = CONFIG_BLOB_MAGIC;
452   old.schema = 1;
453   old.revision = 7;
454   old.sensorEnabled = (SensorMask)(SENSOR_MASK_ALL & ~(1u << SENSOR_TDS));
455   old.lampAutoEnabled = true;
456   old.lamp = kLamp;
457   old.ph = kPh;
458   old.tds = kTds;
459   old.crc = crc32Ieee(&old, (uint32_t)offsetof(ConfigBlobV1, crc));
```

Respect user overrides across refresh and migration / continued

test/native/test_protocol/test_main.c / lines 460-472

```
460   const ConfigDefaults d = defaults();
461   ConfigBlob b;
462   TEST_ASSERT_TRUE(configBlobMigrateV1(&old, sizeof(old), &d, &b));
463   TEST_ASSERT_TRUE(configBlobValid(&b));
464   TEST_ASSERT_EQUAL_UINT16(7, b.revision);
465   TEST_ASSERT_EQUAL_HEX8(old.sensorEnabled, b.sensorEnabled);  // the user's choice
  >   survives the update
466   TEST_ASSERT_EQUAL_HEX8(0, b.calOverridden);
467   TEST_ASSERT_FALSE(b.light.valid);
468   old.crc ^= 1u;
469   TEST_ASSERT_FALSE(configBlobMigrateV1(&old, sizeof(old), &d, &b));
470   TEST_ASSERT_FALSE(configBlobMigrateV1(&old, sizeof(old) - 1, &d, &b));
471   TEST_ASSERT_TRUE(sizeof(ConfigBlob) != sizeof(ConfigBlobV1));  // so getBlob() can
  >   tell them apart
472 }
```

Migration accepts exactly the old size and valid CRC, preserves the sensor choice and establishes the new fields. An older binary layout is decoded deliberately; it is not cast into the new structure and hoped to match.

Reject a plausible-looking but unsuitable fit

test/native/test_protocol/test_main.c / lines 473-496

```
473
474 // ------------------------------------------------------------ calibration fits /
  >   capture
475
476 static void test_calibration_fits(void) {
477   LinearCalibration c;
478   // Buffers 7.00 at 2500 mV and 4.00 at 3030 mV: slope -0.00566 pH/mV.
479   TEST_ASSERT_TRUE(phFitFromBuffers(2500.0f, 7.0f, 3030.0f, 4.0f, 0.002f, 0.02f, 5,
  >   &c));
480   TEST_ASSERT_TRUE(c.valid);
481   TEST_ASSERT_FLOAT_WITHIN(1e-3f, 7.0f, linearApply(&c, 2500.0f));
482   TEST_ASSERT_FLOAT_WITHIN(1e-3f, 4.0f, linearApply(&c, 3030.0f));
483   TEST_ASSERT_FALSE(phFitFromBuffers(2500.0f, 7.0f, 2510.0f, 4.0f, 0.002f, 0.02f, 5,
  >   &c));  // too steep
484   TEST_ASSERT_FALSE(c.valid);
485   TEST_ASSERT_FALSE(phFitFromBuffers(2500.0f, 7.0f, 2600.0f, 6.5f, 0.002f, 0.02f, 5,
  >   &c));  // < 1 pH apart
486   TEST_ASSERT_FALSE(phFitFromBuffers(2500.0f, 7.0f, 2500.0f, 4.0f, 0.002f, 0.02f, 5,
  >   &c));  // same mV
487   // TDS K: 650 ppm nominal in a 707 ppm solution.
488   TEST_ASSERT_TRUE(linearGainFromPoint(650.0f, 707.0f, 0.5f, 2.0f, 0, &c));
489   TEST_ASSERT_FLOAT_WITHIN(1e-4f, 707.0f / 650.0f, c.slope);
490   TEST_ASSERT_EQUAL_FLOAT(0.0f, c.offset);
491   TEST_ASSERT_FALSE(linearGainFromPoint(100.0f, 707.0f, 0.5f, 2.0f, 0, &c));  // K = 7
492   TEST_ASSERT_FALSE(linearGainFromPoint(0.0f, 707.0f, 0.5f, 2.0f, 0, &c));
493   // Light: 1200 mV under a 3000 lux reference.
494   TEST_ASSERT_TRUE(linearGainFromPoint(1200.0f, 3000.0f, 0.01f, 100.0f, 0, &c));
495   TEST_ASSERT_FLOAT_WITHIN(1e-3f, 3000.0f, linearApply(&c, 1200.0f));
496 }
```

pH maps the two reference points, then rejects excessive slope, insufficient target separation and coincident x values. TDS and light verify one-point gains and reject impossible input or out-of-policy gain.

These examples teach why a mathematically computable coefficient may still be refused. Coordinates and application plausibility both constrain a calibration.

Reconstruct calibration x from raw published data

test/native/test_protocol/test_main.c / lines 497-524

```
497
498 static Reading reading(SensorId s, MetricId m, Unit u, Health h, float value, int32_t
  >   raw, MonoMs end) {
499   Reading r;
500   readingReset(&r, s, m, u);
501   r.health = h;
502   r.value = value;
503   r.hasRaw = true;
504   r.raw = raw;
505   r.intervalEndMs = end;
506   return r;
507 }
508
509 static void test_capture_from_snapshot(void) {
510   Snapshot snap;
511   snapshotReset(&snap);
512   const CaptureLimits lim = {5000, 30000, 10000, 15000};
513   float x;
514   uint32_t age;
515   // Nothing measured yet.
516   TEST_ASSERT_EQUAL(CAPTURE_STALE, calCaptureFromSnapshot(SENSOR_PH, &snap, 10000, &lim,
  >    &x, &age));
517   // pH: pin 1500 mV through 10k/15k -> module 2500 mV.
518   snap.readings[METRIC_PH] = reading(SENSOR_PH, METRIC_PH, UNIT_PH, HEALTH_UNCALIBRATED,
  >    7.0f, 1500, 9000);
519   TEST_ASSERT_EQUAL(CAPTURE_OK, calCaptureFromSnapshot(SENSOR_PH, &snap, 10000, &lim,
  >   &x, &age));
520   TEST_ASSERT_FLOAT_WITHIN(0.01f, 2500.0f, x);
521   TEST_ASSERT_EQUAL_UINT32(1000, age);
522   TEST_ASSERT_EQUAL(CAPTURE_STALE, calCaptureFromSnapshot(SENSOR_PH, &snap, 20000, &lim,
  >    &x, &age));  // too old
523   snap.readings[METRIC_PH].health = HEALTH_UNPOWERED;
524   TEST_ASSERT_EQUAL(CAPTURE_STALE, calCaptureFromSnapshot(SENSOR_PH, &snap, 10000, &lim,
  >    &x, &age));
```

The fixture includes raw ADC millivolts even after the displayed value has become pH or lux. pH reverses its divider; light uses raw pin millivolts. TDS requires an independently usable water-temperature record.

Reconstruct calibration x from raw published data / continued

test/native/test_protocol/test_main.c / lines 525-538

```
525   // Light: raw pin mV even after a lux fit.
526   snap.readings[METRIC_LIGHT_LEVEL] =
527       reading(SENSOR_LIGHT, METRIC_LIGHT_LEVEL, UNIT_LUX, HEALTH_OK, 3000.0f, 1200,
  >   9500);
528   TEST_ASSERT_EQUAL(CAPTURE_OK, calCaptureFromSnapshot(SENSOR_LIGHT, &snap, 10000, &lim,
  >    &x, &age));
529   TEST_ASSERT_EQUAL_FLOAT(1200.0f, x);
530   // TDS: refused without a usable water temperature, then the nominal ppm at 25 degC.
531   snap.readings[METRIC_TDS] = reading(SENSOR_TDS, METRIC_TDS, UNIT_PPM,
  >   HEALTH_UNCALIBRATED, 0.0f, 1000, 9500);
532   TEST_ASSERT_EQUAL(CAPTURE_NO_TEMPERATURE, calCaptureFromSnapshot(SENSOR_TDS, &snap,
  >   10000, &lim, &x, &age));
533   snap.readings[METRIC_WATER_TEMPERATURE] =
534       reading(SENSOR_DS18B20, METRIC_WATER_TEMPERATURE, UNIT_DEG_C, HEALTH_OK, 25.0f,
  >   400, 9000);
535   TEST_ASSERT_EQUAL(CAPTURE_OK, calCaptureFromSnapshot(SENSOR_TDS, &snap, 10000, &lim,
  >   &x, &age));
536   TEST_ASSERT_FLOAT_WITHIN(0.5f, (133.42f - 255.86f + 857.39f) * 0.5f, x);  // 1 V on
  >   the curve
537   TEST_ASSERT_EQUAL(CAPTURE_UNSUPPORTED, calCaptureFromSnapshot(SENSOR_BME280, &snap,
  >   10000, &lim, &x, &age));
538 }
```

A stale, unpowered or unsupported capture is rejected. These checks explain why capture reads a complete retained record rather than accepting an arbitrary displayed float.

Give each reader its own event position

test/native/test_protocol/test_main.c / lines 539-566

```
539
540 // ------------------------------------------------------------- journal
541
542 static EventJournal s_journal;
543
544 static void test_journal_cursors_are_independent(void) {
545   journalInit(&s_journal);
546   JournalCursor ble, lan;
547   journalCursorInit(&s_journal, &ble);
548   journalCursorInit(&s_journal, &lan);
549   JournalEvent e = {0};
550   for (uint8_t i = 0; i < 3; ++i) {
551     e.ruleId = i;
552     journalAppend(&s_journal, &e);
553   }
554   JournalEvent out;
555   bool gap;
556   TEST_ASSERT_TRUE(journalNext(&s_journal, &ble, &out, &gap));
557   TEST_ASSERT_EQUAL_UINT32(1, out.n);
558   TEST_ASSERT_FALSE(gap);
559   for (uint32_t n = 1; n <= 3; ++n) {
560     TEST_ASSERT_TRUE(journalNext(&s_journal, &lan, &out, &gap));
561     TEST_ASSERT_EQUAL_UINT32(n, out.n);
562   }
563   TEST_ASSERT_FALSE(journalNext(&s_journal, &lan, &out, &gap));
564   TEST_ASSERT_TRUE(journalNext(&s_journal, &ble, &out, &gap));
565   TEST_ASSERT_EQUAL_UINT32(2, out.n);
566 }
```

Two cursors start together, then advance at different speeds without consuming each other's history. The overrun case appends more than the fixed capacity and expects the oldest retained sequence plus gap:true.

Give each reader its own event position / continued

test/native/test_protocol/test_main.c / lines 567-579

```
567
568 static void test_journal_overrun_reports_gap(void) {
569   journalInit(&s_journal);
570   JournalCursor c;
571   journalCursorInit(&s_journal, &c);
572   JournalEvent e = {0};
573   for (int i = 0; i < EVENT_JOURNAL_LEN + 10; ++i) journalAppend(&s_journal, &e);
574   JournalEvent out;
575   bool gap;
576   TEST_ASSERT_TRUE(journalNext(&s_journal, &c, &out, &gap));
577   TEST_ASSERT_TRUE(gap);
578   TEST_ASSERT_EQUAL_UINT32(11, out.n);  // oldest still stored
579   TEST_ASSERT_TRUE(journalNext(&s_journal, &c, &out, &gap));
```

test/native/test_protocol/test_main.c / lines 580-582

```
580   TEST_ASSERT_FALSE(gap);
581   TEST_ASSERT_EQUAL_UINT32(12, out.n);
582 }
```

The local variable named lan is only a second test reader. It does not mean a LAN transport is still implemented in this firmware.

Check retry bounds without a random generator

test/native/test_protocol/test_main.c / lines 583-602

```
583
584 // ------------------------------------------------------------- backoff
585
586 static void test_backoff_doubles_with_bounded_jitter(void) {
587   Backoff b;
588   backoffInit(&b, 1000, 60000, 20);
589   const uint32_t expected[] = {1000, 2000, 4000, 8000, 16000, 32000, 60000, 60000};
590   for (size_t i = 0; i < sizeof(expected) / sizeof(expected[0]); ++i) {
591     const uint32_t base = b.currentMs;
592     TEST_ASSERT_EQUAL_UINT32(expected[i], base);
593     const uint32_t lo = backoffNext(&b, 0);
594     b.currentMs = base;
595     const uint32_t hi = backoffNext(&b, 0xFFFFFFFFu);
596     TEST_ASSERT_TRUE(lo >= base - base / 5);
597     TEST_ASSERT_TRUE(hi <= base + base / 5);
598     TEST_ASSERT_TRUE(hi <= 60000);
599   }
600   backoffReset(&b);
601   TEST_ASSERT_EQUAL_UINT32(1000, b.currentMs);
602 }
```

The test supplies deterministic random words and checks the stored base sequence from 1000 to 60000ms. It checks bounds on selected outputs and restores the initial value through reset.

Supplying 0xFFFFFFFF does not necessarily choose the exact maximum offset because remainder arithmetic depends on span. The assertions check boundedness, not an exhaustive probability distribution.

Test command outcome against active state and expiry

test/native/test_protocol/test_main.c / lines 603-629

```
603
604 // ------------------------------------------------------------ safety
605
606 static const SafetyConfig kSafety = {true, {40.0f, 120.0f, 0, 0, 5}, 3000};
607 static const AlarmRule kRules[2] = {
608     {.ruleId = 0, .enabled = true, .metric = METRIC_AIR_TEMPERATURE, .comparison =
  >   COMPARISON_ABOVE,
609      .tripThreshold = 40.0f, .clearThreshold = 37.0f, .persistMs = 0, .severity =
  >   SEVERITY_CRITICAL,
610      .latching = true, .actuatorId = ACTUATOR_RELAY1, .action =
  >   ACTUATOR_ACTION_DEENERGIZE},
611     {.ruleId = 1, .enabled = true, .metric = METRIC_TDS, .comparison = COMPARISON_ABOVE,
612      .tripThreshold = 1500.0f, .clearThreshold = 1400.0f, .persistMs = 0, .severity =
  >   SEVERITY_WARNING,
613      .latching = false, .actuatorId = ACTUATOR_NONE, .action = ACTUATOR_ACTION_NONE},
614 };
615
616 static SafetyCore s_core;
617
618 static AlarmEvent airEvent(RuleDecision d, float v, MonoMs end) {
619   AlarmEvent e;
620   memset(&e, 0, sizeof(e));
621   e.ruleId = 0;
622   e.decision = d;
623   readingReset(&e.sample, SENSOR_BME280, METRIC_AIR_TEMPERATURE, UNIT_DEG_C);
624   e.sample.health = HEALTH_OK;
625   e.sample.value = v;
626   e.sample.intervalEndMs = end;
627   return e;
628 }
629
```

test/native/test_protocol/test_main.c / lines 630-643

```
630 static void test_safety_reset_latch_command(void) {
631   safetyCoreInit(&s_core, &kSafety, kRules, 2, 0);
632   snapshotReset(&s_snap);
633   SafetyOutputs out;
634   const AlarmEvent trip = airEvent(RULE_DECISION_TRIP, 45.0f, 100);
635   SafetyInputs in = {&s_snap, &trip, 1, 100};
636   safetyCoreStep(&s_core, &in, &out);
637   TEST_ASSERT_EQUAL_HEX16(0x0001, out.latchedMask);
638
639   // Still active: refused.
640   const SafetyCommand reset = {SAFETY_CMD_RESET_LATCH, 0, 5000, 77};
641   SafetyInputs cmd = {&s_snap, NULL, 0, 200, &reset, 1, 0};
642   safetyCoreStep(&s_core, &cmd, &out);
643   TEST_ASSERT_EQUAL(1, out.resultCount);
```

Test command outcome against active state and expiry / continued

test/native/test_protocol/test_main.c / lines 644-657

```
644     TEST_ASSERT_EQUAL_UINT32(77, out.results[0].tag);
645     TEST_ASSERT_EQUAL(SAFETY_CMD_STILL_ACTIVE, out.results[0].outcome);
646     TEST_ASSERT_TRUE(out.interlock);
647
648     // Recovery sample clears "active"; an expired reset is still refused.
649     const AlarmEvent clear = airEvent(RULE_DECISION_CLEAR, 30.0f, 300);
650     SafetyInputs rec = {&s_snap, &clear, 1, 300};
651     safetyCoreStep(&s_core, &rec, &out);
652     TEST_ASSERT_EQUAL_HEX16(0x0001, out.latchedMask);
653     SafetyInputs late = {&s_snap, NULL, 0, 6000, &reset, 1, 0};
654     safetyCoreStep(&s_core, &late, &out);
655     TEST_ASSERT_EQUAL(SAFETY_CMD_EXPIRED, out.results[0].outcome);
656     TEST_ASSERT_EQUAL_HEX16(0x0001, out.latchedMask);
657
```

A hot trip creates a latch. Reset while active is refused; a clear removes active state but an expired reset is still refused. A fresh reset applies, and an unsupported rule id returns BAD_RULE.

test/native/test_protocol/test_main.c / lines 658-669

```
658     const SafetyCommand fresh = {SAFETY_CMD_RESET_LATCH, 0, 9000, 78};
659     SafetyInputs ok = {&s_snap, NULL, 0, 6100, &fresh, 1, 0};
660     safetyCoreStep(&s_core, &ok, &out);
661     TEST_ASSERT_EQUAL(SAFETY_CMD_APPLIED, out.results[0].outcome);
662     TEST_ASSERT_EQUAL_HEX16(0x0000, out.latchedMask);
663     TEST_ASSERT_FALSE(out.interlock);
664
665     const SafetyCommand bad = {SAFETY_CMD_RESET_LATCH, 5, 9000, 79};
666     SafetyInputs b = {&s_snap, NULL, 0, 6200, &bad, 1, 0};
667     safetyCoreStep(&s_core, &b, &out);
668     TEST_ASSERT_EQUAL(SAFETY_CMD_BAD_RULE, out.results[0].outcome);
669 }
```

The result tag survives into each outcome, allowing the transport to match the answer to the original pending request. The test connects time policy, safety policy and asynchronous correlation.

Keep disabled sensors from erasing a protective event

test/native/test_protocol/test_main.c / lines 670-697

```
670
671 static void test_safety_disabled_sensor_cannot_silence_alarm(void) {
672   // A latched or active rule blocks disabling its sensor.
673   TEST_ASSERT_TRUE(safetyRulesBlockSensor(kRules, 2, 0x0001, 0, SENSOR_BME280));
674   TEST_ASSERT_TRUE(safetyRulesBlockSensor(kRules, 2, 0, 0x0002, SENSOR_TDS));
675   TEST_ASSERT_FALSE(safetyRulesBlockSensor(kRules, 2, 0x0001, 0, SENSOR_TDS));
676   TEST_ASSERT_FALSE(safetyRulesBlockSensor(kRules, 2, 0, 0, SENSOR_BME280));
677
678   // Disabled and unlatched: no stale report. Latched: still reported, and the
679   // interlock holds, even with the input switched off.
680   safetyCoreInit(&s_core, &kSafety, kRules, 2, 0);
681   snapshotReset(&s_snap);
682   SafetyOutputs out;
683   const SensorMask off = (SensorMask)((1u << SENSOR_BME280) | (1u << SENSOR_TDS));
684   SafetyInputs quiet = {&s_snap, NULL, 0, 100, NULL, 0, off};
685   safetyCoreStep(&s_core, &quiet, &out);
686   TEST_ASSERT_EQUAL_HEX16(0x0000, out.staleMask);
687
688   const AlarmEvent trip = airEvent(RULE_DECISION_TRIP, 45.0f, 200);
689   SafetyInputs t = {&s_snap, &trip, 1, 200, NULL, 0, off};
690   safetyCoreStep(&s_core, &t, &out);
691   SafetyInputs later = {&s_snap, NULL, 0, 5000, NULL, 0, off};
692   safetyCoreStep(&s_core, &later, &out);
693   TEST_ASSERT_EQUAL_HEX16(0x0001, out.staleMask);
694   TEST_ASSERT_EQUAL_HEX16(0x0001, out.latchedMask);
695   TEST_ASSERT_TRUE(out.interlock);
696   TEST_ASSERT_FALSE(out.relayOn);
697 }
```

The policy first blocks disabling a sensor with an active or latched rule. The core tests also show that a retained latch remains reported after its sensor is disabled. The final mapping loop round-trips every supported sensor key.

test/native/test_protocol/test_main.c / lines 698-706

```
698
699 static void test_sensor_keys_round_trip(void) {
700   for (uint8_t s = 0; s < SENSOR_COUNT; ++s) {
701     const char* k = sensorKey((SensorId)s);
702     TEST_ASSERT_EQUAL(s, sensorFromKey(k, strlen(k)));
703   }
704   TEST_ASSERT_EQUAL(SENSOR_COUNT, sensorFromKey("ph2", 3));
705   TEST_ASSERT_EQUAL(SENSOR_SGP30, sensorOfMetric(METRIC_CO2EQ));
706 }
```

An invalid ph2 key returns SENSOR_COUNT. That sentinel check is the exact contract used by the command parser, so a naming change must remain consistent across these modules.

Register the complete protocol suite

test/native/test_protocol/test_main.c / lines 707-734

```
707
708 int main(int argc, char** argv) {
709   (void)argc;
710   (void)argv;
711   UNITY_BEGIN();
712   RUN_TEST(test_frame_split_across_packets);
713   RUN_TEST(test_frame_two_in_one_packet);
714   RUN_TEST(test_frame_rejects_zero_and_oversize);
715   RUN_TEST(test_telemetry_value_rules);
716   RUN_TEST(test_telemetry_overflow_returns_zero);
717   RUN_TEST(test_state_escapes_and_hides_ip_unless_connected);
718   RUN_TEST(test_results_and_sensor_list);
719   RUN_TEST(test_parse_valid_commands);
720   RUN_TEST(test_parse_calibration_commands);
721   RUN_TEST(test_encode_calibration_and_capture);
722   RUN_TEST(test_parse_rejects_malformed_and_out_of_range);
723   RUN_TEST(test_parse_keeps_id_for_error_reply);
724   RUN_TEST(test_blob_defaults_seal_and_corruption);
725   RUN_TEST(test_blob_sensor_change_and_slot_pick);
726   RUN_TEST(test_blob_refresh_defaults_follows_compiled_values);
727   RUN_TEST(test_blob_user_calibration_survives_refresh_and_reset);
728   RUN_TEST(test_blob_migrates_schema_1);
729   RUN_TEST(test_calibration_fits);
730   RUN_TEST(test_capture_from_snapshot);
731   RUN_TEST(test_journal_cursors_are_independent);
732   RUN_TEST(test_journal_overrun_reports_gap);
733   RUN_TEST(test_backoff_doubles_with_bounded_jitter);
734   RUN_TEST(test_safety_reset_latch_command);
```

The runner registers 27 cases spanning framing, encoders, parsing, blobs, fits, capture, journal, retry and Safety commands. The shared static buffers keep large fixture arrays outside repeated local frames.

test/native/test_protocol/test_main.c / lines 735-740

```
735   RUN_TEST(test_safety_disabled_sensor_cannot_silence_alarm);
736   RUN_TEST(test_sensor_keys_round_trip);
737   RUN_TEST(test_parse_long_integers_without_overflow);
738   RUN_TEST(test_blob_rejects_non_finite_fit);
739   return UNITY_END();
740 }
```

You can extend a behaviour by adding a case whose expected state you can calculate first. Keep target-only timing and library integration in bring-up tests, where the real scheduler and hardware are present.

Follow darkness all the way to a pin command

Use an invented observation: the light window finishes at 31000ms with filtered raw=30mV. The light driver stores that raw value, its interval and health. Assume the compiled raw-mV lamp policy is in use and there is no active interlock.

Step	What changes
Driver finish	A Reading becomes available; no relay write occurs.
Acquisition storeResults	Stamp metadata, replace the light slot in s_work, publish a Snapshot copy.
Safety pass	Peek the Snapshot and check the light age.
Lamp core	30 < 40 asks for ON; the minimum-off dwell has elapsed.
Final policy	Rail and interlocks permit ON.
Safety task	relayWrite(true) requests the output change.

Now change one fact at a time. If the light record is older than 3000ms at the Safety pass, stale handling forces OFF. If a relay interlock is latched, it holds OFF even with fresh darkness. If the 5V rail is invalid, final output gating holds OFF. The float value 30 does not outrank those conditions.

If no new light result ever arrives, the 100ms periodic Safety pass still reevaluates age. If the app is disconnected, local acquisition and Safety still follow the path. If BLE reads the Snapshot later, it sees a retained copy; it does not participate in the relay decision.

This is how to review a complete behaviour. Begin with a concrete input and timestamp, follow every ownership boundary, and name the guard that could reject or postpone the next step. You can now explain the folders because you have followed the operations they contain.

COMMON PITFALL

A system walkthrough needs both the successful path and the condition that prevents it. Following only the happy path hides the purpose of most state fields.

Follow one water reading into two decisions

Use an invented successful frame beginning at 10000ms. External power is valid, the normal worker holds its lease and DS18B20 and TDS are enabled. Suppose water conversion completes with a decoded value of 25.0 degrees Celsius. We will follow that one observation instead of naming every module at once.

The acquisition worker calls the driver's start operation. DS18B20 sends its conversion command and records a future poll time. While the chip converts, the worker can service other due devices or block. The driver is PENDING during that interval; no new accepted water record exists merely because the command was sent.

At a later poll, the driver checks readiness, reads the scratchpad and validates it before exposing a result. The 25.0 is now accompanied by sensor identity, unit, health, interval endpoints, sequence and configuration context. Return to the DS18B20 poll body (p. 206) if any part of that acceptance path is unclear.

REMEMBER

A dependency is a checked observation from an earlier operation. It is not permission to read another driver's private state.

Follow one water reading into two decisions / continued

Point in the path	What exists now	What it permits
After start	Pending device operation	Return to other work; no fresh water value yet.
After checked result	One candidate Reading	Acquisition can apply its acceptance and epoch rules.
After Snapshot publication	A retained complete copy	Safety and encoders can peek independently.
When TDS consumes water	An age-checked dependency	Compensate its own TDS measurement if the dependency is eligible.

The TDS path uses the water record to compute its temperature compensation coefficient. At 25.0 degrees Celsius the coefficient is 1.0. A module voltage of 1000mV then gives about 367.475ppm before any user gain. The temperature did not become a TDS observation; it supplied one input to a different observation's calculation.

Safety also reads retained observations, but its question differs: are the configured protective inputs usable, has an event tripped an interlock, and may the relay follow the lamp request? Two consumers can use the same published data for different declared policies. No consumer gains ownership of the DS18B20 bus by peeking the Snapshot.

Follow a hot air sample to an OFF output

Use the native-test rule you read: trip at 40 degrees Celsius, clear at 37, critical, latching, de-energise relay1. Assume the lamp is already ON and a fresh BME280 air-temperature observation arrives at 41.0. These are worked values from the test policy, not proposed plant limits.

Acquisition evaluates the eligible observation against its retained AlarmRuleState. Crossing the trip threshold produces TRIP once. The state remembers that the rule is active; the next hot sample need not create the same edge again. See evaluateRule (p. 402) for the exact comparison and persistence path.

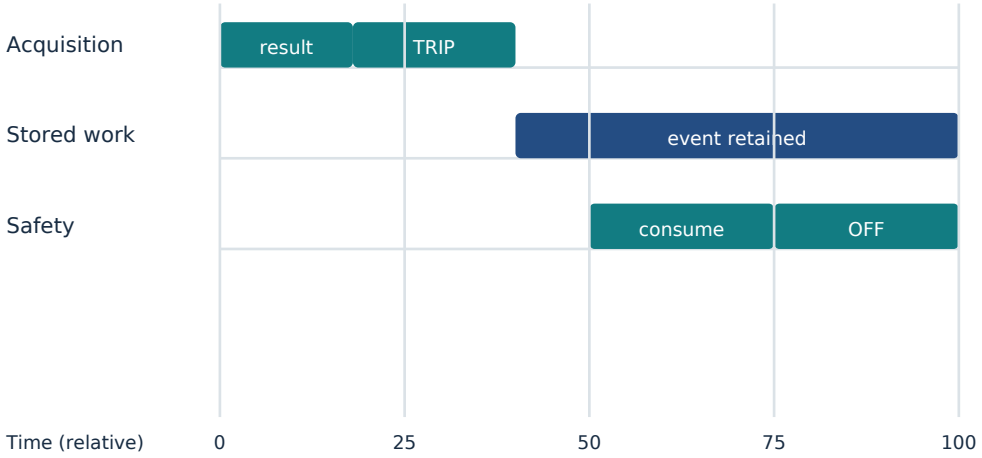
Before waking Safety, the service preserves event evidence. The detail queue normally carries the sample-bearing event; sticky storage protects the critical trip history if that queue cannot retain it. The notification says to inspect available work. It does not carry the 41.0 value itself.

Safety combines new events with its retained rule states and latest Snapshot. A latching TRIP sets both active and latched evidence. That interlock outranks a dark-light lamp request. The output says relay OFF with the interlock reason, and the Safety service writes the physical relay through its owner path.

COMMON PITFALL

An OFF action can be required even if a TRIP and CLEAR both reached Safety in one batch. Read the batch-history test before relying only on the final active mask.

Follow a hot air sample to an OFF output / continued



Illustrative ordering only. Widths do not claim measured execution times.

Now cool to 36.0 and supply an eligible clear observation. Active can clear, but latched stays set. A reset request while still active is refused; a reset after clear can apply if it is fresh and belongs to the current session. Even then, OFF dwell can postpone automatic lamp re-energisation. Resetting a latch is not a direct relay-ON command.

This path explains three separate stores: a Snapshot preserves latest observations, an event queue preserves individual transitions while capacity permits, and a latch preserves a protective obligation until an allowed reset. Replacing them all with one latest float would erase different kinds of evidence.

Follow a sensor-disable request through the owners

Suppose the client sends version 2, id d7, operation set_sensor_enabled, sensor light, enabled false. Begin with an otherwise valid secure connection. The client first serialises the JSON and prefixes its byte length. Delivery may split the header or payload across several characteristic writes.

The NimBLE callback copies incoming chunks and wakes BleTransport. drainRx (p. 666) reconstructs a complete frame, then protocolParseCommand (p. 616) creates a typed Command. A string "false" would be rejected here; a JSON boolean false is accepted.

BleTransport claims a pending slot and creates a tag, current session and expiry. It copies a ConfigRequest into Configuration's queue. A successful enqueue means accepted for processing. If that queue or the pending table is full, the reply is busy and this request does not obtain a promise of later application.

Stage	Evidence	What the client should infer
Parsed	Fields passed codec validation	The request is structurally accepted.
Queued	An owner retained the request	Execution is still pending.
Stored	The candidate passed its persistence path	Durable settings have advanced.
Applied	Acquisition acknowledged the chosen revision	The owner is now using the accepted configuration.
Result delivered	Tag and session still matched	This response belongs to this request and connection history.

REMEMBER

The service chapter shows failure after storage but before observed application. Keep durable, requested and applied revisions separate when diagnosing a timeout.

Follow a sensor-disable request through the owners / continued

Configuration rechecks expiry and session before doing work. It also checks whether the requested sensor change is blocked by protective rule state. If accepted, it prepares the next blob, uses the flash gate for persistence and asks acquisition to apply the revision at its controlled boundary. Return to configurationSubmit (p. 565) to see the queue entry point, then follow its owner body in source order.

Acquisition can cancel or prepare affected hardware and invalidate the appropriate retained records. Only its acknowledgement allows the apply path to advance. With light disabled, the automatic lamp policy cannot assume fresh darkness; the configuration change reaches control through published state rather than a BLE callback toggling a GPIO.

The completion callback copies a compact result into the BLE result queue and wakes its owner. drainResults (p. 667) validates sequence and session before using id d7 in a reply. A late answer from a previous use of that slot is discarded. This extra history is why a queue slot number alone is not a sufficient request identity.

Follow two pH buffers into later measurements

You place the probe in a known buffer, wait for a settled usable observation and request cal_capture. The transport peeks the Snapshot, checks enabled state and current rail epoch, then uses raw ADC millivolts. It does not start a fresh ADC burst in the BLE callback.

Suppose raw pin voltage is 1500mV with the compiled 10k/15k divider. Module voltage is 1500 times 25/15 = 2500mV. The capture x is 25000 because its wire scale is ten counts per millivolt. If the buffer is pH 7.000, its target y is 7000. These two integers have different units and different scales.

For a second invented buffer, use module voltage 3030mV and pH 4.000: x2 = 30300, y2 = 4000. Configuration converts those wire integers back to physical coordinates before fitting. Slope is (4 - 7)/(3030 - 2500), about -0.00566038 pH per millivolt. Offset is 7 minus slope times 2500, about 21.15094.

Coordinate	Physical value	Wire integer
First x	2500.0mV module	25000
First y	pH 7.000	7000
Second x	3030.0mV module	30300
Second y	pH 4.000	4000

The fit must pass coordinate, finite-number and slope/separation checks. Storing a fit marks its override status and gives it revision history. After the owning acquisition path applies that configuration, later pH readings carry the calibration revision used for their values. Equal pH numbers produced under different fits can therefore be distinguished.

A captured old voltage does not become new because you requested it now. calCaptureFromSnapshot (p. 525) checks observation age and health. Rail gating occurs before that helper in the transport. If external power changed epochs, a retained value from the previous epoch must not become a calibration point.

COMMON PITFALL

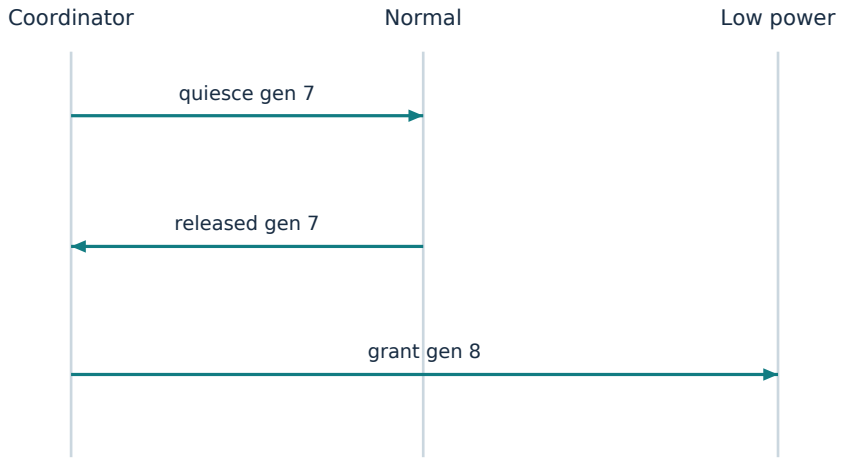
Do not fit pH directly against GPIO millivolts if the fitter expects recovered module millivolts. Using the wrong x scale changes the coefficient even if the two reference points appear to work in a separate calculation.

Follow external power loss without overlapping owners

Assume normal acquisition holds generation 7 and an HC-SR04 operation is pending on the 5V branch. External power begins to fall. The first observed sense edge invalidates the rail history and wakes PowerCoord; the edge does not complete a full mode transition inside the ISR.

The sense task path qualifies the level after its configured interval. Meanwhile, a consumer sees rail invalid or an epoch mismatch and refuses old 5V readings. This acceptance rule acts independently of whether the power state machine has already changed its mode label.

Once loss is accepted by policy, PowerCoord requests normal-worker quiescence for generation 7. That worker cancels pending operations, leaves interfaces in their defined power-off state and releases the lease only after cleanup. The coordinator must observe the matching release before granting generation 8 to the low-power worker.



The names are ownership roles. A new generation makes a new grant distinguishable from earlier work.

REMEMBER

A lease controls access. Rail validity controls measurement eligibility. Network acknowledgement controls transition completion. These are related facts with separate jobs.

Follow external power loss without overlapping owners / continued

In low power, the retained 3.3V sensors may still be serviced according to that worker's allowed rail mask. pH, TDS and HC-SR04 cannot supply valid new 5V observations from their lost branch. Wi-Fi receives a stop request and later acknowledges radio-off; BLE and other retained work follow the policy you read, rather than being inferred from the Wi-Fi task name.

If release misses its deadline, the mode reports DEGRADED. A timeout does not grant permission to start the second worker over the same bus objects. A late valid release can allow recovery later. This is a design choice favouring exclusive ownership over pretending the timeout proved cleanup finished.

On external-power return, stability must belong to the current epoch. A new low-high pulse can restart that qualification even if the task samples high at both ends. Return then quiesces the low worker before granting normal. The native power tests let you replay all these inputs without waiting for a real unplug.

Follow a producer that stops publishing

Suppose the last light result ends at 31000ms and is eligible when published. Then acquisition stalls. The Snapshot queue still contains its retained copy, so snapshotPeek may continue succeeding. A successful peek proves a copy exists; it does not prove the producer is alive.

Safety wakes through its normal bounded wait even without a new publication. At 34000ms the age is exactly 3000ms; the test policy still accepts that boundary. At 34100ms it is beyond the limit, so stale-data policy forces the lamp OFF. No new failed sensor result was needed to discover age.

Supervisor separately observes acquisition's old progress stamp. After its configured deadline and boot grace it warns about a stalled responsibility. The task watchdog is yet another path: the stalled subscribed owner eventually fails to reset its own watchdog. These three mechanisms answer data freshness, task progress and watchdog liveness respectively.

Mechanism	Input it checks	Useful conclusion
Reading age	Now minus interval endpoint	This observation is too old for this policy.
Progress deadline	Now minus meaningful-work stamp	This owner has not reported timely progress.
Task watchdog	Subscribed owner reset behaviour	The watched execution responsibility missed the watchdog contract.

If another task fed the watchdog on acquisition's behalf, the first two problems would remain while the watchdog evidence became misleading. This is why the owner resets itself and Supervisor only observes. The architecture connects several independent checks instead of asking one queue or one timer to prove everything.

When acquisition resumes, a new usable result does not erase the recent OFF transition's dwell requirement. Recovery is a state transition with its own history, not a jump back into the middle of the stalled operation.

COMMON PITFALL

Polling a retained mailbox faster cannot make its content younger. More consumer iterations create no new physical observations.

Choose a boundary by the fact it must preserve

You can now evaluate alternative architectures with concrete costs. One task per sensor would give each sensor a separate stack and execution loop. Shared I2C would still need one agreed serialisation policy, and TDS would still need a fresh water-temperature dependency. More tasks would not remove those coordination questions.

This implementation instead gives sensor access to the active acquisition owner and uses stepped driver operations. That reduces competing bus callers and makes release-before-grant power handover explicit. The cost is that a long HAL call can delay other work in that same owner. Scheduling budgets and overlap guards need real duration measurements; a function table cannot enforce them.

REMEMBER

A useful architecture lets you name the owner, the stored evidence and the acceptance rule at every boundary.

Choose a boundary by the fact it must preserve / continued

Need	Choice in this controller	What another choice would require
Latest complete observations	One-item Snapshot overwrite/peek	An event stream would need backlog and age policy.
Distinct alarm transitions	Detail queue plus sticky critical evidence	A mailbox alone could erase a short TRIP.
Wake an owner promptly	Task notification	Data must still live in a queue or protected object.
One writer for settings	Configuration request queue	Direct callbacks would need coordinated persistence and apply boundaries.
Exclusive device access	One active acquisition lease	Multiple owners would need a different complete access protocol.

When extending the controller, start with the fact that must survive: the latest state, every event, a request identity or exclusive ownership. Then choose storage and wake-up behaviour that preserve it. A queue, notification, mutex and critical section are different tools because they preserve different properties.

For a new sensor, trace one complete candidate result: pin operation, physical wait, validation, Reading identity and unit, dependency, publication, consumer acceptance and cleanup on rail loss. You have read examples of every step in this source. Use their contracts together, then check the new physical timing and electrical assumptions on the actual board.